

INTERNATIONAL STANDARD

NORME INTERNATIONALE



**OPC Unified Architecture –
Part 4: Services**

**Architecture Unifiée OPC –
Partie 4: Services**



THIS PUBLICATION IS COPYRIGHT PROTECTED

Copyright © 2015 IEC, Geneva, Switzerland

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized in any form or by any means, electronic or mechanical, including photocopying and microfilm, without permission in writing from either IEC or IEC's member National Committee in the country of the requester. If you have any questions about IEC copyright or have an enquiry about obtaining additional rights to this publication, please contact the address below or your local IEC member National Committee for further information.

Droits de reproduction réservés. Sauf indication contraire, aucune partie de cette publication ne peut être reproduite ni utilisée sous quelque forme que ce soit et par aucun procédé, électronique ou mécanique, y compris la photocopie et les microfilms, sans l'accord écrit de l'IEC ou du Comité national de l'IEC du pays du demandeur. Si vous avez des questions sur le copyright de l'IEC ou si vous désirez obtenir des droits supplémentaires sur cette publication, utilisez les coordonnées ci-après ou contactez le Comité national de l'IEC de votre pays de résidence.

IEC Central Office
3, rue de Varembe
CH-1211 Geneva 20
Switzerland

Tel.: +41 22 919 02 11
Fax: +41 22 919 03 00
info@iec.ch
www.iec.ch

About the IEC

The International Electrotechnical Commission (IEC) is the leading global organization that prepares and publishes International Standards for all electrical, electronic and related technologies.

About IEC publications

The technical content of IEC publications is kept under constant review by the IEC. Please make sure that you have the latest edition, a corrigenda or an amendment might have been published.

IEC Catalogue - webstore.iec.ch/catalogue

The stand-alone application for consulting the entire bibliographical information on IEC International Standards, Technical Specifications, Technical Reports and other documents. Available for PC, Mac OS, Android Tablets and iPad.

IEC publications search - www.iec.ch/searchpub

The advanced search enables to find IEC publications by a variety of criteria (reference number, text, technical committee,...). It also gives information on projects, replaced and withdrawn publications.

IEC Just Published - webstore.iec.ch/justpublished

Stay up to date on all new IEC publications. Just Published details all new publications released. Available online and also once a month by email.

Electropedia - www.electropedia.org

The world's leading online dictionary of electronic and electrical terms containing more than 30 000 terms and definitions in English and French, with equivalent terms in 15 additional languages. Also known as the International Electrotechnical Vocabulary (IEV) online.

IEC Glossary - std.iec.ch/glossary

More than 60 000 electrotechnical terminology entries in English and French extracted from the Terms and Definitions clause of IEC publications issued since 2002. Some entries have been collected from earlier publications of IEC TC 37, 77, 86 and CISPR.

IEC Customer Service Centre - webstore.iec.ch/csc

If you wish to give us your feedback on this publication or need further assistance, please contact the Customer Service Centre: csc@iec.ch.

A propos de l'IEC

La Commission Electrotechnique Internationale (IEC) est la première organisation mondiale qui élabore et publie des Normes internationales pour tout ce qui a trait à l'électricité, à l'électronique et aux technologies apparentées.

A propos des publications IEC

Le contenu technique des publications IEC est constamment revu. Veuillez vous assurer que vous possédez l'édition la plus récente, un corrigendum ou amendement peut avoir été publié.

Catalogue IEC - webstore.iec.ch/catalogue

Application autonome pour consulter tous les renseignements bibliographiques sur les Normes internationales, Spécifications techniques, Rapports techniques et autres documents de l'IEC. Disponible pour PC, Mac OS, tablettes Android et iPad.

Recherche de publications IEC - www.iec.ch/searchpub

La recherche avancée permet de trouver des publications IEC en utilisant différents critères (numéro de référence, texte, comité d'études,...). Elle donne aussi des informations sur les projets et les publications remplacées ou retirées.

IEC Just Published - webstore.iec.ch/justpublished

Restez informé sur les nouvelles publications IEC. Just Published détaille les nouvelles publications parues. Disponible en ligne et aussi une fois par mois par email.

Electropedia - www.electropedia.org

Le premier dictionnaire en ligne de termes électroniques et électriques. Il contient plus de 30 000 termes et définitions en anglais et en français, ainsi que les termes équivalents dans 15 langues additionnelles. Egalement appelé Vocabulaire Electrotechnique International (IEV) en ligne.

Glossaire IEC - std.iec.ch/glossary

Plus de 60 000 entrées terminologiques électrotechniques, en anglais et en français, extraites des articles Termes et Définitions des publications IEC parues depuis 2002. Plus certaines entrées antérieures extraites des publications des CE 37, 77, 86 et CISPR de l'IEC.

Service Clients - webstore.iec.ch/csc

Si vous désirez nous donner des commentaires sur cette publication ou si vous avez des questions contactez-nous: csc@iec.ch.

INTERNATIONAL STANDARD

NORME INTERNATIONALE



**OPC Unified Architecture –
Part 4: Services**

**Architecture Unifiée OPC –
Partie 4: Services**

INTERNATIONAL
ELECTROTECHNICAL
COMMISSION

COMMISSION
ELECTROTECHNIQUE
INTERNATIONALE

ICS 25.040.40; 35.100

ISBN 978-2-8322-2369-7

Warning! Make sure that you obtained this publication from an authorized distributor. Attention! Veuillez vous assurer que vous avez obtenu cette publication via un distributeur agréé.
--

CONTENTS

FOREWORD	12
1 Scope	14
2 Normative references	14
3 Terms, definitions and conventions	15
3.1 Terms and definitions	15
3.2 Abbreviations and symbols	15
3.3 Conventions for Service definitions	16
4 Overview	17
4.1 Service Set model	17
4.2 Request/response Service procedures	21
5 Service Sets	21
5.1 General	21
5.2 Service request and response header	21
5.3 Service results	21
5.4 Discovery Service Set	23
5.4.1 Overview	23
5.4.2 FindServers	24
5.4.3 GetEndpoints	25
5.4.4 RegisterServer	28
5.5 SecureChannel Service Set	31
5.5.1 Overview	31
5.5.2 OpenSecureChannel	32
5.5.3 CloseSecureChannel	35
5.6 Session Service Set	36
5.6.1 Overview	36
5.6.2 CreateSession	36
5.6.3 ActivateSession	40
5.6.4 CloseSession	43
5.6.5 Cancel	43
5.7 NodeManagement Service Set	44
5.7.1 Overview	44
5.7.2 AddNodes	44
5.7.3 AddReferences	46
5.7.4 DeleteNodes	47
5.7.5 DeleteReferences	48
5.8 View Service Set	49
5.8.1 Overview	49
5.8.2 Browse	50
5.8.3 BrowseNext	52
5.8.4 TranslateBrowsePathsToNodeIds	54
5.8.5 RegisterNodes	56
5.8.6 UnregisterNodes	57
5.9 Query Service Set	57
5.9.1 Overview	57
5.9.2 Querying Views	58
5.9.3 QueryFirst	58

5.9.4	QueryNext	62
5.10	Attribute Service Set	63
5.10.1	Overview	63
5.10.2	Read	63
5.10.3	HistoryRead	65
5.10.4	Write	67
5.10.5	HistoryUpdate	69
5.11	Method Service Set	70
5.11.1	Overview	70
5.11.2	Call	71
5.12	MonitoredItem Service Set	73
5.12.1	MonitoredItem model	73
5.12.2	CreateMonitoredItems	79
5.12.3	ModifyMonitoredItems	81
5.12.4	SetMonitoringMode	83
5.12.5	SetTriggering	84
5.12.6	DeleteMonitoredItems	85
5.13	Subscription Service Set	86
5.13.1	Subscription model	86
5.13.2	CreateSubscription	93
5.13.3	ModifySubscription	94
5.13.4	SetPublishingMode	95
5.13.5	Publish	96
5.13.6	Republish	98
5.13.7	TransferSubscriptions	98
5.13.8	DeleteSubscriptions	100
6	Service behaviours	101
6.1	Security	101
6.1.1	Overview	101
6.1.2	Obtaining and Installing an Application Instance Certificate	101
6.1.3	Determining if a Certificate is Trusted	102
6.1.4	Creating a SecureChannel	104
6.1.5	Creating a Session	106
6.1.6	Impersonating a User	107
6.2	Software Certificates	107
6.2.1	Overview	107
6.2.2	Obtaining and Installing a Software Certificate	107
6.2.3	Validating a Software Certificate	109
6.3	Auditing	109
6.3.1	Overview	109
6.3.2	General audit logs	109
6.3.3	General audit Events	109
6.3.4	Auditing for Discovery Service Set	110
6.3.5	Auditing for SecureChannel Service Set	110
6.3.6	Auditing for Session Service Set	110
6.3.7	Auditing for NodeManagement Service Set	111
6.3.8	Auditing for Attribute Service Set	111
6.3.9	Auditing for Method Service Set	111
6.3.10	Auditing for View, Query, MonitoredItem and Subscription Service Set	112

6.4	Redundancy.....	112
6.4.1	Redundancy overview	112
6.4.2	Server redundancy overview	112
6.4.3	Client redundancy	116
6.4.4	Network redundancy	116
6.5	Re-establishing connections	117
7	Common parameter type definitions.....	118
7.1	ApplicationDescription	118
7.2	ApplicationInstanceCertificate	118
7.3	BrowseResult.....	119
7.4	ContentFilter	119
7.4.1	ContentFilter structure	119
7.4.2	ContentFilterResult	120
7.4.3	FilterOperator	121
7.4.4	FilterOperand parameters	127
7.5	Counter.....	128
7.6	ContinuationPoint	129
7.7	DataValue	129
7.7.1	General.....	129
7.7.2	PicoSeconds.....	130
7.7.3	SourceTimestamp	130
7.7.4	ServerTimestamp.....	130
7.7.5	StatusCode assigned to a value	131
7.8	DiagnosticInfo	131
7.9	EndpointDescription	132
7.10	ExpandedNodeId	133
7.11	ExtensibleParameter	133
7.12	Index	134
7.13	IntegerId	134
7.14	MessageSecurityMode	134
7.15	MonitoringParameters	134
7.16	MonitoringFilter parameters	135
7.16.1	Overview	135
7.16.2	DataChangeFilter	136
7.16.3	EventFilter	136
7.16.4	AggregateFilter	139
7.17	MonitoringMode	140
7.18	NodeAttributes parameters	140
7.18.1	Overview	140
7.18.2	ObjectAttributes parameter	141
7.18.3	VariableAttributes parameter	141
7.18.4	MethodAttributes parameter.....	142
7.18.5	ObjectTypeAttributes parameter.....	142
7.18.6	VariableTypeAttributes parameter.....	142
7.18.7	ReferenceTypeAttributes parameter.....	143
7.18.8	DataTypeAttributes parameter	143
7.18.9	ViewAttributes parameter.....	143
7.19	NotificationData parameters	144
7.19.1	Overview	144

7.19.2	DataChangeNotification parameter	144
7.19.3	EventNotificationList parameter	145
7.19.4	StatusChangeNotification parameter	145
7.20	NotificationMessage	145
7.21	NumericRange	146
7.22	QueryDataSet	147
7.23	ReadValueId	147
7.24	ReferenceDescription	148
7.25	RelativePath	149
7.26	RequestHeader	149
7.27	ResponseHeader	151
7.28	ServiceFault	151
7.29	SessionAuthenticationToken	151
7.30	SignatureData	153
7.31	SignedSoftwareCertificate	153
7.32	SoftwareCertificate	153
7.33	StatusCode	154
7.33.1	General	154
7.33.2	Common StatusCodes	156
7.34	TimestampsToReturn	159
7.35	UserIdentityToken parameters	159
7.35.1	Overview	159
7.35.2	AnonymousIdentityToken	160
7.35.3	UserNameIdentityToken	160
7.35.4	X509IdentityTokens	161
7.35.5	IssuedIdentityToken	161
7.36	UserTokenPolicy	162
7.37	ViewDescription	163
Annex A	(informative) BNF definitions	164
A.1	Overview over BNF	164
A.2	BNF of RelativePath	164
A.3	BNF of NumericRange	165
Annex B	(informative) Content Filter and Query Examples	166
B.1	Simple ContentFilter examples	166
B.1.1	Overview	166
B.1.2	Example 1	166
B.1.3	Example 2	167
B.2	Complex Examples of Query Filters	167
B.2.1	Overview	167
B.2.2	Used type model	168
B.2.3	Example Notes	170
B.2.4	Example 1	171
B.2.5	Example 2	172
B.2.6	Example 3	173
B.2.7	Example 4	175
B.2.8	Example 5	176
B.2.9	Example 6	178
B.2.10	Example 7	179
B.2.11	Example 8	181

B.2.12	Example 9.....	182
Figure 1	– Discovery Service Set	17
Figure 2	– SecureChannel Service Set.....	18
Figure 3	– Session Service Set	18
Figure 4	– NodeManagement Service Set	18
Figure 5	– View Service Set.....	19
Figure 6	– Attribute Service Set	19
Figure 7	– Method Service Set.....	20
Figure 8	– MonitoredItem and Subscription Service Sets	20
Figure 9	– Discovery process.....	23
Figure 10	– Using a Gateway Server.....	27
Figure 11	– The Registration Process – Manually Launched Servers	28
Figure 12	– The Registration Process – Automatically Launched Servers	29
Figure 13	– SecureChannel and Session Services	32
Figure 14	– Multiplexing Users on a Session.....	38
Figure 15	– MonitoredItem Model.....	74
Figure 16	– Typical delay in change detection.....	75
Figure 17	– Queue overflow handling.....	77
Figure 18	– Triggering Model	78
Figure 19	– Obtaining and Installing an Application Instance Certificate.....	102
Figure 20	– Determining if a Application Instance Certificate is Trusted	104
Figure 21	– Establishing a SecureChannel.....	105
Figure 22	– Establishing a Session	106
Figure 23	– Impersonating a User	107
Figure 24	– Obtaining and Installing a Software Certificate	108
Figure 25	– Transparent Redundancy setup.....	113
Figure 26	– Non-Transparent Redundancy setup	113
Figure 27	– Server proxy for redundancy	116
Figure 28	– Reconnect Sequence	117
Figure 29	– Logical layers of a Server.....	152
Figure 30	– Obtaining a SessionAuthenticationToken	152
Figure B.1	– Filter Logic Tree Example	166
Figure B.2	– Filter Logic Tree Example	167
Figure B.3	– Example Type Nodes	169
Figure B.4	– Example Instance Nodes	170
Figure B.5	– Example 1 Filter.....	171
Figure B.6	– Example 2 Filter Logic Tree	172
Figure B.7	– Example 3 Filter Logic Tree	174
Figure B.8	– Example 4 Filter Logic Tree	176
Figure B.9	– Example 5 Filter Logic Tree	177
Figure B.10	– Example 6 Filter Logic Tree	178
Figure B.11	– Example 7 Filter Logic Tree	180

Figure B.12 – Example 8 Filter Logic Tree	181
Figure B.13 – Example 9 Filter Logic Tree	183
Table 1 – Service Definition Table	16
Table 2 – Parameter Types defined in IEC 62541-3	17
Table 3 – FindServers Service Parameters	25
Table 4 – GetEndpoints Service Parameters	27
Table 5 – RegisterServer Service Parameters	30
Table 6 – RegisterServer Service Result Codes	30
Table 7 – OpenSecureChannel Service Parameters	34
Table 8 – OpenSecureChannel Service Result Codes	35
Table 9 – CloseSecureChannel Service Parameters	35
Table 10 – CloseSecureChannel Service Result Codes	35
Table 11 – CreateSession Service Parameters	38
Table 12 – CreateSession Service Result Codes	40
Table 13 – ActivateSession Service Parameters	42
Table 14 – ActivateSession Service Result Codes	43
Table 15 – CloseSession Service Parameters	43
Table 16 – CloseSession Service Result Codes	43
Table 17 – Cancel Service Parameters	44
Table 18 – AddNodes Service Parameters	45
Table 19 – AddNodes Service Result Codes	45
Table 20 – AddNodes Operation Level Result Codes	46
Table 21 – AddReferences Service Parameters	46
Table 22 – AddReferences Service Result Codes	47
Table 23 – AddReferences Operation Level Result Codes	47
Table 24 – DeleteNodes Service Parameters	48
Table 25 – DeleteNodes Service Result Codes	48
Table 26 – DeleteNodes Operation Level Result Codes	48
Table 27 – DeleteReferences Service Parameters	49
Table 28 – DeleteReferences Service Result Codes	49
Table 29 – DeleteReferences Operation Level Result Codes	49
Table 30 – Browse Service Parameters	51
Table 31 – Browse Service Result Codes	52
Table 32 – Browse Operation Level Result Codes	52
Table 33 – BrowseNext Service Parameters	53
Table 34 – BrowseNext Service Result Codes	53
Table 35 – BrowseNext Operation Level Result Codes	54
Table 36 – TranslateBrowsePathsToNodeIds Service Parameters	55
Table 37 – TranslateBrowsePathsToNodeIds Service Result Codes	55
Table 38 – TranslateBrowsePathsToNodeIds Operation Level Result Codes	56
Table 39 – RegisterNodes Service Parameters	56
Table 40 – RegisterNodes Service Result Codes	57

Table 41 – UnregisterNodes Service Parameters	57
Table 42 – UnregisterNodes Service Result Codes	57
Table 43 – QueryFirst Request Parameters	60
Table 44 – QueryFirst Response Parameters	61
Table 45 – QueryFirst Service Result Codes	62
Table 46 – QueryFirst Operation Level Result Codes	62
Table 47 – QueryNext Service Parameters	63
Table 48 – QueryNext Service Result Codes	63
Table 49 – Read Service Parameters	64
Table 50 – Read Service Result Codes	64
Table 51 – Read Operation Level Result Codes	65
Table 52 – HistoryRead Service Parameters	66
Table 53 – HistoryRead Service Result Codes	67
Table 54 – HistoryRead Operation Level Result Codes	67
Table 55 – Write Service Parameters	68
Table 56 – Write Service Result Codes	69
Table 57 – Write Operation Level Result Codes	69
Table 58 – HistoryUpdate Service Parameters	70
Table 59 – HistoryUpdate Service Result Codes	70
Table 60 – HistoryUpdate Operation Level Result Codes	70
Table 61 – Call Service Parameters	72
Table 62 – Call Service Result Codes	72
Table 63 – Call Operation Level Result Codes	73
Table 64 – CreateMonitoredItems Service Parameters	80
Table 65 – CreateMonitoredItems Service Result Codes	80
Table 66 – CreateMonitoredItems Operation Level Result Codes	81
Table 67 – ModifyMonitoredItems Service Parameters	82
Table 68 – ModifyMonitoredItems Service Result Codes	82
Table 69 – ModifyMonitoredItems Operation Level Result Codes	83
Table 70 – SetMonitoringMode Service Parameters	83
Table 71 – SetMonitoringMode Service Result Codes	83
Table 72 – SetMonitoringMode Operation Level Result Codes	84
Table 73 – SetTriggering Service Parameters	84
Table 74 – SetTriggering Service Result Codes	85
Table 75 – SetTriggering Operation Level Result Codes	85
Table 76 – DeleteMonitoredItems Service Parameters	85
Table 77 – DeleteMonitoredItems Service Result Codes	86
Table 78 – DeleteMonitoredItems Operation Level Result Codes	86
Table 79 – Subscription States	88
Table 80 – Subscription State Table	89
Table 81 – State variables and parameters	91
Table 82 – Functions	92
Table 83 – CreateSubscription Service Parameters	93

Table 84 – CreateSubscription Service Result Codes	94
Table 85 – ModifySubscription Service Parameters	94
Table 86 – ModifySubscription Service Result Codes	95
Table 87 – SetPublishingMode Service Parameters	95
Table 88 – SetPublishingMode Service Result Codes	96
Table 89 – SetPublishingMode Operation Level Result Codes	96
Table 90 – Publish Service Parameters	97
Table 91 – Publish Service Result Codes	97
Table 92 – Publish Operation Level Result Codes	98
Table 93 – Republish Service Parameters	98
Table 94 – Republish Service Result Codes	98
Table 95 – TransferSubscriptions Service Parameters	99
Table 96 – TransferSubscriptions Service Result Codes	99
Table 97 – TransferSubscriptions Operation Level Result Codes	100
Table 98 – DeleteSubscriptions Service Parameters	100
Table 99 – DeleteSubscriptions Service Result Codes	100
Table 100 – DeleteSubscriptions Operation Level Result Codes	101
Table 101 – Certificate Validation Steps	103
Table 102 – Redundancy failover actions	114
Table 103 – ApplicationDescription	118
Table 104 – ApplicationInstanceCertificate	119
Table 105 – BrowseResult	119
Table 106 – ContentFilter Structure	120
Table 107 – ContentFilterResult Structure	120
Table 108 – ContentFilterResult Result Codes	120
Table 109 – ContentFilterResult Operand Result Codes	121
Table 110 – Basic FilterOperator Definition	121
Table 111 – Complex FilterOperator Definition	123
Table 112 – Wildcard characters	124
Table 113 – Conversion Rules	125
Table 114 – Data Precedence Rules	126
Table 115 – Logical AND Truth Table	126
Table 116 – Logical OR Truth Table	127
Table 117 – FilterOperand parameter Typelds	127
Table 118 – ElementOperand	127
Table 119 – LiteralOperand	127
Table 120 – AttributeOperand	128
Table 121 – SimpleAttributeOperand	128
Table 122 – DataValue	129
Table 123 – DiagnosticInfo	132
Table 124 – EndpointDescription	133
Table 125 – ExpandedNodeId	133
Table 126 – ExtensibleParameter Base Type	134

Table 127 – MessageSecurityMode Values	134
Table 128 – MonitoringParameters	135
Table 129 – MonitoringFilter parameterTypelds	136
Table 130 – DataChangeFilter	136
Table 131 – EventFilter structure	138
Table 132 – EventFilterResult structure	138
Table 133 – EventFilterResult Result Codes	138
Table 134 – AggregateFilter structure	139
Table 135 – AggregateFilterResult structure	140
Table 136 – MonitoringMode Values	140
Table 137 – NodeAttributes parameterTypelds	140
Table 138 – Bit mask for specified Attributes	141
Table 139 – ObjectAttributes	141
Table 140 – VariableAttributes	142
Table 141 – MethodAttributes	142
Table 142 – ObjectTypeAttributes	142
Table 143 – VariableTypeAttributes	143
Table 144 – ReferenceTypeAttributes	143
Table 145 – DataTypeAttributes	143
Table 146 – ViewAttributes	144
Table 147 – NotificationData parameterTypelds	144
Table 148 – DataChangeNotification	145
Table 149 – EventNotificationList	145
Table 150 – StatusChangeNotification	145
Table 151 – NotificationMessage	146
Table 152 – NumericRange	147
Table 153 – QueryDataSet	147
Table 154 – ReadValueId	148
Table 155 – ReferenceDescription	148
Table 156 – RelativePath	149
Table 157 – RequestHeader	150
Table 158 – ResponseHeader	151
Table 159 – ServiceFault	151
Table 160 – SignatureData	153
Table 161 – SignedSoftwareCertificate	153
Table 162 – SoftwareCertificate	154
Table 163 – StatusCode Bit Assignments	155
Table 164 – DataValue InfoBits	156
Table 165 – Common Service Result Codes	157
Table 166 – Common Operation Level Result Codes	158
Table 167 – TimestampsToReturn Values	159
Table 168 – UserIdentityToken parameterTypelds	159
Table 169 – UserIdentityToken Encrypted Token Format	160

Table 170 – AnonymousIdentityToken	160
Table 171 – UserNameIdentityToken	161
Table 172 – EncryptionAlgorithm selection.....	161
Table 173 – X509 Identity Token.....	161
Table 174 – IssuedIdentityToken	162
Table 175 – UserTokenPolicy	162
Table 176 – ViewDescription.....	163
Table A.1 – RelativePath	164
Table A.2 – <i>RelativePath</i> Examples	165
Table B.1 – ContentFilter Example.....	167
Table B.2 – ContentFilter Example.....	167
Table B.3 – Example 1 NodeTypeDescription	171
Table B.4 – Example 1 ContentFilter.....	171
Table B.5 – Example 1 QueryDataSets	172
Table B.6 – Example 2 NodeTypeDescription	172
Table B.7 – Example 2 ContentFilter.....	173
Table B.8 – Example 2 QueryDataSets	173
Table B.9 – Example 3 – NodeTypeDescription	173
Table B.10 – Example 3 ContentFilter.....	175
Table B.11 – Example 3 QueryDataSets	175
Table B.12 – Example 4 NodeTypeDescription.....	176
Table B.13 – Example 4 ContentFilter.....	176
Table B.14 – Example 4 QueryDataSets	176
Table B.15 – Example 5 NodeTypeDescription.....	177
Table B.16 – Example 5 ContentFilter.....	177
Table B.17 – Example 5 QueryDataSets	177
Table B.18 – Example 6 NodeTypeDescription.....	178
Table B.19 – Example 6 ContentFilter.....	178
Table B.20 – Example 6 QueryDataSets	179
Table B.21 – Example 6 QueryDataSets without Additional Information	179
Table B.22 – Example 7 NodeTypeDescription.....	180
Table B.23 – Example 7 ContentFilter.....	180
Table B.24 – Example 7 QueryDataSets	181
Table B.25 – Example 8 NodeTypeDescription.....	181
Table B.26 – Example 8 ContentFilter.....	182
Table B.27 – Example 8 QueryDataSets	182
Table B.28 – Example 9 NodeTypeDescription.....	182
Table B.29 – Example 9 ContentFilter.....	183
Table B.30 – Example 9 QueryDataSets	183

INTERNATIONAL ELECTROTECHNICAL COMMISSION

OPC Unified Architecture –**Part 4: Services****FOREWORD**

- 1) The International Electrotechnical Commission (IEC) is a worldwide organization for standardization comprising all national electrotechnical committees (IEC National Committees). The object of IEC is to promote international co-operation on all questions concerning standardization in the electrical and electronic fields. To this end and in addition to other activities, IEC publishes International Standards, Technical Specifications, Technical Reports, Publicly Available Specifications (PAS) and Guides (hereafter referred to as “IEC Publication(s)”). Their preparation is entrusted to technical committees; any IEC National Committee interested in the subject dealt with may participate in this preparatory work. International, governmental and non-governmental organizations liaising with the IEC also participate in this preparation. IEC collaborates closely with the International Organization for Standardization (ISO) in accordance with conditions determined by agreement between the two organizations.
- 2) The formal decisions or agreements of IEC on technical matters express, as nearly as possible, an international consensus of opinion on the relevant subjects since each technical committee has representation from all interested IEC National Committees.
- 3) IEC Publications have the form of recommendations for international use and are accepted by IEC National Committees in that sense. While all reasonable efforts are made to ensure that the technical content of IEC Publications is accurate, IEC cannot be held responsible for the way in which they are used or for any misinterpretation by any end user.
- 4) In order to promote international uniformity, IEC National Committees undertake to apply IEC Publications transparently to the maximum extent possible in their national and regional publications. Any divergence between any IEC Publication and the corresponding national or regional publication shall be clearly indicated in the latter.
- 5) IEC itself does not provide any attestation of conformity. Independent certification bodies provide conformity assessment services and, in some areas, access to IEC marks of conformity. IEC is not responsible for any services carried out by independent certification bodies.
- 6) All users should ensure that they have the latest edition of this publication.
- 7) No liability shall attach to IEC or its directors, employees, servants or agents including individual experts and members of its technical committees and IEC National Committees for any personal injury, property damage or other damage of any nature whatsoever, whether direct or indirect, or for costs (including legal fees) and expenses arising out of the publication, use of, or reliance upon, this IEC Publication or any other IEC Publications.
- 8) Attention is drawn to the Normative references cited in this publication. Use of the referenced publications is indispensable for the correct application of this publication.
- 9) Attention is drawn to the possibility that some of the elements of this IEC Publication may be the subject of patent rights. IEC shall not be held responsible for identifying any or all such patent rights.

International Standard IEC 62541-4 has been prepared by subcommittee 65E: Devices and integration in enterprise systems, of IEC technical committee 65: Industrial-process measurement, control and automation.

This second edition cancels and replaces the first edition published in 2011. This edition constitutes a technical revision.

This edition includes the following significant technical changes with respect to the previous edition:

- a) Update for 6.4 Redundancy.
Added non-transparent redundancy option HotAndMirrored and reworked most of the redundancy description.
- b) Clarifications for Publish and Reconnect scenarios.
Reworked different parts of the specification to make sure no data is lost during short communication interruptions and clients can always detect for how long they lost

information during connection interruption. Added new clause 6.5 Re-establishing connections that describes the exact reconnect sequence for clients loosing connection to a server. Changed the minimum requirement for the retransmission queue of sent NotificationMessages from one keep-alive interval to minimum two times the minimum number of Publish requests per Session. Added clarification in which data value the overflow bit is set depending on the discard oldest setting. Changed discard handling for discardOldest is FALSE. The new value is replacing the last value put into the queue for FALSE. Added exception that the overflow bit is not set if the queue size one.

- c) Handling of MonitoredItem changes in short network interruption scenarios.
Added new method GetMonitoredItems in Part 5. This method can be used to get the list of monitored items in a subscription if CreateMonitoredItems failed due to a network interruption and the client does not know if the creation succeeded in the server.
- d) Update for 6.1.3 Determining if a Certificate is Trusted
Revised rules for certificate validation.
- e) Revised definition of parameters semaphoreFile and isOnline in Service RegisterServer
- f) Services ModifySubscription and ModifyMonitoredItems
Clarified that changes are applied directly and will take effect as soon as practical but not later than twice the new time interval.
- g) There is a long list of minor changes to eliminate ambiguity.

The text of this standard is based on the following documents:

CDV	Report on voting
65E/375/CDV	65E/403/RVC

Full information on the voting for the approval of this standard can be found in the report on voting indicated in the above table.

This publication has been drafted in accordance with the ISO/IEC Directives, Part 2.

A list of all parts of the IEC 62541 series, published under the general title *OPC Unified Architecture*, can be found on the IEC website.

The committee has decided that the contents of this publication will remain unchanged until the stability date indicated on the IEC web site under "<http://webstore.iec.ch>" in the data related to the specific publication. At this date, the publication will be

- reconfirmed,
- withdrawn,
- replaced by a revised edition, or
- amended.

IMPORTANT – The “colour inside” logo on the cover page of this publication indicates that it contains colours which are considered to be useful for the correct understanding of its contents. Users should therefore print this publication using a colour printer.

OPC Unified Architecture –

Part 4: Services

1 Scope

This part of IEC 62541 defines the OPC Unified Architecture (OPC UA) *Services*. The *Services* described are the collection of abstract Remote Procedure Calls (RPC) that are implemented by OPC UA *Servers* and called by OPC UA *Clients*. All interactions between OPC UA *Clients* and *Servers* occur via these *Services*. The defined *Services* are considered abstract because no particular RPC mechanism for implementation is defined in this part. IEC 62541-6 specifies one or more concrete mappings supported for implementation. For example, one mapping in IEC 62541-6 is to XML Web Services. In that case the *Services* described in this part appear as the Web service methods in the WSDL contract.

Not all OPC UA *Servers* will need to implement all of the defined *Services*. IEC 62541-7 defines the *Profiles* that dictate which *Services* need to be implemented in order to be compliant with a particular *Profile*.

2 Normative references

The following documents, in whole or in part, are normatively referenced in this document and are indispensable for its application. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

IEC TR 62541-1, *OPC Unified Architecture – Part 1: Overview and Concepts*

IEC TR 62541-2, *OPC Unified Architecture – Part 2: Security Model*

IEC 62541-3, *OPC unified architecture – Part 3: Address Space Model*

IEC 62541-5, *OPC unified architecture – Part 5: Information Model*

IEC 62541-6, *OPC unified architecture – Part 6: Mappings*

IEC 62541-7, *OPC unified architecture – Part 7: Profiles*

IEC 62541-8, *OPC unified architecture – Part 8: Data Access*

IEC 62541-11, *OPC Unified Architecture – Part 11: Historical Access*

IEC 62541-13 *OPC Unified Architecture – Part 13: Aggregates*

3 Terms, definitions and conventions

3.1 Terms and definitions

For the purposes of this document, the terms and definitions given in IEC TR 62541-1, IEC TR 62541-2, and IEC 62541-3, as well as the following apply.

3.1.1

Deadband

permitted range for value changes that will not trigger a data change *Notification*

Note 1 to entry: *Deadband* can be applied as a filter when subscribing to *Variables* and is used to keep noisy signals from updating the *Client* unnecessarily. This standard defines *AbsoluteDeadband* as a common filter. IEC 62541-8 defines an additional *Deadband* filter.

3.1.2

Endpoint

physical address available on a network that allows *Clients* to access one or more *Services* provided by a *Server*

Note 1 to entry: Each *Server* may have multiple *Endpoints*. The address of an *Endpoint* shall include a *HostName*.

3.1.3

Gateway Server

Server that acts as an intermediary for one or more *Servers*

Note 1 to entry: *Gateway Servers* may be deployed to limit external access, provide protocol conversion or to provide features that the underlying *Servers* do not support.

3.1.4

Hostname

unique identifier for a machine on a network

Note 1 to entry: This identifier shall be unique within a local network; however, it may also be globally unique. The identifier can be an IP address.

3.1.5

Security Token

identifier for a cryptographic key set

Note 1 to entry: All *Security Tokens* belong to a security context which is in case of OPC UA the *Secure Channel*.

3.1.6

SoftwareCertificate

digital certificate for a software product that can be installed on several hosts to describe the capabilities of the software product

Note 1 to entry: Different installations of one software product could have the same software certificate. Software certificates are not relevant for security. They are used to identify a software product and its supported features. *SoftwareCertificates* are described in 6.2.

3.2 Abbreviations and symbols

API Application Programming Interface

BNF Backus-Naur Form

CA Certificate Authority

CRL Certificate Revocation List

CTL Certificate Trust List

DA Data Access
 NAT Network Address Translation
 UA Unified Architecture
 URI Uniform Resource Identifier
 URL Uniform Resource Locator

3.3 Conventions for Service definitions

OPC UA *Services* contain parameters that are conveyed between the *Client* and the *Server*. The OPC UA *Service* specifications use tables to describe *Service* parameters, as shown in Table 1. Parameters are organised in this table into request parameters and response parameters.

Table 1 – Service Definition Table

Name	Type	Description
Request		Defines the request parameters of the <i>Service</i>
Simple Parameter Name		Description of this parameter
Constructed Parameter Name		Description of the constructed parameter
Component Parameter Name		Description of the component parameter
Response		Defines the response parameters of the <i>Service</i>

The Name, Type and Description columns contain the name, data type and description of each parameter. All parameters are mandatory, although some may be unused under certain circumstances. The Description column specifies the value to be supplied when a parameter is unused.

Two types of parameters are defined in these tables, simple and constructed. Simple parameters have a simple data type, such as *Boolean* or *String*.

Constructed parameters are composed of two or more component parameters, which can be simple or constructed. Component parameter names are indented below the constructed parameter name.

The data types used in these tables may be base types, common types to multiple *Services* or *Service*-specific types. Base data types are defined in IEC 62541-3. The base types used in *Services* are listed in Table 2. Data types that are common to multiple *Services* are defined in Clause 7. Data types that are *Service*-specific are defined in the parameter table of the *Service*.

Table 2 – Parameter Types defined in IEC 62541-3

Parameter Type
BaseDataType
Boolean
ByteString
Double
Duration
Guid
Int32
LocaleId
NodeId
QualifiedName
String
UInt16
UInt32
UInteger
UtcTime
XmlElement

The parameters of the Request and Indication service primitives are represented in Table 1 as Request parameters. Likewise, the parameters of the Response and Confirmation service primitives are represented in Table 1 as Response parameters. All request and response parameters are conveyed between the sender and receiver without change. Therefore, separate columns for request, indication, response and confirmation parameter values are not needed and have been intentionally omitted to improve readability.

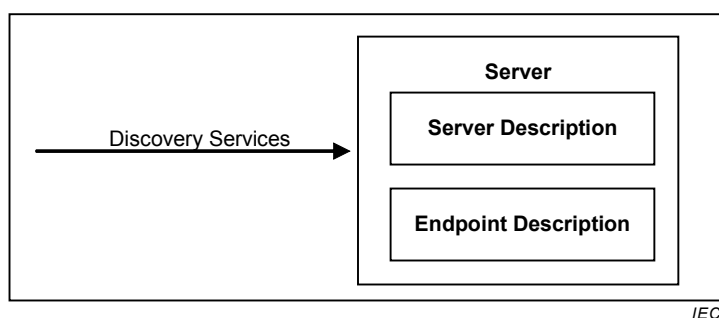
4 Overview

4.1 Service Set model

This clause specifies the OPC UA *Services*. The OPC UA *Service* definitions are abstract descriptions and do not represent a specification for implementation. The mapping between the abstract descriptions and the *Communication Stack* derived from these *Services* are defined in IEC 62541-6. In the case of an implementation as web services, the OPC UA *Services* correspond to the web service and an OPC UA *Service* corresponds to an operation of the web service.

These *Services* are organised into *Service Sets*. Each *Service Set* defines a set of related *Services*. The organisation in *Service Sets* is a logical grouping used in this standard and is not used in the implementation.

The *Discovery Service Set*, illustrated in Figure 1, defines *Services* that allow a *Client* to discover the *Endpoints* implemented by a *Server* and to read the security configuration for each of those *Endpoints*.

**Figure 1 – Discovery Service Set**

The *SecureChannel Service Set*, illustrated in Figure 2, defines *Services* that allow a *Client* to establish a communication channel to ensure the *Confidentiality* and *Integrity of Messages* exchanged with the *Server*.

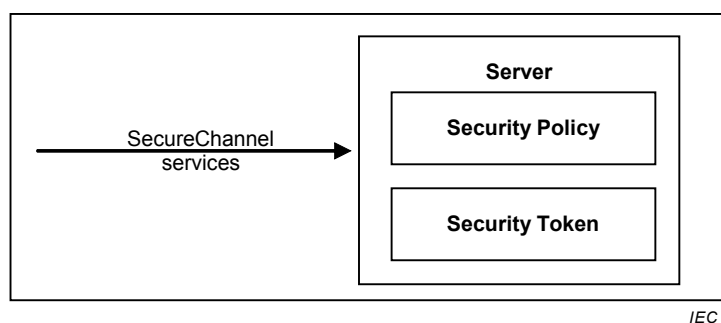


Figure 2 – SecureChannel Service Set

The *Session Service Set*, illustrated in Figure 3, defines *Services* that allow the *Client* to authenticate the user on whose behalf it is acting and to manage *Sessions*.

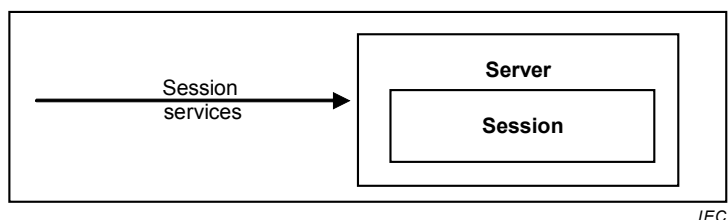


Figure 3 – Session Service Set

The *NodeManagement Service Set*, illustrated in Figure 4, defines *Services* that allow the *Client* to add, modify and delete *Nodes* in the *AddressSpace*.

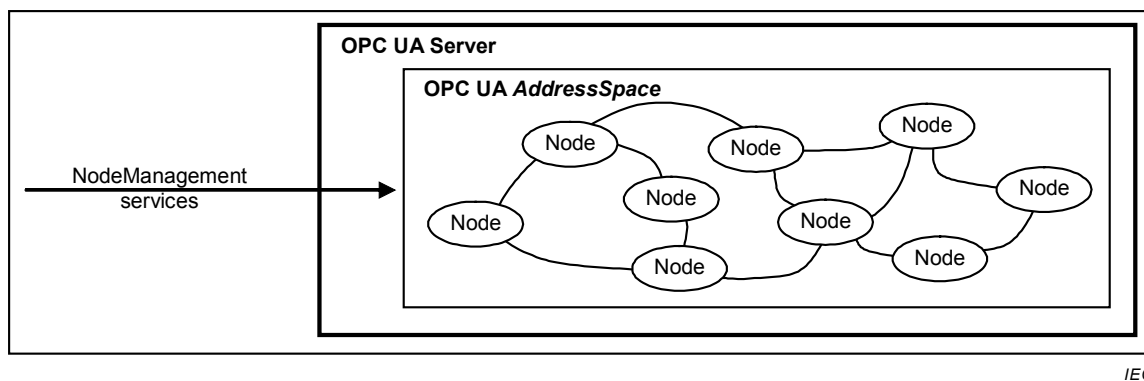


Figure 4 – NodeManagement Service Set

The *View Service Set*, illustrated in Figure 5, defines *Services* that allow *Clients* to browse through the *AddressSpace* or subsets of the *AddressSpace* called *Views*. The *Query Service Set* allows *Clients* to get a subset of data from the *AddressSpace* or the *View*.

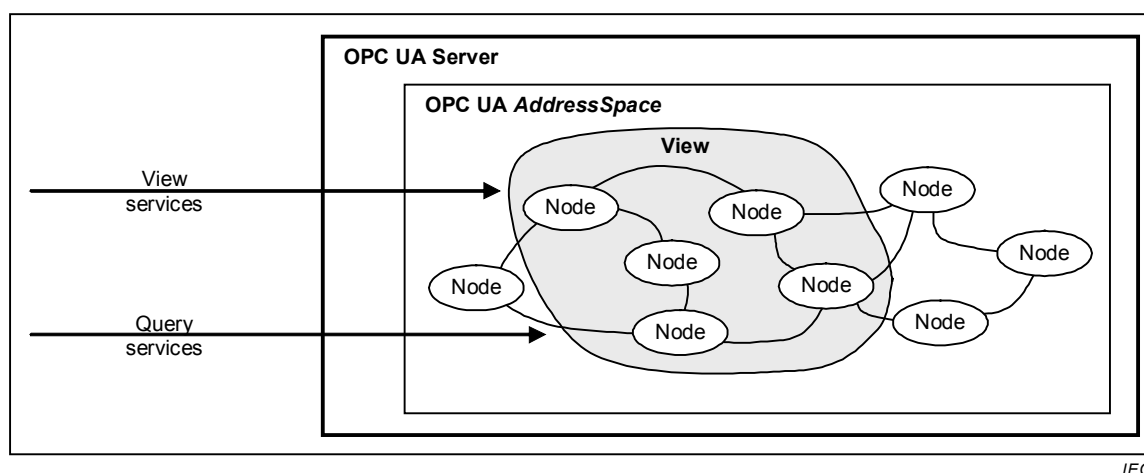


Figure 5 – View Service Set

The *Attribute Service Set* is illustrated in Figure 6. It defines *Services* that allow *Clients* to read and write *Attributes* of *Nodes*, including their historical values. Since the value of a *Variable* is modelled as an *Attribute*, these *Services* allow *Clients* to read and write the values of *Variables*.

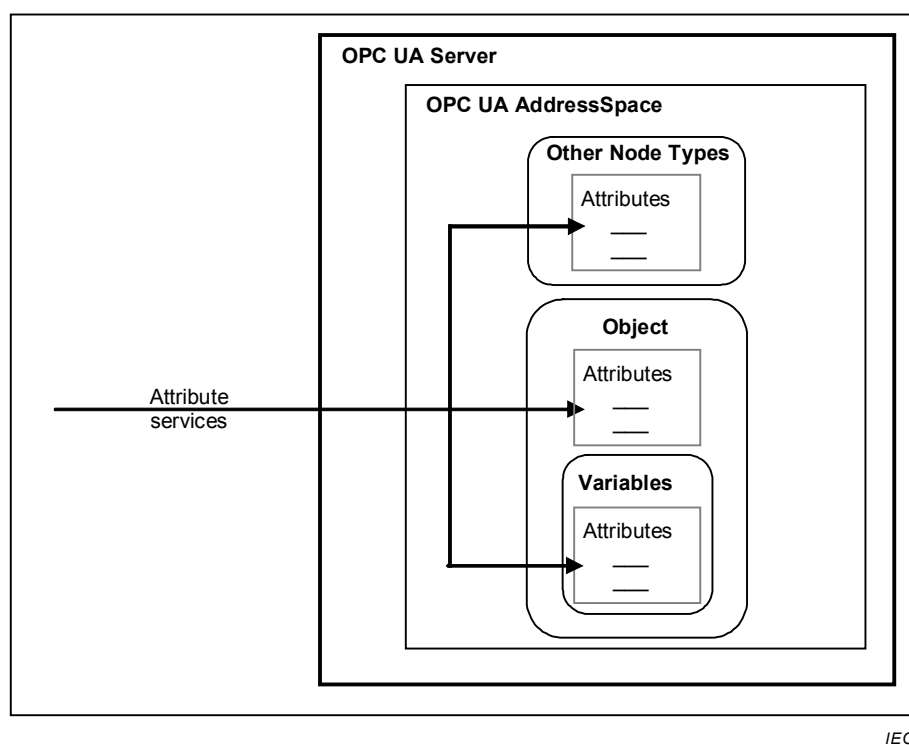


Figure 6 – Attribute Service Set

The *Method Service Set* is illustrated in Figure 7. It defines *Services* that allow *Clients* to call methods. Methods run to completion when called. They may be called with method-specific input parameters and may return method-specific output parameters.

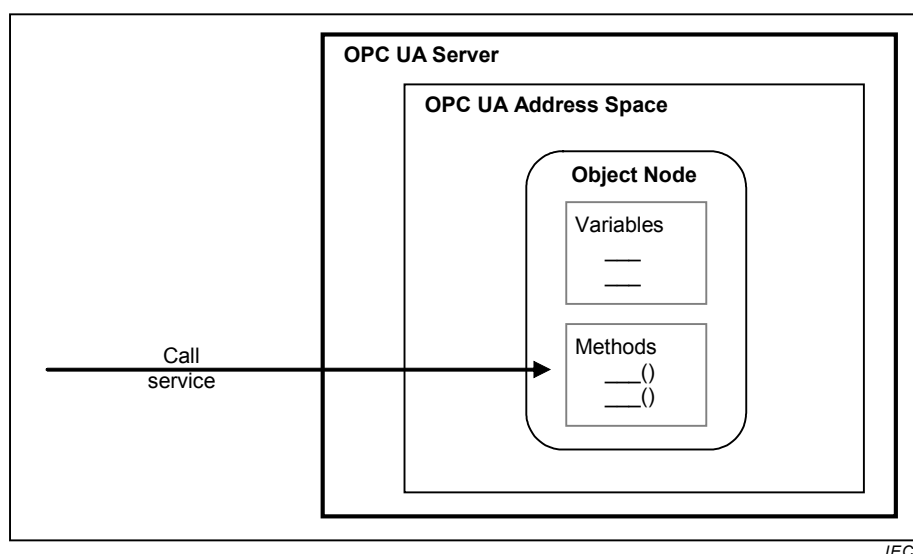


Figure 7 – Method Service Set

The *MonitoredItem Service Set* and the *Subscription Service Set*, illustrated in Figure 8, are used together to subscribe to *Nodes* in the OPC UA *AddressSpace*.

The *MonitoredItem Service Set* defines *Services* that allow *Clients* to create, modify, and delete *MonitoredItems* used to monitor *Attributes* for value changes and *Objects* for *Events*.

These *Notifications* are queued for transfer to the *Client* by *Subscriptions*.

The *Subscription Service Set* defines *Services* that allow *Clients* to create, modify and delete *Subscriptions*. *Subscriptions* send *Notifications* generated by *MonitoredItems* to the *Client*. *Subscription Services* also provide for *Client* recovery from missed *Messages* and communication failures.

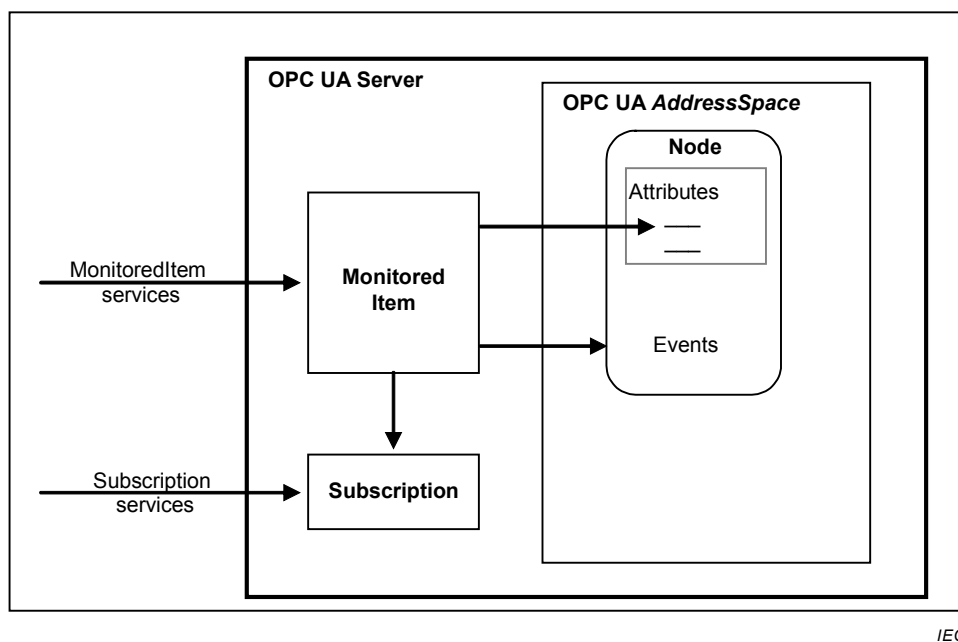


Figure 8 – MonitoredItem and Subscription Service Sets

4.2 Request/response Service procedures

Request/response *Service* procedures describe the processing of requests received by the *Server*, and the subsequent return of responses. The procedures begin with the requesting *Client* submitting a *Service* request *Message* to the *Server*.

Upon receipt of the request, the *Server* processes the *Message* in two steps. In the first step, it attempts to decode and locate the *Service* to execute. The error handling for this step is specific to the communication technology used and is described in IEC 62541-6.

If it succeeds, then it attempts to access each operation identified in the request and perform the requested operation. For each operation in the request, it generates a separate success/failure code that it includes in a positive response *Message* along with any data that is to be returned.

To perform these operations, both the *Client* and the *Server* may make use of the API of a *Communication Stack* to construct and interpret *Messages* and to access the requested operation.

The implementation of each service request or response handling shall check that each service parameter lies within the specified range for that parameter.

5 Service Sets

5.1 General

This clause defines the OPC UA *Service Sets* and their *Services*. Clause 7 contains the definitions of common parameters used by these *Services*. Clause 6.2 describes auditing requirements for all services.

Whether or not a *Server* supports a *Service Set*, or a *Service* within a *Service Set*, is defined by its *Profile*. *Profiles* are described in IEC 62541-7.

5.2 Service request and response header

Each *Service* request has a *RequestHeader* and each *Service* response has a *ResponseHeader*.

The *RequestHeader* structure is defined in 7.26 and contains common request parameters such as *authenticationToken*, *timestamp* and *requestHandle*.

The *ResponseHeader* structure is defined in 7.27 and contains common response parameters such as *serviceResult* and *diagnosticInfo*.

5.3 Service results

Service results are returned at two levels in OPC UA responses, one that indicates the status of the *Service* call, and the other that indicates the status of each operation requested by the *Service*.

Service results are defined via the *StatusCode* (see 7.33).

The status of the *Service* call is represented by the *serviceResult* contained in the *ResponseHeader* (see 7.27). The mechanism for returning this parameter is specific to the communication technology used to convey the *Service* response and is defined in IEC 62541-6.

The status of individual operations in a request is represented by individual *StatusCodes*.

The following cases define the use of these parameters.

- a) A bad code is returned in *serviceResult* if the *Service* itself failed. In this case, a *ServiceFault* is returned. The *ServiceFault* is defined in 7.28.
- b) The good code is returned in *serviceResult* if the *Service* fully or partially succeeded. In this case, other response parameters are returned. The *Client* shall always check the response parameters, especially all *StatusCodes* associated with each operation. These *StatusCodes* may indicate bad or uncertain results for one or more operations requested in the *Service* call.

All *Services* with arrays of operations in the request shall return a bad code in the *serviceResult* if the array is empty.

The *Services* define various specific *StatusCodes* and a *Server* shall use these specific *StatusCodes* as described in the *Service*. A *Client* should be able to handle these *Service* specific *StatusCodes*. In addition, a *Client* shall expect other common *StatusCodes* defined in Table 165 and Table 166. Additional details for *Client* handling of specific *StatusCodes* may be defined in IEC 62541-7.

If the *Server* discovers, through some out-of-band mechanism that the application or user credentials used to create a *Session* or *SecureChannel* have been compromised, then the *Server* should immediately terminate all sessions and channels that use those credentials. In this case, the *Service* result code should be either *Bad_IdentityTokenRejected* or *Bad_CertificateUntrusted*.

Message parsing can fail due to syntax errors or if data contained within the message exceeds ranges supported by the receiver. When this happens messages shall be rejected by the receiver. If the receiver is a *Server* then it shall return a *ServiceFault* with result code of *Bad_DecodingError* or *Bad_EncodingLimitsExceeded*. If the receiver is the *Client* then the *Communication Stack* should report these errors to the *Client* application.

Many applications will place limits on the size of messages and/or data elements contained within these messages. For example, a *Server* may reject requests containing string values longer than a certain length. These limits are typically set by administrators and apply to all connections between a *Client* and a *Server*.

Clients that receive *Bad_EncodingLimitsExceeded* faults from the *Server* will likely have to reformulate their requests. The administrator may need to increase the limits for the *Client* if it receives a response from the *Server* with this fault.

In some cases, parsing errors are fatal and it is not possible to return a fault. For example, the incoming message could exceed the buffer capacity of the receiver. In these cases, these errors may be treated as a communication fault which requires the *SecureChannel* to be re-established (see 5.5).

The *Client* and *Server* reduce the chances of a fatal error by exchanging their message size limits in the *CreateSession* service. This will allow either party to avoid sending a message that causes a communication fault. The *Server* should return a *Bad_ResponseTooLarge* fault if a serialized response message exceeds the message size specified by the *Client*. Similarly, the *Client Communication Stack* should report a *Bad_RequestTooLarge* error to the application before sending a message that exceeds the *Server's* limit.

Note that the message size limits only apply to the raw message body and do not include headers or the effect of applying any security. This means that a message body that is smaller than the specified maximum could still cause a fatal error.

5.4 Discovery Service Set

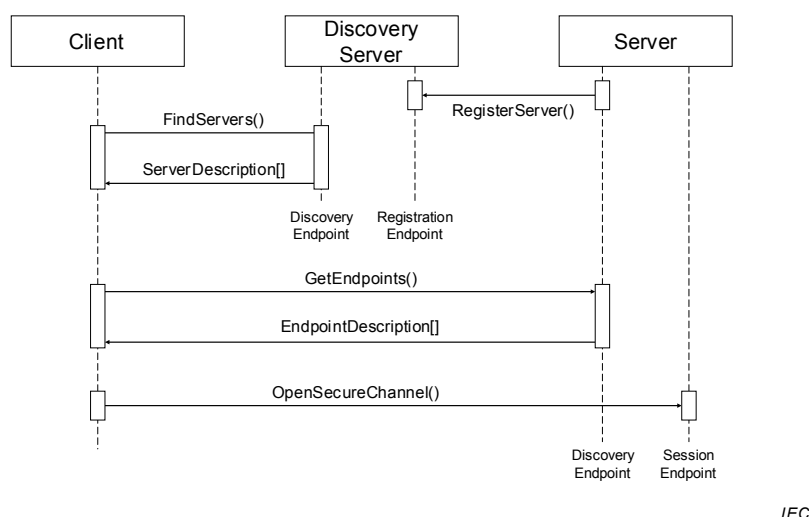
5.4.1 Overview

This *Service Set* defines *Services* used to discover the *Endpoints* implemented by a *Server* and to read the security configuration for those *Endpoints*. The *Discovery Services* are implemented by individual *Servers* and by dedicated *Discovery Servers*. IEC 62541-12¹ describes how to use the *Discovery Services* with dedicated *Discovery Servers*.

Every *Server* shall have a *Discovery Endpoint* that *Clients* can access without establishing a *Session*. This *Endpoint* may or may not be the same *Session Endpoint* that *Clients* use to establish a *SecureChannel*. *Clients* read the security information necessary to establish a *SecureChannel* by calling the *GetEndpoints* Service on the *Discovery Endpoint*.

In addition, *Servers* may register themselves with a well known *Discovery Server* using the *RegisterServer* Service. *Clients* can later discover any registered *Servers* by calling the *FindServers* Service on the *Discovery Server*.

The complete discovery process is illustrated in Figure 9.



IEC

Figure 9 – Discovery process

The URL for a *Discovery Endpoint* shall provide all of the information that the *Client* needs to connect to the *Discovery Endpoint*.

Once a *Client* retrieves the *Endpoints*, the *Client* can save this information and use it to connect directly to the *Server* again without going through the discovery process. If the *Client* finds that it cannot connect then the *Server* configuration may have changed and the *Client* needs to go through the discovery process again.

Discovery Endpoints shall not require any message security, but it may require transport layer security. In production systems, Administrators may disable discovery for security reasons and *Clients* shall rely on cached *EndpointDescriptions*. To provide support for systems with disabled *Discovery Services* *Clients* shall allow Administrators to manually update the *EndpointDescriptions* used to connect to a *Server*. *Servers* shall allow Administrators to disable the *Discovery Endpoint*.

¹ Under consideration.

A *Client* shall be careful when using the information returned from a *Discovery Endpoint* since it has no security. A *Client* does this by comparing the information returned from the *Discovery Endpoint* to the information returned in the *CreateSession* response. A *Client* shall verify that:

- a) The *ApplicationUri* specified in the *Server Certificate* is the same as the *ApplicationUri* provided in the *EndpointDescription*.
- b) The *Server Certificate* returned in *CreateSession* response is the same as the *Certificate* used to create the *SecureChannel*.
- c) The *EndpointDescriptions* returned from the *Discovery Endpoint* are the same as the *EndpointDescriptions* returned in the *CreateSession* response.

If the *Client* detects that one of the above requirements is not fulfilled, then the *Client* shall close the *SecureChannel* and report an error.

A *Client* shall verify the *HostName* specified in the *Server Certificate* is the same as the *HostName* contained in the *endpointUrl* provided in the *EndpointDescription* returned by *CreateSession*. If there is a difference then the *Client* shall report the difference and may close the *SecureChannel*. *Servers* shall add all possible *HostNames* like *MyHost* and *MyHost.local* into the *Server Certificate*. This includes IP addresses of the host or the *HostName* exposed by a NAT router used to connect to the *Server*.

5.4.2 FindServers

5.4.2.1 Description

This *Service* returns the *Servers* known to a *Server* or *Discovery Server*. The behaviour of *Discovery Servers* is described in detail in IEC 62541-12².

The *Client* may reduce the number of results returned by specifying filter criteria. A *Discovery Server* returns an empty list if no *Servers* match the criteria specified by the client. The filter criteria supported by this *Service* are described in 5.4.2.2.

Every *Server* shall provide a *Discovery Endpoint* that supports this *Service*. The *Server* shall always return a record that describes itself, however in some cases more than one record may be returned. *Gateway Servers* shall return a record for each *Server* that they provide access to plus (optionally) a record that allows the *Gateway Server* to be accessed as an ordinary OPC UA *Server*. Non-transparent redundant *Servers* shall provide a record for each *Server* in the redundant set.

Every *Server* shall have a globally unique identifier called the *ServerUri*. This identifier should be a fully qualified domain name; however, it may be a GUID or similar construct that ensures global uniqueness. The *ServerUri* returned by this *Service* shall be the same value that appears in index 0 of the *ServerArray* property (see IEC 62541-5). The *ServerUri* is returned as the *applicationUri* field in the *ApplicationDescription* (see 7.1)

Every *Server* shall also have a human readable identifier called the *ServerName* which is not necessarily globally unique. This identifier may be available in multiple locales.

A *Server* may have multiple *HostNames*. For this reason, the *Client* shall pass the URL it used to connect to the *Endpoint* to this *Service*. The implementation of this *Service* shall use this information to return responses that are accessible to the *Client* via the provided URL.

This *Service* shall not require any message security but it may require transport layer security.

² Under consideration.

Some *Servers* may be accessed via a *Gateway Server* and shall have a value specified for *gatewayServerUri* in their *ApplicationDescription* (see 7.1). The *discoveryUrls* provided in *ApplicationDescription* shall belong to the *Gateway Server*. Some *Discovery Servers* may return multiple records for the same *Server* if that *Server* can be accessed via multiple paths.

This *Service* can be used without security and it is therefore vulnerable to Denial Of Service (DOS) attacks. A *Server* should minimize the amount of processing required to send the response for this *Service*. This can be achieved by preparing the result in advance. The *Server* should also add a short delay before starting processing of a request during high traffic conditions.

The *DiscoveryUrl* returned by this *Service* is ambiguous if there are multiple *TransportProfiles* (e.g. UA XML or UA Binary encoding) associated with the URL scheme. *Clients* that support multiple *TransportProfiles* should attempt to use alternate *TransportProfiles* if the first choice does not succeed.

5.4.2.2 Parameters

Table 3 defines the parameters for the *Service*.

Table 3 – FindServers Service Parameters

Name	Type	Description
Request		
requestHeader	RequestHeader	Common request parameters. The <i>authenticationToken</i> is always omitted. The <i>authenticationToken</i> shall be ignored if it is provided. The type <i>RequestHeader</i> is defined in 7.26.
endpointUrl	String	The network address that the <i>Client</i> used to access the <i>Discovery Endpoint</i> . The <i>Server</i> uses this information for diagnostics and to determine what URLs to return in the response. The <i>Server</i> should return a suitable default URL if it does not recognize the <i>HostName</i> in the URL.
localeIds []	LocaleId	List of locales to use. The server should return the <i>ServerName</i> using one of locales specified. If the server supports more than one of the requested locales then the server shall use the locale that appears first in this list. If the server does not support any of the requested locales it chooses an appropriate default locale. The server chooses an appropriate default locale if this list is empty.
serverUri []	String	List of servers to return. All known servers are returned if the list is empty. A <i>serverUri</i> matches the <i>applicationUri</i> from the <i>ApplicationDescription</i> defined in 7.1.
Response		
responseHeader	ResponseHeader	Common response parameters. The <i>ResponseHeader</i> type is defined in 7.27.
servers []	ApplicationDescription	List of <i>Servers</i> that meet criteria specified in the request. This list is empty if no servers meet the criteria. The <i>ApplicationDescription</i> type is defined in 7.1.

5.4.2.3 Service results

Common *StatusCodes* are defined in Table 165.

5.4.3 GetEndpoints

5.4.3.1 Description

This *Service* returns the *Endpoints* supported by a *Server* and all of the configuration information required to establish a *SecureChannel* and a *Session*.

This *Service* shall not require any message security but it may require transport layer security.

A *Client* may reduce the number of results returned by specifying filter criteria based on *LocaleIds* and *Transport Profile* URIs. The *Server* returns an empty list if no *Endpoints* match the criteria specified by the client. The filter criteria supported by this *Service* are described in 5.4.3.2.

A *Server* may support multiple security configurations for the same *Endpoint*. In this situation, the *Server* shall return separate *EndpointDescription* records for each available configuration. *Clients* should treat each of these configurations as distinct *Endpoints* even if the physical URL happens to be the same.

The security configuration for an *Endpoint* has four components:

- Server Application Instance Certificate
- Message Security Mode
- Security Policy
- Supported User Identity Tokens

The *ApplicationInstanceCertificate* is used to secure the *OpenSecureChannel* request (see 5.5.2). The *MessageSecurityMode* and the *SecurityPolicy* tell the *Client* how to secure messages sent via the *SecureChannel*. The *UserIdentityTokens* tell the client which type of user credentials shall be passed to the *Server* in the *ActivateSession* request (see 5.6.3).

If the *securityPolicyUri* is NONE and none of the *UserTokenPolicies* requires encryption, the *Client* shall ignore the *ApplicationInstanceCertificate*.

Each *EndpointDescription* also specifies a URI for the *Transport Profile* that the *Endpoint* supports. The *Transport Profiles* specify information such as message encoding format and protocol version and are defined in IEC 62541-7. *Clients* shall fetch the *Server's SoftwareCertificates* if they want to discover the complete list of *Profiles* supported by the *Server* (see 7.30).

Messages are secured by applying standard cryptography algorithms to the messages before they are sent over the network. The exact set of algorithms used depends on the *SecurityPolicy* for the *Endpoint*. IEC 62541-7 defines *Profiles* for common *SecurityPolicies* and assigns a unique URI to them. It is expected that applications have built in knowledge of the *SecurityPolicies* that they support, as a result, only the Profile URI for the *SecurityPolicy* is specified in the *EndpointDescription*. A *Client* cannot connect to an *Endpoint* that does not support a *SecurityPolicy* that it recognizes.

An *EndpointDescription* may specify that the message security mode is NONE. This configuration is not recommended unless the applications are communicating on a physically isolated network where the risk of intrusion is extremely small. If the message security is NONE then it is possible for *Clients* to deliberately or accidentally hijack *Sessions* created by other *Clients*.

A *Server* may have multiple *HostNames*. For this reason, the *Client* shall pass the URL it used to connect to the *Endpoint* to this *Service*. The implementation of this *Service* shall use this information to return responses that are accessible to the *Client* via the provided URL.

This *Service* can be used without security and it is therefore vulnerable to Denial Of Service (DOS) attacks. A *Server* should minimize the amount of processing required to send the response for this *Service*. This can be achieved by preparing the result in advance. The *Server* should also add a short delay before starting processing of a request during high traffic conditions.

Some of the *EndpointDescriptions* returned in a response shall specify the *Endpoint* information for a *Gateway Server* that can be used to access another *Server*. In these situations, the *gatewayServerUri* is specified in the *EndpointDescription* and all security

checks used to verify *Certificates* shall use the *gatewayServerUri* (see 6.1.3) instead of the *serverUri*.

To connect to a *Server* via the gateway the *Client* shall first establish a *SecureChannel* with the *Gateway Server*. Then the *Client* shall call the *CreateSession* service and pass the *serverUri* specified in the *EndpointDescription* to the *Gateway Server*. The *Gateway Server* shall then connect to the underlying *Server* on behalf of the *Client*. The process of connecting to a *Server* via a *Gateway Server* is illustrated in Figure 10.

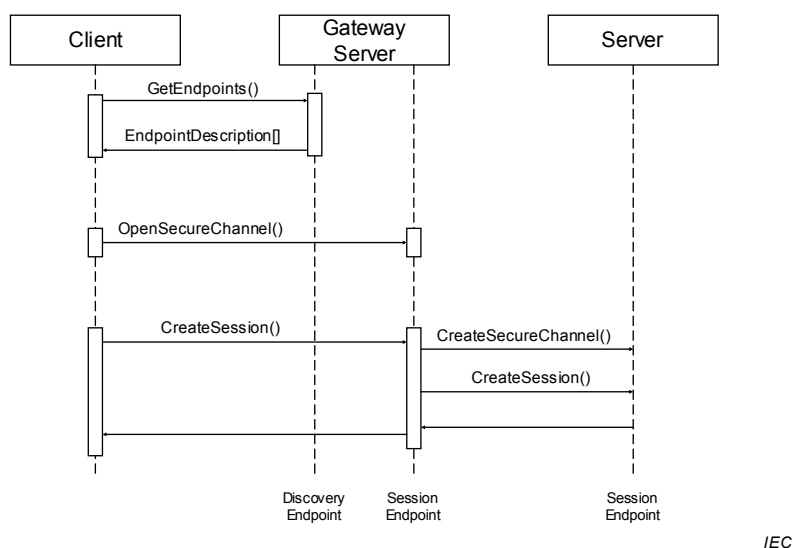


Figure 10 – Using a Gateway Server

5.4.3.2 Parameters

Table 4 defines the parameters for the *Service*.

Table 4 – GetEndpoints Service Parameters

Name	Type	Description
Request		
requestHeader	RequestHeader	Common request parameters. The <i>authenticationToken</i> is always omitted. The <i>authenticationToken</i> shall be ignored if it is provided. The type <i>RequestHeader</i> is defined in 7.26.
endpointUrl	String	The network address that the <i>Client</i> used to access the <i>Discovery Endpoint</i> . The <i>Server</i> uses this information for diagnostics and to determine what URLs to return in the response. The <i>Server</i> should return a suitable default URL if it does not recognize the <i>HostName</i> in the URL.
localeIds []	LocaleId	List of locales to use. Specifies the locale to use when returning human readable strings. This parameter is described in 5.4.2.2.
profileUris []	String	List of <i>Transport Profile</i> that the returned <i>Endpoints</i> shall support. IEC 62541-7 defines URIs for the <i>Transport Profiles</i> . All <i>Endpoints</i> are returned if the list is empty.
Response		
responseHeader	ResponseHeader	Common response parameters. The <i>ResponseHeader</i> type is defined in 7.27.
Endpoints []	EndpointDescription	List of <i>Endpoints</i> that meet criteria specified in the request. This list is empty if no <i>Endpoints</i> meet the criteria. The <i>EndpointDescription</i> type is defined in 7.9.

5.4.3.3 Service Results

Common *StatusCodes* are defined in Table 165.

5.4.4 RegisterServer

5.4.4.1 Description

This *Service* registers a *Server* with a *Discovery Server*. This *Service* will be called by a *Server* or a separate configuration utility. *Clients* will not use this *Service*.

A *Server* shall establish a *SecureChannel* with the *Discovery Server* before calling this *Service*. The *SecureChannel* is described in 5.5. The *Administrator* of the *Server* shall provide the *Server* with an *EndpointDescription* for the *Discovery Server* as part of the configuration process. *Discovery Servers* shall reject registrations if the *serverUri* provided does not match the *applicationUri* in *Server Certificate* used to create the *SecureChannel*.

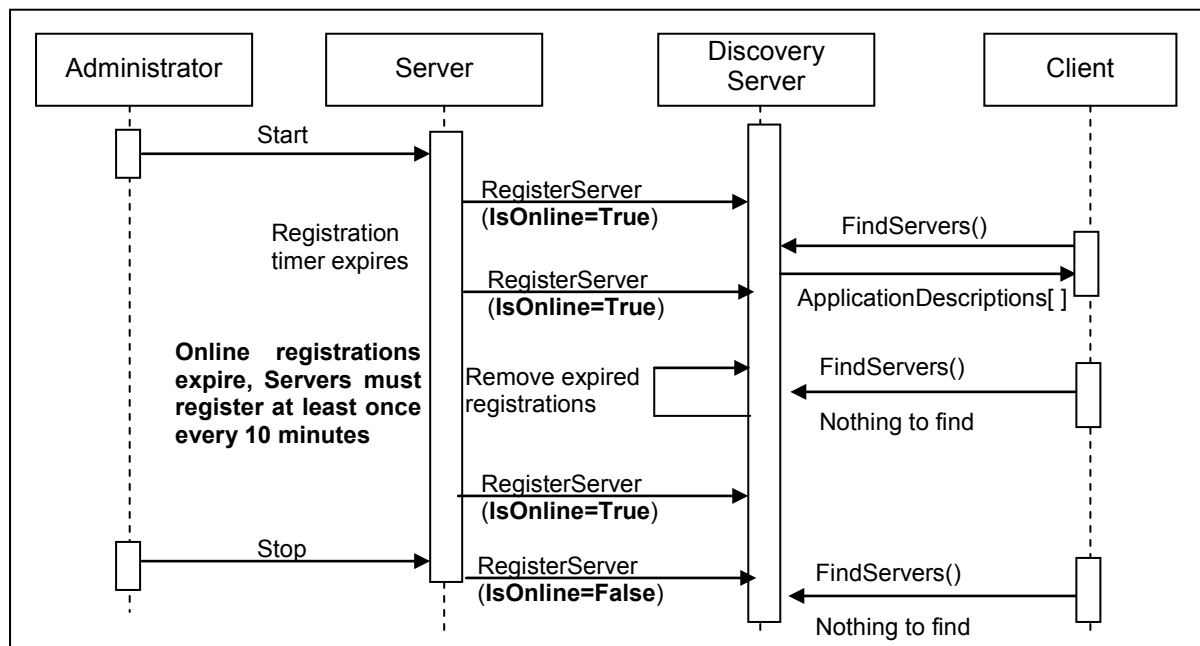
A *Server* only provides its *serverUri* and the URLs of the *Discovery Endpoints* to the *Discovery Server*. *Clients* shall use the *GetEndpoints* service to fetch the most up to date configuration information directly from the *Server*.

The *Server* shall provide a localized name for itself in all locales that it supports.

Servers shall be able to register themselves with a *Discovery Server* running on the same machine. The exact mechanisms depend on the *Discovery Server* implementation and are described in IEC 62541-6.

There are two types of *Server* applications: those which are manually launched including a start by the operating system at boot and those that are automatically launched when a *Client* attempts to connect. The registration process that a *Server* shall use depends on which category it falls into.

The registration process for manually launched *Servers* is illustrated in Figure 11.



IEC

Figure 11 – The Registration Process – Manually Launched Servers

The registration process for automatically launched *Servers* is illustrated in Figure 12.

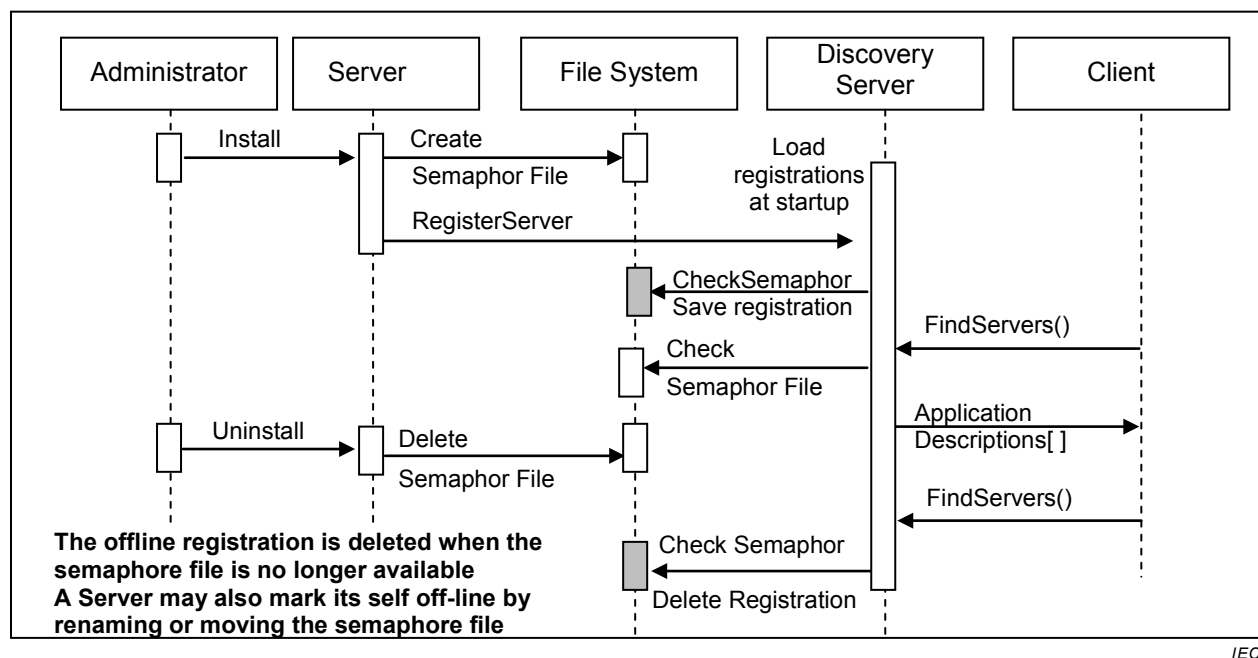


Figure 12 – The Registration Process – Automatically Launched Servers

The registration process is designed to be platform independent, robust and able to minimize problems created by configuration errors. For that reason, *Servers* shall register themselves more than once.

Under normal conditions, manually launched *Servers* shall periodically register with the *Discovery Server* as long as they are able to receive connections from *Clients*. If a *Server* goes offline then it shall register itself once more and indicate that it is going offline. The registration frequency should be configurable; however, the maximum is 10 minutes. If an error occurs during registration (e.g. the *Discovery Server* is not running) then the *Server* shall periodically re-attempt registration. The frequency of these attempts should start at 1 second but gradually increase until the registration frequency is the same as what it would be if no errors occurred. The recommended approach would be to double the period of each attempt until reaching the maximum.

When an automatically launched *Server* (or its install program) registers with the *Discovery Server* it shall provide a path to a semaphore file which the *Discovery Server* can use to determine if the *Server* has been uninstalled from the machine. The *Discovery Server* shall have read access to the file system that contains the file.

5.4.4.2 Parameters

Table 5 defines the parameters for the *Service*.

Table 5 – RegisterServer Service Parameters

Name	Type	Description
Request		
requestHeader	RequestHeader	Common request parameters. The <i>authenticationToken</i> is always omitted. The type <i>RequestHeader</i> is defined in 7.26.
Server	RegisteredServer	The server to register. This structure is defined in-line with the following indented items.
serverUri	String	The globally unique identifier for the <i>Server</i> instance. The <i>serverUri</i> matches the <i>applicationUri</i> from the <i>ApplicationDescription</i> defined in 7.1.
productUri	String	The globally unique identifier for the <i>Server</i> product.
serverNames []	LocalizedText	A list of localized descriptive names for the <i>Server</i> . The list shall have at least one valid entry.
serverType	Enum ApplicationType	The type of application. The enumeration values are defined in Table 103. The value “CLIENT_1” (The application is a <i>Client</i>) is not allowed. The <i>Service</i> result shall be <i>Bad_InvalidArgument</i> in this case.
gatewayServerUri	String	The URI of the <i>Gateway Server</i> associated with the <i>discoveryUrls</i> . This value is only specified by <i>Gateway Servers</i> that wish to register the <i>Servers</i> that they provide access to. For <i>Servers</i> that do not act as a <i>Gateway Server</i> this parameter shall be null.
discoveryUrls []	String	A list of <i>Discovery Endpoints</i> for the <i>Server</i> . The list shall have at least one valid entry.
semaphoreFilePath	String	The path to the semaphore file used to identify an automatically-launched server instance; Manually-launched servers will not use this parameter. If a Semaphore file is provided, the <i>isOnline</i> flag is ignored. If a Semaphore file is provided and exists, the <i>LocalDiscoveryServer</i> shall save the registration information in a persistent data store that it reads whenever the <i>LocalDiscoveryServer</i> starts. If a Semaphore file is specified but does not exist the <i>Discovery Server</i> shall remove the registration from any persistent data store. If the <i>Server</i> has registered with a <i>semaphoreFilePath</i> , the <i>DiscoveryServer</i> shall check that this file exists before returning the <i>ApplicationDescription</i> to the client. If the <i>Server</i> did not register with a <i>semaphoreFilePath</i> (it is null or empty) then the <i>DiscoveryServer</i> does not attempt to verify the existence of the file before returning the <i>ApplicationDescription</i> to the client.
isOnline	Boolean	True if the <i>Server</i> is currently able to accept connections from <i>Clients</i> . The <i>DiscoveryServer</i> shall return <i>ApplicationDescriptions</i> to the <i>Client</i> . The <i>Server</i> is expected to periodically re-register with the <i>DiscoveryServer</i> . False if the <i>Server</i> is currently unable to accept connections from <i>Clients</i> . The <i>DiscoveryServer</i> shall NOT return <i>ApplicationDescriptions</i> to the <i>Client</i> . This parameter is ignored if a <i>semaphoreFilePath</i> is provided.
Response		
ResponseHeader	ResponseHeader	Common response parameters. The type <i>ResponseHeader</i> is defined in 7.27.

5.4.4.3 Service Results

Table 6 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 6 – RegisterServer Service Result Codes

Symbolic Id	Description
Bad_InvalidArgument	See Table 165 for the description of this result code.
Bad_ServerUriInvalid	See Table 165 for the description of this result code.
Bad_ServerNameMissing	No <i>ServerName</i> was specified.
Bad_DiscoveryUriMissing	No discovery URL was specified.
Bad_SemaphoreFileMissing	The semaphore file specified is not valid.

5.5 SecureChannel Service Set

5.5.1 Overview

This *Service Set* defines *Services* used to open a communication channel that ensures the confidentiality and *Integrity* of all *Messages* exchanged with the *Server*. The base concepts for OPC UA security are defined in IEC TR 62541-2.

The *SecureChannel Services* are unlike other *Services* because they are not implemented directly by the *OPC UA Application*. Instead, they are provided by the *Communication Stack* on which the *OPC UA Application* is built. For example, an OPC UA *Server* may be built on a SOAP stack that allows applications to establish a *SecureChannel* using the WS Secure Conversation specification. In these cases, the *OPC UA Application* shall verify that the *Message* it received was in the context of a WS Secure Conversation. IEC 62541-6 describes how the *SecureChannel Services* are implemented.

A *SecureChannel* is a long-running logical connection between a single *Client* and a single *Server*. This channel maintains a set of keys known only to the *Client* and *Server*, which are used to authenticate and encrypt *Messages* sent across the network. The *SecureChannel Services* allow the *Client* and *Server* to securely negotiate the keys to use.

An *EndpointDescription* tells a *Client* how to establish a *SecureChannel* with a given *Endpoint*. A *Client* may obtain the *EndpointDescription* from a *Discovery Server*, via some non-UA defined directory server or from its own configuration.

The exact algorithms used to authenticate and encrypt *Messages* are described in the *SecurityPolicy* field of the *EndpointDescription*. A *Client* shall use these algorithms when it creates a *SecureChannel*.

It should be noted that some *SecurityPolicies* defined in IEC 62541-7 will turn off authentication and encryption resulting in a *SecureChannel* that provides no security.

When a *Client* and *Server* are communicating via a *SecureChannel*, they shall verify that all incoming *Messages* have been signed and encrypted according to the requirements specified in the *EndpointDescription*. An *OPC UA Application* shall not process any *Message* that does not conform to these requirements.

The relationship between the *SecureChannel* and the *OPC UA Application* depends on the implementation technology. IEC 62541-6 defines any requirements that depend on the technology used.

The correlation between the *OPC UA Application Session* and the *SecureChannel* is illustrated in Figure 13. The *Communication Stack* is used by the *OPC UA Applications* to exchange *Messages*. In the first step, the *SecureChannel Services* are used to establish a *SecureChannel* between the two *Communication Stacks* which allows the secure exchange of *Messages*. In the second step, the *OPC UA Applications* use the *Session Service Set* to establish an *OPC UA Application Session*.

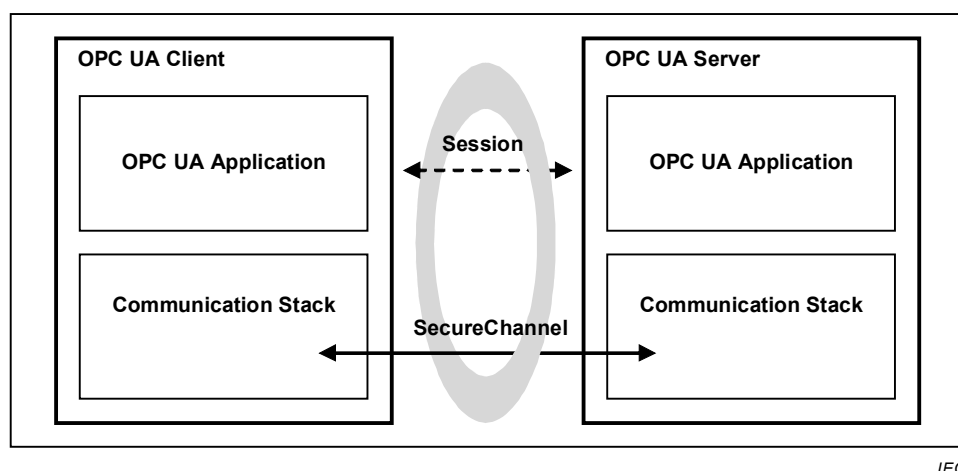


Figure 13 – SecureChannel and Session Services

Once a *Client* has established a *Session* it may wish to access the *Session* from a different *SecureChannel*. The *Client* can do this by validating the new *SecureChannel* with the *ActivateSession Service* described in 5.6.3.

If a *Server* acts as a *Client* to other *Servers*, which is commonly referred to as *Server chaining*, then the *Server* shall be able to maintain user level security. By this we mean that the user identity should be passed to the underlying server or it should be mapped to an appropriate user identity in the underlying server. It is unacceptable to ignore user level security. This is required to ensure that security is maintained and that a user does not obtain information that they should not have access to. Whenever possible a *Server* should impersonate the original *Client* by passing the original *Client's* user identity to the underlying *Server* when it calls the *ActivateSession Service*. If impersonation is not an option then the *Server* shall map the original *Client's* user identity onto a new user identity which the underlying *Server* does recognize.

5.5.2 OpenSecureChannel

5.5.2.1 Description

This *Service* is used to open or renew a *SecureChannel* that can be used to ensure *Confidentiality* and *Integrity* for *Message* exchange during a *Session*. This *Service* requires the *Communication Stack* to apply the various security algorithms to the *Messages* as they are sent and received. Specific implementations of this *Service* for different *Communication Stacks* are described in IEC 62541-6.

Each *SecureChannel* has a globally-unique identifier and is valid for a specific combination of *Client* and *Server* application instances. Each channel contains one or more *SecurityTokens* that identify a set of cryptography keys that are used to encrypt and authenticate *Messages*. *SecurityTokens* also have globally-unique identifiers which are attached to each *Message* secured with the token. This allows an authorized receiver to know how to decrypt and verify the *Message*.

SecurityTokens have a finite lifetime negotiated with this *Service*. However, differences between the system clocks on different machines and network latencies mean that valid *Messages* could arrive after the token has expired. To prevent valid *Messages* from being discarded, the applications should do the following:

- a) *Clients* should request a new *SecurityToken* after 75 % of its lifetime has elapsed. This should ensure that *Clients* will receive the new *SecurityToken* before the old one actually expires.

- b) *Servers* shall use the existing *SecurityToken* to secure outgoing *Messages* until the *SecurityToken* expires or the *Server* receives a *Message* secured with a new *SecurityToken*. This should ensure that *Clients* do not reject *Messages* secured with the new *SecurityToken* that arrive before the *Client* receives the new *SecurityToken*.
- c) *Clients* should accept *Messages* secured by an expired *SecurityToken* for up to 25 % of the token lifetime. This should ensure that *Messages* sent by the *Server* before the token expired are not rejected because of network delays.

Each *SecureChannel* exists until it is explicitly closed or until the last token has expired and the overlap period has elapsed. A *Server* application should limit the number of *SecureChannels*. To protect against misbehaving *Clients* and denial of service attacks, the *Server* shall close the oldest *SecureChannel* that has no *Session* assigned before reaching the maximum number of supported *SecureChannels*.

The *OpenSecureChannel* request and response *Messages* shall be signed with the sender's *Certificate*. These *Messages* shall always be encrypted. If the transport layer does not provide encryption, then these *Messages* shall be encrypted with the receiver's *Certificate*. These requirements for *OpenSecureChannel* only apply if the *securityPolicyUri* is not None.

The *Certificates* used in the *OpenSecureChannel* service shall be the *Application Instance Certificates*. *Clients* and *Servers* shall verify that the same *Certificates* were used in the *CreateSession* and *ActivateSession* services. *Certificates* are not provided and shall not be verified if the *securityPolicyUri* is None.

If the *securityPolicyUri* is not None, a *Client* shall verify the *HostName* specified in the *Server Certificate* is the same as the *HostName* contained in the *endpointUrl*. If there is a difference then the *Client* shall report the difference and may choose to not open the *SecureChannel*. *Servers* shall add all possible *HostNames* like *MyHost* and *MyHost.local* into the *Server Certificate*. This includes IP addresses of the host or the *HostName* exposed by a NAT router used to connect to the *Server*.

Clients should be prepared to replace the *HostName* returned in the *EndPointDescription* with the *HostName* or the IP addresses they used to call *GetEndpoints*.

5.5.2.2 Parameters

Table 7 defines the parameters for the *Service*.

Unlike other *Services*, the parameters for this *Service* provide only an abstract definition. The concrete representation on the network depends on the mappings defined in IEC 62541-6.

Table 7 – OpenSecureChannel Service Parameters

Name	Type	Description
Request		
requestHeader	RequestHeader	Common request parameters. The <i>authenticationToken</i> is always omitted. The type <i>RequestHeader</i> is defined in 7.26.
clientCertificate	ApplicationInstanceCertificate	A <i>Certificate</i> that identifies the <i>Client</i> . The <i>OpenSecureChannel</i> request shall be signed with this <i>Certificate</i> . The <i>ApplicationInstanceCertificate</i> type is defined in 7.2. If the <i>securityPolicyUri</i> is None, the <i>Server</i> shall ignore the <i>ApplicationInstanceCertificate</i> .
requestType	Enum SecurityToken RequestType	The type of <i>SecurityToken</i> request: An enumeration that shall be one of the following: ISSUE_0 creates a new <i>SecurityToken</i> for a new <i>SecureChannel</i> . RENEW_1 creates a new <i>SecurityToken</i> for an existing <i>SecureChannel</i> .
secureChannelId	ByteString	The identifier for the <i>SecureChannel</i> that the new token should belong to. This parameter shall be null when creating a new <i>SecureChannel</i> .
securityMode	Enum MessageSecurityMode	The type of security to apply to the messages. The type <i>MessageSecurityMode</i> type is defined in 7.14. A <i>SecureChannel</i> may have to be created even if the <i>securityMode</i> is NONE. The exact behaviour depends on the mapping used and is described in the IEC 62541-6.
securityPolicyUri	String	The URI for <i>SecurityPolicy</i> to use when securing messages sent over the <i>SecureChannel</i> . The set of known URIs and the <i>SecurityPolicies</i> associated with them are defined in IEC 62541-7.
clientNonce	ByteString	A random number that shall not be used in any other request. A new <i>clientNonce</i> shall be generated for each time a <i>SecureChannel</i> is renewed. This parameter shall have a length equal to key size used for the symmetric encryption algorithm that is identified by the <i>securityPolicyUri</i> .
requestedLifetime	Duration	The requested lifetime, in milliseconds, for the new <i>SecurityToken</i> . It specifies when the <i>Client</i> expects to renew the <i>SecureChannel</i> by calling the <i>OpenSecureChannel Service</i> again. If a <i>SecureChannel</i> is not renewed, then all <i>Messages</i> sent using the current <i>SecurityTokens</i> shall be rejected by the receiver.
Response		
responseHeader	ResponseHeader	Common response parameters (see 7.27 for <i>ResponseHeader</i> type definition).
securityToken	ChannelSecurityToken	Describes the new <i>SecurityToken</i> issued by the <i>Server</i> . This structure is defined in-line with the following indented items.
channelId	ByteString	A unique identifier for the <i>SecureChannel</i> . This is the identifier that shall be supplied whenever the <i>SecureChannel</i> is renewed.
tokenId	ByteString	A unique identifier for a single <i>SecurityToken</i> within the channel. This is the identifier that shall be passed with each <i>Message</i> secured with the <i>SecurityToken</i> .
createdAt	UtcTime	The time when the <i>SecurityToken</i> was created.
revisedLifetime	Duration	The lifetime of the <i>SecurityToken</i> in milliseconds. The UTC expiration time for the token may be calculated by adding the lifetime to the <i>createdAt</i> time.
serverNonce	ByteString	A random number that shall not be used in any other request. A new <i>serverNonce</i> shall be generated for each time a <i>SecureChannel</i> is renewed. This parameter shall have a length equal to key size used for the symmetric encryption algorithm that is identified by the <i>securityPolicyUri</i> .

5.5.2.3 Service results

Table 8 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 8 – OpenSecureChannel Service Result Codes

Symbolic Id	Description
Bad_SecurityChecksFailed	See Table 165 for the description of this result code.
Bad_CertificateTimeInvalid	See Table 165 for the description of this result code.
Bad_CertificateIssuerTimeInvalid	See Table 165 for the description of this result code.
Bad_CertificateHostNameInvalid	See Table 165 for the description of this result code.
Bad_CertificateUriInvalid	See Table 165 for the description of this result code.
Bad_CertificateUseNotAllowed	See Table 165 for the description of this result code.
Bad_CertificateIssuerUseNotAllowed	See Table 165 for the description of this result code.
Bad_CertificateUntrusted	See Table 165 for the description of this result code.
Bad_CertificateRevocationUnknown	See Table 165 for the description of this result code.
Bad_CertificateIssuerRevocationUnknown	See Table 165 for the description of this result code.
Bad_CertificateRevoked	See Table 165 for the description of this result code.
Bad_CertificateIssuerRevoked	See Table 165 for the description of this result code.
Bad_RequestTypeInvalid	The security token request type is not valid.
Bad_SecurityModeRejected	The security mode does not meet the requirements set by the <i>Server</i> .
Bad_SecurityPolicyRejected	The security policy does not meet the requirements set by the <i>Server</i> .
Bad_SecureChannelIdInvalid	See Table 165 for the description of this result code.
Bad_NonceInvalid	See Table 165 for the description of this result code. A server shall check the minimum length of the client nonce and return this status if the length is below 32 bytes. A check for duplicated nonces can only be done in OpenSecureChannel calls with the request type RENEW_1.

5.5.3 CloseSecureChannel

5.5.3.1 Description

This *Service* is used to terminate a *SecureChannel*.

The request *Messages* shall be signed with the appropriate key associated with the current token for the *SecureChannel*.

5.5.3.2 Parameters

Table 9 defines the parameters for the *Service*.

Table 9 – CloseSecureChannel Service Parameters

Name	Type	Description
Request		
requestHeader	RequestHeader	Common request parameters. The <i>sessionId</i> is always omitted. The type <i>RequestHeader</i> is defined in 7.26.
secureChannelId	ByteString	The identifier for the <i>SecureChannel</i> to close.
Response		
responseHeader	ResponseHeader	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).

5.5.3.3 Service results

Table 10 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 10 – CloseSecureChannel Service Result Codes

Symbolic Id	Description
Bad_SecureChannelIdInvalid	See Table 165 for the description of this result code.

5.6 Session Service Set

5.6.1 Overview

This *Service Set* defines *Services* for an application layer connection establishment in the context of a *Session*.

5.6.2 CreateSession

5.6.2.1 Description

This *Service* is used by an OPC UA *Client* to create a *Session* and the *Server* returns two values which uniquely identify the *Session*. The first value is the *sessionId* which is used to identify the *Session* in the audit logs and in the *Server*'s address space. The second is the *authenticationToken* which is used to associate an incoming request with a *Session*.

Before calling this *Service*, the *Client* shall create a *SecureChannel* with the *OpenSecureChannel* *Service* to ensure the *Integrity* of all *Messages* exchanged during a *Session*. This *SecureChannel* has a unique identifier which the *Server* shall associate with the *authenticationToken*. The *Server* may accept requests with the *authenticationToken* only if they are associated with the same *SecureChannel* that was used to create the *Session*. The *Client* may associate a new *SecureChannel* with the *Session* by calling the *ActivateSession* method.

The *SecureChannel* is always managed by the *Communication Stack* which means it shall provide APIs which the *Server* can use to find out information about the *SecureChannel* used for any given request. The *Communication Stack* shall, at a minimum, provide the *SecurityPolicy* and *SecurityMode* used by the *SecureChannel*. It shall also provide a *SecureChannelId* which uniquely identifies the *SecureChannel* or the *Client Certificate* used to establish the *SecureChannel*. The *Server* uses one of these to identify the *SecureChannel* used to send a request. Clause 7.29 describes how to create the *authenticationToken* for different types of *Communication Stack*.

Depending upon on the *SecurityPolicy* and the *SecurityMode* of the *SecureChannel*, the exchange of *ApplicationInstanceCertificates* and *Nonces* may be optional and the signatures may be empty. See IEC 62541-7 for the definition of *SecurityPolicies* and the handling of these parameters.

The *Server* returns its *EndpointDescriptions* in the response. *Clients* use this information to determine whether the list of *EndpointDescriptions* returned from the *Discovery Endpoint* matches the *Endpoints* that the *Server* has. If there is a difference then the *Client* shall close the *Session* and report an error. The *Server* returns all *EndpointDescriptions* for the *serverUri* specified by the *Client* in the request. The *Client* only verifies *EndpointDescriptions* with a *transportProfileUri* that matches the *profileUri* specified in the original *GetEndpoints* request. A *Client* may skip this check if the *EndpointDescriptions* were provided by a trusted source such as the *Administrator*.

The *Session* created with this *Service* shall not be used until the *Client* calls the *ActivateSession* *Service* and provides its *SoftwareCertificates* and proves possession of its *Application Instance Certificate* and any user identity token that it provided.

The *SoftwareCertificates* parameter in the *Server* response is deprecated to reduce the message size for OPC UA Applications with limited resources. The *SoftwareCertificates* are provided in the *Server*'s *AddressSpace* as defined in IEC 62541-5. A *SoftwareCertificate* identifies the capabilities of the *Server* and also contains the list of OPC UA *Profiles* supported by the *Server*. OPC UA *Profiles* are defined in IEC 62541-7.

Additional *Certificates* issued by other organisations may be included to identify additional *Server* capabilities. Examples of these *Profiles* include support for specific information models and support for access to specific types of devices.

When a *Session* is created, the *Server* adds an entry for the *Client* in its *SessionDiagnosticsArray Variable*. See IEC 62541-5 for a description of this *Variable*.

Sessions are created to be independent of the underlying communications connection. Therefore, if a communications connection fails, the *Session* is not immediately affected. The exact mechanism to recover from an underlying communication connection error depends on the *SecureChannel* mapping as described in IEC 62541-6.

Sessions are terminated by the *Server* automatically if the *Client* fails to issue a *Service* request on the *Session* within the timeout period negotiated by the *Server* in the *CreateSession Service* response. This protects the *Server* against *Client* failures and against situations where a failed underlying connection cannot be re-established. *Clients* shall be prepared to submit requests in a timely manner to prevent the *Session* from closing automatically. *Clients* may explicitly terminate *Sessions* using the *CloseSession Service*.

When a *Session* is terminated, all outstanding requests on the *Session* are aborted and *Bad_SessionClosed StatusCodes* are returned to the *Client*. In addition, the *Server* deletes the entry for the *Client* from its *SessionDiagnosticsArray Variable* and notifies any other *Clients* who were subscribed to this entry.

If a *Client* invokes the *CloseSession Service* then all *Subscriptions* associated with the *Session* are also deleted if the *deleteSubscriptions* flag is set to TRUE. If a *Server* terminates a *Session* for any other reason, *Subscriptions* associated with the *Session*, are not deleted. Each *Subscription* has its own lifetime to protect against data loss in the case of a *Session* termination. In these cases, the *Subscription* can be reassigned to another *Client* before its lifetime expires.

Some *Servers*, such as aggregating *Servers*, also act as *Clients* to other *Servers*. These *Servers* typically support more than one system user, acting as their agent to the *Servers* that they represent. Security for these *Servers* is supported at two levels.

First, each OPC UA *Service* request contains a string parameter that is used to carry an audit record id. A *Client*, or any *Server* operating as a *Client*, such as an aggregating *Server*, can create a local audit log entry for a request that it submits. This parameter allows the *Client* to pass the identifier for this entry with the request. If the *Server* also maintains an audit log, then it can include this id in the audit log entry that it writes. When the log is examined and the entry is found, the examiner will be able to relate it directly to the audit log entry created by the *Client*. This capability allows for traceability across audit logs within a system. See IEC TR 62541-2 for additional information on auditing. A *Server* that maintains an audit log shall provide the information in the audit log entries via event *Messages* defined in this standard. The *Server* may choose to only provide the *Audit* information via event *Messages*. The *Audit EventType* is defined in IEC 62541-3.

Second, these aggregating *Servers* may open independent *Sessions* to the underlying *Servers* for each *Client* that accesses data from them. Figure 14 illustrates this concept.

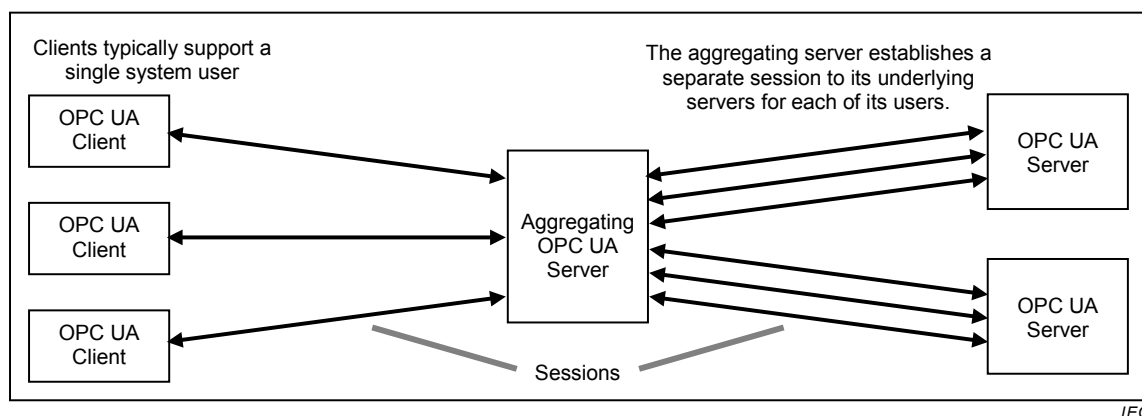


Figure 14 – Multiplexing Users on a Session

5.6.2.2 Parameters

Table 11 defines the parameters for the *Service*.

Table 11 – CreateSession Service Parameters

Name	Type	Description
Request		
requestHeader	RequestHeader	Common request parameters. The <i>authenticationToken</i> is always omitted. The type <i>RequestHeader</i> is defined in 7.26.
clientDescription	Application Description	Information that describes the <i>Client</i> application. The type <i>ApplicationDescription</i> is defined in 7.1.
serverUri	String	This value is only specified if the <i>EndpointDescription</i> has a <i>gatewayServerUri</i> . This value is the <i>applicationUri</i> from the <i>EndpointDescription</i> which is the <i>applicationUri</i> for the underlying <i>Server</i> . The type <i>EndpointDescription</i> is defined in 7.9.
endpointUrl	String	The network address that the <i>Client</i> used to access the <i>Session Endpoint</i> . The <i>HostName</i> portion of the URL should be one of the <i>HostNames</i> for the application that are specified in the <i>Server's ApplicationInstanceCertificate</i> (see 7.2). The <i>Server</i> shall raise an <i>AuditUriMismatchEventType</i> event if the URL does not match the <i>Server's HostNames</i> . <i>AuditUriMismatchEventType</i> event type is defined in IEC 62541-5. The <i>Server</i> uses this information for diagnostics and to determine the set of <i>EndpointDescriptions</i> to return in the response.
sessionName	String	Human readable string that identifies the <i>Session</i> . The <i>Server</i> makes this name and the <i>sessionId</i> visible in its <i>AddressSpace</i> for diagnostic purposes. The <i>Client</i> should provide a name that is unique for the instance of the <i>Client</i> . If this parameter is not specified the <i>Server</i> shall assign a value.
clientNonce	ByteString	A random number that should never be used in any other request. This number shall have a minimum length of 32 bytes. Profiles may increase the required length. The <i>Server</i> shall use this value to prove possession of its <i>Application Instance Certificate</i> in the response.
clientCertificate	ApplicationInstance Certificate	The <i>Application Instance Certificate</i> issued to the <i>Client</i> . The <i>ApplicationInstanceCertificate</i> type is defined in 7.2. If the <i>securityPolicyUri</i> is None, the <i>Server</i> shall ignore the <i>ApplicationInstanceCertificate</i> .
Requested SessionTimeout	Duration	Requested maximum number of milliseconds that a <i>Session</i> should remain open without activity. If the <i>Client</i> fails to issue a <i>Service</i> request within this interval, then the <i>Server</i> shall automatically terminate the <i>Client Session</i> .
maxResponse MessageSize	UInt32	The maximum size, in bytes, for the body of any response message. The <i>Server</i> should return a <i>Bad_ResponseTooLarge</i> service fault if a response message exceeds this limit. The value zero indicates that this parameter is not used. More information on the use of this parameter is provided in 5.3.
Response		
responseHeader	ResponseHeader	Common response parameters (see 7.27 for <i>ResponseHeader</i> type).
sessionId	NodeId	A unique <i>NodeId</i> assigned by the <i>Server</i> to the <i>Session</i> . This identifier is used to access the diagnostics information for the <i>Session</i> in the <i>Server</i> address space. It is also used in the audit logs and any events that report information related to the <i>Session</i> . The <i>Session</i> diagnostic information is described in IEC 62541-5. Audit logs

Name	Type	Description
		and their related events are described in 6.2
authentication Token	Session AuthenticationToken	A unique identifier assigned by the <i>Server</i> to the <i>Session</i> . This identifier shall be passed in the <i>RequestHeader</i> of each request and is used with the <i>SecureChannelId</i> to determine whether a <i>Client</i> has access to the <i>Session</i> . This identifier shall not be reused in a way that the <i>Client</i> or the <i>Server</i> has a chance of confusing them with a previous or existing <i>Session</i> . The <i>SessionAuthenticationToken</i> type is described in 7.29.
revisedSession Timeout	Duration	Actual maximum number of milliseconds that a <i>Session</i> shall remain open without activity. The <i>Server</i> should attempt to honour the <i>Client</i> request for this parameter, but may negotiate this value up or down to meet its own constraints.
serverNonce	ByteString	A random number that should never be used in any other request. This number shall have a minimum length of 32 bytes. The <i>Client</i> shall use this value to prove possession of its <i>Application Instance Certificate</i> in the <i>ActivateSession</i> request. This value may also be used to prove possession of the <i>userIdentityToken</i> it specified in the <i>ActivateSession</i> request.
serverCertificate	ApplicationInstance Certificate	The <i>Application Instance Certificate</i> issued to the <i>Server</i> . A <i>Server</i> shall prove possession by using the private key to sign the <i>Nonce</i> provided by the <i>Client</i> in the request. The <i>Client</i> shall verify that this <i>Certificate</i> is the same as the one it used to create the <i>SecureChannel</i> . The <i>ApplicationInstanceCertificate</i> type is defined in 7.2. If the <i>securityPolicyUri</i> is NONE and none of the <i>UserTokenPolicies</i> requires encryption, the <i>Client</i> shall ignore the <i>ApplicationInstanceCertificate</i> .
serverEndpoints []	Endpoint Description	List of <i>Endpoints</i> that the server supports. The <i>Server</i> shall return a set of <i>EndpointDescriptions</i> available for the <i>serverUri</i> specified in the request. The <i>EndpointDescription</i> type is defined in 7.9. The <i>Client</i> shall verify this list with the list from a <i>Discovery Endpoint</i> if it used a <i>Discovery Endpoint</i> to fetch the <i>EndpointDescriptions</i> . It is recommended that <i>Servers</i> only include the <i>endpointUrl</i> , <i>securityMode</i> , <i>securityPolicyUri</i> , <i>userIdentityTokens</i> , <i>transportProfileUri</i> and <i>securityLevel</i> with all other parameters set to null. Only the recommended parameters shall be verified by the client.
serverSoftware Certificates []	SignedSoftware Certificate	This parameter is deprecated and the array shall be empty. The <i>SoftwareCertificates</i> are provided in the <i>Server AddressSpace</i> as defined in IEC 62541-5.
serverSignature	SignatureData	This is a signature generated with the private key associated with the <i>serverCertificate</i> . This parameter is calculated by appending the <i>clientNonce</i> to the <i>clientCertificate</i> and signing the resulting sequence of bytes. The <i>SignatureAlgorithm</i> shall be the <i>AsymmetricSignatureAlgorithm</i> specified in the <i>SecurityPolicy</i> for the <i>Endpoint</i> . The <i>SignatureData</i> type is defined in 7.30.
maxRequest MessageSize	UInt32	The maximum size, in bytes, for the body of any request message. The <i>Client Communication Stack</i> should return a <i>Bad_RequestTooLarge</i> error to the application if a request message exceeds this limit. The value zero indicates that this parameter is not used. 5.3 provides more information on the use of this parameter.

5.6.2.3 Service results

Table 12 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 12 – CreateSession Service Result Codes

Symbolic Id	Description
Bad_SecureChannelIdInvalid	See Table 165 for the description of this result code.
Bad_NonceInvalid	See Table 165 for the description of this result code. A server shall check the minimum length of the client nonce and return this status if the length is below 32 bytes. A check for duplicated nonces is optional and requires access to the nonce used to create the secure channel.
Bad_SecurityChecksFailed	See Table 165 for the description of this result code.
Bad_CertificateTimeInvalid	See Table 165 for the description of this result code.
Bad_CertificateIssuerTimeInvalid	See Table 165 for the description of this result code.
Bad_CertificateHostNameInvalid	See Table 165 for the description of this result code.
Bad_CertificateUriInvalid	See Table 165 for the description of this result code.
Bad_CertificateUseNotAllowed	See Table 165 for the description of this result code.
Bad_CertificateIssuerUseNotAllowed	See Table 165 for the description of this result code.
Bad_CertificateUntrusted	See Table 165 for the description of this result code.
Bad_CertificateRevocationUnknown	See Table 165 for the description of this result code.
Bad_CertificateIssuerRevocationUnknown	See Table 165 for the description of this result code.
Bad_CertificateRevoked	See Table 165 for the description of this result code.
Bad_CertificateIssuerRevoked	See Table 165 for the description of this result code.
Bad_TooManySessions	The server has reached its maximum number of sessions.
Bad_ServerUriInvalid	See Table 165 for the description of this result code.

5.6.3 ActivateSession

5.6.3.1 Description

This *Service* is used by the *Client* to submit its *SoftwareCertificates* to the *Server* for validation and to specify the identity of the user associated with the *Session*. This *Service* request shall be issued by the *Client* before it issues any other *Service* request after *CreateSession*. Failure to do so shall cause the *Server* to close the *Session*.

Whenever the *Client* calls this *Service* the *Client* shall prove that it is the same application that called the *CreateSession Service*. The *Client* does this by creating a signature with the private key associated with the *clientCertificate* specified in the *CreateSession* request. This signature is created by appending the last *serverNonce* provided by the *Server* to the *serverCertificate* and calculating the signature of the resulting sequence of bytes.

Once used, a *serverNonce* cannot be used again. For that reason, the *Server* returns a new *serverNonce* each time the *ActivateSession Service* is called.

When the *ActivateSession Service* is called for the first time then the *Server* shall reject the request if the *SecureChannel* is not same as the one associated with the *CreateSession* request. Subsequent calls to *ActivateSession* may be associated with different *SecureChannels*. If this is the case then the *Server* shall verify that the *Certificate* the *Client* used to create the new *SecureChannel* is the same as the *Certificate* used to create the original *SecureChannel*. In addition, the *Server* shall verify that the *Client* supplied a *UserIdentityToken* that is identical to the token currently associated with the *Session*. Once the *Server* accepts the new *SecureChannel* it shall reject requests sent via the old *SecureChannel*.

The *ActivateSession Service* is used to associate a user identity with a *Session*. When a *Client* provides a user identity then it shall provide proof that it is authorized to use that user identity. The exact mechanism used to provide this proof depends on the type of the *UserIdentityToken*. If the token is a *UserNameIdentityToken* then the proof is the *password* that is included in the token. If the token is an *X509IdentityToken* then the proof is a signature generated with private key associated with the *Certificate*. The data to sign is created by appending the last *serverNonce* to the *serverCertificate* specified in the *CreateSession* response. If a token includes a secret then it should be encrypted using the public key from the *serverCertificate*.

Clients can change the identity of a user associated with a *Session* by calling the *ActivateSession Service*. The *Server* validates the signatures provided with the request and then validates the new user identity. If no errors occur the *Server* replaces the user identity for the *Session*. Changing the user identity for a *Session* may cause discontinuities in active *Subscriptions* because the *Server* may have to tear down connections to an underlying system and re-establish them using the new credentials.

When a *Client* supplies a list of locale ids in the request, each locale id is required to contain the language component. It may optionally contain the <country/region> component. When the *Server* returns a *LocalizedText* in the context of the *Session*, it also may return both the language and the country/region or just the language as its default locale id.

When a *Server* returns a string to the *Client*, it first determines if there are available translations for it. If there are, then the *Server* returns the string whose locale id exactly matches the locale id with the highest priority in the *Client*-supplied list.

If there are no exact matches, then the *Server* ignores the <country/region> component of the locale id, and returns the string whose <language> component matches the <language> component of the locale id with the highest priority in the *Client* supplied list.

If there still are no matches, then the *Server* returns the string that it has along with the locale id.

A *Gateway Server* is expected to impersonate the user provided by the *Client* when it connects to the underlying *Server*. This means it shall re-calculate the signatures on the *UserIdentityToken* using the nonce provided by the underlying *Server*. The *Gateway Server* will have to use its own user credentials if the *UserIdentityToken* provided by the *Client* does not support impersonation.

5.6.3.2 Parameters

Table 13 defines the parameters for the *Service*.

Table 13 – ActivateSession Service Parameters

Name	Type	Description
Request		
requestHeader	RequestHeader	Common request parameters. The type <i>RequestHeader</i> is defined in 7.26.
clientSignature	SignatureData	This is a signature generated with the private key associated with the <i>clientCertificate</i> . The <i>SignatureAlgorithm</i> shall be the <i>AsymmetricSignatureAlgorithm</i> specified in the <i>SecurityPolicy</i> for the <i>Endpoint</i> . The <i>SignatureData</i> type is defined in 7.30.
clientSoftwareCertificates []	SignedSoftwareCertificate	These are the <i>SoftwareCertificates</i> which have been issued to the <i>Client</i> application. The <i>productUri</i> contained in the <i>SoftwareCertificates</i> shall match the <i>productUri</i> in the <i>ApplicationDescription</i> passed by the <i>Client</i> in the <i>CreateSession</i> requests. <i>Certificates</i> without matching <i>productUri</i> should be ignored. <i>Servers</i> may reject connections from <i>Clients</i> if they are not satisfied with the <i>SoftwareCertificates</i> provided by the <i>Client</i> . This parameter only needs to be specified in the first <i>ActivateSession</i> request after <i>CreateSession</i> . It shall always be omitted if the <i>maxRequestMessageSize</i> returned from the <i>Server</i> in the <i>CreateSession</i> response is less than one megabyte. The <i>SignedSoftwareCertificate</i> type is defined in 7.31.
localeIds []	LocaleId	List of locale ids in priority order for localized strings. The first <i>LocaleId</i> in the list has the highest priority. If the <i>Server</i> returns a localized string to the <i>Client</i> , the <i>Server</i> shall return the translation with the highest priority that it can. If it does not have a translation for any of the locales identified in this list, then it shall return the string value that it has and include the locale id with the string. See IEC 62541-3 for more detail on locale ids. If the <i>Client</i> fails to specify at least one locale id, the <i>Server</i> shall use any that it has. This parameter only needs to be specified during the first call to <i>ActivateSession</i> during a single application <i>Session</i> . If it is not specified the <i>Server</i> shall keep using the current <i>localeIds</i> for the <i>Session</i> .
userIdentityToken	Extensible Parameter UserIdentityToken	The credentials of the user associated with the <i>Client</i> application. The <i>Server</i> uses these credentials to determine whether the <i>Client</i> should be allowed to activate a <i>Session</i> and what resources the <i>Client</i> has access to during this <i>Session</i> . The <i>UserIdentityToken</i> is an extensible parameter type defined in 7.35. The <i>EndpointDescription</i> specifies what <i>UserIdentityTokens</i> the <i>Server</i> shall accept. Null or empty user token shall always be interpreted as anonymous.
userTokenSignature	SignatureData	If the <i>Client</i> specified a user identity token that supports digital signatures, then it shall create a signature and pass it as this parameter. Otherwise the parameter is omitted. The <i>SignatureAlgorithm</i> depends on the identity token type. The <i>SignatureData</i> type is defined in 7.30.
Response		
responseHeader	ResponseHeader	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).
serverNonce	ByteString	A random number that should never be used in any other request. This number shall have a minimum length of 32 bytes. The <i>Client</i> shall use this value to prove possession of its <i>Application Instance Certificate</i> in the next call to <i>ActivateSession</i> request.
results []	StatusCode	List of validation results for the <i>SoftwareCertificates</i> (see 7.33 for <i>StatusCode</i> definition).
diagnosticInfos []	DiagnosticInfo	List of diagnostic information associated with <i>SoftwareCertificate</i> validation errors (see 7.8 for <i>DiagnosticInfo</i> definition). This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the request.

5.6.3.3 Service results

Table 14 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 14 – ActivateSession Service Result Codes

Symbolic Id	Description
Bad_IdentityTokenInvalid	See Table 165 for the description of this result code.
Bad_IdentityTokenRejected	See Table 165 for the description of this result code.
Bad_UserAccessDenied	See Table 165 for the description of this result code.
Bad_ApplicationSignatureInvalid	The signature provided by the client application is missing or invalid.
Bad_UserSignatureInvalid	The user token signature is missing or invalid.
Bad_NoValidCertificates	The <i>Client</i> did not provide at least one <i>Software Certificate</i> that is valid and meets the profile requirements for the <i>Server</i> .
Bad_IdentityChangeNotSupported	The <i>Server</i> does not support changing the user identity assigned to the session.

5.6.4 CloseSession

5.6.4.1 Description

This *Service* is used to terminate a *Session*. The *Server* takes the following actions when it receives a *CloseSession* request:

- It stops accepting requests for the *Session*. All subsequent requests received for the *Session* are discarded.
- It returns negative responses with the *StatusCode* *Bad_SessionClosed* to all requests that are currently outstanding to provide for the timely return of the *CloseSession* response. *Clients* are urged to wait for all outstanding requests to complete before submitting the *CloseSession* request.
- It removes the entry for the *Client* in its *SessionDiagnosticsArray Variable*.

5.6.4.2 Parameters

Table 15 defines the parameters for the *Service*.

Table 15 – CloseSession Service Parameters

Name	Type	Description
Request		
requestHeader	RequestHeader	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).
deleteSubscriptions	Boolean	If the value is TRUE, the Server deletes all Subscriptions associated with the Session. If the value is FALSE, the Server keeps the Subscriptions associated with the Session until they timeout based on their own lifetime.
Response		
responseHeader	ResponseHeader	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).

5.6.4.3 Service results

Table 16 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 16 – CloseSession Service Result Codes

Symbolic Id	Description
Bad_SessionIdInvalid	See Table 165 for the description of this result code.

5.6.5 Cancel

5.6.5.1 Description

This *Service* is used to cancel outstanding *Service* requests. Successfully cancelled service requests shall respond with *Bad_RequestCancelledByClient*.

5.6.5.2 Parameters

Table 17 defines the parameters for the *Service*.

Table 17 – Cancel Service Parameters

Name	Type	Description
Request		
requestHeader	RequestHeader	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).
requestHandle	IntegerId	The <i>requestHandle</i> assigned to one or more requests that should be cancelled. All outstanding requests with the matching <i>requestHandle</i> shall be cancelled.
Response		
responseHeader	ResponseHeader	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).
cancelCount	UInt32	Number of cancelled requests.

5.6.5.3 Service results

Common *StatusCodes* are defined in Table 165.

5.7 NodeManagement Service Set

5.7.1 Overview

This *Service Set* defines *Services* to add and delete *AddressSpace Nodes* and *References* between them. All added *Nodes* continue to exist in the *AddressSpace* even if the *Client* that created them disconnects from the *Server*.

5.7.2 AddNodes

5.7.2.1 Description

This *Service* is used to add one or more *Nodes* into the *AddressSpace* hierarchy. Using this *Service*, each *Node* is added as the *TargetNode* of a *HierarchicalReference* to ensure that the *AddressSpace* is fully connected and that the *Node* is added as a child within the *AddressSpace* hierarchy (see IEC 62541-3).

5.7.2.2 Parameters

Table 18 defines the parameters for the *Service*.

Table 18 – AddNodes Service Parameters

Name	Type	Description
Request		
requestHeader	RequestHeader	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).
nodesToAdd []	AddNodesItem	List of <i>Nodes</i> to add. All <i>Nodes</i> are added as a <i>Reference</i> to an existing <i>Node</i> using a hierarchical <i>ReferenceType</i> . This structure is defined in-line with the following indented items.
parentNodeId	Expanded NodeId	<i>ExpandedNodeId</i> of the parent <i>Node</i> for the <i>Reference</i> . The <i>ExpandedNodeId</i> type is defined in 7.10.
referenceTypeId	NodeId	<i>NodeId</i> of the hierarchical <i>ReferenceType</i> to use for the <i>Reference</i> from the parent <i>Node</i> to the new <i>Node</i> .
requestedNewNodeId	Expanded NodeId	<i>Client</i> requested expanded <i>NodeId</i> of the <i>Node</i> to add. The <i>serverIndex</i> in the expanded <i>NodeId</i> shall be 0. If the <i>Server</i> cannot use this <i>NodeId</i> , it rejects this <i>Node</i> and returns the appropriate error code. If the <i>Client</i> does not want to request a <i>NodeId</i> , then it sets the value of this parameter to the null expanded <i>NodeId</i> . If the <i>Node</i> to add is a <i>ReferenceType Node</i> , its <i>NodeId</i> should be a numeric id. See IEC 62541-3 for a description of <i>ReferenceType NodeIds</i> .
browseName	QualifiedName	The browse name of the <i>Node</i> to add.
nodeClass	NodeClass	<i>NodeClass</i> of the <i>Node</i> to add.
nodeAttributes	Extensible Parameter NodeAttributes	The <i>Attributes</i> that are specific to the <i>NodeClass</i> . The <i>NodeAttributes</i> parameter type is an extensible parameter type specified in 7.18. A <i>Client</i> is allowed to omit values for some or all <i>Attributes</i> . If an <i>Attribute</i> value is omitted, the <i>Server</i> shall use the default values from the <i>TypeDefinitionNode</i> . If a <i>TypeDefinitionNode</i> was not provided the <i>Server</i> shall choose a suitable default value. The <i>Server</i> may still add an optional <i>Attribute</i> to the <i>Node</i> with an appropriate default value even if the <i>Client</i> does not specify a value.
typeDefinition	Expanded NodeId	<i>NodeId</i> of the <i>TypeDefinitionNode</i> for the <i>Node</i> to add. This parameter shall be null for all <i>NodeClasses</i> other than <i>Object</i> and <i>Variable</i> in which case it shall be provided.
Response		
responseHeader	Response Header	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).
results []	AddNodesResult	List of results for the <i>Nodes</i> to add. The size and order of the list matches the size and order of the <i>nodesToAdd</i> request parameter. This structure is defined in-line with the following indented items.
statusCode	StatusCode	<i>StatusCode</i> for the <i>Node</i> to add (see 7.33 for <i>StatusCode</i> definition).
addedNodeId	NodeId	<i>Server</i> assigned <i>NodeId</i> of the added <i>Node</i> . Null <i>NodeId</i> if the operation failed.
diagnosticInfos []	DiagnosticInfo	List of diagnostic information for the <i>Nodes</i> to add (see 7.8 for <i>DiagnosticInfo</i> definition). The size and order of the list matches the size and order of the <i>nodesToAdd</i> request parameter. This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the request.

5.7.2.3 Service results

Table 19 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 19 – AddNodes Service Result Codes

Symbolic Id	Description
Bad_NothingToDo	See Table 165 for the description of this result code.
Bad_TooManyOperations	See Table 165 for the description of this result code.

5.7.2.4 StatusCodes

Table 20 defines values for the operation level *statusCode* parameter that are specific to this *Service*. Common *StatusCodes* are defined in Table 166.

Table 20 – AddNodes Operation Level Result Codes

Symbolic Id	Description
Bad_ParentNodeIdInvalid	The parent node id does not refer to a valid node.
Bad_ReferenceTypeIdInvalid	See Table 166 for the description of this result code.
Bad_ReferenceNotAllowed	The reference could not be created because it violates constraints imposed by the data model.
Bad_NodeIdRejected	The requested node id was rejected either because it was invalid or because the server does not allow node ids to be specified by the client.
Bad_NodeIdExists	The requested node id is already used by another node.
Bad_NodeClassInvalid	See Table 166 for the description of this result code.
Bad_BrowseNameInvalid	See Table 166 for the description of this result code.
Bad_BrowseNameDuplicated	The browse name is not unique among nodes that share the same relationship with the parent.
Bad_NodeAttributesInvalid	The node <i>Attributes</i> are not valid for the node class.
Bad_TypeDefinitionInvalid	See Table 166 for the description of this result code.
Bad_UserAccessDenied	See Table 165 for the description of this result code.

5.7.3 AddReferences

5.7.3.1 Description

This *Service* is used to add one or more *References* to one or more *Nodes*. The *NodeClass* is an input parameter that is used to validate that the *Reference* to be added matches the *NodeClass* of the *TargetNode*. This parameter is not validated if the *Reference* refers to a *TargetNode* in a remote *Server*.

In certain cases, adding new *References* to the *AddressSpace* shall require that the *Server* add new *Server* ids to the *Server's ServerArray Variable*. For this reason, remote *Servers* are identified by their URI and not by their *ServerArray* index. This allows the *Server* to add the remote *Server* URIs to its *ServerArray*.

5.7.3.2 Parameters

Table 21 defines the parameters for the *Service*.

Table 21 – AddReferences Service Parameters

Name	Type	Description
Request		
requestHeader	Request Header	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).
referencesToAdd []	AddReferences Item	List of <i>Reference</i> instances to add to the <i>SourceNode</i> . The <i>targetNodeClass</i> of each <i>Reference</i> in the list shall match the <i>NodeClass</i> of the <i>TargetNode</i> . This structure is defined in-line with the following indented items.
sourceNodeId	NodeId	<i>NodeId</i> of the <i>Node</i> to which the <i>Reference</i> is to be added. The source <i>Node</i> shall always exist in the <i>Server</i> to add the <i>Reference</i> . The <i>isForward</i> parameter can be set to FALSE if the target <i>Node</i> is on the local <i>Server</i> and the source <i>Node</i> on the remote <i>Server</i> .
referenceTypeId	NodeId	<i>NodeId</i> of the <i>ReferenceType</i> that defines the <i>Reference</i> .
isForward	Boolean	If the value is TRUE, the <i>Server</i> creates a forward <i>Reference</i> . If the value is FALSE, the <i>Server</i> creates an inverse <i>Reference</i> .
targetServerUri	String	URI of the remote <i>Server</i> . If this parameter is not null, it overrides the <i>serverIndex</i> in the <i>targetNodeId</i> .
targetNodeId	Expanded NodeId	Expanded <i>NodeId</i> of the <i>TargetNode</i> . The <i>ExpandedNodeId</i> type is defined in 7.10.
targetNodeClass	NodeClass	<i>NodeClass</i> of the <i>TargetNode</i> . The <i>Client</i> shall specify this since the <i>TargetNode</i> might not be accessible directly by the <i>Server</i> .
Response		
responseHeader	Response Header	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).
results []	Status Code	List of <i>StatusCodes</i> for the <i>References</i> to add (see 7.33 for <i>StatusCode</i> definition). The size and order of the list matches the size and order of the <i>referencesToAdd</i> request parameter.
diagnosticInfos []	Diagnostic Info	List of diagnostic information for the <i>References</i> to add (see 7.8 for

Name	Type	Description
Request		
		<i>DiagnosticInfo</i> definition). The size and order of the list matches the size and order of the <i>referencesToAdd</i> request parameter. This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the request.

5.7.3.3 Service results

Table 22 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 22 – AddReferences Service Result Codes

Symbolic Id	Description
Bad_NothingToDo	See Table 165 for the description of this result code.
Bad_TooManyOperations	See Table 165 for the description of this result code.

5.7.3.4 StatusCodes

Table 23 defines values for the *results* parameter that are specific to this *Service*. Common *StatusCodes* are defined in Table 166.

Table 23 – AddReferences Operation Level Result Codes

Symbolic Id	Description
Bad_SourceNodeIdInvalid	See Table 166 for the description of this result code.
Bad_ReferenceTypeIdInvalid	See Table 166 for the description of this result code.
Bad_ServerUriInvalid	See Table 165 for the description of this result code.
Bad_TargetNodeIdInvalid	See Table 166 for the description of this result code.
Bad_NodeClassInvalid	See Table 166 for the description of this result code.
Bad_ReferenceNotAllowed	The reference could not be created because it violates constraints imposed by the data model on this server.
Bad_ReferenceLocalOnly	The reference type is not valid for a reference to a remote <i>Server</i> .
Bad_UserAccessDenied	See Table 165 for the description of this result code.
Bad_DuplicateReferenceNotAllowed	The reference type between the nodes is already defined.
Bad_InvalidSelfReference	The server does not allow this type of self reference on this node.

5.7.4 DeleteNodes

5.7.4.1 Description

This *Service* is used to delete one or more *Nodes* from the *AddressSpace*.

When any of the *Nodes* deleted by an invocation of this *Service* is the *TargetNode* of a *Reference*, then those *References* are left unresolved based on the *deleteTargetReferences* parameter.

When any of the *Nodes* deleted by an invocation of this *Service* is being monitored, then a *Notification* containing the status code *Bad_NodeIdUnknown* is sent to the monitoring *Client* indicating that the *Node* has been deleted.

5.7.4.2 Parameters

Table 24 defines the parameters for the *Service*.

Table 24 – DeleteNodes Service Parameters

Name	Type	Description
Request		
requestHeader	Request Header	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).
nodesToDelete []	DeleteNodes Item	List of <i>Nodes</i> to delete. This structure is defined in-line with the following indented items.
nodeId	NodeId	<i>NodeId</i> of the <i>Node</i> to delete.
deleteTargetReferences	Boolean	A <i>Boolean</i> parameter with the following values: TRUE delete <i>References</i> in <i>TargetNodes</i> that <i>Reference</i> the <i>Node</i> to delete. FALSE delete only the <i>References</i> for which the <i>Node</i> to delete is the source. The <i>Server</i> cannot guarantee that it is able to delete all <i>References</i> from <i>TargetNodes</i> if this parameter is TRUE.
Response		
responseHeader	Response Header	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).
results []	StatusCode	List of <i>StatusCodes</i> for the <i>Nodes</i> to delete (see 7.33 for <i>StatusCode</i> definition). The size and order of the list matches the size and order of the list of the <i>nodesToDelete</i> request parameter.
diagnosticInfos []	Diagnostic Info	List of diagnostic information for the <i>Nodes</i> to delete (see 7.8 for <i>DiagnosticInfo</i> definition). The size and order of the list matches the size and order of the <i>nodesToDelete</i> request parameter. This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the request.

5.7.4.3 Service results

Table 25 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 25 – DeleteNodes Service Result Codes

Symbolic Id	Description
Bad_NothingToDo	See Table 165 for the description of this result code.
Bad_TooManyOperations	See Table 165 for the description of this result code.

5.7.4.4 StatusCodes

Table 26 defines values for the *results* parameter that are specific to this *Service*. Common *StatusCodes* are defined in Table 166.

Table 26 – DeleteNodes Operation Level Result Codes

Symbolic Id	Description
Bad_NodeIdInvalid	See Table 166 for the description of this result code.
Bad_NodeIdUnknown	See Table 166 for the description of this result code.
Bad_UserAccessDenied	See Table 165 for the description of this result code.
Bad_NoDeleteRights	See Table 166 for the description of this result code.
Uncertain_ReferenceNotDeleted	The server was not able to delete all target references.

5.7.5 DeleteReferences

5.7.5.1 Description

This *Service* is used to delete one or more *References* of a *Node*.

When any of the *References* deleted by an invocation of this *Service* are contained in a *View*, then the *ViewVersion Property* is updated if this *Property* is supported.

Table 27 defines the parameters for the *Service*.

Table 27 – DeleteReferences Service Parameters

Name	Type	Description
Request		
requestHeader	RequestHeader	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).
referencesToDelete []	DeleteReferencesItem	List of <i>References</i> to delete. This structure is defined in-line with the following indented items.
sourceNodeId	NodeId	<i>NodeId</i> of the <i>Node</i> that contains the <i>Reference</i> to delete.
referenceTypeId	NodeId	<i>NodeId</i> of the <i>ReferenceType</i> that defines the <i>Reference</i> to delete.
isForward	Boolean	If the value is TRUE, the Server deletes a forward <i>Reference</i> . If the value is FALSE, the Server deletes an inverse <i>Reference</i> .
targetNodeId	ExpandedNodeId	<i>NodeId</i> of the <i>TargetNode</i> of the <i>Reference</i> . If the <i>Server</i> index indicates that the <i>TargetNode</i> is a remote <i>Node</i> , then the <i>nodeId</i> shall contain the absolute namespace URI. If the <i>TargetNode</i> is a local <i>Node</i> the <i>nodeId</i> shall contain the namespace index.
deleteBidirectional	Boolean	A <i>Boolean</i> parameter with the following values: TRUE delete the specified <i>Reference</i> and the opposite <i>Reference</i> from the <i>TargetNode</i> . If the <i>TargetNode</i> is located in a remote <i>Server</i> , the <i>Server</i> is permitted to delete the specified <i>Reference</i> only. FALSE delete only the specified <i>Reference</i> .
Response		
responseHeader	ResponseHeader	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).
results []	StatusCode	List of <i>StatusCodes</i> for the <i>References</i> to delete (see 7.33 for <i>StatusCode</i> definition). The size and order of the list matches the size and order of the <i>referencesToDelete</i> request parameter.
diagnosticInfos []	DiagnosticInfo	List of diagnostic information for the <i>References</i> to delete (see 7.8 for <i>DiagnosticInfo</i> definition). The size and order of the list matches the size and order of the <i>referencesToDelete</i> request parameter. This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the request.

5.7.5.2 Service results

Table 28 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 28 – DeleteReferences Service Result Codes

Symbolic Id	Description
Bad_NothingToDo	See Table 165 for the description of this result code.
Bad_TooManyOperations	See Table 165 for the description of this result code.

5.7.5.3 StatusCodes

Table 29 defines values for the *results* parameter that are specific to this *Service*. Common *StatusCodes* are defined in Table 166.

Table 29 – DeleteReferences Operation Level Result Codes

Symbolic Id	Description
Bad_SourceNodeIdInvalid	See Table 166 for the description of this result code.
Bad_ReferenceTypeIdInvalid	See Table 166 for the description of this result code.
Bad_ServerIndexInvalid	The server index is not valid.
Bad_TargetNodeIdInvalid	See Table 166 for the description of this result code.
Bad_UserAccessDenied	See Table 165 for the description of this result code.
Bad_NoDeleteRights	See Table 166 for the description of this result code.

5.8 View Service Set

5.8.1 Overview

Clients use the browse *Services* of the *View Service Set* to navigate through the *AddressSpace* or through a *View* which is a subset of the *AddressSpace*.

A *View* is a subset of the *AddressSpace* created by the *Server*. Future versions of this standard may also define services to create *Client*-defined *Views*. See IEC 62541-5 for a description of the organisation of views in the *AddressSpace*.

5.8.2 Browse

5.8.2.1 Description

This *Service* is used to discover the *References* of a specified *Node*. The browse can be further limited by the use of a *View*. This Browse *Service* also supports a primitive filtering capability.

5.8.2.2 Parameters

Table 30 defines the parameters for the *Service*.

Table 30 – Browse Service Parameters

Name	Type	Description																		
Request																				
requestHeader	RequestHeader	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).																		
view	ViewDescription	Description of the <i>View</i> to browse (see 7.37 for <i>ViewDescription</i> definition). An empty <i>ViewDescription</i> value indicates the entire <i>AddressSpace</i> . Use of the empty <i>ViewDescription</i> value causes all <i>References</i> of the <i>nodesToBrowse</i> to be returned. Use of any other <i>View</i> causes only the <i>References</i> of the <i>nodesToBrowse</i> that are defined for that <i>View</i> to be returned.																		
requestedMaxReferencesPerNode	Counter	Indicates the maximum number of references to return for each starting Node specified in the request. The value 0 indicates that the <i>Client</i> is imposing no limitation (see 7.5 for <i>Counter</i> definition).																		
nodesToBrowse []	BrowseDescription	A list of nodes to Browse. This structure is defined in-line with the following indented items.																		
nodeId	NodeId	<i>NodeId</i> of the <i>Node</i> to be browsed. If a <i>view</i> is provided, it shall include this Node.																		
browseDirection	Enum BrowseDirection	An enumeration that specifies the direction of <i>References</i> to follow. It has the following values: FORWARD_0 select only forward <i>References</i> . INVERSE_1 select only inverse <i>References</i> . BOTH_2 select forward and inverse <i>References</i> . The returned <i>References</i> do indicate the direction the <i>Server</i> followed in the <i>isForward</i> parameter of the <i>ReferenceDescription</i> . Symmetric <i>References</i> are always considered to be in forward direction therefore the <i>isForward</i> flag is always set to TRUE and symmetric <i>References</i> are not returned if <i>browseDirection</i> is set to <i>INVERSE_1</i> .																		
referenceTypeId	NodeId	Specifies the <i>NodeId</i> of the <i>ReferenceType</i> to follow. Only instances of this <i>ReferenceType</i> or its subtypes are returned. If not specified then all <i>References</i> are returned and <i>includeSubtypes</i> is ignored.																		
includeSubtypes	Boolean	Indicates whether subtypes of the <i>ReferenceType</i> should be included in the browse. If TRUE, then instances of <i>referenceTypeId</i> and all of its subtypes are returned.																		
nodeClassMask	UInt32	Specifies the <i>NodeClasses</i> of the <i>TargetNodes</i> . Only <i>TargetNodes</i> with the selected <i>NodeClasses</i> are returned. The <i>NodeClasses</i> are assigned the following bits: <table><tr><th>Bit</th><th>NodeClass</th></tr><tr><td>0</td><td>Object</td></tr><tr><td>1</td><td>Variable</td></tr><tr><td>2</td><td>Method</td></tr><tr><td>3</td><td>ObjectType</td></tr><tr><td>4</td><td>VariableType</td></tr><tr><td>5</td><td>ReferenceType</td></tr><tr><td>6</td><td>DataType</td></tr><tr><td>7</td><td>View</td></tr></table> If set to zero, then all <i>NodeClasses</i> are returned.	Bit	NodeClass	0	Object	1	Variable	2	Method	3	ObjectType	4	VariableType	5	ReferenceType	6	DataType	7	View
Bit	NodeClass																			
0	Object																			
1	Variable																			
2	Method																			
3	ObjectType																			
4	VariableType																			
5	ReferenceType																			
6	DataType																			
7	View																			
resultMask	UInt32	Specifies the fields in the <i>ReferenceDescription</i> structure that should be returned. The fields are assigned the following bits: <table><tr><th>Bit</th><th>Result</th></tr><tr><td>0</td><td>ReferenceType</td></tr><tr><td>1</td><td>IsForward</td></tr><tr><td>2</td><td>NodeClass</td></tr><tr><td>3</td><td>BrowseName</td></tr><tr><td>4</td><td>DisplayName</td></tr><tr><td>5</td><td>TypeDefinition</td></tr></table> The <i>ReferenceDescription</i> type is defined in 7.24.	Bit	Result	0	ReferenceType	1	IsForward	2	NodeClass	3	BrowseName	4	DisplayName	5	TypeDefinition				
Bit	Result																			
0	ReferenceType																			
1	IsForward																			
2	NodeClass																			
3	BrowseName																			
4	DisplayName																			
5	TypeDefinition																			
Response																				
responseHeader	Response Header	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).																		
results []	BrowseResult	A list of <i>BrowseResults</i> . The size and order of the list matches the size and order of the <i>nodesToBrowse</i> specified in the request. The <i>BrowseResult</i> type is defined in 7.3.																		
diagnosticInfos []	Diagnostic Info	List of diagnostic information for the <i>results</i> (see 7.8 for <i>DiagnosticInfo</i> definition). The size and order of the list matches the size and order of the <i>results</i> response parameter. This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the request.																		

5.8.2.3 Service results

Table 31 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 31 – Browse Service Result Codes

Symbolic Id	Description
Bad_ViewIdUnknown	See Table 165 for the description of this result code.
Bad_ViewTimestampInvalid	See Table 165 for the description of this result code.
Bad_ViewParameterMismatchInvalid	See Table 165 for the description of this result code.
Bad_ViewVersionInvalid	See Table 165 for the description of this result code.
Bad_NothingToDo	See Table 165 for the description of this result code.
Bad_TooManyOperations	See Table 165 for the description of this result code.

5.8.2.4 StatusCodes

Table 32 defines values for the *results* parameter that are specific to this *Service*. Common *StatusCodes* are defined in Table 166.

Table 32 – Browse Operation Level Result Codes

Symbolic Id	Description
Bad_NodeIdInvalid	See Table 166 for the description of this result code.
Bad_NodeIdUnknown	See Table 166 for the description of this result code.
Bad_ReferenceTypeIdInvalid	See Table 166 for the description of this result code.
Bad_BrowseDirectionInvalid	See Table 166 for the description of this result code.
Bad_NodeNotInView	See Table 166 for the description of this result code.
Bad_NoContinuationPoints	See Table 166 for the description of this result code.
Uncertain_NotAllNodesAvailable	Browse results may be incomplete because of the unavailability of a subsystem.

5.8.3 BrowseNext

5.8.3.1 Description

This *Service* is used to request the next set of *Browse* or *BrowseNext* response information that is too large to be sent in a single response. “Too large” in this context means that the *Server* is not able to return a larger response or that the number of results to return exceeds the maximum number of results to return that was specified by the *Client* in the original *Browse* request. The *BrowseNext* shall be submitted on the same *Session* that was used to submit the *Browse* or *BrowseNext* that is being continued.

5.8.3.2 Parameters

Table 33 defines the parameters for the *Service*.

Table 33 – BrowseNext Service Parameters

Name	Type	Description
Request		
requestHeader	Request Header	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).
releaseContinuationPoints	Boolean	A <i>Boolean</i> parameter with the following values: TRUE passed <i>continuationPoints</i> shall be reset to free resources in the <i>Server</i>. The continuation points are released and the results and diagnosticInfos arrays are empty. FALSE passed <i>continuationPoints</i> shall be used to get the next set of browse information. A <i>Client</i> shall always use the continuation point returned by a <i>Browse</i> or <i>BrowseNext</i> response to free the resources for the continuation point in the <i>Server</i>. If the <i>Client</i> does not want to get the next set of browse information, <i>BrowseNext</i> shall be called with this parameter set to TRUE.
continuationPoints []	Continuation Point	A list of <i>Server</i>-defined opaque values that represent continuation points. The value for a continuation point was returned to the <i>Client</i> in a previous <i>Browse</i> or <i>BrowseNext</i> response. These values are used to identify the previously processed <i>Browse</i> or <i>BrowseNext</i> request that is being continued and the point in the result set from which the browse response is to continue. Clients may mix continuation points from different <i>Browse</i> or <i>BrowseNext</i> responses. The <i>ContinuationPoint</i> type is described in 7.6.
Response		
responseHeader	Response Header	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).
results []	BrowseResult	A list of references that met the criteria specified in the original <i>Browse</i> request. The size and order of this list matches the size and order of the <i>continuationPoints</i> request parameter. The <i>BrowseResult</i> type is defined in 7.3.
diagnosticInfos []	Diagnostic Info	List of diagnostic information for the <i>results</i> (see 7.8 for <i>DiagnosticInfo</i> definition). The size and order of the list matches the size and order of the <i>results</i> response parameter. This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the request.

5.8.3.3 Service results

Table 34 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 34 – BrowseNext Service Result Codes

Symbolic Id	Description
Bad_NothingToDo	See Table 165 for the description of this result code.
Bad_TooManyOperations	See Table 165 for the description of this result code.

5.8.3.4 StatusCodes

Table 35 defines values for the *results* parameter that are specific to this *Service*. Common *StatusCodes* are defined in Table 166.

Table 35 – BrowseNext Operation Level Result Codes

Symbolic Id	Description
Bad_NodeIdInvalid	See Table 166 for the description of this result code.
Bad_NodeIdUnknown	See Table 166 for the description of this result code.
Bad_ReferenceTypeIdInvalid	See Table 166 for the description of this result code.
Bad_BrowseDirectionInvalid	See Table 166 for the description of this result code.
Bad_NodeNotInView	See Table 166 for the description of this result code.
Bad_ContinuationPointInvalid	See Table 166 for the description of this result code.

5.8.4 TranslateBrowsePathsToNodeIds

5.8.4.1 Description

This *Service* is used to request that the *Server* translates one or more browse paths to *NodeIds*. Each browse path is constructed of a starting *Node* and a *RelativePath*. The specified starting *Node* identifies the *Node* from which the *RelativePath* is based. The *RelativePath* contains a sequence of *ReferenceTypes* and *BrowseNames*.

One purpose of this *Service* is to allow programming against type definitions. Since *BrowseNames* shall be unique in the context of type definitions, a *Client* may create a browse path that is valid for a type definition and use this path on instances of the type. For example, an *ObjectType* “Boiler” may have a “HeatSensor” *Variable* as *InstanceDeclaration*. A graphical element programmed against the “Boiler” may need to display the *Value* of the “HeatSensor”. If the graphical element would be called on “Boiler1”, an instance of “Boiler”, it would need to call this *Service* specifying the *NodeId* of “Boiler1” as starting *Node* and the *BrowseName* of the “HeatSensor” as browse path. The *Service* would return the *NodeId* of the “HeatSensor” of “Boiler1” and the graphical element could subscribe to its *Value Attribute*.

If a *Node* has multiple targets with the same *BrowseName*, the *Server* shall return a list of *NodeIds*. However, since one of the main purposes of this *Service* is to support programming against type definitions, the *NodeId* of the *Node* based on the type definition of the starting *Node* is returned as the first *NodeId* in the list.

5.8.4.2 Parameters

Table 36 defines the parameters for the *Service*.

Table 36 – TranslateBrowsePathsToNodeIds Service Parameters

Name	Type	Description
Request		
requestHeader	RequestHeader	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).
browsePaths []	BrowsePath	List of browse paths for which <i>NodeIds</i> are being requested. This structure is defined in-line with the following indented items.
startingNode	NodeId	<i>NodeId</i> of the starting <i>Node</i> for the browse path.
relativePath	RelativePath	The path to follow from the <i>startingNode</i> . The last element in the <i>relativePath</i> shall always have a <i>targetName</i> specified. This further restricts the definition of the <i>RelativePath</i> type. The <i>Server</i> shall return <i>Bad_BrowseNameInvalid</i> if the <i>targetName</i> is missing. The <i>RelativePath</i> structure is defined in 7.25.
Response		
responseHeader	ResponseHeader	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).
results []	BrowsePathResult	List of results for the list of browse paths. The size and order of the list matches the size and order of the <i>browsePaths</i> request parameter. This structure is defined in-line with the following indented items.
statusCode	StatusCode	<i>StatusCode</i> for the browse path (see 7.33 for <i>StatusCode</i> definition).
targets []	BrowsePathTarget	List of targets for the <i>relativePath</i> from the <i>startingNode</i> . This structure is defined in-line with the following indented items. A <i>Server</i> may encounter a <i>Reference</i> to a <i>Node</i> in another <i>Server</i> which it cannot follow while it is processing the <i>RelativePath</i> . If this happens the <i>Server</i> returns the <i>NodeId</i> of the external <i>Node</i> and sets the <i>remainingPathIndex</i> parameter to indicate which <i>RelativePath</i> elements still need to be processed. To complete the operation the <i>Client</i> shall connect to the other <i>Server</i> and call this service again using the target as the <i>startingNode</i> and the unprocessed elements as the <i>relativePath</i> .
targetId	ExpandedNodeId	The identifier for a target of the <i>RelativePath</i> .
remainingPathIndex	Index	The index of the first unprocessed element in the <i>RelativePath</i> . This value shall be equal to the maximum value of <i>Index</i> data type if all elements were processed (see 7.12 for <i>Index</i> definition).
diagnosticInfos []	DiagnosticInfo	List of diagnostic information for the list of browse paths (see 7.8 for <i>DiagnosticInfo</i> definition). The size and order of the list matches the size and order of the <i>browsePaths</i> request parameter. This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the request.

5.8.4.3 Service results

Table 37 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in 7.33.

Table 37 – TranslateBrowsePathsToNodeIds Service Result Codes

Symbolic Id	Description
Bad_NothingToDo	See Table 165 for the description of this result code.
Bad_TooManyOperations	See Table 165 for the description of this result code.

5.8.4.4 StatusCodes

Table 38 defines values for the operation level *statusCode* parameters that are specific to this *Service*. Common *StatusCodes* are defined in Table 166.

Table 38 – TranslateBrowsePathsToNodeIds Operation Level Result Codes

Symbolic Id	Description
Bad_NodeIdInvalid	See Table 166 for the description of this result code.
Bad_NodeIdUnknown	See Table 166 for the description of this result code.
Bad_NothingToDo	See Table 165 for the description of this result code. This code indicates that the relativePath contained an empty list.
Bad_BrowseNameInvalid	See Table 166 for the description of this result code. This code indicates that a TargetName was missing in a RelativePath.
Uncertain_ReferenceOutOfServer	The path element has targets which are in another server.
Bad_TooManyMatches	The requested operation has too many matches to return. Users should use queries for large result sets. Servers should allow at least 10 matches before returning this error code.
Bad_QueryTooComplex	The requested operation requires too many resources in the server.
Bad_NoMatch	The requested relativePath cannot be resolved to a target to return.

5.8.5 RegisterNodes

5.8.5.1 Description

A *Server* often has no direct access to the information that it manages. Variables or services might be in underlying systems where additional effort is required to establish a connection to these systems. The *RegisterNodes Service* can be used by *Clients* to register the *Nodes* that they know they will access repeatedly (e.g. Write, Call). It allows *Servers* to set up anything needed so that the access operations will be more efficient. Clients can expect performance improvements when using registered *NodeIds*, but the optimization measures are vendor-specific. For *Variable Nodes Servers* shall concentrate their optimization efforts on the *Value Attribute*.

Registered *NodeIds* are only guaranteed to be valid within the current *Session*. *Clients* shall unregister unneeded *Ids* immediately to free up resources.

RegisterNodes does not validate the *NodeIds* from the request. *Servers* will simply copy unknown *NodeIds* in the response. Structural *NodeId* errors (size violations, invalid id types) will cause the complete *Service* to fail.

For the purpose of *Auditing*, *Servers* shall not use the registered *NodeIds* but only the canonical *NodeIds* which is the value of the *NodeId Attribute*.

5.8.5.2 Parameters

Table 39 defines the parameters for the *Service*.

Table 39 – RegisterNodes Service Parameters

Name	Type	Description
Request		
requestHeader	Request Header	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).
nodesToRegister []	NodeId	List of <i>NodeIds</i> to register that the client has retrieved through browsing, querying or in some other manner.
Response		
responseHeader	Response Header	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).
registeredNodeIds []	NodeId	A list of <i>NodeIds</i> which the <i>Client</i> shall use for subsequent access operations. The size and order of this list matches the size and order of the <i>nodesToRegister</i> request parameter. The <i>Server</i> may return the <i>NodeId</i> from the request or a new (an alias) <i>NodeId</i> . It is recommended that the <i>Server</i> return a numeric <i>NodeIds</i> for aliasing. In case no optimization is supported for a <i>Node</i> , the <i>Server</i> shall return the <i>NodeId</i> from the request.

5.8.5.3 Service results

Table 40 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 40 – RegisterNodes Service Result Codes

Symbolic Id	Description
Bad_NothingToDo	See Table 165 for the description of this result code.
Bad_TooManyOperations	See Table 165 for the description of this result code.
Bad_NodeIdInvalid	See Table 166 for the description of this result code. <i>Servers</i> shall completely reject the <i>RegisterNodes</i> request if any of the <i>NodeIds</i> in the <i>nodesToRegister</i> parameter are structurally invalid.

5.8.6 UnregisterNodes

5.8.6.1 Description

This *Service* is used to unregister *NodeIds* that have been obtained via the *RegisterNodes* service.

UnregisterNodes does not validate the *NodeIds* from the request. *Servers* shall simply unregister *NodeIds* that are known as registered *NodeIds*. Any *NodeIds* that are in the list, but are not registered *NodeIds* are simply ignored.

5.8.6.2 Parameters

Table 46 defines the parameters for the *Service*.

Table 41 – UnregisterNodes Service Parameters

Name	Type	Description
Request		
requestHeader	Request Header	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).
nodesToUnregister []	NodeId	A list of <i>NodeIds</i> that have been obtained via the <i>RegisterNodes</i> service.
Response		
responseHeader	Response Header	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).

5.8.6.3 Service results

Table 47 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 42 – UnregisterNodes Service Result Codes

Symbolic Id	Description
Bad_NothingToDo	See Table 165 for the description of this result code.
Bad_TooManyOperations	See Table 165 for the description of this result code.

5.9 Query Service Set

5.9.1 Overview

This *Service Set* is used to issue a *Query* to a *Server*. OPC UA *Query* is generic in that it provides an underlying storage mechanism independent *Query* capability that can be used to access a wide variety of OPC UA data stores and information management systems. OPC UA *Query* permits a *Client* to access data maintained by a *Server* without any knowledge of the logical schema used for internal storage of the data. Knowledge of the *AddressSpace* is sufficient.

An *OPC UA Application* is expected to use the *OPC UA Query Services* as part of an initialization process or an occasional information synchronization step. For example, *OPC UA Query* would be used for bulk data access of a persistent store to initialise an analysis application with the current state of a system configuration. A *Query* may also be used to initialise or populate data for a report.

A *Query* defines what instances of one or more *TypeDefinitionNodes* in the *AddressSpace* should supply a set of *Attributes*. Results returned by a *Server* are in the form of an array of *QueryDataSets*. The selected *Attribute* values in each *QueryDataSet* come from the definition of the selected *TypeDefinitionNodes* or related *TypeDefinitionNodes* and appear in results in the same order as the *Attributes* that were passed into the *Query*. *Query* also supports *Node* filtering on the basis of *Attribute* values, as well as relationships between *TypeDefinitionNodes*.

See Annex B for example queries.

5.9.2 Querying Views

A *View* is a subset of the *AddressSpace* available in the *Server*. See IEC 62541-5 for a description of the organisation of *Views* in the *AddressSpace*.

For any existing *View*, a *Query* may be used to return a subset of data from the *View*. When an application issues a *Query* against a *View*, only data defined by the *View* is returned. Data not included in the *View* but included in the original *AddressSpace* is not returned.

The *Query Services* supports access to current and historical data. The *Service* supports a *Client* querying a past version of the *AddressSpace*. Clients may specify a *ViewVersion* or a *Timestamp* in a *Query* to access past versions of the *AddressSpace*. *OPC UA Query* is complementary to *Historical Access* in that the former is used to *Query* an *AddressSpace* that existed at a time and the latter is used to *Query* for the value of *Attributes* over time. In this way, a *Query* can be used to retrieve a portion of a past *AddressSpace* so that *Attribute* value history may be accessed using *Historical Access* even if the *Node* is no longer in the current *AddressSpace*.

Servers that support *Query* are expected to be able to access the address space that is associated with the local *Server* and any *Views* that are available on the local *Server*. If a *View* or the address space also references a remote *Server*, query may be able to access the address space of the remote *Server*, but it is not required. If a *Server* does access a remote *Server* the access shall be accomplished using the user identity of the *Client* as described in 5.5.1.

5.9.3 QueryFirst

5.9.3.1 Description

This *Service* is used to issue a *Query* request to the *Server*. The complexity of the *Query* can range from very simple to highly sophisticated. The *Query* can simply request data from instances of a *TypeDefinitionNode* or *TypeDefinitionNode* subject to restrictions specified by the filter. On the other hand, the *Query* can request data from instances of related *Node* types by specifying a *RelativePath* from an originating *TypeDefinitionNode*. In the filter, a separate set of paths can be constructed for limiting the instances that supply data. A filtering path can include multiple *RelatedTo* operators to define a multi-hop path between source instances and target instances. For example, one could filter on students that attend a particular school, but return information about students and their families. In this case, the student school relationship is traversed for filtering, but the student family relationship is traversed to select data. For a complete description of *ContentFilter* see 7.4, also see Clause B.1 for simple examples and Clause B.2 for more complex examples of content filter and queries.

The *Client* provides an array of *NodeTypeDescription* which specify the *NodeId* of a *TypeDefinitionNode* and selects what *Attributes* are to be returned in the response. A client

can also provide a set of *RelativePaths* through the type system starting from an originating *TypeDefinitionNode*. Using these paths, the client selects a set of *Attributes* from *Nodes* that are related to instances of the originating *TypeDefinitionNode*. Additionally, the *Client* can request the *Server* return instances of subtypes of *TypeDefinitionNodes*. If a selected *Attribute* does not exist in a *TypeDefinitionNode* but does exist in a subtype, it is assumed to have a null value in the *TypeDefinitionNode* in question. Therefore, this does not constitute an error condition and a null value is returned for the *Attribute*.

The *Client* can use the filter parameter to limit the result set by restricting *Attributes* and *Properties* to certain values. Another way the *Client* can use a filter to limit the result set is by specifying how instances should be related, using *RelatedTo* operators. In this case, if an instance at the top of the *RelatedTo* path cannot be followed to the bottom of the path via specified hops, no *QueryDataSets* are returned for the starting instance or any of the intermediate instances.

When querying for related instances in the *RelativePath*, the *Client* can optionally ask for *References*. A *Reference* is requested via a *RelativePath* that only includes a *ReferenceType*. If all *References* are desired then the root *ReferenceType* is listed. These *References* are returned as part of the *QueryDataSets*.

Query Services allow a special handling of the *targetName* field in the *RelativePath*. In several Query use cases a type *NodeId* is necessary in the path instead of a *QualifiedName*. Therefore the *Client* is allowed to specify a *NodeId* in the *QualifiedName*. This is done by setting the *namespaceIndex* of the *targetName* to zero and the name part of the *targetName* to the XML representation of the *NodeId*. The XML representation is defined in IEC 62541-6. When matching instances are returned as the target node, the target node shall be an instance of the specified type or subtype of the specified type.

Table 43 defines the request parameters and Table 44 the response parameters for the *QueryFirst Service*.

Table 43 – QueryFirst Request Parameters

Name	Type	Description
Request		
requestHeader	RequestHeader	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).
view	ViewDescription	Specifies a <i>View</i> and temporal context to a <i>Server</i> (see 7.37 for <i>ViewDescription</i> definition).
nodeTypes []	NodeTypeDescription	This is the <i>Node</i> type description. This structure is defined in-line with the following indented items.
typeDefinitionNode	ExpandedNodeId	<i>NodeId</i> of the originating <i>TypeDefinitionNode</i> of the instances for which data is to be returned.
includeSubtypes	Boolean	A flag that indicates whether the <i>Server</i> should include instances of subtypes of the <i>TypeDefinitionNode</i> in the list of instances of the <i>Node</i> type.
dataToReturn []	QueryDataDescription	Specifies an <i>Attribute</i> or <i>Reference</i> from the originating <i>typeDefinitionNode</i> along a given <i>relativePath</i> for which to return data. This structure is defined in-line with the following indented items.
relativePath	RelativePath	Browse path relative to the originating <i>Node</i> that identifies the <i>Node</i> which contains the data that is being requested, where the originating <i>Node</i> is an instance <i>Node</i> of the type defined by the type definition <i>Node</i> . The instance <i>Nodes</i> are further limited by the filter provided as part of this call. For a definition of <i>relativePath</i> see 7.25. This relative path could end on a <i>Reference</i> , in which case the <i>ReferenceDescription</i> of the <i>Reference</i> would be returned as its value. The <i>targetName</i> field of the <i>relativePath</i> may contain a type <i>NodeId</i> . This is done by setting the <i>namespaceIndex</i> of the <i>targetName</i> to zero and the <i>name</i> part of the <i>targetName</i> to the XML representation of the <i>NodeId</i> . The XML representation is defined in IEC 62541-6. When matching instances are returned as the target node, the target node shall be an instance of the specified type or subtype of the specified type.
attributeId	IntegerId	Id of the <i>Attribute</i> . This shall be a valid <i>Attribute</i> Id. The <i>IntegerId</i> is defined in 7.13. The <i>IntegerId</i> for <i>Attributes</i> are defined in IEC 62541-6. If the <i>RelativePath</i> ended in a <i>Reference</i> then this parameter is 0 and ignored by the server.
indexRange	NumericRange	This parameter is used to identify a single element of a structure or an array, or a single range of indexes for arrays. If a range of elements are specified, the values are returned as a composite. The first element is identified by index 0 (zero). The <i>NumericRange</i> type is defined in 7.21. This parameter is null if the specified <i>Attribute</i> is not an array or a structure. However, if the specified <i>Attribute</i> is an array or a structure, and this parameter is null, then all elements are to be included in the range.
filter	ContentFilter	Resulting <i>Nodes</i> shall be limited to the <i>Nodes</i> matching the criteria defined by the filter. <i>ContentFilter</i> is discussed in 7.4. If an empty filter is provided then the entire address space shall be examined and all <i>Nodes</i> that contain a matching requested <i>Attribute</i> or <i>Reference</i> are returned.
maxDataSetsToReturn	Counter	The number of <i>QueryDataSets</i> that the <i>Client</i> wants the <i>Server</i> to return in the response and on each subsequent continuation call response. The <i>Server</i> is allowed to further limit the response, but shall not exceed this limit. A value of 0 indicates that the <i>Client</i> is imposing no limitation.
maxReferencesToReturn	Counter	The number of <i>References</i> that the <i>Client</i> wants the <i>Server</i> to return in the response for each <i>QueryDataSet</i> and on each subsequent continuation call response. The <i>Server</i> is allowed to further limit the response, but shall not exceed this limit. A value of 0 indicates that the <i>Client</i> is imposing no limitation. For example a result where 4 <i>Nodes</i> are being returned, but each has 100 <i>References</i> , if this limit were set to 50 then only the first 50 <i>References</i> for each <i>Node</i> would be returned on the initial call and a continuation point would be set indicating additional data.

Table 44 – QueryFirst Response Parameters

Name	Type	Description
Response		
responseHeader	ResponseHeader	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).
queryDataSets []	QueryDataSet	The array of <i>QueryDataSets</i> . This array is empty if no <i>Nodes</i> or <i>References</i> met the <i>nodeTypes</i> criteria. In this case the continuationPoint parameter shall be empty. The <i>QueryDataSet</i> type is defined in 7.22.
continuationPoint	ContinuationPoint	Server-defined opaque value that identifies the continuation point. The continuation point is used only when the <i>Query</i> results are too large to be returned in a single response. "Too large" in this context means that the <i>Server</i> is not able to return a larger response or that the number of <i>QueryDataSets</i> to return exceeds the maximum number of <i>QueryDataSets</i> to return that was specified by the <i>Client</i> in the request. The continuation point is used in the <i>QueryNext Service</i> . When not used, the value of this parameter is null. If a continuation point is returned, the <i>Client</i> shall call <i>QueryNext</i> to get the next set of <i>QueryDataSets</i> or to free the resources for the continuation point in the <i>Server</i> . A continuation point shall remain active until the <i>Client</i> passes the continuation point to <i>QueryNext</i> or the session is closed. If the maximum continuation points have been reached the oldest continuation point shall be reset. The <i>ContinuationPoint</i> type is described in 7.6.
parsingResults[]	ParsingResult	List of parsing results for <i>QueryFirst</i> . The size and order of the list matches the size and order of the <i>NodeTypes</i> request parameter. This structure is defined in-line with the following indented items. This list is populated with any status codes that are related to the processing of the node types that are part of the query. The array can be empty if no errors were encountered. If any node type encountered an error all node types shall have an associated status code.
statusCode	StatusCode	Parsing result for the requested <i>NodeTypeDescription</i> .
dataStatusCodes []	StatusCode	List of results for <i>dataToReturn</i> . The size and order of the list matches the size and order of the <i>dataToReturn</i> request parameter. The array can be empty if no errors were encountered.
dataDiagnosticInfos []	DiagnosticInfo	List of diagnostic information <i>dataToReturn</i> (see 7.8 for <i>DiagnosticInfo</i> definition). The size and order of the list matches the size and order of the <i>dataToReturn</i> request parameter. This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the query request.
diagnosticInfos []	DiagnosticInfo	List of diagnostic information for the requested <i>NodeTypeDescription</i> . This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the query request.
filterResult	ContentFilter Result	A structure that contains any errors associated with the filter. This structure shall be empty if no errors occurred. The <i>ContentFilterResult</i> type is defined in 7.4.2.

5.9.3.2 Service results

If the *Query* is invalid or cannot be processed, then *QueryDataSets* are not returned and only a *Service* result, filterResult, parsingResults and optional *DiagnosticInfo* is returned. Table 45 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 45 – QueryFirst Service Result Codes

Symbolic Id	Description
Bad_NothingToDo	See Table 165 for the description of this result code.
Bad_TooManyOperations	See Table 165 for the description of this result code.
Bad_ContentFilterInvalid	See Table 166 for the description of this result code.
Bad_ViewIdUnknown	See Table 165 for the description of this result code.
Bad_ViewTimestampInvalid	See Table 165 for the description of this result code.
Bad_ViewParameterMismatchInvalid	See Table 165 for the description of this result code.
Bad_ViewVersionInvalid	See Table 165 for the description of this result code.
Bad_InvalidFilter	The provided filter is invalid, see the filterResult for specific errors
Bad_NodeListError	The NodeTypes provided contain an error, see the parsingResults for specific errors
Bad_InvalidView	The provided ViewDescription is not a valid ViewDescription.
Good_ResultsMayBeIncomplete	The server should have followed a reference to a node in a remote server but did not. The result set may be incomplete.

5.9.3.3 StatusCodes

Table 46 defines values for the parsingResults *statusCode* parameter that are specific to this *Service*. Common *StatusCodes* are defined in Table 166.

Table 46 – QueryFirst Operation Level Result Codes

Symbolic Id	Description
Bad_NodeIdInvalid	See Table 166 for the description of this result code.
Bad_NodeIdUnknown	See Table 166 for the description of this result code.
Bad_NotTypeDefinition	The provided NodeId was not a type definition NodeId.
Bad_AttributeIdInvalid	See Table 166 for the description of this result code.
Bad_IndexRangeInvalid	See Table 166 for the description of this result code.

5.9.4 QueryNext

5.9.4.1 Descriptions

This *Service* is used to request the next set of *QueryFirst* or *QueryNext* response information that is too large to be sent in a single response. “Too large” in this context means that the *Server* is not able to return a larger response or that the number of *QueryDataSets* to return exceeds the maximum number of *QueryDataSets* to return that was specified by the *Client* in the original request. The *QueryNext* shall be submitted on the same session that was used to submit the *QueryFirst* or *QueryNext* that is being continued.

5.9.4.2 Parameters

Table 47 defines the parameters for the *Service*.

Table 47 – QueryNext Service Parameters

Name	Type	Description
Request		
requestHeader	Request Header	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).
releaseContinuationPoint	Boolean	A <i>Boolean</i> parameter with the following values: TRUE passed <i>continuationPoint</i> shall be reset to free resources for the continuation point in the <i>Server</i>. FALSE passed <i>continuationPoint</i> shall be used to get the next set of <i>QueryDataSets</i>. A <i>Client</i> shall always use the continuation point returned by a <i>QueryFirst</i> or <i>QueryNext</i> response to free the resources for the continuation point in the <i>Server</i>. If the <i>Client</i> does not want to get the next set of <i>Query</i> information, <i>QueryNext</i> shall be called with this parameter set to TRUE. If the parameter is set to TRUE all array parameters in the response shall contain empty arrays.
continuationPoint	ContinuationPoint	Server defined opaque value that represents the continuation point. The value of the continuation point was returned to the <i>Client</i> in a previous <i>QueryFirst</i> or <i>QueryNext</i> response. This value is used to identify the previously processed <i>QueryFirst</i> or <i>QueryNext</i> request that is being continued, and the point in the result set from which the browse response is to continue. The <i>ContinuationPoint</i> type is described in 7.6.
Response		
responseHeader	Response Header	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).
queryDataSets []	QueryDataSet	The array of <i>QueryDataSets</i>. The <i>QueryDataSet</i> type is defined in 7.22.
revisedContinuationPoint	ContinuationPoint	Server-defined opaque value that represents the continuation point. It is used only if the information to be returned is too large to be contained in a single response. When not used or when <i>releaseContinuationPoint</i> is set, the value of this parameter is null. The <i>ContinuationPoint</i> type is described in 7.6.

5.9.4.3 Service results

Table 48 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 48 – QueryNext Service Result Codes

Symbolic Id	Description
Bad_ContinuationPointInvalid	See Table 166 for the description of this result code.

5.10 Attribute Service Set

5.10.1 Overview

This *Service Set* provides *Services* to access *Attributes* that are part of *Nodes*.

5.10.2 Read

5.10.2.1 Description

This *Service* is used to read one or more *Attributes* of one or more *Nodes*. For constructed *Attribute* values whose elements are indexed, such as an array, this *Service* allows *Clients* to read the entire set of indexed values as a composite, to read individual elements or to read ranges of elements of the composite.

The *maxAge* parameter is used to direct the *Server* to access the value from the underlying data source, such as a device, if its copy of the data is older than that which the *maxAge* specifies. If the *Server* cannot meet the requested maximum age, it returns its “best effort” value rather than rejecting the request.

5.10.2.2 Parameters

Table 49 defines the parameters for the *Service*.

Table 49 – Read Service Parameters

Name	Type	Description
Request		
requestHeader	RequestHeader	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).
maxAge	Duration	Maximum age of the value to be read in milliseconds. The age of the value is based on the difference between the <i>ServerTimestamp</i> and the time when the <i>Server</i> starts processing the request. For example if the <i>Client</i> specifies a <i>maxAge</i> of 500 milliseconds and it takes 100 milliseconds until the <i>Server</i> starts processing the request, the age of the returned value could be 600 milliseconds prior to the time it was requested. If the <i>Server</i> has one or more values of an <i>Attribute</i> that are within the maximum age, it can return any one of the values or it can read a new value from the data source. The number of values of an <i>Attribute</i> that a <i>Server</i> has depends on the number of <i>MonitoredItems</i> that are defined for the <i>Attribute</i>. In any case, the <i>Client</i> can make no assumption about which copy of the data will be returned. If the <i>Server</i> does not have a value that is within the maximum age, it shall attempt to read a new value from the data source. If the <i>Server</i> cannot meet the requested <i>maxAge</i>, it returns its “best effort” value rather than rejecting the request. This may occur when the time it takes the <i>Server</i> to process and return the new data value after it has been accessed is greater than the specified maximum age. If <i>maxAge</i> is set to 0, the <i>Server</i> shall attempt to read a new value from the data source. If <i>maxAge</i> is set to the max Int32 value or greater, the <i>Server</i> shall attempt to get a cached value. Negative values are invalid for <i>maxAge</i>.
timestampsToReturn	Enum TimestampsToReturn	An enumeration that specifies the <i>Timestamps</i> to be returned for each requested <i>Variable Value Attribute</i> . The <i>TimestampsToReturn</i> enumeration is defined in 7.34.
nodesToRead []	ReadValueId	List of <i>Nodes</i> and their <i>Attributes</i> to read. For each entry in this list, a <i>StatusCode</i> is returned, and if it indicates success, the <i>Attribute Value</i> is also returned. The <i>ReadValueId</i> parameter type is defined in 7.23.
Response		
responseHeader	ResponseHeader	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).
results []	DataValue	List of <i>Attribute</i> values (see 7.7 for <i>DataValue</i> definition). The size and order of this list matches the size and order of the <i>nodesToRead</i> request parameter. There is one entry in this list for each <i>Node</i> contained in the <i>nodesToRead</i> parameter.
diagnosticInfos []	DiagnosticInfo	List of diagnostic information (see 7.8 for <i>DiagnosticInfo</i> definition). The size and order of this list matches the size and order of the <i>nodesToRead</i> request parameter. There is one entry in this list for each <i>Node</i> contained in the <i>nodesToRead</i> parameter. This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the request.

5.10.2.3 Service results

Table 50 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 50 – Read Service Result Codes

Symbolic Id	Description
Bad_NothingToDo	See Table 165 for the description of this result code.
Bad_TooManyOperations	See Table 165 for the description of this result code.
Bad_MaxAgeInvalid	The max age parameter is invalid.
Bad_TimestampsToReturnInvalid	See Table 165 for the description of this result code.

5.10.2.4 StatusCodes

Table 51 defines values for the operation level *statusCode* contained in the *DataValue* structure of each *values* element. Common *StatusCodes* are defined in Table 166.

Table 51 – Read Operation Level Result Codes

Symbolic Id	Description
Bad_NodeIdInvalid	See Table 166 for the description of this result code.
Bad_NodeIdUnknown	See Table 166 for the description of this result code.
Bad_AttributeIdInvalid	See Table 166 for the description of this result code.
Bad_IndexRangeInvalid	See Table 166 for the description of this result code.
Bad_IndexRangeNoData	See Table 166 for the description of this result code.
Bad_DataEncodingInvalid	See Table 166 for the description of this result code.
Bad_DataEncodingUnsupported	See Table 166 for the description of this result code.
Bad_NotReadable	See Table 166 for the description of this result code.
Bad_UserAccessDenied	See Table 165 for the description of this result code.

5.10.3 HistoryRead

5.10.3.1 Description

This *Service* is used to read historical values or *Events* of one or more *Nodes*. For constructed *Attribute* values whose elements are indexed, such as an array, this *Service* allows *Clients* to read the entire set of indexed values as a composite, to read individual elements or to read ranges of elements of the composite. *Servers* may make historical values available to *Clients* using this *Service*, although the historical values themselves are not visible in the *AddressSpace*.

The *AccessLevel Attribute* defined in IEC 62541-3 indicates a *Node's* support for historical values. Several request parameters indicate how the *Server* is to access values from the underlying history data source. The *EventNotifier Attribute* defined in IEC 62541-3 indicates a *Node's* support for historical *Events*.

The *continuationPoint* parameter in the *HistoryRead* is used to mark a point from which to continue the read if not all values could be returned in one response. The value is opaque for the *Client* and is only used to maintain the state information for the *Server* to continue from. A *Server* may use the timestamp of the last returned data item if the timestamp is unique. This can reduce the need in the *Server* to store state information for the continuation point.

For additional details on reading historical data and historical *Events* see IEC 62541-11.

5.10.3.2 Parameters

Table 52 defines the parameters for the *Service*.

Table 52 – HistoryRead Service Parameters

Name	Type	Description
Request		
requestHeader	RequestHeader	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).
historyReadDetails	Extensible Parameter HistoryReadDetails	The details define the types of history reads that can be performed. The <i>HistoryReadDetails</i> parameter type is an extensible parameter type formally defined in IEC 62541-11. The <i>ExtensibleParameter</i> type is defined in 7.11.
timestampsToReturn	Enum TimestampsToReturn	An enumeration that specifies the timestamps to be returned for each requested <i>Variable Value Attribute</i> . The <i>TimestampsToReturn</i> enumeration is defined in 7.34. Specifying a <i>TimestampsToReturn</i> of NEITHER is not valid. A <i>Server</i> shall return a <i>Bad_InvalidTimestampArgument StatusCode</i> in this case. IEC 62541-11 defines exceptions where this parameter shall be ignored.
releaseContinuationPoints	Boolean	A <i>Boolean</i> parameter with the following values: TRUE passed <i>continuationPoints</i> shall be reset to free resources in the <i>Server</i> . FALSE passed <i>continuationPoints</i> shall be used to get the next set of historical information. A <i>Client</i> shall always use the continuation point returned by a <i>HistoryRead</i> response to free the resources for the continuation point in the <i>Server</i> . If the <i>Client</i> does not want to get the next set of historical information, <i>HistoryRead</i> shall be called with this parameter set to TRUE.
nodesToRead []	HistoryReadValuelid	This parameter contains the list of items upon which the historical retrieval is to be performed. This structure is defined in-line with the following indented items.
nodeId	NodeId	If the <i>HistoryReadDetails</i> is RAW, PROCESSED, MODIFIED or ATTIME: The <i>nodeId</i> of the <i>Nodes</i> whose historical values are to be read. The value returned shall always include a timestamp. If the <i>HistoryReadDetails</i> is EVENTS: The <i>NodeId</i> of the <i>Node</i> whose <i>Event</i> history is to be read. If the <i>Node</i> does not support the requested access for historical values or historical <i>Events</i> the appropriate error response for the given <i>Node</i> shall be generated.
indexRange	NumericRange	This parameter is used to identify a single element of an array, or a single range of indexes for arrays. If a range of elements is specified, the values are returned as a composite. The first element is identified by index 0 (zero). The <i>NumericRange</i> type is defined in 7.21. This parameter is null if the value is not an array. However, if the value is an array, and this parameter is null, then all elements are to be included in the range.
dataEncoding	QualifiedName	A <i>QualifiedName</i> that specifies the data encoding to be returned for the <i>Value</i> to be read (see 7.23 for definition how to specify the data encoding). The parameter is ignored when reading history of <i>Events</i> .
continuationPoint	ByteString	For each <i>NodesToRead</i> item this parameter specifies a continuation point returned from a previous <i>HistoryRead</i> call, allowing the <i>Client</i> to continue that read from the last value received. The <i>HistoryRead</i> is used to select an ordered sequence of historical values or events. A continuation point marks a point in that ordered sequence, such that the <i>Server</i> returns the subset of the sequence that follows that point. A null value indicates that this parameter is not used. This continuation point is described in more detail in IEC 62541-11.
Response		
responseHeader	ResponseHeader	Common response parameters (see 7.27 for <i>ResponseHeader</i> type).
results []	HistoryReadResult	List of read results. The size and order of the list matches the size and order of the <i>nodesToRead</i> request parameter. This structure is defined in-line with the following indented items.
statusCode	StatusCode	<i>StatusCode</i> for the <i>NodesToRead</i> item (see 7.33 for <i>StatusCode</i> definition).
continuationPoint	ByteString	This parameter is used only if the number of values to be returned is too large to be returned in a single response. When this parameter is not used, its value is null. <i>Servers</i> shall support at least one continuation point per <i>Session</i> . <i>Servers</i> specify a max history continuation points per <i>Session</i> in the <i>Server</i> capabilities <i>Object</i> defined in IEC 62541-5. A continuation point shall remain active until the <i>Client</i> passes the continuation point to <i>HistoryRead</i> or the <i>Session</i> is closed. If the max continuation points have been reached the oldest continuation point shall be reset.
historyData	Extensible Parameter HistoryData	The history data returned for the <i>Node</i> . The <i>HistoryData</i> parameter type is an extensible parameter type formally defined in IEC 62541-11. It specifies the types of history data that can be returned. The <i>ExtensibleParameter</i> base type is defined in 7.11.
diagnosticInfos []	Diagnostic Info	List of diagnostic information. The size and order of the list matches the size and order of the <i>nodesToRead</i> request parameter. There is one entry in this list

Name	Type	Description
		for each <i>Node</i> contained in the <i>nodesToRead</i> parameter. This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the request.

5.10.3.3 Service results

Table 53 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 53 – HistoryRead Service Result Codes

Symbolic Id	Description
Bad_NothingToDo	See Table 165 for the description of this result code.
Bad_TooManyOperations	See Table 165 for the description of this result code.
Bad_TimestampsToReturnInvalid	See Table 165 for the description of this result code.
Bad_HistoryOperationInvalid	See Table 166 for the description of this result code.
Bad_HistoryOperationUnsupported	See Table 166 for the description of this result code. The requested history operation is not supported by the server.

5.10.3.4 StatusCodes

Table 54 defines values for the operation level *statusCode* parameter that are specific to this *Service*. Common *StatusCodes* are defined in Table 166. History access specific *StatusCodes* are defined in IEC 62541-11.

Table 54 – HistoryRead Operation Level Result Codes

Symbolic Id	Description
Bad_NodeIdInvalid	See Table 166 for the description of this result code.
Bad_NodeIdUnknown	See Table 166 for the description of this result code.
Bad_DataEncodingInvalid	See Table 166 for the description of this result code.
Bad_DataEncodingUnsupported	See Table 166 for the description of this result code.
Bad_UserAccessDenied	See Table 165 for the description of this result code.
Bad_ContinuationPointInvalid	See Table 165 for the description of this result code.
Bad_InvalidTimestampArgument	The defined timestamp to return was invalid.
Bad_HistoryOperationUnsupported	See Table 166 for the description of this result code. The requested history operation is not supported for the requested node.

5.10.4 Write

5.10.4.1 Description

This *Service* is used to write values to one or more *Attributes* of one or more *Nodes*. For constructed *Attribute* values whose elements are indexed, such as an array, this *Service* allows *Clients* to write the entire set of indexed values as a composite, to write individual elements or to write ranges of elements of the composite.

The values are written to the data source, such as a device, and the *Service* does not return until it writes the values or determines that the value cannot be written. In certain cases, the *Server* will successfully write to an intermediate system or *Server*, and will not know if the data source was updated properly. In these cases, the *Server* should report a success code that indicates that the write was not verified. In the cases where the *Server* is able to verify that it has successfully written to the data source, it reports an unconditional success.

The order the operations are processed in the *Server* is not defined and depends on the different data sources and the internal *Server* logic. If an *Attribute* and *Node* combination is contained in more than one operation, the order of the processing is undefined. If a *Client* requires sequential processing the *Client* needs separate *Service* calls.

It is possible that the *Server* may successfully write some *Attributes*, but not others. Rollback is the responsibility of the *Client*.

If a *Server* allows writing of *Attributes* with the *DataType* *LocalizedText*, the *Client* can add or overwrite the text for a locale by writing the text with the associated *LocaleId*. Writing a null *String* for the text for a locale shall delete the *String* for that locale. Writing a null *String* for the *locale* and a non-null *String* for the *text* is setting the *text* for an invariant locale. Writing a null *String* for the *text* and a null *String* for the *locale* shall delete the entries for all locales. If a *Client* attempts to write a *locale* that is either syntactically invalid or not supported, the *Server* returns *Bad_LocaleNotSupported*.

5.10.4.2 Parameters

Table 55 defines the parameters for the *Service*.

Table 55 – Write Service Parameters

Name	Type	Description
Request		
requestHeader	RequestHeader	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).
nodesToWrite []	WriteValue	List of <i>Nodes</i> and their <i>Attributes</i> to write. This structure is defined in-line with the following indented items.
nodeId	NodeId	<i>NodeId</i> of the <i>Node</i> that contains the <i>Attributes</i> .
attributeId	IntegerId	Id of the <i>Attribute</i> . This shall be a valid <i>Attribute</i> id. The <i>IntegerId</i> is defined in 7.13. The <i>IntegerIds</i> for the <i>Attributes</i> are defined in IEC 62541-6.
indexRange	NumericRange	This parameter is used to identify a single element of an array, or a single range of indexes for arrays. The first element is identified by index 0 (zero). The <i>NumericRange</i> type is defined in 7.21. This parameter is not used if the specified <i>Attribute</i> is not an array. However, if the specified <i>Attribute</i> is an array and this parameter is not used, then all elements are to be included in the range. The parameter is null if not used. A <i>Server</i> shall return a <i>Bad_WriteNotSupported</i> error if an <i>indexRange</i> is provided and writing of <i>indexRange</i> is not possible for the <i>Node</i> .
value	DataValue	The <i>Node's</i> <i>Attribute</i> value (see 7.7 for <i>DataValue</i> definition). If the <i>indexRange</i> parameter is specified then the <i>Value</i> shall be an array even if only one element is being written. If the <i>SourceTimestamp</i> or the <i>ServerTimestamp</i> is specified, the <i>Server</i> shall use these values. The <i>Server</i> returns a <i>Bad_WriteNotSupported</i> error if it does not support writing of timestamps. A <i>Server</i> shall return a <i>Bad_TypeMismatch</i> error if the data type of the written value is not the same type or subtype of the <i>Attribute's</i> <i>DataType</i> . Based on the <i>DataType</i> hierarchy, subtypes of the <i>Attribute</i> <i>DataType</i> shall be accepted by the <i>Server</i> . For the <i>Value</i> <i>Attribute</i> the <i>DataType</i> is defined through the <i>DataType</i> <i>Attribute</i> . A <i>ByteString</i> is structurally the same as a one dimensional array of <i>Byte</i> . A <i>Server</i> shall accept a <i>ByteString</i> if an array of <i>Byte</i> is expected. The <i>Server</i> returns a <i>Bad_DataEncodingUnsupported</i> error if it does not support the provided data encoding. <i>Simple DataTypes</i> (see IEC 62541-3) use the same representation on the wire as their supertypes and therefore writing a value of a simple <i>DataType</i> cannot be distinguished from writing a value of its supertype. The <i>Server</i> shall assume that by receiving the correct wire representation for a simple <i>DataType</i> the correct type was chosen. <i>Servers</i> are allowed to impose additional data validations on the value independent of the encoding (e.g. having an image in GIF format in a <i>ByteString</i>). In this case the <i>Server</i> shall return a <i>Bad_TypeMismatch</i> error if the validation fails.
Response		
responseHeader	ResponseHeader	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).
results []	StatusCode	List of results for the <i>Nodes</i> to write (see 7.33 for <i>StatusCode</i> definition). The size and order of the list matches the size and order of the <i>nodesToWrite</i> request parameter. There is one entry in this list for each <i>Node</i> contained in the <i>nodesToWrite</i> parameter.
diagnosticInfos []	DiagnosticInfo	List of diagnostic information for the <i>Nodes</i> to write (see 7.8 for <i>DiagnosticInfo</i> definition). The size and order of the list matches the size and order of the <i>nodesToWrite</i> request parameter. This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the request.

5.10.4.3 Service results

Table 56 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 56 – Write Service Result Codes

Symbolic Id	Description
Bad_NothingToDo	See Table 165 for the description of this result code.
Bad_TooManyOperations	See Table 165 for the description of this result code.

5.10.4.4 StatusCodes

Table 57 defines values for the *results* parameter that are specific to this *Service*. Common *StatusCodes* are defined in Table 166.

Table 57 – Write Operation Level Result Codes

Symbolic Id	Description
Good_CompletesAsynchronously	See Table 165 for the description of this result code. The value was successfully written to an intermediate system but the <i>Server</i> does not know if the data source was updated properly.
Bad_NodeIdInvalid	See Table 166 for the description of this result code.
Bad_NodeIdUnknown	See Table 166 for the description of this result code.
Bad_AttributeIdInvalid	See Table 166 for the description of this result code.
Bad_IndexRangeInvalid	See Table 166 for the description of this result code.
Bad_IndexRangeNoData	See Table 166 for the description of this result code.
Bad_WriteNotSupported	The requested write operation is not supported. If a <i>Client</i> attempts to write any value, quality, timestamp combination and the <i>Server</i> does not support the requested combination (which could be a single quantity such as just timestamp), then the <i>Server</i> shall not perform any write on this <i>Node</i> and shall return this <i>StatusCode</i> for this <i>Node</i> . It is also used if writing an <i>IndexRange</i> is not supported for a <i>Node</i> .
Bad_NotWritable	See Table 166 for the description of this result code.
Bad_UserAccessDenied	See Table 165 for the description of this result code. The current user does not have permission to write the attribute.
Bad_OutOfRange	See Table 166 for the description of this result code. If a <i>Client</i> attempts to write a value outside the valid range like a value not contained in the enumeration data type of the <i>Node</i> , the <i>Server</i> shall return this <i>StatusCode</i> for this <i>Node</i> .
Bad_TypeMismatch	See Table 166 for the description of this result code.
Bad_DataEncodingUnsupported	See Table 166 for the description of this result code.
Bad_NoCommunication	See Table 166 for the description of this result code.
Bad_LocaleNotSupported	The locale in the requested write operation is not supported.

5.10.5 HistoryUpdate

5.10.5.1 Description

This *Service* is used to update historical values or *Events* of one or more *Nodes*. Several request parameters indicate how the *Server* is to update the historical value or *Event*. Valid actions are Insert, Replace or Delete.

5.10.5.2 Parameters

Table 58 defines the parameters for the *Service*.

Table 58 – HistoryUpdate Service Parameters

Name	Type	Description
Request		
requestHeader	RequestHeader	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).
historyUpdateDetails []	Extensible Parameter HistoryUpdate Details	The details defined for this update. The <i>HistoryUpdateDetails</i> parameter type is an extensible parameter type formally defined in IEC 62541-11. It specifies the types of history updates that can be performed. The <i>ExtensibleParameter</i> type is defined in 7.11.
Response		
responseHeader	ResponseHeader	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).
results []	HistoryUpdate Result	List of update results for the history update details. The size and order of the list matches the size and order of the details element of the <i>historyUpdateDetails</i> parameter specified in the request. This structure is defined in-line with the following indented items.
statusCode	StatusCode	<i>StatusCode</i> for the update of the <i>Node</i> (see 7.33 for <i>StatusCode</i> definition).
operationResults []	StatusCode	List of <i>StatusCodes</i> for the operations to be performed on a <i>Node</i> . The size and order of the list matches the size and order of any list defined by the details element being reported by this result entry.
diagnosticInfos []	DiagnosticInfo	List of diagnostic information for the operations to be performed on a <i>Node</i> (see 7.8 for <i>DiagnosticInfo</i> definition). The size and order of the list matches the size and order of any list defined by the details element being reported by this <i>updateResults</i> entry. This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the request.
diagnosticInfos []	DiagnosticInfo	List of diagnostic information for the history update details. The size and order of the list matches the size and order of the details element of the <i>historyUpdateDetails</i> parameter specified in the request. This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the request.

5.10.5.3 Service results

Table 59 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 59 – HistoryUpdate Service Result Codes

Symbolic Id	Description
Bad_NothingToDo	See Table 165 for the description of this result code.
Bad_TooManyOperations	See Table 165 for the description of this result code.

5.10.5.4 StatusCodes

Table 60 defines values for the *statusCode* and *operationResults* parameters that are specific to this *Service*. Common *StatusCodes* are defined in Table 166. History access specific *StatusCodes* are defined in IEC 62541-11.

Table 60 – HistoryUpdate Operation Level Result Codes

Symbolic Id	Description
Bad_NotWritable	See Table 166 for the description of this result code.
Bad_HistoryOperationInvalid	See Table 166 for the description of this result code.
Bad_HistoryOperationUnsupported	See Table 166 for the description of this result code.
Bad_UserAccessDenied	See Table 165 for the description of this result code. The current user does not have permission to update the history.

5.11 Method Service Set

5.11.1 Overview

Methods represent the function calls of *Objects*. They are defined in IEC 62541-3. *Methods* are invoked and return only after completion (successful or unsuccessful). Execution times for methods may vary, depending on the function that they perform.

The *Method Service Set* defines the means to invoke methods. A *method* shall be a *component* of an *Object*. Discovery is provided through the *Browse* and *Query Services*. *Clients* discover the *methods* supported by a *Server* by browsing for the owning *Objects References* that identify their supported *methods*.

Because *Methods* may control some aspect of plant operations, method invocation may depend on environmental or other conditions. This may be especially true when attempting to re-invoke a method immediately after it has completed execution. Conditions that are required to invoke the method might not yet have returned to the state that permits the method to start again.

5.11.2 Call

5.11.2.1 Description

This *Service* is used to call (invoke) a list of *Methods*. Each *method* call is invoked within the context of an existing *Session*. If the *Session* is terminated, the results of the *method's* execution cannot be returned to the *Client* and are discarded. This is independent of the task actually performed at the *Server*.

The order the operations are processed in the *Server* is not defined and depends on the different tasks and the internal *Server* logic. If a *Method* is contained in more than one operation, the order of the processing is undefined. If a *Client* requires sequential processing the *Client* needs separate *Service* calls.

This *Service* provides for passing input and output arguments to/from a method. These arguments are defined by *Properties* of the method.

5.11.2.2 Parameters

Table 61 defines the parameters for the *Service*.

Table 61 – Call Service Parameters

Name	Type	Description
Request		
requestHeader	RequestHeader	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).
methodsToCall []	CallMethodRequest	List of <i>Methods</i> to call. This structure is defined in-line with the following indented items.
objectId	NodeId	The <i>NodeId</i> shall be that of the <i>Object</i> or <i>ObjectType</i> that is the source of a <i>HasComponent Reference</i> (or subtype of <i>HasComponent Reference</i>) to the <i>Method</i> specified in <i>methodId</i> . See IEC 62541-3 for a description of <i>Objects</i> and their <i>Methods</i> .
methodId	NodeId	<i>NodeId</i> of the <i>Method</i> to invoke. If the <i>objectId</i> is the <i>NodeId</i> of an <i>Object</i> , it is allowed to use the <i>NodeId</i> of a <i>Method</i> that is the target of a <i>HasComponent Reference</i> from the <i>ObjectType</i> of the <i>Object</i> .
inputArguments []	BaseDataType	List of input argument values. An empty list indicates that there are no input arguments. The size and order of this list matches the size and order of the input arguments defined by the input <i>InputArguments Property</i> of the <i>Method</i> . The name, a description and the data type of each argument are defined by the <i>Argument</i> structure in each element of the method's <i>InputArguments Property</i> .
Response		
responseHeader	ResponseHeader	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).
results []	CallMethodResult	Result for the <i>Method</i> calls. This structure is defined in-line with the following indented items.
statusCode	StatusCode	<i>StatusCode</i> of the <i>Method</i> executed in the server. This <i>StatusCode</i> is set to the <i>Bad_InvalidArgument StatusCode</i> if at least one input argument broke a constraint (e.g. wrong data type, value out of range). This <i>StatusCode</i> is set to a bad <i>StatusCode</i> if the <i>Method</i> execution failed in the server, e.g. based on an exception.
inputArgumentResults []	StatusCode	List of <i>StatusCodes</i> corresponding to the <i>inputArguments</i> . This list is empty unless the operation level result is <i>Bad_InvalidArgument</i> . If this list is populated, it has the same length as the <i>inputArgument</i> list.
inputArgumentDiagnosticInfos []	DiagnosticInfo	List of diagnostic information corresponding to the <i>inputArguments</i> . This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the request.
outputArguments []	BaseDataType	List of output argument values. An empty list indicates that there are no output arguments. The size and order of this list matches the size and order of the output arguments defined by the <i>OutputArguments Property</i> of the <i>Method</i> . The name, a description and the data type of each argument are defined by the <i>Argument</i> structure in each element of the methods <i>OutputArguments Property</i> .
diagnosticInfos []	DiagnosticInfo	List of diagnostic information for the <i>statusCode</i> of the <i>results</i> . This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the request.

5.11.2.3 Service results

Table 62 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 62 – Call Service Result Codes

Symbolic Id	Description
Bad_NothingToDo	See Table 165 for the description of this result code.
Bad_TooManyOperations	See Table 165 for the description of this result code.

5.11.2.4 StatusCodes

Table 63 defines values for the *statusCode* and *inputArgumentResults* parameters that are specific to this *Service*. Common *StatusCodes* are defined in Table 166.

Table 63 – Call Operation Level Result Codes

Symbolic Id	Description
Bad_NodeIdInvalid	See Table 166 for the description of this result code. Used to indicate that the specified object is not valid.
Bad_NodeIdUnknown	See Table 166 for the description of this result code. Used to indicate that the specified object is not valid.
Bad_ArgumentsMissing	The client did not specify all of the input arguments for the method.
Bad_TooManyArguments	The client specified more input arguments than defined for the method.
Bad_InvalidArgument	See Table 165 for the description of this result code. Used to indicate in the operation level results that one or more of the input arguments are invalid. The <i>inputArgumentResults</i> contain the specific status code for each invalid argument.
Bad_UserAccessDenied	See Table 165 for the description of this result code.
Bad_MethodInvalid	The method id does not refer to a method for the specified object.
Bad_OutOfRange	See Table 166 for the description of this result code. Used to indicate that an input argument is outside the acceptable range.
Bad_TypeMismatch	See Table 166 for the description of this result code. Used to indicate that an input argument does not have the correct data type. A <i>ByteString</i> is structurally the same as a one dimensional array of <i>Byte</i> . A server shall accept a <i>ByteString</i> if an array of <i>Byte</i> is expected.
Bad_NoCommunication	See Table 166 for the description of this result code.

5.12 MonitoredItem Service Set

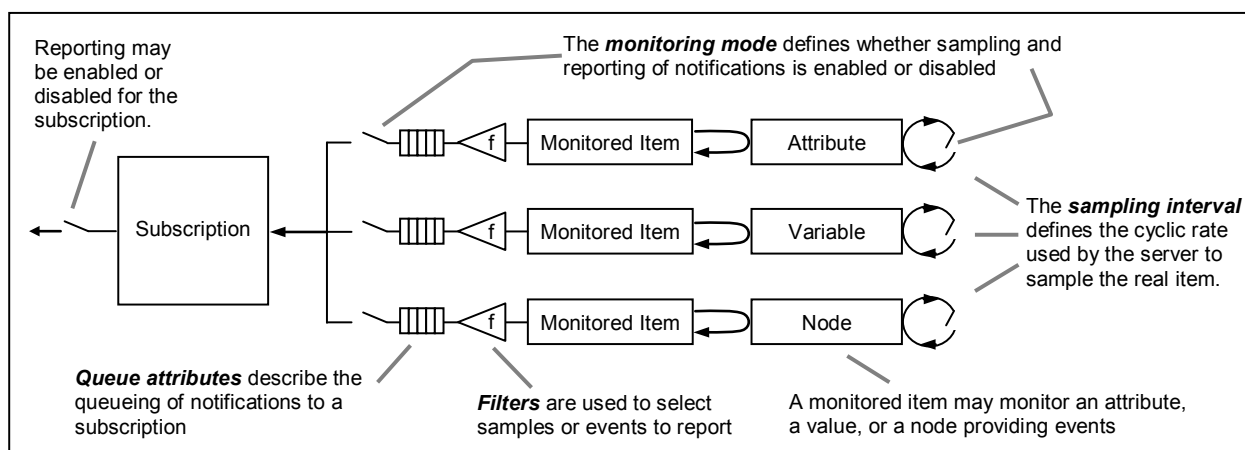
5.12.1 MonitoredItem model

5.12.1.1 Overview

Clients define *MonitoredItems* to subscribe to data and *Events*. Each *MonitoredItem* identifies the item to be monitored and the *Subscription* to use to send *Notifications*. The item to be monitored may be any *Node Attribute*.

Notifications are data structures that describe the occurrence of data changes and *Events*. They are packaged into *NotificationMessages* for transfer to the *Client*. The *Subscription* periodically sends *NotificationMessages* at a user-specified publishing interval, and the cycle during which these messages are sent is called a publishing cycle.

Four primary parameters are defined for *MonitoredItems* that tell the *Server* how the item is to be sampled, evaluated and reported. These parameters are the sampling interval, the monitoring mode, the filter and the queue parameter. Figure 15 illustrates these concepts.



IEC

Figure 15 – MonitoredItem Model

Attributes, other than the *Value Attribute*, are only monitored for a change in value. The filter is not used for these *Attributes*. Any change in value for these *Attributes* causes a *Notification* to be generated.

The *Value Attribute* is used when monitoring *Variables*. *Variable* values are monitored for a change in value or a change in their status. The filters defined in this standard (see 7.16.2) and in IEC 62541-8 are used to determine if the value change is large enough to cause a *Notification* to be generated for the *Variable*.

Objects and views can be used to monitor *Events*. *Events* are only available from *Nodes* where the *SubscribeToEvents* bit of the *EventNotifier Attribute* is set. The filter defined in this standard (see 7.16.3) is used to determine if an *Event* received from the *Node* is sent to the *Client*. The filter also allows selecting fields of the *EventType* that will be contained in the *Event* such as *EventId*, *EventType*, *SourceNode*, *Time* and *Description*.

IEC 62541-3 describes the *Event* model and the base *EventTypes*.

The *Properties* of the base *EventTypes* and the representation of the base *EventTypes* in the *AddressSpace* are specified in IEC 62541-5.

5.12.1.2 Sampling interval

Each *MonitoredItem* created by the *Client* is assigned a sampling interval that is either inherited from the publishing interval of the *Subscription* or that is defined specifically to override that rate. A negative number indicates that the default sampling interval defined by the publishing interval of the *Subscription* is requested. The sampling interval indicates the fastest rate at which the *Server* should sample its underlying source for data changes.

A *Client* shall define a sampling interval of 0 if it subscribes for *Events*.

The assigned sampling interval defines a “best effort” cyclic rate that the *Server* uses to sample the item from its source. “Best effort” in this context means that the *Server* does its best to sample at this rate. Sampling at rates faster than this rate is acceptable, but not necessary to meet the needs of the *Client*. How the *Server* deals with the sampling rate and how often it actually polls its data source internally is a *Server* implementation detail. However, the time between values returned to the *Client* shall be greater or equal to the sampling interval.

The *Client* may also specify 0 for the sampling interval, which indicates that the *Server* should use the fastest practical rate. It is expected that *Servers* will support only a limited set of

sampling intervals to optimize their operation. If the exact interval requested by the *Client* is not supported by the *Server*, then the *Server* assigns to the *MonitoredItem* the most appropriate interval as determined by the *Server*. It returns this assigned interval to the *Client*. The *Server Capabilities Object* defined in IEC 62541-5 identifies the sampling intervals supported by the *Server*.

The *Server* may support data that is collected based on a sampling model or generated based on an exception-based model. The fastest supported sampling interval may be equal to 0, which indicates that the data item is exception-based rather than being sampled at some period. An exception-based model means that the underlying system does not require sampling and reports data changes.

The *Client* may use the revised sampling interval values as a hint for setting the publishing interval as well as the keep-alive count of a *Subscription*. If, for example, the smallest revised sampling interval of the *MonitoredItems* is 5 seconds, then the time before a keep-alive is sent should be longer than 5 seconds.

Note that, in many cases, the OPC UA *Server* provides access to a decoupled system and therefore has no knowledge of the data update logic. In this case, even though the OPC UA *Server* samples at the negotiated rate, the data might be updated by the underlying system at a much slower rate. In this case, changes can only be detected at this slower rate.

If the behaviour by which the underlying system updates the item is known, it will be available via the *MinimumSamplingInterval Attribute* defined in IEC 62541-3. If the *Server* specifies a value for the *MinimumSamplingInterval Attribute* it shall always return a *revisedSamplingInterval* that is equal or higher than the *MinimumSamplingInterval* if the *Client* subscribes to the *Value Attribute*.

Clients should also be aware that the sampling by the OPC UA *Server* and the update cycle of the underlying system are usually not synchronized. This can cause additional delays in change detection, as illustrated in Figure 16.

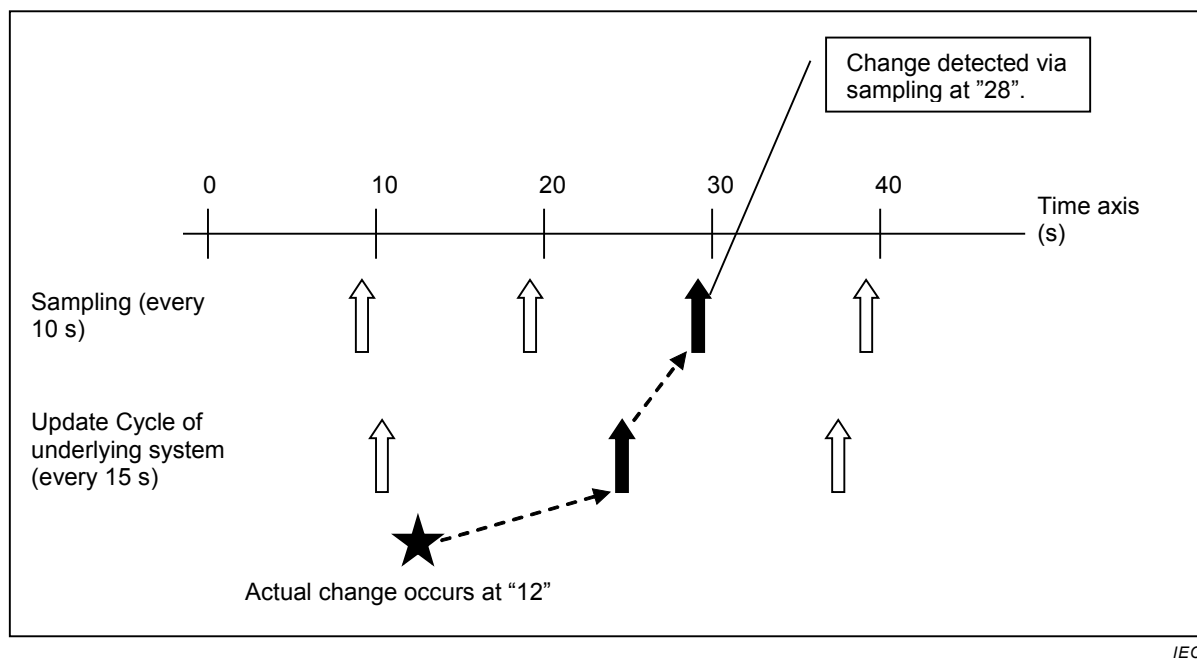


Figure 16 – Typical delay in change detection

5.12.1.3 Monitoring mode

The monitoring mode parameter is used to enable and disable the sampling of a *MonitoredItem*, and also to provide for independently enabling and disabling the reporting of *Notifications*. This capability allows a *MonitoredItem* to be configured to sample, sample and report, or neither. Disabling sampling does not change the values of any of the other *MonitoredItem* parameter, such as its sampling interval.

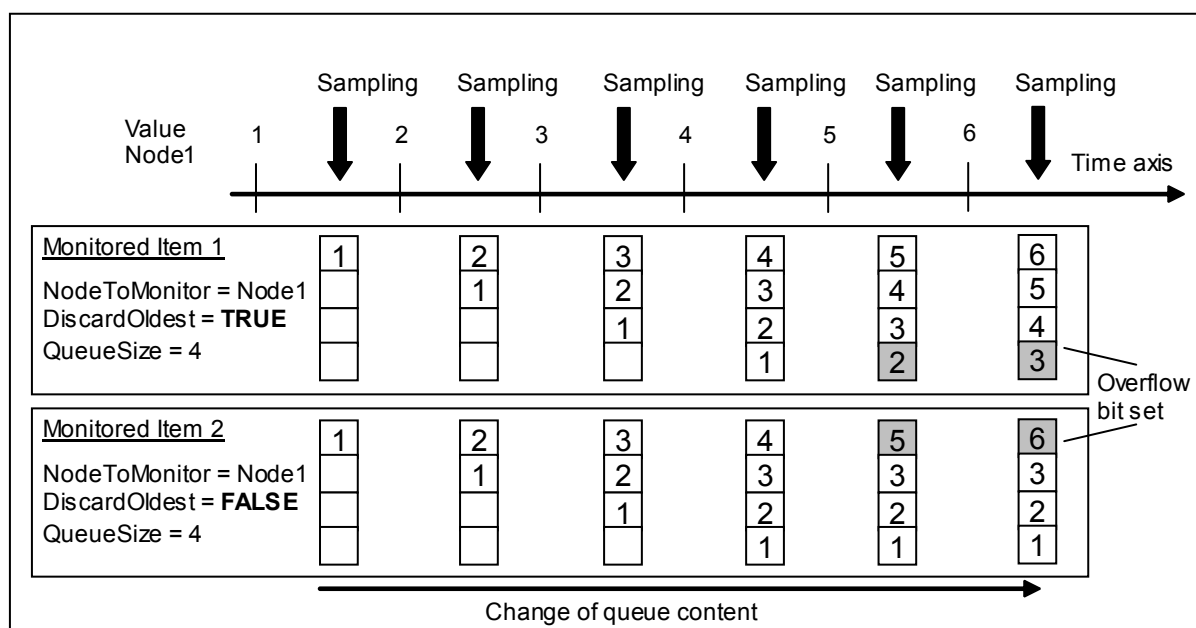
When a *MonitoredItem* is enabled (i.e. when the *MonitoringMode* is changed from *DISABLED* to *SAMPLING* or *REPORTING*) or it is created in the enabled state, the *Server* shall report the first sample as soon as possible and the time of this sample becomes the starting point for the next sampling interval.

5.12.1.4 Filter

Each time a *MonitoredItem* is sampled, the *Server* evaluates the sample using the filter defined for the *MonitoredItem*. The filter parameter defines the criteria that the *Server* uses to determine if a *Notification* should be generated for the sample. The type of filter is dependent on the type of the item that is being monitored. For example, the *DataChangeFilter* and the *AggregateFilter* are used when monitoring *Variable Values* and the *EventFilter* is used when monitoring *Events*. Sampling and evaluation, including the use of filters, are described in this standard. Additional filters may be defined in other parts of this series of standards.

5.12.1.5 Queue parameters

If the sample passes the filter criteria, a *Notification* is generated and queued for transfer by the *Subscription*. The size of the queue is defined when the *MonitoredItem* is created. When the queue is full and a new *Notification* is received, the *Server* either discards the oldest *Notification* and queues the new one, or it replaces the last value added to the queue with the new one. The *MonitoredItem* is configured for one of these discard policies when the *MonitoredItem* is created. If a *Notification* is discarded for a *DataValue* and the size of the queue is larger than one, then the *Overflow* bit (flag) in the *InfoBits* portion of the *DataValue statusCode* is set. If *discardOldest* is TRUE, the oldest value gets deleted from the queue and the next value in the queue gets the flag set. If *discardOldest* is FALSE, the last value added to the queue gets replaced with the new value. The new value gets the flag set to indicate the lost values in the next *NotificationMessage*. Figure 17 illustrates the queue overflow handling.



IEC

Figure 17 – Queue overflow handling

If the queue size is one, the queue becomes a buffer that always contains the newest *Notification*. In this case, if the sampling interval of the *MonitoredItem* is faster than the publishing interval of the *Subscription*, the *MonitoredItem* will be over sampling and the *Client* will always receive the most up-to-date value. The discard policy is ignored if the queue size is one.

On the other hand, the *Client* may want to subscribe to a continuous stream of *Notifications* without any gaps, but does not want them reported at the sampling interval. In this case, the *MonitoredItem* would be created with a queue size large enough to hold all *Notifications* generated between two consecutive publishing cycles. Then, at each publishing cycle, the *Subscription* would send all *Notifications* queued for the *MonitoredItem* to the *Client*. The *Server* shall return *Notifications* for any particular item in the same order they are in the queue.

The *Server* may be sampling at a faster rate than the sampling interval to support other *Clients*; the *Client* should only expect values at the negotiated sampling interval. The *Server* may deliver fewer values than dictated by the sampling interval, based on the filter and implementation constraints. If a *DataChangeFilter* is configured for a *MonitoredItem*, it is always applied to the newest value in the queue compared to the current sample.

If, for example, the *AbsoluteDeadband* in the *DataChangeFilter* is “10”, the queue could consist of values in the following order:

- 100
- 111
- 100
- 89
- 100

Queuing of data may result in unexpected behaviour when using a *Deadband* filter and the number of encountered changes is larger than the number of values that can be maintained. The new first value in the queue may not exceed the *Deadband* limit of the previous value sent to the *Client*.

The queue size is the maximum value supported by the *Server* when monitoring *Events*. In this case, the *Server* is responsible for the *Event* buffer. If *Events* are lost, an *Event* of the type *EventQueueOverflowEventType* is placed in the queue. This *Event* is generated when the first *Event* has to be discarded on a *MonitoredItem* subscribing for *Events*. It is put into the Queue of the *MonitoredItem* in addition to the size of the Queue defined for this *MonitoredItem* without discarding any other *Event*. If *discardOldest* is set to TRUE it is put at the beginning of the queue and is never discarded, otherwise at the end. An aggregating *Server* shall not pass on such an *Event*. It shall be handled like other connection error scenarios.

5.12.1.6 Triggering model

The *MonitoredItems Service* allows the addition of items that are reported only when some other item (the triggering item) triggers. This is done by creating links between the triggered items and the items to report. The monitoring mode of the items to report is set to sampling-only so that it will sample and queue *Notifications* without reporting them. Figure 18 illustrates this concept.

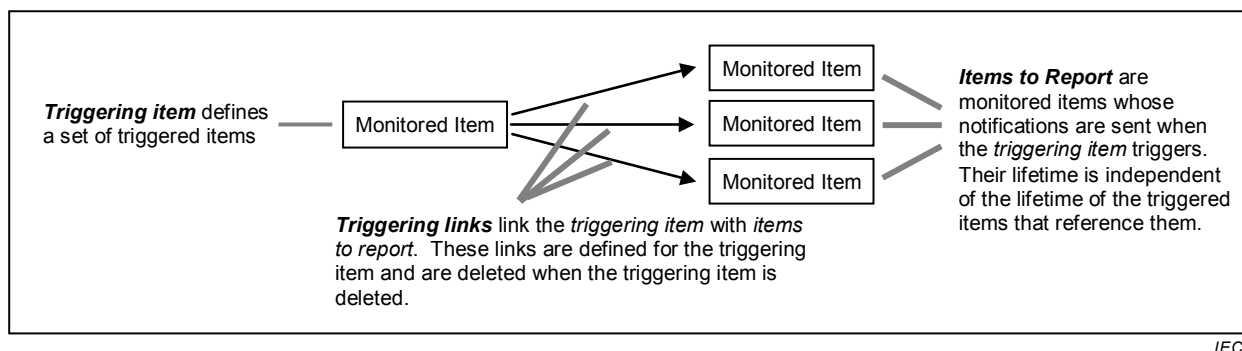


Figure 18 – Triggering Model

The triggering mechanism is a useful feature that allows *Clients* to reduce the data volume on the wire by configuring some items to sample frequently but only report when some other *Event* happens.

The following triggering behaviours are specified.

- If the monitoring mode of the triggering item is *SAMPLING*, then it is not reported when the triggering item triggers the items to report.
- If the monitoring mode of the triggering item is *REPORTING*, then it is reported when the triggering item triggers the items to report.
- If the monitoring mode of the triggering item is *DISABLED*, then the triggering item does not trigger the items to report.
- If the monitoring mode of the item to report is *SAMPLING*, then it is reported when the triggering item triggers the items to report.
- If the monitoring mode of the item to report is *REPORTING*, this effectively causes the triggering item to be ignored. All notifications of the items to report are sent after the publishing interval expires.
- If the monitoring mode of the item to report is *DISABLED*, then there will be no sampling of the item to report and therefore no notifications to report.
- The first trigger shall occur when the first notification is queued for the triggering item after the creation of the link.

Clients create and delete triggering links between a triggering item and a set of items to report. If the *MonitoredItem* that represents an item to report is deleted before its associated

triggering link is deleted, the triggering link is also deleted, but the triggering item is otherwise unaffected.

Deletion of a *MonitoredItem* should not be confused with the removal of the *Attribute* that it monitors. If the *Node* that contains the *Attribute* being monitored is deleted, the *MonitoredItem* generates a *Notification* with a *StatusCode* *Bad_NodeIdUnknown* that indicates the deletion, but the *MonitoredItem* is not deleted.

5.12.2 CreateMonitoredItems

5.12.2.1 Description

This *Service* is used to create and add one or more *MonitoredItems* to a *Subscription*. A *MonitoredItem* is deleted automatically by the *Server* when the *Subscription* is deleted. Deleting a *MonitoredItem* causes its entire set of triggered item links to be deleted, but has no effect on the *MonitoredItems* referenced by the triggered items.

Calling the *CreateMonitoredItems Service* repetitively to add a small number of *MonitoredItems* each time may adversely affect the performance of the *Server*. Instead, *Clients* should add a complete set of *MonitoredItems* to a *Subscription* whenever possible.

When a *MonitoredItem* is added, the *Server* performs initialization processing for it. The initialization processing is defined by the *Notification* type of the item being monitored. *Notification* types are specified in this standard and in the Access Type Specification parts of this series of standards, such as IEC 62541-8. See IEC TR 62541-1 for a description of the Access Type Parts.

When a user adds a monitored item that the user is denied read access to, the add operation for the item shall succeed and the bad status *Bad_NotReadable* or *Bad_UserAccessDenied* shall be returned in the Publish response. This is the same behaviour for the case where the access rights are changed after the call to *CreateMonitoredItems*. If the access rights change to read rights, the *Server* shall start sending data for the *MonitoredItem*. The same procedure shall be applied for an *IndexRange* that does not deliver data for the current value but could deliver data in the future.

Monitored *Nodes* can be removed from the *AddressSpace* after the creation of a *MonitoredItem*. This does not affect the validity of the *MonitoredItem* but a *Bad_NodeIdUnknown* shall be returned in the Publish response.

The return diagnostic info setting in the request header of the *CreateMonitoredItems* or the last *ModifyMonitoredItems Service* is applied to the *Monitored Items* and is used as the diagnostic information settings when sending Notifications in the *Publish* response.

Illegal request values for parameters that can be revised do not generate errors. Instead the server will choose default values and indicate them in the corresponding revised parameter.

It is strongly recommended by OPC UA that a *Client* reuses a *Subscription* after a short network interruption by activating the existing *Session* on a new *SecureChannel* as described in 6.5. If a *Client* called *CreateMonitoredItems* during the network interruption and the call succeeded in the *Server* but did not return to the *Client*, then the *Client* does not know if the call succeeded. The *Client* may receive data changes for these monitored items but is not able to remove them since it does not know the *Server* handle for each monitored item. There is also no way for the *Client* to detect if the create succeeded. To delete and recreate the *Subscription* is also not an option since there may be several monitored items operating normally that should not be interrupted. To resolve this situation, the *Server Object* provides a *Method* *GetMonitoredItems* that returns the list of server and client handles for the monitored items in a *Subscription*. This *Method* is defined in IEC 62541-5.

5.12.2.2 Parameters

Table 64 defines the parameters for the *Service*.

Table 64 – CreateMonitoredItems Service Parameters

Name	Type	Description
Request		
requestHeader	RequestHeader	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).
subscriptionId	IntegerId	The <i>Server</i> -assigned identifier for the <i>Subscription</i> that will report <i>Notifications</i> for this <i>MonitoredItem</i> (see 7.13 for <i>IntegerId</i> definition).
timestampsToReturn	Enum Timestamps ToReturn	An enumeration that specifies the timestamp <i>Attributes</i> to be transmitted for each <i>MonitoredItem</i> . The <i>TimestampsToReturn</i> enumeration is defined in 7.34. When monitoring <i>Events</i> , this applies only to <i>Event</i> fields that are of type <i>DataValue</i> .
itemsToCreate []	MonitoredItem CreateRequest	A list of <i>MonitoredItems</i> to be created and assigned to the specified <i>Subscription</i> . This structure is defined in-line with the following indented items.
itemToMonitor	ReadValued	Identifies an item in the <i>AddressSpace</i> to monitor. To monitor for <i>Events</i> , the <i>attributedId</i> element of the <i>ReadValued</i> structure is the id of the <i>EventNotifierAttribute</i> . The <i>ReadValued</i> type is defined in 7.23.
monitoringMode	Enum MonitoringMode	The monitoring mode to be set for the <i>MonitoredItem</i> . The <i>MonitoringMode</i> enumeration is defined in 7.17.
requestedParameters	Monitoring Parameters	The requested monitoring parameters. <i>Servers</i> negotiate the values of these parameters based on the <i>Subscription</i> and the capabilities of the <i>Server</i> . The <i>MonitoringParameters</i> type is defined in 7.15.
Response		
responseHeader	Response Header	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).
results []	MonitoredItem CreateResult	List of results for the <i>MonitoredItems</i> to create. The size and order of the list matches the size and order of the <i>itemsToCreate</i> request parameter. This structure is defined in-line with the following indented items.
statusCode	StatusCode	<i>StatusCode</i> for the <i>MonitoredItem</i> to create (see 7.33 for <i>StatusCode</i> definition).
monitoredItemId	IntegerId	<i>Server</i> -assigned id for the <i>MonitoredItem</i> (see 7.13 for <i>IntegerId</i> definition). This id is unique within the <i>Subscription</i> , but might not be unique within the <i>Server</i> or <i>Session</i> . This parameter is present only if the <i>statusCode</i> indicates that the <i>MonitoredItem</i> was successfully created.
revisedSamplingInterval	Duration	The actual sampling interval that the <i>Server</i> will use. This value is based on a number of factors, including capabilities of the underlying system. The <i>Server</i> shall always return a <i>revisedSamplingInterval</i> that is equal or higher than the requested <i>samplingInterval</i> . If the requested <i>samplingInterval</i> is higher than the maximum sampling interval supported by the <i>Server</i> , the maximum sampling interval is returned.
revisedQueueSize	Counter	The actual queue size that the <i>Server</i> will use.
filterResult	Extensible Parameter MonitoringFilter Result	Contains any revised parameter values or error results associated with the <i>MonitoringFilter</i> specified in <i>requestedParameters</i> . This parameter may be omitted if no errors occurred. The <i>MonitoringFilterResult</i> parameter type is an extensible parameter type specified in 7.16.
diagnosticInfos []	DiagnosticInfo	List of diagnostic information for the <i>MonitoredItems</i> to create (see 7.8 for <i>DiagnosticInfo</i> definition). The size and order of the list matches the size and order of the <i>itemsToCreate</i> request parameter. This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the request.

5.12.2.3 Service results

Table 65 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 65 – CreateMonitoredItems Service Result Codes

Symbolic Id	Description
Bad_NothingToDo	See Table 165 for the description of this result code.
Bad_TooManyOperations	See Table 165 for the description of this result code.
Bad_TimestampsToReturnInvalid	See Table 165 for the description of this result code.
Bad_SubscriptionIdInvalid	See Table 165 for the description of this result code.

5.12.2.4 StatusCodes

Table 66 defines values for the operation level *statusCode* parameter that are specific to this *Service*. Common *StatusCodes* are defined in Table 166.

Table 66 – CreateMonitoredItems Operation Level Result Codes

Symbolic Id	Description
Bad_MonitoringModelInvalid	See Table 166 for the description of this result code.
Bad_NodeIdInvalid	See Table 166 for the description of this result code.
Bad_NodeIdUnknown	See Table 166 for the description of this result code.
Bad_AttributeIdInvalid	See Table 166 for the description of this result code.
Bad_IndexRangeInvalid	See Table 166 for the description of this result code.
Bad_IndexRangeNoData	See Table 166 for the description of this result code. If the <i>ArrayDimensions</i> have a fixed length that can not change and no data exists within the range of indexes specified, <i>Bad_IndexRangeNoData</i> is returned in <i>CreateMonitoredItems</i> . Otherwise if the length of the array is dynamic, the <i>Server</i> shall return this status in a <i>Publish</i> response for the <i>MonitoredItem</i> if no data exists within the range.
Bad_DataEncodingInvalid	See Table 166 for the description of this result code.
Bad_DataEncodingUnsupported	See Table 166 for the description of this result code.
Bad_MonitoredItemFilterInvalid	See Table 166 for the description of this result code.
Bad_MonitoredItemFilterUnsupported	See Table 166 for the description of this result code.
Bad_FilterNotAllowed	See Table 165 for the description of this result code.
Bad_TooManyMonitoredItems	The <i>Server</i> has reached its maximum number of monitored items.

5.12.3 ModifyMonitoredItems

5.12.3.1 Description

This *Service* is used to modify *MonitoredItems* of a *Subscription*. Changes to the *MonitoredItem* settings shall be applied immediately by the *Server*. They take effect as soon as practical but not later than twice the new *revisedSamplingInterval*.

The return diagnostic info setting in the request header of the *CreateMonitoredItems* or the last *ModifyMonitoredItems* *Service* is applied to the *Monitored Items* and is used as the diagnostic information settings when sending Notifications in the *Publish* response.

Illegal request values for parameters that can be revised do not generate errors. Instead the server will choose default values and indicate them in the corresponding revised parameter.

5.12.3.2 Parameters

Table 67 defines the parameters for the *Service*.

Table 67 – ModifyMonitoredItems Service Parameters

Name	Type	Description
Request		
requestHeader	RequestHeader	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).
subscriptionId	IntegerId	The <i>Server</i> -assigned identifier for the <i>Subscription</i> used to qualify the <i>monitoredItemId</i> (see 7.13 for <i>IntegerId</i> definition).
timestampsToReturn	Enum Timestamps ToReturn	An enumeration that specifies the timestamp <i>Attributes</i> to be transmitted for each <i>MonitoredItem</i> to be modified. The <i>TimestampsToReturn</i> enumeration is defined in 7.34. When monitoring <i>Events</i> , this applies only to <i>Event</i> fields that are of type <i>DataValue</i> .
itemsToModify []	MonitoredItemMo difyRequest	The list of <i>MonitoredItems</i> to modify. This structure is defined in-line with the following indented items.
monitoredItemId	IntegerId	<i>Server</i> -assigned id for the <i>MonitoredItem</i> .
requestedParameters	Monitoring Parameters	The requested values for the monitoring parameters. The <i>MonitoringParameters</i> type is defined in 7.15. If the number of notifications in the queue exceeds the new queue size, the notifications exceeding the size shall be discarded following the configured discard policy.
Response		
responseHeader	Response Header	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).
results []	MonitoredItemMo difyResult	List of results for the <i>MonitoredItems</i> to modify. The size and order of the list matches the size and order of the <i>itemsToModify</i> request parameter. This structure is defined in-line with the following indented items.
statusCode	StatusCode	<i>StatusCode</i> for the <i>MonitoredItem</i> to be modified (see 7.33 for <i>StatusCode</i> definition).
revisedSampling Interval	Duration	The actual sampling interval that the <i>Server</i> will use. The <i>Server</i> returns the value it will actually use for the sampling interval. This value is based on a number of factors, including capabilities of the underlying system. The <i>Server</i> shall always return a <i>revisedSamplingInterval</i> that is equal or higher than the requested <i>samplingInterval</i> . If the requested <i>samplingInterval</i> is higher than the maximum sampling interval supported by the <i>Server</i> , the maximum sampling interval is returned.
revisedQueueSize	Counter	The actual queue size that the <i>Server</i> will use.
filterResult	Extensible Parameter MonitoringFilterR esult	Contains any revised parameter values or error results associated with the <i>MonitoringFilter</i> specified in the request. This parameter may be omitted if no errors occurred. The <i>MonitoringFilterResult</i> parameter type is an extensible parameter type specified in 7.16.
diagnosticInfos []	DiagnosticInfo	List of diagnostic information for the <i>MonitoredItems</i> to modify (see 7.8 for <i>DiagnosticInfo</i> definition). The size and order of the list matches the size and order of the <i>itemsToModify</i> request parameter. This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the request.

5.12.3.3 Service results

Table 68 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 68 – ModifyMonitoredItems Service Result Codes

Symbolic Id	Description
Bad_NothingToDo	See Table 165 for the description of this result code.
Bad_TooManyOperations	See Table 165 for the description of this result code.
Bad_TimestampsToReturnInvalid	See Table 165 for the description of this result code.
Bad_SubscriptionIdInvalid	See Table 165 for the description of this result code.

5.12.3.4 StatusCodes

Table 69 defines values for the operation level *statusCode* parameter that are specific to this *Service*. Common *StatusCodes* are defined in Table 166.

Table 69 – ModifyMonitoredItems Operation Level Result Codes

Symbolic Id	Description
Bad_MonitoredItemIdInvalid	See Table 166 for the description of this result code.
Bad_MonitoredItemFilterInvalid	See Table 166 for the description of this result code.
Bad_MonitoredItemFilterUnsupported	See Table 166 for the description of this result code.
Bad_FilterNotAllowed	See Table 165 for the description of this result code.

5.12.4 SetMonitoringMode

5.12.4.1 Description

This *Service* is used to set the monitoring mode for one or more *MonitoredItems* of a *Subscription*. Setting the mode to DISABLED causes all queued *Notifications* to be deleted.

5.12.4.2 Parameters

Table 70 defines the parameters for the *Service*.

Table 70 – SetMonitoringMode Service Parameters

Name	Type	Description
Request		
requestHeader	RequestHeader	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).
subscriptionId	IntegerId	The <i>Server</i> -assigned identifier for the <i>Subscription</i> used to qualify the <i>monitoredItemIds</i> (see 7.13 for <i>IntegerId</i> definition).
monitoringMode	Enum MonitoringMode	The monitoring mode to be set for the <i>MonitoredItems</i> . The <i>MonitoringMode</i> enumeration is defined in 7.17.
monitoredItemIds []	IntegerId	List of <i>Server</i> -assigned ids for the <i>MonitoredItems</i> .
Response		
responseHeader	Response Header	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).
results []	StatusCode	List of <i>StatusCodes</i> for the <i>MonitoredItems</i> to enable/disable (see 7.33 for <i>StatusCode</i> definition). The size and order of the list matches the size and order of the <i>monitoredItemIds</i> request parameter.
diagnosticInfos []	DiagnosticInfo	List of diagnostic information for the <i>MonitoredItems</i> to enable/disable (see 7.8 for <i>DiagnosticInfo</i> definition). The size and order of the list matches the size and order of the <i>monitoredItemIds</i> request parameter. This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the request.

5.12.4.3 Service results

Table 71 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 71 – SetMonitoringMode Service Result Codes

Symbolic Id	Description
Bad_NothingToDo	See Table 165 for the description of this result code.
Bad_TooManyOperations	See Table 165 for the description of this result code.
Bad_SubscriptionIdInvalid	See Table 165 for the description of this result code.
Bad_MonitoringModeInvalid	See Table 166 for the description of this result code.

5.12.4.4 StatusCodes

Table 72 defines values for the operation level *statusCode* parameter that are specific to this *Service*. Common *StatusCodes* are defined in Table 166.

Table 72 – SetMonitoringMode Operation Level Result Codes

Value	Description
Bad_MonitoredItemIdInvalid	See Table 166 for the description of this result code.

5.12.5 SetTriggering

5.12.5.1 Description

This *Service* is used to create and delete triggering links for a triggering item. The triggering item and the items to report shall belong to the same *Subscription*.

Each triggering link links a triggering item to an item to report. Each link is represented by the *MonitoredItem* id for the item to report. An error code is returned if this id is invalid.

See 5.12.1.6 for a description of the triggering model.

5.12.5.2 Parameters

Table 73 defines the parameters for the *Service*.

Table 73 – SetTriggering Service Parameters

Name	Type	Description
Request		
requestHeader	Request Header	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).
subscriptionId	IntegerId	The <i>Server</i> -assigned identifier for the <i>Subscription</i> that contains the triggering item and the items to report (see 7.13 for <i>IntegerId</i> definition).
triggeringItemId	IntegerId	<i>Server</i> -assigned id for the <i>MonitoredItem</i> used as the triggering item.
linksToAdd []	IntegerId	The list of <i>Server</i> -assigned ids of the items to report that are to be added as triggering links. The list of <i>linksToRemove</i> is processed before the <i>linksToAdd</i> .
linksToRemove []	IntegerId	The list of <i>Server</i> -assigned ids of the items to report for the triggering links to be deleted. The list of <i>linksToRemove</i> is processed before the <i>linksToAdd</i> .
Response		
responseHeader	Response Header	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).
addResults []	StatusCode	List of <i>StatusCodes</i> for the items to add (see 7.33 for <i>StatusCode</i> definition). The size and order of the list matches the size and order of the <i>linksToAdd</i> parameter specified in the request.
addDiagnosticInfos []	Diagnostic Info	List of diagnostic information for the links to add (see 7.8 for <i>DiagnosticInfo</i> definition). The size and order of the list matches the size and order of the <i>linksToAdd</i> request parameter. This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the request.
removeResults []	StatusCode	List of <i>StatusCodes</i> for the items to delete. The size and order of the list matches the size and order of the <i>linksToRemove</i> parameter specified in the request.
removeDiagnosticInfos []	Diagnostic Info	List of diagnostic information for the links to delete. The size and order of the list matches the size and order of the <i>linksToRemove</i> request parameter. This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the request.

5.12.5.3 Service results

Table 74 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in 7.33.

Table 74 – SetTriggering Service Result Codes

Symbolic Id	Description
Bad_NothingToDo	See Table 165 for the description of this result code.
Bad_TooManyOperations	See Table 165 for the description of this result code.
Bad_SubscriptionIdInvalid	See Table 165 for the description of this result code.
Bad_MonitoredItemIdInvalid	See Table 166 for the description of this result code.

5.12.5.4 StatusCodes

Table 75 defines values for the results parameters that are specific to this *Service*. Common *StatusCodes* are defined in Table 166.

Table 75 – SetTriggering Operation Level Result Codes

Symbolic Id	Description
Bad_MonitoredItemIdInvalid	See Table 166 for the description of this result code.

5.12.6 DeleteMonitoredItems

5.12.6.1 Description

This *Service* is used to remove one or more *MonitoredItems* of a *Subscription*. When a *MonitoredItem* is deleted, its triggered item links are also deleted.

Successful removal of a *MonitoredItem*, however, might not remove *Notifications* for the *MonitoredItem* that are in the process of being sent by the *Subscription*. Therefore, *Clients* may receive *Notifications* for the *MonitoredItem* after they have received a positive response that the *MonitoredItem* has been deleted.

5.12.6.2 Parameters

Table 76 defines the parameters for the *Service*.

Table 76 – DeleteMonitoredItems Service Parameters

Name	Type	Description
Request		
requestHeader	RequestHeader	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).
subscriptionId	IntegerId	The <i>Server</i> -assigned identifier for the <i>Subscription</i> that contains the <i>MonitoredItems</i> to be deleted (see 7.13 for <i>IntegerId</i> definition).
monitoredItemIds []	IntegerId	List of <i>Server</i> -assigned ids for the <i>MonitoredItems</i> to be deleted.
Response		
responseHeader	Response Header	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).
results []	StatusCode	List of <i>StatusCodes</i> for the <i>MonitoredItems</i> to delete (see 7.33 for <i>StatusCode</i> definition). The size and order of the list matches the size and order of the <i>monitoredItemIds</i> request parameter.
diagnosticInfos []	DiagnosticInfo	List of diagnostic information for the <i>MonitoredItems</i> to delete (see 7.8 for <i>DiagnosticInfo</i> definition). The size and order of the list matches the size and order of the <i>monitoredItemIds</i> request parameter. This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the request.

5.12.6.3 Service results

Table 77 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 77 – DeleteMonitoredItems Service Result Codes

Symbolic Id	Description
Bad_NothingToDo	See Table 165 for the description of this result code.
Bad_TooManyOperations	See Table 165 for the description of this result code.
Bad_SubscriptionIdInvalid	See Table 165 for the description of this result code.

5.12.6.4 StatusCodes

Table 78 defines values for the *results* parameter that are specific to this *Service*. Common *StatusCodes* are defined in Table 166.

Table 78 – DeleteMonitoredItems Operation Level Result Codes

Symbolic Id	Description
Bad_MonitoredItemIdInvalid	See Table 166 for the description of this result code.

5.13 Subscription Service Set

5.13.1 Subscription model

5.13.1.1 Description

Subscriptions are used to report *Notifications* to the *Client*. Their general behaviour is summarized below. Their precise behaviour is described in 5.13.1.2.

- Subscriptions* have a set of *MonitoredItems* assigned to them by the *Client*. *MonitoredItems* generate *Notifications* that are to be reported to the *Client* by the *Subscription* (see 5.12.1 for a description of *MonitoredItems*).
- Subscriptions* have a publishing interval. The publishing interval of a *Subscription* defines the cyclic rate at which the *Subscription* executes. Each time it executes, it attempts to send a *NotificationMessage* to the *Client*. *NotificationMessages* contain *Notifications* that have not yet been reported to *Client*.
- NotificationMessages* are sent to the *Client* in response to *Publish* requests. *Publish* requests are normally queued to the *Session* as they are received, and one is de-queued and processed by a subscription related to this *Session* for each publishing cycle, if there are *Notifications* to report. When there are not, the *Publish* request is not de-queued from the *Session*, and the *Server* waits until the next cycle and checks again for *Notifications*.
- At the beginning of a cycle, if there are *Notifications* to send but there are no *Publish* requests queued, the *Server* enters a wait state for a *Publish* request to be received. When one is received, it is processed immediately without waiting for the next publishing cycle.
- NotificationMessages* are uniquely identified by sequence numbers that enable *Clients* to detect missed *Messages*. The publishing interval also defines the default sampling interval for its *MonitoredItems*.
- Subscriptions* have a keep-alive counter that counts the number of consecutive publishing cycles in which there have been no *Notifications* to report to the *Client*. When the maximum keep-alive count is reached, a *Publish* request is de-queued and used to return a keep-alive *Message*. This keep-alive *Message* informs the *Client* that the *Subscription* is still active. Each keep-alive *Message* is a response to a *Publish* request in which the *notificationMessage* parameter does not contain any *Notifications* and that contains the sequence number of the next *NotificationMessage* that is to be sent. In the clauses that follow, the term *NotificationMessage* refers to a response to a *Publish* request in which the *notificationMessage* parameter actually contains one or more *Notifications*, as opposed to a keep-alive *Message* in which this parameter contains no *Notifications*. The maximum keep-alive count is set by the *Client* during *Subscription* creation and may be subsequently modified using the *ModifySubscription Service*. Similar to *Notification* processing described in (c) above, if there are no *Publish* requests queued, the *Server* waits for the next one to be received and sends the keep-alive immediately without waiting for the next publishing cycle.

- g) Publishing by a *Subscription* may be enabled or disabled by the *Client* when created, or subsequently using the *SetPublishingMode Service*. Disabling causes the *Subscription* to cease sending *NotificationMessages* to the *Client*. However, the *Subscription* continues to execute cyclically and continues to send keep-alive *Messages* to the *Client*.
- h) *Subscriptions* have a lifetime counter that counts the number of consecutive publishing cycles in which there have been no *Publish* requests available to send a *Publish* response for the *Subscription*. Any *Service* call that uses the *SubscriptionId* or the processing of a *Publish* response resets the lifetime counter of this *Subscription*. When this counter reaches the value calculated for the lifetime of a *Subscription* based on the *MaxKeepAliveCount* parameter in the *CreateSubscription Service* (5.13.2), the *Subscription* is closed. Closing the *Subscription* causes its *MonitoredItems* to be deleted. In addition the *Server* shall issue a *StatusChangeNotification notificationMessage* with the status code *Bad_Timeout*. The *StatusChangeNotification notificationMessage* type is defined in 7.19.4.
- i) *Sessions* maintain a retransmission queue of sent *NotificationMessages*. *NotificationMessages* are retained in this queue until they are acknowledged. The *Session* shall maintain a retransmission queue size of at least two times the number of *Publish* requests per *Session* the *Server* supports. The minimum size of the retransmission queue may be changed by a *Profile* in IEC 62541-7. The minimum number of *Publish* requests per *Session* the *Server* shall support is defined in IEC 62541-7. *Clients* are required to acknowledge *NotificationMessages* as they are received. In the case of a retransmission queue overflow, the oldest sent *NotificationMessage* gets deleted. If a *Subscription* is transferred to another *Session*, the queued *NotificationMessages* for this *Subscription* are moved from the old to the new *Session*.

The sequence number is an unsigned 32-bit integer that is incremented by one for each *NotificationMessage* sent. The value 0 is never used for the sequence number. The first *NotificationMessage* sent on a *Subscription* has a sequence number of 1. If the sequence number rolls over, it rolls over to 1.

When a *Subscription* is created, the first *Message* is sent at the end of the first publishing cycle to inform the *Client* that the *Subscription* is operational. A *NotificationMessage* is sent if there are *Notifications* ready to be reported. If there are none, a keep-alive *Message* is sent instead that contains a sequence number of 1, indicating that the first *NotificationMessage* has not yet been sent. This is the only time a keep-alive *Message* is sent without waiting for the maximum keep-alive count to be reached, as specified in (f) above.

The value of the sequence number is never reset during the lifetime of a *Subscription*. Therefore, the same sequence number shall not be reused on a *Subscription* until over four billion *NotificationMessages* have been sent. At a continuous rate of one thousand *NotificationMessages* per second on a given *Subscription*, it would take roughly fifty days for the same sequence number to be reused. This allows *Clients* to safely treat sequence numbers as unique.

Sequence numbers are also used by *Clients* to acknowledge the receipt of *NotificationMessages*. *Publish* requests allow the *Client* to acknowledge all *Notifications* up to a specific sequence number and to acknowledge the sequence number of the last *NotificationMessage* received. One or more gaps may exist in between. Acknowledgements allow the *Server* to delete *NotificationMessages* from its retransmission queue.

Clients may ask for retransmission of selected *NotificationMessages* using the *Republish Service*. This *Service* returns the requested *Message*.

5.13.1.2 State table

The state table formally describes the operation of the *Subscription*. The following model of operations is described by this state table. This description applies when publishing is enabled or disabled for the *Subscription*.

After creation of the *Subscription*, the *Server* starts the publishing timer and restarts it whenever it expires. If the timer expires the number of times defined for the *Subscription* lifetime without having received a *Subscription Service* request from the *Client*, the *Subscription* assumes that the *Client* is no longer present, and terminates.

Clients send *Publish* requests to *Servers* to receive *Notifications*. *Publish* requests are not directed to any one *Subscription* and, therefore, may be used by any *Subscription*. Each contains acknowledgements for one or more *Subscriptions*. These acknowledgements are processed when the *Publish* request is received. The *Server* then queues the request in a queue shared by all *Subscriptions*, except in the following cases.

- a) The previous *Publish* response indicated that there were still more *Notifications* ready to be transferred and there were no more *Publish* requests queued to transfer them.
- b) The publishing timer of a *Subscription* expired and there were either *Notifications* to be sent or a keep-alive *Message* to be sent.

In these cases, the newly received *Publish* request is processed immediately by the first *Subscription* to encounter either case (a) or case (b).

Each time the publishing timer expires, it is immediately reset. If there are *Notifications* or a keep-alive *Message* to be sent, it de-queues and processes a *Publish* request. When a *Subscription* processes a *Publish* request, it accesses the queues of its *MonitoredItems* and de-queues its *Notifications*, if any. It returns these *Notifications* in the response, setting the *moreNotifications* flag if it was not able to return all available *Notifications* in the response.

If there were *Notifications* or a keep-alive *Message* to be sent but there were no *Publish* requests queued, the *Subscription* assumes that the *Publish* request is late and waits for the next *Publish* request to be received, as described in case (b).

If the *Subscription* is disabled when the publishing timer expires or if there are no *Notifications* available, it enters the keep-alive state and sets the keep-alive counter to its maximum value as defined for the *Subscription*.

While in the keep-alive state, it checks for *Notifications* each time the publishing timer expires. If one or more *Notifications* have been generated, a *Publish* request is de-queued and a *NotificationMessage* is returned in the response. However, if the publishing timer expires without a *Notification* becoming available, a *Publish* request is de-queued and a keep-alive *Message* is returned in the response. The *Subscription* then returns to the normal state of waiting for the publishing timer to expire again. If, in either of these cases, there are no *Publish* requests queued, the *Subscription* waits for the next *Publish* request to be received, as described in case (b).

The *Subscription* states are defined in Table 79.

Table 79 – Subscription States

State	Description
CLOSED	The <i>Subscription</i> has not yet been created or has terminated.
CREATING	The <i>Subscription</i> is being created.
NORMAL	The <i>Subscription</i> is cyclically checking for <i>Notifications</i> from its <i>MonitoredItems</i> . The keep-alive counter is not used in this state.
LATE	The publishing timer has expired and there are <i>Notifications</i> available or a keep-alive <i>Message</i> is ready to be sent, but there are no <i>Publish</i> requests queued. When in this state, the next <i>Publish</i> request is processed when it is received. The keep-alive counter is not used in this state.
KEEPAIVE	The <i>Subscription</i> is cyclically checking for <i>Notifications</i> from its <i>MonitoredItems</i> or for the keep-alive counter to count down to 0 from its maximum.

The state table is described in Table 80. The following rules and conventions apply.

- a) *Events* represent the receipt of *Service* requests and the occurrence internal *Events*, such as timer expirations.

- b) *Service* requests *Events* may be accompanied by conditions that test *Service* parameter values. Parameter names begin with a lower case letter.
- c) Internal *Events* may be accompanied by conditions that test state *Variable* values. State *Variables* are defined in 5.13.1.3. They begin with an upper case letter.
- d) *Service* request and internal *Events* may be accompanied by conditions represented by functions whose return value is tested. Functions are identified by “()” after their name. They are described in 5.13.1.4.
- e) When an *Event* is received, the first transition for the current state is located and the transitions are searched sequentially for the first transition that meets the *Event* or conditions criteria. If none are found, the *Event* is ignored.
- f) Actions are described by functions and state *Variable* manipulations.
- g) The LifetimeTimerExpires *Event* is triggered when its corresponding counter reaches zero.

Table 80 – Subscription State Table

#	Current State	Event/Conditions	Action	Next State
1	CLOSED	Receive CreateSubscription Request	CreateSubscription()	CREATING
2	CREATING	CreateSubscription fails	ReturnNegativeResponse()	CLOSED
3	CREATING	CreateSubscription succeeds	InitializeSubscription() MessageSent = FALSE ReturnResponse()	NORMAL
4	NORMAL	Receive <i>Publish</i> Request && (PublishingEnabled == FALSE (PublishingEnabled == TRUE && MoreNotifications == FALSE))	DeleteAkedNotificationMsgs() EnqueuePublishingReq()	NORMAL
5	NORMAL	Receive <i>Publish</i> Request && PublishingEnabled == TRUE && MoreNotifications == TRUE	ResetLifetimeCounter() DeleteAkedNotificationMsgs() ReturnNotifications() MessageSent = TRUE	NORMAL
6	NORMAL	PublishingTimer Expires && PublishingReqQueued == TRUE && PublishingEnabled == TRUE && NotificationsAvailable == TRUE	ResetLifetimeCounter() StartPublishingTimer() DequeuePublishReq() ReturnNotifications() MessageSent == TRUE	NORMAL
7	NORMAL	PublishingTimer Expires && PublishingReqQueued == TRUE && MessageSent == FALSE && (PublishingEnabled == FALSE (PublishingEnabled == TRUE && NotificationsAvailable == FALSE))	ResetLifetimeCounter() StartPublishingTimer() DequeuePublishReq() ReturnKeepAlive() MessageSent == TRUE	NORMAL
8	NORMAL	PublishingTimer Expires && PublishingReqQueued == FALSE && (MessageSent == FALSE (PublishingEnabled == TRUE && NotificationsAvailable == TRUE))	StartPublishingTimer()	LATE
9	NORMAL	PublishingTimer Expires && MessageSent == TRUE && (PublishingEnabled == FALSE (PublishingEnabled == TRUE && NotificationsAvailable == FALSE))	StartPublishingTimer() ResetKeepAliveCounter()	KEEPALIVE

#	Current State	Event/Conditions	Action	Next State
		)		
10	LATE	Receive <i>Publish</i> Request && PublishingEnabled == TRUE && (NotificationsAvailable == TRUE MoreNotifications == TRUE)	ResetLifetimeCounter() DeleteAkedNotificationMsgs() ReturnNotifications() MessageSent = TRUE	NORMAL
11	LATE	Receive <i>Publish</i> Request && (PublishingEnabled == FALSE (PublishingEnabled == TRUE && NotificationsAvailable == FALSE && MoreNotifications == FALSE))	ResetLifetimeCounter() DeleteAkedNotificationMsgs() ReturnKeepAlive() MessageSent = TRUE	KEEPALIVE
12	LATE	PublishingTimer Expires	StartPublishingTimer()	LATE
13	KEEPALIVE	Receive <i>Publish</i> Request	DeleteAkedNotificationMsgs() EnqueuePublishingReq()	KEEPALIVE
14	KEEPALIVE	PublishingTimer Expires && PublishingEnabled == TRUE && NotificationsAvailable == TRUE && PublishingReqQueued == TRUE	ResetLifetimeCounter() StartPublishingTimer() DequeuePublishReq() ReturnNotifications() MessageSent == TRUE	NORMAL
15	KEEPALIVE	PublishingTimer Expires && PublishingReqQueued == TRUE && KeepAliveCounter == 1 && (PublishingEnabled == FALSE (PublishingEnabled == TRUE && NotificationsAvailable == FALSE))	StartPublishingTimer() DequeuePublishReq() ReturnKeepAlive() ResetKeepAliveCounter()	KEEPALIVE
16	KEEPALIVE	PublishingTimer Expires && KeepAliveCounter > 1 && (PublishingEnabled == FALSE (PublishingEnabled == TRUE && NotificationsAvailable == FALSE))	StartPublishingTimer() KeepAliveCounter--	KEEPALIVE
17	KEEPALIVE	PublishingTimer Expires && PublishingReqQueued == FALSE && (KeepAliveCounter == 1 (KeepAliveCounter > 1 && PublishingEnabled == TRUE && NotificationsAvailable == TRUE))	StartPublishingTimer()	LATE
18	NORMAL LATE KEEPALIVE	Receive ModifySubscription Request	ResetLifetimeCounter() UpdateSubscriptionParams() ReturnResponse()	SAME
19	NORMAL LATE KEEPALIVE	Receive SetPublishingMode Request	ResetLifetimeCounter() SetPublishingEnabled() MoreNotifications = FALSE ReturnResponse()	SAME
20	NORMAL LATE KEEPALIVE	Receive Republish Request && RequestedMessageFound == TRUE	ResetLifetimeCounter() ReturnResponse()	SAME
21	NORMAL LATE KEEPALIVE	Receive Republish Request && RequestedMessageFound == FALSE	ResetLifetimeCounter() ReturnNegativeResponse()	SAME
22	NORMAL LATE KEEPALIVE	Receive TransferSubscriptions Request && SessionChanged() == FALSE	ResetLifetimeCounter() ReturnNegativeResponse ()	SAME

#	Current State	Event/Conditions	Action	Next State
23	NORMAL LATE KEEPALIVE	Receive TransferSubscriptions Request && SessionChanged() == TRUE && ClientValidated() ==TRUE	SetSession() ResetLifetimeCounter() ReturnResponse() IssueStatusChangeNotification()	SAME
24	NORMAL LATE KEEPALIVE	Receive TransferSubscriptions Request && SessionChanged() == TRUE && ClientValidated() == FALSE	ReturnNegativeResponse()	SAME
25	NORMAL LATE KEEPALIVE	Receive DeleteSubscriptions Request && SubscriptionAssignedToClient ==TRUE	DeleteMonitoredItems() DeleteClientPublReqQueue()	CLOSED
26	NORMAL LATE KEEPALIVE	Receive DeleteSubscriptions Request && SubscriptionAssignedToClient ==FALSE	ResetLifetimeCounter() ReturnNegativeResponse()	SAME
27	NORMAL LATE KEEPALIVE	LifetimeCounter == 1 The LifetimeCounter is decremented if PublishingTimer expires and PublishingReqQueued == FALSE The LifetimeCounter is reset if PublishingReqQueued == TRUE.	DeleteMonitoredItems() IssueStatusChangeNotification()	CLOSED

5.13.1.3 State Variables and parameters

The state *Variables* are defined alphabetically in Table 81.

Table 81 – State variables and parameters

State Variable	Description
MoreNotifications	A boolean value that is set to TRUE only by the CreateNotificationMsg() when there were too many <i>Notifications</i> for a single <i>NotificationMessage</i> .
LatePublishRequest	A boolean value that is set to TRUE to reflect that, the last time the publishing timer expired, there were no <i>Publish</i> requests queued.
LifetimeCounter	A value that contains the number of consecutive publishing timer expirations without <i>Client</i> activity before the <i>Subscription</i> is terminated.
MessageSent	A boolean value that is set to TRUE to mean that either a <i>NotificationMessage</i> or a keep-alive <i>Message</i> has been sent on the <i>Subscription</i> . It is a flag that is used to ensure that either a <i>NotificationMessage</i> or a keep-alive <i>Message</i> is sent out the first time the publishing timer expires.
NotificationsAvailable	A boolean value that is set to TRUE only when there is at least one <i>MonitoredItem</i> that is in the reporting mode and that has a <i>Notification</i> queued or there is at least one item to report whose triggering item has triggered and that has a <i>Notification</i> queued. The transition of this state <i>Variable</i> from FALSE to TRUE creates the “New <i>Notification</i> Queued” <i>Event</i> in the state table.
PublishingEnabled	The parameter that requests publishing to be enabled or disabled.
PublishingReqQueued	A boolean value that is set to TRUE only when there is a <i>Publish</i> request <i>Message</i> queued to the <i>Subscription</i> .
RequestedMessageFound	A boolean value that is set to TRUE only when the <i>Message</i> requested to be retransmitted was found in the retransmission queue.
SeqNum	The value that records the value of the sequence number used in <i>NotificationMessages</i> .
SubscriptionAssignedToClient	A boolean value that is set to TRUE only when the <i>Subscription</i> requested to be deleted is assigned to the <i>Client</i> that issued the request. A <i>Subscription</i> is assigned to the <i>Client</i> that created it. That assignment can only be changed through successful completion of the <i>TransferSubscriptions Service</i> .

5.13.1.4 Functions

The action functions are defined alphabetically in Table 82.

Table 82 – Functions

Function	Description
ClientValidated()	A boolean function that returns TRUE only when the <i>Client</i> that is submitting a TransferSubscriptions request is operating on behalf of the same user and supports the same <i>Profiles</i> as the <i>Client</i> of the previous <i>Session</i> .
CreateNotificationMsg()	Increment the SeqNum and create a <i>NotificationMessage</i> from the <i>MonitoredItems</i> assigned to the <i>Subscription</i> . Save the newly-created <i>NotificationMessage</i> in the retransmission queue. If all available <i>Notifications</i> can be sent in the <i>Publish</i> response, the MoreNotifications state <i>Variable</i> is set to FALSE. Otherwise, it is set to TRUE.
CreateSubscription()	Attempt to create the <i>Subscription</i> .
DeleteAckedNotificationMsgs()	Delete the <i>NotificationMessages</i> from the retransmission queue that were acknowledged by the request.
DeleteClientPublReqQueue()	Clear the <i>Publish</i> request queue for the <i>Client</i> that is sending the DeleteSubscriptions request, if there are no more <i>Subscriptions</i> assigned to that <i>Client</i> .
DeleteMonitoredItems()	Delete all <i>MonitoredItems</i> assigned to the <i>Subscription</i> .
DequeuePublishReq()	De-queue a publishing request in first-in first-out order. Validate if the publish request is still valid by checking the timeoutHint in the RequestHeader. If the request timed out, send a <i>Bad_Timeout</i> service result for the request and de-queue another publish request. ResetLifetimeCounter()
EnqueuePublishingReq()	Enqueue the publishing request.
InitializeSubscription()	ResetLifetimeCounter() MoreNotifications = FALSE PublishRateChange = FALSE PublishingEnabled = value of publishingEnabled parameter in the CreateSubscription request PublishingReqQueued = FALSE SeqNum = 0 SetSession() StartPublishingTimer()
IssueStatusChangeNotification()	Issue a <i>StatusChangeNotification notificationMessage</i> with a status code for the status change of the <i>Subscription</i> . The <i>StatusChangeNotification notificationMessage</i> type is defined in 7.19.4. <i>Bad_Timeout</i> status code is used if the lifetime expires and <i>Good_SubscriptionTransferred</i> is used if the <i>Subscriptions</i> was transferred to another <i>Session</i> .
ResetKeepAliveCounter()	Reset the keep-alive counter to the maximum keep-alive count of the <i>Subscription</i> . The maximum keep-alive count is set by the <i>Client</i> when the <i>Subscription</i> is created and may be modified using the <i>ModifySubscription Service</i> .
ResetLifetimeCounter()	Reset the LifetimeCounter <i>Variable</i> to the value specified for the lifetime of a <i>Subscription</i> in the <i>CreateSubscription Service</i> (5.13.2).
ReturnKeepAlive()	CreateKeepAliveMsg() ReturnResponse()
ReturnNegativeResponse ()	Return a <i>Service</i> response indicating the appropriate <i>Service</i> level error. No parameters are returned other than the responseHeader that contains the <i>Service</i> level <i>StatusCode</i> .
ReturnNotifications()	CreateNotificationMsg() ReturnResponse() If (MoreNotifications == TRUE) && (PublishingReqQueued == TRUE) { DequeuePublishReq() Loop through this function again }
ReturnResponse()	Return the appropriate response, setting the appropriate parameter values and <i>StatusCodes</i> defined for the <i>Service</i> .
SessionChanged()	A boolean function that returns TRUE only when the <i>Session</i> used to send a TransferSubscriptions request is different from the <i>Client Session</i> currently associated with the <i>Subscription</i> .
SetPublishingEnabled ()	Set the PublishingEnabled state <i>Variable</i> to the value of the publishingEnabled parameter received in the request.
SetSession	Set the <i>Session</i> information for the <i>Subscription</i> to match the <i>Session</i> on which the TransferSubscriptions request was issued.
StartPublishingTimer()	Start or restart the publishing timer and decrement the LifetimeCounter <i>Variable</i> .
UpdateSubscriptionParams()	Negotiate and update the <i>Subscription</i> parameters. If the new keep-alive interval is less than the current value of the keep-alive counter, perform ResetKeepAliveCounter() and ResetLifetimeCounter().

5.13.2 CreateSubscription

5.13.2.1 Description

This *Service* is used to create a *Subscription*. *Subscriptions* monitor a set of *MonitoredItems* for *Notifications* and return them to the *Client* in response to *Publish* requests.

Illegal request values for parameters that can be revised do not generate errors. Instead the server will choose default values and indicate them in the corresponding revised parameter.

5.13.2.2 Parameters

Table 83 defines the parameters for the *Service*.

Table 83 – CreateSubscription Service Parameters

Name	Type	Description
Request		
requestHeader	Request Header	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).
requestedPublishingInterval	Duration	This interval defines the cyclic rate that the <i>Subscription</i> is being requested to return <i>Notifications</i> to the <i>Client</i> . This interval is expressed in milliseconds. This interval is represented by the publishing timer in the <i>Subscription</i> state table (see 5.13.1.2). The negotiated value for this parameter returned in the response is used as the default sampling interval for <i>MonitoredItems</i> assigned to this <i>Subscription</i> . If the requested value is 0 or negative, the server shall revise with the fastest supported publishing interval.
requestedLifetimeCount	Counter	Requested lifetime count (see 7.5 for <i>Counter</i> definition). The lifetime count shall be a minimum of three times the keep-alive count. When the publishing timer has expired this number of times without a <i>Publish</i> request being available to send a <i>NotificationMessage</i> , then the <i>Subscription</i> shall be deleted by the <i>Server</i> .
requestedMaxKeepAliveCount	Counter	Requested maximum keep-alive count (see 7.5 for <i>Counter</i> definition). When the publishing timer has expired this number of times without requiring any <i>NotificationMessage</i> to be sent, the <i>Subscription</i> sends a keep-alive <i>Message</i> to the <i>Client</i> . The negotiated value for this parameter is returned in the response. If the requested value is 0, the server shall revise with the smallest supported keep-alive count.
maxNotificationsPerPublish	Counter	The maximum number of notifications that the <i>Client</i> wishes to receive in a single <i>Publish</i> response. A value of zero indicates that there is no limit. The number of notifications per <i>Publish</i> is the sum of monitoredItems in the <i>DataChangeNotification</i> and events in the <i>EventNotificationList</i> .
publishingEnabled	Boolean	A <i>Boolean</i> parameter with the following values: TRUE publishing is enabled for the <i>Subscription</i> . FALSE publishing is disabled for the <i>Subscription</i> . The value of this parameter does not affect the value of the monitoring mode <i>Attribute</i> of <i>MonitoredItems</i> .
priority	Byte	Indicates the relative priority of the <i>Subscription</i> . When more than one <i>Subscription</i> needs to send <i>Notifications</i> , the <i>Server</i> should de-queue a <i>Publish</i> request to the <i>Subscription</i> with the highest <i>priority</i> number. For <i>Subscriptions</i> with equal <i>priority</i> the <i>Server</i> should de-queue <i>Publish</i> requests in a round-robin fashion. A <i>Client</i> that does not require special priority settings should set this value to zero.
Response		
responseHeader	Response Header	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).
subscriptionId	IntegerId	The <i>Server</i> -assigned identifier for the <i>Subscription</i> (see 7.13 for <i>IntegerId</i> definition). This identifier shall be unique for the entire <i>Server</i> , not just for the <i>Session</i> , in order to allow the <i>Subscription</i> to be transferred to another <i>Session</i> using the <i>TransferSubscriptions</i> service.
revisedPublishingInterval	Duration	The actual publishing interval that the <i>Server</i> will use, expressed in milliseconds. The <i>Server</i> should attempt to honour the <i>Client</i> request for this parameter, but may negotiate this value up or down to meet its own constraints.
revisedLifetimeCount	Counter	The lifetime of the <i>Subscription</i> shall be a minimum of three times the keep-alive interval negotiated by the <i>Server</i> .

Name	Type	Description
Request		
revisedMaxKeepAliveCount	Counter	The actual maximum keep-alive count (see 7.5 for <i>Counter</i> definition). The <i>Server</i> should attempt to honour the <i>Client</i> request for this parameter, but may negotiate this value up or down to meet its own constraints.

5.13.2.3 Service results

Table 84 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 84 – CreateSubscription Service Result Codes

Symbolic Id	Description
Bad_TooManySubscriptions	The <i>Server</i> has reached its maximum number of subscriptions.

5.13.3 ModifySubscription

5.13.3.1 Description

This *Service* is used to modify a *Subscription*.

Illegal request values for parameters that can be revised do not generate errors. Instead the server will choose default values and indicate them in the corresponding revised parameter.

Changes to the *Subscription* settings shall be applied immediately by the *Server*. They take effect as soon as practical but not later than twice the new *revisedPublishingInterval*.

5.13.3.2 Parameters

Table 85 defines the parameters for the *Service*.

Table 85 – ModifySubscription Service Parameters

Name	Type	Description
Request		
requestHeader	RequestHeader	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).
subscriptionId	IntegerId	The <i>Server</i> -assigned identifier for the <i>Subscription</i> (see 7.13 for <i>IntegerId</i> definition).
requestedPublishingInterval	Duration	This interval defines the cyclic rate that the <i>Subscription</i> is being requested to return <i>Notifications</i> to the <i>Client</i> . This interval is expressed in milliseconds. This interval is represented by the publishing timer in the <i>Subscription</i> state table (see 5.13.1.2). The negotiated value for this parameter returned in the response is used as the default sampling interval for <i>MonitoredItems</i> assigned to this <i>Subscription</i> . If the requested value is 0 or negative, the server shall revise with the fastest supported publishing interval.
requestedLifetimeCount	Counter	Requested lifetime count (see 7.5 for <i>Counter</i> definition). The lifetime count shall be a minimum of three times the keep keep-alive count. When the publishing timer has expired this number of times without a <i>Publish</i> request being available to send a <i>NotificationMessage</i> , then the <i>Subscription</i> shall be deleted by the <i>Server</i> .
requestedMaxKeepAliveCount	Counter	Requested maximum keep-alive count (see 7.5 for <i>Counter</i> definition). When the publishing timer has expired this number of times without requiring any <i>NotificationMessage</i> to be sent, the <i>Subscription</i> sends a keep-alive <i>Message</i> to the <i>Client</i> . The negotiated value for this parameter is returned in the response. If the requested value is 0, the server shall revise with the smallest supported keep-alive count.
maxNotificationsPerPublish	Counter	The maximum number of notifications that the <i>Client</i> wishes to receive in a single <i>Publish</i> response. A value of zero indicates that there is no limit.
priority	Byte	Indicates the relative priority of the <i>Subscription</i> . When more than one <i>Subscription</i> needs to send <i>Notifications</i> , the <i>Server</i> should de-queue a <i>Publish</i> request to the <i>Subscription</i> with the highest <i>priority</i> number. For

Name	Type	Description
Request		
		<i>Subscriptions</i> with equal <i>priority</i> the <i>Server</i> should de-queue <i>Publish</i> requests in a round-robin fashion. Any <i>Subscription</i> that needs to send a keep-alive <i>Message</i> shall take precedence regardless of its <i>priority</i> , in order to prevent the <i>Subscription</i> from expiring. A <i>Client</i> that does not require special priority settings should set this value to zero.
Response		
responseHeader	ResponseHeader	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).
revisedPublishingInterval	Duration	The actual publishing interval that the <i>Server</i> will use, expressed in milliseconds. The <i>Server</i> should attempt to honour the <i>Client</i> request for this parameter, but may negotiate this value up or down to meet its own constraints.
revisedLifetimeCount	Counter	The lifetime of the <i>Subscription</i> shall be a minimum of three times the keep-alive interval negotiated by the <i>Server</i> .
revisedMaxKeepAliveCount	Counter	The actual maximum keep-alive count (see 7.5 for <i>Counter</i> definition). The <i>Server</i> should attempt to honour the <i>Client</i> request for this parameter, but may negotiate this value up or down to meet its own constraints.

5.13.3.3 Service results

Table 86 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 86 – ModifySubscription Service Result Codes

Symbolic Id	Description
Bad_SubscriptionIdInvalid	See Table 165 for the description of this result code.

5.13.4 SetPublishingMode

5.13.4.1 Description

This *Service* is used to enable sending of *Notifications* on one or more *Subscriptions*.

5.13.4.2 Parameters

Table 87 defines the parameters for the *Service*.

Table 87 – SetPublishingMode Service Parameters

Name	Type	Description
Request		
requestHeader	RequestHeader	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).
publishingEnabled	Boolean	A <i>Boolean</i> parameter with the following values: TRUE publishing of <i>NotificationMessages</i> is enabled for the <i>Subscription</i> . FALSE publishing of <i>NotificationMessages</i> is disabled for the <i>Subscription</i> . The value of this parameter does not affect the value of the monitoring mode <i>Attribute of MonitoredItems</i> . Setting this value to FALSE does not discontinue the sending of keep-alive <i>Messages</i> .
subscriptionIds []	IntegerId	List of <i>Server</i> -assigned identifiers for the <i>Subscriptions</i> to enable or disable (see 7.13 for <i>IntegerId</i> definition).
Response		
responseHeader	ResponseHeader	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).
results []	StatusCode	List of <i>StatusCodes</i> for the <i>Subscriptions</i> to enable/disable (see 7.33 for <i>StatusCode</i> definition). The size and order of the list matches the size and order of the <i>subscriptionIds</i> request parameter.
diagnosticInfos []	DiagnosticInfo	List of diagnostic information for the <i>Subscriptions</i> to enable/disable (see 7.8 for <i>DiagnosticInfo</i> definition). The size and order of the list matches the size and order of the <i>subscriptionIds</i> request parameter. This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the request.

5.13.4.3 Service results

Table 88 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 88 – SetPublishingMode Service Result Codes

Symbolic Id	Description
Bad_NothingToDo	See Table 165 for the description of this result code.
Bad_TooManyOperations	See Table 165 for the description of this result code.

5.13.4.4 StatusCodes

Table 89 defines values for the *results* parameter that are specific to this *Service*. Common *StatusCodes* are defined in Table 166.

Table 89 – SetPublishingMode Operation Level Result Codes

Symbolic Id	Description
Bad_SubscriptionIdInvalid	See Table 165 for the description of this result code.

5.13.5 Publish

5.13.5.1 Description

This *Service* is used for two purposes. First, it is used to acknowledge the receipt of *NotificationMessages* for one or more *Subscriptions*. Second, it is used to request the *Server* to return a *NotificationMessage* or a keep-alive *Message*. Since *Publish* requests are not directed to a specific *Subscription*, they may be used by any *Subscription*. 5.13.1.2 describes the use of the *Publish Service*.

Client strategies for issuing *Publish* requests may vary depending on the networking delays between the *Client* and the *Server*. In many cases, the *Client* may wish to issue a *Publish* request immediately after creating a *Subscription*, and thereafter, immediately after receiving a *Publish* response.

In other cases, especially in high latency networks, the *Client* may wish to pipeline *Publish* requests to ensure cyclic reporting from the *Server*. Pipelining involves sending more than one *Publish* request for each *Subscription* before receiving a response. For example, if the network introduces a delay between the *Client* and the *Server* of 5 seconds and the publishing interval for a *Subscription* is one second, then the *Client* will have to issue *Publish* requests every second instead of waiting for a response to be received before sending the next request.

A server should limit the number of active *Publish* requests to avoid an infinite number since it is expected that the *Publish* requests are queued in the *Server*. But a *Server* shall accept more queued *Publish* requests than created *Subscriptions*. It is expected that a *Server* supports several *Publish* requests per *Subscription*. When a *Server* receives a new *Publish* request that exceeds its limit it shall de-queue the oldest *Publish* request and return a response with the result set to *Bad_TooManyPublishRequests*. If a *Client* receives this *Service* result for a *Publish* request it shall not issue another *Publish* request before one of its outstanding *Publish* requests is returned from the *Server*.

Clients can limit the size of *Publish* responses with the *maxNotificationsPerPublish* parameter passed to the *CreateSubscription Service*. However, this could still result in a message that is too large for the *Client* or *Server* to process. In this situation, the *Client* will find that either the *SecureChannel* goes into a fault state and needs to be re-established or the *Publish* response returns an error and calling the *Republish Service* also returns an error. If either situation

occurs then the *Client* will have to adjust its message processing limits or the parameters for the *Subscription and/or MonitoredItems*.

The return diagnostic info setting in the request header of the *CreateMonitoredItems* or the last *ModifyMonitoredItems Service* is applied to the *Monitored Items* and is used as the diagnostic information settings when sending Notifications in the *Publish* response.

5.13.5.2 Parameters

Table 90 defines the parameters for the *Service*.

Table 90 – Publish Service Parameters

Name	Type	Description
Request		
requestHeader	RequestHeader	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).
subscription Acknowledgements []	Subscription Acknowledgement	The list of acknowledgements for one or more <i>Subscriptions</i> . This list may contain multiple acknowledgements for the same <i>Subscription</i> (multiple entries with the same <i>subscriptionId</i>). This structure is defined in-line with the following indented items.
subscriptionId	IntegerId	The <i>Server</i> assigned identifier for a <i>Subscription</i> (see 7.13 for <i>IntegerId</i> definition).
sequenceNumber	Counter	The sequence number being acknowledged (see 7.5 for <i>Counter</i> definition). The <i>Server</i> may delete the <i>Message</i> with this sequence number from its retransmission queue.
Response		
responseHeader	ResponseHeader	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).
subscriptionId	IntegerId	The <i>Server</i> -assigned identifier for the <i>Subscription</i> for which <i>Notifications</i> are being returned (see 7.13 for <i>IntegerId</i> definition). The value 0 is used to indicate that there were no <i>Subscriptions</i> defined for which a response could be sent.
availableSequence Numbers []	Counter	A list of sequence number ranges that identify unacknowledged <i>NotificationMessages</i> that are available for retransmission from the <i>Subscription</i> 's retransmission queue. This list is prepared after processing the acknowledgements in the request (see 7.5 for <i>Counter</i> definition).
moreNotifications	Boolean	A <i>Boolean</i> parameter with the following values: TRUE the number of <i>Notifications</i> that were ready to be sent could not be sent in a single response. FALSE all <i>Notifications</i> that were ready are included in the response.
notificationMessage	Notification Message	The <i>NotificationMessage</i> that contains the list of <i>Notifications</i> . The <i>NotificationMessage</i> parameter type is specified in 7.20.
results []	StatusCodes	List of results for the acknowledgements (see 7.33 for <i>StatusCodes</i> definition). The size and order of the list matches the size and order of the <i>subscriptionAcknowledgements</i> request parameter.
diagnosticInfos []	DiagnosticInfo	List of diagnostic information for the acknowledgements (see 7.8 for <i>DiagnosticInfo</i> definition). The size and order of the list matches the size and order of the <i>subscriptionAcknowledgements</i> request parameter. This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the request.

5.13.5.3 Service results

Table 91 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 91 – Publish Service Result Codes

Symbolic Id	Description
Bad_TooManyPublishRequests	The server has reached the maximum number of queued publish requests.
Bad_NoSubscription	There is no subscription available for this session.

5.13.5.4 StatusCodes

Table 92 defines values for the *results* parameter that are specific to this *Service*. Common *StatusCodes* are defined in Table 166.

Table 92 – Publish Operation Level Result Codes

Symbolic Id	Description
Bad_SubscriptionIdInvalid	See Table 165 for the description of this result code.
Bad_SequenceNumberUnknown	The sequence number is unknown to the server.

5.13.6 Republish

5.13.6.1 Description

This *Service* requests the *Subscription* to republish a *NotificationMessage* from its retransmission queue. If the *Server* does not have the requested *Message* in its retransmission queue, it returns an error response.

See 5.13.1.2 for the detail description of the behaviour of this *Service*.

See 6.5 for a description of the issues and strategies regarding reconnect handling and *Republish*.

5.13.6.2 Parameters

Table 93 defines the parameters for the *Service*.

Table 93 – Republish Service Parameters

Name	Type	Description
Request		
requestHeader	RequestHeader	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).
subscriptionId	IntegerId	The <i>Server</i> assigned identifier for the <i>Subscription</i> to be republished (see 7.13 for <i>IntegerId</i> definition).
retransmitSequence Number	Counter	The sequence number of a specific <i>NotificationMessage</i> to be republished (see 7.5 for <i>Counter</i> definition).
Response		
responseHeader	ResponseHeader	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).
notificationMessage	Notification Message	The requested <i>NotificationMessage</i> . The <i>NotificationMessage</i> parameter type is specified in 7.18.

5.13.6.3 Service results

Table 94 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 94 – Republish Service Result Codes

Symbolic Id	Description
Bad_SubscriptionIdInvalid	See Table 165 for the description of this result code.
Bad_MessageNotAvailable	The requested message is no longer available.

5.13.7 TransferSubscriptions

5.13.7.1 Description

This *Service* is used to transfer a *Subscription* and its *MonitoredItems* from one *Session* to another. For example, a *Client* may need to reopen a *Session* and then transfer its *Subscriptions* to that *Session*. It may also be used by one *Client* to take over a *Subscription* from another *Client* by transferring the *Subscription* to its *Session*.

The *authenticationToken* contained in the request header identifies the *Session* to which the *Subscription* and *MonitoredItems* shall be transferred. The *Server* shall validate that the *Client* of that *Session* is operating on behalf of the same user and that the potentially new *Client* supports the *Profiles* that are necessary for the *Subscription*. If the *Server* transfers the

Subscription, it returns the sequence numbers of the *NotificationMessages* that are available for retransmission. The *Client* should acknowledge all *Messages* in this list for which it will not request retransmission.

If the *Server* transfers the *Subscription* to the new *Session*, the *Server* shall issue a *StatusChangeNotification notificationMessage* with the status code *Good_SubscriptionTransferred* to the old *Session*. The *StatusChangeNotification notificationMessage* type is defined in 7.19.4.

5.13.7.2 Parameters

Table 95 defines the parameters for the *Service*.

Table 95 – TransferSubscriptions Service Parameters

Name	Type	Description
Request		
requestHeader	RequestHeader	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).
subscriptionIds []	IntegerId	List of identifiers for the <i>Subscriptions</i> to be transferred to the new <i>Client</i> (see 7.13 for <i>IntegerId</i> definition). These identifiers are transferred from the primary <i>Client</i> to a backup <i>Client</i> via external mechanisms.
sendInitialValues	Boolean	A <i>Boolean</i> parameter with the following values: TRUE the first Publish response after the TransferSubscriptions call shall contain the current values of all Monitored Items in the Subscription where the Monitoring Mode is set to Reporting. FALSE the first Publish response after the TransferSubscriptions call shall contain only the value changes since the last Publish response was sent. This parameter only applies to MonitoredItems used for monitoring Attribute changes.
Response		
responseHeader	ResponseHeader	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).
results []	TransferResult	List of results for the <i>Subscriptions</i> to transfer. The size and order of the list matches the size and order of the <i>subscriptionIds</i> request parameter. This structure is defined in-line with the following indented items.
statusCode	StatusCode	<i>StatusCode</i> for each <i>Subscription</i> to be transferred (see 7.33 for <i>StatusCode</i> definition).
availableSequence Numbers []	Counter	A list of sequence number ranges that identify <i>NotificationMessages</i> that are in the <i>Subscription's</i> retransmission queue. This parameter is null if the transfer of the <i>Subscription</i> failed. The <i>Counter</i> type is defined in 7.5.
diagnosticInfos []	DiagnosticInfo	List of diagnostic information for the <i>Subscriptions</i> to transfer (see 7.8 for <i>DiagnosticInfo</i> definition). The size and order of the list matches the size and order of the <i>subscriptionIds</i> request parameter. This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the request.

5.13.7.3 Service results

Table 96 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 96 – TransferSubscriptions Service Result Codes

Symbolic Id	Description
Bad_NothingToDo	See Table 165 for the description of this result code.
Bad_TooManyOperations	See Table 165 for the description of this result code.
Bad_InsufficientClientProfile	The <i>Client</i> of the current <i>Session</i> does not support one or more <i>Profiles</i> that are necessary for the <i>Subscription</i> .

5.13.7.4 StatusCodes

Table 97 defines values for the operation level *statusCode* parameter that are specific to this *Service*. Common *StatusCodes* are defined in Table 166.

Table 97 – TransferSubscriptions Operation Level Result Codes

Symbolic Id	Description
Bad_SubscriptionIdInvalid	See Table 165 for the description of this result code.
Bad_UserAccessDenied	See Table 165 for the description of this result code. The <i>Client</i> of the current <i>Session</i> is not operating on behalf of the same user as the <i>Session</i> that owns the <i>Subscription</i> .

5.13.8 DeleteSubscriptions

5.13.8.1 Description

This *Service* is invoked to delete one or more *Subscriptions* that belong to the *Client's Session*.

Successful completion of this *Service* causes all *MonitoredItems* that use the *Subscription* to be deleted. If this is the last *Subscription* for the *Session*, then all *Publish* requests still queued for that *Session* are de-queued and shall be returned with *Bad_NoSubscription*.

Subscriptions that were transferred to another *Session* shall be deleted by the *Client* that owns the *Session*.

5.13.8.2 Parameters

Table 98 defines the parameters for the *Service*.

Table 98 – DeleteSubscriptions Service Parameters

Name	Type	Description
Request		
requestHeader	RequestHeader	Common request parameters (see 7.26 for <i>RequestHeader</i> definition).
subscriptionIds []	IntegerId	The <i>Server</i> -assigned identifier for the <i>Subscription</i> (see 7.13 for <i>IntegerId</i> definition).
Response		
responseHeader	ResponseHeader	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).
results []	StatusCodes	List of <i>StatusCodes</i> for the <i>Subscriptions</i> to delete (see 7.33 for <i>StatusCode</i> definition). The size and order of the list matches the size and order of the <i>subscriptionIds</i> request parameter.
diagnosticInfos []	DiagnosticInfo	List of diagnostic information for the <i>Subscriptions</i> to delete (see 7.8 for <i>DiagnosticInfo</i> definition). The size and order of the list matches the size and order of the <i>subscriptionIds</i> request parameter. This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the request.

5.13.8.3 Service results

Table 99 defines the *Service* results specific to this *Service*. Common *StatusCodes* are defined in Table 165.

Table 99 – DeleteSubscriptions Service Result Codes

Symbolic Id	Description
Bad_NothingToDo	See Table 165 for the description of this result code.
Bad_TooManyOperations	See Table 165 for the description of this result code.

5.13.8.4 StatusCodes

Table 100 defines values for the *results* parameter that are specific to this *Service*. Common *StatusCodes* are defined in Table 166.

Table 100 – DeleteSubscriptions Operation Level Result Codes

Symbolic Id	Description
Bad_SubscriptionIdInvalid	See Table 165 for the description of this result code.

6 Service behaviours

6.1 Security

6.1.1 Overview

The OPC UA *Services* define a number of mechanisms to meet the security requirements outlined in IEC TR 62541-2. This clause describes a number of important security-related procedures that *OPC UA Applications* shall follow.

6.1.2 Obtaining and Installing an Application Instance Certificate

All *OPC UA Applications* require an *Application Instance Certificate* which shall contain the following information:

- The network name or address of the computer where the application runs;
- The name of the organisation that administers or owns the application;
- The name of the application;
- The URI of the application instance;
- The name of the *Certificate Authority* that issued the *Certificate*;
- The issue and expiry date for the *Certificate*;
- The public key issued to the application by the *Certificate Authority* (CA);
- A digital signature created by the *Certificate Authority* (CA).

In addition, each *Application Instance Certificate* has a private key which should be stored in a location that can only be accessed by the application. If this private key is compromised, the administrator shall assign a new *Application Instance Certificate* and private key to the application.

This *Certificate* may be generated automatically when the application is installed. In this situation the private key assigned to the *Certificate* shall be used to create the *Certificate* signature. *Certificates* created in this way are called self-signed *Certificates*.

If the administrator responsible for the application decides that a self-signed *Certificate* does not meet the security requirements of the organisation, then the administrator should install a *Certificate* issued by a *Certification Authority*. The steps involved in requesting an *Application Instance Certificate* from a *Certificate Authority* are shown in Figure 19.

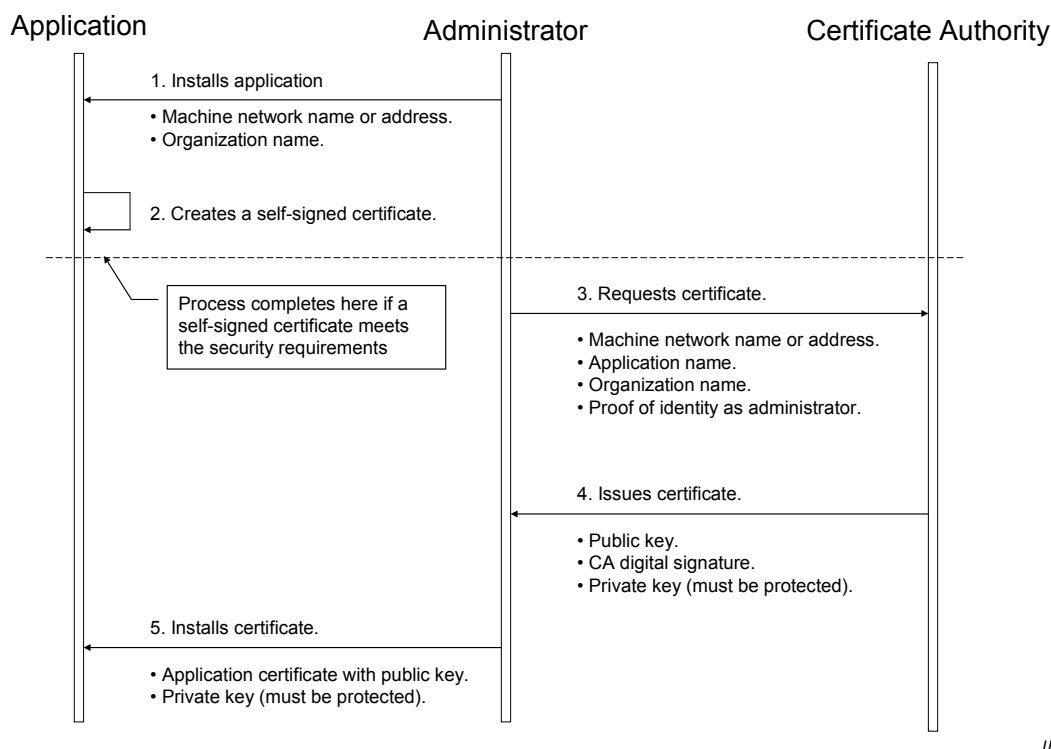


Figure 19 – Obtaining and Installing an Application Instance Certificate

The figure above illustrates the interactions between the application, the *Administrator* and the *Certificate Authority*. The *Application* is as *OPC UA Application* installed on a single machine. The *Administrator* is the person responsible for managing the machine and the *OPC UA Application*. The *Certificate Authority* is an entity that can issue digital *Certificates* that meet the requirements of the organisation deploying the *OPC UA Application*.

If the *Administrator* decides that a self-signed *Certificate* meets the security requirements for the organisation, then the *Administrator* may skip Steps 3 through 5. Application vendors shall always create a default self-signed *Certificate* during the installation process. Every *OPC UA Application* shall allow the *Administrators* to replace *Application Instance Certificates* with *Certificates* that meet their requirements.

When the *Administrator* requests a new *Certificate* from a *Certificate Authority*, the *Certificate Authority* may require that the *Administrator* provide proof of authorization to request *Certificates* for the organisation that will own the *Certificate*. The exact mechanism used to provide this proof depends on the *Certificate Authority*.

Vendors may choose to automate the process of acquiring *Certificates* from an authority. If this is the case, the *Administrator* would still go through the steps illustrated in Figure 19, however, the installation program for the application would do them automatically and only prompt the *Administrator* to provide information about the application instance being installed.

6.1.3 Determining if a Certificate is Trusted

Applications shall never communicate with another application that they do not trust. An *Application* decides if another application is trusted by checking whether the *Application Instance Certificate* for the other application is trusted. Applications shall rely on lists of *Certificates* provided by the *Administrator* to determine trust. There are two separate lists: a list of trusted *Applications* and a list of trusted *Certificate Authorities* (CAs). If an application is not directly trusted (i.e. its *Certificate* is not in the list of trusted applications) then the application shall build a chain of *Certificates* back to a trusted CA.

When building a chain each *Certificate* in the chain shall be validated. If any validation error occurs then the trust check fails. Some validation errors are non-critical which means they can be suppressed by a user of an *Application* with the appropriate privileges. Suppressed validation errors are always reported via auditing (i.e. an appropriate Audit event is raised).

Building a trust chain requires access to all *Certificates* in the chain. These *Certificates* may be stored locally or they may be provided with the application *Certificate*. Processing fails with *Bad_SecurityChecksFailed* if a CA *Certificate* cannot be found.

Table 101 specifies the steps used to validate a *Certificate* in the order that they shall be followed. These steps are repeated for each *Certificate* in the chain. Each validation step has a unique error status and audit event type that shall be reported if the check fails. The audit event is in addition to any audit event that was generated for the particular *Service* that was invoked. The *Service* audit event in its message text shall include the audit *EventID* of the *AuditCertificateEventType* (for more details, see 6.2). Processing halts if an error occurs, unless it is non-critical and it has been suppressed.

ApplicationInstanceCertificates shall not be used in a *Client* or *Server* until they have been evaluated and marked as trusted. This can happen automatically by a PKI trust chain or in an offline manner where the *Certificate* is marked as trusted by an administrator after evaluation.

Table 101 – Certificate Validation Steps

Step	Error/AuditEvent	Description
Certificate Structure	Bad_SecurityChecksFailed AuditCertificateInvalidEventType	The <i>Certificate</i> structure is verified. This error may not be suppressed.
Signature	Bad_SecurityChecksFailed AuditCertificateInvalidEventType	A <i>Certificate</i> with an invalid signature shall always be rejected. A <i>Certificate</i> signature is invalid if the <i>Issuer Certificate</i> is unknown. A self-signed <i>Certificate</i> is its own issuer.
Trust List Check	Bad_SecurityChecksFailed AuditCertificateUntrustedEventType	If the <i>Application Instance Certificate</i> is not trusted and none of the CA <i>Certificates</i> in the chain is trusted, the result of the <i>Certificate</i> validation shall be <i>Bad_SecurityChecksFailed</i> .
Validity Period	Bad_CertificateTimeInvalid Bad_CertificateIssuerTimeInvalid AuditCertificateExpiredEventType	The current time shall be after the start of the validity period and before the end. This error may be suppressed.
Host Name	Bad_CertificateHostNameInvalid AuditCertificateDataMismatchEventType	The <i>HostName</i> in the URL used to connect to the <i>Server</i> shall be the same as one of the <i>HostNames</i> specified in the <i>Certificate</i> . This check is skipped for CA <i>Certificates</i> . This error may be suppressed.
URI	Bad_CertificateUriInvalid AuditCertificateDataMismatchEventType	<i>Application and Software Certificates</i> contain an application or product URI that shall match the URI specified in the <i>ApplicationDescription</i> provided with the <i>Certificate</i> . This check is skipped for CA <i>Certificates</i> . This error may not be suppressed. The <i>gatewayServerUri</i> is used to validate an <i>Application Certificate</i> when connecting to a <i>Gateway Server</i> (see 7.1).
Certificate Usage	Bad_CertificateUseNotAllowed Bad_CertificateIssuerUseNotAllowed AuditCertificateMismatchEventType	Each <i>Certificate</i> has a set of uses for the <i>Certificate</i> (see IEC 62541-6). These uses shall match use requested for the <i>Certificate</i> (i.e. Application, Software or CA). This error may be suppressed unless the <i>Certificate</i> indicates that the usage is mandatory.
Find Revocation List	Bad_CertificateRevocationUnknown Bad_CertificateIssuerRevocationUnknown AuditCertificateRevokedEventType	Each CA <i>Certificate</i> may have a revocation list. This check fails if this list is not available (i.e. a network interruption prevents the application from accessing the list). No error is reported if the <i>Administrator</i> disables revocation checks for a CA <i>Certificate</i> . This error may be suppressed.
Revocation Check	Bad_CertificateRevoked Bad_CertificateIssuerRevoked AuditCertificateRevokedEventType	The <i>Certificate</i> has been revoked and may not be used. This error may not be suppressed.

Certificates are usually placed in a central location called a *CertificateStore*. Figure 20 illustrates the interactions between the *Application*, the *Administrator* and the *CertificateStore*. The *CertificateStore* could be on the local machine or in some central server. The exact mechanisms used to access the *CertificateStore* depend on the application and PKI environment set up by the *Administrator*.

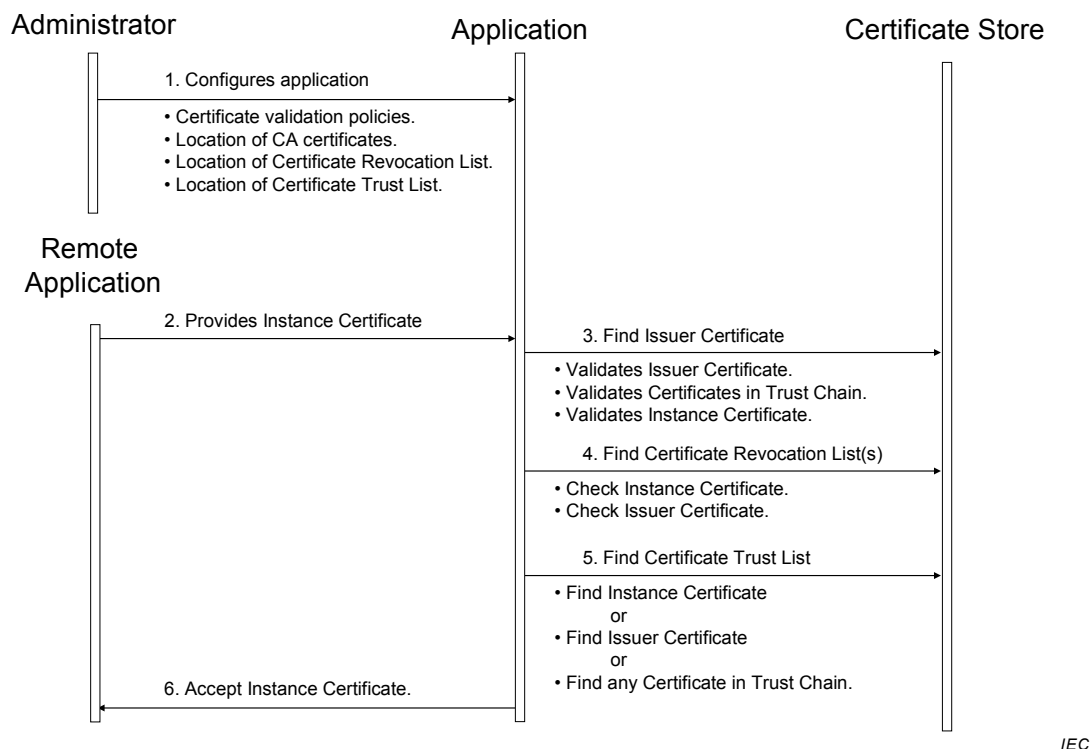
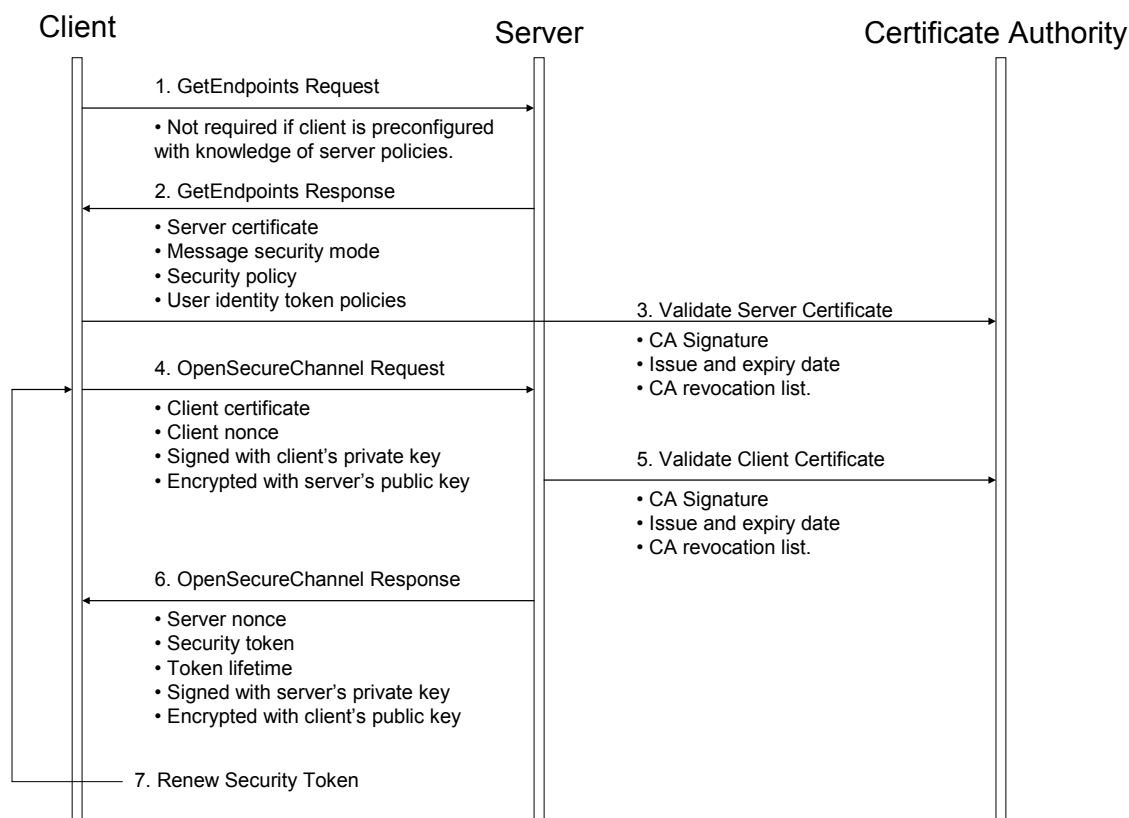


Figure 20 – Determining if a Application Instance Certificate is Trusted

6.1.4 Creating a SecureChannel

All *OPC UA Applications* shall establish a *SecureChannel* before creating a *Session*. This *SecureChannel* requires that both applications have access to *Certificates* that can be used to encrypt and sign *Messages* exchange. The *Application Instance Certificates* installed by following the process described in 6.1.2 may be used for this purpose.

The steps involved in establishing a *SecureChannel* are shown in Figure 21.



IEC

Figure 21 – Establishing a SecureChannel

Figure 21 above assumes *Client* and *Server* have online access to a *CertificateAuthority* (CA). If online access is not available and if the administrator has installed the CA public key on the local machine, then the *Client* and *Server* shall still validate the application *Certificates* using that key. The figure shows only one CA, however, there is no requirement that the *Client* and *Server* *Certificates* be issued by the same authority. A self-signed *Application Instance Certificate* does not need to be verified with a CA. Any *Certificate* shall be rejected if it is not in a trust list provided by the administrator.

Both the *Client* and *Server* shall have a list of *Certificates* that they have been configured to trust (sometimes called the *Certificate Trust List* or CTL). These trusted *Certificates* may be *Certificates* for *Certificate Authorities* or they may be *OPC UA Application Instance Certificates*. *OPC UA Applications* shall be configured to reject connections with applications that do not have a trusted *Certificate*.

Certificates can be compromised, which means they should no longer be trusted. Administrators can revoke a *Certificate* by removing it from the trust list for all applications or the CA can add the *Certificate* to the *Certificate Revocation List* (CRL) for the *Issuer Certificate*. Administrators may save a local copy of the CRL for each *Issuer Certificate* when online access is not available.

A *Client* does not need to call *GetEndpoints* each time it connects to the *Server*. This information should change rarely and the *Client* can cache it locally. If the *Server* rejects the *OpenSecureChannel* request the *Client* should call *GetEndpoints* and make sure the *Server* configuration has not changed.

There are two security risks which a *Client* shall be aware of when using the *GetEndpoints Service*. The first could come from a rogue *Discovery Server* that tries to direct the *Client* to a rogue *Server*. For this reason the *Client* shall verify that the *ServerCertificate* in the *EndpointDescription* is a trusted *Certificate* before it calls *CreateSession*.

The second security risk comes from a third party that alters the contents of the *EndpointDescriptions* as they are transferred over the network back to the *Client*. The *Client* protects itself against this by comparing the list of *EndpointDescriptions* returned from the *GetEndpoints Service* with list returned in the *CreateSession* response.

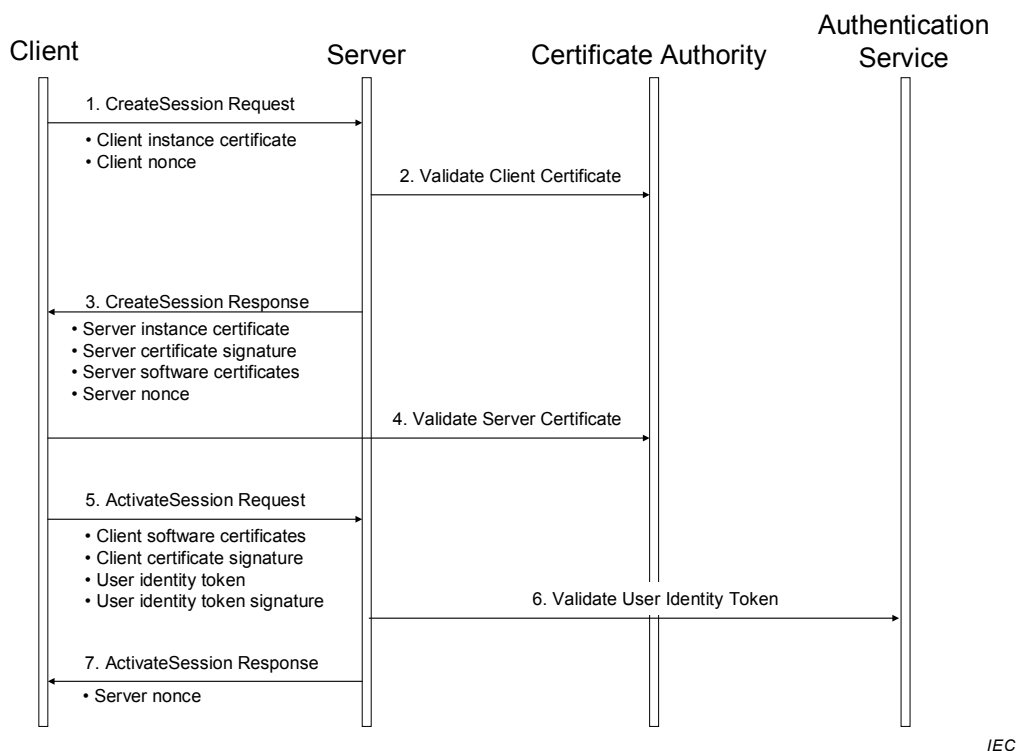
The exact mechanisms for using the security token to sign and encrypt *Messages* exchanged over the *SecureChannel* are described in IEC 62541-6. The process for renewing tokens is also described in detail in IEC 62541-6.

In many cases, the *Certificates* used to establish the *SecureChannel* will be the *Application Instance Certificates*. However, some *Communication Stacks* might not support *Certificates* that are specific to a single application. Instead, they expect all communication to be secured with a *Certificate* specific to a user or the entire machine. For this reason, *OPC UA Applications* will need to exchange their *Application Instance Certificates* when creating a *Session*.

6.1.5 Creating a Session

Once an OPC UA *Client* has established a *SecureChannel* with a *Server* it can create an OPC UA *Session*.

The steps involved in establishing a *Session* are shown in Figure 22.



IEC

Figure 22 – Establishing a Session

Figure 22 above illustrates the interactions between a *Client*, a *Server*, a *Certificate Authority* (CA) and an authentication service. The CA is responsible for issuing the *Application Instance Certificates*. If the *Client* or *Server* does not have online access to the CA, then they shall validate the *Application Instance Certificates* using the CA public key that the administrator shall install on the local machine.

The authentication service is a central database that can verify that user token provided by the *Client*. This authentication service may also tell the *Server* which access rights the user has. The authentication service depends on the user identity token. It could be a *Certificate*

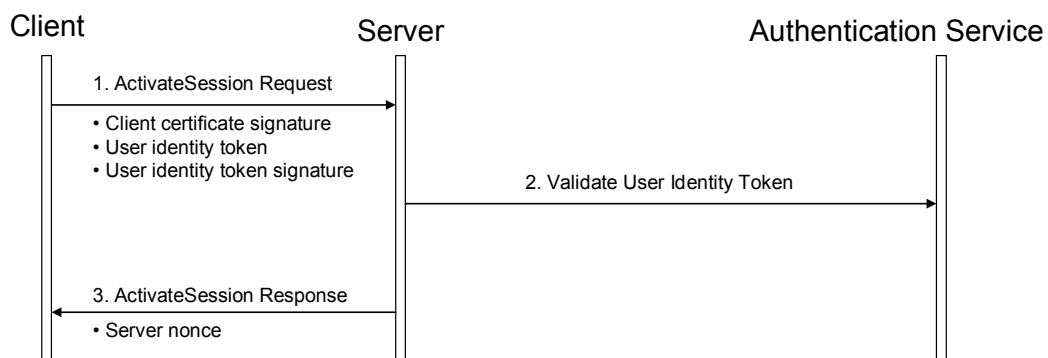
Authority, a Kerberos ticket granting service, a WS-Trust *Server* or a proprietary database of some sort.

The *Client* and *Server* shall prove possession of their *Application Instance Certificates* by signing the *Certificates* with a nonce appended. The exact mechanism used to create the proof of possession signatures is described in 5.6.2. Similarly, the *Client* shall prove possession of some types of user identity tokens by creating signatures with the secret associated with the token.

6.1.6 Impersonating a User

Once an OPC UA *Client* has established a *Session* with a *Server* it can change the user identity associated with the *Session* by calling the *ActivateSession* service.

The steps involved in impersonating a user are shown in Figure 23.



IEC

Figure 23 – Impersonating a User

6.2 Software Certificates

6.2.1 Overview

All OPC UA Applications may have one or more software *Certificates* that are issued by certification authorities and specify the profiles that the application supports. This clause describes how *SoftwareCertificates* are obtained, installed and validated.

SoftwareCertificates are used to identify a software product that is an OPC UA Application and to describe the capabilities of the product. The capabilities are described through *Profiles* defined in IEC 62541-7. *SoftwareCertificates* are not relevant for security in OPC UA.

6.2.2 Obtaining and Installing a Software Certificate

The software *Certificates* contain the following information:

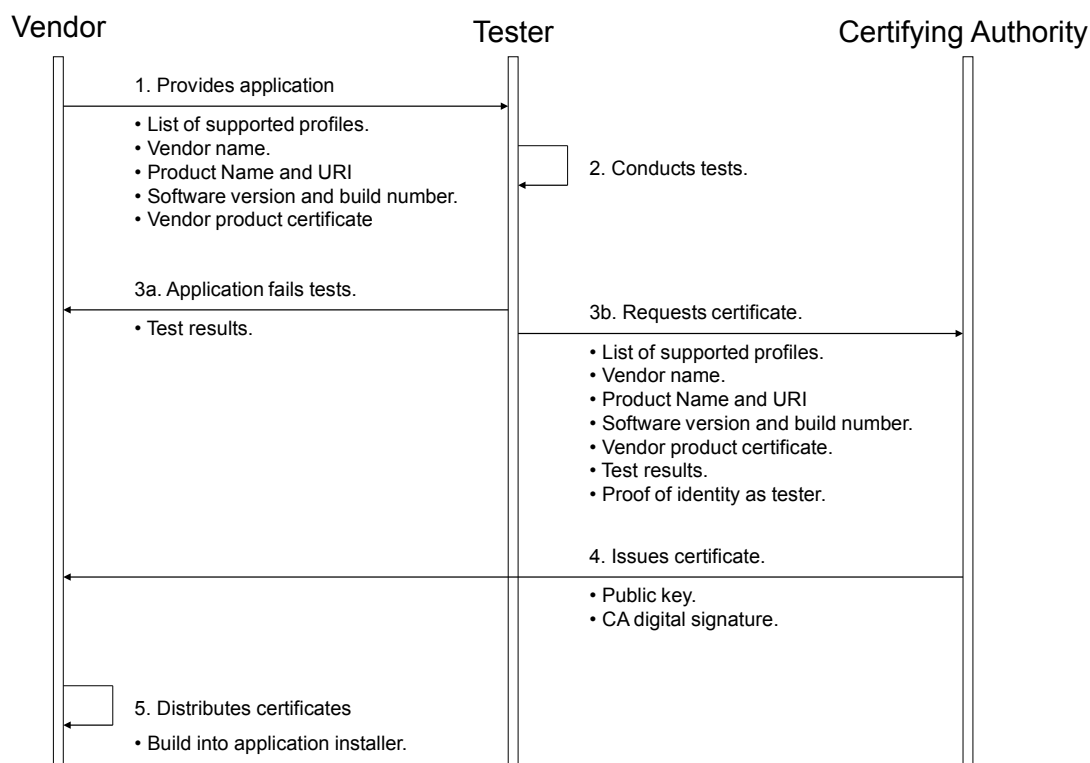
- The name of the vendor responsible for the product;
- The name of the product;
- The URI for the product;
- The software version and build number;
- The vendor product *Certificate*;
- A list of profiles supported by the application;
- The certification testing status for each supported profile;

- The name of the testing authority that issued the *Certificate*;
- The issue and expiry date for the *Certificate*;
- The public key issued to the application by the certifying authority;
- A digital signature created by the certifying authority.

The product vendor is responsible for completing the certification process and requesting software *Certificates* from the certification authorities. The vendor shall install the software *Certificates* when an application is installed on a machine. When distributing these *Certificates*, the application vendors should take precautions to prevent unauthorized users from acquiring their software *Certificates* and using them for applications that the vendor did not develop. Misused software *Certificates* are not a security risk, but vendors could find that they are blamed for interoperability problems caused by *Certificates* used by unauthorized applications.

Vendors may choose to assign their own *Certificate* to the application (called the Vendor Product *Certificate*). This *Certificate* would then be incorporated in the *SoftwareCertificate*. The private key for the vendor product *Certificate* is managed by the vendor.

The steps involved in acquiring and installing a software *Certificate* from a *Certificate Authority* are shown in Figure 24.



IEC

Figure 24 – Obtaining and Installing a Software Certificate

The figure above illustrates the interactions between the vendor, the tester and the certifying authority. The vendor is the organisation responsible for the *OPC UA Application*. The tester may be the vendor (for self-certification) or it may be a third-party testing facility. The certifying authority is a PKI *Certificate* authority managed by the organisation that created the *OPC UA Application* profiles and certification programmes.

The certifying authority will issue *Certificates* only to vendors it trusts. For that reason, the tester shall provide proof of identity before the certifying authority will issue a *Certificate*.

6.2.3 Validating a Software Certificate

OPC UA Applications shall validate the *SoftwareCertificates* provided by the applications that they communicate with.

A *SoftwareCertificate* is valid if:

- The signature on the *SoftwareCertificate* is valid;
- The *SoftwareCertificate* has passed its issue date and it has not expired;
- The *SoftwareCertificate* has not been revoked by the issuer;
- The *Issuer Certificate* is valid and has not been revoked by the CA that issued it.

The steps used to validate *SoftwareCertificates* are almost the same as the steps described for *Application Instance Certificates* in 6.1.3. The only difference is *SoftwareCertificates* are not checked against the *TrustList* for the *Application*.

SoftwareCertificates always contain an *Application Instance Certificate* which is owned by the vendor that distributes the application. The application *Certificate* may also be in the *Certificate* chain used to issue *Application Instance Certificates*. For this reason, *OPC UA Applications* may reject *SoftwareCertificates* provided by applications if the application *Certificate* is not part of the *Certificate* chain for the *Application Instance Certificate*. *OPC UA Applications* should allow *Administrators* to require this behaviour. Profiles defined in IEC 62541-7 may further specify expected *Certificate* handling behaviours.

6.3 Auditing

6.3.1 Overview

Auditing is a requirement in many systems. It provides a means of tracking activities that occur as part of normal operation of the system. It also provides a means of tracking abnormal behaviour. It is also a requirement from a security standpoint. For more information on the security aspects of auditing, see IEC TR 62541-2. This subclause describes what is expected of an *OPC UA Server* and *Client* with respect to auditing and it details the audit requirements for each service set. Auditing can be accomplished using one or both of the following methods:

- a) The *OPC UA Application* that generates the audit event can log the audit entry in a log file or other storage location;
- b) The *OPC UA Server* that generates the audit event can publish the audit event using the *OPC UA* event mechanism. This allows an external *OPC UA Client* to subscribe to and log the audit entries to a log file or other storage location.

6.3.2 General audit logs

Each *OPC UA Service* request contains a string parameter that is used to carry an audit record id. A *Client* or any *Server* operating as a *Client*, such as an aggregating *Server*, can create a local audit log entry for a request that it submits. This parameter allows this *Client* to pass the identifier for this entry with the request. If this *Server* also maintains an audit log, it should include this id in its audit log entry that it writes. When this log is examined and that entry is found, the examiner will be able to relate it directly to the audit log entry created by the *Client*. This capability allows for traceability across audit logs within a system.

6.3.3 General audit Events

A *Server* that maintains an audit log shall provide the audit log entries via *Event Messages*. The *AuditEventType* and its sub-types are defined in IEC 62541-3. An audit *Event Message* also includes the audit record Id. The details of the *AuditEventType* and its subtypes are defined in IEC 62541-5. A *Server* that is an aggregating *Server* that supports auditing shall also subscribe for audit events for all of the *Servers* that it is aggregating (assuming they provide auditing). The combined stream should be available from the aggregating *Server*.

6.3.4 Auditing for Discovery Service Set

This *Service Set* can be separated into two groups: *Services* that are called by OPC UA *Clients* and *Services* that are invoked by OPC UA *Servers*. The *FindServers* and *GetEndpoints* *Services* that are called by OPC UA *Clients* may generate audit entries for failed *Service* invocations. The *RegisterServer* *Service* that is invoked by OPC UA *Servers* shall generate audit entries for all new registrations and for failed *Service* invocations. These audit entries shall include the *Server* URI, *Server* names, *Discovery* URIs and *isOnline* status. Audit entries should not be generated for *RegisterServer* invocation that does not cause changes to the registered *Servers*.

6.3.5 Auditing for SecureChannel Service Set

All *Services* in this *Service Set* for *Servers* that support auditing may generate audit entries and shall generate audit *Events* for failed service invocations and for successful invocation of the *OpenSecureChannel* and *CloseSecureChannel* *Services*. The *Client* generated audit entries should be setup prior to the actual call, allowing the correct audit record *Id* to be provided. The *OpenSecureChannel* *Service* shall generate an audit *Event* of type *AuditOpenSecureChannelEventType* or a subtype of it for the *requestType* *ISSUE_0*. Audit *Events* for the *requestType* *RENEW_1* are only created if the renew fails. The *CloseSecureChannel* service shall generate an audit *Event* of type *AuditChannelEventType* or a subtype of it. Both of these *Event* types are subtypes of the *AuditChannelEventType*. See IEC 62541-5 for the detailed assignment of the *SourceNode*, the *SourceName* and additional parameters. For the failure cases the *Message* for *Events* of this type should include a description of why the service failed. This description should be more detailed than what was returned to the client. From a security point of view a client only needs to know that it failed, but from an *Auditing* point of view the exact details of the failure need to be known. In the case of *Certificate* validation errors the description should include the audit *EventId* of the specific *AuditCertificateEventType* that was generated to report the *Certificate* error. The *AuditCertificateEvent* shall also contain the detailed *Certificate* validation error. The additional parameters should include the details of the request. It is understood that these events may be generated by the underlying *Communication Stacks* in many cases, but they shall be made available to the *Server* and the *Server* shall report them.

6.3.6 Auditing for Session Service Set

All *Services* in this *Service Set* for *Servers* that support auditing may generate audit entries and shall generate audit *Events* for both successful and failed *Service* invocations. These *Services* shall generate an audit *Event* of type *AuditSessionEventType* or a subtype of it. In particular, they shall generate the base *EventType* or the appropriate subtype, depending on the service that was invoked. The *CreateSession* service shall generate *AuditCreateSessionEventType* events or sub-types of it. The *ActivateSession* service shall generate *AuditActivateSessionEventType* events or subtypes of it. When the *ActivateSession* *Service* is called to change the user identity then the server shall generate *AuditActivateSessionEventType* events or subtypes of it. The *CloseSession* service shall generate the base *EventType* of *AuditSessionEventType* or subtypes of it. See IEC 62541-5 for the detailed assignment of the *SourceNode*, the *SourceName* and additional parameters. For the failure case the *Message* for *Events* of this type should include a description of why the *Service* failed. The additional parameters should include the details of the request.

This *Service Set* shall also generate additional audit events in the cases when *Certificate* validation errors occur. These audit events are generated in addition to the *AuditSessionEventTypes*. See IEC 62541-3 for the definition of *AuditCertificateEventType* and its subtypes.

For *Clients*, that support auditing, accessing the services in the *Session Service Set* shall generate audit entries for both successful and failed invocations of the *Service*. These audit entries should be setup prior to the actual *Service* invocation, allowing the invocation to contain the correct audit record *id*.

6.3.7 Auditing for NodeManagement Service Set

All *Services* in this *Service Set* for *Servers* that support auditing may generate audit entries and shall generate audit *Events* for both successful and failed *Service* invocations. These *Services* shall generate an audit *Event* of type *AuditNodeManagementEventType* or subtypes of it. See IEC 62541-5 for the detailed assignment of the *SourceNode*, the *SourceName* and additional parameters. For the failure case, the *Message* for *Events* of this type should include a description of why the service failed. The additional parameters should include the details of the request.

For *Clients* that support auditing, accessing the *Services* in the *NodeManagement Service Set* shall generate audit entries for both successful and failed invocations of the *Service*. All audit entries should be setup prior to the actual *Service* invocation, allowing the invocation to contain the correct audit record id.

6.3.8 Auditing for Attribute Service Set

The *Write* or *HistoryUpdate* *Services* in this *Service Set* for *Servers* that support auditing may generate audit entries and shall generate audit *Events* for both successful and failed *Service* invocations. These *Services* shall generate an audit *Event* of type *AuditUpdateEventType* or subtypes of it. In particular, the *Write Service* shall generate an audit event of type *AuditWriteUpdateEventType* or a subtype of it. The *HistoryUpdate Service* shall generate an audit *Event* of type *AuditHistoryUpdateEvent* or a subtype of it. Three subtypes of *AuditHistoryUpdateEvent* are defined as *AuditHistoryEventUpdateEventType*, *AuditHistoryValueUpdateEventType* and *AuditHistoryDeleteEventType*. The subtype depends on the type of operation being performed, historical event update, historical data value update or a historical delete. See IEC 62541-5 for the detailed assignment of the *SourceNode*, the *SourceName* and additional parameters. For the failure case the *Message* for *Events* of this type should include a description of why the *Service* failed. The additional parameters should include the details of the request.

The *Read* and *HistoryRead Services* may generate audit entries and audit *Events* for failed *Service* invocations. These *Services* should generate an audit *Event* of type *AuditEventType* or a subtype of it. See IEC 62541-5 for the detailed assignment of the *SourceNode*, *SourceName* and additional parameters. The *Message* for *Events* of this type should include a description of why the *Service* failed.

For *Clients* that support auditing, accessing the *Write* or *HistoryUpdate* services in the *Attribute Service Set* shall generate audit entries for both successful and failed invocations of the *Service*. Invocations of the other *Services* in this *Service Set* may generate audit entries. All audit entries should be setup prior to the actual *Service* invocation, allowing the invocation to contain the correct audit record id.

6.3.9 Auditing for Method Service Set

All *Services* in this *Service Set* for *Servers* that support auditing may generate audit entries and shall generate audit *Events* for both successful and failed service invocations if the invocation modifies the address space, writes a value or modifies the state of the system (alarm acknowledge, batch sequencing or other system changes). These method calls shall generate an audit *Event* of type *AuditUpdateMethodEventType* or subtypes of it. Methods that do not modify the address space, write values or modify the state of the system may generate events. See IEC 62541-5 for the detailed assignment of the *SourceNode*, *SourceName* and additional parameters.

For *Clients* that support auditing, accessing the *Method Service Set* shall generate audit entries for both successful and failed invocations of the *Service*, if the invocation modifies the address space, writes a value or modifies the state of the system (alarm acknowledge, batch sequencing or other system changes). Invocations of the other *Methods* may generate audit entries. All audit entries should be setup prior to the actual *Service* invocation, allowing the invocation to contain the correct audit record id.

6.3.10 Auditing for View, Query, MonitoredItem and Subscription Service Set

All of the *Services* in these four *Service Sets* only provide the *Client* with information, with the exception of the *TransferSubscriptions Service* in the *Subscription Service Set*. In general, these services will not generate audit entries or audit *Event Messages*. The *TransferSubscriptions Service* shall generate an audit *Event* of type *AuditSessionEventType* or subtypes of it for both successful and failed *Service* invocations. See IEC 62541-5 for the detailed assignment of the *SourceNode*, the *SourceName* and additional parameters. For the failure case, the *Message* for *Events* of this type should include a description of why the service failed.

For *Clients* that support auditing, accessing the *TransferSubscriptions Service* in the *Subscription Service Set* shall generate audit entries for both successful and failed invocations of the *Service*. Invocations of the other *Services* in this *Service Set* do not require audit entries. All audit entries should be setup prior to the actual *Service* invocation, allowing the invocation to contain the correct audit record id.

6.4 Redundancy

6.4.1 Redundancy overview

Redundancy in OPC UA ensures that both *Clients* and *Server* can be redundant. OPC UA does not provide redundancy; it provides the data structures and services by which redundancy may be achieved in a standardized manner.

6.4.2 Server redundancy overview

6.4.2.1 General

Server redundancy comes in two modes, transparent and non-transparent. By definition, in transparent redundancy the failover of *Server* responsibilities from one *Server* to another is transparent to the *Client*: the *Client* does not care or even know that failover has occurred; the *Client* does not need to do anything at all to keep data flowing. In contrast, non-transparent failover requires some activity on the part of the *Client*.

The *ServerRedundancy Object* defined in IEC 62541-5 indicates the mode supported by the *Server*. The *ServerRedundancyType ObjectType* and its subtypes *TransparentRedundancyType* and *NonTransparentRedundancyType* defined in IEC 62541-5 specify information for the supported redundancy mode.

The two areas where redundancy creates specific needs are in keeping the *Server* and *Client* information synchronised across *Servers*, and in controlling the failover of data flow from one *Server* to another.

Independent of the used redundancy mode it is expected that all *Servers* in the redundant set have an identical address space including identical *NodeIds* and the identical logic for setting the service level.

6.4.2.2 Transparent redundancy

For transparent redundancy, all that OPC UA provides is the data structures to allow the *Client* to identify which *Servers* are in the redundant set, what the service level of each *Server* is and which *Server* is currently responsible for the *Client Session*. This information is specified in *TransparentRedundancyType ObjectType* defined in IEC 62541-5.

All OPC UA interactions within a given session shall be supported by one *Server* and the *Client* is able to identify which *Server* that is, allowing a complete audit trail for the data. It is the responsibility of the *Servers* to ensure that information is synchronised between the *Servers*. A functional *Server* will take over the *Session* and *Subscriptions* from the failed

Server. Failover may require a transport layer reconnect of the *Client* but the *Endpoint* URL of the *Server* shall not change. See 6.5 for more details on re-establishing connections.

Figure 25 shows a typical transparent redundancy setup.

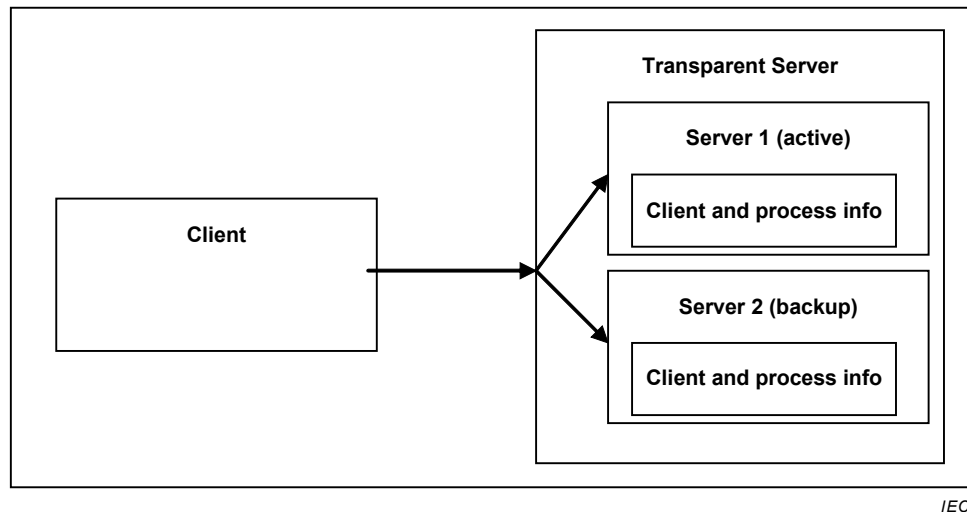


Figure 25 – Transparent Redundancy setup

6.4.2.3 Non-transparent redundancy

For non-transparent redundancy, OPC UA provides the data structures to allow the *Client* to identify what *Servers* are available in the redundant set and also *Server* information which tells the *Client* what modes of failover the *Server* supports. This information allows the *Client* to determine what actions it may need to take in order to accomplish failover. This information is specified in *NonTransparentRedundancyType ObjectType* defined in IEC 62541-5.

Figure 26 shows a typical non-transparent redundancy setup.

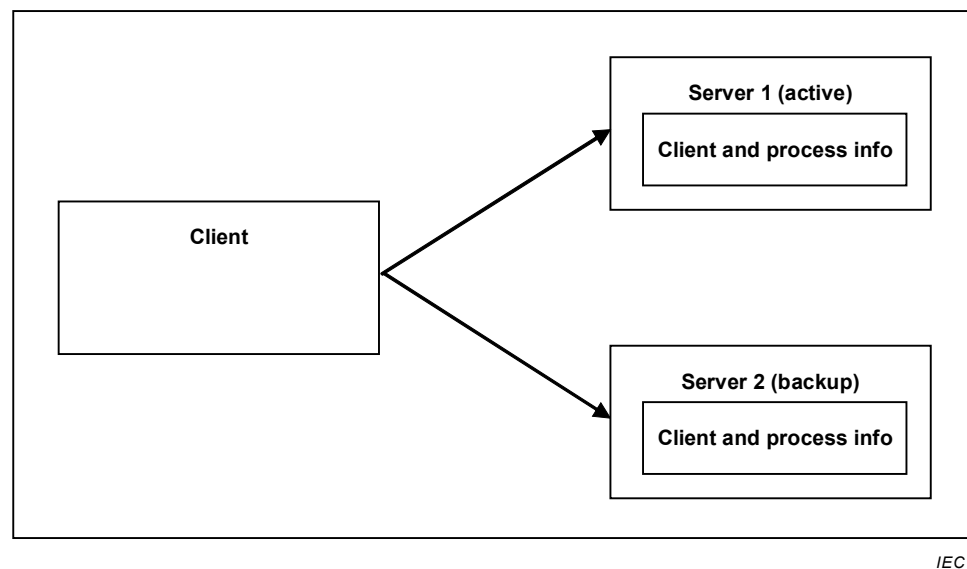


Figure 26 – Non-Transparent Redundancy setup

For non-transparent redundancy the *Server* has additional concepts of *Cold*, *Warm*, *Hot* and *HotAndMirrored* failover modes. The failover mode provides information about the failover

capabilities the *Server* supports and allows a *Client* to determine the available failover actions described in Table 102.

Cold failovers are for *Servers* where only one *Server* can be active at a time. The *Client* can only connect to one of the *Servers* in the redundant set.

Warm failovers are for *Servers* where the backup *Servers* can be active, but cannot connect to actual data points (typically, a system where the underlying devices are limited to a single connection). The *ServiceLevel Variable* defined in IEC 62541-5 indicates the ability of the *Server* to provide its data to the *Client*. The *Client* shall use the *Server* with the highest service level. The designation of the active *Server* is vendor specific.

Hot failovers are for *Servers* where more than one *Server* can be active and fully operational. The *ServiceLevel Variable* defined in IEC 62541-5 should be used by the *Client* to find the *Servers* with the highest service level to achieve load balancing.

HotAndMirrored failovers are for *Servers* that are mirroring their internal states to all *Servers* in the redundant set and more than one *Server* can be active and fully operational. Mirroring state minimally includes *Sessions*, *Subscriptions*, registered *Nodes*, sequence numbers and sent *Notifications*. This allows *Clients* to fail over without creating a new context for communication. The *ServiceLevel Variable* defined in IEC 62541-5 should be used by the *Client* to find the *Servers* with the highest service level to achieve load balancing. This failover mode is similar to the transparent redundancy. The advantage is that the *Client* has full control over selecting the *Server*. The disadvantage is that the *Client* needs to be able to handle failovers.

Each *Server* maintains a list of *ServerUris* for all redundant *Servers* in the redundant set. The list is provided together with the failover mode in the *ServerRedundancy Object* defined in IEC 62541-5. To enable clients to connect to all *Servers* in the list, each *Server* in the list shall provide the *ApplicationDescription* for all *Servers* in the redundant set through the *FindServers Service*. This information is needed by the *Client* to translate the *ServerUri* into information needed to connect to the other *Servers* in the redundant set. Therefore a *Client* needs to know only one of the redundant *Servers* to find the other *Servers* based on the provided information.

Table 102 defines the list of failover actions.

Table 102 – Redundancy failover actions

Failover mode and <i>Client</i> options	Cold	Warm	Hot (1)	Hot (2)	HotAndMirrored
On initial connection in addition to actions on active server:					
Connect to more than one OPC UA <i>Server</i> .		X	X	X	Optional for status check
Creating <i>Subscriptions</i> and adding monitored items.		X	X	X	
Activating sampling on the <i>Subscriptions</i> .			X	X	
Activate publishing.				X	
At Failover:					
CreateSecureChannel to backup OPC UA <i>Server</i>	X				X
CreateSession on backup OPC UA <i>Server</i>	X				
ActivateSession on backup OPC UA <i>Server</i>	X				X
Creating <i>Subscriptions</i> and adding monitored items.	X				
Activating sampling on the <i>Subscriptions</i> .	X	X			
Activate publishing.	X	X	X		

A *Client* connected to a *Cold* failover redundant set can only connect to one *Server*. For failover the *Client* connects to a backup *Server*, creates *Subscriptions* and *MonitoredItems* and activates publishing. There is a loss of data from the time the connection to the active *Server* is interrupted until the time the *Client* gets *Publish Responses* from the backup *Server*.

A *Client* connected to a *Warm* failover redundant set can connect and create *Subscriptions* and *MonitoredItems* on more than one *Server*. Sampling and publishing can only be activated

on one *Server*. This one *Server* can be found by reading the *ServiceLevel Variable* from all *Servers*. The *Server* with the highest *ServiceLevel* is used. For failover the *Client* activates sampling and publishing on the *Server* with the highest *ServiceLevel*. There is a loss of data from the time the connection to the active *Server* is interrupted until the time the *Client* gets *Publish Responses* from the backup *Server*.

A *Client* connected to a *Hot* failover redundant set has two options to ensure that there is no loss of data in a failover scenario.

- *Hot (1)*
The *Client* creates *MonitoredItems* and is sampling on more than one *Server* but activates publishing only on the *Server* with the highest *ServiceLevel*. The *Client* should setup the queue size for the *MonitoredItems* such that it can buffer all changes during the failover time. The failover time is the time between the connection interruption and the time the *Client* gets *Publish Responses* from the backup *Server*.
- *Hot (2)*
The *Client* receives the same information from more than one *Server* by sampling and publishing on more than one *Server*. This mode ensures that there is no interruption in the data stream and no data loss but the *Client* should ensure that it detects duplicate data and it shall process two data streams.

A *Client* connected to a *HotAndMirrored* failover redundant set can assume that the *Servers* are mirroring their state, including connection information and queues. Therefore the *Client* just connects to one *Server* and in case of a failover re-establishes its connection using a different *Server*. In order to validate the capability to connect to other redundant *Servers* it is allowed to create *Sessions* with other *Servers* and maintain the open connections by periodically reading the *ServiceLevel*. A *Client* shall not create *Subscriptions* on the backup *Servers* for status monitoring. Failover works as described in 6.5. The only difference is that the new *SecureChannel* is created on another *Server* with the highest *ServiceLevel*. *Sessions* and *Subscriptions* can be reused since they are mirrored to all *Servers*.

A *Client* can always use a lesser mode than the server supports. For example, the *Server* supports *Hot* failover mode and the *Client* can use the *Warm* actions. In the case of failover mode *HotAndMirrored*, the *Client* shall not use a lesser mode as this would generate unnecessary load on the *Servers*.

6.4.2.4 Hiding Failover with a Server Proxy (Informative)

A vendor can use the non-transparent redundancy features to create a *Server* proxy running on the *Client* machine that makes redundancy transparent to *Clients*. This reduces the amount of functionality that shall be designed into the *Client* and to enable simpler *Clients* to take advantage of non-transparent redundancy. The *Server* proxy simply duplicates *Subscriptions* and modifications to *Subscriptions*, by passing the calls on to both *Servers*, but only enabling publishing and sampling on one *Server*. When the proxy detects a failure, it enables publishing and/or sampling on the backup *Server*, just as the *Client* would if it were a redundancy-aware *Client*. The proxy shall run on the *Client* machine to avoid another single point of failure.

Figure 27 shows the *Server* proxy used to provide redundancy to non redundancy aware *Clients*.

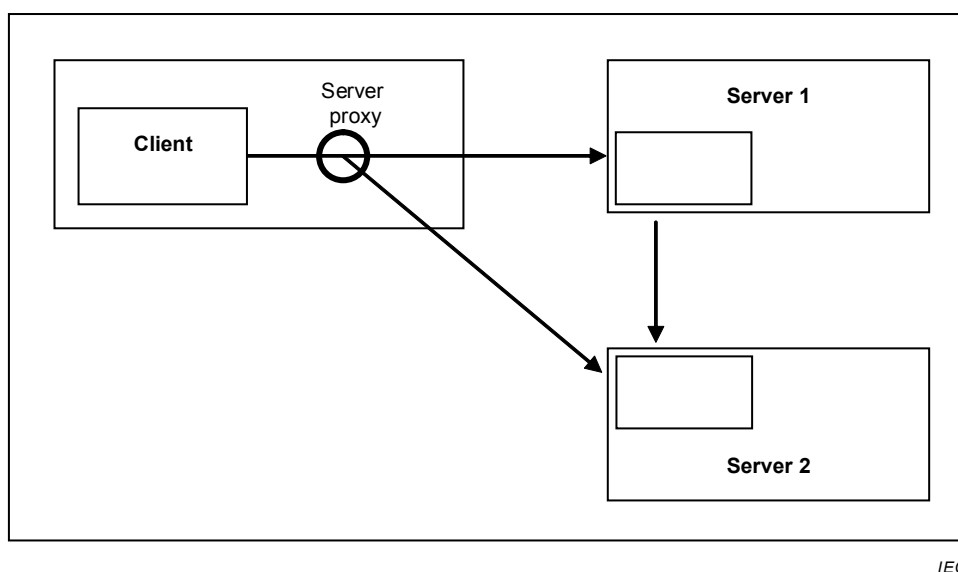


Figure 27 – Server proxy for redundancy

6.4.3 Client redundancy

Client redundancy is supported in OPC UA by the *TransferSubscriptions* call and by exposing *Client* information in the *Server* information structures. Since *Subscription* lifetime is not tied to the *Session* in which it was created, backup *Clients* can monitor the active *Client*'s *Session* with the *Server*, just as they would monitor any other data variable. If the active *Client* ceases to be active, the *Server* shall send a data update to any *Client* which has that variable monitored. Upon receiving such notification, a backup *Client* would then instruct the *Server* to transfer the *Subscriptions* to its own session. If the *Subscription* is crafted carefully, with sufficient resources to buffer data during the change-over, there need be no data loss from a *Client* failover.

OPC UA does not provide a standardized mechanism for conveying the *SessionId* and *SubscriptionIds* from the active *Client* to the backup *Clients*, but as long as the backup *Clients* know the *Client* name of the active *Client*, this information is readily available using the *SessionDiagnostics* and *SubscriptionDiagnostics* portions of the *ServerDiagnostics* data.

6.4.4 Network redundancy

Redundant networks can be used with OPC UA either transparent or non-transparent.

In the transparent case a single *Server Endpoint* can be reached through different network paths. This case is completely handled by the network infrastructure. The selected network path and failover are transparent to the *Client*.

In the non-transparent case the *Server* provides different *Endpoints* for the different network paths. This requires both the *Server* and the *Client* to support multiple network connections. In this case the *Client* is responsible for selecting the *Endpoint* and for failover. For failover the normal reconnect scenario described in 6.5 can be used. Only the *SecureChannel* is created with another *Endpoint*. *Sessions* and *Subscriptions* can be reused.

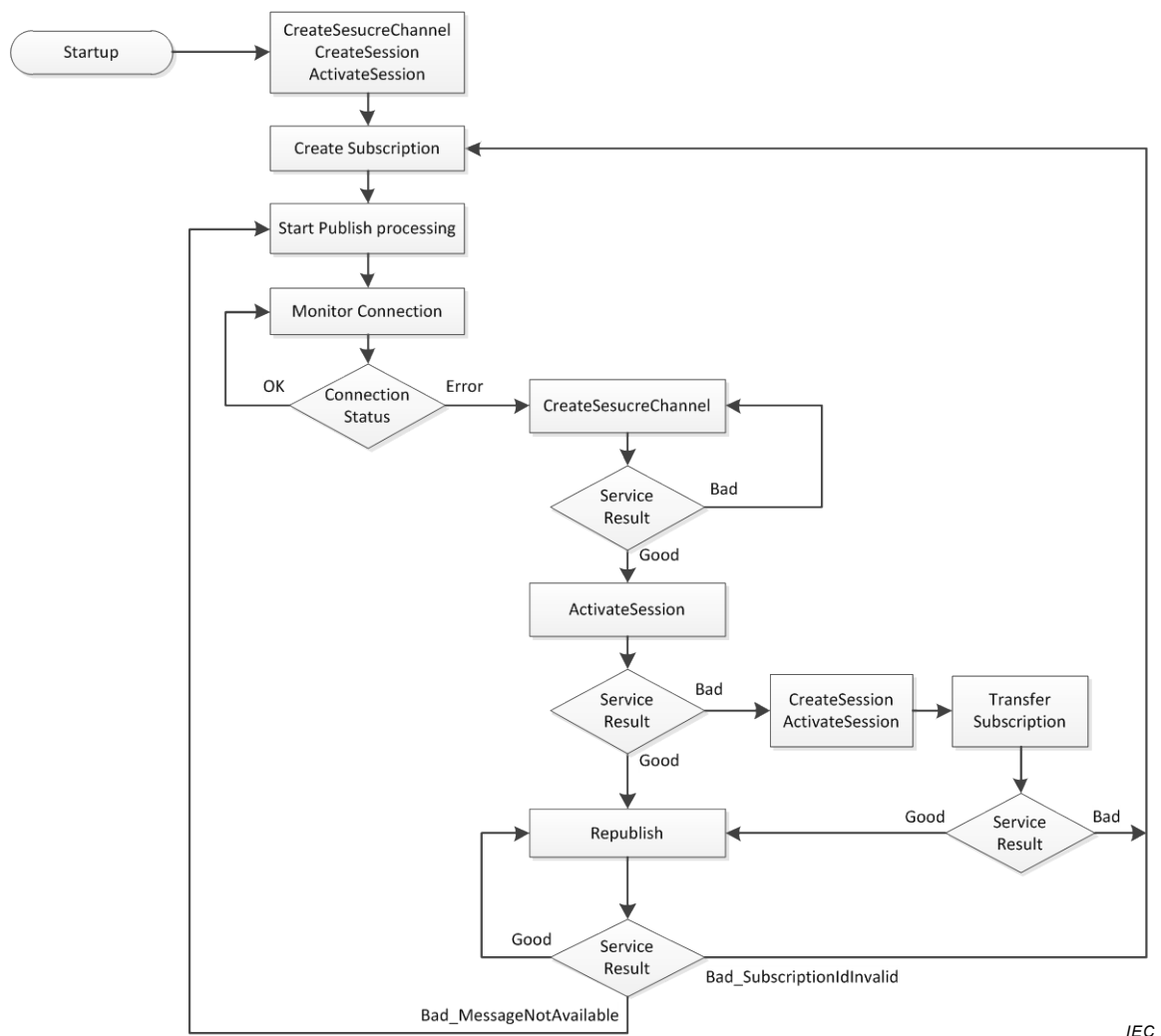
The information about the different network paths is specified in *NonTransparentRedundancyType ObjectType* defined in IEC 62541-5.

Network redundancy can be combined with server redundancy.

6.5 Re-establishing connections

After a *Client* establishes a connection to a *Server* and creates a *Subscription*, the *Client* monitors the connection status. Figure 28 shows the steps to connect a *Client* to a *Server* and the general logic for reconnect handling. Not all possible error scenarios are covered.

The preferred mechanism for a *Client* to monitor the connection status is through the keep-alive of the *Subscription*. A *Client* should subscribe for the *State Variable* in the *ServerStatus* to detect shutdown or other failure states. If no *Subscription* is created or the *Server* does not support *Subscriptions*, the connection can be monitored by periodically reading the *State Variable*.



IEC

Figure 28 – Reconnect Sequence

When a *Client* loses the connection to the *Server*, the goal is to reconnect without losing information. To do this the *Client* shall re-establish the connection by creating a new *SecureChannel* and activating the *Session* with the *Service ActivateSession*. This assigns the new *SecureChannel* to the existing *Session* and allows the *Client* to reuse the *Session* and *Subscriptions* in the *Server*. This will result in the *Client* receiving data and event *Notifications* without losing information provided the queues in the *MonitoredItems* do not overflow.

The *Client* shall only create a new *Session* if *ActivateSession* fails. *TransferSubscriptions* is used to transfer the *Subscription* to the new *Session*. If *TransferSubscriptions* fails, the *Client* needs to create a new *Subscription*.

When the connection is lost, *Publish* responses may have been sent but not received by the *Client*.

After re-establishing the connection the *Client* shall call *Republish* in a loop, starting with the next expected sequence number and incrementing the sequence number until the *Server* returns the status *Bad_MessageNotAvailable*. After receiving this status, the *Client* shall start sending *Publish* requests with the normal *Publish* handling. This sequence ensures that the lost *NotificationMessages* queued in the *Server* are not overwritten by new *Publish* responses.

If the *Client* detects missing sequence numbers in the *Publish* and is not able to get the lost *NotificationMessages* through *Republish*, the *Client* should read the values of all data *MonitoredItems* to make sure the *Client* has the latest values for all *MonitoredItems*.

Independent of the detailed recovery strategy, the *Client* should make sure that it does not overwrite newer data in the *Client* with older values provided through *Republish*.

If the *Republish* returns *Bad_SubscriptionIdInvalid*, then the *Client* needs to create a new *Subscription*.

7 Common parameter type definitions

7.1 ApplicationDescription

The components of this parameter are defined in Table 103.

Table 103 – ApplicationDescription

Name	Type	Description
ApplicationDescription	structure	Specifies an application that is available.
applicationUri	String	The globally unique identifier for the application instance. This URI is used as <i>ServerUri</i> in <i>Services</i> if the application is a <i>Server</i> .
productUri	String	The globally unique identifier for the product.
applicationName	LocalizedText	A localized descriptive name for the application.
applicationType	Enum ApplicationType	The type of application. This value is an enumeration with one of the following values: SERVER_0 The application is a <i>Server</i> . CLIENT_1 The application is a <i>Client</i> . CLIENTANDSERVER_2 The application is a <i>Client</i> and a <i>Server</i> . DISCOVERYSERVER_3 The application is a <i>DiscoveryServer</i> .
gatewayServerUri	String	A URI that identifies the <i>Gateway Server</i> associated with the <i>discoveryUrls</i> . This value is not specified if the <i>Server</i> can be accessed directly. This field is not used if the <i>applicationType</i> is CLIENT_1.
discoveryProfileUri	String	A URI that identifies the discovery profile supported by the URLs provided. This field is not used if the <i>applicationType</i> is CLIENT_1. If this value is not specified then the Endpoints shall support the Discovery Services defined in 5.4. Alternate discovery profiles are defined in IEC 62541-7.
discoveryUrls []	String	A list of URLs for the discovery <i>Endpoints</i> provided by the application. If the <i>applicationType</i> is CLIENT_1, this field shall contain an empty list.

7.2 ApplicationInstanceCertificate

An *ApplicationInstanceCertificate* is a *ByteString* containing an encoded *Certificate*. The encoding of an *ApplicationInstanceCertificate* depends on the security technology mapping and is defined completely in IEC 62541-6. Table 104 specifies the information that shall be contained in an *ApplicationInstanceCertificate*.

Table 104 – ApplicationInstanceCertificate

Name	Type	Description
ApplicationInstanceCertificate	structure	<i>ApplicationInstanceCertificate</i> with signature created by a <i>Certificate Authority</i> .
version	String	An identifier for the version of the <i>Certificate</i> encoding.
serialNumber	ByteString	A unique identifier for the <i>Certificate</i> assigned by the Issuer.
signatureAlgorithm	String	The algorithm used to sign the <i>Certificate</i> . The syntax of this field depends on the <i>Certificate</i> encoding.
signature	ByteString	The signature created by the Issuer.
issuer	Structure	A name that identifies the <i>Issuer Certificate</i> used to create the signature.
validFrom	UtcTime	When the <i>Certificate</i> becomes valid.
validTo	UtcTime	When the <i>Certificate</i> expires.
subject	Structure	A name that identifies the application instance that the <i>Certificate</i> describes. This field shall contain the <i>productName</i> and the name of the organization responsible for the application instance.
applicationUri	String	The <i>applicationUri</i> specified in the <i>ApplicationDescription</i> . The <i>ApplicationDescription</i> is described in 7.1.
hostnames []	String	The name of the machine where the application instance runs. A machine may have multiple names if is accessible via multiple networks. The hostname may be a numeric network address or a descriptive name. <i>Server Certificates</i> shall have at least one hostname defined.
publicKey	ByteString	The public key associated with the <i>Certificate</i> .
keyUsage []	String	Specifies how the <i>Certificate</i> key may be used. <i>ApplicationInstanceCertificates</i> shall support Digital Signature, Non-Repudiation Key Encryption, Data Encryption and Client/Server Authorization. The contents of this field depend on the <i>Certificate</i> encoding.

7.3 BrowseResult

The components of this parameter are defined in Table 105.

Table 105 – BrowseResult

Name	Type	Description
BrowseResult	structure	The results of a Browse operation.
statusCode	StatusCode	The status for the <i>BrowseDescription</i> . This value is set to <i>Good</i> if there are still references to return for the <i>BrowseDescription</i> .
continuationPoint	ContinuationPoint	A <i>Server</i> defined opaque value that identifies the continuation point. The <i>ContinuationPoint</i> type is defined in 7.6.
References []	ReferenceDescription	The set of references that meet the criteria specified in the <i>BrowseDescription</i> . Empty, if no <i>References</i> met the criteria. The Reference Description type is defined in 7.24.

7.4 ContentFilter

7.4.1 ContentFilter structure

The ContentFilter structure defines a collection of elements that define filtering criteria. Each element in the collection describes an operator and an array of operands to be used by the operator. The operators that can be used in a ContentFilter are described in Table 110. The filter is evaluated by evaluating the first entry in the element array starting with the first operand in the operand array. The operands of an element may contain References to sub-elements resulting in the evaluation continuing to the referenced elements in the element array. The evaluation shall not introduce loops. For example evaluation starting from element “A” shall never be able to return to element “A”. However there may be more than one path leading to another element “B”. If an element cannot be traced back to the starting element it is ignored. Extra operands for any operator shall result in an error. Annex B provides examples using the ContentFilter structure.

Table 106 defines the *ContentFilter* structure.

Table 106 – ContentFilter Structure

Name	Type	Description
ContentFilter	structure	
elements []	ContentFilterElement	List of operators and their operands that compose the filter criteria. The filter is evaluated by starting with the first entry in this array. This structure is defined in-line with the following indented items.
filterOperator	Enum FilterOperator	Filter operator to be evaluated. The <i>FilterOperator</i> enumeration is defined in Table 110.
filterOperands []	Extensible Parameter FilterOperand	Operands used by the selected operator. The number and use depend on the operators defined in Table 110. This array needs at least one entry. This extensible parameter type is the <i>FilterOperand</i> parameter type specified in 7.4.4. It specifies the list of valid <i>FilterOperand</i> values.

7.4.2 ContentFilterResult

The components of this data type are defined in Table 107.

Table 107 – ContentFilterResult Structure

Name	Type	Description
ContentFilterResult	structure	A structure that contains any errors associated with the filter.
elementResults []	ContentFilter ElementResult	A list of results for individual elements in the filter. The size and order of the list matches the size and order of the elements in the <i>ContentFilter</i> parameter. This structure is defined in-line with the following indented items.
statusCode	StatusCode	The status code for a single element.
operandStatusCodes []	StatusCode	A list of status codes for the operands in an element. The size and order of the list matches the size and order of the operands in the <i>ContentFilterElement</i> . This list is empty if no operand errors occurred.
operandDiagnosticInfos []	DiagnosticInfo	A list of diagnostic information for the operands in an element. The size and order of the list matches the size and order of the operands in the <i>ContentFilterElement</i> . This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the operands.
elementDiagnosticInfos []	DiagnosticInfo	A list of diagnostic information for individual elements in the filter. The size and order of the list matches the size and order of the elements in the <i>filter</i> request parameter. This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the elements.

Table 108 defines values for the *statusCode* parameter that are specific to this structure. Common *StatusCodes* are defined in Table 166.

Table 108 – ContentFilterResult Result Codes

Symbolic Id	Description
Bad_FilterOperandCountMismatch	The number of operands provided for the filter operator was less than expected for the operand provided.
Bad_FilterOperatorInvalid	An unrecognized operator was provided in a filter.
Bad_FilterOperatorUnsupported	A valid operator was provided, but the server does not provide support for this filter operator.

Table 109 defines values for the *operandStatusCode* parameter that are specific to this structure. Common *StatusCodes* are defined in Table 166.

Table 109 – ContentFilterResult Operand Result Codes

Symbolic Id	Description
Bad_FilterOperandInvalid	See Table 166 for the description of this result code.
Bad_FilterElementInvalid	The referenced element is not a valid element in the content filter.
Bad_FilterLiteralInvalid	The referenced literal is not a valid BaseDataType.
Bad_AttributeIdInvalid	The attribute id is not a valid attribute id in the system.
Bad_IndexRangeInvalid	See Table 166 for the description of this result code.
Bad_NodeIdInvalid	See Table 166 for the description of this result code.
Bad_NodeIdUnknown	See Table 166 for the description of this result code.
Bad_NotTypeDefinition	The provided NodeId was not a type definition NodeId.

7.4.3 FilterOperator

Table 110 defines the basic operators that can be used in a *ContentFilter*. See Table 111 for a description of advanced operators. See 7.4.4 for a definition of operands.

Table 110 – Basic FilterOperator Definition

Operator	Number of Operands	Description
Equals_0	2	TRUE if operand[0] is equal to operand[1]. If the operands are of different types, the system shall perform any implicit conversion to a common type. This operator resolves to FALSE if no implicit conversion is available and the operands are of different types. This operator returns FALSE if the implicit conversion fails. See the discussion on data type precedence in Table 114 for more information how to convert operands of different types.
IsNull_1	1	TRUE if operand[0] is a null value.
GreaterThan_2	2	TRUE if operand[0] is greater than operand[1]. The following restrictions apply to the operands: [0]: Any operand that resolves to an ordered value. [1]: Any operand that resolves to an ordered value. The same conversion rules as defined for <i>Equals</i> apply.
LessThan_3	2	TRUE if operand[0] is less than operand[1]. The same conversion rules and restrictions as defined for <i>GreaterThan</i> apply.
GreaterThanOrEqual_4	2	TRUE if operand[0] is greater than or equal to operand[1]. The same conversion rules and restrictions as defined for <i>GreaterThan</i> apply.
LessThanOrEqual_5	2	TRUE if operand[0] is less than or equal to operand[1]. The same conversion rules and restrictions as defined for <i>GreaterThan</i> apply.
Like_6	2	TRUE if operand[0] matches a pattern defined by operand[1]. See Table 112 for the definition of the pattern syntax. The following restrictions apply to the operands: [0]: Any operand that resolves to a String. [1]: Any operand that resolves to a String. This operator resolves to FALSE if no operand can be resolved to a string.
Not_7	1	TRUE if operand[0] is FALSE. The following restrictions apply to the operands: [0]: Any operand that resolves to a Boolean. If the operand cannot be resolved to a Boolean, the result is a NULL. See below for a discussion on the handling of NULL.
Between_8	3	TRUE if operand[0] is greater or equal to operand[1] and less than or equal to operand[2]. The following restrictions apply to the operands: [0]: Any operand that resolves to an ordered value. [1]: Any operand that resolves to an ordered value. [2]: Any operand that resolves to an ordered value. If the operands are of different types, the system shall perform any implicit conversion to match all operands to a common type. If no implicit conversion is available and the operands are of different types, the particular result is FALSE. See the discussion on data type precedence in Table 114 for more information how to convert operands of different types.
InList_9	2..n	TRUE if operand[0] is equal to one or more of the remaining operands. The Equals Operator is evaluated for operand[0] and each remaining operand in the list. If any Equals evaluation is TRUE, InList returns TRUE.
And_10	2	TRUE if operand[0] and operand[1] are TRUE. The following restrictions apply to the operands: [0]: Any operand that resolves to a Boolean.

Operator	Number of Operands	Description
		[1]: Any operand that resolves to a Boolean. If any operand cannot be resolved to a Boolean it is considered a NULL. See below for a discussion on the handling of NULL.
Or_11	2	TRUE if operand[0] or operand[1] are TRUE. The following restrictions apply to the operands: [0]: Any operand that resolves to a Boolean. [1]: Any operand that resolves to a Boolean. If any operand cannot be resolved to a Boolean it is considered a NULL. See below for a discussion on the handling of NULL.
Cast_12	2	Converts operand[0] to a value with a data type with a NodeId identified by operand[1]. The following restrictions apply to the operands: [0]: Any operand. [1]: Any operand that resolves to a NodeId or ExpandedNodeId where the <i>Node</i> is of the <i>NodeClass DataType</i> . If there is any error in conversion or in any of the parameters then the Cast Operation evaluates to a NULL. See below for a discussion on the handling of NULL.
BitwiseAnd_16	2	The result is an integer which matches the size of the largest operand and contains a bitwise And operation of the two operands where both have been converted to the same size (largest of the two operands). The following restrictions apply to the operands: [0]: Any operand that resolves to a integer. [1]: Any operand that resolves to a integer. If any operand cannot be resolved to a integer it is considered a NULL. See below for a discussion on the handling of NULL.
BitwiseOr_17	2	The result is an integer which matches the size of the largest operand and contains a bitwise Or operation of the two operands where both have been converted to the same size (largest of the two operands). The following restrictions apply to the operands: [0]: Any operand that resolves to a Integer. [1]: Any operand that resolves to a Integer. If any operand cannot be resolved to a Integer it is considered a NULL. See below for a discussion on the handling of NULL.

Many operands have restrictions on their type. This requires the operand to be evaluated to determine what the type is. In some cases the type is specified in the operand (i.e. a *LiteralOperand*). In other cases the type requires that the value of an attribute be read. An *ElementOperand* evaluates to a Boolean value unless the operator is a Cast or a nested *RelatedTo* operator.

Table 111 defines complex operators that require a target node (i.e. row) to evaluate. These operators shall be re-evaluated for each possible target node in the result set.

Table 111 – Complex FilterOperator Definition

Operator	Number of Operands	Description
InView_13	1	TRUE if the target <i>Node</i> is contained in the <i>View</i> defined by operand[0]. The following restrictions apply to the operands: [0]: Any operand that resolves to a <i>NodeId</i> that identifies a <i>View Node</i> . If operand[0] does not resolve to a <i>NodeId</i> that identifies a <i>View Node</i> , this operation shall always be False.
OfType_14	1	TRUE if the target <i>Node</i> is of type operand[0] or of a subtype of operand[0]. The following restrictions apply to the operands: [0]: Any operand that resolves to a <i>NodeId</i> that identifies an <i>ObjectType</i> or <i>VariableType Node</i> . If operand[0] does not resolve to a <i>NodeId</i> that identifies an <i>ObjectType</i> or <i>VariableType Node</i> , this operation shall always be False.
RelatedTo_15	6	TRUE if the target <i>Node</i> is of type Operand[0] and is related to a <i>NodeId</i> of the type defined in Operand[1] by the <i>Reference</i> type defined in Operand[2]. Operand[0] or Operand[1] can also point to an element <i>Reference</i> where the referred to element is another <i>RelatedTo</i> operator. This allows chaining of relationships (e.g. A is related to B is related to C), where the relationship is defined by the <i>ReferenceType</i> defined in Operand[2]. In this case, the referred to element returns a list of <i>NodeIds</i> instead of TRUE or FALSE. In this case if any errors occur or any of the operands cannot be resolved to an appropriate value, the result of the chained relationship is an empty list of nodes. Operand[3] defines the number of hops for which the relationship should be followed. If Operand[3] is 1, then objects shall be directly related. If a hop is greater than 1, then a <i>NodeId</i> of the type described in Operand[1] is checked for at the depth specified by the hop. In this case, the type of the intermediate <i>Node</i> is undefined, and only the <i>Reference</i> type used to reach the end <i>Node</i> is defined. If the requested number of hops cannot be followed, then the result is FALSE, i.e., an empty <i>Node</i> list. If Operand[3] is 0, the relationship is followed to its logical end in a forward direction and each <i>Node</i> is checked to be of the type specified in Operand[1]. If any <i>Node</i> satisfies this criterion, then the result is TRUE, i.e., the <i>NodeId</i> is included in the sub-list. Operand [4] defines if Operands [0] and [1] should include support for subtypes of the types defined by these operands. A TRUE indicates support for subtypes Operand [5] defines if Operand [2] should include support for subtypes of the reference type. A TRUE indicates support for subtypes. The following restrictions apply to the operands: [0]: Any operand that resolves to a <i>NodeId</i> or <i>ExpandedNodeId</i> that identifies an <i>ObjectType</i> or <i>VariableType Node</i> or a reference to another element which is a <i>RelatedTo</i> operator. [1]: Any operand that resolves to a <i>NodeId</i> or <i>ExpandedNodeId</i> that identifies an <i>ObjectType</i> or <i>VariableType Node</i> or a reference to another element which is a <i>RelatedTo</i> operator. [2]: Any operand that resolves to a <i>NodeId</i> that identifies a <i>ReferenceType Node</i>. [3]: Any operand that resolves to a value implicitly convertible to Uint32. [4]: Any operand that resolves to a value implicitly convertible to a boolean; if this operand does not resolve to a Boolean, then a value of FALSE is used. [5]: Any operand that resolves to a value implicitly convertible to a boolean; if this operand does not resolve to a Boolean, then a value of FALSE is used. If none of the operands [0],[1],[2],[3] resolves to an appropriate value then the result of this operation shall always be False (or an Empty set in the case of a nested <i>RelatedTo</i> operand). See examples for <i>RelatedTo</i> in B.2.

The *RelatedTo* operator can be used to identify if a given type, set as operand[1], is a subtype of another type set as operand[0] by setting operand[2] to the *HasSubtype ReferenceType* and operand[3] to 0.

The *Like* operator can be used to perform wildcard comparisons. Several special characters can be included in the second operand of the *Like* operator. The valid characters are defined in Table 112. The wildcard characters can be combined in a single string (i.e. 'Th[ia][ts]%' would match 'That is fine', 'This is fine', 'That as one', 'This it is', 'Then at any', etc.). The *Like* operator is case sensitive.

Table 112 – Wildcard characters

Special Character	Description
%	Match any string of zero or more characters (i.e. 'main%' would match any string that starts with 'main', '%en%' would match any string that contains the letters 'en' such as 'entail', 'green' and 'content'.) If a '%' sign is intended in a string the list operand can be used (i.e. 5[%] would match '5%').
_	Match any single character (i.e. '_ould' would match 'would', 'could'). If the '_' is intended in a string then the list operand can be used (i.e. 5[_] would match '5_').
\	Escape character allows literal interpretation (i.e. \ is \, \% is %, _ is _)
[]	Match any single character in a list (i.e. 'abc[13-68]' would match 'abc1', 'abc3', 'abc4', 'abc5', 'abc6', and 'abc8'. 'xyz[c-f]' would match 'xyzc', 'xyzd', 'xyze', 'xyzf').
[^]	Not Matching any single character in a list. The ^ shall be the first character inside on the []. (i.e. 'ABC[^13-5]' would NOT match 'ABC1', 'ABC3', 'ABC4', and 'ABC5'. xyz[^dgh] would NOT match 'xyzd', 'xyzg', 'xyzh'.)

Table 113 defines the conversion rules for the operand values. The types are automatically converted if an implicit conversion exists (I). If an explicit conversion exists (E) then type can be converted with the cast operator. If no conversion exists (X) the then types cannot be converted, however, some servers may support application specific explicit conversions. The types used in the table are defined in IEC 62541-3. A data type that is not in the table does not have any defined conversions.

Table 113 – Conversion Rules

Target Type (To)	Source Type (From)	Boolean	Byte	ByteString	DateTime	Double	ExpandedNodeId	Float	Guid	Int16	Int32	Int64	NodeId	SByte	StatusCode	String	LocalizedText	QualifiedName	UInt16	UInt32	UInt64	XmlElement
Boolean		-	I	X	X	I	X	I	X	I	I	I	X	I	X	E	X	X	I	I	I	X
Byte		E	-	X	X	I	X	I	X	I	I	I	X	I	X	E	X	X	I	I	I	X
ByteString		X	X	-	X	X	X	E	X	X	X	X	X	X	X	X	X	X	X	X	X	X
DateTime		X	X	X	-	X	X	X	X	X	X	X	X	X	X	E	X	X	X	X	X	X
Double		E	E	X	X	-	X	E	X	E	E	E	X	E	X	E	X	X	E	E	E	X
ExpandedNodeId		X	X	X	X	X	-	X	X	X	X	X	E	X	X	I	X	X	X	X	X	X
Float		E	E	X	X	I	X	-	X	E	E	E	X	E	X	E	X	X	E	E	E	X
Guid		X	X	E	X	X	X	-	X	X	X	X	X	X	X	E	X	X	X	X	X	X
Int16		E	E	X	X	I	X	I	X	-	I	I	X	E	X	E	X	X	E	I	I	X
Int32		E	E	X	X	I	X	I	X	E	-	I	X	E	E	E	X	X	E	E	I	X
Int64		E	E	X	X	I	X	I	X	E	E	-	X	E	E	E	X	X	E	E	E	X
NodeId		X	X	X	X	X	I	X	X	X	X	X	-	X	X	I	X	X	X	X	X	X
SByte		E	E	X	X	I	X	I	X	I	I	I	X	-	X	E	X	X	I	I	I	X
StatusCode		X	X	X	X	X	X	X	X	X	I	I	X	X	-	X	X	X	E	I	I	X
String		I	I	X	E	I	E	I	I	I	I	I	E	I	X	-	E	E	I	I	I	X
LocalizedText		X	X	X	X	X	X	X	X	X	X	X	X	X	X	I	-	X	X	X	X	X
QualifiedName		X	X	X	X	X	X	X	X	X	X	X	X	X	X	I	I	-	X	X	X	X
UInt16		E	E	X	X	I	X	I	X	I	I	I	X	E	I	E	X	X	-	I	I	X
UInt32		E	E	X	X	I	X	I	X	E	I	I	X	E	E	E	X	X	E	-	I	X
UInt64		E	E	X	X	I	X	I	X	E	E	I	X	E	E	E	X	X	E	E	-	X
XmlElement		X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	-

Arrays of a source type can be converted to arrays of the target type by converting each element. A conversion error for any element causes the entire conversion to fail.

Arrays of length 1 can be implicitly converted to a scalar value of the same type.

Guid, *NodeId* and *ExpandedNodeId* are converted to and from *String* using the syntax defined in IEC 62541-6.

Floating point values are rounded by adding 0.5 and truncating when they are converted to integer values.

Converting a negative value to an unsigned type causes a conversion error. If the conversion fails the result is a null value.

Converting a value that is outside the range of the target type causes a conversion error. If the conversion fails the result is a null value.

ByteString is converted to *String* by formatting the bytes as a sequence of hexadecimal digits.

LocalizedText values are converted to *Strings* by dropping the *Locale*. *Strings* are converted to *LocalizedText* values by setting the *Locale* to “”.

QualifiedName values are converted to *Strings* by dropping the *NamespaceIndex*. *Strings* are converted to *QualifiedName* values by setting the *NamespaceIndex* to 0.

A *StatusCode* can be converted to and from a *UInt32* and *Int32* by copying the bits. Only the top 16-bits of the *StatusCode* are copied when it is converted to and from a *UInt16* or *Int16* value.

Boolean values are converted to '1' when true and '0' when false. Non zero numeric values are converted to true *Boolean* values. Numeric values of 0 are converted to false *Boolean* values. *String* values containing "true", "false", "1" or "0" can be converted to *Boolean* values. Other string values cause a conversion error. In this case *Strings* are case-insensitive.

It is sometimes possible to use implicit casts when operands with different data types are used in an operation. In this situation the precedence rules defined in Table 114 are used to determine which implicit conversion to use. The first data type in the list (top down) has the most precedence. If a data type is not in this table then it cannot be converted implicitly while evaluating an operation.

For example, assume that A = 1,1 (*Float*) and B = 1 (*Int32*) and that these values are used with an *Equals* operator. This operation would be evaluated by casting the *Int32* value to a *Float* since the *Float* data type has more precedence.

Table 114 – Data Precedence Rules

Rank	Data Type
1	Double
2	Float
3	Int64
4	UInt64
5	Int32
6	UInt32
7	StatusCode
8	Int16
9	UInt16
10	SByte
11	Byte
12	Boolean
13	Guid
14	String
15	ExpandedNodeId
16	NodeId
17	LocalizedText
18	QualifiedName

Operands may contain null values (i.e. values which do not exist). When this happens, the element always evaluates to NULL (unless the *IsNull_1* operator has been specified). Table 115 defines how to combine elements that evaluate to NULL with other elements in a logical AND operation.

Table 115 – Logical AND Truth Table

	TRUE	FALSE	NULL
TRUE	TRUE	FALSE	NULL
FALSE	FALSE	FALSE	FALSE
NULL	NULL	FALSE	NULL

Table 116 defines how to combine elements that evaluate to NULL with other elements in a logical OR operation.

Table 116 – Logical OR Truth Table

	TRUE	FALSE	NULL
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	NULL
NULL	TRUE	NULL	NULL

The NOT operator always evaluates to NULL if applied to a NULL operand.

A *ContentFilter* which evaluates to NULL after all elements are evaluated is evaluated as false.

7.4.4 FilterOperand parameters

7.4.4.1 Overview

The *ContentFilter* structure specified in 7.4 defines a collection of elements that makes up filter criteria and contains different types of *FilterOperands*. The *FilterOperand* parameter is an extensible parameter. This parameter is defined in Table 117. The *ExtensibleParameter* type is defined in 7.11.

Table 117 – FilterOperand parameter Typelds

Symbolic Id	Description
Element	Specifies an index into the array of elements. This type is used to build a logic tree of sub-elements by linking the operand of one element to a sub-element.
Literal	Specifies a literal value.
Attribute	Specifies any <i>Attribute</i> of an <i>Object</i> or <i>Variable Node</i> using a <i>Node</i> in the type system and relative path constructed from <i>ReferenceTypes</i> and <i>BrowseNames</i> .
SimpleAttribute	Specifies any <i>Attribute</i> of an <i>Object</i> or <i>Variable Node</i> using a <i>TypeDefinition</i> and a relative path constructed from <i>BrowseNames</i> .

7.4.4.2 ElementOperand

The *ElementOperand* provides the linking to sub-elements within a *ContentFilter*. The link is in the form of an integer that is used to index into the array of elements contained in the *ContentFilter*. An index is considered valid if its value is greater than the element index it is part of and it does not *Reference* a non-existent element. *Clients* shall construct filters in this way to avoid circular and invalid *References*. *Servers* should protect against invalid indexes by verifying the index prior to using it.

Table 118 defines the *ElementOperand* type.

Table 118 – ElementOperand

Name	Type	Description
ElementOperand	structure	ElementOperand value.
index	UInt32	Index into the element array.

7.4.4.3 LiteralOperand

Table 119 defines the *LiteralOperand* type.

Table 119 – LiteralOperand

Name	Type	Description
LiteralOperand	structure	LiteralOperand value.
value	BaseDataType	A literal value.

7.4.4.4 AttributeOperand

Table 120 defines the *AttributeOperand* type.

Table 120 – AttributeOperand

Name	Type	Description
AttributeOperand	structure	Attribute of a <i>Node</i> in the address space.
nodeId	NodeId	<i>NodeId</i> of a <i>Node</i> from the type system.
alias	String	An optional parameter used to identify or refer to an alias. An alias is a symbolic name that can be used to alias this operand and use it in other locations in the filter structure.
browsePath	RelativePath	Browse path relative to the <i>Node</i> identified by the <i>nodeId</i> parameter. See 7.25 for the definition of <i>RelativePath</i> .
attributeId	IntegerId	Id of the <i>Attribute</i> . This shall be a valid <i>AttributeId</i> . The <i>IntegerId</i> is defined in 7.13. The <i>IntegerIds</i> for the <i>Attributes</i> are defined in IEC 62541-6.
indexRange	NumericRange	This parameter is used to identify a single element of an array or a single range of indexes for an array. The first element is identified by index 0 (zero). The <i>NumericRange</i> type is defined in 7.21. This parameter is not used if the specified <i>Attribute</i> is not an array. However, if the specified <i>Attribute</i> is an array and this parameter is not used, then all elements are to be included in the range. The parameter is null if not used.

7.4.4.5 SimpleAttributeOperand

The *SimpleAttributeOperand* is a simplified form of the *AttributeOperand* and all of the rules that apply to the *AttributeOperand* also apply to the *SimpleAttributeOperand*. The examples provided in B.1 only use *AttributeOperand*, however, the *AttributeOperand* can be replaced by a *SimpleAttributeOperand* whenever all *ReferenceTypes* in the *RelativePath* are subtypes of *HierarchicalReferences* and the targets are *Object* or *Variable Nodes* and an *Alias* is not required.

Table 121 defines the *SimpleAttributeOperand* type.

Table 121 – SimpleAttributeOperand

Name	Type	Description
SimpleAttributeOperand	structure	Attribute of a <i>Node</i> in the address space.
typeId	NodeId	<i>NodeId</i> of a <i>TypeDefinitionNode</i> . This parameter restricts the operand to instances of the <i>TypeDefinitionNode</i> or one of its subtypes.
browsePath []	QualifiedName	A relative path to a <i>Node</i> . This parameter specifies a relative path using a list of <i>BrowseNames</i> instead of the <i>RelativePath</i> structure used in the <i>AttributeOperand</i> . The list of <i>BrowseNames</i> is equivalent to a <i>RelativePath</i> that specifies forward references which are subtypes of the <i>HierarchicalReferences ReferenceType</i> . All <i>Nodes</i> followed by the <i>browsePath</i> shall be of the <i>NodeClass Object</i> or <i>Variable</i> . If this list is empty the <i>Node</i> is the instance of the <i>TypeDefinition</i> .
attributeId	IntegerId	Id of the <i>Attribute</i> . The <i>IntegerId</i> is defined in 7.13. The <i>Value Attribute</i> shall be supported by all <i>Servers</i> . The support of other <i>Attributes</i> depends on requirements set in Profiles or other parts of this specification.
indexRange	NumericRange	This parameter is used to identify a single element of an array, or a single range of indexes for an array. The first element is identified by index 0 (zero). This parameter is ignored if the selected <i>Node</i> is not a <i>Variable</i> or the <i>Value</i> of a <i>Variable</i> is not an array. The parameter is null if not specified. All values in the array are used if this parameter is not specified. The <i>NumericRange</i> type is defined in 7.21.

7.5 Counter

This primitive data type is a UInt32 that represents the value of a counter. The initial value of a counter is specified by its use. Modulus arithmetic is used for all calculations, where the modulus is max value + 1. Therefore,

$$x + y = (x + y) \bmod (\text{max value} + 1)$$

For example:

$$\text{max value} + 1 = 0$$

$$\text{max value} + 2 = 1$$

7.6 ContinuationPoint

A *ContinuationPoint* is used to pause a *Browse* or *QueryFirst* operation and allow it to be restarted later by calling *BrowseNext* or *QueryNext*. Operations are paused when the number of results found exceeds the limits set by either the *Client* or the *Server*.

The *Client* specifies the maximum number of results per operation in the request message. A *Server* shall not return more than this number of results but it may return fewer results. The *Server* allocates a *ContinuationPoint* if there are more results to return. The *Server* shall always return at least one result if it returns a *ContinuationPoint*.

Servers shall support at least one *ContinuationPoint* per *Session*. *Servers* specify a maximum number of *ContinuationPoints* per *Session* in the *ServerCapabilities Object* defined in IEC 62541-5. *ContinuationPoints* remain active until the *Client* retrieves the remaining results, the *Client* releases the *ContinuationPoint* or the *Session* is closed. A *Server* shall automatically free *ContinuationPoints* from prior requests if they are needed to process a new request. The *Server* returns a *Bad_ContinuationPointInvalid* error if a *Client* tries to use a *ContinuationPoint* that has been released.

Requests will often specify multiple operations that may or may not require a *ContinuationPoint*. A *Server* shall process the operations until it runs out of *ContinuationPoints*. Once that happens the *Server* shall return a *Bad_NoContinuationPoints* error for any remaining operations.

A *Client* restarts an operation by passing the *ContinuationPoint* back to the *Server*. *Server* should always be able to reuse the *ContinuationPoint* provided so *Servers* shall never return *Bad_NoContinuationPoints* error when continuing a previously halted operation.

A *ContinuationPoint* is a subtype of the *ByteString* data type.

7.7 DataValue

7.7.1 General

The components of this parameter are defined in Table 122.

Table 122 – DataValue

Name	Type	Description
DataValue	structure	The value and associated information.
value	BaseDataType	The data value. If the <i>StatusCode</i> indicates an error then the value is to be ignored and the <i>Server</i> shall set it to null.
statusCode	StatusCode	The <i>StatusCode</i> that defines with the <i>Server's</i> ability to access/provide the value. The <i>StatusCode</i> type is defined in 7.33
sourceTimestamp	UtcTime	The source timestamp for the value.
sourcePicoSeconds	UInteger	Specifies the number of 10 picoseconds (1,0 e-11 seconds) intervals which shall be added to the sourceTimestamp.
serverTimestamp	UtcTime	The <i>Server</i> timestamp for the value.
serverPicoSeconds	UInteger	Specifies the number of 10 picoseconds (1,0 e-11 seconds) intervals which shall be added to the serverTimestamp.

7.7.2 PicoSeconds

Some applications require high resolution timestamps. The *PicoSeconds* fields allow applications to specify timestamps with a resolution of 10 picoseconds. The actual size of the *PicoSeconds* field depends on the resolution of the *UtcTime DataType*. For example, if the *UtcTime DataType* has a resolution of 100 nanoseconds then the *PicoSeconds* field would have to store values up to 10 000 in order to provide the resolution of 10 picoseconds. The resolution of the *UtcTime DataType* depends on the *Mappings* defined in IEC 62541-6.

7.7.3 SourceTimestamp

The *sourceTimestamp* is used to reflect the timestamp that was applied to a *Variable* value by the data source. Once a value has been assigned a source timestamp, the source timestamp for that value instance never changes. In this context, “value instance” refers to the value received, independent of its actual value.

The *sourceTimestamp* shall be UTC time and should indicate the time of the last change of the *value* or *statusCode*.

The *sourceTimestamp* should be generated as close as possible to the source of the value but the timestamp needs to be set always by the same physical clock. In the case of redundant sources, the clocks of the sources should be synchronised.

If the OPC UA Server receives the *Variable* value from another OPC UA Server, then the OPC UA Server shall always pass the source timestamp without changes. If the source that applies the timestamp is not available, the source timestamp is set to null. For example, if a value could not be read because of some error during processing like invalid arguments passed in the request then the *sourceTimestamp* shall be null.

In the case of a bad or uncertain status *sourceTimestamp* is used to reflect the time that the source recognized the non-good status or the time the Server last tried to recover from the bad or uncertain status.

The *sourceTimestamp* is only returned with a *Value Attribute*. For all other *Attributes* the returned *sourceTimestamp* is set to null.

7.7.4 ServerTimestamp

The *serverTimestamp* is used to reflect the time that the Server received a *Variable* value or knew it to be accurate.

In the case of a bad or uncertain status, *serverTimestamp* is used to reflect the time that the Server received the status or that the Server last tried to recover from the bad or uncertain status.

In the case where the OPC UA Server subscribes to a value from another OPC UA Server, each Server applies its own *serverTimestamp*. This is in contrast to the *sourceTimestamp* in which only the originator of the data is allowed to apply the *sourceTimestamp*.

If the Server subscribes to the value from another Server every ten seconds and the value changes, then the *serverTimestamp* is updated each time a new value is received. If the value does not change, then new values will not be received on the *Subscription*. However, in the absence of errors, the receiving Server applies a new *serverTimestamp* every ten seconds because not receiving a value means that the value has not changed. Thus, the *serverTimestamp* reflects the time at which the Server knew the value to be accurate.

This concept also applies to OPC UA Servers that receive values from exception-based data sources. For example, suppose that a Server is receiving values from an exception-based device, and that

- a) the device is checking values every 0,5 seconds,
- b) the connection to the device is good,
- c) the device sent an update 3 minutes ago with a value of 1,234.

In this case, the *Server* value would be 1,234 and the *serverTimestamp* would be updated every 0,5 seconds after the receipt of the value.

7.7.5 **StatusCode assigned to a value**

The *StatusCode* is used to indicate the conditions under which a *Variable* value was generated, and thereby can be used as an indicator of the usability of the value. The *StatusCode* is defined in 7.33.

Overall condition (severity)

- A *StatusCode* with severity Good means that the value is of good quality.
- A *StatusCode* with severity Uncertain means that the quality of the value is uncertain for reasons indicated by the *SubCode*.
- A *StatusCode* with severity Bad means that the value is not usable for reasons indicated by the *SubCode*.

Rules

- The *StatusCode* indicates the usability of the value. Therefore, It is required that *Clients* minimally check the *StatusCode Severity* of all results, even if they do not check the other fields, before accessing and using the value.
- A *Server*, which does not support status information, shall return a severity code of Good. It is also acceptable for a *Server* to simply return a severity and a non-specific (0) Substatus.
- If the *Server* has no known value – in particular when *Severity* is BAD, it shall return a NULL value.

7.8 **DiagnosticInfo**

The components of this parameter are defined in Table 123.

Table 123 – DiagnosticInfo

Name	Type	Description
DiagnosticInfo	structure	Vendor-specific diagnostic information.
namespaceUri	Int32	The <i>symbolicId</i> is defined within the context of a namespace. This namespace is represented as a string and is conveyed to the <i>Client</i> in the <i>stringTable</i> parameter of the <i>ResponseHeader</i> parameter defined in 7.27. The <i>namespaceIndex</i> parameter contains the index into the <i>stringTable</i> for this string. -1 indicates that no string is specified. The <i>namespaceUri</i> shall not be the standard OPC UA namespace. There are no <i>symbolicIds</i> provided for standard <i>StatusCodes</i> .
symbolicId	Int32	The <i>symbolicId</i> shall be used to identify a vendor-specific error or condition; typically the result of some server internal operation. The maximum length of this string is 32 characters. <i>Servers</i> wishing to return a numeric return code should convert the return code into a string and use this string as <i>symbolicId</i> (e.g., "0xC0040007" or "-4"). This symbolic identifier string is conveyed to the <i>Client</i> in the <i>stringTable</i> parameter of the <i>ResponseHeader</i> parameter defined in 7.27. The <i>symbolicId</i> parameter contains the index into the <i>stringTable</i> for this string. -1 indicates that no string is specified. The <i>symbolicId</i> shall not contain <i>StatusCodes</i> . If the <i>localizedText</i> contains a translation for the description of a <i>StatusCode</i> , the <i>symbolicId</i> is -1.
locale	Int32	The locale part of the vendor-specific localized text describing the symbolic id. This localized text string is conveyed to the <i>Client</i> in the <i>stringTable</i> parameter of the <i>ResponseHeader</i> parameter defined in 7.27. The <i>localizedTextIndex</i> parameter contains the index into the <i>stringTable</i> for this string. -1 indicates that no string is specified.
localizedText	Int32	A vendor-specific localized text string describes the symbolic id. The maximum length of this text string is 256 characters. This localized text string is conveyed to the <i>Client</i> in the <i>stringTable</i> parameter of the <i>ResponseHeader</i> parameter defined in 7.27. The <i>localizedTextIndex</i> parameter contains the index into the <i>stringTable</i> for this string. -1 indicates that no string is specified. The <i>localizedText</i> refers to the <i>symbolicId</i> if present or the string that describes the standard <i>StatusCode</i> if the server provides translations. If the index is -1, the server has no translation to return and the client should use the invariant <i>StatusCode</i> description from the specification.
additionalInfo	String	Vendor-specific diagnostic information.
innerStatusCode	StatusCode	The <i>StatusCode</i> from the inner operation. Many applications will make calls into underlying systems during OPC UA request processing. An OPC UA <i>Server</i> has the option of reporting the status from the underlying system in the diagnostic info.
innerDiagnosticInfo	DiagnosticInfo	The diagnostic info associated with the inner <i>StatusCode</i> .

7.9 EndpointDescription

The components of this parameter are defined in Table 124.

Table 124 – EndpointDescription

Name	Type	Description
EndpointDescription	structure	Describes an <i>Endpoint</i> for a <i>Server</i> .
endpointUri	String	The URL for the <i>Endpoint</i> described.
Server	ApplicationDescription	The description for the <i>Server</i> that the <i>Endpoint</i> belongs to. The <i>ApplicationDescription</i> type is defined in 7.1.
serverCertificate	ApplicationInstanceCertificate	The <i>Application Instance Certificate</i> issued to the <i>Server</i> . The <i>ApplicationInstanceCertificate</i> type is defined in 7.2.
securityMode	Enum MessageSecurityMode	The type of security to apply to the messages. The type <i>MessageSecurityMode</i> type is defined in 7.14. A <i>SecureChannel</i> may have to be created even if the <i>securityMode</i> is NONE. The exact behaviour depends on the mapping used and is described in the IEC 62541-6.
securityPolicyUri	String	The URI for <i>SecurityPolicy</i> to use when securing messages. The set of known URIs and the <i>SecurityPolicies</i> associated with them are defined in IEC 62541-7.
userIdentityTokens []	UserTokenPolicy	The user identity tokens that the <i>Server</i> will accept. The <i>Client</i> shall pass one of the <i>UserIdentityTokens</i> in the <i>ActivateSession</i> request. The <i>UserTokenPolicy</i> type is described in 7.36.
transportProfileUri	String	The URI of the <i>Transport Profile</i> supported by the <i>Endpoint</i> . IEC 62541-7 defines URIs for the <i>Transport Profiles</i> .
securityLevel	Byte	A numeric value that indicates how secure the <i>EndpointDescription</i> is compared to other <i>EndpointDescriptions</i> for the same <i>Server</i> . A value of 0 indicates that the <i>EndpointDescription</i> is not recommended and is only supported for backward compatibility. A higher value indicates better security.

7.10 ExpandedNodeId

The components of this parameter are defined in Table 125. *ExpandedNodeId* allows the namespace to be specified explicitly as a string or with an index in the *Server's* namespace table.

Table 125 – ExpandedNodeId

Name	Type	Description
ExpandedNodeId	structure	The <i>NodeId</i> with the namespace expanded to its string representation.
serverIndex	Index	Index that identifies the <i>Server</i> that contains the <i>TargetNode</i> . This <i>Server</i> may be the local <i>Server</i> or a remote <i>Server</i> . This index is the index of that <i>Server</i> in the local <i>Server's</i> <i>Server</i> table. The index of the local <i>Server</i> in the <i>Server</i> table is always 0. All remote <i>Servers</i> have indexes greater than 0. The <i>Server</i> table is contained in the <i>Server Object</i> in the <i>AddressSpace</i> (see IEC 62541-3 and IEC 62541-5). The <i>Client</i> may read the <i>Server</i> table <i>Variable</i> to access the description of the target <i>Server</i> .
namespaceUri	String	The URI of the namespace. If this parameter is specified then the namespace index is ignored. Subclause 5.4 and IEC 62541-12 ³ describe the discovery mechanism that can be used to resolve URIs into URLs.
namespaceIndex	Index	The index in the <i>Server's</i> namespace table. This parameter shall be 0 and is ignored in the <i>Server</i> if the namespace URI is specified.
identifierType	IdType	Type of the identifier element of the <i>NodeId</i> .
identifier	*	The identifier for a <i>Node</i> in the address space of an OPC UA <i>Server</i> (see <i>NodeId</i> definition in IEC 62541-3).

7.11 ExtensibleParameter

The extensible parameter types can only be extended by additional parts of series of standards.

³ Under consideration.

The *ExtensibleParameter* defines a data structure with two elements. The *parameterTypeId* specifies the data type encoding of the second element. Therefore the second element is specified as "--". The *ExtensibleParameter* base type is defined in Table 126.

Concrete extensible parameters that are common to OPC UA are defined in Clause 7. Additional parts of series of standards can define additional extensible parameter types.

Table 126 – ExtensibleParameter Base Type

Name	Type	Description
ExtensibleParameter	structure	Specifies the details of an extensible parameter type.
parameterTypeId	NodeId	Identifies the data type of the parameter that follows.
parameterData	--	The details for the extensible parameter type.

7.12 Index

This primitive data type is a UInt32 that identifies an element of an array.

7.13 IntegerId

This primitive data type is a UInt32 that is used as an identifier, such as a handle. All values, except for 0, are valid.

7.14 MessageSecurityMode

The *MessageSecurityMode* is an enumeration that specifies what security should be applied to messages exchanges during a Session. The possible values are described in Table 127.

Table 127 – MessageSecurityMode Values

Value	Description
INVALID_0	The MessageSecurityMode is invalid. This value is the default value to avoid an accidental choice of no security is applied.. This choice will always be rejected.
NONE_1	No security is applied.
SIGN_2	All messages are signed but not encrypted.
SIGNANDENCRYPT_3	All messages are signed and encrypted.

7.15 MonitoringParameters

The components of this parameter are defined in Table 128.

Table 128 – MonitoringParameters

Name	Type	Description												
MonitoringParameters	structure	Parameters that define the monitoring characteristics of a <i>MonitoredItem</i> .												
clientHandle	IntegerId	<i>Client</i> -supplied id of the <i>MonitoredItem</i> . This id is used in <i>Notifications</i> generated for the list <i>Node</i> . The <i>IntegerId</i> type is defined in 7.13.												
samplingInterval	Duration	The interval that defines the fastest rate at which the <i>MonitoredItem</i>(s) should be accessed and evaluated. This interval is defined in milliseconds. The value 0 indicates that the <i>Server</i> should use the fastest practical rate. The value -1 indicates that the default sampling interval defined by the publishing interval of the <i>Subscription</i> is requested. A different sampling interval is used if the publishing interval is not a supported sampling interval. Any negative number is interpreted as -1. The sampling interval is not changed if the publishing interval is changed by a subsequent call to the <i>ModifySubscription Service</i>. The <i>Server</i> uses this parameter to assign the <i>MonitoredItems</i> to a sampling interval that it supports. The assigned interval is provided in the <i>revisedSamplingInterval</i> parameter. The <i>Server</i> shall always return a <i>revisedSamplingInterval</i> that is equal or higher than the requested <i>samplingInterval</i>. If the requested <i>samplingInterval</i> is higher than the maximum sampling interval supported by the <i>Server</i>, the maximum sampling interval is returned.												
filter	Extensible Parameter MonitoringFilter	A filter used by the <i>Server</i> to determine if the <i>MonitoredItem</i> should generate a <i>Notification</i> . If not used, this parameter is null. The <i>MonitoringFilter</i> parameter type is an extensible parameter type specified in 7.16. It specifies the types of filters that can be used.												
queueSize	Counter	The requested size of the <i>MonitoredItem</i> queue. The following values have special meaning for data monitored items: <table><thead><tr><th>Value</th><th>Meaning</th></tr></thead><tbody><tr><td>0 or 1</td><td>the server returns the default queue size which shall be 1 as <i>revisedQueueSize</i> for data monitored items. The queue has a single entry, effectively disabling queuing.</td></tr></tbody></table> For values larger than one a first-in-first-out queue is to be used. The <i>Server</i> may limit the size in <i>revisedQueueSize</i>. In the case of a queue overflow, the <i>Overflow</i> bit (flag) in the <i>InfoBits</i> portion of the <i>DataValue statusCode</i> is set in the new value. The following values have special meaning for event monitored items: <table><thead><tr><th>Value</th><th>Meaning</th></tr></thead><tbody><tr><td>0</td><td>the <i>Server</i> returns the default queue size for <i>Event Notifications</i> as <i>revisedQueueSize</i> for event monitored items.</td></tr><tr><td>1</td><td>the <i>Server</i> returns the minimum queue size the <i>Server</i> requires for <i>Event Notifications</i> as <i>revisedQueueSize</i>.</td></tr><tr><td>MaxUInt32</td><td>the <i>Server</i> returns the maximum queue size that the <i>Server</i> can support for <i>Event Notifications</i> as <i>revisedQueueSize</i>.</td></tr></tbody></table> If a <i>Client</i> chooses a value between the minimum and maximum settings of the <i>Server</i> the value shall be returned in the <i>revisedQueueSize</i>. If the requested <i>queueSize</i> is outside the minimum or maximum, the <i>Server</i> shall return the corresponding bounding value. In the case of a queue overflow, an <i>Event</i> of the type <i>EventQueueOverflowEventType</i> is generated.	Value	Meaning	0 or 1	the server returns the default queue size which shall be 1 as <i>revisedQueueSize</i> for data monitored items. The queue has a single entry, effectively disabling queuing.	Value	Meaning	0	the <i>Server</i> returns the default queue size for <i>Event Notifications</i> as <i>revisedQueueSize</i> for event monitored items.	1	the <i>Server</i> returns the minimum queue size the <i>Server</i> requires for <i>Event Notifications</i> as <i>revisedQueueSize</i> .	MaxUInt32	the <i>Server</i> returns the maximum queue size that the <i>Server</i> can support for <i>Event Notifications</i> as <i>revisedQueueSize</i> .
Value	Meaning													
0 or 1	the server returns the default queue size which shall be 1 as <i>revisedQueueSize</i> for data monitored items. The queue has a single entry, effectively disabling queuing.													
Value	Meaning													
0	the <i>Server</i> returns the default queue size for <i>Event Notifications</i> as <i>revisedQueueSize</i> for event monitored items.													
1	the <i>Server</i> returns the minimum queue size the <i>Server</i> requires for <i>Event Notifications</i> as <i>revisedQueueSize</i> .													
MaxUInt32	the <i>Server</i> returns the maximum queue size that the <i>Server</i> can support for <i>Event Notifications</i> as <i>revisedQueueSize</i> .													
discardOldest	Boolean	A boolean parameter that specifies the discard policy when the queue is full and a new <i>Notification</i> is to be queued. It has the following values: <table><tbody><tr><td>TRUE</td><td>the oldest (first) <i>Notification</i> in the queue is discarded. The new <i>Notification</i> is added to the end of the queue.</td></tr><tr><td>FALSE</td><td>the last <i>Notification</i> added to the queue gets replaced with the new <i>Notification</i>.</td></tr></tbody></table>	TRUE	the oldest (first) <i>Notification</i> in the queue is discarded. The new <i>Notification</i> is added to the end of the queue.	FALSE	the last <i>Notification</i> added to the queue gets replaced with the new <i>Notification</i> .								
TRUE	the oldest (first) <i>Notification</i> in the queue is discarded. The new <i>Notification</i> is added to the end of the queue.													
FALSE	the last <i>Notification</i> added to the queue gets replaced with the new <i>Notification</i> .													

7.16 MonitoringFilter parameters

7.16.1 Overview

The *CreateMonitoredItem Service* allows specifying a filter for each *MonitoredItem*. The *MonitoringFilter* is an extensible parameter whose structure depends on the type of item being monitored. The *parameterTypeIds* are defined in Table 129. Other types can be defined by additional parts of this multi-part specification or other specifications based on OPC UA. The *ExtensibleParameter* type is defined in 7.11.

Each *MonitoringFilter* may have an associated *MonitoringFilterResult* structure which returns revised parameters and/or error information to clients in the response. The result structures, when they exist, are described in the section that defines the *MonitoringFilter*.

Table 129 – MonitoringFilter parameterTypeIds

Symbolic Id	Description
DataChangeFilter	The change in a data value that shall cause a <i>Notification</i> to be generated.
EventFilter	If a <i>Notification</i> conforms to the <i>EventFilter</i> , the <i>Notification</i> is sent to the <i>Client</i> .
AggregateFilter	The <i>Aggregate</i> and its intervals when it will be calculated and a <i>Notification</i> is generated.

7.16.2 DataChangeFilter

The *DataChangeFilter* defines the conditions under which a *DataChange Notification* should be reported and, optionally, a range or band for value changes where no *DataChange Notification* is generated. This range is called *Deadband*. The *DataChangeFilter* is defined in Table 130.

Table 130 – DataChangeFilter

Name	Type	Description
DataChangeFilter	structure	
trigger	Enum DataChangeTrigger	Specifies the conditions under which a data change notification should be reported. It has the following values: STATUS_0 Report a notification ONLY if the <i>StatusCode</i> associated with the value changes. See Table 166 for <i>StatusCodes</i> defined in this standard. IEC 62541-8 specifies additional <i>StatusCodes</i> that are valid in particular for device data. STATUS_VALUE_1 Report a notification if either the <i>StatusCode</i> or the value change. The <i>Deadband</i> filter can be used in addition for filtering value changes. This is the default setting if no filter is set. STATUS_VALUE_TIMESTAMP_2 Report a notification if either <i>StatusCode</i>, value or the <i>SourceTimestamp</i> change. If a <i>Deadband</i> filter is specified, this trigger has the same behaviour as STATUS_VALUE_1. If the <i>DataChangeFilter</i> is not applied to the monitored item, STATUS_VALUE_1 is the default reporting behaviour.
deadbandType	UInt32	A value that defines the <i>Deadband</i> type and behaviour. Value <u>deadbandType</u> None_0 No <i>Deadband</i> calculation should be applied. Absolute_1 AbsoluteDeadband (see below) Percent_2 PercentDeadband (This type is specified in IEC 62541-8).
deadbandValue	Double	The <i>Deadband</i> is applied only if * the <i>trigger</i> includes value changes and * the <i>deadbandType</i> is set appropriately. Deadband is ignored if the status of the data item changes. DeadbandType = AbsoluteDeadband: For this type the <i>deadbandValue</i> contains the absolute change in a data value that shall cause a <i>Notification</i> to be generated. This parameter applies only to <i>Variables</i> with any <i>Number</i> data type. An exception that causes a <i>DataChange Notification</i> based on an <i>AbsoluteDeadband</i> is determined as follows: Generate a Notification if (absolute value of (last cached value – current value) > AbsoluteDeadband) The last cached value is defined as the last value pushed to the queue. If the item is an array of values, the entire array is returned if any array element exceeds the <i>AbsoluteDeadband</i> , or the size or dimension of the array changes. DeadbandType = PercentDeadband: This type is specified in IEC 62541-8

The *DataChangeFilter* does not have an associated result structure.

7.16.3 EventFilter

The *EventFilter* provides for the filtering and content selection of *Event Subscriptions*.

If an *Event Notification* conforms to the filter defined by the *where* parameter of the *EventFilter*, then the *Notification* is sent to the *Client*.

Each *Event Notification* shall include the fields defined by the *selectClauses* parameter of the *EventFilter*. The defined *EventTypes* are specified in IEC 62541-5.

The *selectClause* and *whereClause* parameters are specified with the *SimpleAttributeOperand* structure (see 7.4.4.5). This structure requires the *NodeId* of an *EventType* supported by the *Server* and a path to an *InstanceDeclaration*. An *InstanceDeclaration* is a *Node* which can be found by following forward hierarchical references from the fully inherited *EventType* where the *Node* is also the source of a *HasModellingRule* reference. *EventTypes*, *InstanceDeclarations* and *Modelling Rules* are described completely in IEC 62541-3.

In some cases the same *BrowsePath* will apply to multiple *EventTypes*. If the *Client* specifies the *BaseEventType* in the *SimpleAttributeOperand* then the *Server* shall evaluate the *BrowsePath* without considering the *Type*.

Each *InstanceDeclaration* in the path shall be *Object* or *Variable Node*. The final *Node* in the path may be an *Object Node*, however, *Object Nodes* are only available for *Events* which are visible in the *Server's Address Space*.

The *SimpleAttributeOperand* structure allows the *Client* to specify any *Attribute*, however, the *Server* is only required to support the *Value Attribute* for *Variable Nodes* and the *NodeId Attribute* for *Object Nodes*. That said, profiles defined in IEC 62541-7 may make support for additional *Attributes* mandatory.

The *SimpleAttributeOperand* structure is used in the *selectClause* to select the value to return if an *Event* meets the criteria specified by the *whereClause*. A null value is returned in the corresponding event field in the Publish response if the selected *field* is not part of the *Event* or an error was returned in the *selectClauseResults* of the *EventFilterResult*. If the selected *field* is supported but not available at the time of the event notification, the event field shall contain a *StatusCode* that indicates the reason for the unavailability. For example, the *Server* shall set the event field to *Bad_UserAccessDenied* if the value is not accessible to the user associated with the *Session*. If a *Value Attribute* has an uncertain or bad *StatusCode* associated with it then the *Server* shall provide the *StatusCode* instead of the *Value Attribute*. The *Server* shall set the event field to *Bad_EncodingLimitsExceeded* if a value exceeds the *maxResponseMessageSize*. The *EventId*, *EventType* and *ReceiveTime* cannot contain a *StatusCode* or a null value.

The *Server* shall validate the *selectClauses* when a *Client* creates or updates the *EventFilter*. Any errors which are true for all possible *Events* are returned in the *selectClauseResults* parameter described in Table 132. Some *Servers*, like aggregating *Servers*, may not know all possible *EventTypes* at the time the *EventFilter* is set. These *Servers* do not return errors for unknown *EventTypes* or *BrowsePaths*. The *Server* shall not report errors that might occur depending on the state or the *Server* or type of *Event*. For example, a *selectClause* that requests a single element in an array would always produce an error if the *DataType* of the *Attribute* is a scalar. However, even if the *DataType* is an array an error could occur if the requested index does not exist for a particular *Event*, the *Server* would not report an error in the *selectClauseResults* parameter if the latter situation existed.

The *SimpleAttributeOperand* is used in the *whereClause* to select a value which forms part of a logical expression. These logical expressions are then used to determine whether a particular *Event* should be reported to the *Client*. The *Server* shall use a null value if any error occurs when a *whereClause* is evaluated for a particular *Event*. If a *Value Attribute* has an uncertain or bad *StatusCode* associated with it then the *Server* shall use a null value instead of the *Value*.

Any basic *FilterOperator* in Table 110 may be used in the *whereClause*, however, only the *OfType FilterOperator* from Table 111 is permitted.

The *Server* shall validate the *whereClauses* when a *Client* creates or updates the *EventFilter*. Any structural errors in the construction of the filter and any errors which are true for all possible *Events* are returned in the *whereClauseResult* parameter described in Table 132. Errors that could occur depending on the state of the *Server* or the *Event* are not reported. Some *Servers*, like aggregating *Servers*, may not know all possible *EventTypes* at the time the *EventFilter* is set. These *Servers* do not return errors for unknown *EventTypes* or *BrowsePaths*.

SubscriptionControlEvents are special *Events* which are used to provide control information to the *Client*. These *Events* are only published to the *MonitoredItems* in the *Subscription* that produced the *SubscriptionControlEvent*. These *Events* bypass the *whereClause*.

Table 131 defines the *EventFilter* structure.

Table 131 – EventFilter structure

Name	Type	Description
EventFilter	structure	
selectClauses []	SimpleAttributeOperand	List of the values to return with each <i>Event</i> in a <i>Notification</i> . At least one valid clause shall be specified. See 7.4.4.5 for the definition of <i>SimpleAttributeOperand</i> .
whereClause	ContentFilter	Limit the <i>Notifications</i> to those <i>Events</i> that match the criteria defined by this ContentFilter. The ContentFilter structure is described in 7.4. The <i>AttributeOperand</i> structure may not be used in an <i>EventFilter</i> .

Table 132 defines the *EventFilterResult* structure. This is the *MonitoringFilterResult* associated with the *EventFilter MonitoringFilter*.

Table 132 – EventFilterResult structure

Name	Type	Description
EventFilterResult	structure	
selectClauseResults []	StatusCodes	List of status codes for the elements in the select clause. The size and order of the list matches the size and order of the elements in the <i>selectClauses</i> request parameter. The Server returns null for unavailable or rejected <i>Event</i> fields.
selectClauseDiagnosticInfos []	DiagnosticInfo	A list of diagnostic information for individual elements in the select clause. The size and order of the list matches the size and order of the elements in the <i>selectClauses</i> request parameter. This list is empty if diagnostics information was not requested in the request header or if no diagnostic information was encountered in processing of the select clauses.
whereClauseResult	ContentFilterResult	An results associated with the <i>whereClause</i> request parameter. The <i>ContentFilterResult</i> type is defined in 7.4.2.

Table 133 defines values for the *selectClauseResults* parameter. Common *StatusCodes* are defined in Table 166.

Table 133 – EventFilterResult Result Codes

Symbolic Id	Description
Bad_TypeDefinitionInvalid	See Table 166 for the description of this result code. The typeId is not the NodeId for BaseEventType or a subtype of it.
Bad_NodeIdUnknown	See Table 166 for the description of this result code. The browsePath is specified but it will never exist in any Event.
Bad_BrowseNameInvalid	See Table 166 for the description of this result code. The browsePath is specified and contains a null element.
Bad_AttributeIdInvalid	See Table 166 for the description of this result code. The node specified by the browse path will never allow the given attribute id to be returned.
Bad_IndexRangeInvalid	See Table 166 for the description of this result code.
Bad_TypeMismatch	See Table 166 for the description of this result code. The indexRange is valid but the value of the Attribute is never an array.

7.16.4 AggregateFilter

The *AggregateFilter* defines the *Aggregate* function that should be used to calculate the values to be returned. See IEC 62541-13 for details on possible *Aggregate* functions. It specifies a *startTime* of the first *Aggregate* to be calculated. The *samplingInterval* of the *MonitoringAttributes* (see 7.15) defines how the server should internally sample the underlying data source. The *processingInterval* specifies the size of a time-period where the *Aggregate* is calculated. The *queueSize* from the *MonitoringAttributes* specifies the number of processed values that should be kept.

The intention of the *AggregateFilter* is not to read historical data, the *HistoryRead* service should be used for this purpose. However, it is allowed that the *startTime* is set to a time that is in the past when received from the server. The number of *Aggregates* to be calculated in the past should not exceed the *queueSize* defined in the *MonitoringAttributes* since the values exceeding the *queueSize* would directly be discharged and never returned to the client.

The *startTime* and the *processingInterval* can be revised by the server, but the *startTime* should remain in the same boundary ($\text{startTime} + \text{revisedProcessingInterval} * n = \text{revisedStartTime}$). That behaviour simplifies accessing historical values of the *Aggregates* using the same boundaries by calling the *HistoryRead* service. The extensible *Parameter AggregateFilterResult* is used to return the revised values for the *AggregateFilter*.

Some underlying systems may poll data and produce multiple samples with the same value. Other systems may only report changes to the values. The definition for each *Aggregate* type explains how to handle the two different scenarios.

The *MonitoredItem* only reports values for intervals that have completed when the publish timer expires. Unused data is carried over and used to calculate a value returned in the next publish.

The *ServerTimestamp* for each interval shall be the time of the end of the processing interval.

The *AggregateFilter* is defined in Table 134.

Table 134 – AggregateFilter structure

Name	Type	Description
AggregateFilter	structure	
startTime	UtcTime	Beginning of period to calculate the <i>Aggregate</i> the first time. The size of each period used to calculate the <i>Aggregate</i> is defined by the <i>samplingInterval</i> of the <i>MonitoringAttributes</i> (see 7.15).
aggregateType	NodeId	The <i>NodeId</i> of the <i>AggregateFunctionType</i> object that indicates the <i>Aggregate</i> to be used when retrieving processed data. See IEC 62541-13 for details.
processingInterval	Duration	The period be used to compute the <i>Aggregate</i> .
aggregateConfiguration	Aggregate Configuration	This parameter allows <i>Clients</i> to override the <i>Aggregate</i> configuration settings supplied by the <i>AggregateConfiguration</i> object on a per monitored item basis. See IEC 62541-13 for more information on <i>Aggregate</i> configurations. If the <i>Server</i> does not support the ability to override the <i>Aggregate</i> configuration settings it shall return a <i>StatusCode</i> of <i>Bad_AggregateListMismatch</i> . This structure is defined in-line with the following indented items.
useServerCapabilities Defaults	Boolean	If value = TRUE use <i>Aggregate</i> configuration settings as outlined by the <i>AggregateConfiguration</i> object. If value=FALSE use configuration settings as outlined in the following <i>aggregateConfiguration</i> parameters. Default is TRUE.
treatUncertainAsBad	Boolean	As described in IEC 62541-13.
percentDataBad	Byte	As described in IEC 62541-13.
percentDataGood	Byte	As described in IEC 62541-13.
useSloped Extrapolation	Boolean	As described in IEC 62541-13.

The *AggregateFilterResult* defines the revised *AggregateFilter* the server can return when an *AggregateFilter* is defined for a *MonitoredItem* in the *CreateMonitoredItems* or *ModifyMonitoredItems* services. The *AggregateFilterResult* is defined in Table 135. This is the *MonitoringFilterResult* associated with the *AggregateFilter* *MonitoringFilter*.

Table 135 – AggregateFilterResult structure

Name	Type	Description
AggregateFilterResult	structure	
revisedStartTime	UtcTime	The actual StartTime interval that the Server shall use. This value is based on a number of factors, including capabilities of the server to access historical data. The revisedStartTime should remain in the same boundary as the startTime (startTime + samplingInterval * n = revisedStartTime).
revisedProcessingInterval	Duration	The actual processingInterval that the Server shall use. The revisedProcessingInterval shall be at least twice the revisedSamplingInterval for the MonitoredItem.

7.17 MonitoringMode

The *MonitoringMode* is an enumeration that specifies whether sampling and reporting are enabled or disabled for a *MonitoredItem*. The value of the publishing enabled parameter for a *Subscription* does not affect the value of the monitoring mode for a *MonitoredItem* of the *Subscription*. The values of this parameter are defined in Table 136.

Table 136 – MonitoringMode Values

Value	Description
DISABLED_0	The item being monitored is not sampled or evaluated, and <i>Notifications</i> are not generated or queued. <i>Notification</i> reporting is disabled.
SAMPLING_1	The item being monitored is sampled and evaluated, and <i>Notifications</i> are generated and queued. <i>Notification</i> reporting is disabled.
REPORTING_2	The item being monitored is sampled and evaluated, and <i>Notifications</i> are generated and queued. <i>Notification</i> reporting is enabled.

7.18 NodeAttributes parameters

7.18.1 Overview

The *AddNodes Service* allows specifying the *Attributes* for the *Nodes* to add. The *NodeAttributes* is an extensible parameter whose structure depends on the type of the *Attribute* being added. It identifies the *NodeClass* that defines the structure of the *Attributes* that follow. The *parameterTypeIds* are defined in Table 137. The *ExtensibleParameter* type is defined in 7.11.

Table 137 – NodeAttributes parameterTypeIds

Symbolic Id	Description
ObjectAttributes	Defines the <i>Attributes</i> for the <i>Object NodeClass</i> .
VariableAttributes	Defines the <i>Attributes</i> for the <i>Variable NodeClass</i> .
MethodAttributes	Defines the <i>Attributes</i> for the <i>Method NodeClass</i> .
ObjectTypeAttributes	Defines the <i>Attributes</i> for the <i>ObjectType NodeClass</i> .
VariableTypeAttributes	Defines the <i>Attributes</i> for the <i>VariableType NodeClass</i> .
ReferenceTypeAttributes	Defines the <i>Attributes</i> for the <i>ReferenceType NodeClass</i> .
DataTypeAttributes	Defines the <i>Attributes</i> for the <i>DataType NodeClass</i> .
ViewAttributes	Defines the <i>Attributes</i> for the <i>View NodeClass</i> .

Table 138 defines the bit mask used in the *NodeAttributes* parameters to specify which *Attributes* are set by the *Client*.

Table 138 – Bit mask for specified Attributes

Field	Bit	Description
AccessLevel	0	Indicates if the AccessLevel Attribute is set.
ArrayDimensions	1	Indicates if the ArrayDimensions Attribute is set.
Reserved	2	Reserved to be consistent with WriteMask defined in IEC 62541-3.
ContainsNoLoops	3	Indicates if the ContainsNoLoops Attribute is set.
DataType	4	Indicates if the DataType Attribute is set.
Description	5	Indicates if the Description Attribute is set.
DisplayName	6	Indicates if the DisplayName Attribute is set.
EventNotifier	7	Indicates if the EventNotifier Attribute is set.
Executable	8	Indicates if the Executable Attribute is set.
Historizing	9	Indicates if the Historizing Attribute is set.
InverseName	10	Indicates if the InverseName Attribute is set.
IsAbstract	11	Indicates if the IsAbstract Attribute is set.
MinimumSamplingInterval	12	Indicates if the MinimumSamplingInterval Attribute is set.
Reserved	13	Reserved to be consistent with WriteMask defined in IEC 62541-3.
Reserved	14	Reserved to be consistent with WriteMask defined in IEC 62541-3.
Symmetric	15	Indicates if the Symmetric Attribute is set.
UserAccessLevel	16	Indicates if the UserAccessLevel Attribute is set.
UserExecutable	17	Indicates if the UserExecutable Attribute is set.
UserWriteMask	18	Indicates if the UserWriteMask Attribute is set.
ValueRank	19	Indicates if the ValueRank Attribute is set.
WriteMask	20	Indicates if the WriteMask Attribute is set.
Value	21	Indicates if the Value Attribute is set.
Reserved	22:32	Reserved for future use. Shall always be zero.

7.18.2 ObjectAttributes parameter

Table 139 defines the *ObjectAttributes* parameter.

Table 139 – ObjectAttributes

Name	Type	Description
ObjectAttributes	structure	Defines the <i>Attributes</i> for the <i>Object NodeClass</i> .
specifiedAttributes	UInt32	A bit mask that indicates which fields contain valid values. A field shall be ignored if the corresponding bit is set to 0. The bit values are defined in Table 138.
displayName	LocalizedText	See IEC 62541-3 for the description of this <i>Attribute</i> .
description	LocalizedText	See IEC 62541-3 for the description of this <i>Attribute</i> .
eventNotifier	Byte	See IEC 62541-3 for the description of this <i>Attribute</i> .
writeMask	UInt32	See IEC 62541-3 for the description of this <i>Attribute</i> .
userWriteMask	UInt32	See IEC 62541-3 for the description of this <i>Attribute</i> .

7.18.3 VariableAttributes parameter

Table 140 defines the *VariableAttributes* parameter.

Table 140 – VariableAttributes

Name	Type	Description
VariableAttributes	structure	Defines the <i>Attributes</i> for the <i>Variable NodeClass</i>
specifiedAttributes	UInt32	A bit mask that indicates which fields contain valid values. A field shall be ignored if the corresponding bit is set to 0. The bit values are defined in Table 138.
displayName	LocalizedText	See IEC 62541-3 for the description of this <i>Attribute</i> .
description	LocalizedText	See IEC 62541-3 for the description of this <i>Attribute</i> .
value	Defined by the <i>DataType Attribute</i>	See IEC 62541-3 for the description of this <i>Attribute</i> .
dataType	NodeId	See IEC 62541-3 for the description of this <i>Attribute</i> .
valueRank	Int32	See IEC 62541-3 for the description of this <i>Attribute</i> .
arrayDimensions	UInt32 []	See IEC 62541-3 for the description of this <i>Attribute</i> .
accessLevel	Byte	See IEC 62541-3 for the description of this <i>Attribute</i> .
userAccessLevel	Byte	See IEC 62541-3 for the description of this <i>Attribute</i> .
minimumSamplingInterval	Duration	See IEC 62541-3 for the description of this <i>Attribute</i> .
historizing	Boolean	See IEC 62541-3 for the description of this <i>Attribute</i> .
writeMask	UInt32	See IEC 62541-3 for the description of this <i>Attribute</i> .
userWriteMask	UInt32	See IEC 62541-3 for the description of this <i>Attribute</i> .

7.18.4 MethodAttributes parameter

Table 141 defines the *MethodAttributes* parameter.

Table 141 – MethodAttributes

Name	Type	Description
BaseAttributes	structure	Defines the <i>Attributes</i> for the <i>Method NodeClass</i>
specifiedAttributes	UInt32	A bit mask that indicates which fields contain valid values. A field shall be ignored if the corresponding bit is set to 0. The bit values are defined in Table 138.
displayName	LocalizedText	See IEC 62541-3 for the description of this <i>Attribute</i> .
description	LocalizedText	See IEC 62541-3 for the description of this <i>Attribute</i> .
executable	Boolean	See IEC 62541-3 for the description of this <i>Attribute</i> .
userExecutable	Boolean	See IEC 62541-3 for the description of this <i>Attribute</i> .
writeMask	UInt32	See IEC 62541-3 for the description of this <i>Attribute</i> .
userWriteMask	UInt32	See IEC 62541-3 for the description of this <i>Attribute</i> .

7.18.5 ObjectTypeAttributes parameter

Table 142 defines the *ObjectTypeAttributes* parameter.

Table 142 – ObjectTypeAttributes

Name	Type	Description
ObjectTypeAttributes	structure	Defines the <i>Attributes</i> for the <i>ObjectType NodeClass</i> .
specifiedAttributes	UInt32	A bit mask that indicates which fields contain valid values. A field shall be ignored if the corresponding bit is set to 0. The bit values are defined in Table 138.
displayName	LocalizedText	See IEC 62541-3 for the description of this <i>Attribute</i> .
description	LocalizedText	See IEC 62541-3 for the description of this <i>Attribute</i> .
isAbstract	Boolean	See IEC 62541-3 for the description of this <i>Attribute</i> .
writeMask	UInt32	See IEC 62541-3 for the description of this <i>Attribute</i> .
userWriteMask	UInt32	See IEC 62541-3 for the description of this <i>Attribute</i> .

7.18.6 VariableTypeAttributes parameter

Table 143 defines the *VariableTypeAttributes* parameter.

Table 143 – VariableTypeAttributes

Name	Type	Description
VariableTypeAttributes	structure	Defines the <i>Attributes</i> for the <i>VariableType NodeClass</i> .
specifiedAttributes	UInt32	A bit mask that indicates which fields contain valid values. A field shall be ignored if the corresponding bit is set to 0. The bit values are defined in Table 138.
displayName	LocalizedText	See IEC 62541-3 for the description of this <i>Attribute</i> .
description	LocalizedText	See IEC 62541-3 for the description of this <i>Attribute</i> .
value	Defined by the <i>DataType Attribute</i>	See IEC 62541-3 for the description of this <i>Attribute</i> .
dataType	NodeId	See IEC 62541-3 for the description of this <i>Attribute</i> .
valueRank	Int32	See IEC 62541-3 for the description of this <i>Attribute</i> .
arrayDimensions	UInt32 []	See IEC 62541-3 for the description of this <i>Attribute</i> .
isAbstract	Boolean	See IEC 62541-3 for the description of this <i>Attribute</i> .
writeMask	UInt32	See IEC 62541-3 for the description of this <i>Attribute</i> .
userWriteMask	UInt32	See IEC 62541-3 for the description of this <i>Attribute</i> .

7.18.7 ReferenceTypeAttributes parameter

Table 144 defines the *ReferenceTypeAttributes* parameter.

Table 144 – ReferenceTypeAttributes

Name	Type	Description
ReferenceTypeAttributes	structure	Defines the <i>Attributes</i> for the <i>ReferenceType NodeClass</i> .
specifiedAttributes	UInt32	A bit mask that indicates which fields contain valid values. A field shall be ignored if the corresponding bit is set to 0. The bit values are defined in Table 138.
displayName	LocalizedText	See IEC 62541-3 for the description of this <i>Attribute</i> .
description	LocalizedText	See IEC 62541-3 for the description of this <i>Attribute</i> .
isAbstract	Boolean	See IEC 62541-3 for the description of this <i>Attribute</i> .
symmetric	Boolean	See IEC 62541-3 for the description of this <i>Attribute</i> .
inverseName	LocalizedText	See IEC 62541-3 for the description of this <i>Attribute</i> .
writeMask	UInt32	See IEC 62541-3 for the description of this <i>Attribute</i> .
userWriteMask	UInt32	See IEC 62541-3 for the description of this <i>Attribute</i> .

7.18.8 DataTypeAttributes parameter

Table 145 defines the *DataTypeAttributes* parameter.

Table 145 – DataTypeAttributes

Name	Type	Description
DataTypeAttributes	structure	Defines the <i>Attributes</i> for the <i>DataType NodeClass</i> .
specifiedAttributes	UInt32	A bit mask that indicates which fields contain valid values. A field shall be ignored if the corresponding bit is set to 0. The bit values are defined in Table 138.
displayName	LocalizedText	See IEC 62541-3 for the description of this <i>Attribute</i> .
description	LocalizedText	See IEC 62541-3 for the description of this <i>Attribute</i> .
isAbstract	Boolean	See IEC 62541-3 for the description of this <i>Attribute</i> .
writeMask	UInt32	See IEC 62541-3 for the description of this <i>Attribute</i> .
userWriteMask	UInt32	See IEC 62541-3 for the description of this <i>Attribute</i> .

7.18.9 ViewAttributes parameter

Table 146 defines the *ViewAttributes* parameter.

Table 146 – ViewAttributes

Name	Type	Description
ViewAttributes	structure	Defines the <i>Attributes</i> for the <i>View NodeClass</i> .
specifiedAttributes	UInt32	A bit mask that indicates which fields contain valid values. A field shall be ignored if the corresponding bit is set to 0. The bit values are defined in Table 138.
displayName	LocalizedText	See IEC 62541-3 for the description of this <i>Attribute</i> .
description	LocalizedText	See IEC 62541-3 for the description of this <i>Attribute</i> .
containsNoLoops	Boolean	See IEC 62541-3 for the description of this <i>Attribute</i> .
eventNotifier	Byte	See IEC 62541-3 for the description of this <i>Attribute</i> .
writeMask	UInt32	See IEC 62541-3 for the description of this <i>Attribute</i> .
userWriteMask	UInt32	See IEC 62541-3 for the description of this <i>Attribute</i> .

7.19 NotificationData parameters

7.19.1 Overview

The *NotificationMessage* structure used in the *Subscription Service* set allows specifying different types of *NotificationData*. The *NotificationData* parameter is an extensible parameter whose structure depends on the type of *Notification* being sent. This parameter is defined in Table 147. Other types can be defined by additional parts of series of standards or other specifications based on OPC UA. The *ExtensibleParameter* type is defined in 7.11.

There may be multiple notifications for a single *MonitoredItem* in a single *NotificationData* structure. When that happens the *Server* shall ensure the notifications appear in the same order that they were queued in the *MonitoredItem*. These notifications do not need to appear as a contiguous block.

Table 147 – NotificationData parameterTypeIds

Symbolic Id	Description
DataChange	<i>Notification</i> data parameter used for data change <i>Notifications</i> .
Event	<i>Notification</i> data parameter used for <i>Event Notifications</i> .
StatusChange	<i>Notification</i> data parameter used for Subscription status change <i>Notifications</i> .

7.19.2 DataChangeNotification parameter

Table 148 defines the *NotificationData* parameter used for data change notifications. This structure contains the monitored data items that are to be reported. Monitored data items are reported under two conditions:

- if the *MonitoringMode* is set to REPORTING and a change in value or its status (represented by its *StatusCode*) is detected;
- if the *MonitoringMode* is set to SAMPLING, the *MonitoredItem* is linked to a triggering item and the triggering item triggers.

See 5.12 for a description of the *MonitoredItem Service* set, and in particular the *MonitoringItemModel* and the *TriggeringModel*.

After creating a *MonitoredItem*, the current value or status of the monitored *Attribute* shall be queued without applying the filter. If the current value is not available after the first sampling interval the first *Notification* shall be queued after getting the initial value or status from the data source.

Table 148 – DataChangeNotification

Name	Type	Description
DataChangeNotification	structure	Data change <i>Notification</i> data.
monitoredItems []	MonitoredItem Notification	The list of <i>MonitoredItems</i> for which a change has been detected. This structure is defined in-line with the following indented items.
clientHandle	IntegerId	<i>Client</i> -supplied handle for the <i>MonitoredItem</i> . The <i>IntegerId</i> type is defined in 7.13.
Value	DataValue	The <i>StatusCode</i> , value and timestamp(s) of the monitored <i>Attribute</i> depending on the sampling and queuing configuration. If the <i>StatusCode</i> indicates an error then the value is to be ignored. If not every detected change has been returned since the <i>Server's</i> queue buffer for the <i>MonitoredItem</i> reached its limit and had to purge out data and the size of the queue is larger than one, the <i>Overflow</i> bit in the <i>DataValue InfoBits</i> of the <i>statusCode</i> is set. <i>DataValue</i> is a common type defined in 7.7.
diagnosticInfos []	DiagnosticInfo	List of diagnostic information. The size and order of this list matches the size and order of the <i>monitoredItems</i> parameter. There is one entry in this list for each <i>Node</i> contained in the <i>monitoredItems</i> parameter. This list is empty if diagnostics information was not requested or is not available for any of the <i>MonitoredItems</i> . <i>DiagnosticInfo</i> is a common type defined in 7.8.

7.19.3 EventNotificationList parameter

Table 149 defines the *NotificationData* parameter used for *EventNotifications*.

The *EventNotificationList* defines a table structure that is used to return *Event* fields to a *Client Subscription*. The structure is in the form of a table consisting of one or more *Events*, each containing an array of one or more fields. The selection and order of the fields returned for each *Event* is identical to the selected parameter of the *EventFilter*.

Table 149 – EventNotificationList

Name	Type	Description
EventNotificationList	structure	Event <i>Notification</i> data.
events []	EventFieldList	The list of <i>Events</i> being delivered. This structure is defined in-line with the following indented items.
clientHandle	IntegerId	<i>Client</i> -supplied handle for the <i>MonitoredItem</i> . The <i>IntegerId</i> type is defined in 7.13.
eventFields []	BaseDataType	List of selected <i>Event</i> fields. This shall be a one to one match with the fields selected in the <i>EventFilter</i> . 7.16.3 specifies how the <i>Server</i> shall deal with error conditions.

7.19.4 StatusChangeNotification parameter

Table 150 defines the *NotificationData* parameter used for a *StatusChangeNotification*.

The *StatusChangeNotification* informs the client about a change in the status of a *Subscription*.

Table 150 – StatusChangeNotification

Name	Type	Description
StatusChangeNotification	structure	Event <i>Notification</i> data
status	StatusCode	The <i>StatusCode</i> that indicates the status change.
diagnosticInfo	DiagnosticInfo	Diagnostic information for the status change

7.20 NotificationMessage

The components of this parameter are defined in Table 151.

Table 151 – NotificationMessage

Name	Type	Description
NotificationMessage	structure	The <i>Message</i> that contains one or more <i>Notifications</i> .
sequenceNumber	Counter	The sequence number of the <i>NotificationMessage</i> .
publishTime	UtcTime	The time that this <i>Message</i> was sent to the <i>Client</i> . If this <i>Message</i> is retransmitted to the <i>Client</i> , this parameter contains the time it was first transmitted to the <i>Client</i> .
notificationData []	Extensible Parameter NotificationData	The list of <i>NotificationData</i> structures. The <i>NotificationData</i> parameter type is an extensible parameter type specified in 7.19. It specifies the types of <i>Notifications</i> that can be sent. The <i>ExtensibleParameter</i> type is specified in 7.11. Notifications of the same type should be grouped into one <i>NotificationData</i> element. If a <i>Subscription</i> contains <i>MonitoredItems</i> for events and data, this array should have not more than 2 elements. If the <i>Subscription</i> contains <i>MonitoredItems</i> only for data or only for events, the array size should always be one for this <i>Subscription</i> .

7.21 NumericRange

This parameter is defined in Table 152. A formal BNF definition of the numeric range can be found in Clause A.3.

The syntax for the string contains one of the following two constructs. The first construct is the string representation of an individual integer. For example, “6” is valid, but “6,0” and “3,2” are not. The minimum and maximum values that can be expressed are defined by the use of this parameter and not by this parameter type definition. The second construct is a range represented by two integers separated by the colon (“:”) character. The first integer shall always have a lower value than the second. For example, “5:7” is valid, while “7:5” and “5:5” are not. The minimum and maximum values that can be expressed by these integers are defined by the use of this parameter, and not by this parameter type definition. No other characters, including white-space characters, are permitted.

Multi-dimensional arrays can be indexed by specifying a range for each dimension separated by a ‘,’. For example, a 2x2 block in a 4x4 matrix could be selected with the range “1:2,0:1”. A single element in a multi-dimensional array can be selected by specifying a single number instead of a range. For example, “1,1” specifies selects the [1,1] element in a two dimensional array.

Dimensions are specified in the order that they appear in the *ArrayDimensions Attribute*. All dimensions shall be specified for a *NumericRange* to be valid.

All indexes start with 0. The maximum value for any index is one less than the length of the dimension.

When reading a value, the indexes may not specify a range that is within the bounds of the array. The *Server* shall return a partial result if some elements exist within the range. The *Server* shall return a *Bad_IndexRangeNoData* if no elements exist within the range.

Bad_IndexRangeInvalid is only used for invalid syntax of the *NumericRange*. All other invalid requests with a valid syntax shall result in *Bad_IndexRangeNoData*.

When writing a value, the size of the array shall match the size specified by the *NumericRange*. The *Server* shall return an error if it cannot write all elements specified by the *Client*.

The *NumericRange* can also be used to specify substrings for *ByteString* and *String* values. Arrays of *ByteString* and *String* values are treated as two dimensional arrays where the final index specifies the substring range within the *ByteString* or *String* value. The entire *ByteString* or *String* value is selected if the final index is omitted.

Table 152 – NumericRange

Name	Type	Description
NumericRange	String	A number or a numeric range. A null string indicates that this parameter is not used.

7.22 QueryDataSet

The components of this parameter are defined in Table 153.

Table 153 – QueryDataSet

Name	Type	Description
QueryDataSet	structure	Data related to a <i>Node</i> returned in a Query response.
nodeId	ExpandedNodeId	The <i>NodeId</i> for this <i>Node</i> description.
typeDefinitionNode	ExpandedNodeId	The <i>NodeId</i> for the type definition for this <i>Node</i> description.
values []	BaseDataType	Values for the selected <i>Attributes</i> . The order of returned items matches the order of the requested items. There is an entry for each requested item for the given <i>TypeDefinitionNode</i> that matches the selected instance, this includes any related nodes that were specified using a relative path from the selected instance's <i>TypeDefinitionNode</i> . If no values were found for a given requested item a null value is returned for that item. If a value has a bad status, the <i>StatusCode</i> is returned instead of the value. If multiple values exist for a requested item then an array of values is returned. If the requested item is a reference then a <i>ReferenceDescription</i> or array of <i>ReferenceDescription</i> is returned for that item. If the <i>QueryDataSet</i> is returned in a <i>QueryNext</i> to continue a list of <i>ReferenceDescription</i> , the <i>values</i> array will have the same size but the other values already returned are null.

7.23 ReadValueId

The components of this parameter are defined in Table 154.

Table 154 – ReadValueId

Name	Type	Description
ReadValueId	structure	Identifier for an item to read or to monitor.
nodeId	NodeId	<i>NodeId</i> of a <i>Node</i> .
attributeId	IntegerId	Id of the <i>Attribute</i> . This shall be a valid <i>Attribute</i> id. The <i>IntegerId</i> is defined in 7.13. The <i>IntegerIds</i> for the <i>Attributes</i> are defined in IEC 62541-6.
indexRange	NumericRange	This parameter is used to identify a single element of an array, or a single range of indexes for arrays. If a range of elements is specified, the values are returned as a composite. The first element is identified by index 0 (zero). The <i>NumericRange</i> type is defined in 7.21. This parameter is null if the specified <i>Attribute</i> is not an array. However, if the specified <i>Attribute</i> is an array, and this parameter is null, then all elements are to be included in the range.
dataEncoding	QualifiedName	This parameter specifies the <i>BrowseName</i> of the <i>DataTypeEncoding</i> that the <i>Server</i> should use when returning the <i>Value Attribute</i> of a <i>Variable</i> . It is an error to specify this parameter for other <i>Attributes</i> . A <i>Client</i> can discover what <i>DataTypeEncodings</i> are available by following the <i>HasEncoding Reference</i> from the <i>DataType Node</i> for a <i>Variable</i> . OPC UA defines <i>BrowseNames</i> which <i>Servers</i> shall recognize even if the <i>DataType Nodes</i> are not visible in the <i>Server</i> address space. These <i>BrowseNames</i> are: Default Binary The default or native binary (or non-XML) encoding. Default XML The default XML encoding. Each <i>DataType</i> shall support at least one of these encodings. <i>DataTypes</i> that do not have a true binary encoding (e.g. they only have a non-XML text encoding) should use the Default Binary name to identify the encoding that is considered to be the default non-XML encoding. <i>DataTypes</i> that support at least one XML-based encoding shall identify one of the encodings as the Default XML encoding. Other standards bodies may define other well-known data encodings that could be supported. If this parameter is not specified then the <i>Server</i> shall choose either the Default Binary or Default XML encoding according to what <i>Message</i> encoding (see IEC 62541-6) is used for the <i>Session</i> . If the <i>Server</i> does not support the encoding that matches the <i>Message</i> encoding then the <i>Server</i> shall choose the default encoding that it does support. If this parameter is specified for a <i>MonitoredItem</i> , the <i>Server</i> shall set the <i>StructureChanged</i> bit in the <i>StatusCode</i> (see 7.33) if the <i>DataTypeEncoding</i> changes. The <i>DataTypeEncoding</i> changes if the <i>DataTypeVersion</i> of the <i>DataTypeDescription</i> or the <i>DataTypeDictionary</i> associated with the <i>DataTypeEncoding</i> changes.

7.24 ReferenceDescription

The components of this parameter are defined in Table 155.

Table 155 – ReferenceDescription

Name	Type	Description
ReferenceDescription	structure	Reference parameters returned for the <i>Browse Service</i> .
referenceTypeId	NodeId	<i>NodeId</i> of the <i>ReferenceType</i> that defines the <i>Reference</i> .
isForward	Boolean	If the value is TRUE, the <i>Server</i> followed a forward <i>Reference</i> . If the value is FALSE, the <i>Server</i> followed an inverse <i>Reference</i> .
nodeId	Expanded NodeId	<i>NodeId</i> of the <i>TargetNode</i> as assigned by the <i>Server</i> identified by the <i>Server</i> index. The <i>ExpandedNodeId</i> type is defined in 7.10. If the <i>serverIndex</i> indicates that the <i>TargetNode</i> is a remote <i>Node</i> , then the <i>nodeId</i> shall contain the absolute namespace URI. If the <i>TargetNode</i> is a local <i>Node</i> the <i>nodeId</i> shall contain the namespace index.
browseName ¹⁾	QualifiedName	The <i>BrowseName</i> of the <i>TargetNode</i> .
displayName	LocalizedText	The <i>DisplayName</i> of the <i>TargetNode</i> .
nodeClass ¹⁾	NodeClass	<i>NodeClass</i> of the <i>TargetNode</i> .
typeDefinition ¹⁾	Expanded NodeId	Type definition <i>NodeId</i> of the <i>TargetNode</i> . Type definitions are only available for the <i>NodeClasses Object</i> and <i>Variable</i> . For all other <i>NodeClasses</i> a null <i>NodeId</i> shall be returned.
¹⁾ If the <i>Server</i> index indicates that the <i>TargetNode</i> is a remote <i>Node</i> , then the <i>browseName</i> , <i>nodeClass</i> and <i>typeDefinition</i> may be null or empty. If they are not, they might not be up to date because the local <i>Server</i> might not continuously monitor the remote <i>Server</i> for changes.		

7.25 RelativePath

The components of this parameter are defined in Table 156.

Table 156 – RelativePath

Name	Type	Description
RelativePath	structure	Defines a sequence of <i>References</i> and <i>BrowseNames</i> to follow.
elements []	RelativePath Element	A sequence of <i>References</i> and <i>BrowseNames</i> to follow. This structure is defined in-line with the following indented items. Each element in the sequence is processed by finding the targets and then using those targets as the starting nodes for the next element. The targets of the final element are the target of the <i>RelativePath</i> .
referenceTypeld	Nodeld	The type of reference to follow from the current node. The current path cannot be followed any further if the referenceTypeld is not available on the Node instance. If not specified then all <i>References</i> are included and the parameter includeSubtypes is ignored.
isInverse	Boolean	Indicates whether the inverse <i>Reference</i> should be followed. The inverse reference is followed if this value is TRUE.
includeSubtypes	Boolean	Indicates whether subtypes of the <i>ReferenceType</i> should be followed. Subtypes are included if this value is TRUE.
targetName	QualifiedName	The <i>BrowseName</i> of the target node. The final element may have an empty <i>targetName</i> . In this situation all targets of the references identified by the referenceTypeld are the targets of the <i>RelativePath</i> . The <i>targetName</i> shall be specified for all other elements. The current path cannot be followed any further if no targets with the specified <i>BrowseName</i> exist.

A *RelativePath* can be applied to any starting *Node*. The targets of the *RelativePath* are the set of *Nodes* that are found by sequentially following the elements in *RelativePath*.

A text format for the *RelativePath* can be found in Clause A.2. This format is used in examples that explain the *Services* that make use of the *RelativePath* structure.

7.26 RequestHeader

The components of this parameter are defined in Table 157.

Table 157 – RequestHeader

Name	Type	Description																																				
RequestHeader	structure	Common parameters for all requests submitted on a <i>Session</i> .																																				
authenticationToken	Session AuthenticationToken	The secret <i>Session</i> identifier used to verify that the request is associated with the <i>Session</i> . The <i>SessionAuthenticationToken</i> type is defined in 7.29.																																				
timestamp	UtcTime	The time the <i>Client</i> sent the request. The parameter is only used for diagnostic and logging purposes in the server.																																				
requestHandle	IntegerId	A <i>requestHandle</i> associated with the request. This client defined handle can be used to cancel the request. It is also returned in the response.																																				
returnDiagnostics	UInt32	A bit mask that identifies the types of vendor-specific diagnostics to be returned in <i>diagnosticInfo</i> response parameters. The value of this parameter may consist of zero, one or more of the following values. No value indicates that diagnostics are not to be returned. <table><tr><th>Bit Value</th><th>Diagnostics to return</th></tr><tr><td>0x0000 0001</td><td>ServiceLevel / SymbolicId</td></tr><tr><td>0x0000 0002</td><td>ServiceLevel / LocalizedText</td></tr><tr><td>0x0000 0004</td><td>ServiceLevel / AdditionalInfo</td></tr><tr><td>0x0000 0008</td><td>ServiceLevel / Inner <i>StatusCode</i></td></tr><tr><td>0x0000 0010</td><td>ServiceLevel / Inner Diagnostics</td></tr><tr><td>0x0000 0020</td><td>OperationLevel / SymbolicId</td></tr><tr><td>0x0000 0040</td><td>OperationLevel / LocalizedText</td></tr><tr><td>0x0000 0080</td><td>OperationLevel / AdditionalInfo</td></tr><tr><td>0x0000 0100</td><td>OperationLevel / Inner <i>StatusCode</i></td></tr><tr><td>0x0000 0200</td><td>OperationLevel / Inner Diagnostics</td></tr></table> Each of these values is composed of two components, <i>level</i> and <i>type</i>, as described below. If none are requested, as indicated by a 0 value, or if no diagnostic information was encountered in processing of the request, then diagnostics information is not returned. Level: <table><tr><td>ServiceLevel</td><td>return diagnostics in the <i>diagnosticInfo</i> of the <i>Service</i>.</td></tr><tr><td>OperationLevel</td><td>return diagnostics in the <i>diagnosticInfo</i> defined for individual operations requested in the <i>Service</i>.</td></tr></table> Type: <table><tr><td>SymbolicId</td><td>return a namespace-qualified, symbolic identifier for an error or condition. The maximum length of this identifier is 32 characters.</td></tr><tr><td>LocalizedText</td><td>return up to 256 bytes of localized text that describes the symbolic id.</td></tr><tr><td>AdditionalInfo</td><td>return a byte string that contains additional diagnostic information, such as a memory image. The format of this byte string is vendor-specific, and may depend on the type of error or condition encountered.</td></tr><tr><td>InnerStatusCode</td><td>return the inner <i>StatusCode</i> associated with the operation or <i>Service</i>.</td></tr><tr><td>InnerDiagnostics</td><td>return the inner diagnostic info associated with the operation or <i>Service</i>. The contents of the inner diagnostic info structure are determined by other bits in the mask. Note that setting this bit could cause multiple levels of nested diagnostic info structures to be returned.</td></tr></table>	Bit Value	Diagnostics to return	0x0000 0001	ServiceLevel / SymbolicId	0x0000 0002	ServiceLevel / LocalizedText	0x0000 0004	ServiceLevel / AdditionalInfo	0x0000 0008	ServiceLevel / Inner <i>StatusCode</i>	0x0000 0010	ServiceLevel / Inner Diagnostics	0x0000 0020	OperationLevel / SymbolicId	0x0000 0040	OperationLevel / LocalizedText	0x0000 0080	OperationLevel / AdditionalInfo	0x0000 0100	OperationLevel / Inner <i>StatusCode</i>	0x0000 0200	OperationLevel / Inner Diagnostics	ServiceLevel	return diagnostics in the <i>diagnosticInfo</i> of the <i>Service</i> .	OperationLevel	return diagnostics in the <i>diagnosticInfo</i> defined for individual operations requested in the <i>Service</i> .	SymbolicId	return a namespace-qualified, symbolic identifier for an error or condition. The maximum length of this identifier is 32 characters.	LocalizedText	return up to 256 bytes of localized text that describes the symbolic id.	AdditionalInfo	return a byte string that contains additional diagnostic information, such as a memory image. The format of this byte string is vendor-specific, and may depend on the type of error or condition encountered.	InnerStatusCode	return the inner <i>StatusCode</i> associated with the operation or <i>Service</i> .	InnerDiagnostics	return the inner diagnostic info associated with the operation or <i>Service</i> . The contents of the inner diagnostic info structure are determined by other bits in the mask. Note that setting this bit could cause multiple levels of nested diagnostic info structures to be returned.
Bit Value	Diagnostics to return																																					
0x0000 0001	ServiceLevel / SymbolicId																																					
0x0000 0002	ServiceLevel / LocalizedText																																					
0x0000 0004	ServiceLevel / AdditionalInfo																																					
0x0000 0008	ServiceLevel / Inner <i>StatusCode</i>																																					
0x0000 0010	ServiceLevel / Inner Diagnostics																																					
0x0000 0020	OperationLevel / SymbolicId																																					
0x0000 0040	OperationLevel / LocalizedText																																					
0x0000 0080	OperationLevel / AdditionalInfo																																					
0x0000 0100	OperationLevel / Inner <i>StatusCode</i>																																					
0x0000 0200	OperationLevel / Inner Diagnostics																																					
ServiceLevel	return diagnostics in the <i>diagnosticInfo</i> of the <i>Service</i> .																																					
OperationLevel	return diagnostics in the <i>diagnosticInfo</i> defined for individual operations requested in the <i>Service</i> .																																					
SymbolicId	return a namespace-qualified, symbolic identifier for an error or condition. The maximum length of this identifier is 32 characters.																																					
LocalizedText	return up to 256 bytes of localized text that describes the symbolic id.																																					
AdditionalInfo	return a byte string that contains additional diagnostic information, such as a memory image. The format of this byte string is vendor-specific, and may depend on the type of error or condition encountered.																																					
InnerStatusCode	return the inner <i>StatusCode</i> associated with the operation or <i>Service</i> .																																					
InnerDiagnostics	return the inner diagnostic info associated with the operation or <i>Service</i> . The contents of the inner diagnostic info structure are determined by other bits in the mask. Note that setting this bit could cause multiple levels of nested diagnostic info structures to be returned.																																					
auditEntryId	String	An identifier that identifies the <i>Client</i>'s security audit log entry associated with this request. An empty string value means that this parameter is not used. The <i>AuditEntryId</i> typically contains who initiated the action and from where it was initiated. The <i>AuditEventId</i> is included in the <i>AuditEvent</i> to allow human readers to correlate an <i>Event</i> with the initiating action. More details of the <i>Audit</i> mechanisms are defined in 6.2 and in IEC 62541-3.																																				
timeoutHint	UInt32	This timeout in milliseconds is used in the <i>Client</i> side <i>Communication Stack</i> to set the timeout on a per-call base. For a <i>Server</i> this timeout is only a hint and can be used to cancel long running operations to free resources. If the <i>Server</i> detects a timeout, he can cancel the operation by sending the <i>Service</i> result <i>Bad_Timeout</i>. The <i>Server</i> should wait at minimum the timeout after he received the request before cancelling the operation. The <i>Server</i> shall check the <i>timeoutHint</i> parameter of a <i>PublishRequest</i> before processing a <i>PublishResponse</i>. If the request timed out, a <i>Bad_Timeout Service</i> result is sent and another <i>PublishRequest</i> is used. The value of 0 indicates no timeout.																																				
additionalHeader	Extensible Parameter AdditionalHeader	Reserved for future use. Applications that do not understand the header should ignore it.																																				

7.27 ResponseHeader

The components of this parameter are defined in Table 158.

Table 158 – ResponseHeader

Name	Type	Description
ResponseHeader	structure	Common parameters for all responses.
timestamp	UtcTime	The time the <i>Server</i> sent the response.
requestHandle	IntegerId	The requestHandle given by the <i>Client</i> to the request.
serviceResult	StatusCode	OPC UA-defined result of the <i>Service</i> invocation. The <i>StatusCode</i> type is defined in 7.33.
serviceDiagnostics	DiagnosticInfo	Diagnostic information for the <i>Service</i> invocation. This parameter is empty if diagnostics information was not requested in the request header. The <i>DiagnosticInfo</i> type is defined in 7.8.
stringTable []	String	There is one string in this list for each unique namespace, symbolic identifier, and localized text string contained in all of the diagnostics information parameters contained in the response (see 7.8). Each is identified within this table by its zero-based index.
additionalHeader	Extensible Parameter AdditionalHeader	Reserved for future use. Applications that do not understand the header should ignore it.

7.28 ServiceFault

The components of this parameter are defined in Table 159.

The *ServiceFault* parameter is returned instead of the *Service* response message when a service level error occurs. The *requestHandle* in the *ResponseHeader* should be set to what was provided in the *RequestHeader* even if these values were not valid. The level of diagnostics returned in the *ResponseHeader* is specified by the *returnDiagnostics* parameter in the *RequestHeader*.

The exact use of this parameter depends on the mappings defined in IEC 62541-6.

Table 159 – ServiceFault

Name	Type	Description
ServiceFault	structure	An error response sent when a service level error occurs.
responseHeader	ResponseHeader	Common response parameters (see 7.27 for <i>ResponseHeader</i> definition).

7.29 SessionAuthenticationToken

The *SessionAuthenticationToken* type is an opaque identifier that is used to identify requests associated with a particular *Session*. This identifier is used in conjunction with the *SecureChannelId* or *Client Certificate* to authenticate incoming messages. It is the secret form of the *sessionId* for internal use in the *Client* and *Server Applications*.

A *Server* returns a *SessionAuthenticationToken* in the *CreateSession* response. The *Client* then sends this value with every request which allows the *Server* to verify that the sender of the request is the same as the sender of the original *CreateSession* request.

For the purposes of this discussion, a *Server* consists of application (code) and a *Communication Stack* as shown in Figure 29. The security provided by the *SessionAuthenticationToken* depends on a trust relationship between the *Server* application and the *Communication Stack*. The *Communication Stack* shall be able to verify the sender of the message and it uses the *SecureChannelId* or the *Client Certificate* to identify the sender to the *Server*. In these cases, the *SessionAuthenticationToken* is a UInt32 identifier that allows the *Server* to distinguish between different *Sessions* created by the same sender.

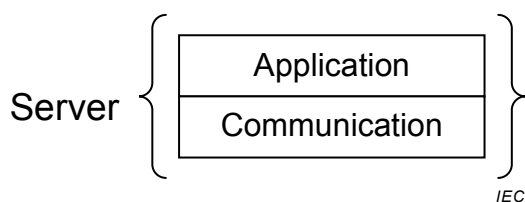
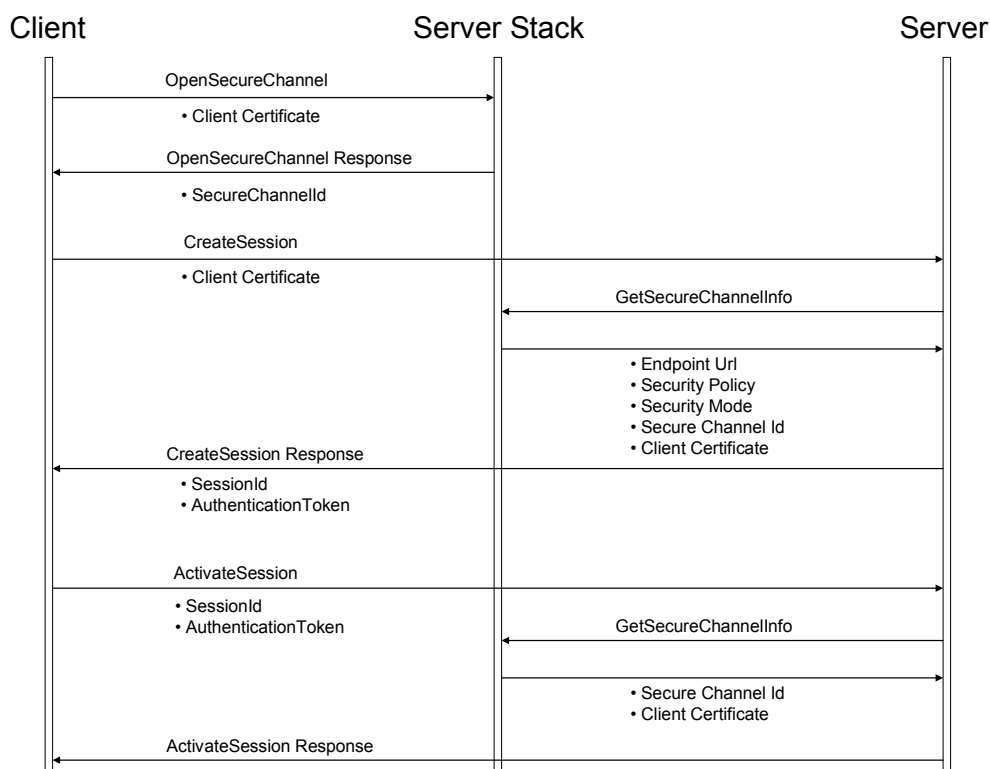


Figure 29 – Logical layers of a Server

In some cases, the application and the *Communication Stack* cannot exchange information at runtime which means the application will not have access to the *SecureChannelId* or the *Certificate* used to create the *SecureChannel*. In these cases the application shall create a random *ByteString* value that is at least 32 bytes long. This value shall be kept secret and shall always be exchanged over a *SecureChannel* with encryption enabled. The Administrator is responsible for ensuring that encryption is enabled. The *Profiles* in IEC 62541-7 may define additional requirements for a *ByteString SessionAuthenticationToken*.

Client and *Server* applications should be written to be independent of the *SecureChannel* implementation. Therefore, they should always treat the *SessionAuthenticationToken* as secret information even if it is not required when using some *SecureChannel* implementations.

Figure 30 illustrates the information exchanged between the Client, the Server and the Server *Communication Stack* when the *Client* obtains a *SessionAuthenticationToken*. In this figure the *GetSecureChannelInfo* step represents an API that depends on the *Communication Stack* implementation.



IEC

Figure 30 – Obtaining a SessionAuthenticationToken

The *SessionAuthenticationToken* is a subtype of the *NodeId* data type; however, it is never used to identify a *Node* in the address space. Servers may assign a value to the *NamespaceIndex*; however, its meaning is *Server* specific.

7.30 SignatureData

The components of this parameter are defined in Table 160.

Table 160 – SignatureData

Name	Type	Description
SignatureData	structure	Contains a digital signature created with a <i>Certificate</i> .
signature	ByteString	This is a signature generated with the private key associated with a <i>Certificate</i> .
algorithm	String	A string containing the URI of the <i>algorithm</i> . The URI string values are defined as part of the security profiles specified in IEC 62541-7.

7.31 SignedSoftwareCertificate

A *SignedSoftwareCertificate* is a *ByteString* containing an encoded *Certificate*. The encoding of a *SignedSoftwareCertificate* depends on the security technology mapping and is defined completely in IEC 62541-6. Table 161 specifies the information that shall be contained in a *SignedSoftwareCertificate*.

Table 161 – SignedSoftwareCertificate

Name	Type	Description
SignedSoftwareCertificate	structure	A <i>SoftwareCertificate</i> with a signature created by Certifying Authority.
version	String	An identifier for the version of the <i>Certificate</i> encoding.
serialNumber	ByteString	A unique identifier for the <i>Certificate</i> assigned by the Issuer.
signatureAlgorithm	String	The algorithm used to sign the <i>Certificate</i> . The syntax of this field depends on the <i>Certificate</i> encoding.
signature	ByteString	The signature created by the Issuer.
issuer	Structure	A name that identifies the <i>Issuer Certificate</i> used to create the signature.
validFrom	UtcTime	When the <i>Certificate</i> becomes valid.
validTo	UtcTime	When the <i>Certificate</i> expires.
subject	Structure	A name that identifies the product which the <i>Certificate</i> describes. This field shall contain the <i>productName</i> and <i>vendorName</i> from the <i>SoftwareCertificate</i> .
subjectAltName []	Structure	A list of alternate names for the product. This list shall include the <i>productUri</i> specified in the <i>SoftwareCertificate</i> .
publicKey	ByteString	The public key associated with the <i>Certificate</i> .
keyUsage []	String	Specifies how the <i>Certificate</i> key may be used. <i>SignedSoftwareCertificate</i> may only be used for creating Digital Signatures and Non-Repudiation. The contents of this field depends on the <i>Certificate</i> encoding.
softwareCertificate	ByteString	The XML encoded form of the <i>SoftwareCertificate</i> stored as UTF8 text. IEC 62541-6 describes the XML representation for the <i>SoftwareCertificate</i> .

7.32 SoftwareCertificate

The components of this parameter are defined in Table 162.

Table 162 – SoftwareCertificate

Name	Type	Description
SoftwareCertificate	Structure	A <i>Certificate</i> describing a product.
productName	String	The name of the product that is certified. This field shall be specified.
productUri	String	A globally unique identifier for the product that is certified. This field shall be specified.
vendorName	String	The name of the vendor responsible for the product. This field shall be specified.
vendorProductCertificate	ByteString	The DER encoded form of the X509 <i>Certificate</i> which is assigned to the product by the vendor. This field may be omitted.
softwareVersion	String	Software version. This field shall be specified.
buildNumber	String	Build number. This field shall be specified.
buildDate	UtcTime	Date and time of the build. This field shall be specified.
issuedBy	String	URI of the certifying authority. This field shall be specified.
issueDate	UtcTime	Specifies when the <i>Certificate</i> was issued by the certifying authority. This field shall be specified.
supportedProfiles []	structure	List of supported <i>Profiles</i> . This structure is defined in-line with the following indented items.
organizationUri	String	A URI that identifies the organization that defined the profile.
profileId	String	A string that identifies the <i>Profile</i> .
complianceTool	String	A string that identifies the tool or certification method used for compliance testing.
complianceDate	UtcTime	Date and time of the compliance test.
complianceLevel	Enum Compliance Level	An enumeration that specifies the compliance level of the <i>Profile</i> . It has the following values: UNTESTED_0 the profiled capability has not been tested successfully PARTIAL_1 the profiled capability has been partially tested and has passed critical tests, as defined by the certifying authority. SELFTESTED_2 the profiled capability has been successfully tested using a self-test system authorized by the certifying authority. CERTIFIED_3 the profiled capability has been successfully tested by a testing organisation authorized by the certifying authority.
unsupportedUnitIds []	String	The identifiers for the optional conformance units that were not tested. See IEC 62541-7 for a detailed explanation.

7.33 StatusCode

7.33.1 General

A *StatusCode* in OPC UA is numerical value that is used to report the outcome of an operation performed by an OPC UA *Server*. This code may have associated diagnostic information that describes the status in more detail; however, the code by itself is intended to provide *Client* applications with enough information to make decisions on how to process the results of an OPC UA *Service*.

The *StatusCode* is a 32-bit unsigned integer. The top 16 bits represent the numeric value of the code that shall be used for detecting specific errors or conditions. The bottom 16 bits are bit flags that contain additional information but do not affect the meaning of the *StatusCode*.

All OPC UA *Clients* shall always check the *StatusCode* associated with a result before using it. Results that have an uncertain/warning status associated with them shall be used with care since these results might not be valid in all situations. Results with a bad/failed status shall never be used.

OPC UA *Servers* should return good/success *StatusCodes* if the operation completed normally and the result is always valid. Different *StatusCode* values can provide additional information to the *Client*.

OPC UA *Servers* should use uncertain/warning *StatusCodes* if they could not complete the operation in the manner requested by the *Client*, however, the operation did not fail entirely.

The exact bit assignments are shown in Table 163.

Table 163 – StatusCode Bit Assignments

Field	Bit Range	Description
Severity	30:31	Indicates whether the <i>StatusCode</i> represents a good, bad or uncertain condition. These bits have the following meanings: Good 00 Indicates that the operation was successful and the associated results may be used. Uncertain 01 Indicates that the operation was partially successful and that associated results might not be suitable for some purposes. Warning 10 Indicates that the operation failed and any associated results cannot be used. Bad Failure 11 Reserved for future use. All <i>Clients</i> should treat a <i>StatusCode</i> with this severity as “Bad”.
Reserved	29:29	Reserved for use in OPC UA application specific APIs. This bit shall always be zero on the wire but may be used by OPC UA application specific APIs for API specific status codes.
Reserved	28:28	Reserved for future use. Shall always be zero.
SubCode	16:27	The code is a numeric value assigned to represent different conditions. Each code has a symbolic name and a numeric value. All descriptions in the OPC UA specification refer to the symbolic name. IEC 62541-6 maps the symbolic names onto a numeric value.
StructureChanged	15:15	Indicates that the structure of the associated data value has changed since the last <i>Notification</i> . <i>Clients</i> should not process the data value unless they re-read the metadata. <i>Servers</i> shall set this bit if the <i>DataTypeEncoding</i> used for a <i>Variable</i> changes. 7.23 describes how the <i>DataTypeEncoding</i> is specified for a <i>Variable</i> . <i>Servers</i> shall also set this bit if the <i>EnumStrings Property</i> of the <i>DataType</i> of the <i>Variable</i> changes. This bit is provided to warn <i>Clients</i> that parse complex data values that their parsing routines could fail because the serialized form of the data value has changed. This bit has meaning only for <i>StatusCodes</i> returned as part of a data change <i>Notification</i> or the <i>HistoryRead</i> . <i>StatusCodes</i> used in other contexts shall always set this bit to zero.
SemanticsChanged	14:14	Indicates that the semantics of the associated data value have changed. <i>Clients</i> should not process the data value until they re-read the metadata associated with the <i>Variable</i> . <i>Servers</i> should set this bit if the metadata has changed in way that could cause application errors if the <i>Client</i> does not re-read the metadata. For example, a change to the engineering units could create problems if the <i>Client</i> uses the value to perform calculations. IEC 62541-8 defines the conditions where a <i>Server</i> shall set this bit for a DA <i>Variable</i> . Other specifications may define additional conditions. A <i>Server</i> may define other conditions that cause this bit to be set. This bit has meaning only for <i>StatusCodes</i> returned as part of a data change <i>Notification</i> or the <i>HistoryRead</i> . <i>StatusCodes</i> used in other contexts shall always set this bit to zero.
Reserved	12:13	Reserved for future use. Shall always be zero.
InfoType	10:11	The type of information contained in the info bits. These bits have the following meanings: NotUsed 00 The info bits are not used and shall be set to zero. DataValue 01 The <i>StatusCode</i> and its info bits are associated with a data value returned from the <i>Server</i> . This flag is only used in combination with <i>StatusCodes</i> defined in IEC 62541-8. Reserved 1X Reserved for future use. The info bits shall be ignored.
InfoBits	0:9	Additional information bits that qualify the <i>StatusCode</i> . The structure of these bits depends on the Info Type field.

Table 164 describes the structure of the *InfoBits* when the Info Type is set to *DataValue* (01).

Table 164 – DataValue InfoBits

Info Type	Bit Range	Description																					
LimitBits	8:9	The limit bits associated with the data value. The limits bits have the following meanings: <table> <tr> <th>Limit</th><th>Bits</th><th>Description</th></tr> <tr> <td>None</td><td>00</td><td>The value is free to change.</td></tr> <tr> <td>Low</td><td>01</td><td>The value is at the lower limit for the data source.</td></tr> <tr> <td>High</td><td>10</td><td>The value is at the higher limit for the data source.</td></tr> <tr> <td>Constant</td><td>11</td><td>The value is constant and cannot change.</td></tr> </table>	Limit	Bits	Description	None	00	The value is free to change.	Low	01	The value is at the lower limit for the data source.	High	10	The value is at the higher limit for the data source.	Constant	11	The value is constant and cannot change.						
Limit	Bits	Description																					
None	00	The value is free to change.																					
Low	01	The value is at the lower limit for the data source.																					
High	10	The value is at the higher limit for the data source.																					
Constant	11	The value is constant and cannot change.																					
Overflow	7	This bit shall only be set if the <i>MonitoredItem</i> queue size is greater than 1. If this bit is set, not every detected change has been returned since the <i>Server's</i> queue buffer for the <i>MonitoredItem</i> reached its limit and had to purge out data.																					
Reserved	5:6	Reserved for future use. Shall always be zero.																					
HistorianBits	0:4	These bits are set only when reading historical data. They indicate where the data value came from and provide information that affects how the <i>Client</i> uses the data value. The historian bits have the following meaning: <table> <tr> <td>Raw</td><td>XXX00</td><td>A raw data value.</td></tr> <tr> <td>Calculated</td><td>XXX01</td><td>A data value which was calculated.</td></tr> <tr> <td>Interpolated</td><td>XXX10</td><td>A data value which was interpolated.</td></tr> <tr> <td>Reserved</td><td>XXX11</td><td>Undefined.</td></tr> <tr> <td>Partial</td><td>XX1XX</td><td>A data value which was calculated with an incomplete interval.</td></tr> <tr> <td>Extra Data</td><td>X1XXX</td><td>A raw data value that hides other data at the same timestamp.</td></tr> <tr> <td>Multi Value</td><td>1XXXX</td><td>Multiple values match the <i>Aggregate</i> criteria (i.e. multiple minimum values at different timestamps within the same interval).</td></tr> </table> IEC 62541-11 describes how these bits are used in more detail.	Raw	XXX00	A raw data value.	Calculated	XXX01	A data value which was calculated.	Interpolated	XXX10	A data value which was interpolated.	Reserved	XXX11	Undefined.	Partial	XX1XX	A data value which was calculated with an incomplete interval.	Extra Data	X1XXX	A raw data value that hides other data at the same timestamp.	Multi Value	1XXXX	Multiple values match the <i>Aggregate</i> criteria (i.e. multiple minimum values at different timestamps within the same interval).
Raw	XXX00	A raw data value.																					
Calculated	XXX01	A data value which was calculated.																					
Interpolated	XXX10	A data value which was interpolated.																					
Reserved	XXX11	Undefined.																					
Partial	XX1XX	A data value which was calculated with an incomplete interval.																					
Extra Data	X1XXX	A raw data value that hides other data at the same timestamp.																					
Multi Value	1XXXX	Multiple values match the <i>Aggregate</i> criteria (i.e. multiple minimum values at different timestamps within the same interval).																					

7.33.2 Common StatusCodes

Table 165 defines the common *StatusCodes* for all *Service* results used in more than one service. It does not provide a complete list. These *StatusCodes* may also be used as operation level result code. IEC 62541-6 maps the symbolic names to a numeric value and provides a complete list of *StatusCodes* including codes defines in other parts.

Table 165 – Common Service Result Codes

Symbolic Id	Description
Good	The operation was successful.
Good_CompletesAsynchronously	The processing will complete asynchronously.
Good_SubscriptionTransferred	The subscription was transferred to another session.
Bad_CertificateHostNameInvalid	The <i>HostName</i> used to connect to a <i>Server</i> does not match a <i>HostName</i> in the <i>Certificate</i> .
Bad_CertificateIssuerRevocationUnknown	It was not possible to determine if the <i>Issuer Certificate</i> has been revoked.
Bad_CertificateIssuerUseNotAllowed	The <i>Issuer Certificate</i> may not be used for the requested operation.
Bad_CertificateIssuerTimeInvalid	An <i>Issuer Certificate</i> has expired or is not yet valid.
Bad_CertificateIssuerRevoked	The <i>Issuer Certificate</i> has been revoked.
Bad_CertificateInvalid	The <i>Certificate</i> provided as a parameter is not valid.
Bad_CertificateRevocationUnknown	It was not possible to determine if the <i>Certificate</i> has been revoked.
Bad_CertificateRevoked	The <i>Certificate</i> has been revoked.
Bad_CertificateTimeInvalid	The <i>Certificate</i> has expired or is not yet valid.
Bad_CertificateUriInvalid	The URI specified in the <i>ApplicationDescription</i> does not match the URI in the <i>Certificate</i> .
Bad_CertificateUntrusted	The <i>Certificate</i> is not trusted.
Bad_CertificateUseNotAllowed	The <i>Certificate</i> may not be used for the requested operation.
Bad_CommunicationError	A low level communication error occurred.
Bad_DataTypeIdUnknown	The extension object cannot be (de)serialized because the data type id is not recognized.
Bad_DecodingError	Decoding halted because of invalid data in the stream.
Bad_EncodingError	Encoding halted because of invalid data in the objects being serialized.
Bad_EncodingLimitsExceeded	The message encoding/decoding limits imposed by the <i>Communication Stack</i> have been exceeded.
Bad_IdentityTokenInvalid	The user identity token is not valid.
Bad_IdentityTokenRejected	The user identity token is valid but the server has rejected it.
Bad_InternalError	An internal error occurred as a result of a programming or configuration error.
Bad_InvalidArgument	One or more arguments are invalid. Each service defines parameter-specific <i>StatusCodes</i> and these <i>StatusCodes</i> shall be used instead of this general error code. This error code shall be used only by the <i>Communication Stack</i> and in services where it is defined in the list of valid <i>StatusCodes</i> for the service.
Bad_InvalidState	The operation cannot be completed because the object is closed, uninitialized or in some other invalid state.
Bad_InvalidTimestamp	The timestamp is outside the range allowed by the server.
Bad_NonceInvalid	The nonce does appear to be not a random value or it is not the correct length.
Bad_NothingToDo	There was nothing to do because the client passed a list of operations with no elements.
Bad_OutOfMemory	Not enough memory to complete the operation.
Bad_RequestCancelledByClient	The request was cancelled by the client.
Bad_RequestTooLarge	The request message size exceeds limits set by the server.
Bad_ResponseTooLarge	The response message size exceeds limits set by the client.
Bad_RequestHeaderInvalid	The header for the request is missing or invalid.
Bad_ResourceUnavailable	An operating system resource is not available.
Bad_SecureChannelIdInvalid	The specified secure channel is not longer valid.
Bad_SecurityChecksFailed	An error occurred while verifying security.
Bad_ServerHalted	The server has stopped and cannot process any requests.
Bad_ServerNotConnected	The operation could not complete because the client is not connected to the server.
Bad_ServerUriInvalid	The <i>Server</i> URI is not valid.
Bad_ServiceUnsupported	The server does not support the requested service.
Bad_SessionIdInvalid	The session id is not valid.
Bad_SessionClosed	The session was closed by the client.
Bad_SessionNotActivated	The session cannot be used because <i>ActivateSession</i> has not been called.
Bad_Shutdown	The operation was cancelled because the application is shutting down.
Bad_SubscriptionIdInvalid	The subscription id is not valid.
Bad_Timeout	The operation timed out.
Bad_TimestampsToReturnInvalid	The timestamps to return parameter is invalid.
Bad_TooManyOperations	The request could not be processed because it specified too many operations.
Bad_UnexpectedError	An unexpected error occurred.
Bad_UnknownResponse	An unrecognized response was received from the server.
Bad_UserAccessDenied	User does not have permission to perform the requested operation.
Bad_ViewIdUnknown	The view id does not refer to a valid view Node.

Symbolic Id	Description
Bad_ViewTimestampInvalid	The view timestamp is not available or not supported.
Bad_ViewParameterMismatchInvalid	The view parameters are not consistent with each other.
Bad_ViewVersionInvalid	The view version is not available or not supported.

Table 166 defines the common *StatusCodes* for all operation level results used in more than one service. It does not provide a complete list. IEC 62541-6 maps the symbolic names to a numeric value and provides a complete list of StatusCodes including codes defines in other parts. The common *Service* result codes can be also contained in the operation level.

Table 166 – Common Operation Level Result Codes

Symbolic Id	Description
Good_Clamped	The value written was accepted but was clamped.
Good_Overload	Sampling has slowed down due to resource limitations.
Uncertain	The value is uncertain but no specific reason is known.
Bad	The value is bad but no specific reason is known.
Bad_AttributeIdInvalid	The attribute is not supported for the specified node.
Bad_BrowseDirectionInvalid	The browse direction is not valid.
Bad_BrowseNameInvalid	The browse name is invalid.
Bad_ContentFilterInvalid	The content filter is not valid.
Bad_ContinuationPointInvalid	The continuation point provided is not longer valid. This status is returned if the continuation point was deleted or the address space was changed between the browse calls.
Bad_DataEncodingInvalid	The data encoding is invalid. This result is used if no <i>dataEncoding</i> can be applied because an <i>Attribute</i> other than <i>Value</i> was requested or the <i>DataType</i> of the <i>Value Attribute</i> is not a subtype of the <i>Structure DataType</i> .
Bad_DataEncodingUnsupported	The server does not support the requested data encoding for the node. This result is used if a <i>dataEncoding</i> can be applied but the passed data encoding is not known to the <i>Server</i> .
Bad_EventFilterInvalid	The event filter is not valid.
Bad_FilterNotAllowed	A monitoring filter cannot be used in combination with the attribute specified.
Bad_FilterOperandInvalid	The operand used in a content filter is not valid.
Bad_HistoryOperationInvalid	The history details parameter is not valid.
Bad_HistoryOperationUnsupported	The server does not support the requested operation.
Bad_IndexRangeInvalid	The syntax of the index range parameter is invalid.
Bad_IndexRangeNoData	No data exists within the range of indexes specified.
Bad_MonitoredItemFilterInvalid	The monitored item filter parameter is not valid.
Bad_MonitoredItemFilterUnsupported	The server does not support the requested monitored item filter.
Bad_MonitoredItemIdInvalid	The monitoring item id does not refer to a valid monitored item.
Bad_MonitoringModelInvalid	The monitoring mode is invalid.
Bad_NoCommunication	Communication with the data source is defined, but not established, and there is no last known value available. This status/sub-status is used for cached values before the first value is received or for Write and Call if the communication is not established.
Bad_NoContinuationPoints	The operation could not be processed because all continuation points have been allocated.
Bad_NodeClassInvalid	The node class is not valid.
Bad_NodeIdInvalid	The syntax of the node id is not valid.
Bad_NodeIdUnknown	The node id refers to a node that does not exist in the server address space.
Bad_NoDeleteRights	The <i>Server</i> will not allow the node to be deleted.
Bad_NodeNotInView	The nodesToBrowse is not part of the view.
Bad_NotFound	A requested item was not found or a search operation ended without success.
Bad_NotImplemented	Requested operation is not implemented.
Bad_NotReadable	The access level does not allow reading or subscribing to the <i>Node</i> .
Bad_NotSupported	The requested operation is not supported.
Bad_NotWritable	The access level does not allow writing to the <i>Node</i> .
Bad_ObjectDeleted	The object cannot be used because it has been deleted.
Bad_OutOfRange	The value was out of range.
Bad_ReferenceTypeIdInvalid	The reference type id does not refer to a valid reference type node.
Bad_SourceNodeIdInvalid	The source node id does not refer to a valid node.
Bad_StructureMissing	A mandatory structured parameter was missing or null.

Symbolic Id	Description
Bad_TargetNodeIdInvalid	The target node id does not refer to a valid node.
Bad_TypeDefinitionInvalid	The type definition node id does not reference an appropriate type node.
Bad_TypeMismatch	The value supplied for the attribute is not of the same type as the attribute's value.
Bad_WaitingForInitialData	Waiting for the server to obtain values from the underlying data source. After creating a <i>MonitoredItem</i> , it may take some time for the server to actually obtain values for these items. In such cases the server can send a <i>Notification</i> with this status prior to the <i>Notification</i> with the first value or status from the data source.

7.34 TimestampsToReturn

The *TimestampsToReturn* is an enumeration that specifies the *Timestamp Attributes* to be transmitted for *MonitoredItems* or *Nodes* in *Read* and *HistoryRead*. The values of this parameter are defined in Table 167.

Table 167 – TimestampsToReturn Values

Value	Description
SOURCE_0	Return the source timestamp.
SERVER_1	Return the <i>Server</i> timestamp.
BOTH_2	Return both the source and <i>Server</i> timestamps.
NEITHER_3	Return neither timestamp. This is the default value for <i>MonitoredItems</i> if a <i>Variable</i> value is not being accessed. For <i>HistoryRead</i> this is not a valid setting.

7.35 UserIdentityToken parameters

7.35.1 Overview

The *UserIdentityToken* structure used in the *Server Service Set* allows *Clients* to specify the identity of the user they are acting on behalf of. The exact mechanism used to identify users depends on the system configuration. The different types of identity tokens are based on the most common mechanisms that are used in systems today. Table 168 defines the current set of user identity tokens. The *ExtensibleParameter* type is defined in 7.11.

Table 168 – UserIdentityToken parameterTypeIds

Symbolic Id	Description
AnonymousIdentityToken	No user information is available.
UserNameIdentityToken	A user identified by user name and password.
X509IdentityToken	A user identified by an X509v3 <i>Certificate</i> .
IssuedIdentityToken	A user identified by a WS-SecurityToken.

The *Client* shall always prove possession of a *UserIdentityToken* when it passes it to the *Server*. Some tokens include a secret such as a password which the *Server* will accept as proof. In order to protect these secrets the *Token* may be encrypted before it is passed to the *Server*. The profiles defined in IEC 62541-7 specify where encryption shall be applied. Other types of tokens allow the *Client* to create a signature with the secret associated with the *Token*. In these cases, the *Client* proves possession of a *UserIdentityToken* by appending the last *ServerNonce* to the *ServerCertificate* and uses the secret to produce a *Signature* which is passed to the *Server*.

Each *UserIdentityToken* allowed by an *Endpoint* shall have a *UserTokenPolicy* specified in the *EndpointDescription*. The *UserTokenPolicy* specifies what *SecurityPolicy* to use when encrypting or signing. If this *SecurityPolicy* is omitted then the *Client* uses the *SecurityPolicy* in the *EndpointDescription*. If the matching *SecurityPolicy* is set to None then no encryption or signature is required. It is recommended that Applications never set the *SecurityPolicy* to None for UserTokens that include a secret because these secrets could be used by an attacker to gain access to the system.

Table 169 describes how to serialize *UserIdentityTokens* before applying encryption.

Table 169 – UserIdentityToken Encrypted Token Format

Name	Type	Description
length	Byte[4]	The length of the encrypted data including the <i>ServerNonce</i> but excluding the <i>length</i> field. This field is a 4 byte unsigned integer encoded with the least significant bytes appearing first.
tokenData	Byte[*]	The token data.
serverNonce	Byte[*]	The last <i>ServerNonce</i> returned by the server in the <i>CreateSession</i> or <i>ActivateSession</i> response.

7.35.2 AnonymousIdentityToken

The *AnonymousIdentityToken* is used to indicate that the Client has no user credentials.

Table 170 defines the *AnonymousIdentityToken* parameter.

Table 170 – AnonymousIdentityToken

Name	Type	Description
AnonymousIdentityToken	structure	An anonymous user identity.
policyId	String	An identifier for the <i>UserTokenPolicy</i> that the token conforms to. The <i>UserTokenPolicy</i> structure is defined in 7.36.

7.35.3 UserNameIdentityToken

The *UserNameIdentityToken* is used to pass simple username/password credentials to the *Server*.

This token shall be encrypted by the *Client* if required by the *Server* in the *SecurityPolicy* of the *UserTokenPolicy* in the *EndpointDescription*. The *Server* should specify a *SecurityPolicy* for the *UserTokenPolicy* if the *SecureChannel* has a *SecurityPolicy* of None and no transport layer encryption is available. If None is specified for the *UserTokenPolicy* and *SecurityPolicy* is None then the password only contains the UTF-8 encoded password. This configuration should not be used unless the network is encrypted in some other manner such as a VPN. The use of this configuration without network encryption would result in a serious security fault, in that it would cause the appearance of a secure user access, but it would make the password visible in clear text.

If the token is encrypted the password shall be converted to a UTF8 *ByteString* and then serialized as shown in Table 169.

The *Server* shall decrypt the password and verify the *ServerNonce*.

If the *SecurityPolicy* is None then the password only contains the UTF-8 encoded password.

Table 171 defines the *UserNameIdentityToken* parameter.

Table 171 – UserNameIdentityToken

Name	Type	Description
UserNameIdentityToken	structure	UserName value.
policyId	String	An identifier for the <i>UserTokenPolicy</i> that the token conforms to. The <i>UserTokenPolicy</i> structure is defined in 7.36.
userName	String	A string that identifies the user.
password	ByteString	The password for the user. The password can be an empty string. This parameter shall be encrypted with the Server's Certificate using the algorithm specified by the <i>SecurityPolicy</i> .
encryptionAlgorithm	String	A string containing the URI of the <i>AsymmetricEncryptionAlgorithm</i> . The URI string values are defined names that may be used as part of the security profiles specified in IEC 62541-7. This parameter is null if the password is not encrypted.

Table 172 describes the dependencies for selecting the *AsymmetricEncryptionAlgorithm* for the *UserNameIdentityToken*. The *SecureChannel SecurityPolicy* URI is specified in the *EndpointDescription* and used in subsequent *OpenSecureChannel* request. The *UserTokenPolicy SecurityPolicy* URI is specified in the *EndpointDescription*. The *encryptionAlgorithm* is specified in the *UserNameIdentityToken* or *IssuedIdentityToken* provided by the *Client* in the *ActivateSession* call.

Table 172 – EncryptionAlgorithm selection

SecureChannel SecurityPolicy	UserTokenPolicy SecurityPolicy	UserNameIdentityToken EncryptionAlgorithm
Security Policy – None	Null	Null
Security Policy – None	Security Policy – None	Null
Security Policy – None	Security Policy – Other	Asymmetric algorithm for "Other"
Security Policy – Other	Null	Asymmetric algorithm for "Other"
Security Policy – Other	Security Policy – Yet another	Asymmetric algorithm for "Yet another"
Security Policy – Other	Security Policy – Other	Asymmetric algorithm for "Other"
Security Policy – Other	Security Policy – None	Null

7.35.4 X509IdentityTokens

The *X509IdentityToken* is used to pass an X509v3 *Certificate* which is issued by the user.

This token shall always be accompanied by a signature in the *userTokenSignature* parameter of *ActivateSession* if required by the *SecurityPolicy*. The *Server* should specify a *SecurityPolicy* for the *UserTokenPolicy* if the *SecureChannel* has a *SecurityPolicy* of None.

Table 173 defines the *X509IdentityToken* parameter.

Table 173 – X509 Identity Token

Name	Type	Description
X509IdentityToken	structure	X509v3 value.
policyId	String	An identifier for the <i>UserTokenPolicy</i> that the token conforms to. The <i>UserTokenPolicy</i> structure is defined in 7.36.
certificateData	ByteString	The X509 <i>Certificate</i> in DER format.

7.35.5 IssuedIdentityToken

The *IssuedIdentityToken* is used to pass WS-Security compliant *SecurityTokens* to the *Server*.

WS-Security defines a number of token profiles that may be used to represent different types of *SecurityTokens*. For example, Kerberos and SAML tokens have WSS token profiles and shall be exchanged in OPC UA as XML Security Tokens.

The WSS X509 and Username tokens should not be exchanged as XML security tokens. OPC UA applications should use the appropriate OPC UA identity tokens to pass the information contained in these types of WSS *SecurityTokens*.

These tokens may be encrypted or require a signature. IEC 62541-7 defines profiles that include user related security; they also include any requirements for encryption and signatures. Additional security profiles specify encryption and signature algorithms.

If the token is encrypted then the XML shall be converted to an UTF8 *ByteString* and then serialized as shown in Table 169.

The *Server* shall decrypt the tokenData and verify the *ServerNonce*.

If the *SecurityPolicy* is None or if the token only requires signing then the tokenData contains the UTF-8 encoded XML representation of the token.

Table 174 defines the *IssuedIdentityToken* parameter.

Table 174 – IssuedIdentityToken

Name	Type	Description
IssuedIdentityToken	structure	WSS value.
policyId	String	An identifier for the <i>UserTokenPolicy</i> that the token conforms to. The <i>UserTokenPolicy</i> structure is defined in 7.36.
tokenData	ByteString	The XML representation of the token. This parameter may be encrypted with the <i>Server's Certificate</i> .
encryptionAlgorithm	String	A string containing the URI of the <i>AsymmetricEncryptionAlgorithm</i> . The list of OPC UA-defined names that may be used is specified in IEC 62541-7. This parameter is null if the token is not encrypted. The <i>encryptionAlgorithm</i> is <i>AsymmetricEncryptionAlgorithm</i> for the security policy identified by the <i>securityPolicyUri</i> for the <i>UserTokenPolicy</i> .

7.36 UserTokenPolicy

The components of this parameter are defined in Table 175.

Table 175 – UserTokenPolicy

Name	Type	Description
UserTokenPolicy	structure	Specifies a <i>UserIdentityToken</i> that a <i>Server</i> will accept.
policyId	String	An identifier for the <i>UserTokenPolicy</i> assigned by the <i>Server</i> . The <i>Client</i> specifies this value when it constructs a <i>UserIdentityToken</i> that conforms to the policy. This value is only unique within the context of a single <i>Server</i> .
tokenType	Enum <i>UserIdentityTokenType</i>	The type of user identity token required. This value is an enumeration with one of the following values: ANONYMOUS_0 No token is required. USERNAME_1 A username/password token. CERTIFICATE_2 An X509v3 <i>Certificate</i> token. ISSUEDTOKEN_3 Any WS-Security defined token. A <i>tokenType</i> of ANONYMOUS indicates that the <i>Server</i> does not require any user identification. In this case the <i>Client Application Instance Certificate</i> is used as the user identification.
issuedTokenType	String	A URI for the type of token. IEC 62541-6 defines URIs for common issued token types. Vendors may specify their own token. This field may only be specified if <i>TokenType</i> is ISSUEDTOKEN_3.
issuerEndpointUrl	String	An optional URL for the token issuing service. The meaning of this value depends on the <i>issuedTokenType</i> .
securityPolicyUri	String	The security policy to use when encrypting or signing the <i>UserIdentityToken</i> when it is passed to the <i>Server</i> in the <i>ActivateSession</i> request. Clause 7.35 describes how this parameter is used. The security policy for the <i>SecureChannel</i> is used if this value is omitted.

7.37 ViewDescription

The components of this parameter are defined in Table 176.

Table 176 – ViewDescription

Name	Type	Description
ViewDescription	structure	Specifies a <i>View</i> .
viewId	NodeId	<i>NodeId</i> of the <i>View</i> to <i>Query</i> . A null value indicates the entire <i>AddressSpace</i> .
timestamp	UtcTime	The time date desired. The corresponding version is the one with the closest previous creation timestamp. Either the <i>Timestamp</i> or the <i>viewVersion</i> parameter may be set by a <i>Client</i> , but not both. If <i>ViewVersion</i> is set this parameter shall be null.
viewVersion	UInt32	The version number for the <i>View</i> desired. When <i>Nodes</i> are added to or removed from a <i>View</i> , the value of a <i>View</i> 's <i>ViewVersion Property</i> is updated. Either the <i>Timestamp</i> or the <i>viewVersion</i> parameter may be set by a <i>Client</i> , but not both. The <i>ViewVersion Property</i> is defined in IEC 62541-3. If <i>timestamp</i> is set this parameter shall be 0. The current view is used if timestamp is null and viewVersion is 0.

Annex A (informative)

BNF definitions

A.1 Overview over BNF

The BNF (Backus-Naur form) used in this annex uses `<` and `>` to mark symbols, `[` and `]` to identify optional paths and `|` to identify alternatives. If the `(` and `)` symbols are used, it indicates sets.

A.2 BNF of RelativePath

A *RelativePath* is a structure that describes a sequence of *References* and *Nodes* to follow. This annex describes a text format for a *RelativePath* that can be used in documentation or in files used to store configuration information.

The components of a *RelativePath* text format are specified in Table A.1.

Table A.1 – RelativePath

Symbol	Meaning										
/	The forward slash character indicates that the <i>Server</i> is to follow any subtype of <i>HierarchicalReferences</i> .										
.	The period (dot) character indicates that the <i>Server</i> is to follow any subtype of a <i>Aggregates ReferenceType</i> .										
<[#!ns:]ReferenceType>	A string delimited by the `<` and `>` symbols specifies the <i>BrowseName</i> of a <i>ReferenceType</i> to follow. By default, any <i>References</i> of the subtypes the <i>ReferenceType</i> are followed as well. A `#` placed in front of the <i>BrowseName</i> indicates that subtypes should not be followed. A `!` in front of the <i>BrowseName</i> is used to indicate that the inverse <i>Reference</i> should be followed. The <i>BrowseName</i> may be qualified with a namespace index (indicated by a numeric prefix followed by a colon). This namespace index is used specify the namespace component of the <i>BrowseName</i> for the <i>ReferenceType</i> . If the namespace prefix is omitted then namespace index 0 is used.										
[ns:]BrowseName	A string that follows a `/`, `.` or `>` symbol specifies the <i>BrowseName</i> of a target <i>Node</i> to return or follow. This <i>BrowseName</i> may be prefixed by its namespace index. If the namespace prefix is omitted then namespace index 0 is used. Omitting the final <i>BrowseName</i> from a path is equivalent to a wildcard operation that matches all <i>Nodes</i> which are the target of the <i>Reference</i> specified by the path.										
&	The & sign character is the escape character. It is used to specify reserved characters that appear within a <i>BrowseName</i> . A reserved character is escaped by inserting the `&` in front of it. Examples of <i>BrowseNames</i> with escaped characters are: <table> <tr> <th>Received browse path name</th><th>Resolves to</th></tr> <tr> <td>"&/Name_1"</td><td>"/Name_1"</td></tr> <tr> <td>"&.Name_2"</td><td>".Name_2"</td></tr> <tr> <td>"&:Name_3"</td><td>":Name_3"</td></tr> <tr> <td>"&&Name_4"</td><td>"&Name_4"</td></tr> </table>	Received browse path name	Resolves to	"&/Name_1"	"/Name_1"	"&.Name_2"	".Name_2"	"&:Name_3"	":Name_3"	"&&Name_4"	"&Name_4"
Received browse path name	Resolves to										
"&/Name_1"	"/Name_1"										
"&.Name_2"	".Name_2"										
"&:Name_3"	":Name_3"										
"&&Name_4"	"&Name_4"										

Table A.2 provides *RelativePaths* examples in text format.

Table A.2 – *RelativePath* Examples

Browse Path	Description
"/2:Block&.Output"	Follows any forward hierarchical <i>Reference</i> with target <i>BrowseName</i> = "2:Block.Output".
"/3:Truck.0:NodeVersion"	Follows any forward hierarchical <i>Reference</i> with target <i>BrowseName</i> = "3:Truck" and from there a forward <i>Aggregates Reference</i> to a target with <i>BrowseName</i> "0:NodeVersion".
"<1:ConnectedTo>1:Boiler/1:HeatSensor"	Follows any forward <i>Reference</i> with a <i>BrowseName</i> = '1:ConnectedTo' and finds targets with <i>BrowseName</i> = '1:Boiler'. From there follows any hierarchical <i>Reference</i> and find targets with <i>BrowseName</i> = '1:HeatSensor'.
"<1:ConnectedTo>1:Boiler/"	Follows any forward <i>Reference</i> with a <i>BrowseName</i> = '1:ConnectedTo' and finds targets with <i>BrowseName</i> = '1:Boiler'. From there it finds all targets of hierarchical <i>References</i> .
"<0:HasChild>2:Wheel"	Follows any forward <i>Reference</i> with a <i>BrowseName</i> = 'HasChild' and qualified with the default OPC UA namespace. Then find targets with <i>BrowseName</i> = 'Wheel' qualified with namespace index '2'.
"<!HasChild>Truck"	Follows any inverse <i>Reference</i> with a <i>BrowseName</i> = 'HasChild'. Then find targets with <i>BrowseName</i> = 'Truck'. In both cases, the namespace component of the <i>BrowseName</i> is assumed to be 0.
"<0:HasChild>"	Finds all targets of forward <i>References</i> with a <i>BrowseName</i> = 'HasChild' and qualified with the default OPC UA namespace.

The following BNF describes the syntax of the *RelativePath* text format.

```

<relative-path> ::= <reference-type> <browse-name> [relative-path]

<reference-type> ::= '/' | '.' | '<' ['#'] ['!'] <browse-name> '>'

<browse-name> ::= [<namespace-index> ':' ] <name>

<namespace-index> ::= <digit> [<digit>]

<digit> ::= '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'

<name> ::= (<name-char> | '&' <reserved-char>) [<name>]

<reserved-char> ::= '/' | '.' | '<' | '>' | ':' | '#' | '!' | '&'

<name-char> ::= All valid characters for a String (see IEC 62541-3) excluding reserved-chars.

```

A.3 BNF of *NumericRange*

The following BNF describes the syntax of the *NumericRange* parameter type.

```

<numeric-range> ::= <dimension> [',' <dimension>]

<dimension> ::= <index> [':' <index>]

<index> ::= <digit> [<digit>]

<digit> ::= '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'

```

Annex B (informative)

Content Filter and Query Examples

B.1 Simple ContentFilter examples

B.1.1 Overview

These examples provide fairly simple content filters. Filter similar to these examples may be used in processing events.

The following conventions apply to these examples with regard to how Attribute operands are used (for a definition of this operand see 7.4.4):

- **AttributeOperand:** Refers to a *Node*, an *Attribute* of a *Node* or the *Value Attribute* of a *Property* associated with a *Node*. In the examples, the character names of NodeIds are used instead of an actual nodeId, this also applies to Attribute Ids.
- The string representation of relative paths is used instead of the actual structure.
- The NamespaceIndex used in all examples is 12 (it could just as easily have been 4 or 23 or any value). For more information about NamespaceIndex, see IEC 62541-3. The use of the NamespaceIndex illustrates that the information model being used in the examples is not a model defined by this standard, but one created for the examples.

B.1.2 Example 1

For example the logic describe by '(((AType.A = 5) or InList(BType.B, 3,5,7)) and BaseObjectType.displayName LIKE "Main%")' would result in a logic tree as shown in Figure B.1 and a ContentFilter as shown in Table B.1. For this example to return anything AType and BType shall both be subtypes of BaseObjectType, or the resulting "And" operation would always be false.

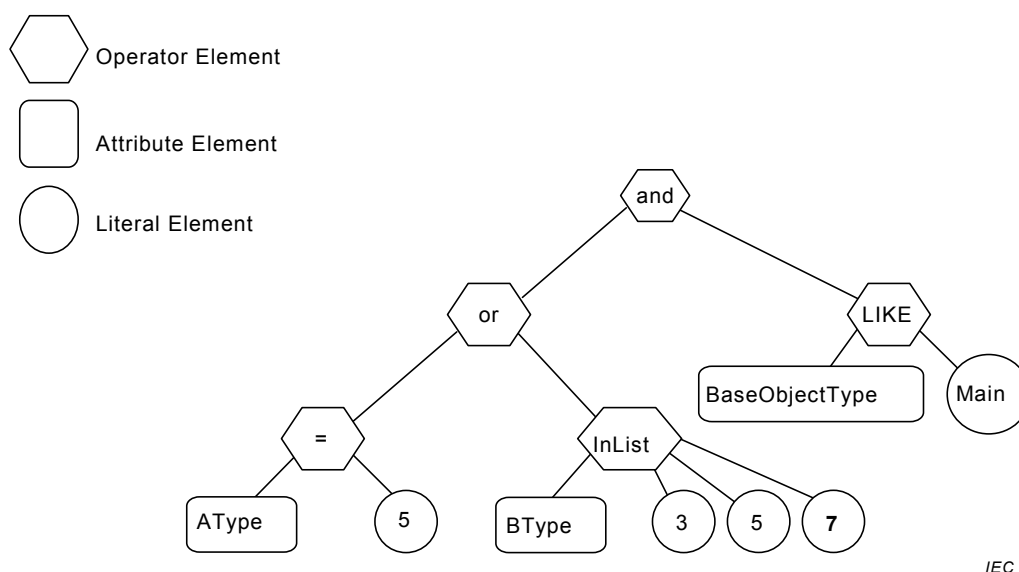


Figure B.1 – Filter Logic Tree Example

Table B.1 describes the elements, operators and operands used in the example.

Table B.1 – ContentFilter Example

Element[]	Operator	Operand[0]	Operand[1]	Operand[2]	Operand[3]
0	And	ElementOperand = 1	Element Operand = 4		
1	Or	ElementOperand = 2	Element Operand = 3		
2	Equals	AttributeOperand = NodeId: AType, BrowsePath: ".12:A", Attribute:value	LiteralOperand = '5'		
3	InList	AttributeOperand = NodeId: BType, BrowsePath: ".12:B", Attribute:value	LiteralOperand = '3'	LiteralOperand = '5'	LiteralOperand = '7'
4	Like	AttributeOperand = NodeId: BaseObjectType, BrowsePath: ".", Attribute: displayName	LiteralOperand = "Main%"		

B.1.3 Example 2

As another example a filter to select all *SystemEvents* (including derived types) that are contained in the *Area1 View* or the *Area2 View* would result in a logic tree as shown in Figure B.2 and a ContentFilter as shown in Table B.2.

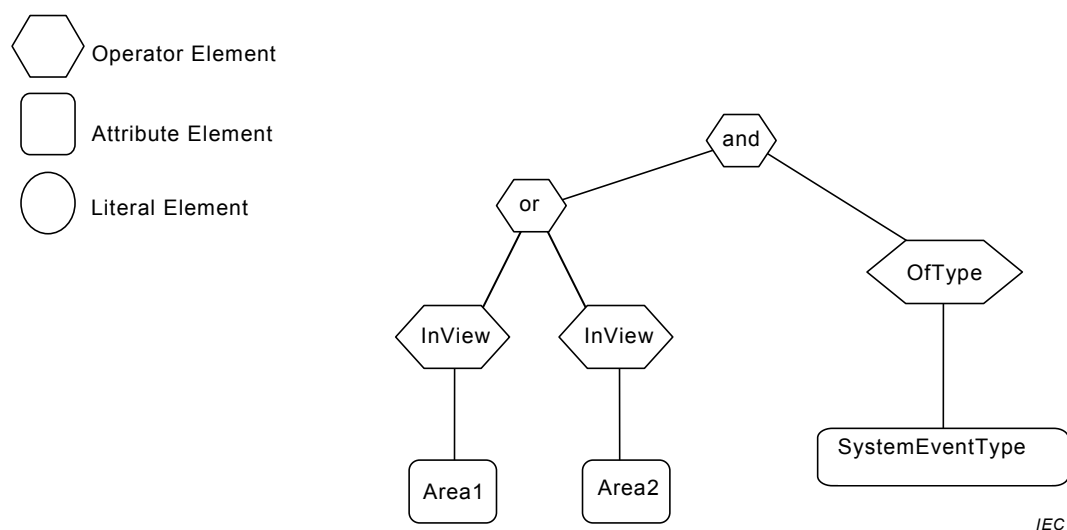
**Figure B.2 – Filter Logic Tree Example**

Table B.2 describes the elements, operators and operands used in the example.

Table B.2 – ContentFilter Example

Element[]	Operator	Operand[0]	Operand[1]
0	And	ElementOperand = 1	ElementOperand = 4
1	Or	ElementOperand = 2	ElementOperand = 3
2	InView	AttributeOperand = NodeId: Area1, BrowsePath: ".", Attribute: NodeId	
3	InView	AttributeOperand = NodeId: Area2, BrowsePath: ".", Attribute: NodeId	
4	OfType	AttributeOperand = NodeId: SystemEventType, BrowsePath: ".", Attribute: NodeId"	

B.2 Complex Examples of Query Filters**B.2.1 Overview**

These query examples illustrate complex filters. The following conventions apply to these examples with regard to Attribute operands (for a definition of these operands, see 7.4.4).

- *AttributeOperand*: Refers to a *Node*, an *Attribute* of a *Node* or the *Value Attribute* of a *Property* associated with a *Node*. In the examples character names of *ExpandedNodeId* are used instead of an actual *ExpandedNodeId*, this also applies to *Attribute* Ids.
- The string representation of relative paths is used instead of the actual structure.
- The *NamespaceIndex* used in all examples is 12 (it could just as easily have been 4 or 23 or any value). For more information about *NamespaceIndex*, see IEC 62541-3. The use of the *NamespaceIndex* illustrates that the information model being used in the examples is not a model defined by this standard, but one created for the examples.

B.2.2 Used type model

The following examples use the type model described below. All *Property* values are assumed to be string unless otherwise noted

New Reference types:

- “HasChild” derived from *HierarchicalReference*.
- “HasAnimal” derived from *HierarchicalReference*.
- “HasPet” derived from *HasAnimal*.
- “HasFarmAnimal” derived from *HasAnimal*.
- “HasSchedule” derived from *HierarchicalReference*.

PersonType derived from *BaseObjectType* adds:

- HasProperty "LastName".
- HasProperty "FirstName".
- HasProperty "StreetAddress".
- HasProperty "City".
- HasProperty "ZipCode".
- May have HasChild reference to a node of type *PersonType*.
- May have HasAnimal reference to a node of type *AnimalType* (or a subtype of this *Reference* type).

AnimalType derived from *BaseObjectType* adds:

- May have HasSchedule reference to a node of type *FeedingScheduleType*.
- HasProperty "Name".

DogType derived from *AnimalType* adds:

- HasProperty "NickName".
- HasProperty "DogBreed".
- HasProperty "License".

CatType derived from *AnimalType* adds:

- HasProperty "NickName".
- HasProperty "CatBreed".

PigType derived from *AnimalType* adds:

- HasProperty "PigBreed".

ScheduleType derived from *BaseObjectType* adds:

- HasProperty "Period".

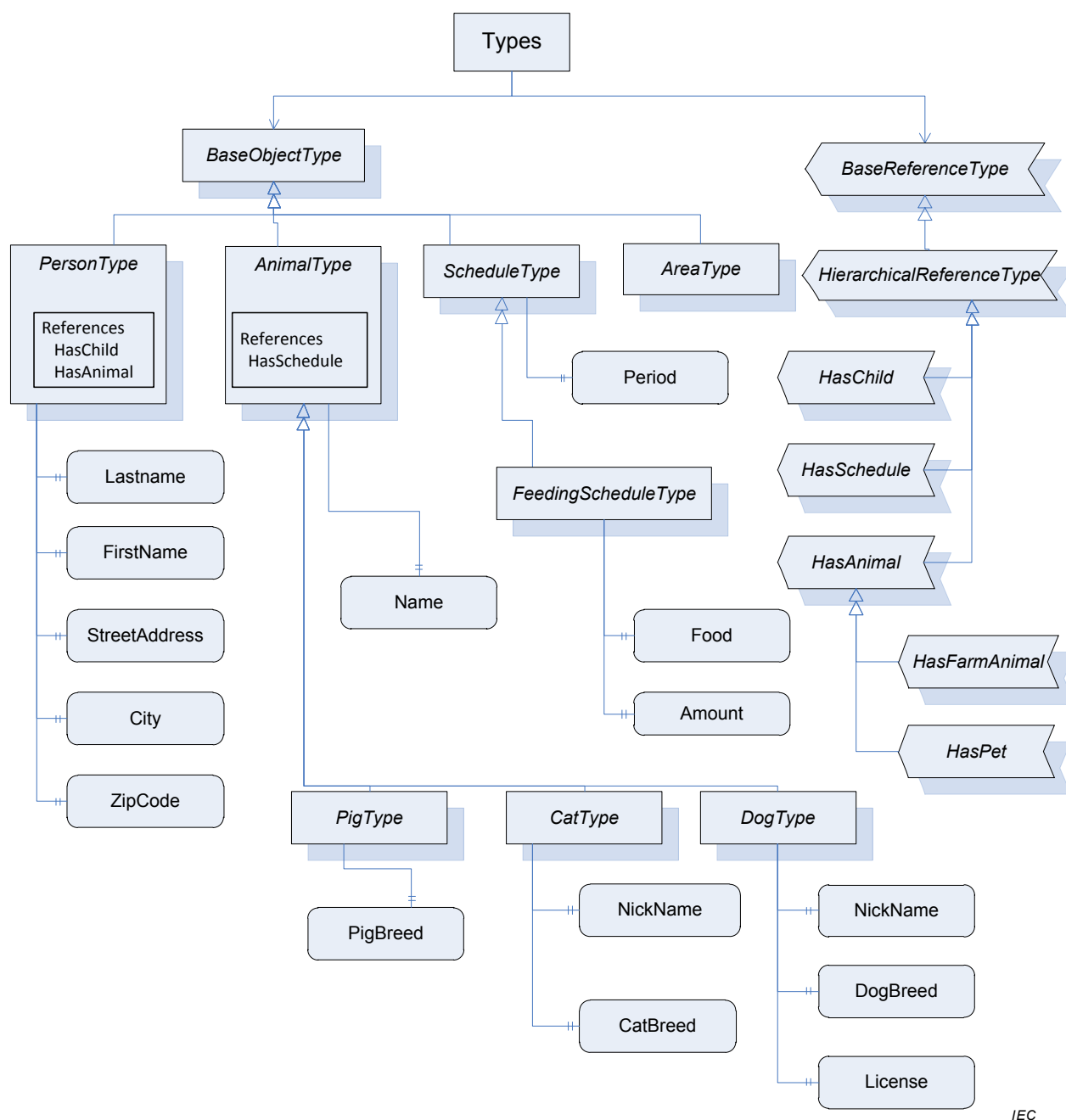
FeedingScheduleType derived from *ScheduleType* adds:

- HasProperty "Food".
- HasProperty "Amount" (Stored as an *Int32*).

AreaType derived from *BaseObjectType* is just a simple *Folder* and contains no *Properties*.

This example type system is shown in Figure B.3. In this Figure, the OPC UA notation is used for all *References* to *ObjectTypes*, *Variables*, *Properties* and subtypes. Additionally, supported *References* are contained in an inner box. The actual references only exist in the instances,

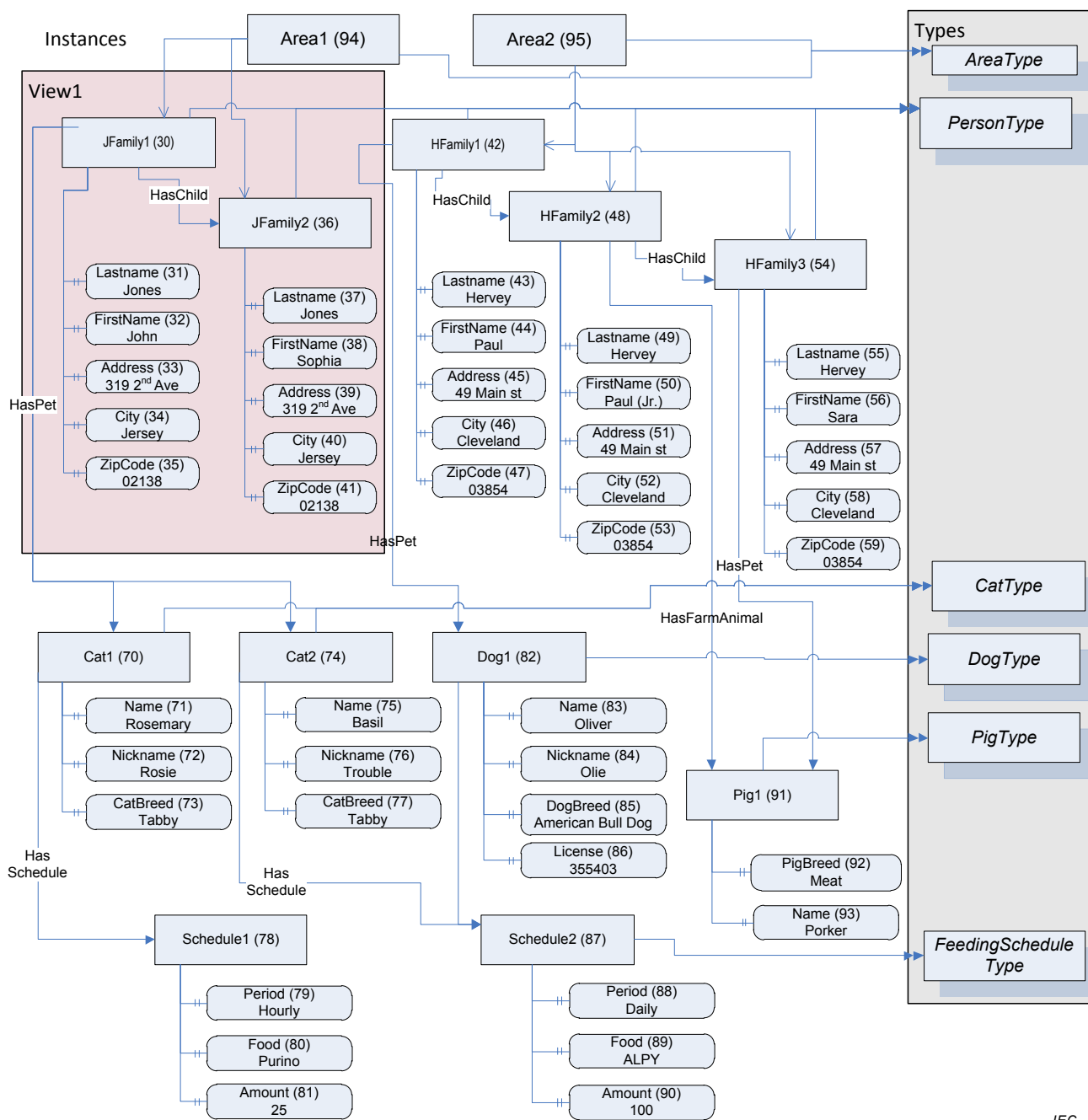
thus, no connections to other *Objects* are shown in the Figure and they are subtypes of the listed *Reference*.



IEC

Figure B.3 – Example Type Nodes

A corresponding example set of instances is shown in Figure B.4. These instances include a type *Reference* for *Objects*. Properties also have type *References*, but the *References* are omitted for simplicity. The name of the *Object* is provided in the box and a numeric instance *NodeId* in brackets. Standard *ReferenceTypes* use the OPC UA notation, custom *ReferenceTypes* are listed as a named *Reference*. For *Properties*, the *BrowseName*, *NodeId*, and *Value* are shown. The *Nodes* that are included in a *View* (View1) are enclosed in the coloured box. Two *Area* nodes are included for grouping of the existing person nodes. All custom nodes are defined in namespace 12 which is not included in Figure B.4.



IEC

Figure B.4 – Example Instance Nodes

B.2.3 Example Notes

For all of the examples in 7.4.4, the type definition *Node* is listed in its symbolic form, in the actual call it would be the *ExpandedNodeId* assigned to the *Node*. The *Attribute* is the symbolic name of the *Attribute*, in the actual call they would be translated to the *IntegerId* of the *Attribute*. Also in all of the examples the *BrowseName* is included in the result table for clarity; normally this would not be returned.

All of the examples include the following items:

- an English description of the object of the query,
- an SQL like description of the query,

- a table that has a *NodeTypeDescription* of the items that are to be returned
- a figure illustrating the query filter.
- A table describing the content filter
- A table describing the resulting dataset

The examples assume namespace 12 is the namespace for all of the custom definitions described for the examples.

B.2.4 Example 1

This example requests a simple layered filter, a person has a pet and the pet has a schedule.

Example 1: Get *PersonType.LastName*, *AnimalType.Name*, *ScheduleType.Period* where the Person Has a Pet and that Pet Has a Schedule.

The *NodeTypeDescription* parameters used in the example are described in Table B.3.

Table B.3 – Example 1 *NodeTypeDescription*

Type Definition Node	Include Subtypes	QueryDataDescription		
		Relative Path	Attribute	Index Range
PersonType	FALSE	“.12:LastName”	value	N/A
		“<12:HasPet>12:AnimalType. 12:Name”	value	N/A
		“<12:HasPet>12:AnimalType<12:HasSchedule>12:Schedule. 12:Period”	value	N/A

The corresponding *ContentFilter* is illustrated in Figure B.5.

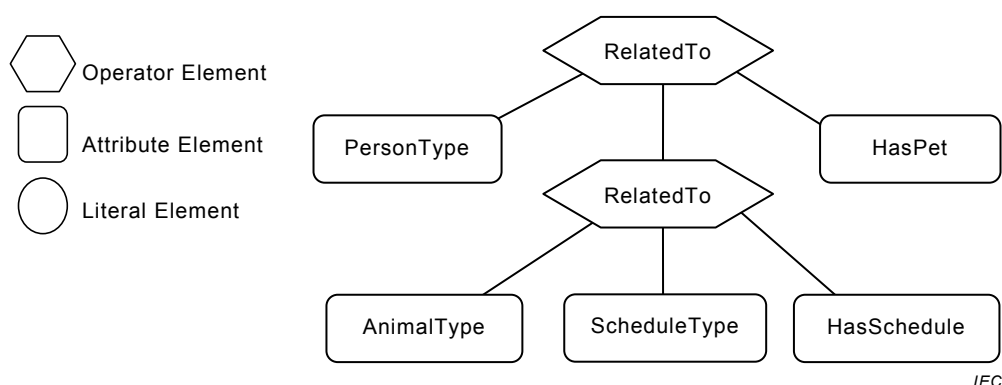


Figure B.5 – Example 1 Filter

Table B.4 describes the *ContentFilter* elements, operators and operands used in the example.

Table B.4 – Example 1 *ContentFilter*

Element[]	filterOperator	filterOperand[0]	filterOperand[1]	filterOperand[2]	filterOperand[3]
1	RelatedTo_15	AttributeOperand = Nodeid: PersonType, BrowsePath “.”, Attribute: Nodeid	ElementOperand = 2	AttributeOperand = Nodeid: HasPet, BrowsePath “.”, Attribute: Nodeid	LiteralOperand = “1”
2	RelatedTo_15	AttributeOperand = Nodeid: AnimalType, BrowsePath “.”, Attribute: Nodeid	AttributeOperand = Nodeid: ScheduleType, BrowsePath “.”, Attribute: Nodeid	AttributeOperand = Nodeid: HasSchedule, BrowsePath “.”, Attribute: Nodeid	LiteralOperand= “1”

Table B.5 describes the *QueryDataSet* that results from this query if it were executed against the instances described in Figure B.4

Table B.5 – Example 1 QueryDataSets

NodeId	TypeDefinition NodeId	RelativePath	Value
12:30 (JFamily1)	PersonType	".12:LastName"	Jones
		"<12:HasPet>12:AnimalType. 12:Name"	Rosemary
			Basil
		"<12:HasPet>12:AnimalType<12:HasSchedule> 12:Schedule.12:Period"	Hourly
12:42(HFamily1)	PersonType	".12:LastName"	Hervey
		"<12:HasPet>12:AnimalType. 12:Name"	Oliver
		"<12:HasPet>12:AnimalType<12:HasSchedule> 12:Schedule.12:Period"	Daily

NOTE The RelativePath column and browse name (in parentheses in the *NodeId* column) are not in the QueryDataSet and are only shown here for clarity. The *TypeDefinition NodeId* would be an integer not the symbolic name that is included in the table.

The Value column is returned as an array for each *Node* description, where the order of the items in the array would correspond to the order of the items that were requested for the given Node Type. In Addition, if a single *Attribute* has multiple values then it would be returned as an array within the larger array, for example in this table Rosemary and Basil would be returned in a array for the .<HasPet>.AnimalType.Name item. They are show as separate rows for ease of viewing. The actual value array for JFamily1 would be ("Jones", {"RoseMary", "Basil"}, {"Hourly", "Daily"})

B.2.5 Example 2

The second example illustrates receiving a list of disjoint *Nodes* and also illustrates that an array of results can be received.

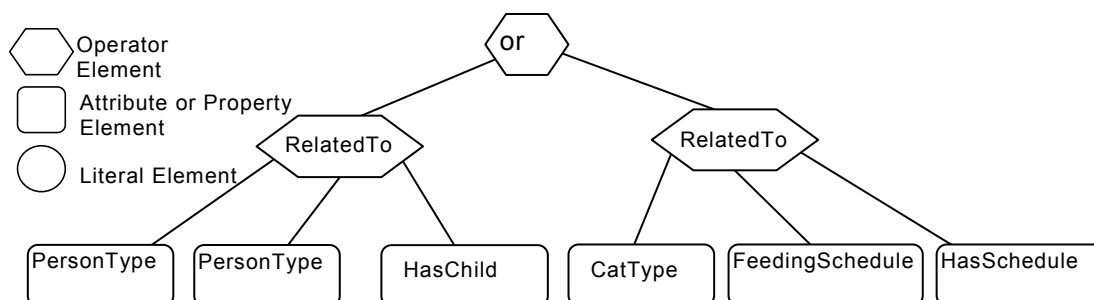
Example 2: Get PersonType.LastName, AnimalType.Name where a person has a child or (a pet is of type cat and has a feeding schedule).

The NodeTypeDescription parameters used in the example are described in Table B.6.

Table B.6 – Example 2 NodeTypeDescription

Type Definition Node	Include Subtypes	QueryDataDescription		
		Relative Path	Attribute	Index Range
PersonType	FALSE	".12:LastName"	Value	N/A
AnimalType	TRUE	".12:Name"	Value	N/A

The corresponding ContentFilter is illustrated in Figure B.6.



IEC

Figure B.6 – Example 2 Filter Logic Tree

Table B.7 describes the elements, operators and operands used in the example. It is worth noting that a CatType is a subtype of AnimalType.

Table B.7 – Example 2 ContentFilter

Element[]	Operator	Operand[0]	Operand[1]	Operand[2]	Operand[3]
0	Or	ElementOperand=1	ElementOperand = 2		
1	RelatedTo	AttributeOperand = NodId: PersonType, BrowsePath “.”, Attribute: NodId	AttributeOperand = NodId: PersonType, BrowsePath “.”, Attribute: NodId	AttributeOperand = NodId: HasChild, BrowsePath “.”, Attribute: NodId	LiteralOperand = ‘1’
2	RelatedTo	AttributeOperand = NodId: CatType, BrowsePath “.”, Attribute: NodId	AttributeOperand = NodId: FeedingScheduleType, BrowsePath “.”, Attribute: NodId	AttributeOperand = NodId: HasSchedule, BrowsePath “.”, Attribute: NodId	LiteralOperand = ‘1’

The results from this query would contain the *QueryDataSets* shown in Table B.8.

Table B.8 – Example 2 QueryDataSets

NodId	TypeDefinition NodId	RelativePath	Value
12:30 (Jfamily1)	PersonType	. 12:LastName	Jones
12:42 (HFamily1)	PersonType	. 12:LastName	Hervey
12:48 (HFamily2)	PersonType	. 12:LastName	Hervey
12:70 (Cat1)	CatType	. 12:Name	Rosemary
12:74 (Cat2)	CatType	. 12:Name	Basil

NOTE The relative path column and browse name (in parentheses in the *NodId* column) are not in the *QueryDataSet* and are only shown here for clarity. The *TypeDefinitionNodId* would be a *NodId* not the symbolic name that is included in the table.

B.2.6 Example 3

The third example provides a more complex *Query* in which the results are filtered on multiple criteria.

Example 3: Get PersonType.LastName, AnimalType.Name, ScheduleType.Period where a person has a pet and the animal has a feeding schedule and the person has a Zipcode = ‘02138’ and (the Schedule.Period is Daily or Hourly) and Amount to feed is > 10.

Table B.9 describes the NodeTypeIdDescription parameters used in the example.

Table B.9 – Example 3 – NodeTypeIdDescription

Type Definition Node	Include Subtypes	QueryDataDescription		
		RelativePath	Attribute	Index Range
PersonType	FALSE	“12:LastName”	Value	N/A
		“<12:HasPet>12:AnimalType. 12:Name”	Value	N/A
		“<12:HasPet>12:AnimalType<12:HasSchedule> 12:FeedingSchedule.Period”	Value	N/A

The corresponding *ContentFilter* is illustrated in Figure B.7.

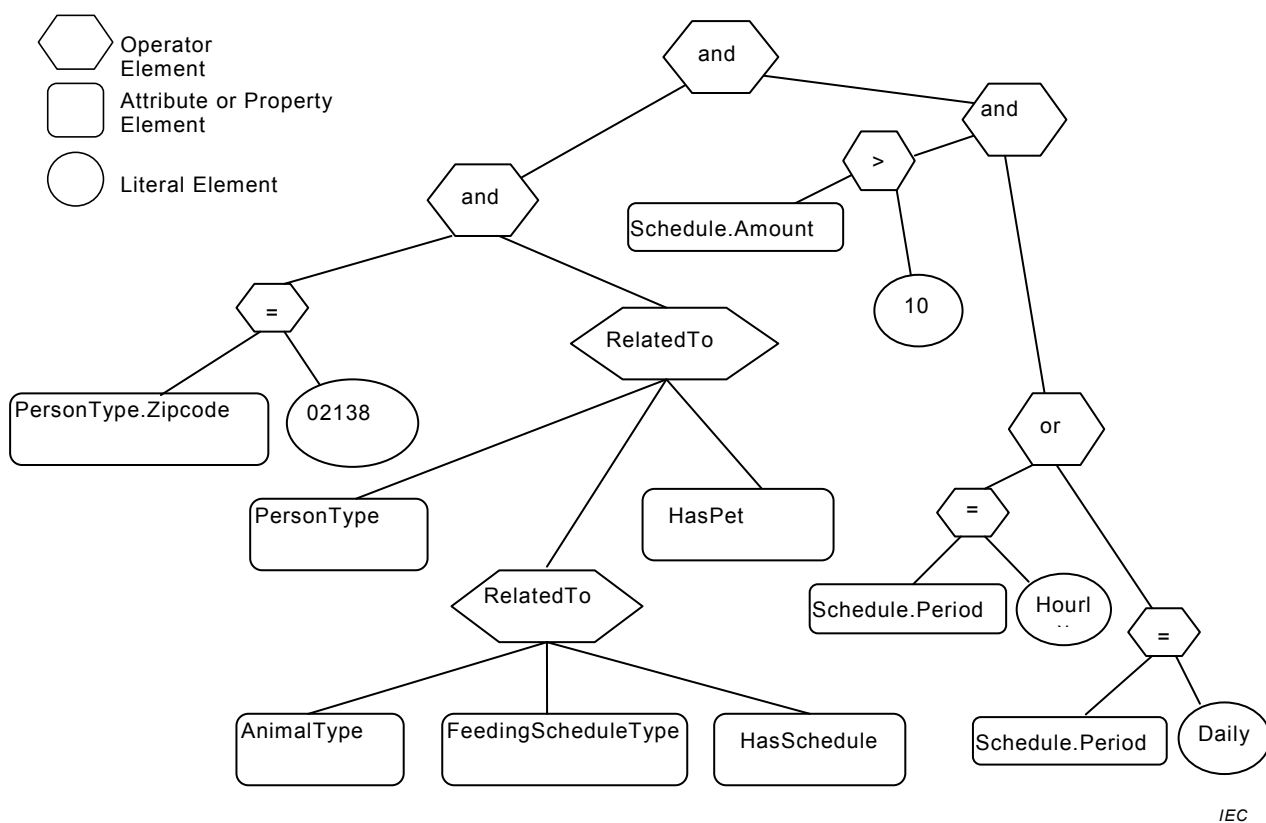


Figure B.7 – Example 3 Filter Logic Tree

Table B.10 describes the elements, operators and operands used in the example.

Table B.10 – Example 3 ContentFilter

Element[]	Operator	Operand[0]	Operand[1]	Operand[2]	Operand[3]
0	And	Element Operand = 1	ElementOperand = 2		
1	And	ElementOperand = 4	ElementOperand = 6		
2	And	ElementOperand = 3	ElementOperand = 9		
3	Or	ElementOperand = 7	ElementOperand = 8		
4	RelatedTo	AttributeOperand = NodeId: 12:PersonType, BrowsePath “.”, Attribute: NodeId	ElementOperand = 5	AttributeOperand = NodeId: 12:HasPet, BrowsePath “.”, Attribute: NodeId	LiteralOperand = ‘1’
5	RelatedTo	AttributeOperand = Node: 12:AnimalType, BrowsePath “.”, Attribute: NodeId Alias: AT	AttributeOperand = NodeId: 12:FeedingScheduleType, BrowsePath “.”, Attribute: NodeId Alias: FST	AttributeOperand = NodeId: 12:HasSchedule, BrowsePath “.”, Attribute: NodeId	LiteralOperand = ‘1’
6	Equals	AttributeOperand = NodeId: 12:PersonType BrowsePath 12:Zipcode “.”, Attribute: Value	LiteralOperand = ‘02138’		
7	Equals	AttributeOperand = NodeId: 12:PersonType BrowsePath “12:HasPet>12:AnimalType<12: HasSchedule>12: FeedingSchedule/12:Period”, Attribute: Value Alias: FST	LiteralOperand = ‘Daily’		
8	Equals	AttributeOperand = NodeId: 12:PersonType BrowsePath “12:HasPet>12:AnimalType<12: HasSchedule>12: FeedingSchedule/12:Period”, Attribute: Value Alias: FST	LiteralOperand = ‘Hourly’		
9	Greater Than	AttributeOperand = NodeId: 12:PersonType BrowsePath “12:HasPet>12:AnimalType<12: HasSchedule>12: FeedingSchedule/12:Amount”, Attribute: Value Alias: FST	ElementOperand = 10		
10	Cast	LiteralOperand = 10	AttributeOperand = NodeId: Int32, BrowsePath “.”, Attribute: NodeId		

The results from this query would contain the *QueryDataSets* shown in Table B.11.

Table B.11 – Example 3 QueryDataSets

NodeId	TypeDefinition NodeId	RelativePath	Value
12:30 (JFamily1)	PersonType	“.12:LastName”	Jones
		“<12:HasPet>12:PersonType. 12:Name”	Rosemary
			Basil
		“<12:HasPet>12:AnimalType<12:HasSchedule>12:FeedingSchedule. 12:Period”	Hourly
			Daily

NOTE The RelativePath column and browse name (in parentheses in the *NodeId* column) are not in the *QueryDataSet* and are only shown here for clarity. The TypeDefinitionNodeId would be an integer not the symbolic name that is included in the table.

B.2.7 Example 4

The fourth example provides an illustration of the Hop parameter that is part of the RelatedTo Operator.

Example 4: Get PersonType.LastName where a person has a child who has a child who has a pet.

Table B.12 describes the NodeTypeDescription parameters used in the example.

Table B.12 – Example 4 NodeTypeDescription

Type Definition Node	Include Subtypes	QueryDataDescription		
		Relative Path	Attribute	Index Range
PersonType	FALSE	“.12:LastName”	value	N/A

The corresponding *ContentFilter* is illustrated in Figure B.8.

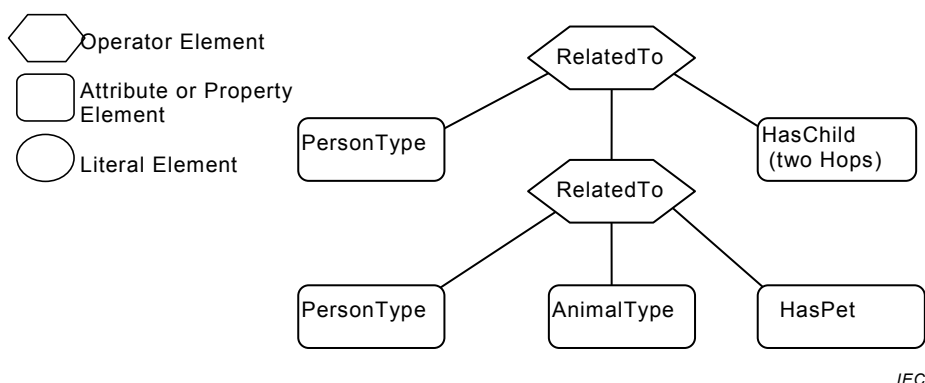


Figure B.8 – Example 4 Filter Logic Tree

Table B.13 describes the elements, operators and operands used in the example.

Table B.13 – Example 4 ContentFilter

Element[]	Operator	Operand[0]	Operand[1]	Operand[2]	Operand[3]
0	RelatedTo	AttributeOperand = NodeId: 12:PersonType, BrowsePath “.”, Attribute: NodeId	Element Operand = 1	AttributeOperand = NodeId: 12:HasChild, BrowsePath “.”, Attribute: NodeId	LiteralOperand = ‘2’
1	RelatedTo	AttributeOperand = NodeId: 12:PersonType, BrowsePath “.”, Attribute: NodeId	AttributeOperand = NodeId: 12:AnimalType, BrowsePath “.”, Attribute: NodeId	AttributeOperand = NodeId: 12:HasPet, BrowsePath “.”, Attribute: NodeId	LiteralOperand = ‘1’

The results from this query would contain the *QueryDataSets* shown in Table B.14. It is worth noting that the pig “Pig1” is referenced as a pet by Sara, but is referenced as a farm animal by Sara’s parent Paul.

Table B.14 – Example 4 QueryDataSets

NodeId	TypeDefinition NodeId	RelativePath	Value
12:42 (HFamily1)	PersonType	“.12:LastName”	Hervey

NOTE The RelativePath column and browse name (in parentheses in the *NodeId* column) are not in the *QueryDataSet* and are only shown here for clarity. The TypeDefinition NodeId would be an integer not the symbolic name that is included in the table.

B.2.8 Example 5

The fifth example provides an illustration of the use of alias.

Example 5: Get the last names of children that have the same first name as a parent of theirs

Table B.15 describes the NodeTypeDescription parameters used in the example.

Table B.15 – Example 5 NodeTypeDescription

Type Definition Node	Include Subtypes	QueryDataDescription		
		Relative Path	Attribute	Index Range
PersonType	FALSE	"<12:HasChild>12:PersonType.12:LastName"	Value	N/A

The corresponding *ContentFilter* is illustrated in Figure B.9.

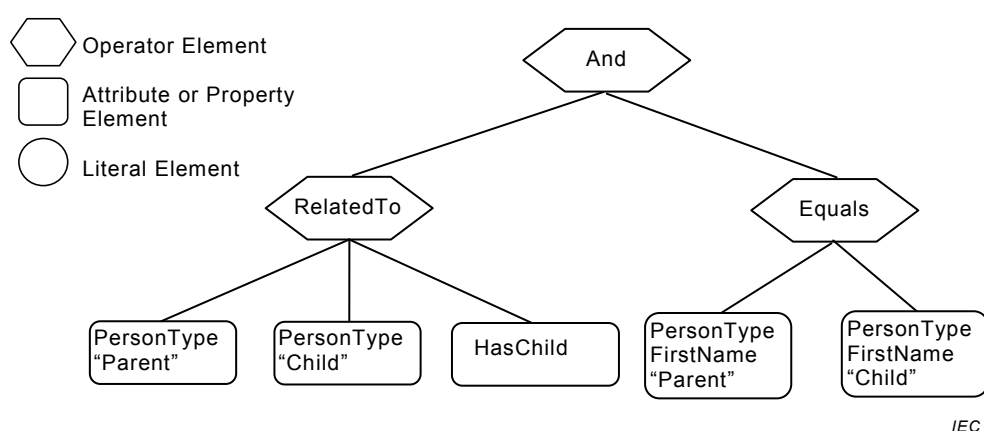


Figure B.9 – Example 5 Filter Logic Tree

In this example, one *Reference* to PersonType is aliased to “Parent” and another *Reference* to PersonType is aliased to “Child”. The value of Parent.firstName and Child.firstName are then compared. Table B.16 describes the elements, operators and operands used in the example.

Table B.16 – Example 5 ContentFilter

Element[]	Operator	Operand[0]	Operand[1]	Operand[2]	Operand[3]
0	And	ElementOperand = 1	ElementOperand = 2		
1	RelatedTo	AttributeOperand = NodeId: 12:PersonType, BrowsePath “.”, Attribute: NodeId, Alias: “Parent”	AttributeOperand = NodeId: 12:PersonType, BrowsePath “.”, Attribute: NodeId, Alias: “Child”	AttributeOperand = NodeId: 12:HasChild, Attribute: NodeId	LiteralOperand = “1”
2	Equals	AttributeOperand = NodeId: 12:PersonType, BrowsePath “/12:FirstName”, Attribute: Value, Alias: “Parent”	AttributeOperand = NodeId: 12:PersonType, BrowsePath “/12:FirstName”, Attribute: Value, Alias: “Child”		

The results from this query would contain the *QueryDataSets* shown in Table B.17.

Table B.17 – Example 5 QueryDataSets

NodeId	TypeDefinition NodeId	RelativePath	Value
12:42 (HFamily1)	PersonType	"<12:HasChild>12:PersonType.12:LastName"	Hervey

NOTE The *RelativePath* column and browse name (in parentheses in the *NodeId* column) are not in the *QueryDataSet* and are only shown here for clarity. The *TypeDefinitionNodeId* would be an integer not the symbolic name that is included in the table.

B.2.9 Example 6

The sixth example provides an illustration a different type of request, one in which the *Client* is interested in displaying part of the address space of the server. This request includes listing a *Reference* as something that is to be returned.

Example 6: Get PersonType.NodeId, AnimalType.NodeId, PersonType.HasChild Reference, PersonType.HasAnimal Reference where a person has a child who has a Animal.

Table B.18 describes the *NodeTypeDescription* parameters used in the example.

Table B.18 – Example 6 NodeTypeDescription

Type Definition Node	Include Subtypes	QueryDataDescription		
		Relative Path	Attribute	Index Range
PersonType	FALSE	".12:NodeId"	value	N/A
		<12:HasChild>12:PersonType<12:HasAnimal>12:AnimalType.NodeId	value	N/A
		<12:HasChild>	value	N/A
		<12:HasChild>12:PersonType<12:HasAnimal>	value	N/A

The corresponding *ContentFilter* is illustrated in Figure B.10.

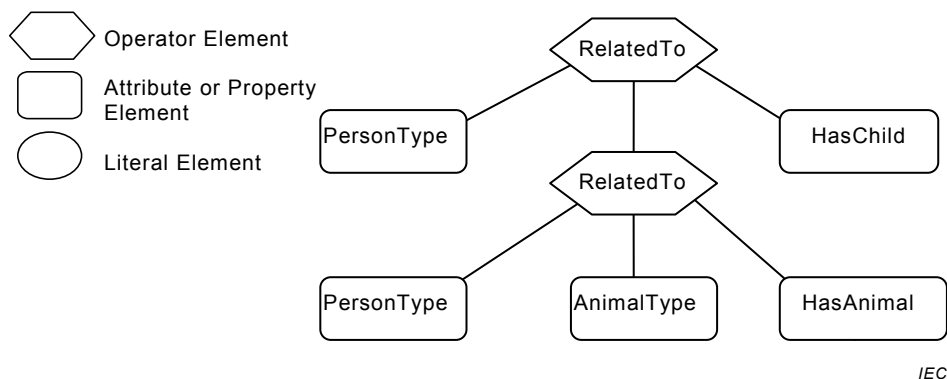


Figure B.10 – Example 6 Filter Logic Tree

Table B.19 describes the elements, operators and operands used in the example.

Table B.19 – Example 6 ContentFilter

Element[]	Operator	Operand[0]	Operand[1]	Operand[2]	Operand[3]
0	RelatedTo	AttributeOperand = NodeId: 12:PersonType, BrowsePath ".", Attribute: NodeId	ElementOperand = 1	AttributeOperand = Node: 12:HasChild, BrowsePath ".", Attribute: NodeId	LiteralOperand = '1'
1	RelatedTo	AttributeOperand = NodeId: 12:PersonType, BrowsePath ".", Attribute: NodeId	AttributeOperand = NodeId: 12:AnimalType, BrowsePath ".", Attribute: NodeId	AttributeOperand = NodeId: 12:HasAnimal, BrowsePath ".", Attribute: NodeId	LiteralOperand = '1'

The results from this query would contain the *QueryDataSets* shown in Table B.20.

Table B.20 – Example 6 QueryDataSets

NodeId	TypeDefinition NodeId	RelativePath	Value
12:42 (HFamily1)	PersonType	“.NodeId”	12:42 (HFamily1)
		<12:HasChild>12:PersonType<12:HasAnimal> 12:AnimalType.NodeId	12:91 (Pig1)
		<12:HasChild>	HasChild <i>ReferenceDescription</i>
		<12:HasChild>12:PersonType<12:HasAnimal>	HasFarmAnimal <i>ReferenceDescription</i>
12:48 (HFamily2)	PersonType	“.NodeId”	12:48 (HFamily2)
		<12:HasChild>12:PersonType<12:HasAnimal> 12:AnimalType.NodeId	12:91 (Pig1)
		<12:HasChild>	HasChild <i>ReferenceDescription</i>
		<12:HasChild>12:PersonType<12:HasAnimal>	HasPet <i>ReferenceDescription</i>

NOTE The RelativePath and browse name (in parentheses) is not in the *QueryDataSet* and is only shown here for clarity and the TypeDefinitionNodeId would be an integer, not the symbolic name that is included in the table. The value field would in this case be the *NodeId* where it was requested, but for the example the browse name is provided in parentheses and in the case of *Reference* types on the browse name is provided. For the *References* listed in Table B.20, the value would be a *ReferenceDescription* which are described in 7.24.

Table B.21 provides an example of the same *QueryDataSet* as shown in Table B.20 without any additional fields and minimal symbolic Ids. There is an entry for each requested Attribute, in the cases where an Attribute would return multiple entries the entries are separated by comas. If a structure is being returned then the structure is enclosed in square brackets. In the case of a *ReferenceDescription* the structure contains a structure and DisplayName and BrowseName are assumed to be the same and defined in Figure B.4.

Table B.21 – Example 6 QueryDataSets without Additional Information

NodeId	TypeDefinition NodeId	Value
12:42	PersonType	12:42
		12:91
		[HasChild,TRUE,[48,HFamily2,HFamily2,PersonType]],
		[HasFarmAnimal,TRUE[91,Pig1,Pig1,PigType]
12:48	PersonType	12:54
		12:91
		[HasChild,TRUE,[54,HFamily3,HFamily3,PersonType]]
		[HasPet, TRUE,[91,Pig1,Pig1,PigType]]

The PersonType, HasChild, PigType, HasPet, HasFarmAnimal identifiers used in the above table would be translated to actual *ExtendedNodeIds*.

B.2.10 Example 7

The seventh example provides an illustration a request in which a *Client* wants to display part of the address space based on a starting point that was obtained via browsing. This request includes listing *References* as something that is to be returned. In this case the Person Browsed to Area2 and wanted to *Query* for information below this starting point.

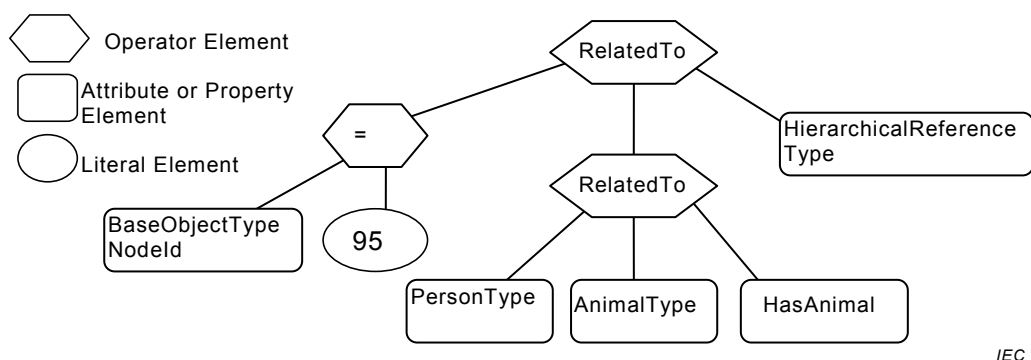
Example 7: Get PersonType.NodeId, AnimalType.NodeId, PersonType.HasChild Reference, PersonType.HasAnimal Reference where the person is in Area2 (Cleveland nodes) and the person has a child.

Table B.22 describes the NodeTypeDescription parameters used in the example.

Table B.22 – Example 7 NodeTypeDescription

Type Definition Node	Include Subtypes	QueryDataDescription		
		Relative Path	Attribute	Index Range
PersonType	FALSE	“.NodeId”	Value	N/A
		<12:HasChild>	Value	N/A
		<12:HasAnimal>NodeId	Value	N/A
		<12:HasAnimal>	Value	N/A

The corresponding *ContentFilter* is illustrated in Figure B.11. Note that the *Browse* call would typically return a *NodeId*, thus the first filter is for the *BaseObjectType* with a *NodeId* of 95 where 95 is the *NodeId* associated with the Area2 node, all *Nodes* descend from *BaseObjectType*, and *NodeId* is a base *Property* so this filter will work for all *Queries* of this nature.



IEC

Figure B.11 – Example 7 Filter Logic Tree

Table B.23 describes the elements, operators and operands used in the example.

Table B.23 – Example 7 ContentFilter

Element[]	Operator	Operand[0]	Operand[1]	Operand[2]	Operand[3]
0	RelatedTo	ElementOperand = 2	ElementOperand = 1	AttributeOperand = Node:HierarchicalReference, BrowsePath “.”, Attribute:NodeId	LiteralOperand = ‘1’
1	RelatedTo	AttributeOperand = NodeId: 12:PersonType, BrowsePath “.”, Attribute: NodeId	AttributeOperand = NodeId: 12:PersonType, BrowsePath “.”, Attribute: NodeId	AttributeOperand = NodeId: 12:HasChild, BrowsePath “.”, Attribute: NodeId	LiteralOperand = ‘1’
2	Equals	AttributeOperand = NodeId: BaseObjectType, BrowsePath “.”, Attribute: NodeId,	LiteralOperand = ‘95’		

The results from this *Query* would contain the *QueryDataSets* shown in Table B.24.

Table B.24 – Example 7 QueryDataSets

NodeId	TypeDefinition NodeId	RelativePath	Value
12:42 (HFamily1)	PersonType	".NodeId"	12:42 (HFamily1)
		<12:HasChild>	HasChild <i>ReferenceDescription</i>
		<12:HasAnimal>12:AnimalType.NodeId	NULL
		<12:HasAnimal>	HasFarmAnimal <i>ReferenceDescription</i>
12:48 (HFamily2)	PersonType	".NodeId"	12:48 (HFamily2)
		<12:HasChild>	HasChild <i>ReferenceDescription</i>
		<12:HasAnimal>12:AnimalType.NodeId	12:91 (Pig1)
		<12:HasAnimal>	HasFarmAnimal <i>ReferenceDescription</i>

NOTE The RelativePath and browse name (in parentheses) is not in the *QueryDataSet* and is only shown here for clarity and the TypeDefinitionNodeId would be an integer not the symbolic name that is included in the table. The value field would in this case be the *NodeId* where it was requested, but for the example the browse name is provided in parentheses and in the case of *Reference* types on the browse name is provided. For the *References* listed in Table B.24, the value would be a *ReferenceDescription* which are described in 7.24.

B.2.11 Example 8

The eighth example provides an illustration of a request in which the address space is restricted by a *Server* defined *View*. This request is the same as in the second example which illustrates receiving a list of disjoint *Nodes* and also illustrates that an array of results can be received. It is **important** to note that all of the parameters and the *ContentFilter* are the same, only the *View* description would be specified as "View1".

Example 8: Get PersonType.LastName, AnimalType.Name where a person has a child or (a pet is of type cat and has a feeding schedule) limited by the address space in View1.

The NodeTypeDescription parameters used in the example are described in Table B.25

Table B.25 – Example 8 NodeTypeDescription

Type Definition Node	Include Subtypes	QueryDataDescription		
		Relative Path	Attribute	Index Range
PersonType	FALSE	".12:LastName"	value	N/A
AnimalType	TRUE	"12.Name"	value	N/A

The corresponding *ContentFilter* is illustrated in Figure B.12.

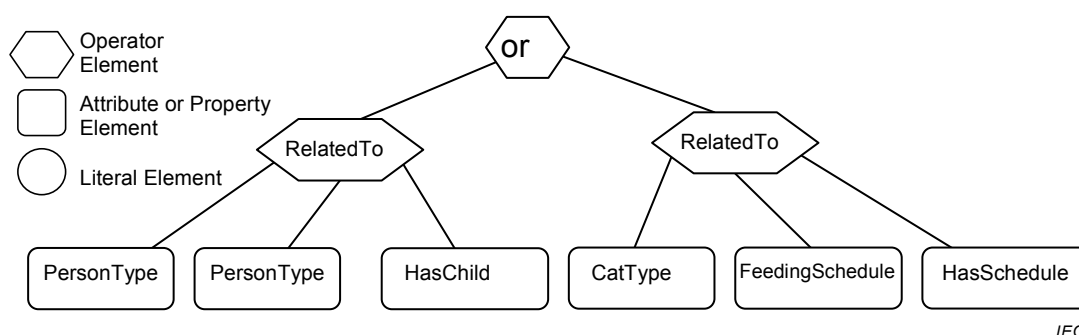
**Figure B.12 – Example 8 Filter Logic Tree**

Table B.26 describes the elements, operators and operands used in the example. It is worth noting that a *CatType* is a subtype of *AnimalType*.

Table B.26 – Example 8 ContentFilter

Element[]	Operator	Operand[0]	Operand[1]	Operand[2]	Operand[3]
0	Or	ElementOperand=1	ElementOperand = 2		
1	RelatedTo	AttributeOperand = NodId: 12:PersonType, BrowsePath “.”, Attribute: NodId	AttributeOperand = NodId: 12:PersonType, BrowsePath “.”, Attribute: NodId	AttributeOperand = NodId: 12:HasChild, BrowsePath “.”, Attribute: NodId	LiteralOperand = ‘1’
2	RelatedTo	AttributeOperand = NodId: 12:CatType, BrowsePath “.”, Attribute: NodId	AttributeOperand = NodId: 12:FeedingScheduleType, BrowsePath “.”, Attribute: NodId	AttributeOperand = NodId: 12:HasSchedule, BrowsePath “.”, Attribute: NodId	LiteralOperand = ‘1’

The results from this query would contain the *QueryDataSets* shown in Table B.27. If this is compared to the result set from example 2, the only difference is the omission of the Cat *Nodes*. These *Nodes* are not in the *View* and thus are not included in the result set.

Table B.27 – Example 8 QueryDataSets

NodId	TypeDefinition NodId	RelativePath	Value
12:30 (Jfamily1)	Persontype	.12:LastName	Jones

NOTE The RelativePath column and browse name (in parentheses in the *NodId* column) are not in the *QueryDataSet* and are only shown here for clarity. The TypeDefinition NodId would be an integer not the symbolic name that is included in the table.

B.2.12 Example 9

The ninth example provides a further illustration for a request in which the address space is restricted by a *Server* defined *View*. This request is similar to the second example except that some of the requested nodes are expressed in terms of a relative path. It is **important** to note that the *ContentFilter* is the same, only the *View* description would be specified as “View1”.

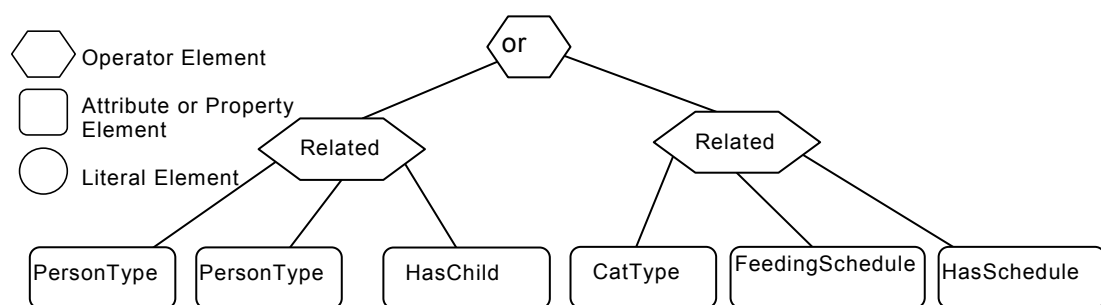
Example 9: Get PersonType.LastName, AnimalType.Name where a person has a child or (a pet is of type cat and has a feeding schedule) limited by the address space in View1.

Table B.28 describes the NodeTypeDescription parameters used in the example.

Table B.28 – Example 9 NodeTypeDescription

Type Definition Node	Include Subtypes	QueryDataDescription		
		Relative Path	Attribute	Index Range
PersonType	FALSE	“.NodId”	value	N/A
		<12:HasChild>12:PersonType<12:HasAnimal>12:AnimalType.NodId	value	N/A
		<12:HasChild>	value	N/A
		<12:HasChild>12:PersonType<12:HasAnimal>	value	N/A
PersonType	FALSE	“.12:LastName”	value	N/A
		<12:HasAnimal>12:AnimalType.12:Name	value	N/A
AnimalType	TRUE	“.12:name”	value	N/A

The corresponding *ContentFilter* is illustrated in Figure B.13.



IEC

Figure B.13 – Example 9 Filter Logic Tree

Table B.29 describes the elements, operators and operands used in the example.

Table B.29 – Example 9 ContentFilter

Element[]	Operator	Operand[0]	Operand[1]	Operand[2]	Operand[3]
0	Or	ElementOperand=1	ElementOperand = 2		
1	RelatedTo	AttributeOperand = NodeId: 12:PersonType, BrowsePath “.”, Attribute: NodeId	AttributeOperand = NodeId: 12:PersonType, BrowsePath “.”, Attribute: NodeId	AttributeOperand = NodeId: 12:HasChild, BrowsePath “.”, Attribute: NodeId	LiteralOperand = ‘1’
2	RelatedTo	AttributeOperand = NodeId: 12:CatType, BrowsePath “.”, Attribute: NodeId	AttributeOperand = NodeId: 12:FeedingScheduleType, BrowsePath “.”, Attribute: NodeId	AttributeOperand = NodeId: 12:HasSchedule, BrowsePath “.”, Attribute: NodeId	LiteralOperand = ‘1’

The results from this *Query* would contain the *QueryDataSets* shown in Table B.30. If this is compared to the result set from example 2, the Pet *Nodes* are included in the list, even though they are outside of the *View*. This is possible since the name referenced via the relative path and the root *Node* is in the *View*.

Table B.30 – Example 9 QueryDataSets

NodeId	TypeDefinition NodeId	RelativePath	Value
12:30 (Jfamily1)	PersonType	. 12:LastName	Jones
		<12:HasAnimal>12:AnimalType. 12:Name	Rosemary
		<12:HasAnimal>12:AnimalType. 12:Name	Basil

NOTE The RelativePath column and browse name (in parentheses in the *NodeId* column) are not in the QueryDataSet and are only shown here for clarity. The TypeDefinition NodeId would be an integer not the symbolic name that is included in the table.

SOMMAIRE

AVANT-PROPOS	194
1 Domaine d'application	197
2 Références normatives	197
3 Termes, définitions et conventions.....	198
3.1 Termes et définitions	198
3.2 Abréviations et symboles	198
3.3 Conventions pour les définitions des Services	199
4 Vue d'ensemble	200
4.1 Modèle de jeux de services	200
4.2 Procédures des services Request/Response (Demande/Réponse).....	205
5 Jeux de services (Service Sets).....	206
5.1 Généralités	206
5.2 En-tête de demande de services et de réponse.....	206
5.3 Résultats des services	206
5.4 Jeu de services de découverte (Discovery Service Set)	207
5.4.1 Vue d'ensemble	207
5.4.2 Trouver des serveurs (FindServers).....	209
5.4.3 Déterminer les points d'extrémité (GetEndpoints)	211
5.4.4 Enregistrer un serveur (RegisterServer).....	214
5.5 Jeu de services de canal sécurisé (SecureChannel Service Set).....	218
5.5.1 Vue d'ensemble	218
5.5.2 Ouvrir un canal sécurisé (OpenSecureChannel)	219
5.5.3 Fermer un canal sécurisé (CloseSecureChannel)	222
5.6 Jeu de services de session (Session Service Set).....	223
5.6.1 Vue d'ensemble	223
5.6.2 Créer une session (CreateSession)	223
5.6.3 Activer une session (ActivateSession)	227
5.6.4 Fermer une session (CloseSession)	230
5.6.5 Annuler (Cancel).....	231
5.7 Jeu de services de gestion de nœuds (NodeManagement Service Set).....	231
5.7.1 Vue d'ensemble	231
5.7.2 Ajouter des nœuds (AddNodes)	231
5.7.3 Ajouter des références (AddReferences)	233
5.7.4 Supprimer des nœuds (DeleteNodes)	235
5.7.5 Supprimer des références (DeleteReferences).....	236
5.8 Jeu de services pour les vues (View Service Set)	238
5.8.1 Vue d'ensemble	238
5.8.2 Parcourir (Browse).....	238
5.8.3 Parcourir le prochain (BrowseNext)	240
5.8.4 Traduire les chemins de navigation en identifiants de nœuds (TranslateBrowsePathsToNodeIds).....	242
5.8.5 Enregistrer les nœuds (RegisterNodes)	244
5.8.6 Supprimer l'enregistrement de nœuds (UnregisterNodes)	245
5.9 Jeu de services d'interrogation (Query Service Set)	246
5.9.1 Vue d'ensemble	246
5.9.2 Interroger des vues (Querying Views)	246

5.9.3	Émettre une demande (QueryFirst)	247
5.9.4	Poursuivre la demande (QueryNext)	250
5.10	Jeu de services pour les attributs (Attribute Service Set)	251
5.10.1	Vue d'ensemble	251
5.10.2	Lecture (Read)	251
5.10.3	Lecture de l'historique (HistoryRead)	253
5.10.4	Écrire (Write)	255
5.10.5	Mise à jour de l'historique (HistoryUpdate)	258
5.11	Jeu de services de méthodes (Method Service Set)	260
5.11.1	Vue d'ensemble	260
5.11.2	Appel (Call)	260
5.12	Jeu de services des éléments surveillés (MonitoredItem Service Set)	262
5.12.1	Modèle pour les éléments surveillés (MonitoredItem Model)	262
5.12.2	Créer des éléments surveillés (CreateMonitoredItems)	269
5.12.3	Modifier des éléments surveillés (ModifyMonitoredItems)	271
5.12.4	Gérer des modes de surveillance (SetMonitoringMode)	273
5.12.5	Gérer des déclenchements (SetTriggering)	274
5.12.6	Supprimer des éléments surveillés (DeleteMonitoredItems)	275
5.13	Jeu de services d'abonnements (Subscription Service Set)	276
5.13.1	Modèle d'abonnement (SubscriptionModel)	276
5.13.2	Créer un abonnement (CreateSubscription)	284
5.13.3	Modifier un abonnement (ModifySubscription)	285
5.13.4	Demander un mode d'édition (SetPublishingMode)	286
5.13.5	Editer (Publish)	287
5.13.6	Éditer à nouveau (Republish)	290
5.13.7	Transférer un abonnement (TransferSubscriptions)	290
5.13.8	Supprimer des abonnements (DeleteSubscriptions)	292
6	Comportements des services	293
6.1	Sécurité	293
6.1.1	Vue d'ensemble	293
6.1.2	Obtenir et installer un certificat d'instance d'application (Application Instance Certificate)	293
6.1.3	Déterminer si un Certificat est Fiable	295
6.1.4	Créer un Canal Sécurisé (SecureChannel)	298
6.1.5	Créer une Session	300
6.1.6	Se substituer à un utilisateur	301
6.2	Certificats de logiciel (Software Certificates)	302
6.2.1	Vue d'ensemble	302
6.2.2	Obtenir et installer un Certificat de logiciel	302
6.2.3	Valider un Certificat de logiciel	304
6.3	Audits	304
6.3.1	Vue d'ensemble	304
6.3.2	Journaux d'audits généraux	304
6.3.3	Événements d'audit généraux	305
6.3.4	Audits pour jeu de services Discovery	305
6.3.5	Audits pour jeu de services SecureChannel	305
6.3.6	Audits pour jeu de services Session	305
6.3.7	Audits pour jeu de services NodeManagement	306
6.3.8	Audits pour jeu de services Attribute	306

6.3.9	Audits pour jeu de services de Méthodes	307
6.3.10	Audits pour le View Service Set, le Query Service Set, le MonitoredItem Service Set et le Subscription Service Set	307
6.4	Redondance	307
6.4.1	Vue d'ensemble de la redondance	307
6.4.2	Vue d'ensemble de la redondance du serveur	308
6.4.3	Redondance client	313
6.4.4	Redondance du réseau	313
6.5	Rétablir les connexions	313
7	Définitions de types de paramètres communs	315
7.1	Description d'une application (ApplicationDescription)	315
7.2	Certificat d'instance d'application (ApplicationInstanceCertificate)	315
7.3	Parcourir les résultats (BrowseResult)	316
7.4	Composantes d'un filtre (ContentFilter)	316
7.4.1	Structure de composantes d'un filtre (ContentFilter structure)	316
7.4.2	Composantes des résultats d'un filtre (ContentFilterResult)	317
7.4.3	Opérateurs d'un filtre (FilterOperator)	318
7.4.4	Paramètres de FilterOperand	324
7.5	Compteur (Counter)	326
7.6	Point de continuation (ContinuationPoint)	326
7.7	Valeur d'une donnée (DataValue)	327
7.7.1	Généralités	327
7.7.2	PicoSeconds	327
7.7.3	SourceTimestamp	327
7.7.4	ServerTimestamp	328
7.7.5	StatusCode affecté à une valeur	328
7.8	Information de diagnostic (DiagnosticInfo)	329
7.9	Description d'un point d'extrémité (EndpointDescription)	329
7.10	Identifiant étendu d'un nœud (ExpandedNodeId)	330
7.11	Paramètre extensible (ExtensibleParameter)	331
7.12	Indice (Index)	331
7.13	IntegerId	331
7.14	Mode de sécurité des messages (MessageSecurityMode)	331
7.15	Surveillance des paramètres (MonitoringParameters)	331
7.16	Paramètres des filtres de surveillance (MonitoringFilter)	333
7.16.1	Vue d'ensemble	333
7.16.2	DataChangeFilter	333
7.16.3	EventFilter	334
7.16.4	AggregateFilter	336
7.17	Mode de surveillance (MonitoringMode)	338
7.18	Paramètres des attributs d'un nœud (NodeAttributes)	338
7.18.1	Vue d'ensemble	338
7.18.2	Paramètre d'ObjectAttributes	339
7.18.3	Paramètre de VariableAttributes	339
7.18.4	Paramètre de MethodAttributes	340
7.18.5	Paramètre d'ObjectTypeAttributes	340
7.18.6	Paramètre de VariableTypeAttributes	340
7.18.7	Paramètre de ReferenceTypeAttributes	341
7.18.8	Paramètre de DataTypeAttributes	341

7.18.9	Paramètre de ViewAttributes	341
7.19	Paramètres des données de notification (NotificationData)	342
7.19.1	Vue d'ensemble	342
7.19.2	Paramètre de DataChangeNotification	342
7.19.3	Paramètre d'EventNotificationList	343
7.19.4	Paramètre de StatusChangeNotification	343
7.20	Message de notification (NotificationMessage).....	343
7.21	Plage numérique (NumericRange)	344
7.22	Jeu de données de demande (QueryDataSet).....	345
7.23	Lecture de l'identifiant d'une valeur (ReadValueId)	345
7.24	Description d'une référence (ReferenceDescription)	346
7.25	Chemin relatif (RelativePath)	346
7.26	Entête d'une demande (RequestHeader).....	347
7.27	Entête d'une réponse (ResponseHeader).....	348
7.28	Service en défaut (ServiceFault)	349
7.29	Authentification du jeton d'une session (SessionAuthenticationToken)	349
7.30	Signature d'une donnée (SignatureData)	351
7.31	Certificat d'un logiciel signé (SignedSoftwareCertificate).....	352
7.32	Certificat d'un logiciel (SoftwareCertificate).....	352
7.33	Code de statut (StatusCode).....	353
7.33.1	Généralités	353
7.33.2	StatusCodes communs	355
7.34	Horodatages à retourner (TimestampsToReturn).....	358
7.35	Paramètres du jeton d'identité d'utilisateur (UserIdentityToken)	358
7.35.1	Vue d'ensemble	358
7.35.2	AnonymousIdentityToken.....	359
7.35.3	UserNameIdentityToken	359
7.35.4	X509IdentityTokens	360
7.35.5	IssuedIdentityToken.....	360
7.36	Politique pour le jeton utilisateur (UserTokenPolicy)	361
7.37	Description d'une vue (ViewDescription)	362
Annexe A (informative)	Définitions BNF	363
A.1	Vue d'ensemble sur BNF.....	363
A.2	BNF de RelativePath.....	363
A.3	BNF de NumericRange	364
Annexe B (informative)	Composition des Filtres et exemples d'interrogation (Query)	365
B.1	Exemples simples de ContentFilter	365
B.1.1	Vue d'ensemble	365
B.1.2	Exemple 1	365
B.1.3	Exemple 2	366
B.2	Exemples complexes de Filtre d'Interrogation (Query Filter)	367
B.2.1	Vue d'ensemble	367
B.2.2	Modèle de type utilisé	367
B.2.3	Remarques relatives aux exemples	370
B.2.4	Exemple 1	371
B.2.5	Exemple 2	372
B.2.6	Exemple 3	373
B.2.7	Exemple 4	375
B.2.8	Exemple 5	377

B.2.9	Exemple 6	378
B.2.10	Exemple 7	380
B.2.11	Exemple 8	381
B.2.12	Exemple 9	382
Figure 1	– Jeu de services de découverte (Discovery Service Set)	201
Figure 2	– Jeu de services de canal sécurisé (SecureChannel Service Set)	201
Figure 3	– Jeu de services de session (Session Service Set)	202
Figure 4	– Jeu de services de gestion de nœuds (NodeManagement Service Set)	202
Figure 5	– Jeu de services de vues (View Service Set)	203
Figure 6	– Jeu de services d'attributs (Attribute Service Set)	204
Figure 7	– Jeu de services de méthodes (Method Service Set)	204
Figure 8	– Jeux de services d'éléments surveillés et d'abonnements (MonitoredItem et Subscription Service Sets)	205
Figure 9	– Processus de découverte	208
Figure 10	– Utilisation d'un Gateway Server (Serveur passerelle)	213
Figure 11	– Processus d'enregistrement – Serveurs lancés manuellement	215
Figure 12	– Processus d'enregistrement – Serveurs lancés automatiquement	216
Figure 13	– Services SecureChannel et Session	219
Figure 14	– Multiplexage d'utilisateurs sur une Session	225
Figure 15	– Modèle MonitoredItem	263
Figure 16	– Retard type pour détecter des modifications	265
Figure 17	– Gestion du dépassement de capacité de la file d'attente	266
Figure 18	– Modèle de déclenchement	268
Figure 19	– Obtenir et installer un certificat d'instance d'application	294
Figure 20	– Déterminer si un Certificat d'Instance d'Application est Fiable	297
Figure 21	– Etablir un Canal Sécurisé	299
Figure 22	– Etablir une Session	301
Figure 23	– Se substituer à un utilisateur	302
Figure 24	– Obtenir et installer un Certificat de logiciel	303
Figure 25	– Configuration type de la redondance transparente	309
Figure 26	– Configuration de la redondance non transparente	310
Figure 27	– Proxy Serveur pour la redondance	312
Figure 28	– Séquence de reconnexion	314
Figure 29	– Couches logiques d'un <i>Serveur</i>	350
Figure 30	– Obtenir un SessionAuthenticationToken	351
Figure B.1	– Exemple d'arbre logique de filtre	366
Figure B.2	– Exemple d'arbre logique de filtre	366
Figure B.3	– Exemples de nœuds Type	369
Figure B.4	– Exemple de nœuds Instance	370
Figure B.5	– Exemple 1 Filtre	371
Figure B.6	– Exemple 2 Arbre logique d'un filtre	373
Figure B.7	– Exemple 3 Arbre logique d'un filtre	374
Figure B.8	– Exemple 4 Arbre logique d'un filtre	376

Figure B.9 – Exemple 5 Arbre logique d'un filtre	377
Figure B.10 – Exemple 6 Arbre logique d'un filtre	378
Figure B.11 – Exemple 7 Arbre logique d'un filtre	380
Figure B.12 – Exemple 8 Arbre logique d'un filtre	382
Figure B.13 – Exemple 9 Arbre logique d'un filtre	383
Tableau 1 – Tableau de définition des services.....	199
Tableau 2 – Types de paramètre définis dans l'IEC 62541-3.....	200
Tableau 3 – Paramètres du service FindServers	211
Tableau 4 – Paramètres du service GetEndpoints.....	213
Tableau 5 – Paramètres du service RegisterServer.....	217
Tableau 6 – Codes des résultats de services RegisterServer	218
Tableau 7 – Paramètres du service OpenSecureChannel.....	221
Tableau 8 – Codes des résultats de services OpenSecureChannel	222
Tableau 9 – Paramètres du service CloseSecureChannel	222
Tableau 10 – Codes des résultats de services CloseSecureChannel.....	223
Tableau 11 – Paramètres du service CreateSession	226
Tableau 12 – Codes des résultats de services CreateSession	227
Tableau 13 – Paramètres du service ActivateSession	229
Tableau 14 – Codes des résultats de services ActivateSession	230
Tableau 15 – Paramètres du service CloseSession.....	230
Tableau 16 – Codes des résultats de services CloseSession	231
Tableau 17 – Paramètres du service Cancel	231
Tableau 18 – Paramètres du service AddNodes	232
Tableau 19 – Codes des résultats de services AddNodes	232
Tableau 20 – Codes de résultats de niveau opération des AddNodes	233
Tableau 21 – Paramètres du service AddReferences	234
Tableau 22 – Codes des résultats de services AddReferences	234
Tableau 23 – Codes de résultats de niveau opération des AddReferences.....	235
Tableau 24 – Paramètres du service DeleteNodes	235
Tableau 25 – Codes de résultats des services DeleteNodes	236
Tableau 26 – Codes de résultats de niveau opération des DeleteNodes	236
Tableau 27 – Paramètres du service DeleteReferences	237
Tableau 28 – Codes de résultats des services DeleteReferences	237
Tableau 29 – Codes de résultats de niveau opération des DeleteReferences.....	237
Tableau 30 – Paramètres du service Browse	239
Tableau 31 – Codes de résultats des services Browse.....	240
Tableau 32 – Codes de résultats de niveau opération de Browse.....	240
Tableau 33 – Paramètres du service BrowseNext	241
Tableau 34 – Codes de résultats des services BrowseNext.....	241
Tableau 35 – Codes de résultats de niveau opération de BrowseNext	242
Tableau 36 – Paramètres du service TranslateBrowsePathsToNodeIds	243
Tableau 37 – Codes de résultats des services TranslateBrowsePathsToNodeIds	243

Tableau 38 – Codes de résultats de niveau opération de TranslateBrowsePathsToNodeIds	244
Tableau 39 – Paramètres du service RegisterNodes	245
Tableau 40 – Codes de résultats des services RegisterNodes	245
Tableau 41 – Paramètres du service UnregisterNodes	245
Tableau 42 – Codes de résultats des services UnregisterNodes	246
Tableau 43 – Paramètres de demande QueryFirst	248
Tableau 44 – Paramètres de réponse QueryFirst	249
Tableau 45 – Codes de résultats des services QueryFirst	250
Tableau 46 – Codes de résultats de niveau opération de QueryFirst	250
Tableau 47 – Paramètres du service QueryNext	251
Tableau 48 – Codes de résultats des services QueryNext	251
Tableau 49 – Paramètres du service Read	252
Tableau 50 – Codes de résultats des services Read	253
Tableau 51 – Codes de résultats de niveau opération de Read	253
Tableau 52 – Paramètres du service HistoryRead	254
Tableau 53 – Codes de résultats des services HistoryRead	255
Tableau 54 – Codes de résultats de niveau opération de HistoryRead	255
Tableau 55 – Paramètres du service Write	257
Tableau 56 – Codes de résultats des services Write	257
Tableau 57 – Codes de résultats de niveau opération de Write	258
Tableau 58 – Paramètres du service HistoryUpdate	259
Tableau 59 – Codes de résultats des services HistoryUpdate	259
Tableau 60 – Codes de résultats de niveau opération de HistoryUpdate	259
Tableau 61 – Paramètres du service Call	261
Tableau 62 – Codes de résultats des services Call	262
Tableau 63 – Codes de résultats de niveau opération de Call	262
Tableau 64 – Paramètres du service CreateMonitoredItems	270
Tableau 65 – Codes de résultats des services CreateMonitoredItems	271
Tableau 66 – Codes de résultats de niveau opération de CreateMonitoredItems	271
Tableau 67 – Paramètres du service ModifyMonitoredItems	272
Tableau 68 – Codes de résultats des services ModifyMonitoredItems	272
Tableau 69 – Codes de résultats de niveau opération de ModifyMonitoredItems	273
Tableau 70 – Paramètres du service SetMonitoringMode	273
Tableau 71 – Codes de résultats des services SetMonitoringMode	273
Tableau 72 – Codes de résultats de niveau opération de SetMonitoringMode	274
Tableau 73 – Paramètres du service SetTriggering	274
Tableau 74 – Codes de résultats des services SetTriggering	275
Tableau 75 – Codes de résultats de niveau opération de SetTriggering	275
Tableau 76 – Paramètres du service DeleteMonitoredItems	275
Tableau 77 – Codes de résultats des services DeleteMonitoredItems	276
Tableau 78 – Codes de résultats de niveau opération de DeleteMonitoredItems	276
Tableau 79 – États d'un abonnement	279

Tableau 80 – Table d'états d'un abonnement	280
Tableau 81 – Variables d'état et paramètres	282
Tableau 82 – Fonctions	283
Tableau 83 – Paramètres du service CreateSubscription	284
Tableau 84 – Codes de résultats des services CreateSubscription	285
Tableau 85 – Paramètres du service ModifySubscription	285
Tableau 86 – Codes de résultats des services ModifySubscription	286
Tableau 87 – Paramètres du service SetPublishingMode	286
Tableau 88 – Codes de résultats des services SetPublishingMode	287
Tableau 89 – Codes de résultats de niveau opération de SetPublishingMode	287
Tableau 90 – Paramètres du service Publish	289
Tableau 91 – Codes de résultats des services Publish	289
Tableau 92 – Codes de résultats de niveau opération de Publish	289
Tableau 93 – Paramètres du service Republish	290
Tableau 94 – Codes de résultats des services Republish	290
Tableau 95 – Paramètres du service TransferSubscriptions	291
Tableau 96 – Codes de résultats des services TransferSubscriptions	291
Tableau 97 – Codes de résultats de niveau opération de TransferSubscriptions	292
Tableau 98 – Paramètres du service DeleteSubscriptions	292
Tableau 99 – Codes de résultats des services DeleteSubscriptions	292
Tableau 100 – Codes de résultats de niveau opération de DeleteSubscriptions	293
Tableau 101 – Étapes de validation d'un certificat	296
Tableau 102 – Actions de basculement de redondance	311
Tableau 103 – ApplicationDescription	315
Tableau 104 – ApplicationInstanceCertificate	316
Tableau 105 – BrowseResult	316
Tableau 106 – Structure ContentFilter	317
Tableau 107 – Structure ContentFilterResult	317
Tableau 108 – Codes de résultats ContentFilterResult	317
Tableau 109 – Codes de résultats d'opérande ContentFilterResult	318
Tableau 110 – Définition de FilterOperator de base	318
Tableau 111 – Définition de FilterOperator complexe	320
Tableau 112 – Caractères de remplacement	321
Tableau 113 – Règles de conversion	322
Tableau 114 – Règles de préséance des données	323
Tableau 115 – Table de vérité du AND (ET) logique	324
Tableau 116 – Table de vérité du OR (OU) logique	324
Tableau 117 – Typelds du paramètre FilterOperand	324
Tableau 118 – ElementOperand	324
Tableau 119 – LiteralOperand	325
Tableau 120 – AttributeOperand	325
Tableau 121 – SimpleAttributeOperand	326
Tableau 122 – DataValue	327

Tableau 123 – DiagnosticInfo.....	329
Tableau 124 – EndpointDescription.....	330
Tableau 125 – ExpandedNodeId	330
Tableau 126 – Type de base ExtensibleParameter	331
Tableau 127 – Valeurs de MessageSecurityMode	331
Tableau 128 – MonitoringParameters.....	332
Tableau 129 – Typelds du paramètre MonitoringFilter.....	333
Tableau 130 – DataChangeFilter	334
Tableau 131 – Structure EventFilter.....	336
Tableau 132 – Structure EventFilterResult.....	336
Tableau 133 – Codes de résultats EventFilterResult	336
Tableau 134 – Structure AggregateFilter.....	337
Tableau 135 – Structure AggregateFilterResult.....	338
Tableau 136 – Valeurs de MonitoringMode	338
Tableau 137 –Typelds du paramètre NodeAttributes.....	338
Tableau 138 – Masque de bits pour les Attributs spécifiés	339
Tableau 139 – ObjectAttributes	339
Tableau 140 – VariableAttributes	340
Tableau 141 – MethodAttributes	340
Tableau 142 – ObjectTypeAttributes	340
Tableau 143 – VariableTypeAttributes	341
Tableau 144 – ReferenceTypeAttributes	341
Tableau 145 – DataTypeAttributes	341
Tableau 146 – ViewAttributes	342
Tableau 147 – Typelds du paramètre NotificationData	342
Tableau 148 – DataChangeNotification	343
Tableau 149 – EventNotificationList	343
Tableau 150 – StatusChangeNotification	343
Tableau 151 – NotificationMessage	344
Tableau 152 – NumericRange.....	345
Tableau 153 – QueryDataSet.....	345
Tableau 154 – ReadValueId.....	345
Tableau 155 – ReferenceDescription	346
Tableau 156 – RelativePath	347
Tableau 157 – RequestHeader.....	347
Tableau 158 – ResponseHeader	349
Tableau 159 – ServiceFault	349
Tableau 160 – SignatureData.....	352
Tableau 161 – SignedSoftwareCertificate	352
Tableau 162 – SoftwareCertificate	353
Tableau 163 – Affectations de bits de StatusCode	354
Tableau 164 – DataValue InfoBits.....	355
Tableau 165 – Codes de résultats des services communs	356

Tableau 166 – Codes de résultats de niveau opération communs	357
Tableau 167 – Valeurs de TimestampsToReturn	358
Tableau 168 – Typelds du paramètre UserIdentityToken.....	358
Tableau 169 – Format de jeton chiffré UserIdentityToken	359
Tableau 170 – AnonymousIdentityToken.....	359
Tableau 171 – UserNameIdentityToken	360
Tableau 172 – Sélection d'EncryptionAlgorithm	360
Tableau 173 – Jeton d'identité X509	360
Tableau 174 – IssuedIdentityToken.....	361
Tableau 175 – UserTokenPolicy	362
Tableau 176 – ViewDescription.....	362
Tableau A.1 – RelativePath	363
Tableau A.2 – Exemples de <i>RelativePath</i>	364
Tableau B.1 – Exemple de ContentFilter	366
Tableau B.2 – Exemple de ContentFilter	367
Tableau B.3 – Exemple 1 NodeTypeDescription.....	371
Tableau B.4 – Exemple 1 ContentFilter	372
Tableau B.5 – Exemple 1 QueryDataSets	372
Tableau B.6 – Exemple 2 NodeTypeDescription.....	372
Tableau B.7 – Exemple 2 ContentFilter	373
Tableau B.8 – Exemple 2 QueryDataSets	373
Tableau B.9 – Exemple 3 – NodeTypeDescription	374
Tableau B.10 – Exemple 3 ContentFilter	375
Tableau B.11 – Exemple 3 QueryDataSets	375
Tableau B.12 – Exemple 4 NodeTypeDescription.....	376
Tableau B.13 – Exemple 4 ContentFilter	376
Tableau B.14 – Exemple 4 QueryDataSets	376
Tableau B.15 – Exemple 5 NodeTypeDescription.....	377
Tableau B.16 – Exemple 5 ContentFilter	377
Tableau B.17 – Exemple 5 QueryDataSets	378
Tableau B.18 – Exemple 6 NodeTypeDescription.....	378
Tableau B.19 – Exemple 6 ContentFilter	379
Tableau B.20 – Exemple 6 QueryDataSets	379
Tableau B.21 – Exemple 6 QueryDataSets sans informations supplémentaires.....	379
Tableau B.22 – Exemple 7 NodeTypeDescription.....	380
Tableau B.23 – Exemple 7 ContentFilter	381
Tableau B.24 – Exemple 7 QueryDataSets	381
Tableau B.25 – Exemple 8 NodeTypeDescription.....	381
Tableau B.26 – Exemple 8 ContentFilter	382
Tableau B.27 – Exemple 8 QueryDataSets	382
Tableau B.28 – Exemple 9 NodeTypeDescription.....	383
Tableau B.29 – Exemple 9 ContentFilter	383
Tableau B.30 – Exemple 9 QueryDataSets	384

COMMISSION ÉLECTROTECHNIQUE INTERNATIONALE

Architecture unifiée OPC –

Partie 4: Services

AVANT-PROPOS

- 1) La Commission Electrotechnique Internationale (IEC) est une organisation mondiale de normalisation composée de l'ensemble des comités électrotechniques nationaux (Comités nationaux de l'IEC). L'IEC a pour objet de favoriser la coopération internationale pour toutes les questions de normalisation dans les domaines de l'électricité et de l'électronique. À cet effet, l'IEC – entre autres activités – publie des Normes internationales, des Spécifications techniques, des Rapports techniques, des Spécifications accessibles au public (PAS) et des Guides (ci-après dénommés "Publication(s) de l'IEC"). Leur élaboration est confiée à des comités d'études, aux travaux desquels tout Comité national intéressé par le sujet traité peut participer. Les organisations internationales, gouvernementales et non gouvernementales, en liaison avec l'IEC, participent également aux travaux. L'IEC collabore étroitement avec l'Organisation Internationale de Normalisation (ISO), selon des conditions fixées par accord entre les deux organisations.
- 2) Les décisions ou accords officiels de l'IEC concernant les questions techniques représentent, dans la mesure du possible, un accord international sur les sujets étudiés, étant donné que les Comités nationaux de l'IEC intéressés sont représentés dans chaque comité d'études.
- 3) Les Publications de l'IEC se présentent sous la forme de recommandations internationales et sont agréées comme telles par les Comités nationaux de l'IEC. Tous les efforts raisonnables sont entrepris afin que l'IEC s'assure de l'exactitude du contenu technique de ses publications; l'IEC ne peut pas être tenue responsable de l'éventuelle mauvaise utilisation ou interprétation qui en est faite par un quelconque utilisateur final.
- 4) Dans le but d'encourager l'uniformité internationale, les Comités nationaux de l'IEC s'engagent, dans toute la mesure possible, à appliquer de façon transparente les Publications de l'IEC dans leurs publications nationales et régionales. Toutes divergences entre toutes Publications de l'IEC et toutes publications nationales ou régionales correspondantes doivent être indiquées en termes clairs dans ces dernières.
- 5) L'IEC elle-même ne fournit aucune attestation de conformité. Des organismes de certification indépendants fournissent des services d'évaluation de conformité et, dans certains secteurs, accèdent aux marques de conformité de l'IEC. L'IEC n'est responsable d'aucun des services effectués par les organismes de certification indépendants.
- 6) Tous les utilisateurs doivent s'assurer qu'ils sont en possession de la dernière édition de cette publication.
- 7) Aucune responsabilité ne doit être imputée à l'IEC, à ses administrateurs, employés, auxiliaires ou mandataires, y compris ses experts particuliers et les membres de ses comités d'études et des Comités nationaux de l'IEC, pour tout préjudice causé en cas de dommages corporels et matériels, ou de tout autre dommage de quelque nature que ce soit, directe ou indirecte, ou pour supporter les coûts (y compris les frais de justice) et les dépenses découlant de la publication ou de l'utilisation de cette Publication de l'IEC ou de toute autre Publication de l'IEC, ou au crédit qui lui est accordé.
- 8) L'attention est attirée sur les références normatives citées dans cette publication. L'utilisation d'éditions référencées est obligatoire pour une application correcte de la présente publication.
- 9) L'attention est attirée sur le fait que certains des éléments de la présente Publication de l'IEC peuvent faire l'objet de droits de brevet. L'IEC ne saurait être tenue pour responsable de ne pas avoir identifié de tels droits de brevets et de ne pas avoir signalé leur existence.

La Norme internationale IEC 62541-4 a été établie par le sous-comité 65E: Les dispositifs et leur intégration dans les systèmes de l'entreprise, du comité d'études 65 de l'IEC: Mesure, commande et automation dans les processus industriels.

Cette deuxième édition annule et remplace la première édition parue en 2011. Cette édition constitue une révision technique.

Cette édition inclut les modifications techniques majeures suivantes par rapport à l'édition précédente:

- a) Mise à jour de 6.4 Redondance.
Ajout de l'option de redondance non transparente HotAndMirrored et reformulation dans l'ensemble de la description de la redondance.

- b) Clarifications pour les scénarios Publish et Reconnect.
Reformulation de différentes parties de la spécification pour s'assurer qu'aucune donnée n'est perdue pendant de courtes interruptions de la communication et que les clients peuvent toujours détecter la durée de la perte d'informations pendant une interruption de la connexion. Ajout d'un nouveau paragraphe 6.5 Rétablir les connexions qui décrit la séquence exacte de reconnexion pour les clients n'ayant plus de connexion à un serveur. Modification de l'exigence minimale relative à la file d'attente de retransmission pour l'envoi des NotificationMessages entre un intervalle d'entretien et au minimum deux fois le nombre minimal de demandes Publish par Session. Clarification concernant la valeur de données dans laquelle le bit de dépassement de capacité est établi en fonction du réglage de rejet le plus ancien. Modification de la gestion de rejet lorsque discardOldest est FALSE. La nouvelle valeur remplace l'ancienne mise en file d'attente pour FALSE. Ajout de l'exception selon laquelle le bit de dépassement de capacité n'est pas établi si la taille de la file d'attente est un.
- c) Gestion des modifications de MonitoredItem dans les scénarios de courte interruption du réseau.
Ajout d'une nouvelle méthode GetMonitoredItems dans la Partie 5. Cette méthode peut être utilisée pour établir la liste des éléments surveillés dans un abonnement en cas d'échec de CreateMonitoredItems en raison d'une interruption du réseau et lorsque le client ne sait pas si la création a été réussie dans le serveur.
- d) Mise à jour de 6.1.3 Déterminer si un Certificat est fiable
Révision des règles de validation du certificat.
- e) Révision de la définition des paramètres semaphoreFile et isOnline dans le Service RegisterServer
- f) Services ModifySubscription et ModifyMonitoredItems
Clarification précisant que les modifications s'appliquent directement et prennent effet le plus tôt possible sans toutefois dépasser une durée égale à deux fois le nouvel intervalle de temps.
- g) Établissement d'une longue liste de modifications mineures permettant de lever les ambiguïtés.

Le texte de cette norme est issu des documents suivants:

CDV	Rapport de vote
65E/375/CDV	65E/403/RVC

Le rapport de vote indiqué dans le tableau ci-dessus donne toute information sur le vote ayant abouti à l'approbation de cette norme.

Cette publication a été rédigée selon les Directives ISO/IEC, Partie 2.

Une liste de toutes les parties de la série IEC 62541, publiées sous le titre général *Architecture Unifiée OPC*, peut être consultée sur le site web de l'IEC.

Le comité a décidé que le contenu de cette publication ne sera pas modifié avant la date de stabilité indiquée sur le site web de l'IEC sous "<http://webstore.iec.ch>" dans les données relatives à la publication recherchée. A cette date, la publication sera

- reconduite,
- supprimée,
- remplacée par une édition révisée, ou
- amendée.

IMPORTANT – Le logo "*colour inside*" qui se trouve sur la page de couverture de cette publication indique qu'elle contient des couleurs qui sont considérées comme utiles à une bonne compréhension de son contenu. Les utilisateurs devraient, par conséquent, imprimer cette publication en utilisant une imprimante couleur.

Architecture unifiée OPC –

Partie 4: Services

1 Domaine d'application

La présente partie de l'IEC 62541 définit les *Services* de l'Architecture Unifiée OPC (OPC UA). Les *Services* décrits sont le recueil d'appels de procédures abstraites distantes (RPC – Remote Procedure Calls) qui sont mises en œuvre par les *Serveurs* OPC UA et qui sont appelées par les *Clients* OPC UA. Toutes les interactions entre *Clients* et *Serveurs* OPC UA ont lieu via ces *Services*. Les *Services* définis sont considérés comme abstraits, car pour leur mise en œuvre aucun mécanisme RPC particulier n'est spécifié dans la présente partie. L'IEC 62541-6 spécifie le(les) mécanisme(s) concret(s) de mise en correspondance (mapping) pour la mise en œuvre. Par exemple, dans l'IEC 62541-6, un des mécanismes de mise en correspondance est basé sur l'utilisation des Services Web XML. Dans ce cas, les *Services* décrits dans la présente partie apparaissent comme les méthodes de service web dans le contrat WSDL.

Tous les *Serveurs* OPC UA n'auront pas besoin de mettre en œuvre la totalité des *Services* définis. L'IEC 62541-7 définit les *Profiles* (profils) qui dictent les *Services* qui auront besoin d'être mis en œuvre afin d'être conformes à un *Profile* particulier.

2 Références normatives

Les documents suivants sont cités en référence de manière normative, en intégralité ou en partie, dans le présent document et sont indispensables pour son application. Pour les références datées, seule l'édition citée s'applique. Pour les références non datées, la dernière édition du document de référence s'applique (y compris les éventuels amendements).

IEC TR 62541-1, *OPC Unified Architecture – Part 1: Overview and Concepts* (disponible en anglais seulement)

IEC TR 62541-2, *OPC Unified Architecture – Part-2: Security Model* (disponible en anglais seulement)

IEC 62541-3, *OPC Unified Architecture – Part 3: Address Space Model* (disponible en anglais seulement)

IEC 62541-5, *Architecture unifiée OPC – Partie 5: Modèle d'Information*

IEC 62541-6, *Architecture unifiée OPC – Partie 6: Correspondances*

IEC 62541-7, *Architecture unifiée OPC – Partie 7: Profils*

IEC 62541-8, *Architecture unifiée OPC – Partie 8: Accès aux données*

IEC 62541-11, *OPC Unified Architecture – Part 11: Historical Access* (disponible en anglais seulement)

IEC 62541-13, *OPC Unified Architecture – Part 13: Aggregates* (disponible en anglais seulement)

3 Termes, définitions et conventions

3.1 Termes et définitions

Pour les besoins du présent document, les termes et définitions donnés dans l'IEC TR 62541-1, l'IEC TR 62541-2 et l'IEC 62541-3 ainsi que les suivants s'appliquent.

3.1.1

bande morte (Deadband)

plage permise pour les variations de valeur qui ne déclencheront pas une *Notification* de changement de données

Note 1 à l'article: *Bande morte* peut s'appliquer comme un filtre lors de l'abonnement à des *Variables* et sert à empêcher que des signaux bruyants ne mettent inutilement un *Client* à jour. La présente Norme définit *AbsoluteDeadband* comme un filtre commun. L'IEC 62541-8 définit un filtre *Deadband* supplémentaire.

3.1.2

point d'extrémité (Endpoint)

adresse physique disponible sur un réseau qui permet à des *Clients* d'accéder à un ou plusieurs *Services* fournis par un *Serveur* (server)

Note 1 à l'article: Chaque *Serveur* peut avoir plusieurs *Points d'extrémité*. L'adresse d'un *Point d'extrémité* doit inclure un *HostName* (un nom d'hôte).

3.1.3

serveur passerelle (Gateway Server)

serveur qui agit comme un intermédiaire pour un ou plusieurs *Serveurs*

Note 1 à l'article: Les *Serveurs passerelles* peuvent être déployés pour limiter l'accès externe, assurer la conversion de protocole ou fournir des caractéristiques que les *Serveurs* sous-jacents ne prennent pas en charge.

3.1.4

nom d'hôte (HostName)

identificateur unique pour une machine sur un réseau

Note 1 à l'article: Cet identificateur doit être unique dans un réseau local, mais il peut toutefois être également unique au niveau global. L'identificateur peut être une adresse IP.

3.1.5

jeton de sécurité (Security Token)

identificateur pour un jeu de clés cryptographiques

Note 1 à l'article: Tous les *Security Tokens* (Jetons de sécurité) appartiennent à un contexte de sécurité qui est, dans le cas d'OPC UA, le canal sécurisé (*Secure Channel*).

3.1.6

certificat de logiciel (SoftwareCertificate)

certificat numérique pour un produit logiciel qui peut être installé sur plusieurs hôtes pour décrire les fonctions du produit logiciel

Note 1 à l'article: Des installations différentes du même produit logiciel peuvent avoir le même certificat de logiciel. Les certificats de logiciel ne sont pas pertinents pour la sécurité. Ils sont utilisés pour identifier un produit logiciel et ses caractéristiques prises en charge. Les certificats de logiciel sont décrits en 6.2.

3.2 Abréviations et symboles

API Application Programming Interface (Interface de programmation d'application)

BNF Backus-Naur Form (Notation Backus-Naur)

CA Certificate Authority (Autorité de certification)

CRL Certificate Revocation List (Liste d'annulation de certificats)

CTL Certificate Trust List (Liste de certificats de confiance)

DA	Data Access (Accès aux données)
NAT	Network Address Translation (Traduction d'adresse réseau)
UA	Unified Architecture (Architecture Unifiée)
URI	Uniform Resource Identifier (Identificateur uniforme de ressource)
URL	Uniform Resource Locator (Localisateur uniforme de ressource)

3.3 Conventions pour les définitions des Services

Les *Services* OPC UA ont des paramètres qui sont acheminés entre le *Client* et le *Serveur*. Les spécifications des *Services* OPC UA utilisent des tableaux pour décrire les paramètres des *Services* comme montré au Tableau 1. Dans ce tableau, les paramètres sont organisés en paramètres de demande et paramètres de réponse.

Tableau 1 – Tableau de définition des services

Nom	Type	Description
Request (Demande)		Définit les paramètres de demande relatifs au <i>Service</i>
Nom de paramètre simple		Description de ce paramètre
Nom de paramètre construit		Description du paramètre construit
Nom de paramètre composant		Description du paramètre composant
Response (Réponse)		Définit les paramètres de réponse relatifs au <i>Service</i>

Les colonnes Nom, Type et Description donnent le nom, le type de données et la description de chaque paramètre. Tous les paramètres sont obligatoires, même si certains peuvent être inutilisés dans certaines circonstances. La colonne Description spécifie la valeur à fournir lorsqu'un paramètre est inutilisé.

Deux types de paramètres sont définis dans ces tableaux, à savoir simple et construit. Les paramètres simples ont un type de données simple, tel que *Boolean* (booléen) ou *String* (chaîne).

Les paramètres construits sont des paramètres composés de deux paramètres ou plus, qui peuvent être simples ou construits. Les noms des paramètres composants sont indentés sous le nom du paramètre construit.

Les types de données utilisés dans ces tableaux peuvent être des types de base, des types communs à plusieurs *Services* ou des types spécifiques à chaque *Service*. Les types de données de base sont définis dans l'IEC 62541-3. Les types de base utilisés dans les *Services* sont énumérés dans le Tableau 2. Les types de données qui sont communs à plusieurs *Services* sont définis à l'Article 7. Les types de données qui sont spécifiques à un *Service* sont définis dans le tableau de paramètres du *Service* en question.

Tableau 2 – Types de paramètre définis dans l'IEC 62541-3

Type du paramètre
BaseDataType
Boolean
ByteString
Double
Duration
Guid
Int32
LocaleId
NodeId
QualifiedName
String
UInt16
UInt32
UInteger
UtcTime
XmlElement

Les paramètres des primitives du service Request et Indication sont représentés au Tableau 1 comme des paramètres Request. De même, les paramètres des primitives du service Response et Confirmation sont représentés au Tableau 1 comme des paramètres Response. Tous les paramètres de demande et de réponse sont inchangés et acheminés entre l'expéditeur et le récipiendaire. Par conséquent, des colonnes distinctes pour les valeurs des paramètres de demande, d'indication, de réponse et de confirmation sont inutiles et ont été intentionnellement omises afin d'améliorer la lisibilité.

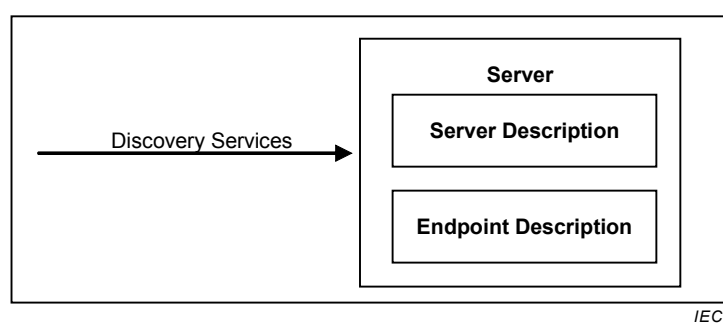
4 Vue d'ensemble

4.1 Modèle de jeux de services

Le présent article spécifie les *Services* OPC UA. Les définitions des *Services* OPC UA sont des descriptions abstraites, mais ne sont pas une spécification de mise en œuvre (implémentation). La mise en correspondance entre les descriptions abstraites et la *Communication Stack (Pile de communication)* associée à ces *Services* sont définies dans l'IEC 62541-6. Dans le cas d'une implémentation basée sur des services web, les *Services* OPC UA correspondent au service web et un *Service* OPC UA correspond à une opération du service web.

Ces *Services* sont organisés en *Service Sets* (Jeux de services). Chaque *Service Set* définit un jeu de *Services* connexes. L'organisation en *Service Sets* est un regroupement logique présenté dans la présente Norme, mais qui n'est pas utilisé pour l'implémentation.

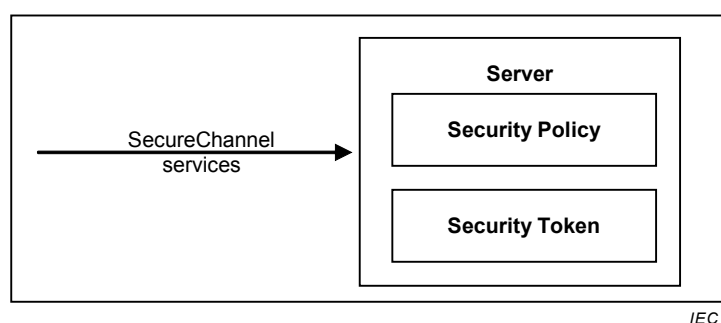
Le *Discovery Service Set* (Jeu de services de découverte), illustré à la Figure 1, définit les *Services* qui permettent à un *Client* de découvrir les *Endpoints* implémentés par un *Serveur* et de connaître la configuration de sécurité de chacun de ces *Endpoints*.



Anglais	Français
Discovery services	Services Discovery
Server	Serveur
Server description	Description du serveur
Endpoint description	Description du point d'extrémité

Figure 1 – Jeu de services de découverte (Discovery Service Set)

Le *SecureChannel Service Set* (Jeu de services de canal sécurisé), illustré à la Figure 2, définit les *Services* qui permettent à un *Client* d'établir un canal de communication pour assurer la *Confidentiality* (Confidentialité) et l'*Integrity* (Intégrité) des *Messages* échangés avec le *Serveur*.



Anglais	Français
SecureChannel services	Services SecureChannel
Server	Serveur
Security Policy	Politique de sécurité
Security Token	Jeton de sécurité

Figure 2 – Jeu de services de canal sécurisé (SecureChannel Service Set)

Le *Session Service Set* (Jeu de services de session), illustré à la Figure 3, définit les *Services* qui permettent au *Client* d'authentifier l'utilisateur au nom duquel il agit et de gérer des *Sessions*.

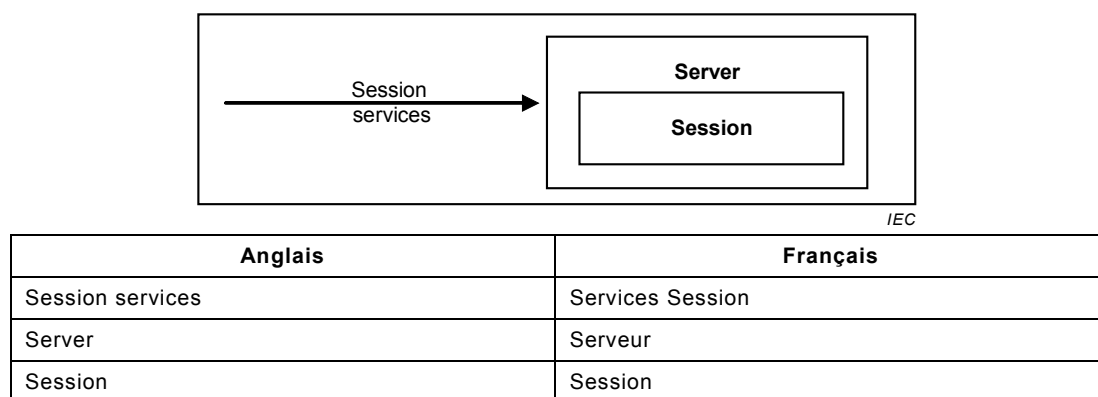


Figure 3 – Jeu de services de session (Session Service Set)

Le *NodeManagement Service Set* (Jeu de services de gestion de nœuds), illustré à la Figure 4, définit les *Services* qui permettent au *Client* d'ajouter, de modifier et de supprimer des *Nodes* (Nœuds) dans l'*AddressSpace* (Espace d'adresses).

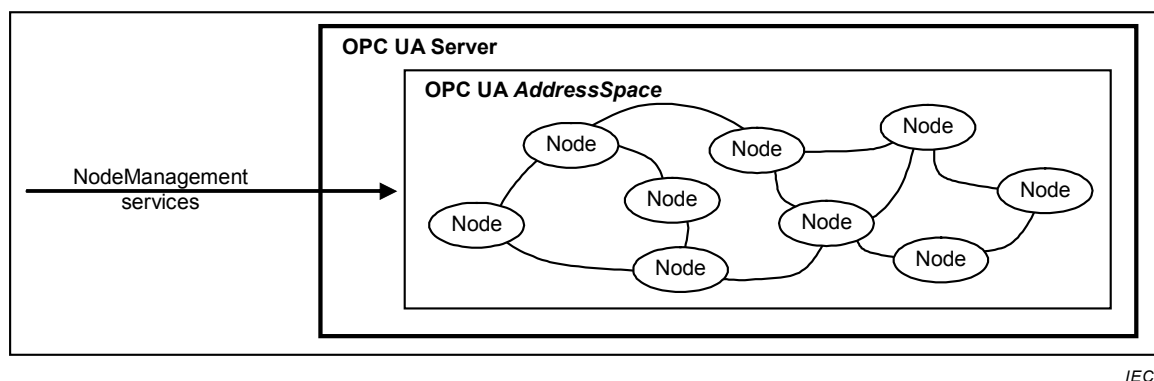
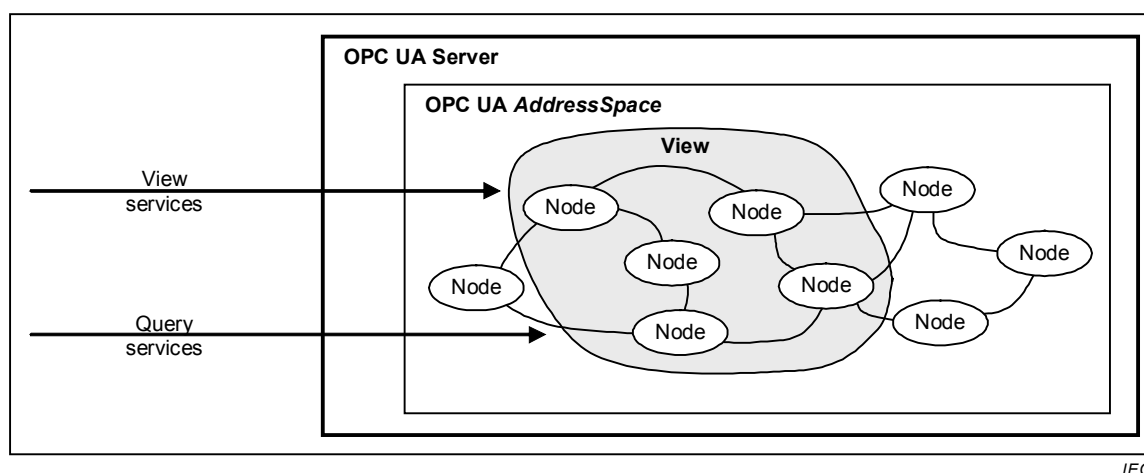


Figure 4 – Jeu de services de gestion de nœuds (NodeManagement Service Set)

Le *View Service Set* (Jeu de services de vues), illustré à la Figure 5, définit les *Services* qui permettent aux *Clients* de parcourir l'*Espace d'Adresses* ou des sous-ensembles de l'*Espace d'Adresses* appelés *Views* (Vues). Le *Query Service Set* (Jeu de services d'interrogation) permet aux *Clients* de récupérer un sous-ensemble de données de l'*Espace d'Adresses* ou de la *Vue*.

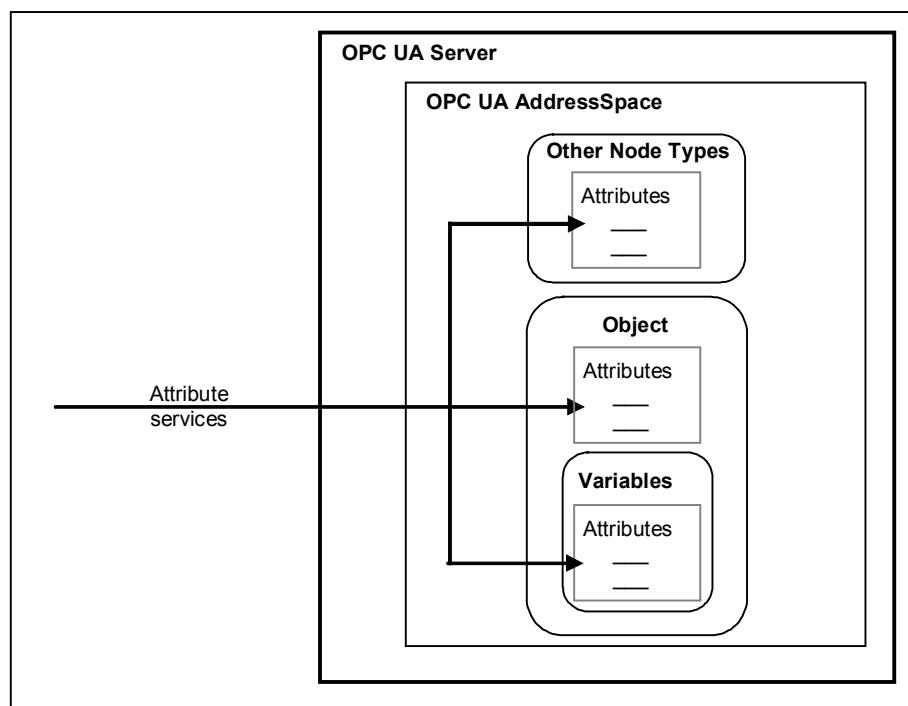


IEC

Anglais	Français
View services	Services View (vues)
Query services	Services Query (interrogation)
OPC UA Server	Serveur OPC UA
OPC UA AddressSpace	Espace d'adresses OPC UA
Node	Nœud
View	Vue

Figure 5 – Jeu de services de vues (View Service Set)

L'*Attribute Service Set* (Jeu de services d'attributs) est illustré à la Figure 6. Il définit les *Services* qui permettent aux *Clients* de lire et écrire les *Attributs* de *Nœuds*, y compris les valeurs de l'historique. La valeur d'une *Variable* étant modélisée comme *Attribut*, ces *Services* permettent aux *Clients* de lire et écrire les valeurs des *Variables*.



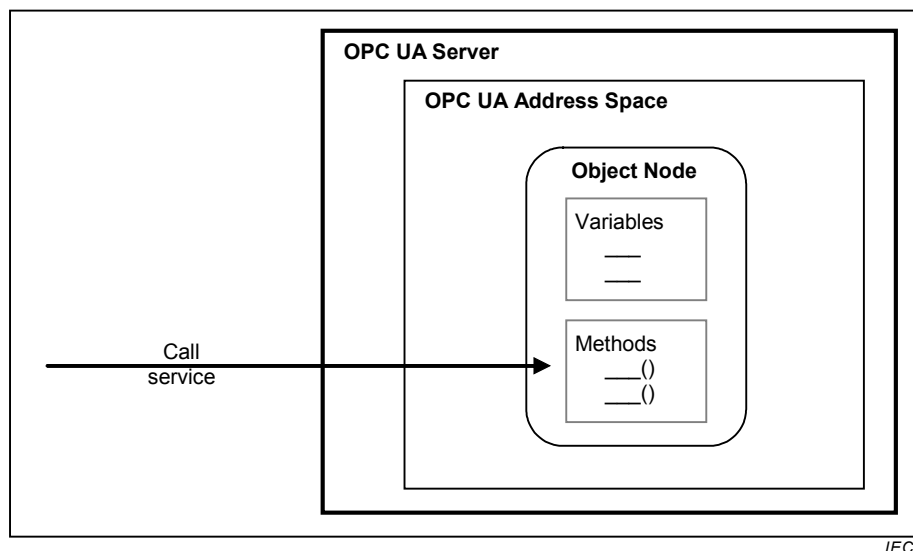
IEC

Anglais	Français
Attribute services	Services d'attributs

Anglais	Français
Other Node Types	Autres types de nœuds
OPC UA Server	Serveur OPC UA
OPC UA AddressSpace	Espace d'adresses OPC UA
Attributes	Attributs
Object	Objet
Variables	Variables

Figure 6 – Jeu de services d'attributs (Attribute Service Set)

Le *Method Service Set* (Jeu de services de méthodes) est illustré à la Figure 7. Il définit les *Services* qui permettent aux *Clients* d'appeler des méthodes. Les méthodes s'exécutent totalement lorsqu'elles sont appelées. Elles peuvent être appelées avec des paramètres d'entrée spécifiques à la méthode et peuvent retourner des paramètres de sortie spécifiques à la méthode.



IEC

Anglais	Français
Call service	Service Call
Object Node	Object Node
OPC UA Server	Serveur OPC UA
OPC UA AddressSpace	Espace d'adresses OPC UA
Methods	Méthodes
Variables	Variables

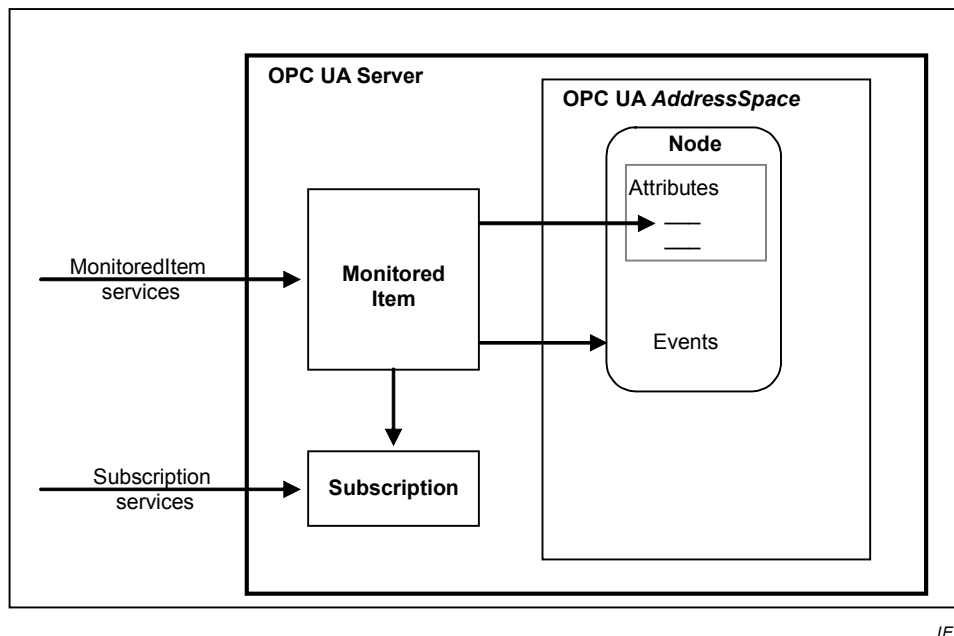
Figure 7 – Jeu de services de méthodes (Method Service Set)

Le *MonitoredItem Service Set* (Jeu de services d'éléments surveillés) et le *Subscription Service Set* (Jeu de services d'abonnements), illustrés à la Figure 8, sont utilisés ensemble pour s'abonner à des *Nœuds* de l'*Espace d'Adresses* OPC UA.

Le *MonitoredItem Service Set* définit les *Services* qui permettent aux *Clients* de créer, modifier et supprimer des *MonitoredItems* (Éléments surveillés) utilisés pour surveiller les *Attributs* afin de déceler des changements de valeur et les *Objects* afin de détecter des *Événements*.

Ces *Notifications* sont mises en file d'attente en vue de leur transfert vers le *Client* en utilisant le mécanisme de *Subscriptions* (Abonnements).

Le *Subscription Service Set* (Jeu de services d'abonnements) définit les *Services* qui permettent aux *Clients* de créer, modifier et supprimer des *Subscriptions*. Les *Subscriptions* envoient des *Notifications* générées par des *MonitoredItems* au *Client*. Les *Subscription Services* (Services d'abonnements) prévoient aussi la reprise *Client* après des *Messages* manqués et des défaillances de communications.



IEC

Anglais	Français
MonitoredItem services	Services MonitoredItem (éléments surveillés)
Subscription services	Services Subscription (abonnements)
Node	Nœud
OPC UA Server	Serveur OPC UA
OPC UA AddressSpace	Espace d'adresses OPC UA
Attributes	Attributs
Events	Événements
MonitoredItem	Élément surveillé
Subscription	Abonnement

**Figure 8 – Jeux de services d'éléments surveillés et d'abonnements
(MonitoredItem et Subscription Service Sets)**

4.2 Procédures des services Request/Response (Demande/Réponse)

Les procédures des *Services Request/Response* décrivent le traitement des demandes reçues par le *Serveur* et des réponses renvoyées ensuite. Les procédures débutent par la présentation au *Serveur* d'un *Message* de demande de *Service* de la part du *Client* demandeur.

À la réception de la demande, le *Serveur* traite le *Message* en deux étapes. Dans la première étape, il tente de décoder et de localiser le *Service* à exécuter. La gestion des erreurs pour cette étape est spécifique à la technologie de communication utilisée et elle est décrite dans l'IEC 62541-6.

S'il réussit, il tente d'accéder à chaque opération identifiée dans la demande et exécute l'opération demandée. Pour chaque opération indiquée dans la demande, il génère un code de succès/échec distinct qu'il insère dans un *Message* de réponse positive accompagné des éventuelles données qui sont à retourner.

Pour exécuter ces opérations, le *Client* et le *Serveur* peuvent se servir de l'API de la *Pile de communication* pour construire et interpréter les *Messages* et accéder à l'opération demandée.

La mise en œuvre de la gestion de chaque demande de service ou de chaque réponse doit vérifier que chaque paramètre de service s'inscrit bien dans la plage spécifiée pour chaque paramètre.

5 Jeux de services (Service Sets)

5.1 Généralités

Le présent article définit les *Service Sets* d'OPC UA et leurs *Services*. L'Article 7 contient les définitions des paramètres communs utilisés par ces *Services*. Les exigences relatives à l'audit sont décrites en 6.2 pour tous les services.

Le *Server Profile* définit si un *Serveur* prend ou non en charge un *Service Set*, ou un *Service* inclus dans un *Service Set*. Les *Profiles* sont décrits dans l'IEC 62541-7.

5.2 En-tête de demande de services et de réponse

Chaque demande de *Service* a un *RequestHeader* (En-tête de demande) et chaque réponse relative à un *Service* a un *ResponseHeader* (En-tête de réponse).

La structure du *RequestHeader* est définie en 7.26 et contient les paramètres communs aux demandes tels qu'*authenticationToken* (Jeton d'authentification), *timestamp* (Horodatage) et *requestHandle* (Gestion de demande).

La structure du *ResponseHeader* est définie en 7.27 et contient des paramètres communs aux réponses tels que *serviceResult* (Résultat de service) et *diagnosticInfo* (Informations de diagnostic).

5.3 Résultats des services

Les résultats d'un *Service* sont communiqués à deux niveaux dans les réponses OPC UA: l'un indique le statut de l'appel de *Service* tandis que l'autre indique le statut de chaque opération demandée par le *Service*.

Les résultats de *Service* sont définis via le *StatusCode* (Code de statut) (voir 7.33).

Le statut de l'appel de *Service* est représenté par le *serviceResult* contenu dans le *ResponseHeader* (voir 7.27). Le mécanisme pour retourner ce paramètre est spécifique à la technologie de communication utilisée pour acheminer la réponse de *Service* et il est défini dans l'IEC 62541-6.

Le statut des opérations individuelles d'une demande est représenté par des *StatusCodes* individuels.

Les cas ci-après définissent l'utilisation de ces paramètres.

- a) Un code bad (mauvais) est retourné dans le *serviceResult* si le *Service* lui-même a failli. Dans ce cas, un *ServiceFault* (Défaut de service) est renvoyé. Le *ServiceFault* est défini en 7.28.
- b) Le code good (bon) est retourné dans le *serviceResult* si le *Service* a partiellement ou totalement réussi. Dans ce cas, d'autres paramètres de réponse sont retournés. Le *Client* doit toujours vérifier les paramètres de réponse, notamment tous les *StatusCodes* associés à chaque opération. Ces *StatusCodes* peuvent indiquer des résultats incorrects ou incertains pour une ou plusieurs opérations demandées lors de l'appel de *Service*.

Tous les *Services* avec des matrices d'opérations dans la demande doivent retourner un code bad dans le *serviceResult* si la matrice est vide.

Les *Services* définissent divers *StatusCodes* spécifiques et un *Serveur* doit utiliser ces *StatusCodes* spécifiques comme décrit dans le *Service*. Il convient qu'un *Client* puisse gérer ces *StatusCodes* spécifiques au *Service*. En outre, un *Client* doit prendre en compte d'autres *StatusCodes* communs définis au Tableau 165 et au Tableau 166. Des détails complémentaires pour la gestion de *StatusCodes* spécifiques par le *Client* peuvent être définis dans l'IEC 62541-7.

Si le *Serveur* découvre, par des mécanismes hors bande, que l'intégrité des authentifiants de l'application ou de l'utilisateur utilisés pour créer une *Session* ou un *Canal Sécurisé* a été compromise, il convient que le *Serveur* interrompe immédiatement toutes les sessions et les canaux qui utilisent ces authentifiants. Dans ce cas, il convient que le code de résultat du *Service* soit *Bad_IdentityTokenRejected* ou *Bad_CertificateUntrusted*.

L'analyse de message peut échouer en raison d'erreurs de syntaxe ou si les données du message sortent de la plage prise en charge par le récipiendaire. Lorsque cela se produit, les messages doivent être rejetés par le récipiendaire. Si le récipiendaire est un *Serveur*, il doit retourner un *ServiceFault* avec le code de résultat *Bad_DecodingError* ou *Bad_EncodingLimitsExceeded*. Si le récipiendaire est le *Client*, il convient que la *Pile de Communication* rende compte de ces erreurs à l'application *Client*.

De nombreuses applications imposent des limites à la taille des messages et/ou des éléments de données contenus dans ces messages. Par exemple, un *Serveur* peut rejeter des demandes contenant des valeurs chaînes plus longues qu'une certaine longueur. Ces limites sont typiquement établies par des administrateurs et s'appliquent à toutes les connexions entre un *Client* et un *Serveur*.

Les *Clients* qui reçoivent des défauts *Bad_EncodingLimitsExceeded* de la part du *Serveur* auront vraisemblablement à reformuler leurs demandes. L'administrateur peut avoir besoin d'augmenter les limites pour le *Client* s'il reçoit une réponse du *Serveur* comportant ce défaut.

Dans certains cas, les erreurs d'analyse sont fatales et il n'est pas possible de retourner un défaut. Par exemple, le message entrant peut dépasser la capacité du tampon du récipiendaire. Dans ces cas, ces erreurs peuvent être traitées comme un défaut de communication qui exige le rétablissement du *Canal Sécurisé* (voir 5.5).

Le *Client* et le *Serveur* réduisent les risques d'erreur fatale en échangeant leurs limites de taille de message dans le service *CreateSession*. Cela permet à chaque partie prenante d'éviter d'envoyer un message qui génère un défaut de communication. Il convient que le *Serveur* retourne un défaut *Bad_ResponseTooLarge* si un message de réponse sérialisé dépasse la taille de message spécifiée par le *Client*. De même, il convient que la *Pile de communication (Communication Stack)* du *Client* rapporte une erreur *Bad_RequestTooLarge* à l'application avant d'envoyer un message qui dépasse la limite du *Serveur*.

À signaler que les limites de taille de message s'appliquent seulement au corps de message et n'incluent pas les en-têtes ou les effets d'une application d'une quelconque politique de sécurité. Cela signifie qu'un corps de message qui est plus petit que le maximum spécifié peut tout de même générer une erreur fatale.

5.4 Jeu de services de découverte (Discovery Service Set)

5.4.1 Vue d'ensemble

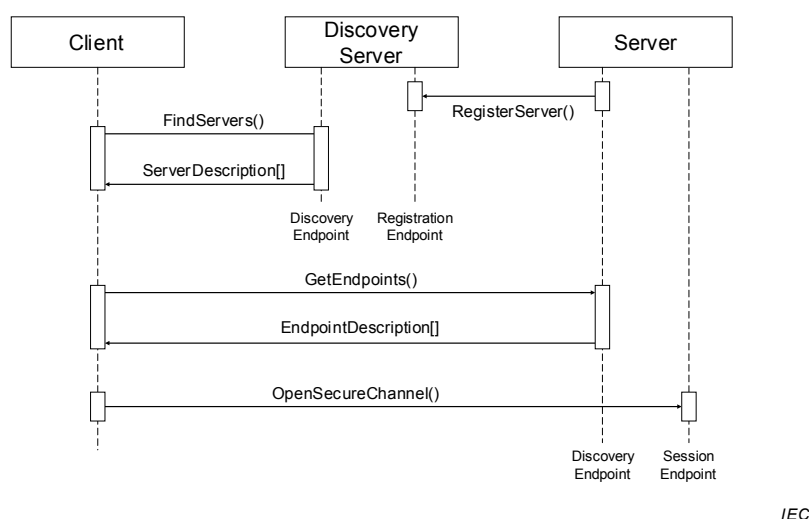
Le *Service Set* définit les *Services* utilisés pour découvrir les *Endpoints (Points d'extrémité)* implémentés par un *Serveur* et pour lire la configuration de sécurité pour ces *Endpoints*. Les *Discovery Services* (Services de Découverte) sont mis en œuvre par des *Serveurs* individuels

et par des *Discovery Servers* (Serveurs de découverte) spécialisés. L'IEC 62541-12¹ décrit comment les *Discovery Servers* spécialisés utilisent les *Discovery Services*.

Chaque *Serveur* doit avoir un *Discovery Endpoint* (Point d'extrémité de découverte) auquel les *Clients* accèdent sans établir une *Session*. Cet *Endpoint* peut ou peut ne pas être le même *Session Endpoint* (Point d'extrémité de session) que les *Clients* utilisent pour établir un *Canal Sécurisé*. Les clients lisent les informations de sécurité nécessaires pour établir un *Canal Sécurisé* en appelant le *GetEndpoints Service* (Service de récupération des points d'extrémité) sur le *Discovery Endpoint*.

En outre, les *Serveurs* peuvent s'enregistrer auprès d'un *Discovery Server* connu en utilisant le service *RegisterServer* (Enregistrement de serveur). Les *Clients* peuvent découvrir plus tard tous les éventuels *Serveurs* enregistrés en appelant le *FindServers Service* (Service Trouver des serveurs) sur le *Discovery Server*.

Le processus de découverte complet est illustré à la Figure 9.



Anglais	Français
Discovery serveur	Serveur de découverte
Server	Serveur
Discovery endpoint	Point d'extrémité de découverte
Registration endpoint	Point d'extrémité d'enregistrement
Session endpoint	Point d'extrémité de session

Figure 9 – Processus de découverte

L'URL (localisateur uniforme de ressource, *Uniform Resource Locator*) pour un *Discovery Endpoint* doit fournir toutes les informations dont le *Client* a besoin pour se connecter au *Discovery Endpoint*.

Une fois qu'un *Client* récupère les *Endpoints*, le *Client* peut sauvegarder cette information et l'utiliser pour se reconnecter directement au *Serveur* sans passer par le processus de découverte. Si le *Client* s'aperçoit qu'il ne peut pas se connecter, la configuration du *Serveur* a pu changer et il est nécessaire que le *Client* repasse par le processus de découverte.

¹ A l'étude.

Les *Discovery Endpoints* ne doivent nécessiter aucune sécurité pour les messages, mais ils peuvent requérir une sécurité de la couche transport. Dans les systèmes de production, les *Administrateurs* peuvent désactiver la découverte pour des raisons de sécurité et les *Clients* doivent compter sur des *EndpointDescriptions* (descriptions de points d'extrémité) placées en cache. Pour prendre en charge des systèmes ayant les *Discovery Services* désactivés, les *Clients* doivent permettre aux *Administrateurs* de mettre à jour manuellement les *EndpointDescriptions* utilisées pour se connecter à un *Serveur*. Les *Serveurs* doivent permettre aux *Administrateurs* de désactiver le *Discovery Endpoint*.

Le *Client* doit être très prudent pour utiliser les informations retournées par un *Discovery Endpoint*, car elles ne sont pas sûres. Le *Client* le fait en comparant les informations retournées par le *Discovery Endpoint* aux informations retournées dans la réponse *CreateSession*. Le *Client* doit s'assurer que:

- a) L'*ApplicationUri* spécifié dans le *Serveur Certificate* est le même que l'*ApplicationUri* fourni dans l'*EndpointDescription*.
- b) Le *Server Certificate* retourné dans la réponse *CreateSession* est le même que le *Certificate* utilisé pour créer le *Canal Sécurisé*.
- c) Les *EndpointDescriptions* retournées par le *Discovery Endpoint* sont les mêmes que les *EndpointDescriptions* retournées dans la réponse *CreateSession*.

Si le *Client* détecte que l'une des exigences ci-dessus n'est pas remplie, le *Client* doit alors fermer le *Canal Sécurisé* et émettre un rapport d'erreur.

Le *Client* doit vérifier que le *HostName* spécifié dans le *Server Certificate* est le même que le *HostName* contenu dans l'*endpointUrl* fourni dans l'*EndpointDescription* retournée par *CreateSession*. Si ce n'est pas le cas, le *Client* doit rapporter la différence et peut fermer le *Canal Sécurisé*. Les *Serveurs* doivent ajouter tous les *HostNames* possibles tels que *MyHost* et *MyHost.Local* dans le *Server Certificate*. Ceci comprend les adresses IP de l'hôte ou le *HostName* présenté par un routeur NAT utilisé pour se connecter au *Serveur*.

5.4.2 Trouver des serveurs (FindServers)

5.4.2.1 Description

Ce *Service* retourne les *Serveurs* connus auprès d'un *Serveur* ou d'un *Discovery Server*. Le comportement des *Discovery Servers* est décrit en détail dans l'IEC 62541-12².

Le *Client* peut réduire le nombre de résultats retournés en spécifiant des critères de filtrage. Un *Discovery Server* retourne une liste vide si aucun *Serveur* ne correspond aux critères spécifiés par le client. Les critères de filtrage pris en charge par ce *Service* sont décrits en 5.4.2.2.

Chaque *Serveur* doit fournir un *Discovery Endpoint* qui prend en charge ce *Service*. Le *Serveur* doit toujours retourner un enregistrement qui le décrit, cependant, dans certains cas, plusieurs enregistrements peuvent être retournés. Les *Gateway Servers* (serveurs passerelles) doivent retourner un enregistrement pour chaque *Serveur* auquel ils accordent l'accès et en plus de manière facultative un enregistrement qui permet au *Gateway Server* (serveur passerelle) d'être accessible comme un *Serveur* OPC UA ordinaire. Les *Serveurs* redondants non transparents doivent fournir un enregistrement pour chaque *Serveur* dans le jeu redondant.

Chaque *Serveur* doit avoir un identificateur globalement unique appelé *ServerUri*. Il convient que cet identificateur soit un nom de domaine totalement qualifié; cependant, cela peut être un GUID ou construct similaire qui assure l'unicité au niveau global. Le *ServerUri* retourné par ce *Service* doit avoir la valeur qui apparaît en indice 0 de la propriété *ServerArray* (matrice de

² A l'étude.

serveurs) (voir l'IEC 62541-5). Le *ServerUri* est retourné comme le champ *applicationUri* dans l'*ApplicationDescription* (voir 7.1)

Chaque *Serveur* doit aussi avoir un identificateur interprétable par l'utilisateur appelé *ServerName* (nom de serveur) qui au niveau global n'est pas nécessairement unique. Cet identificateur peut être disponible dans plusieurs paramètres de lieu.

Un *Serveur* peut avoir plusieurs *HostNames*. C'est pourquoi le *Client* doit passer à ce *Service* l'URL qu'il a utilisé pour se connecter à l'*Endpoint*. L'implémentation de ce *Service* doit utiliser cette information pour retourner des réponses qui sont accessibles au *Client* via l'URL fourni.

Ce *Service* ne doit nécessiter aucune sécurité pour les messages, mais il peut requérir une sécurité de la couche transport.

Certains *Serveurs* peuvent être accessibles via un *Gateway Server* et doivent avoir une valeur spécifiée pour *gatewayServerUri* (URI de serveur passerelle) dans leur *ApplicationDescription* (voir 7.1). Les *discoveryUrls* fournis dans *ApplicationDescription* doivent appartenir au *Gateway Server*. Certains *Discovery Servers* peuvent retourner plusieurs enregistrements pour le même *Serveur* si le *Serveur* en question peut être accessible par plusieurs chemins.

Ce *Service* peut être utilisé sans sécurité et il est donc vulnérable aux attaques par déni de service (DOS, *Denial Of Service*). Il convient qu'un *Serveur* réduise au maximum les traitements requis pour envoyer une réponse via ce *Service*. Cela peut être fait en préparant le résultat à l'avance. Il convient que le *Serveur* ajoute aussi un faible retard avant de commencer le traitement d'une demande s'il y a beaucoup de demandes.

Le *DiscoveryUrl* retourné par ce *Service* est ambigu s'il y a plusieurs *TransportProfiles* (par exemple, codage UA XML ou binaire UA) associés au schéma d'URL. Il convient que les *Clients* qui prennent en charge plusieurs *TransportProfiles* tentent d'utiliser des *TransportProfiles* de remplacement si le premier choix échoue.

5.4.2.2 Paramètres

Le Tableau 3 définit les paramètres pour le *Service*.

Tableau 3 – Paramètres du service FindServers

Nom	Type	Description
Request		
requestHeader	RequestHeader	Paramètres communs aux demandes. L' <i>authenticationToken</i> (jeton d'authentification) est toujours omis. S'il est fourni, l' <i>authenticationToken</i> doit être ignoré. Le type <i>RequestHeader</i> est défini en 7.26.
endpointUrl	String	L'adresse réseau que le <i>Client</i> a utilisée pour accéder au <i>Discovery Endpoint</i> . Le <i>Serveur</i> utilise cette information pour des diagnostics et pour déterminer les URL à retourner dans la réponse. Il convient que le <i>Serveur</i> retourne une URL par défaut adéquat s'il ne reconnaît pas le <i>HostName</i> dans l'URL.
localeIds []	LocaleId	Liste des paramètres de lieu à utiliser. Il convient que le serveur retourne le <i>ServerName</i> en utilisant un des paramètres de lieu spécifiés. Si le serveur prend en charge plus d'un des paramètres de lieu demandés, il doit utiliser celui qui apparaît le premier dans la liste. Si le serveur ne prend en charge aucun des paramètres de lieu demandés, il choisit un paramètre de lieu adéquat par défaut. Le serveur choisit un paramètre de lieu adéquat par défaut si cette liste est vide.
serverUris []	String	Liste des serveurs à retourner. Tous les serveurs connus sont retournés si la liste est vide. Un <i>serverUri</i> correspond à l' <i>applicationUri</i> de l' <i>ApplicationDescription</i> défini en 7.1.
Response		
responseHeader	ResponseHeader	Paramètres communs aux réponses. Le type <i>ResponseHeader</i> est défini en 7.27.
servers []	ApplicationDescription	Liste des <i>Serveurs</i> qui répondent aux critères spécifiés dans la demande. Cette liste est vide si aucun serveur ne répond aux critères. Le type <i>ApplicationDescription</i> est défini en 7.1.

5.4.2.3 Résultats des services

Les *StatusCodes* communs sont définis au Tableau 165.

5.4.3 Déterminer les points d'extrémité (GetEndpoints)

5.4.3.1 Description

Ce *Service* retourne les *Endpoints* pris en charge par un *Serveur* et la totalité des informations de configuration nécessaires pour établir un *Canal Sécurisé* et une *Session*.

Cela ne doit nécessiter aucune sécurité pour les messages, mais peut requérir une sécurité de la couche transport.

Un *Client* peut réduire le nombre de résultats retournés en spécifiant des critères de filtrage sur la base de *LocaleIds* et d'URI de *Transport Profile*. Le *Serveur* retourne une liste vide si aucun *Endpoint* ne correspond aux critères spécifiés par le client. Les critères de filtrage pris en charge par *Service* sont décrits en 5.4.3.2.

Un *Serveur* peut prendre en charge plusieurs configurations de sécurité pour le même *Endpoint*. Dans cette situation, le *Serveur* doit retourner des enregistrements d'*EndpointDescription* distincts pour chaque configuration disponible. Il convient que les *Clients* traitent chacune de ces configurations comme des *Endpoints* distincts même si l'URL physique se trouve être le même.

La configuration de sécurité pour un *Endpoint* comporte quatre composants:

- Server Application Instance Certificate (Certificat d'instance d'application serveur)
- Message Security Mode (Mode de sécurité de message)
- Security Policy (Politique de sécurité)
- Supported User Identity Tokens (Jetons d'identité d'utilisateur pris en charge)

L'*ApplicationInstanceCertificate* est utilisé pour sécuriser la demande d'*OpenSecureChannel* (voir 5.5.2). Le *MessageSecurityMode* et le *SecurityPolicy* indiquent au *Client* comment sécuriser les messages envoyés via le *Canal Sécurisé*. Les *UserIdentityTokens* (jetons d'identité d'utilisateur) indiquent au client le type d'authentifiant d'utilisateur qui doit être passé au *Serveur* dans la demande *ActivateSession* (voir 5.6.3).

Si le *securityPolicyUri* est NONE et qu'aucune *UserTokenPolicy* ne nécessite de chiffrement, le *Client* doit ignorer l'*ApplicationInstanceCertificate*.

Chaque *EndpointDescription* spécifie aussi un URI pour le *Transport Profile* (profil de transport) que l'*Endpoint* prend en charge. Les *Transport Profiles* (Profils de transport) spécifient les informations telles que le format de codage de message et la version de protocole et sont définis dans l'IEC 62541-7. Les *Clients* doivent chercher les *SoftwareCertificates* (Certificats de logiciel) du *Serveur* s'ils veulent découvrir la liste complète des *Profiles* pris en compte par le *Serveur* (voir 7.30).

Les messages sont sécurisés en appliquant aux messages avant de les envoyer sur le réseau des algorithmes de chiffrement normalisés. Le jeu exact d'algorithmes utilisés dépend de la *SecurityPolicy* pour l'*Endpoint*. L'IEC 62541-7 définit des *Profiles* pour des *SecurityPolicies* communes et leur affecte un URI unique. Il est prévu que les applications connaissent les *SecurityPolicies* (Politiques de sécurité) qu'elles prennent en charge et, par conséquent, seul l'URI de profil pour la *SecurityPolicy* est spécifié dans l'*EndpointDescription*. Un *Client* ne peut pas se connecter à un *Endpoint* qui ne prend pas en charge une *SecurityPolicy* qu'il connaît.

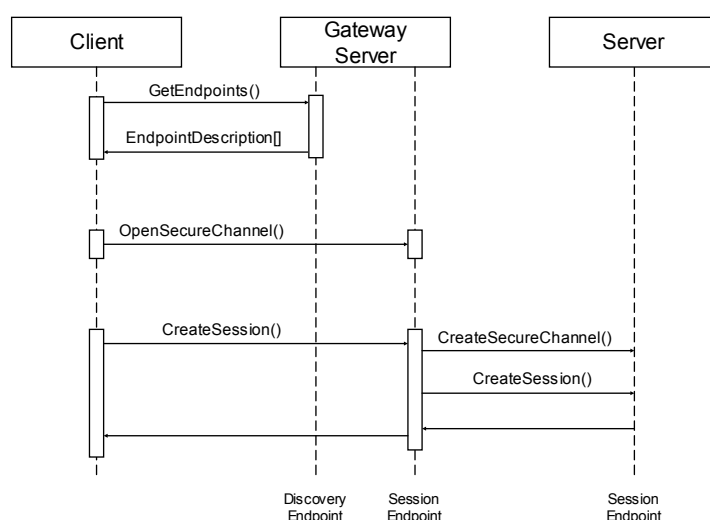
Une *EndpointDescription* peut spécifier que le mode de sécurité de message est NONE (Aucun/Rien). Cette configuration n'est pas recommandée sauf si les applications communiquent sur un réseau physiquement isolé dans lequel le risque d'intrusion est extrêmement faible. Si la sécurité de message est NONE, il est possible pour les *Clients* de pirater, délibérément ou accidentellement, des *Sessions* créées par d'autres *Clients*.

Un *Serveur* peut avoir plusieurs *HostNames*. Pour cette raison, le *Client* doit passer à ce *Service* l'URL qu'il a utilisée pour se connecter à l'*Endpoint*. L'implémentation de ce *Service* doit utiliser cette information pour retourner des réponses qui sont accessibles au *Client* via l'URL fournie.

Ce *Service* peut être utilisé sans sécurité et il est donc vulnérable aux attaques par déni de service (DOS, Denial Of Service). Il convient qu'un *Serveur* réduise au maximum la quantité de traitement nécessaire pour envoyer la réponse à ce *Service*. Cela peut être fait en préparant le résultat à l'avance. Il convient que le *Serveur* ajoute aussi un faible retard avant de commencer le traitement d'une demande s'il y a beaucoup de demandes.

Certaines des *EndpointDescriptions* retournées dans une réponse doivent spécifier les informations d'*Endpoint* pour un *Gateway Server* qui peut être utilisé pour accéder à un autre *Serveur*. Dans ce cas, le *gatewayServerUri* (l'URI de serveur passerelle) est spécifié dans l'*EndpointDescription* et tous les contrôles de sécurité utilisés pour vérifier les *Certificates* doivent utiliser le *gatewayServerUri* (voir 6.1.3) au lieu du *serverUri* (URI du serveur).

Pour se connecter à un *Serveur* via la passerelle, le *Client* doit d'abord établir un *Canal Sécurisé* avec le *Gateway Server*. Le *Client* doit appeler le service *CreateSession* et passe au *Gateway Server* le *serverUri* spécifié dans l'*EndpointDescription*. Le *Gateway Server* doit alors se connecter au *Serveur* sous-jacent pour le compte du *Client*. Le processus de connexion au *Serveur* via un *Gateway Server* est illustré à la Figure 10.



Anglais	Français
Gateway server	Serveur passerelle
Server	Serveur
Discovery endpoint	Point d'extrémité de découverte
Session endpoint	Point d'extrémité de session

Figure 10 – Utilisation d'un Gateway Server (Serveur passerelle)

5.4.3.2 Paramètres

Le Tableau 4 définit les paramètres pour le *Service*.

Tableau 4 – Paramètres du service GetEndpoints

Nom	Type	Description
Request		
requestHeader	RequestHeader	Paramètres communs aux demandes. L' <i>authenticationToken</i> (jeton d'authentification) est toujours omis. S'il est fourni, l' <i>authenticationToken</i> doit être ignoré. Le type <i>RequestHeader</i> est défini en 7.26.
endpointUrl	String	L'adresse réseau que le <i>Client</i> a utilisée pour accéder au <i>Discovery Endpoint</i> . Le <i>Serveur</i> utilise cette information pour des diagnostics et pour déterminer les URL à retourner dans la réponse. Il convient que le <i>Serveur</i> retourne une URL adéquate par défaut s'il ne reconnaît pas le <i>HostName</i> dans l'URL.
localeIds []	LocaleId	Liste des paramètres de lieu à utiliser. Spécifie le paramètre de lieu à utiliser pour retourner des chaînes lisibles par l'homme. Ce paramètre est décrit en 5.4.2.2.
profileUris []	String	Liste de <i>Transport Profile</i> que doivent prendre en charge les <i>Endpoints</i> retournés. L'IEC 62541-7 définit les URI pour les <i>Transport Profiles</i> . Tous les <i>Endpoints</i> sont retournés si la liste est vide.
Response		
responseHeader	ResponseHeader	Paramètres communs aux réponses. Le type <i>ResponseHeader</i> est défini en 7.27.
Endpoints []	EndpointDescription	Liste des <i>Endpoints</i> qui répondent aux critères spécifiés dans la demande. Cette liste est vide si aucun <i>Endpoint</i> ne répond aux critères. Le type <i>EndpointDescription</i> est défini en 7.9.

5.4.3.3 Résultats des services

Les *StatusCodes* communs sont définis au Tableau 165.

5.4.4 Enregistrer un serveur (RegisterServer)

5.4.4.1 Description

Ce *Service* enregistre un *Serveur* auprès d'un *Discovery Server* (Serveur de découverte). Ce *Service* est appelé par un *Serveur* ou par un utilitaire de configuration distinct. Les *Clients* n'utilisent pas ce *Service*.

Un *Serveur* doit établir un *Canal Sécurisé* avec le *Discovery Server* avant d'appeler ce *Service*. Le *Canal Sécurisé* est décrit en 5.5. L'*Administrateur* du *Serveur* doit fournir au *Serveur* une *EndpointDescription* pour le *Discovery Server*, cela fait partie intégrante du processus de configuration. Les *Discovery Servers* doivent rejeter les enregistrements si le *serverUri* fourni ne correspond pas à l'*applicationUri* contenue dans le *Server Certificate* utilisé pour créer le *Canal Sécurisé*.

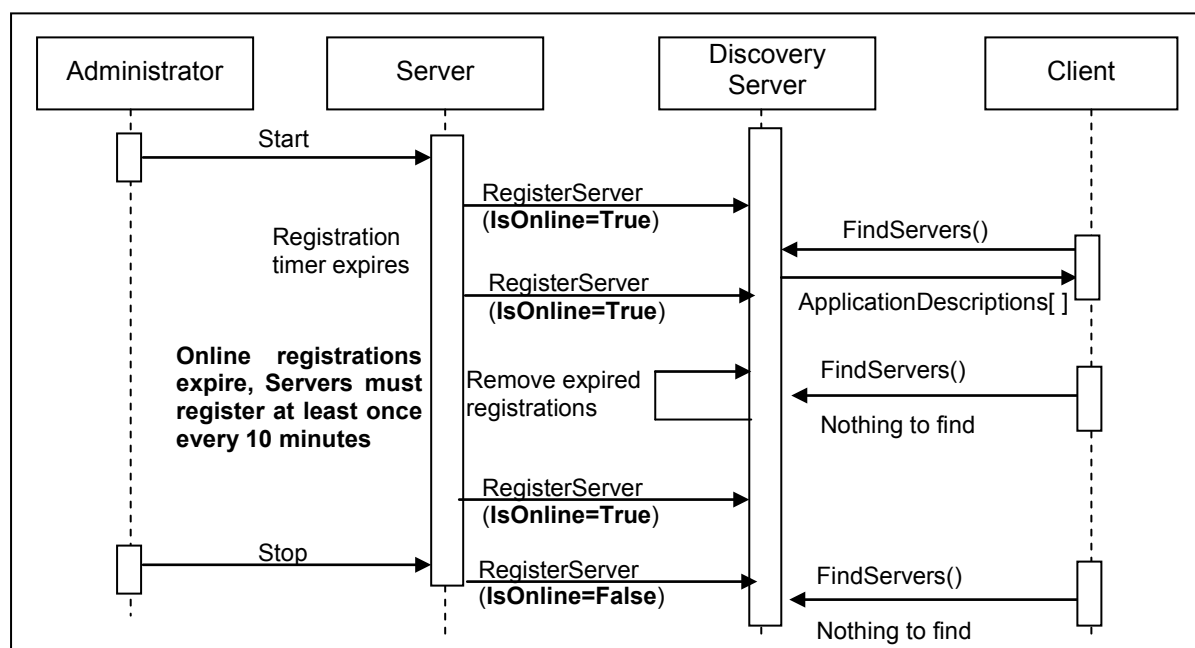
Un *Serveur* fournit uniquement son *serverUri* et les URL des *Discovery Endpoints* au *Discovery Server*. Les *Clients* doivent utiliser le service *GetEndpoints* pour rechercher directement auprès du *Serveur* les informations de configurations les plus récentes.

Le *Serveur* doit fournir un nom localisé pour lui-même dans tous paramètres de lieu qu'il prend en charge.

Les *Serveurs* doivent être à même de s'enregistrer auprès d'un *Discovery Server* tournant sur la même machine. Les mécanismes exacts dépendent de l'implémentation du *Discovery Server* et sont décrits dans l'IEC 62541-6.

Il existe deux types d'applications d'un *Serveur*: celles qui sont lancées manuellement, y compris par un démarrage par le système d'exploitation à l'amorçage et celles qui sont lancées automatiquement lorsqu'un *Client* tente de se connecter. Le processus d'enregistrement qu'un *Serveur* doit utiliser dépend de la catégorie dans laquelle il s'inscrit.

Le processus d'enregistrement pour les *Serveurs* lancés manuellement est illustré à la Figure 11.



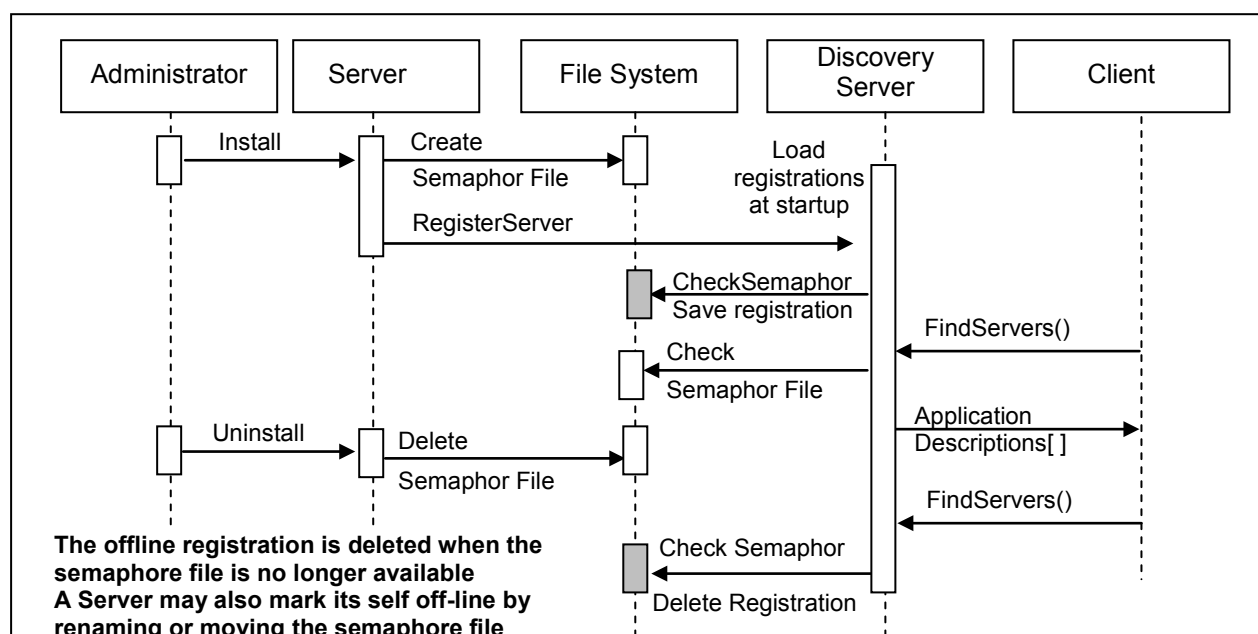
IEC

Anglais	Français
Administrator	Administrateur

Anglais	Français
Server	Serveur
Discovery server	Serveur de découverte
Registration timer expires	La minuterie d'enregistrement expire
Online registrations expire, servers must register at least once every 10 minutes	L'enregistrement en ligne expire, les serveurs doivent s'enregistrer au moins toutes les 10 minutes
Remove expired registrations	Retirer les enregistrements expirés
Start	Démarrage
Stop	Arrêt
Nothing to find	Rien à trouver

Figure 11 – Processus d'enregistrement – Serveurs lancés manuellement

Le processus d'enregistrement pour les *Serveurs* lancés automatiquement est illustré à la Figure 12.



IEC

Anglais	Français
Administrator	Administrateur
Server	Serveur
File system	Système de fichiers
Install	Installer
Create	Créer
Semaphor file	Fichier sémaphore
Load registrations at startup	Charger les enregistrements au démarrage
Save registration	Sauvegarder les enregistrements
Delete	Supprimer
Uninstall	Désinstaller
Check semaphor	Vérifier le fichier sémaphore
Delete registration	Supprimer l'enregistrement
The offline registration is deleted when the semaphor file is no longer available	L'enregistrement hors ligne est supprimé lorsque le fichier sémaphore n'est plus disponible

Anglais	Français
A server may also mark itself off-line by renaming or moving the semaphore file	Un serveur peut aussi se marquer lui-même hors ligne en renommant ou en déplaçant le fichier sémaphore

Figure 12 – Processus d'enregistrement – Serveurs lancés automatiquement

Le processus d'enregistrement est conçu pour être indépendant de la plateforme, être robuste et capable de réduire au maximum les problèmes créés par des erreurs de configuration. Pour cette raison, les *Serveurs* doivent s'enregistrer plus d'une fois.

Dans des conditions normales, les *Serveurs* lancés manuellement doivent s'enregistrer régulièrement auprès du *Discovery Server* tant qu'ils peuvent recevoir des connexions des *Clients*. Si un *Serveur* passe hors ligne, il doit s'enregistrer une fois de plus et indiquer s'il passe hors ligne. Il convient que la fréquence d'enregistrement soit configurable; cependant, la valeur maximale est de 10 minutes. Si une erreur se produit au cours de l'enregistrement (par exemple, le *Discovery Server* n'est pas en marche), le *Serveur* doit de nouveau tenter périodiquement de s'enregistrer. Il convient que la fréquence de ces tentatives commence à 1 s mais augmente progressivement jusqu'à ce que la fréquence d'enregistrement atteigne la valeur qu'elle aurait eue si aucune erreur ne s'était produite. L'approche recommandée est de doubler la période de chaque tentative jusqu'à ce que le maximum soit atteint.

Lorsqu'un *Serveur* lancé automatiquement (ou son programme d'installation) s'enregistre auprès du *Discovery Server*, il doit fournir un chemin vers un fichier sémaphore que le *Discovery Server* peut utiliser pour déterminer si le *Serveur* a été désinstallé de la machine. Le *Discovery Server* doit avoir eu accès au système de fichiers qui contient le fichier.

5.4.4.2 Paramètres

Le Tableau 5 définit les paramètres pour le *Service*.

Tableau 5 – Paramètres du service RegisterServer

Nom	Type	Description
Request		
requestHeader	RequestHeader	Paramètres communs aux demandes. L' <i>authenticationToken</i> (jeton d'authentification) est toujours omis. Le type <i>RequestHeader</i> est défini en 7.26.
Server	RegisteredServer	Le serveur à enregistrer. Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.
serverUri	String	L'identificateur globalement unique pour l'instance de <i>Serveur</i> . Le <i>serverUri</i> correspond à l' <i>applicationUri</i> de l' <i>ApplicationDescription</i> défini en 7.1.
productUri	String	L'identificateur globalement unique pour le produit <i>Serveur</i> .
serverNames []	LocalizedText	Une liste des noms descriptifs localisés pour le <i>Serveur</i> . La liste doit avoir au moins une entrée valide.
serverType	Enum ApplicationType	Le type d'application. Les valeurs d'énumération sont définies au Tableau 103. La valeur "CLIENT_1" (L'application est un <i>Client</i> .) n'est pas autorisée. Le résultat du <i>Service</i> doit être <i>Bad_InvalidArgument</i> dans ce cas.
gatewayServerUri	String	L'URI du <i>Gateway Server</i> associé aux <i>discoveryUrls</i> . Cette valeur est uniquement spécifiée par les <i>Gateway Servers</i> qui souhaitent enregistrer les <i>Serveurs</i> auxquels ils donnent accès. Pour les <i>Serveurs</i> qui n'agissent pas comme un <i>Gateway Server</i> , ce paramètre doit être null.
discoveryUrls []	String	Une liste de <i>Discovery Endpoints</i> pour le <i>Serveur</i> . La liste doit avoir au moins une entrée valide.
semaphoreFilePath	String	Le chemin menant au fichier sémaphore utilisé pour identifier l'instance de serveur lancé automatiquement. Les serveurs lancés manuellement n'utiliseront pas ce paramètre. Si un fichier sémaphore est fourni, le fanion <i>isOnline</i> est ignoré. Si un fichier sémaphore est fourni et existe, le <i>LocalDiscoveryServer</i> doit sauvegarder les informations relatives à l'enregistrement dans une unité de stockage persistante de données qu'il lit lorsque le <i>LocalDiscoveryServer</i> démarre. Si un fichier sémaphore est spécifié, mais n'existe pas, le <i>Discovery Server</i> doit retirer l'enregistrement de l'unité de stockage persistante de données. Si le <i>Serveur</i> a enregistré avec un <i>semaphoreFilePath</i> , le <i>DiscoveryServer</i> doit vérifier que ce fichier existe avant de renvoyer l' <i>ApplicationDescription</i> au client. Si le <i>Serveur</i> n'a pas enregistré avec un <i>semaphoreFilePath</i> (s'il est null ou vide) le <i>DiscoveryServer</i> ne tente pas de vérifier l'existence du fichier avant de renvoyer l' <i>ApplicationDescription</i> au client.
isOnline	Boolean	True (vrai) si le <i>Serveur</i> est actuellement capable d'accepter des connexions de <i>Clients</i> . Le <i>DiscoveryServer</i> doit renvoyer l' <i>ApplicationDescriptions</i> au <i>Client</i> . Il est prévu que le <i>Serveur</i> se réenregistre régulièrement avec le <i>DiscoveryServer</i> . False (faux) si le <i>Serveur</i> est réellement incapable d'accepter des connexions de <i>Clients</i> . Le <i>DiscoveryServer</i> ne doit PAS renvoyer les <i>ApplicationDescriptions</i> au <i>Client</i> . Ce paramètre est ignoré si un <i>semaphoreFilePath</i> est fourni.
Response		
ResponseHeader	ResponseHeader	Paramètres communs aux réponses. Le type <i>ResponseHeader</i> est défini en 7.27.

5.4.4.3 Résultats des services

Le Tableau 6 définit les résultats de *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 6 – Codes des résultats de services RegisterServer

Id symbolique	Description
Bad_InvalidArgument	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_ServerUrlInvalid	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_ServerNameMissing	Aucun <i>ServerName</i> n'a été spécifié.
Bad_DiscoveryUrlMissing	Aucun <i>DiscoveryUrl</i> n'a été spécifié.
Bad_SemaphoreFileMissing	Le fichier sémaphore spécifié n'est pas valide.

5.5 Jeu de services de canal sécurisé (SecureChannel Service Set)

5.5.1 Vue d'ensemble

Ce *Service Set* (jeu de services) définit les *Services* utilisés pour ouvrir un canal de communication qui assure la *Confidentiality* et l'*Integrity* de tous les *Messages* échangés avec le *Serveur*. Les concepts de base pour la sécurité OPC UA sont définis dans l'IEC TR 62541-2.

Les *SecureChannel Services* (services de canal sécurisé) ne sont pas comme d'autres *Services*, car ils ne sont pas implémentés directement par l'*Application OPC UA*. En revanche, ils sont fournis par la *Pile de Communication* sur laquelle est bâtie l'*Application OPC UA*. Par exemple, un *Serveur OPC UA* peut être bâti sur une pile SOAP (Simple Object Access Protocol, Protocole d'accès à objet simple) qui permet à des applications d'établir un *Canal Sécurisé* en utilisant la spécification WS Secure Conversation. Dans ces cas, l'*Application OPC UA* doit vérifier que le *Message* qu'elle a reçu était dans le contexte d'une WS Secure Conversation. L'IEC 62541-6 décrit comment les *Services SecureChannel* sont mis en œuvre.

Un *Canal Sécurisé* est une connexion logique de longue durée établie entre un seul *Client* et un seul *Serveur*. Ce canal maintient un jeu de clés connues seulement du *Client* et du *Serveur*, qui sont utilisées pour authentifier et chiffrer des *Messages* à travers le réseau. Les *Services SecureChannel* permettent au *Client* et au *Serveur* de négocier en toute sécurité les clés à utiliser.

Une *EndpointDescription* indique au *Client* comment établir un *Canal Sécurisé* avec un *Endpoint* donné. Un *Client* peut obtenir l'*EndpointDescription* provenant d'un *Discovery Server*, via quelque serveur non défini par l'UA, ou provenant de sa propre configuration.

Les algorithmes exacts utilisés pour authentifier et chiffrer les *Messages* sont décrits dans le champ *SecurityPolicy* de l'*EndpointDescription*. Un *Client* doit utiliser ces algorithmes lorsqu'il crée un *Canal Sécurisé*.

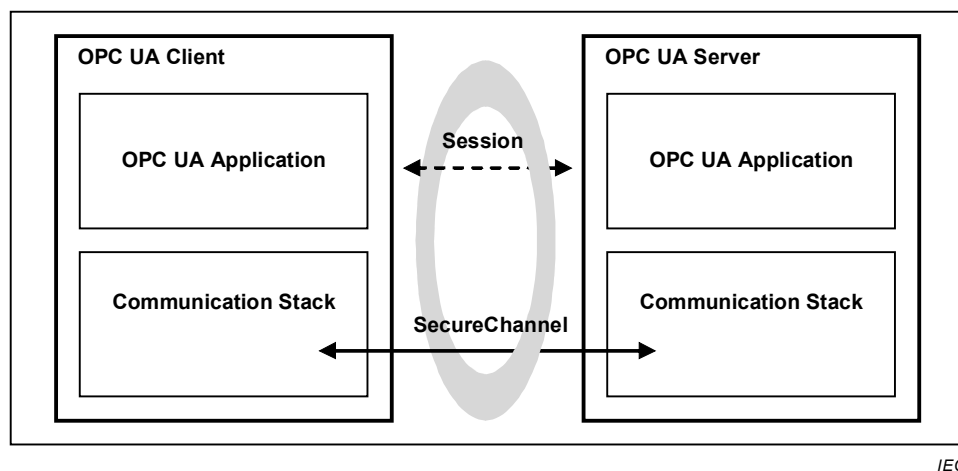
Il convient de noter que certaines *SecurityPolicies* définies dans l'IEC 62541-7 désactivent l'authentification et le chiffrement, conduisant à un *Canal Sécurisé* qui ne procure aucune sécurité.

Lorsqu'un *Client* et un *Serveur* sont en communication via un *Canal Sécurisé*, ils doivent vérifier que tous les *Messages* entrants ont été signés et chiffrés conformément aux exigences spécifiées dans l'*EndpointDescription*. Une *Application OPC UA* ne doit traiter aucun *Message* qui n'est pas conforme à ces exigences.

La relation entre le *Canal Sécurisé* et l'*Application OPC UA* dépend de la technologie d'implémentation. L'IEC 62541-6 définit toutes les exigences induites par la technologie utilisée.

La corrélation entre une *Session d'Application OPC UA* et le *Canal Sécurisé* est illustrée à la Figure 13. La *Pile de Communication* (*Communication Stack*) est utilisée par les *Applications OPC UA* pour échanger des *Messages*. Dans une première étape, les *Services SecureChannel* sont utilisés pour établir un *Canal Sécurisé* entre les deux *Piles de Communication* qui permet l'échange sécurisé de *Messages*. Dans une seconde étape, les

Applications OPC UA utilisent le *Session Service Set* (Jeu de services de session) pour établir une *Session d'Application OPC UA*.



Anglais	Français
OPC UA Client	Client OPC UA
OPC UA Application	Application OPC UA
Communication Stack	Pile de Communication
OPC UA Server	Serveur OPC UA

Figure 13 – Services SecureChannel et Session

Une fois qu'un *Client* a établi une *Session*, il peut souhaiter accéder à cette *Session* à partir d'un *Canal Sécurisé* différent. Le *Client* peut le faire en validant le nouveau *Canal Sécurisé* avec le *Service ActivateSession* décrit en 5.6.3.

Si un *Serveur* agit comme *Client* auprès d'autres *Serveurs*, ce qui est communément appelé chaînage de *Serveurs*, le *Serveur* doit être à même de maintenir le niveau de sécurité utilisateur. Cela signifie qu'il convient que l'identité de l'utilisateur soit transmise au serveur sous-jacent ou qu'elle soit mise en correspondance avec une identité d'utilisateur appropriée dans le serveur sous-jacent. Il est inacceptable d'ignorer la sécurité de niveau utilisateur. Elle est indispensable pour s'assurer que la sécurité est maintenue et qu'un utilisateur n'obtiendra pas d'informations auxquelles il convient qu'il n'ait pas accès. Autant que possible, il convient qu'un *Serveur* se substitue au *Client* d'origine en transmettant l'identité d'utilisateur du *Client* d'origine au *Serveur* sous-jacent lorsqu'il appelle le *Service ActivateSession*. Si la substitution d'identité n'est pas une option, le *Serveur* doit faire correspondre l'identité d'utilisateur du *Client* avec une nouvelle identité d'utilisateur que le *Serveur* sous-jacent reconnaît effectivement.

5.5.2 Ouvrir un canal sécurisé (OpenSecureChannel)

5.5.2.1 Description

Ce *Service* est utilisé pour ouvrir ou renouveler un *Canal Sécurisé* qui peut être utilisé pour assurer la *Confidentiality* et l'*Integrity* pour l'échange de *Messages* pendant cette *Session*. Ce *Service* exige de la *Pile de Communication* qu'elle applique les divers algorithmes de sécurité aux *Messages* à mesure qu'ils sont envoyés et reçus. Des mises en œuvre spécifiques de ce *Service* pour différentes *Piles de Communication* (*Communication Stacks*) sont décrites dans l'IEC 62541-6.

Chaque *Canal Sécurisé* a un identificateur globalement unique et est valide pour une combinaison spécifique d'instances d'applications *Client* et *Serveur*. Chaque canal contient un ou plusieurs *SecurityTokens* (jetons de sécurité) qui identifient un jeu de clés cryptographiques qui sont utilisées pour chiffrer et authentifier des *Messages*. Les

SecurityTokens ont aussi des identificateurs globalement uniques qui sont joints à chaque *Message* sécurisé avec le jeton. Cela permet à un récipiendaire autorisé de savoir comment déchiffrer et vérifier les *Messages*.

Les *SecurityTokens* ont une durée de vie finie négociée avec ce *Service*. Cependant, les différences entre les horloges système de plusieurs machines et les temps de latence réseau impliquent que des *Messages* valides peuvent arriver après l'expiration du jeton. Pour éviter que des *Messages* valides soient rejetés, il convient que les applications exécutent les actions suivantes:

- a) il convient que les *Clients* demandent un nouveau *SecurityToken* après 75 % de leur durée de vie. Il convient que cela garantisse que les *Clients* reçoivent le nouveau *SecurityToken* avant que l'ancien n'expire effectivement.
- b) les *Serveurs* doivent utiliser le *SecurityToken* existant pour sécuriser les *Messages* sortants jusqu'à ce que le *SecurityToken* expire ou que le *Serveur* reçoive un *Message* sécurisé avec un nouveau *SecurityToken*. Il convient que cela garantisse que les *Clients* ne rejettent pas des *Messages* sécurisés reçus avec le nouveau *SecurityToken* avant que le *Client* ne reçoive le nouveau *SecurityToken*.
- c) il convient que les *Clients* acceptent des *Messages* sécurisés par un *SecurityToken* expiré pendant une durée pouvant atteindre 25 % de la durée de vie du jeton. Il convient que cela garantisse que les *Messages* envoyés par le *Serveur* avant l'expiration du jeton ne soient pas rejetés en raison des retards du réseau.

Chaque *Canal Sécurisé* existe jusqu'à ce qu'il soit explicitement fermé ou jusqu'à ce que le dernier jeton ait expiré et que la période de chevauchement ait expiré. Il convient qu'une application *Serveur* limite le nombre de *Canaux Sécurisés*. Afin d'assurer une protection contre des *Clients* mal intentionnés et des attaques de déni de service, le *Serveur* doit fermer le plus ancien *Canal Sécurisé* qui n'a pas de *Session* assignée avant d'atteindre le nombre maximal de *Canaux Sécurisés* pris en charge.

Les *Messages* de demande d'*OpenSecureChannel* et de réponse doivent être signés avec le *Certificat* de l'expéditeur. Ces *Messages* doivent toujours être chiffrés. Si la couche transport ne fournit pas de chiffrement, ces *Messages* doivent être chiffrés avec le *Certificat* du récipiendaire. Ces exigences pour *OpenSecureChannel* ne s'appliquent que si le *securityPolicyUri* n'est pas None.

Les *Certificats* utilisés dans le service *OpenSecureChannel* doivent être les *Certificats d'Instance d'Application (Application Instance Certificates)*. Les *Clients* et *Serveurs* doivent vérifier que les mêmes *Certificats* ont été utilisés dans les services *CreateSession* et *ActivateSession*. Les *Certificats* ne sont pas fournis et ne doivent pas être vérifiés si le *securityPolicyUri* est None.

Si le *securityPolicyUri* n'est pas None, un *Client* doit vérifier que le *HostName* spécifié dans le *Certificat* du *Serveur* est le même que le *HostName* contenu dans l'*endpointUrl*. En cas de différence, le *Client* doit alors signaler la différence et peut choisir de ne pas ouvrir le *Canal Sécurisé*. Les *Serveurs* doivent ajouter tous les *HostNames* possibles tels que *MyHost* et *MyHost.local* dans le *Certificat* du *Serveur*. Ceci comprend les adresses IP de l'hôte ou l'*HostName* présenté par un router NAT utilisé pour établir la connexion avec le *Serveur*.

Il convient que les *Clients* soient prêts à remplacer le *HostName* retourné dans l'*EndPointDescription* par le *HostName* ou les adresses IP qu'ils utilisent pour appeler les *GetEndpoints*.

5.5.2.2 Paramètres

Le Tableau 7 définit les paramètres pour le *Service*.

Contrairement aux autres *Services*, les paramètres applicables à ce *Service* ne fournissent qu'une définition abstraite. La représentation concrète sur le réseau dépend des mises en correspondance définies dans l'IEC 62541-6.

Tableau 7 – Paramètres du service OpenSecureChannel

Nom	Type	Description
Request		
requestHeader	RequestHeader	Paramètres communs aux demandes. L' <i>authenticationToken</i> (jeton d'authentification) est toujours omis. Le type <i>RequestHeader</i> est défini en 7.26.
clientCertificate	ApplicationInstanceCertificate	Un <i>Certificat</i> qui identifie le <i>Client</i> . La demande d' <i>OpenSecureChannel</i> doit être signée avec ce <i>Certificat</i> . Le type <i>ApplicationInstanceCertificate</i> est défini en 7.2. Si le <i>securityPolicyUri</i> est None, le <i>Serveur</i> doit ignorer l' <i>ApplicationInstanceCertificate</i> .
requestType	Enum SecurityToken RequestType	Le type de demande de <i>SecurityToken</i> : Une énumération qui doit être l'une des suivantes: ISSUE_0 crée un nouveau <i>SecurityToken</i> pour un nouveau <i>Canal Sécurisé</i> . RENEW_1 crée un nouveau <i>SecurityToken</i> pour un <i>Canal Sécurisé</i> existant.
secureChannelId	ByteString	L'identificateur du <i>Canal Sécurisé</i> auquel il convient que le nouveau jeton appartienne. Ce paramètre doit être null lors de la création d'un nouveau <i>Canal Sécurisé</i> .
securityMode	Enum MessageSecurityMode	Le type de sécurité à appliquer aux messages. Le type <i>MessageSecurityMode</i> est défini en 7.14. Il est possible qu'il faille créer un <i>Canal Sécurisé</i> même si le <i>securityMode</i> est NONE. Le comportement exact dépend de la mise en correspondance utilisée et est décrit dans l'IEC 62541-6.
securityPolicyUri	String	L'URI pour la <i>SecurityPolicy</i> à utiliser lors de la sécurisation des messages envoyés sur le <i>Canal Sécurisé</i> . Le jeu d'URI connus et les <i>SecurityPolitiques</i> qui leur sont associées sont définis dans l'IEC 62541-7.
clientNonce	ByteString	Un nombre aléatoire qui ne doit être utilisé par aucune autre demande. Un nouveau <i>clientNonce</i> doit être généré chaque fois qu'un <i>Canal Sécurisé</i> est renouvelé. Ce paramètre doit avoir une longueur égale à la taille de clé utilisée pour l'algorithme de chiffrement symétrique qui est identifié par le <i>securityPolicyUri</i> .
requestedLifetime	Duration	La durée de vie demandée, en millisecondes, pour le nouveau <i>SecurityToken</i> . Il spécifie quand le <i>Client</i> pense renouveler le <i>Canal Sécurisé</i> en appelant de nouveau le <i>Service OpenSecureChannel</i> . Si un <i>Canal Sécurisé</i> n'est pas renouvelé, tous les <i>Messages</i> envoyés en utilisant les <i>SecurityTokens</i> courants doivent être rejetés par le récipiendaire.
Response		
responseHeader	ResponseHeader	Paramètres communs aux réponses (voir 7.27 pour la définition du type <i>ResponseHeader</i>).
securityToken	ChannelSecurityToken	Décrit le nouveau <i>SecurityToken</i> émis par le <i>Serveur</i> . Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.
channelId	ByteString	Un identificateur unique pour le <i>Canal Sécurisé</i> . Il s'agit de l'identificateur qui doit être fourni chaque fois que le <i>Canal Sécurisé</i> est renouvelé.
tokenId	ByteString	Un identificateur unique pour un seul <i>SecurityToken</i> dans le canal. Il s'agit de l'identificateur qui doit être transmis avec chaque <i>Message</i> sécurisé avec le <i>SecurityToken</i> .
createdAt	UtcTime	Le moment où le <i>SecurityToken</i> a été créé.
revisedLifetime	Duration	La durée de vie du <i>SecurityToken</i> en millisecondes. L'heure d'expiration TUC pour le jeton peut être calculée en ajoutant la durée de vie au temps <i>createdAt</i> .
serverNonce	ByteString	Un nombre aléatoire qui ne doit être utilisé par aucune autre demande. Un nouveau <i>serverNonce</i> doit être généré chaque fois qu'un <i>Canal Sécurisé</i> est renouvelé. Ce paramètre doit avoir une longueur égale à la taille de clé utilisée pour l'algorithme de chiffrement symétrique qui est identifié par le <i>securityPolicyUri</i> .

5.5.2.3 Résultats des services

Le Tableau 8 définit les résultats de *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 8 – Codes des résultats de services OpenSecureChannel

Id symbolique	Description
Bad_SecurityChecksFailed	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_CertificateTimeInvalid	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_CertificateIssuerTimeInvalid	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_CertificateHostNameInvalid	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_CertificateUriInvalid	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_CertificateUseNotAllowed	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_CertificateIssuerUseNotAllowed	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_CertificateUntrusted	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_CertificateRevocationUnknown	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_CertificateIssuerRevocationUnknown	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_CertificateRevoked	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_CertificateIssuerRevoked	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_RequestTypeInvalid	Le type de la demande de jeton de sécurité n'est pas valide.
Bad_SecurityModeRejected	Le mode de sécurité ne satisfait pas aux exigences établies par le <i>Serveur</i> .
Bad_SecurityPolicyRejected	La politique de sécurité ne satisfait pas aux exigences établies par le <i>Serveur</i> .
Bad_SecureChannelIdInvalid	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_NonceInvalid	Voir le Tableau 165 pour la description de ce code de résultat. Un serveur doit vérifier la longueur minimale du nonce du client et renvoyer ce statut si la longueur est inférieure à 32 octets. Une vérification des nonces dupliqués ne peut être faite que dans les appels OpenSecureChannel avec le type de demande RENEW_1.

5.5.3 Fermer un canal sécurisé (CloseSecureChannel)

5.5.3.1 Description

Ce *Service* est utilisé pour mettre fin à un *Canal Sécurisé*.

Les *Messages* de demande doivent être signés avec la clé appropriée associée au jeton courant pour le *Canal Sécurisé*.

5.5.3.2 Paramètres

Le Tableau 9 définit les paramètres pour le *Service*.

Tableau 9 – Paramètres du service CloseSecureChannel

Nom	Type	Description
Request		
requestHeader	RequestHeader	Paramètres communs aux demandes. Le <i>sessionId</i> (identificateur de session) est toujours omis. Le type <i>RequestHeader</i> est défini en 7.26.
secureChannelId	ByteString	L'identificateur pour le <i>Canal Sécurisé</i> à fermer.
Response		
responseHeader	ResponseHeader	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).

5.5.3.3 Résultats des services

Le Tableau 10 définit les résultats des *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 10 – Codes des résultats de services CloseSecureChannel

Id symbolique	Description
Bad_SecureChannelIdInvalid	Voir le Tableau 165 pour la description de ce code de résultat.

5.6 Jeu de services de session (Session Service Set)

5.6.1 Vue d'ensemble

Ce *Service Set* définit des *Services* pour l'établissement de connexions de couche application dans le contexte d'une *Session*.

5.6.2 Créer une session (CreateSession)

5.6.2.1 Description

Ce *Service* est utilisé par un *Client* OPC UA pour créer une *Session* et le *Serveur* retourne deux valeurs qui identifient de façon unique cette *Session*. La première valeur est le *sessionId* qui est utilisé pour identifier cette *Session* dans les journaux d'audit et dans l'espace d'adresses du *Serveur*. La seconde est l'*authenticationToken* qui est utilisé pour associer une demande entrante à une *Session*.

Avant d'appeler ce *Service*, le *Client* doit créer un *Canal Sécurisé* avec le *Service OpenSecureChannel* pour garantir l'*Integrity* de tous les *Messages* échangés au cours de cette *Session*. Ce *Canal Sécurisé* a un identificateur unique, que le *Serveur* doit associer à l'*authenticationToken*. Le *Serveur* peut accepter des demandes avec l'*authenticationToken* seulement si elles sont associées au *Canal Sécurisé* qui a été utilisé pour créer cette *Session*. Le *Client* peut associer un nouveau *Canal Sécurisé* à la *Session* en appelant la méthode *ActivateSession*.

Le *Canal Sécurisé* est toujours géré par la *Pile de Communication*, ce qui signifie qu'il doit fournir les API que le *Serveur* peut utiliser pour trouver des informations du *Canal Sécurisé* utilisé pour n'importe quelle demande donnée. La *Pile de Communication* doit, au minimum, fournir la *SecurityPolicy* et le *SecurityMode* utilisés par le *Canal Sécurisé*. Elle doit aussi fournir un *Canal Sécurisé Id* (identificateur de canal sécurisé) qui identifie de façon unique le *Canal Sécurisé* ou le *Client Certificate* (certificat de client) utilisé pour établir le *Canal Sécurisé*. Le *Serveur* utilise l'un de ceux-ci pour identifier le *Canal Sécurisé* utilisé pour envoyer une demande. Le paragraphe 7.29 décrit comment créer l'*authenticationToken* pour différents types de *Pile de Communication*.

En fonction de la *SecurityPolicy* et du *SecurityMode* du *Canal Sécurisé*, l'échange des *ApplicationInstanceCertificates* (Certificats d'instances d'application) et des *Nonces* (number used once, nombre utilisé une seule fois) peut être facultatif et les signatures peuvent être vides. Voir l'IEC 62541-7 pour la définition des *SecurityPolicies* et la gestion de ces paramètres.

Le *Serveur* retourne ses *EndpointDescriptions* dans la réponse. Les *Clients* utilisent cette information pour déterminer si, oui ou non, la liste des *EndpointDescriptions* retournées par le *Discovery Endpoint* correspond aux *Endpoints* que le *Serveur* possède. S'il y a une différence, le *Client* doit alors fermer cette *Session* et rapporter l'erreur. Le *Serveur* retourne toutes les *EndpointDescriptions* pour le *serverUri* spécifié par le *Client* dans la demande. Le *Client* vérifie uniquement les *EndpointDescriptions* ayant un *transportProfileUri* (URI de profil de transport) qui correspond au *profileUri* (URI de profil) spécifié dans la demande de *GetEndpoints* initiale. Un *Client* peut sauter cette vérification si les *EndpointDescriptions* ont été fournies par une source de confiance telle que l'*Administrateur*.

Une *Session* créée avec ce *Service* ne doit pas être utilisée jusqu'à ce que le *Client* appelle le *Service ActivateSession*, fournisse ses *SoftwareCertificates* et prouve la possession de son *Certificat d'Instance d'Application* et de tout jeton d'identité utilisateur qu'il fournit.

Le paramètre *SoftwareCertificates* dans la réponse du *Serveur* est dévalorisé de façon à réduire la taille du message pour les Applications OPC UA aux ressources limitées. Les *SoftwareCertificates* sont fournis dans l'*Espace d'Adresses* du *Serveur*, comme défini dans l'IEC 62541-5. Un *SoftwareCertificate* identifie les fonctions du *Serveur* et contient également la liste des *Profils* OPC UA pris en charge par le *Serveur*. Les *Profils* OPC UA sont définis dans l'IEC 62541-7.

Les *Certificats* supplémentaires émis par d'autres organisations peuvent être inclus pour identifier des fonctions supplémentaires du *Serveur*. Les exemples de ces *Profils* incluent la prise en charge pour des modèles spécifiques d'informations et la prise en charge pour l'accès aux types spécifiques d'appareils.

Lorsqu'une *Session* est créée, le *Serveur* ajoute une entrée pour le *Client* dans sa *Variable SessionDiagnosticsArray*. Voir l'IEC 62541-5 pour la description de cette *Variable*.

Les *Sessions* sont créées de façon à être indépendantes de la connexion de communications sous-jacente. Par conséquent, une défaillance de la connexion de communications n'affecte pas immédiatement la *Session*. Le mécanisme exact de reprise sur une erreur de la connexion de communications sous-jacente dépend de la mise en correspondance de *SecureChannel* décrite dans l'IEC 62541-6.

Les *Sessions* sont automatiquement fermées par le *Serveur* si le *Client* oublie d'émettre une demande de *Service* sur la *Session* dans la période d'attente de temporisation négociée avec le *Serveur* dans la réponse à la demande de *Service CreateSession*. Le *Serveur* est ainsi protégé des défaillances du *Client* et des situations dans lesquelles la connexion sous-jacente défectueuse ne peut pas être rétablie. Les *Clients* doivent être prêts à transmettre des demandes dans le temps prévu afin d'empêcher la fermeture automatique de cette *Session*. Les *Clients* peuvent explicitement mettre fin à des *Sessions* au moyen du *Service CloseSession* (*Service Fermer la Session*).

Lorsqu'il est mis fin à une *Session*, toutes les demandes en cours sur cette *Session* sont abandonnées et des *StatusCodes Bad_SessionClosed* sont retournés au *Client*. De plus, le *Serveur* supprime de sa *Variable SessionDiagnosticsArray* (*Matrice de diagnostic de session*) l'entrée pour le *Client* et en notifie les autres éventuels *Clients* qui s'étaient abonnés à cette entrée.

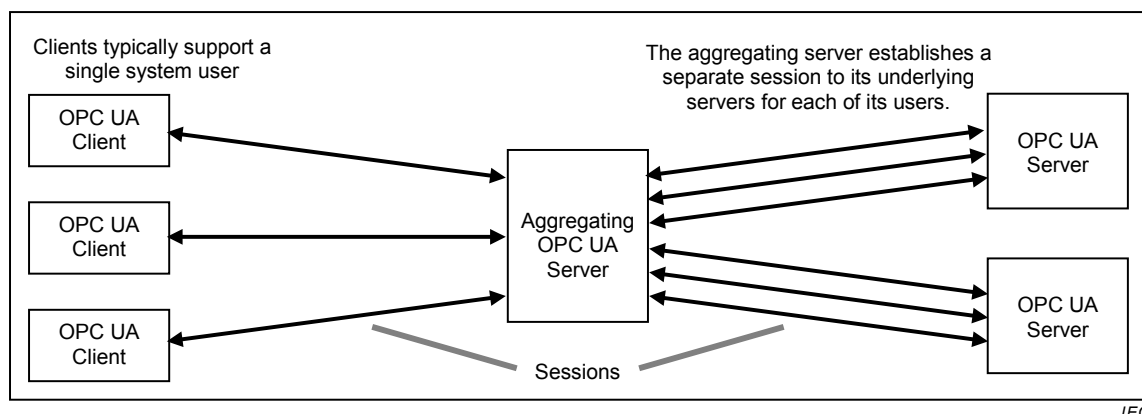
Si un *Client* invoque le *Service CloseSession*, tous les *Subscriptions* (*Abonnements*) associés à la *Session* sont également supprimés si le *fanion deleteSubscriptions* (*Effacer les abonnements*) est mis à TRUE. Si un *Serveur* met fin à une *Session* pour une autre raison quelconque, les *Subscriptions* associés à cette *Session* ne sont pas supprimés. Chaque *Subscription* a sa propre durée de vie pour se protéger des pertes de données en cas de fermeture d'une *Session*. Dans ces cas, le *Subscription* peut être réaffecté à un autre *Client* avant que sa durée de vie n'expire.

Certains *Serveurs*, tels que les *Serveurs* combinés, agissent aussi comme *Clients* pour d'autres *Serveurs*. Les *Serveurs* prennent typiquement en charge plus d'un utilisateur système, agissant comme leurs agents auprès des *Serveurs* qu'ils représentent. La sécurité pour ces *Serveurs* est prise en charge à deux niveaux.

Premièrement, chaque demande de *Service* OPC UA contient un paramètre chaîne qui est utilisé pour contenir un id d'enregistrement d'audit. Un *Client* ou tout *Serveur* opérant comme *Client*, tel qu'un *Serveur* combiné par exemple, peut créer une entrée locale de journal d'audits pour une demande qu'il présente. Ce paramètre permet au *Client* de transmettre l'identificateur pour cette entrée avec la demande. Si le *Serveur* maintient aussi un journal d'audits, il peut alors inclure cet id dans l'entrée de journal d'audits qu'il écrit. Lorsque le journal est examiné et l'entrée trouvée, l'examineur peut la relier directement à l'entrée de journal d'audits créée par le *Client*. Cette fonction permet la traçabilité à travers les journaux d'audits dans un système. Voir l'IEC TR 62541-2 pour des informations complémentaires relatives aux audits. Un *Serveur* qui maintient un journal d'audits doit fournir les informations

dans les entrées du journal d'audits via des *Messages* d'événements définis dans la présente Norme. Le *Serveur* peut choisir de fournir uniquement les informations d'*Audit* via des *Messages* d'événements. L'*EventType Audit* est défini dans l'IEC 62541-3.

Deuxièmement, ces *Serveurs* combinés peuvent ouvrir des *Sessions* indépendantes sur les *Serveurs* sous-jacents pour chaque *Client* qui, à partir d'eux, accède aux données. La Figure 14 illustre ce concept.



Anglais	Français
Clients typically support a single system user	Les clients prennent typiquement en charge un seul utilisateur système
OPC UA Client	Client OPC UA
Aggregating OPC UA Server	Serveur combiné OPC UA
The aggregating server establishes a separate session to its underlying servers for each of its users	Le serveur combiné établit une session séparée dans ses serveurs sous-jacents pour chacun de ses utilisateurs
OPC UA Server	Serveur OPC UA

Figure 14 – Multiplexage d'utilisateurs sur une Session

5.6.2.2 Paramètres

Le Tableau 11 définit les paramètres pour le *Service*.

Tableau 11 – Paramètres du service CreateSession

Nom	Type	Description
Request		
requestHeader	RequestHeader	Paramètres communs aux demandes. L' <i>authenticationToken</i> (jeton d'authentification) est toujours omis. Le type <i>RequestHeader</i> est défini en 7.26.
clientDescription	Application Description	Informations qui décrivent l'application <i>Client</i> . Le type <i>ApplicationDescription</i> est défini en 7.1.
serverUri	String	Cette valeur est seulement spécifiée si l' <i>EndpointDescription</i> a un <i>gatewayServerUri</i> . Cette valeur est l' <i>applicationUri</i> issue de l' <i>EndpointDescription</i> qui est l' <i>applicationUri</i> pour le <i>Serveur</i> sous-jacent. Le type <i>EndpointDescription</i> est défini en 7.9.
endpointUrl	String	L'adresse réseau que le <i>Client</i> a utilisée pour accéder au <i>Session Endpoint</i> . Il convient que la partie <i>HostName</i> de l'URL soit l'un des <i>HostNames</i> pour l'application qui sont spécifiés dans l' <i>ApplicationInstanceCertificate</i> du <i>Serveur</i> (voir 7.2). Le <i>Serveur</i> doit soulever un événement <i>AuditUrlMismatchEventType</i> (un événement de type événement de discordance d'URL d'audit) si l'URL ne correspond pas aux <i>HostNames</i> du <i>Serveur</i> . Le type d'événement <i>AuditUrlMismatchEventType</i> est défini dans l'IEC 62541-5. Le <i>Serveur</i> utilise cette information pour des diagnostics et pour déterminer le jeu d' <i>EndpointDescriptions</i> à retourner dans la réponse.
sessionName	String	Chaîne interprétable par l'utilisateur qui identifie une <i>Session</i> . Le <i>Serveur</i> rend ce nom et le <i>sessionId</i> visibles dans son <i>Espace d'Adresses</i> à des fins de diagnostic. Il convient que le <i>Client</i> fournisse un nom qui est unique pour l'instance du <i>Client</i> . Si ce paramètre n'est pas spécifié, le <i>Serveur</i> doit attribuer une valeur.
clientNonce	ByteString	Un nombre aléatoire qu'il convient de ne jamais utiliser dans aucune autre demande. Ce nombre doit avoir une longueur minimale de 32 octets. Les profils peuvent augmenter la longueur requise. Le <i>Serveur</i> doit utiliser cette valeur pour prouver la possession de son <i>Certificat d'instance d'application</i> dans la réponse.
clientCertificate	ApplicationInstance Certificate	Le <i>Certificat d'instance d'application</i> émis au <i>Client</i> . Le type <i>ApplicationInstanceCertificate</i> est défini en 7.2. Si le <i>securityPolicyUri</i> est None, le <i>Serveur</i> doit ignorer l' <i>ApplicationInstanceCertificate</i> .
Requested SessionTimeout	Duration	Nombre maximal demandé de millisecondes pendant lesquelles il convient qu'une <i>Session</i> reste ouverte sans activité. Si le <i>Client</i> oublie d'émettre une demande de <i>Service</i> dans la limite de cet intervalle, le <i>Serveur</i> doit automatiquement mettre fin à la <i>Session</i> du <i>Client</i> .
maxResponse MessageSize	UInt32	La taille maximale, en octets, pour le corps de tout message de réponse. Il convient que le <i>Serveur</i> retourne un défaut de service <i>Bad_ResponseTooLarge</i> si un message de réponse dépasse cette limite. La valeur zéro indique que ce paramètre n'est pas utilisé. Le paragraphe 5.3 fournit plus d'informations sur l'utilisation de ce paramètre.
Response		
responseHeader	ResponseHeader	Paramètres communs aux réponses (voir 7.27 pour le type <i>ResponseHeader</i>).
sessionId	NodeId	Un <i>NodeId</i> unique affecté par le <i>Serveur</i> à une <i>Session</i> . Cet identificateur est utilisé pour accéder aux informations de diagnostic pour une <i>Session</i> dans l' <i>Espace d'Adresses</i> du <i>Serveur</i> . Il est également utilisé dans les journaux d'audits et les éventuels événements qui rapportent des informations relatives à cette <i>Session</i> . Les informations de diagnostic d'une <i>Session</i> sont décrites dans l'IEC 62541-5. Les journaux d'audits et leurs événements connexes sont décrits en 6.2
authentication Token	Session AuthenticationToken	Un identificateur unique affecté par le <i>Serveur</i> à la <i>Session</i> . Cet identificateur doit être transmis dans le <i>RequestHeader</i> de chaque demande et il est utilisé avec le <i>SecureChannelId</i> pour déterminer si, oui ou non, un <i>Client</i> a accès à la <i>Session</i> . Cet identificateur ne doit pas être réutilisé d'une manière induisant le risque pour le <i>Client</i> ou le <i>Serveur</i> de le confondre avec une <i>Session</i> précédente ou existante. Le type <i>SessionAuthenticationToken</i> est décrit en 7.29.
revisedSession Timeout	Duration	Nombre maximal effectif de millisecondes pendant lesquelles une <i>Session</i> doit rester ouverte sans activité. Il convient que le <i>Serveur</i> tente d'honorer la demande du <i>Client</i> pour ce paramètre, mais il peut négocier cette valeur à la hausse ou à la baisse pour satisfaire à ses propres contraintes.
serverNonce	ByteString	Un nombre aléatoire qu'il convient de ne jamais utiliser dans aucune autre demande. Ce nombre doit avoir une longueur minimale de 32 octets. Le <i>Client</i> doit utiliser cette valeur pour prouver la possession de son <i>Certificat d'instance d'application</i> dans la demande <i>ActivateSession</i> . Cette valeur peut aussi être utilisée pour prouver la possession du <i>userIdentityToken</i> qu'il a spécifié dans la demande <i>ActivateSession</i> .
serverCertificate	ApplicationInstance Certificate	Le <i>Certificat d'instance d'application</i> émis au <i>Serveur</i> . Un <i>Serveur</i> doit prouver la possession en utilisant la clé privée pour signer le <i>Nonce</i> fourni par le <i>Client</i> dans la demande. Le <i>Client</i> doit vérifier que ce <i>Certificat</i> est le même que celui qu'il a utilisé pour créer le <i>Canal Sécurisé</i> . Le type <i>ApplicationInstanceCertificate</i> est défini en 7.2. Si le <i>securityPolicyUri</i> est NONE et qu'aucune <i>UserTokenPolicy</i> ne nécessite de

Nom	Type	Description
		chiffrement, le <i>Client</i> doit ignorer l' <i>ApplicationInstanceCertificate</i> .
serverEndpoints []	Endpoint Description	Liste des <i>Endpoints</i> que le serveur prend en charge. Le <i>Serveur</i> doit retourner un jeu d' <i>EndpointDescriptions</i> disponibles pour le <i>serverUri</i> spécifié dans la demande. Le type <i>EndpointDescription</i> est défini en 7.9. Le <i>Client</i> doit vérifier cette liste avec la liste issue d'un <i>Discovery Endpoint</i> s'il a utilisé un <i>Discovery Endpoint</i> pour rechercher des <i>EndpointDescriptions</i> . Il est recommandé que les <i>Serveurs</i> ne comprennent que l' <i>endpointUrl</i> , le <i>securityMode</i> , le <i>securityPolicyUri</i> , les <i>userIdentityTokens</i> , le <i>transportProfileUri</i> et le <i>securityLevel</i> avec tous les autres paramètres définis sur null. Seuls les paramètres recommandés doivent être vérifiés par le client.
serverSoftwareCertificates []	SignedSoftware Certificate	Ce paramètre est dévalorisé et la matrice doit être vide. Les <i>SoftwareCertificates</i> sont fournis dans l' <i>Espace d'Adresses</i> du <i>Serveur</i> comme défini dans l'IEC 62541-5.
serverSignature	SignatureData	Il s'agit d'une signature générée par la clé privée associée au <i>serverCertificate</i> . Ce paramètre est calculé en accolant le <i>clientNonce</i> au <i>clientCertificate</i> et en signant la séquence d'octets obtenue. Le <i>SignatureAlgorithm</i> (algorithme de signature) doit être l' <i>AsymmetricSignatureAlgorithm</i> (de signature asymétrique) spécifié dans la <i>SecurityPolicy</i> pour l' <i>Endpoint</i> . Le type <i>SignatureData</i> est défini en 7.30.
maxRequest MessageSize	UInt32	La taille maximale, en octets, pour le corps de tout message de demande. Il convient que la <i>Pile de Communication</i> du <i>Client</i> retourne une erreur <i>Bad_RequestTooLarge</i> à l'application si un message de demande dépasse cette limite. La valeur zéro indique que ce paramètre n'est pas utilisé. Le paragraphe 5.3 fournit plus d'informations sur l'utilisation de ce paramètre.

5.6.2.3 Résultats des services

Le Tableau 12 définit les résultats de *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 12 – Codes des résultats de services CreateSession

Id symbolique	Description
Bad_SecureChannelIdInvalid	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_NonceInvalid	Voir le Tableau 165 pour la description de ce code de résultat. Un serveur doit vérifier la longueur minimale du nonce du client et renvoyer ce statut si la longueur est inférieure à 32 octets. Une vérification des nonces dupliqués est facultative et nécessite un accès au nonce utilisé pour créer le canal sécurisé.
Bad_SecurityChecksFailed	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_CertificateTimeInvalid	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_CertificateIssuerTimeInvalid	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_CertificateHostNameInvalid	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_CertificateUriInvalid	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_CertificateUseNotAllowed	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_CertificateIssuerUseNotAllowed	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_CertificateUntrusted	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_CertificateRevocationUnknown	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_CertificateIssuerRevocationUnknown	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_CertificateRevoked	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_CertificateIssuerRevoked	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_TooManySessions	Le serveur a atteint le nombre maximal de sessions.
Bad_ServerUriInvalid	Voir le Tableau 165 pour la description de ce code de résultat.

5.6.3 Activer une session (ActivateSession)

5.6.3.1 Description

Ce *Service* est utilisé par le *Client* pour présenter ses *SoftwareCertificates* au *Serveur* pour validation et pour spécifier l'identité de l'utilisateur associé à la *Session*. Cette demande de *Service* doit être émise par le *Client* avant qu'il n'émette une quelconque autre demande de *Service* après *CreateSession*. Si tel n'est pas le cas, le *Serveur* doit fermer la *Session*.

Chaque fois que le *Client* appelle ce *Service*, le *Client* doit prouver qu'il s'agit de la même application qui a appelé le *Service CreateSession*. Pour ce faire, le *Client* crée une signature

avec la clé privée associée au *clientCertificate* spécifié dans la demande *CreateSession*. Cette signature est créée en accolant le dernier *serverNonce* fourni par le *Serveur* au *serverCertificate* et en calculant la signature de la séquence d'octets obtenue.

Une fois utilisé, un *serverNonce* ne peut plus être réutilisé. C'est pourquoi le *Serveur* retourne un nouveau *serverNonce* chaque fois que le *Service ActivateSession* est appelé.

Lorsque le *Service ActivateSession* est appelé la première fois, le *Serveur* doit rejeter la demande si le *Canal Sécurisé* n'est pas le même que celui qui est associé à la demande *CreateSession*. Les appels consécutifs à *ActivateSession* peuvent être associés à des *Canaux Sécurisés* différents. Si tel est le cas, le *Serveur* doit vérifier que le *Certificat* utilisé par le *Client* pour créer le nouveau *Canal Sécurisé* est le même que le *Certificat* utilisé pour créer le *Canal Sécurisé* d'origine. En outre, le *Serveur* doit vérifier que le *Client* a fourni un *UserIdentityToken* identique au jeton réellement associé à la *Session*. Une fois que le *Serveur* a accepté le nouveau *Canal Sécurisé*, il doit rejeter les demandes envoyées par l'ancien *Canal Sécurisé*.

Le *Service ActivateSession* est utilisé pour associer une identité d'utilisateur à une *Session*. Lorsqu'un *Client* fournit une identité d'utilisateur, il doit fournir la preuve qu'il est habilité à utiliser l'identité d'utilisateur en question. Le mécanisme exact utilisé pour fournir cette preuve dépend du type d'*UserIdentityToken*. Si le jeton est un *UserNameIdentityToken*, la preuve est le *password* (Mot de passe) qui a été inclus dans le jeton. Si le jeton est un *X509IdentityToken*, la preuve est une signature générée avec la clé privée associée au *Certificate*. Les données à signer sont créées en accolant le dernier *serverNonce* au *serverCertificate* spécifié dans la réponse *CreateSession*. Si un jeton inclut un secret, il convient de le déchiffrer à l'aide de la clé publique contenue dans le *serverCertificate*.

Les *Clients* peuvent modifier l'identité d'un utilisateur associé à une *Session* en appelant le *Service ActivateSession*. Le *Serveur* valide les signatures fournies avec la demande puis valide la nouvelle identité d'utilisateur. En l'absence d'erreurs, le *Serveur* remplace l'identité d'utilisateur pour la *Session*. La modification de l'identité utilisateur pour une *Session* peut induire des discontinuités des *Subscriptions* actifs, car le *Serveur* peut devoir casser des connexions à un système sous-jacent et les rétablir en utilisant les nouveaux authentifiants.

Lorsqu'un *Client* fournit une liste d'identificateurs de paramètres de lieu dans la demande, il est requis de chaque identificateur de paramètre de lieu qu'il contienne la composante langue. Il peut facultativement contenir la composante <country/region> (pays/région). Lorsque le *Serveur* retourne un *LocalizedText* dans le contexte d'une *Session*, il peut aussi retourner les deux composantes langue et pays/région ou juste la composante langue comme la valeur par défaut de son identificateur de paramètre de lieu.

Lorsqu'un *Serveur* retourne une chaîne au *Client*, il détermine d'abord s'il existe des traductions pour celle-ci. S'il en existe, le *Serveur* retourne alors la chaîne dont l'identificateur de paramètre de lieu correspond exactement à l'identificateur de paramètre de lieu ayant la priorité la plus haute dans la liste fournie par le *Client*.

S'il n'existe pas de correspondances exactes, le *Serveur* ignore alors la composante <country/region> de l'identificateur de paramètre de lieu et retourne la chaîne dont la composante <language> correspond à la composante <language> de l'identificateur de paramètre de lieu ayant la priorité la plus haute dans la liste fournie par le *Client*.

S'il n'y a pas de correspondances, le *Serveur* retourne alors la chaîne qu'il a avec l'identificateur de paramètre de lieu.

Un *Gateway Server* est censé se substituer à l'utilisateur donné par le *Client* lorsqu'il se connecte au *Serveur* sous-jacent. Cela signifie qu'il doit recalculer les signatures sur l'*UserIdentityToken* en utilisant le nonce fourni par le *Serveur* sous-jacent. Le *Gateway Server* aura à utiliser ses propres authentifiants d'utilisateur si l'*UserIdentityToken* fourni par le *Client* ne prend pas en charge la substitution d'identité.

5.6.3.2 Paramètres

Le Tableau 13 définit les paramètres pour le *Service*.

Tableau 13 – Paramètres du service *ActivateSession*

Nom	Type	Description
Request		
requestHeader	RequestHeader	Paramètres communs aux demandes. Le type <i>RequestHeader</i> est défini en 7.26.
clientSignature	SignatureData	Il s'agit d'une signature générée par la clé privée associée au <i>clientCertificate</i> . Le <i>SignatureAlgorithm</i> (algorithme de signature) doit être l' <i>AsymmetricSignatureAlgorithm</i> (de signature asymétrique) spécifié dans la <i>SecurityPolicy</i> pour l' <i>Endpoint</i> . Le type <i>SignatureData</i> est défini en 7.30.
clientSoftwareCertificates []	SignedSoftwareCertificate	Ce sont les <i>SoftwareCertificates</i> qui ont été émis à l'application <i>Client</i> . Le <i>productUri</i> contenu dans les <i>SoftwareCertificates</i> doit correspondre au <i>productUri</i> dans l' <i>ApplicationDescription</i> utilisée par le <i>Client</i> dans les demandes <i>CreateSession</i> . Il convient d'ignorer les <i>Certificats</i> sans <i>productUri</i> correspondant. Les <i>Serveurs</i> peuvent rejeter des connexions issues des <i>Clients</i> s'ils ne sont pas satisfaits par les <i>SoftwareCertificates</i> fournis par le <i>Client</i> . Il n'est nécessaire de spécifier ce paramètre que dans la première demande <i>ActivateSession</i> après <i>CreateSession</i> . Il doit toujours être omis si la <i>maxRequestMessageSize</i> retournée par le <i>Serveur</i> dans la réponse <i>CreateSession</i> est inférieure à un mégaoctet. Le type <i>SignedSoftwareCertificate</i> est défini en 7.31
localeIds []	LocaleId	Liste des identificateurs de paramètres de lieu dans l'ordre des priorités pour les chaînes localisées. Le premier <i>LocaleId</i> dans la liste a la priorité la plus haute. Si le <i>Serveur</i> retourne une chaîne localisée au <i>Client</i> , le <i>Serveur</i> doit retourner la traduction avec la plus haute priorité qu'il peut. S'il n'a de traduction pour aucun des paramètres de lieu identifiés dans cette liste, il doit retourner la valeur chaîne qu'il a et inclure l'identificateur de paramètre de lieu avec la chaîne. Voir l'IEC 62541-3 pour plus de détails sur les identificateurs de paramètres de lieu. Si le <i>Client</i> ne réussit pas à spécifier au moins un identificateur de paramètre de lieu, le <i>Serveur</i> doit utiliser n'importe lequel en sa possession. Il n'est nécessaire de spécifier ce paramètre que pendant le premier appel au <i>Service ActivateSession</i> au cours d'une seule <i>Session</i> d'application. S'il n'est pas spécifié, le <i>Serveur</i> doit continuer d'utiliser les <i>localeIds</i> courants pour la <i>Session</i> .
userIdentityToken	ExtensibleParameter UserIdentityToken	Les authentifiants de l'utilisateur associé à l'application <i>Client</i> . Le <i>Serveur</i> utilise ces authentifiants pour déterminer s'il convient ou non d'autoriser le <i>Client</i> à activer une <i>Session</i> et pour déterminer également les ressources auxquelles le <i>Client</i> a accès au cours de la <i>Session</i> . L' <i>UserIdentityToken</i> est un type de paramètre extensible défini en 7.35. L' <i>EndpointDescription</i> spécifie les <i>UserIdentityTokens</i> que le <i>Serveur</i> doit accepter. Les jetons d'utilisateur null ou vides doivent toujours être interprétés comme anonyme.
userTokenSignature	SignatureData	Si le <i>Client</i> a spécifié un jeton d'identité d'utilisateur qui prend en charge les signatures numériques, il doit créer une signature et la passer comme étant ce paramètre. Autrement le paramètre est omis. Le <i>SignatureAlgorithm</i> dépend du type de jeton d'identité. Le type <i>SignatureData</i> est défini en 7.30.
Response		
responseHeader	ResponseHeader	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).
serverNonce	ByteString	Un nombre aléatoire qu'il convient de ne jamais utiliser dans aucune autre demande. Ce nombre doit avoir une longueur minimale de 32 octets. Le <i>Client</i> doit utiliser cette valeur pour prouver la possession de son <i>Certificat d'Instance d'Application</i> lors de la prochaine demande d'appel à <i>ActivateSession</i> .
results []	StatusCode	Liste de résultats de validation pour les <i>SoftwareCertificates</i> (voir 7.33 pour la définition de <i>StatusCode</i>).
diagnosticInfos []	DiagnosticInfo	Liste d'informations de diagnostic associées à des erreurs de validation de <i>SoftwareCertificate</i> (voir 7.8 pour la définition de <i>DiagnosticInfo</i>). Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été rencontrée dans le traitement de la demande.

5.6.3.3 Résultats des services

Le Tableau 14 définit les résultats de *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 14 – Codes des résultats de services ActivateSession

Id Symbolique	Description
Bad_IdentityTokenInvalid	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_IdentityTokenRejected	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_UserAccessDenied	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_ApplicationSignatureInvalid	La signature fournie par l'application <i>Client</i> est absente ou non valide.
Bad_UserSignatureInvalid	La signature du jeton d'utilisateur est absente ou non valide.
Bad_NoValidCertificates	Le <i>Client</i> n'a pas fourni au moins un <i>Certificat de Logiciel</i> qui est valide et satisfait aux exigences relatives aux profils pour le <i>Serveur</i> .
Bad_IdentityChangeNotSupported	Le <i>Serveur</i> ne prend pas en charge la modification de l'identité d'utilisateur affectée à la session.

5.6.4 Fermer une session (CloseSession)

5.6.4.1 Description

Ce *Service* est utilisé pour mettre fin à une *Session*. Le *Serveur* entreprend les actions suivantes lorsqu'il reçoit une demande *CloseSession*:

- Il arrête d'accepter des demandes pour cette *Session*. Toutes les demandes ultérieures reçues pour la *Session* sont rejetées.
- Il retourne des réponses négatives avec le *StatusCode* *Bad_SessionClosed* à toutes les demandes qui sont actuellement en cours pour permettre le retour en temps utile de la réponse *CloseSession*. Les *Clients* sont vivement encouragés à attendre que toutes les demandes en cours se terminent avant de présenter une demande *CloseSession*.
- Il retire l'entrée pour le *Client* dans sa *Variable SessionDiagnosticsArray*.

5.6.4.2 Paramètres

Le Tableau 15 définit les paramètres pour le *Service*.

Tableau 15 – Paramètres du service CloseSession

Nom	Type	Description
Request		
requestHeader	RequestHeader	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).
deleteSubscriptions	Boolean	Si la valeur est TRUE, le <i>Serveur</i> supprime tous les Subscriptions (Abonnements) associés à la Session. Si la valeur est FALSE, le <i>Serveur</i> conserve les <i>Abonnements</i> associés à la Session jusqu'à ce qu'ils expirent sur la base de leur propre durée de vie.
Response		
responseHeader	ResponseHeader	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).

5.6.4.3 Résultats des services

Le Tableau 16 définit les résultats de *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 16 – Codes des résultats de services CloseSession

Id symbolique	Description
Bad_SessionIdInvalid	Voir le Tableau 165 pour la description de ce code de résultat.

5.6.5 Annuler (Cancel)

5.6.5.1 Description

Ce *Service* est utilisé pour annuler des demandes de Services en cours. Les demandes de services qui ont été annulées avec succès doivent répondre avec *Bad_RequestCancelledByClient*.

5.6.5.2 Paramètres

Le Tableau 17 définit les paramètres pour le *Service*.

Tableau 17 – Paramètres du service Cancel

Nom	Type	Description
Request		
requestHeader	RequestHeader	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).
requestHandle	IntegerId	Le <i>requestHandle</i> (descripteur de demande) affecté à une ou plusieurs demandes qu'il convient d'annuler. Toutes les demandes en cours avec le <i>requestHandle</i> correspondant doivent être annulées.
Response		
responseHeader	ResponseHeader	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).
cancelCount	UInt32	Nombre de demandes annulées.

5.6.5.3 Résultats des services

Les *StatusCodes* communs sont définis au Tableau 165.

5.7 Jeu de services de gestion de nœuds (NodeManagement Service Set)

5.7.1 Vue d'ensemble

Ce *Service Set* (Jeu de services) définit les *Services* pour ajouter et supprimer des *Nœuds de l'Espace d'Adresses* et les *Références* entre eux. Tous les *Nœuds* ajoutés continuent d'exister dans l'*Espace d'Adresses* même si le *Client* qui les a créés se déconnecte du *Serveur*.

5.7.2 Ajouter des nœuds (AddNodes)

5.7.2.1 Description

Ce *Service* est utilisé pour ajouter un ou plusieurs *Nœuds* dans la hiérarchie de l'*Espace d'Adresses*. En utilisant ce *Service*, chaque *Nœud* est ajouté comme étant le *TargetNode* d'une *HierarchicalReference* pour assurer que l'*Espace d'Adresses* est pleinement connecté et que le *Nœud* est ajouté comme enfant dans la hiérarchie de l'*Espace d'Adresses* (voir l'IEC 62541-3).

5.7.2.2 Paramètres

Le Tableau 18 définit les paramètres pour le *Service*.

Tableau 18 – Paramètres du service AddNodes

Nom	Type	Description
Request		
requestHeader	RequestHeader	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).
nodesToAdd []	AddNodesItem	Liste des <i>Nœuds</i> à ajouter. Tous les <i>Nœuds</i> sont ajoutés comme une <i>Référence</i> à un <i>Nœud</i> existant en utilisant un <i>ReferenceType</i> hiérarchique. Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.
parentNodeid	ExpandedNodeid	<i>ExpandedNodeid</i> (identificateur de nœud étendu) du <i>Nœud</i> parent pour la <i>Référence</i> . Le type <i>ExpandedNodeid</i> est défini en 7.10.
referenceTypeid	Nodeid	<i>Nodeid</i> du <i>ReferenceType</i> hiérarchique à utiliser pour la <i>Référence</i> allant du <i>Nœud</i> parent au nouveau <i>Nœud</i> .
requestedNewNodeid	ExpandedNodeid	<i>Nodeid</i> étendu demandé par le <i>Client</i> pour le <i>Nœud</i> à ajouter. Le <i>serverIndex</i> dans le <i>Nodeid</i> étendu doit être 0. Si le <i>Serveur</i> ne peut pas utiliser ce <i>Nodeid</i> , il rejette ce <i>Nœud</i> et renvoie un code d'erreur approprié. Si le <i>Client</i> ne souhaite pas demander un <i>Nodeid</i> , il met la valeur de ce paramètre au <i>Nodeid</i> étendu null. Si le <i>Nœud</i> à ajouter est un <i>Nœud</i> de <i>ReferenceType</i> , il convient que son <i>Nodeid</i> soit un identificateur numérique. Voir l'IEC 62541-3 pour une description des <i>Nodeids</i> de <i>ReferenceType</i> .
browseName	QualifiedName	Le nom de navigation du <i>Nœud</i> à ajouter.
nodeClass	NodeClass	<i>NodeClass</i> (classe de nœuds) du <i>Nœud</i> à ajouter.
nodeAttributes	ExtensibleParameter NodeAttributes	Les <i>Attributs</i> qui sont spécifiques à la <i>NodeClass</i> . Le type de paramètre <i>NodeAttributes</i> est un type de paramètre extensible spécifié en 7.18. Un <i>Client</i> est autorisé à omettre les valeurs pour certains ou tous les <i>Attributs</i> . Si une valeur d' <i>Attribut</i> est omise, le <i>Serveur</i> doit utiliser les valeurs par défaut provenant du <i>TypeDefinitionNode</i> . Si un <i>TypeDefinitionNode</i> n'a pas été fourni, le <i>Serveur</i> doit choisir une valeur par défaut appropriée. Le <i>Serveur</i> peut encore ajouter un <i>Attribut</i> facultatif au <i>Nœud</i> avec une valeur par défaut appropriée même si le <i>Client</i> ne spécifie pas une valeur.
typeDefinition	ExpandedNodeid	<i>Nodeid</i> du <i>TypeDefinitionNode</i> pour le <i>Nœud</i> à ajouter. Ce paramètre doit être null pour toutes les <i>NodeClasses</i> autres qu' <i>Object</i> et <i>Variable</i> et, dans ce cas, il doit être fourni.
Response		
responseHeader	ResponseHeader	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).
results []	AddNodesResult	Liste de résultats pour les <i>Nœuds</i> à ajouter. La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>nodesToAdd</i> . Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.
statusCode	StatusCode	<i>StatusCode</i> pour le <i>Nœud</i> à ajouter (voir 7.33 pour la définition de <i>StatusCode</i>).
addedNodeid	Nodeid	<i>Nodeid</i> affecté par le <i>Serveur</i> au <i>Nœud</i> ajouté. <i>Nodeid</i> null si l'opération a échoué.
diagnosticInfos []	DiagnosticInfo	Liste d'informations de diagnostic pour les <i>Nœuds</i> à ajouter (voir 7.8 pour la définition de <i>DiagnosticInfo</i>). La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>nodesToAdd</i> . Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été rencontrée dans le traitement de la demande.

5.7.2.3 Résultats des services

Le Tableau 19 définit les résultats de *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 19 – Codes des résultats de services AddNodes

Id symbolique	Description
Bad_NothingToDo	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_TooManyOperations	Voir le Tableau 165 pour la description de ce code de résultat.

5.7.2.4 StatusCodes

Le Tableau 20 définit les valeurs du paramètre *statusCode* de niveau opération qui sont spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 166.

Tableau 20 – Codes de résultats de niveau opération des AddNodes

Id symbolique	Description
Bad_ParentNodeInvalid	L'identificateur de nœud parent ne renvoie pas à un nœud valide.
Bad_ReferenceTypeInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_ReferenceNotAllowed	La référence n'a pas pu être créée, car elle viole des contraintes imposées par le modèle de données.
Bad_NodeIdRejected	L'identificateur de nœud demandé a été rejeté, car il était invalide ou parce que le serveur n'autorise pas que des identificateurs de nœuds soient spécifiés par le client.
Bad_NodeIdExists	L'identificateur de nœud demandé est déjà utilisé par un autre nœud.
Bad_NodeClassInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_BrowseNameInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_BrowseNameDuplicated	Le nom de navigation n'est pas unique parmi les nœuds qui partagent la même relation avec le parent.
Bad_NodeAttributesInvalid	Les <i>Attributs</i> de nœud ne sont pas valides pour la classe de nœuds.
Bad_TypeDefinitionInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_UserAccessDenied	Voir le Tableau 165 pour la description de ce code de résultat.

5.7.3 Ajouter des références (AddReferences)

5.7.3.1 Description

Ce *Service* est utilisé pour ajouter une ou plusieurs *Références* à un ou plusieurs *Nœuds*. La *NodeClass* est un paramètre d'entrée qui est utilisé pour valider le fait que la *Référence* à ajouter corresponde à la *NodeClass* du *TargetNode*. Ce paramètre n'est pas validé si la *Référence* renvoie à un *TargetNode* dans un *Serveur* distant.

Dans certains cas, l'ajout de nouvelles *Références* à l'*Espace d'Adresses* doit exiger que le *Serveur* ajoute de nouveaux identificateurs de *Serveur* à la *Variable ServerArray* (matrice de serveurs) du *Serveur*. C'est pourquoi les *Serveurs* distants sont identifiés par leur URI et non par leur indice de *ServerArray*. Le *Serveur* peut ainsi ajouter les URI de *Serveurs* distants à son *ServerArray*.

5.7.3.2 Paramètres

Le Tableau 21 définit les paramètres pour le *Service*.

Tableau 21 – Paramètres du service AddReferences

Nom	Type	Description
Request		
requestHeader	Request Header	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).
referencesToAdd []	AddReferences Item	Liste d'instances <i>Référence</i> à ajouter au <i>SourceNode</i> . La <i>targetNodeClass</i> de chaque <i>Référence</i> dans la liste doit correspondre à la <i>NodeClass</i> du <i>TargetNode</i> . Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.
sourceNodeId	NodeId	<i>NodeId</i> au <i>Nœud</i> auquel la <i>Référence</i> est à ajouter. Le <i>Nœud</i> source doit toujours exister dans le <i>Serveur</i> pour ajouter la <i>Référence</i> . Le paramètre <i>isForward</i> («est direct») peut être mis à FALSE si le <i>Nœud</i> cible est sur le <i>Serveur</i> local et le <i>Nœud</i> source sur le <i>Serveur</i> distant.
referenceTypeId	NodeId	<i>NodeId</i> du <i>ReferenceType</i> qui définit la <i>Référence</i> .
isForward	Boolean	Si la valeur est TRUE, le <i>Serveur</i> crée une <i>Référence</i> directe (forward Reference). Si la valeur est FALSE, le <i>Serveur</i> crée une <i>Référence</i> inverse (inverse Reference).
targetServerUri	String	URI du <i>Serveur</i> distant. Si ce paramètre n'est pas null, il se substitue au <i>serverIndex</i> dans le <i>targetNodeId</i> .
targetNodeId	Expanded NodeId	<i>NodeId</i> étendu du <i>TargetNode</i> . Le type <i>ExpandedNodeId</i> est défini en 7.10.
targetNodeClass	NodeClass	<i>NodeClass</i> du <i>TargetNode</i> . Le <i>Client</i> doit spécifier cela, car le <i>TargetNode</i> peut ne pas être directement accessible au <i>Serveur</i> .
Response		
responseHeader	Response Header	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).
results []	StatusCodes	Liste des <i>StatusCodes</i> pour les <i>Références</i> à ajouter (voir 7.33 pour la définition des <i>StatusCodes</i>). La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>referencesToAdd</i> .
diagnosticInfos []	Diagnostic Info	Liste d'informations de diagnostic pour les <i>Références</i> à ajouter (voir 7.8 pour la définition de <i>DiagnosticInfo</i>). La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>referencesToAdd</i> . Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été rencontrée dans le traitement de la demande.

5.7.3.3 Résultats des services

Le Tableau 22 définit les résultats de *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 22 – Codes des résultats de services AddReferences

Id symbolique	Description
Bad_NothingToDo	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_TooManyOperations	Voir le Tableau 165 pour la description de ce code de résultat.

5.7.3.4 StatusCodes

Le Tableau 23 définit les valeurs pour le paramètre *results* qui sont spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 166.

Tableau 23 – Codes de résultats de niveau opération des AddReferences

Id symbolique	Description
Bad_SourceNodeIdInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_ReferenceTypeIdInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_ServerUriInvalid	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_TargetNodeIdInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_NodeClassInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_ReferenceNotAllowed	La référence n'a pas pu être créée, car elle viole des contraintes imposées par le modèle de données à ce serveur.
Bad_ReferenceLocalOnly	Le type de référence n'est pas valide pour une référence à un <i>Serveur</i> distant.
Bad_UserAccessDenied	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_DuplicateReferenceNotAllowed	Le type de référence entre les nœuds est déjà défini.
Bad_InvalidSelfReference	Le serveur n'autorise pas ce type d'autoréférence sur ce nœud.

5.7.4 Supprimer des nœuds (DeleteNodes)

5.7.4.1 Description

Ce *Service* est utilisé pour supprimer un ou plusieurs *Nœuds* de l'*Espace d'Adresses*.

Lorsque l'un quelconque des *Nœuds* supprimés par une invocation de ce *Service* est le *TargetNode* d'une *Référence*, ces *Références* sont laissées non résolues sur la base du paramètre *deleteTargetReferences*.

Lorsque l'un quelconque des *Nœuds* supprimés par une invocation de ce *Service* est sous surveillance, une *Notification* contenant le code de statut Bad_NodeIdUnknown est envoyée au *Client* sous surveillance indiquant que le *Nœud* a été supprimé.

5.7.4.2 Paramètres

Le Tableau 24 définit les paramètres pour le *Service*.

Tableau 24 – Paramètres du service DeleteNodes

Nom	Type	Description
Request		
requestHeader	Request Header	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).
nodesToDelete []	DeleteNodes Item	Liste des <i>Nœuds</i> à supprimer. Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.
nodeId	NodeId	<i>NodeId</i> du <i>Nœud</i> à supprimer.
deleteTargetReferences	Boolean	Un paramètre <i>Boolean</i> ayant les valeurs suivantes: TRUE supprimer les <i>Références</i> dans les <i>TargetNodes</i> qui <i>réfèrent</i> le <i>Nœud</i> à supprimer. FALSE supprimer seulement les <i>Références</i> dont le <i>Nœud</i> à supprimer est la source. Le <i>Serveur</i> ne peut pas garantir qu'il est capable de supprimer toutes les <i>Références</i> des <i>TargetNodes</i> si ce paramètre est TRUE.
Response		
responseHeader	Response Header	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).
results []	StatusCode	Liste des <i>StatusCodes</i> pour les <i>Nœuds</i> à supprimer (voir 7.33 pour la définition des <i>StatusCode</i>). La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>nodesToDelete</i> .
diagnosticInfos []	Diagnostic Info	Liste d'informations de diagnostic pour les <i>Nœuds</i> à supprimer (voir 7.8 pour la définition de <i>DiagnosticInfo</i>). La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>nodesToDelete</i> . Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été rencontrée dans le traitement de la demande.

5.7.4.3 Résultats des services

Le Tableau 25 définit les résultats de *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 25 – Codes de résultats des services DeleteNodes

Id symbolique	Description
Bad_NothingToDo	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_TooManyOperations	Voir le Tableau 165 pour la description de ce code de résultat.

5.7.4.4 StatusCodes

Le Tableau 26 définit les valeurs pour le paramètre *results* qui sont spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 166.

Tableau 26 – Codes de résultats de niveau opération des DeleteNodes

Id symbolique	Description
Bad_NodeIdInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_NodeIdUnknown	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_UserAccessDenied	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_NoDeleteRights	Voir le Tableau 166 pour la description de ce code de résultat.
Uncertain_ReferenceNotDeleted	Le serveur n'était pas capable de supprimer toutes les références cibles.

5.7.5 Supprimer des références (DeleteReferences)

5.7.5.1 Description

Ce *Service* est utilisé pour supprimer une ou plusieurs *Références* d'un *Nœud*.

Lorsqu'un nombre quelconque de *Références* supprimées par une invocation de ce *Service* est contenu dans une *Vue*, la *Property ViewVersion* est mise à jour si cette *Property* est prise en charge.

Le Tableau 27 définit les paramètres du *Service*.

Tableau 27 – Paramètres du service DeleteReferences

Nom	Type	Description
Request		
requestHeader	RequestHeader	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).
referencesToDelete []	DeleteReferencesItem	Liste des <i>Références</i> à supprimer. Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.
sourceNodeId	NodeId	<i>NodeId</i> du <i>Nœud</i> qui contient la <i>Référence</i> à supprimer.
referenceTypeId	NodeId	<i>NodeId</i> du <i>ReferenceType</i> qui définit la <i>Référence</i> à supprimer.
isForward	Boolean	Si la valeur est TRUE, le <i>Serveur</i> supprime une <i>Référence</i> directe (forward Reference). Si la valeur est FALSE, le <i>Serveur</i> supprime une <i>Référence</i> inverse (inverse Reference).
targetNodeId	ExpandedNodeId	<i>NodeId</i> du <i>TargetNode</i> de la <i>Référence</i> . Si l'indice du <i>Serveur</i> indique que le <i>TargetNode</i> est un <i>Nœud</i> distant, le <i>nodeId</i> doit contenir l'URI d'espace de noms absolu. Si le <i>TargetNode</i> est un <i>Nœud</i> local, le <i>nodeId</i> doit contenir l'indice d'espace de noms.
deleteBidirectional	Boolean	Un paramètre <i>Boolean</i> ayant les valeurs suivantes: TRUE supprimer la <i>Référence</i> spécifiée et la <i>Référence</i> opposée partant du <i>TargetNode</i> . Si le <i>TargetNode</i> est situé sur un <i>Serveur</i> distant, le <i>Serveur</i> est autorisé à supprimer la <i>Référence</i> spécifiée seulement. FALSE supprimer la <i>Référence</i> spécifiée seulement.
Response		
responseHeader	ResponseHeader	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).
results []	StatusCode	Liste des <i>StatusCodes</i> pour les <i>Références</i> à supprimer (voir 7.33 pour la définition des <i>StatusCodes</i>). La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>referencesToDelete</i> .
diagnosticInfos []	DiagnosticInfo	Liste d'informations de diagnostic pour les <i>Références</i> à supprimer (voir 7.8 pour la définition de <i>DiagnosticInfo</i>). La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>referencesToDelete</i> . Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été trouvée dans le traitement de la demande.

5.7.5.2 Résultats des services

Le Tableau 28 définit les résultats des *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 28 – Codes de résultats des services DeleteReferences

Id symbolique	Description
Bad_NothingToDo	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_TooManyOperations	Voir le Tableau 165 pour la description de ce code de résultat.

5.7.5.3 StatusCodes

Le Tableau 29 définit les valeurs pour le paramètre *results* qui sont spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 166.

Tableau 29 – Codes de résultats de niveau opération des DeleteReferences

Id symbolique	Description
Bad_SourceNodeIdInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_ReferenceTypeIdInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_ServerIndexInvalid	L'indice de serveur n'est pas valide.
Bad_TargetNodeIdInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_UserAccessDenied	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_NoDeleteRights	Voir le Tableau 166 pour la description de ce code de résultat.

5.8 Jeu de services pour les vues (View Service Set)

5.8.1 Vue d'ensemble

Les *Clients* utilisent les *Services* navigation du *View Service Set* (Jeu de services pour les vues) pour naviguer à travers l'*Espace d'Adresses* ou à travers une *Vue* qui est un sous-ensemble de l'*Espace d'Adresses*.

Une *Vue* est un sous-ensemble de l'*Espace d'Adresses* créé par le *Serveur*. Les futures versions de la présente Norme peuvent aussi définir des services pour créer des *Vues* définies par le *Client*. Voir l'IEC 62541-5 pour une description de l'organisation des vues dans l'*Espace d'Adresses*.

5.8.2 Parcourir (Browse)

5.8.2.1 Description

Ce *Service* est utilisé pour découvrir les *Références* d'un *Nœud* spécifié. La navigation peut en outre être limitée par l'utilisation d'une *Vue*. Ce *Service* Browse prend aussi en charge une fonction de filtrage primitive.

5.8.2.2 Paramètres

Le Tableau 30 définit les paramètres du *Service*.

Tableau 30 – Paramètres du service Browse

Nom	Type	Description																		
Request																				
requestHeader	RequestHeader	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).																		
view	ViewDescription	Description de la <i>Vue</i> à parcourir (voir 7.37 pour la définition de <i>ViewDescription</i>). Une valeur vide de <i>ViewDescription</i> indique l' <i>Espace d'Adresses</i> tout entier. L'utilisation de la valeur vide de <i>ViewDescription</i> entraîne que toutes les <i>Références</i> du <i>nodesToBrowse</i> sont retournées. L'utilisation d'une autre <i>Vue</i> entraîne que seules les <i>Références</i> du <i>nodesToBrowse</i> qui sont définies pour la <i>Vue</i> en question seront retournées.																		
requestedMaxReferencesPerNode	Counter	Indique le nombre maximal de références à retourner pour chaque Nœud de départ spécifié dans la demande. La valeur 0 indique que le <i>Client</i> n'impose aucune limitation (voir 7.5 pour la définition de <i>Counter</i>).																		
nodesToBrowse []	BrowseDescription	Une liste de nœuds à parcourir (Browse). Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.																		
nodeId	NodeId	<i>NodeId</i> du Nœud à parcourir. Si une <i>Vue</i> est fournie, elle doit inclure ce <i>Nœud</i> .																		
browseDirection	Enum BrowseDirection	Une énumération qui spécifie le sens des <i>Références</i> à suivre. Cette énumération a les valeurs suivantes: FORWARD_0 sélectionner des <i>Références</i> directes seulement. INVERSE_1 sélectionner des <i>Références</i> inverses seulement. BOTH_2 sélectionner des <i>Références</i> directes et inverses. Les <i>Références</i> retournées indiquent effectivement le sens que le <i>Serveur</i> a suivi dans le paramètre <i>isForward</i> de la <i>ReferenceDescription</i> . Les <i>Références</i> symétriques sont toujours considérées être dans le sens direct; par conséquent, le fanion <i>isForward</i> est toujours mis à TRUE et les <i>Références</i> symétriques ne sont pas retournées si <i>browseDirection</i> (sens de navigation) est mis à <i>INVERSE_1</i> .																		
referenceTypeId	NodeId	Spécifie le <i>NodeId</i> du <i>ReferenceType</i> à suivre. Seules des instances de ce <i>ReferenceType</i> ou de ses sous-types sont retournées. S'il n'est pas spécifié, toutes les <i>Références</i> sont retournées et <i>includeSubtypes</i> est ignoré.																		
includeSubtypes	Boolean	Indique s'il convient d'inclure dans la navigation les sous-types du <i>ReferenceType</i> . Si TRUE, les instances de <i>referenceTypeId</i> et de tous ses sous-types sont retournées.																		
nodeClassMask	UInt32	Spécifie les <i>NodeClasses</i> des <i>TargetNodes</i> . Seuls les <i>TargetNodes</i> avec les <i>NodeClasses</i> sélectionnées sont retournés. Il est affecté aux <i>NodeClasses</i> les bits suivants: <table border="1"><thead><tr><th>Bit</th><th>NodeClass</th></tr></thead><tbody><tr><td>0</td><td>Object</td></tr><tr><td>1</td><td>Variable</td></tr><tr><td>2</td><td>Method</td></tr><tr><td>3</td><td>ObjectType</td></tr><tr><td>4</td><td>VariableType</td></tr><tr><td>5</td><td>ReferenceType</td></tr><tr><td>6</td><td>DataType</td></tr><tr><td>7</td><td>View</td></tr></tbody></table> S'il est mis à zéro, toutes les <i>NodeClasses</i> sont retournées.	Bit	NodeClass	0	Object	1	Variable	2	Method	3	ObjectType	4	VariableType	5	ReferenceType	6	DataType	7	View
Bit	NodeClass																			
0	Object																			
1	Variable																			
2	Method																			
3	ObjectType																			
4	VariableType																			
5	ReferenceType																			
6	DataType																			
7	View																			
resultMask	UInt32	Spécifie les champs dans la structure <i>ReferenceDescription</i> qu'il convient de retourner. Il est affecté aux champs les bits suivants: <table border="1"><thead><tr><th>Bit</th><th>Result</th></tr></thead><tbody><tr><td>0</td><td>ReferenceType</td></tr><tr><td>1</td><td>IsForward</td></tr><tr><td>2</td><td>NodeClass</td></tr><tr><td>3</td><td>BrowseName</td></tr><tr><td>4</td><td>DisplayName</td></tr><tr><td>5</td><td>TypeDefinition</td></tr></tbody></table> Le type <i>ReferenceDescription</i> est défini en 7.24.	Bit	Result	0	ReferenceType	1	IsForward	2	NodeClass	3	BrowseName	4	DisplayName	5	TypeDefinition				
Bit	Result																			
0	ReferenceType																			
1	IsForward																			
2	NodeClass																			
3	BrowseName																			
4	DisplayName																			
5	TypeDefinition																			
Response																				
responseHeader	Response Header	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).																		
results []	BrowseResult	Une liste de <i>BrowseResults</i> (résultats de navigation). La taille et l'ordre de la liste correspondent à la taille et à l'ordre du <i>nodesToBrowse</i> spécifié dans la demande. Le type <i>BrowseResult</i> est défini en 7.3.																		
diagnosticInfos []	Diagnostic Info	Liste d'informations de diagnostic pour les <i>results</i> (voir 7.8 pour la définition de																		

Nom	Type	Description
		<i>DiagnosticInfo</i>). La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de réponse <i>results</i> . Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été rencontrée dans le traitement de la demande.

5.8.2.3 Résultats des services

Le Tableau 31 définit les résultats des *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 31 – Codes de résultats des services Browse

Id symbolique	Description
Bad_ViewIdUnknown	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_ViewTimestampInvalid	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_ViewParameterMismatchInvalid	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_ViewVersionInvalid	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_NothingToDo	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_TooManyOperations	Voir le Tableau 165 pour la description de ce code de résultat.

5.8.2.4 StatusCodes

Le Tableau 32 définit les valeurs pour le paramètre *results* qui sont spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 166.

Tableau 32 – Codes de résultats de niveau opération de Browse

Id symbolique	Description
Bad_NodeIdInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_NodeIdUnknown	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_ReferenceTypeIdInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_BrowseDirectionInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_NodeNotInView	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_NoContinuationPoints	Voir le Tableau 166 pour la description de ce code de résultat.
Uncertain_NotAllNodesAvailable	Les résultats de la navigation (Browse) peuvent être incomplets en raison de l'indisponibilité d'un sous-système.

5.8.3 Parcourir le prochain (BrowseNext)

5.8.3.1 Description

Ce *Service* est utilisé pour demander le prochain jeu d'informations de réponse *Browse* ou *BrowseNext* qui sont trop volumineuses pour être envoyées dans une seule réponse. «Trop volumineuses» signifie dans ce contexte que le *Serveur* n'est pas capable de retourner une réponse plus volumineuse ou que le nombre de résultats à retourner dépasse le nombre maximal de résultats à retourner qui a été spécifié par le *Client* dans la demande *Browse* d'origine. Le *BrowseNext* doit être présenté dans la même *Session* qui a été utilisée pour présenter le *Browse* ou *BrowseNext* qui se poursuit.

5.8.3.2 Paramètres

Le Tableau 33 définit les paramètres du *Service*.

Tableau 33 – Paramètres du service BrowseNext

Nom	Type	Description
Request		
requestHeader	Request Header	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).
releaseContinuationPoints	Boolean	Un paramètre <i>Boolean</i> ayant les valeurs suivantes: TRUE les <i>continuationPoints</i> passés doivent être réinitialisés pour libérer des ressources dans le <i>Serveur</i> . Les points de continuation sont libérés et les résultats et les matrices de <i>diagnosticInfos</i> sont vides. FALSE les <i>continuationPoints</i> passés doivent être utilisés pour récupérer le prochain jeu d'informations de navigation. Un <i>Client</i> doit toujours utiliser le point de continuation retourné par une réponse <i>Browse</i> ou <i>BrowseNext</i> pour libérer les ressources pour le point de continuation dans le <i>Serveur</i> . Si le <i>Client</i> ne souhaite pas récupérer le prochain jeu d'informations de navigation, <i>BrowseNext</i> doit être appelé avec ce paramètre mis à TRUE.
continuationPoints []	Continuation Point	Une liste de valeurs opaques définies par le <i>Serveur</i> qui représentent des points de continuation. La valeur pour un point de continuation a été retournée au <i>Client</i> dans une réponse <i>Browse</i> ou <i>BrowseNext</i> précédente. Ces valeurs sont utilisées pour identifier la demande <i>Browse</i> ou <i>BrowseNext</i> précédemment traitée qui se poursuit et le point dans le jeu de résultats à partir duquel il faut que la réponse de navigation continue. Les Clients peuvent mélanger les points de continuation provenant de réponses <i>Browse</i> ou <i>BrowseNext</i> différentes. Le type <i>ContinuationPoint</i> est décrit en 7.6.
Response		
responseHeader	Response Header	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).
results []	BrowseResult	Une liste de références qui satisfont aux critères spécifiés dans la demande <i>Browse</i> d'origine. La taille et l'ordre de cette liste correspondent à la taille et à l'ordre du paramètre de demande <i>continuationPoints</i> . Le type <i>BrowseResult</i> est défini en 7.3.
diagnosticInfos []	Diagnostic Info	Liste d'informations de diagnostic pour les <i>results</i> (voir 7.8 pour la définition de <i>DiagnosticInfo</i>). La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de réponse <i>results</i> . Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été rencontrée dans le traitement de la demande.

5.8.3.3 Résultats des services

Le Tableau 34 définit les résultats des *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 34 – Codes de résultats des services BrowseNext

Id symbolique	Description
Bad_NothingToDo	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_TooManyOperations	Voir le Tableau 165 pour la description de ce code de résultat.

5.8.3.4 StatusCodes

Le Tableau 35 définit les valeurs pour le paramètre *results* qui sont spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 166.

Tableau 35 – Codes de résultats de niveau opération de BrowseNext

Id symbolique	Description
Bad_NodeIdInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_NodeIdUnknown	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_ReferenceTypeInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_BrowseDirectionInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_NodeNotInView	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_ContinuationPointInvalid	Voir le Tableau 166 pour la description de ce code de résultat.

5.8.4 Traduire les chemins de navigation en identifiants de nœuds (TranslateBrowsePathsToNodeIds)

5.8.4.1 Description

Ce *Service* est utilisé pour demander au *Serveur* de traduire un ou plusieurs chemins de navigation en des *NodeIds*. Chaque chemin de navigation est composé d'un *Nœud* de départ et d'un *RelativePath* (chemin relatif). Le *Nœud* de départ spécifié identifie le *Nœud* sur lequel le *RelativePath* est basé. Le *RelativePath* contient une séquence de *ReferenceTypes* et de *BrowseNames*.

Un objectif de ce *Service* est de permettre la programmation en fonction des définitions de type. Sachant que les *BrowseNames* doivent être uniques dans le contexte des définitions de type, un *Client* peut créer un chemin de navigation qui est valide pour une définition de type et utiliser ce chemin sur des instances du type. Par exemple, un *ObjectType* "Boiler" (chaudière) peut avoir une *Variable* "HeatSensor" (capteur de chaleur) comme *InstanceDeclaration*. Un élément graphique programmé en fonction du "Boiler" peut avoir besoin d'afficher la *Value* du "HeatSensor". Si l'élément graphique était appelé sur "Boiler1", une instance de "Boiler", il aurait besoin d'appeler ce *Service* en spécifiant le *NodeId* de "Boiler1" comme *Nœud* de départ et le *BrowseName* du "HeatSensor" comme chemin de navigation. Le *Service* retournerait le *NodeId* du "HeatSensor" de "Boiler1" et l'élément graphique pourrait s'abonner à son *Attribut Value*.

Si un *Nœud* a plusieurs cibles ayant le même *BrowseName*, le *Serveur* doit retourner une liste de *NodeIds*. Cependant, comme l'un des principaux objectifs de ce *Service* est de prendre en charge la programmation en fonction des définitions de type, le *NodeId* du *Nœud* basé sur la définition de type du *Nœud* de départ est retourné comme le premier *NodeId* dans la liste.

5.8.4.2 Paramètres

Le Tableau 36 définit les paramètres du *Service*.

Tableau 36 – Paramètres du service TranslateBrowsePathsToNodeIds

Nom	Type	Description
Request		
requestHeader	RequestHeader	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).
browsePaths []	BrowsePath	Liste des chemins de navigation pour lesquels les <i>NodeIds</i> sont demandés. Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.
startingNode	NodeId	<i>NodeId</i> du <i>Nœud</i> de départ pour le chemin de navigation.
relativePath	RelativePath	Le chemin à suivre à partir du <i>startingNode</i> . Le dernier élément dans le <i>relativePath</i> doit toujours avoir un <i>targetName</i> spécifié. Cela restreint encore plus la définition du type <i>RelativePath</i> . Le <i>Serveur</i> doit retourner <i>Bad_BrowseNameInvalid</i> si le <i>targetName</i> est absent. La structure <i>RelativePath</i> est définie en 7.25.
Response		
responseHeader	ResponseHeader	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).
results []	BrowsePathResult	Liste de résultats pour la liste de chemins de navigation. La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>browsePaths</i> . Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.
statusCode	StatusCode	<i>StatusCode</i> pour le chemin de navigation (voir 7.33 pour la définition de <i>StatusCode</i>).
targets []	BrowsePathTarget	Liste de cibles pour le <i>relativePath</i> à partir du <i>startingNode</i> . Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants. Un <i>Serveur</i> peut rencontrer une <i>Référence</i> à un <i>Nœud</i> dans un autre <i>Serveur</i> qu'il ne peut pas suivre pendant qu'il traite le <i>RelativePath</i> . Si cela se produit, le <i>Serveur</i> retourne le <i>NodeId</i> du <i>Nœud</i> externe et positionne le paramètre <i>remainingPathIndex</i> pour indiquer les éléments <i>RelativePath</i> qu'il est encore nécessaire de traiter. Pour parachever l'opération, le <i>Client</i> doit se connecter à l'autre <i>Serveur</i> et appeler de nouveau ce service en utilisant la cible comme le <i>startingNode</i> et les éléments non traités comme le <i>relativePath</i> .
targetId	ExpandedNodeId	L'identificateur pour une cible du <i>RelativePath</i> .
remainingPathIndex	Index	L'indice du premier élément non traité dans le <i>RelativePath</i> . Cette valeur doit être égale à la valeur maximale du type de données <i>Index</i> si tous les éléments ont été traités (voir 7.12 pour la définition d' <i>Index</i>).
diagnosticInfos []	DiagnosticInfo	Liste d'informations de diagnostic pour la liste des chemins de navigation (voir 7.8 pour la définition de <i>DiagnosticInfo</i>). La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>browsePaths</i> . Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été rencontrée dans le traitement de la demande.

5.8.4.3 Résultats des services

Le Tableau 37 définit les résultats de *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis en 7.33.

Tableau 37 – Codes de résultats des services TranslateBrowsePathsToNodeIds

Id symbolique	Description
Bad_NothingToDo	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_TooManyOperations	Voir le Tableau 165 pour la description de ce code de résultat.

5.8.4.4 StatusCodes

Le Tableau 38 définit les valeurs pour les paramètres *statusCode* de niveau opération qui sont spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 166.

Tableau 38 – Codes de résultats de niveau opération de TranslateBrowsePathsToNodeIds

Id symbolique	Description
Bad_NodeIdInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_NodeIdUnknown	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_NothingToDo	Voir le Tableau 165 pour la description de ce code de résultat. Ce code indique que le relativePath contenait une liste vide.
Bad_BrowseNameInvalid	Voir le Tableau 166 pour la description de ce code de résultat. Ce code indique qu'un TargetName était absent dans un RelativePath.
Uncertain_ReferenceOutOfServer	L'élément de chemin a des cibles qui sont dans un autre serveur.
Bad_TooManyMatches	L'opération demandée a de trop nombreuses correspondances à retourner. Il convient que les utilisateurs emploient des interrogations pour les volumineux jeux de résultats. Il convient que les <i>Serveurs</i> autorisent au moins 10 correspondances avant de retourner ce code d'erreur.
Bad_QueryTooComplex	L'opération demandée exige un trop grand nombre de ressources dans le serveur.
Bad_NoMatch	Le relativePath demandé ne peut être résolu en une cible à retourner.

5.8.5 Enregistrer les nœuds (RegisterNodes)

5.8.5.1 Description

Un *Serveur* n'a souvent pas d'accès direct aux informations qu'il gère. Les variables ou les services peuvent se trouver dans des systèmes sous-jacents quand un effort supplémentaire est requis pour établir une connexion avec ces systèmes. Le *Service RegisterNodes* peut être utilisé par des *Clients* pour enregistrer les *Nœuds* auxquels ils savent qu'ils vont accéder de façon répétée (par exemple Write, Call). Il permet aux *Serveurs* d'installer tout ce qui est nécessaire pour rendre les opérations d'accès plus efficaces. Les *Clients* peuvent espérer des améliorations de performance lorsqu'ils utilisent des *NodeIds* enregistrés, mais les mesures d'optimisation sont spécifiques au fournisseur. Pour les *Nodes Variable*, les *Serveurs* doivent concentrer leurs efforts d'optimisation sur l'*Attribut Value*.

La validité des *NodeIds* enregistrés ne peut être garantie que dans la *Session* courante. Les *Clients* doivent immédiatement supprimer les enregistrements des identificateurs inutiles pour libérer des ressources.

RegisterNodes ne valide pas les *NodeIds* issus de la demande. Les *Serveurs* copient simplement les *NodeIds* inconnus dans la réponse. Les erreurs de structure des *NodeIds* (violations de taille, types d'identificateur non valides) entraînent la défaillance du *Service* complet.

Pour les besoins de l'*Auditing* (Audits), les *Serveurs* ne doivent pas utiliser les *NodeIds* enregistrés, mais ils doivent utiliser seulement les *NodeIds* canoniques qui sont la valeur de l'*Attribut NodeId*.

5.8.5.2 Paramètres

Le Tableau 39 définit les paramètres du *Service*.

Tableau 39 – Paramètres du service RegisterNodes

Nom	Type	Description
Request		
requestHeader	Request Header	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).
nodesToRegister []	NodeId	Liste des <i>NodeIds</i> à enregistrer que le client a récupérés par navigation, interrogation ou de quelque autre manière.
Response		
responseHeader	Response Header	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).
registeredNodeIds []	NodeId	Une liste de <i>NodeIds</i> que le <i>Client</i> doit utiliser pour des opérations d'accès ultérieures. La taille et l'ordre de cette liste correspondent à la taille et à l'ordre du paramètre de demande <i>nodesToRegister</i> . Le <i>Serveur</i> peut retourner le <i>NodeId</i> de la demande ou un nouveau <i>NodeId</i> (un pseudonyme). Il est recommandé que le <i>Serveur</i> retourne un <i>NodeIds</i> numérique pour la dénomination. Si aucune optimisation n'est prise en charge pour un <i>Nœud</i> , le <i>Serveur</i> doit retourner le <i>NodeId</i> issu de la demande.

5.8.5.3 Résultats des services

Le Tableau 40 définit les résultats des *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 40 – Codes de résultats des services RegisterNodes

Id symbolique	Description
Bad_NothingToDo	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_TooManyOperations	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_NodeIdInvalid	Voir le Tableau 166 pour la description de ce code de résultat. Les <i>Serveurs</i> doivent rejeter complètement la demande <i>RegisterNodes</i> si l'un quelconque des <i>NodeIds</i> dans le paramètre <i>nodesToRegister</i> est structurellement non valide.

5.8.6 Supprimer l'enregistrement de nœuds (UnregisterNodes)

5.8.6.1 Description

Ce *Service* est utilisé pour supprimer l'enregistrement des *NodeIds* qui ont été obtenus via le service *RegisterNodes*.

UnregisterNodes ne valide pas les *NodeIds* issus de la demande. Les *Serveurs* doivent simplement supprimer l'enregistrement des *NodeIds* qui sont connus comme des *NodeIds* enregistrés. Tous les *NodeIds* qui sont dans la liste, mais ne sont pas des *NodeIds* enregistrés, sont simplement ignorés.

5.8.6.2 Paramètres

Le Tableau 41 définit les paramètres du *Service*.

Tableau 41 – Paramètres du service UnregisterNodes

Nom	Type	Description
Request		
requestHeader	Request Header	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).
nodesToUnregister []	NodeId	Une liste des <i>NodeIds</i> qui ont été obtenus via le service <i>RegisterNodes</i> .
Response		
responseHeader	Response Header	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).

5.8.6.3 Résultats des services

Le Tableau 42 définit les résultats des *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 42 – Codes de résultats des services UnregisterNodes

Id symbolique	Description
Bad_NothingToDo	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_TooManyOperations	Voir le Tableau 165 pour la description de ce code de résultat.

5.9 Jeu de services d'interrogation (Query Service Set)

5.9.1 Vue d'ensemble

Ce *Service Set* est utilisé pour émettre une *Query* (interrogation) vers un *Serveur*. La *Query* OPC UA est générique en ce qu'elle fournit un mécanisme de stockage sous-jacent indépendant de la fonction *Query* qui peut être utilisée pour accéder à une grande variété d'unités de stockage de données et de systèmes de gestion d'informations d'OPC UA. La *Query* OPC UA permet à un *Client* d'accéder à des données maintenues par un *Serveur* sans connaître le plan logique utilisé pour le stockage interne des données. La connaissance de l'*Espace d'Adresses* est suffisante.

Une *Application OPC UA* est censée utiliser les *Services Query* d'OPC UA comme une partie d'un processus d'initialisation ou une étape occasionnelle de synchronisation d'informations. Par exemple, *Query* OPC UA serait utilisée pour un accès à des données en vrac d'une unité de stockage persistante pour initialiser une application d'analyse avec l'état courant d'une configuration système. Une *Query* peut aussi être utilisée pour initialiser ou alimenter des données en vue d'un rapport.

Une *Query* définit les instances d'un ou plusieurs *TypeDefinitionNodes* dans l'*Espace d'Adresses* qu'il convient d'appliquer à un jeu d'*Attributs*. Les résultats retournés par un *Serveur* sont sous la forme d'une matrice de *QueryDataSets* (jeu de données d'interrogation). Les valeurs d'*Attributs* sélectionnées dans chaque *QueryDataSet* proviennent de la définition des *TypeDefinitionNodes* ou *TypeDefinitionNodes connexes* sélectionnés et les résultats apparaissent dans le même ordre que les *Attributs* qui ont été passés dans la *Query*. La *Query* prend également en charge le filtrage des *Nœuds* en fonction des valeurs d'*Attributs*, ainsi que les relations entre *TypeDefinitionNodes*.

Voir Annexe B pour des exemples d'interrogations.

5.9.2 Interroger des vues (Querying Views)

Une *Vue* est un sous-ensemble de l'*Espace d'Adresses* disponible dans le *Serveur*. Voir l'IEC 62541-5 pour une description de l'organisation des *Vues* dans l'*Espace d'Adresses*.

Pour toute *Vue* existante, une *Query* peut être utilisée pour retourner un sous-ensemble de données de la *Vue*. Lorsqu'une application émet une *Query* en fonction d'une *Vue*, seules les données définies par la *Vue* sont retournées. Les données non incluses dans la *Vue*, mais incluses dans l'*Espace d'Adresses* d'origine ne sont pas retournées.

Les *Services Query* prennent en charge l'accès à des données courantes et historiques. Le *Service* prend en charge un *Client* qui interroge une version antérieure de l'*Espace d'Adresses*. Les *Clients* peuvent spécifier une *ViewVersion* (Version de vue) ou un *Timestamp* (Horodatage) dans une *Query* pour accéder à des versions antérieures de l'*Espace d'Adresses*. Le service *Query* OPC UA est complémentaire au service Historical Access (Accès à historique) en ce que le premier est utilisé pour *Query* (interroger) un *Espace d'Adresses* qui existait à un moment donné et que le second est utilisé pour *Query* afin d'obtenir la valeur des *Attributs* au fil du temps. Ainsi, une *Query* peut être utilisée pour récupérer une partie d'un *Espace d'Adresses* antérieur et, de ce fait, un historique des

valeurs de l'*Attribut* peut être accessible à l'aide de Historical Access même si le *Nœud* ne se trouve plus dans l'*Espace d'Adresses* courant.

Les *Serveurs* qui prennent en charge *Query* sont censés être capables d'accéder à l'*Espace d'Adresses* qui est associé au *Serveur* local et à n'importe quelles *Vues* qui sont disponibles sur le *Serveur* local. Si une *Vue* ou l'espace d'adresses référence également un *Serveur* distant, l'interrogation peut être capable d'accéder à l'espace d'adresses du *Serveur* distant, mais cela n'est pas obligatoire. Si un *Serveur* accède effectivement à un *Serveur* distant, l'accès doit être accompli en utilisant l'identité d'utilisateur du *Client* tel que décrit en 5.5.1.

5.9.3 Émettre une demande (QueryFirst)

5.9.3.1 Description

Ce *Service* est utilisé pour émettre une demande *Query* vers le *Serveur*. La complexité de la *Query* peut être très simple ou hautement sophistiquée. La *Query* peut simplement demander des données issues d'instances d'un *TypeDefinitionNode* ou *TypeDefinitionNode* soumis à des restrictions spécifiées par le filtre. Par ailleurs, la *Query* peut demander des données issues d'instances de type *Nœuds* connexes en spécifiant un *RelativePath* partant d'un *TypeDefinitionNode* d'origine. Dans le filtre, un jeu séparé de chemins peut être construit pour limiter les instances qui fournissent les données. Un chemin filtrant peut inclure plusieurs opérateurs *RelatedTo* (Relié à) pour définir un chemin à sauts multiples entre des instances sources et des instances cibles. Par exemple, on pourrait passer au filtre des étudiants d'une école particulière, mais retourner des informations relatives aux étudiants et à leurs familles. Dans ce cas, la relation étudiant-école est parcourue pour le filtrage, mais la relation étudiant-famille est parcourue pour sélectionner des données. Pour une description complète de *ContentFilter*, voir 7.4; voir aussi B.1 pour des exemples simples et B.2 pour de plus complexes exemples de filtre de contenu et d'interrogations.

Le *Client* fournit une matrice de *NodeTypeDescription* qui spécifie le *NodeId* d'un *TypeDefinitionNode* et sélectionne les *Attributs* à retourner dans la réponse. Un client peut aussi fournir un jeu de *RelativePaths* à travers le système de type commençant à partir d'un *TypeDefinitionNode* de départ. En utilisant ces chemins, le client sélectionne un jeu d'*Attributs* à partir des *Nœuds* qui sont reliés à des instances du *TypeDefinitionNode* de départ. En outre, le *Client* peut demander au *Serveur* de retourner des instances des sous-types de *TypeDefinitionNodes*. Si un *Attribut* sélectionné n'existe pas dans un *TypeDefinitionNode*, mais existe dans un sous-type, il est admis avoir une valeur null dans le *TypeDefinitionNode* en question. Par conséquent, ceci ne constitue pas une condition d'erreur et une valeur null est retournée pour l'*Attribut*.

Le *Client* peut utiliser le paramètre filtre pour limiter le jeu de résultats en limitant les *Attributs* et les *Propriétés* à certaines valeurs. Une autre manière dont le *Client* peut utiliser un filtre pour limiter le jeu de résultats consiste à spécifier comment il convient de relier les instances, en utilisant des opérateurs *RelatedTo*. Dans ce cas, si une instance au sommet du chemin *RelatedTo* ne peut pas être suivie jusqu'au bas du chemin via des sauts spécifiés, aucun *QueryDataSet* n'est retourné pour l'instance de départ ou l'une quelconque des instances intermédiaires.

Lorsqu'il effectue une interrogation relative à des instances connexes dans le *RelativePath*, le *Client* peut facultativement demander des *Références*. Une *Référence* est demandée via un *RelativePath* qui inclut seulement un *ReferenceType*. Si toutes les *Références* sont désirées, le *ReferenceType* racine est alors énuméré. Ces *Références* sont retournées comme partie des *QueryDataSets*.

Les *Services Query* permettent une gestion particulière du champ *targetName* dans le *RelativePath*. Dans plusieurs cas d'utilisation des *Services Query*, un *NodeId* de type est nécessaire dans le chemin, à la place d'un *QualifiedName*. Le *Client* peut donc spécifier un *NodeId* dans le *QualifiedName*. Cela est réalisé en mettant le *namespaceIndex* du *targetName* à zéro et la partie nom du *targetName* à la représentation en XML du *NodeId*. La représentation en XML est définie dans l'IEC 62541-6. Quand les instances de

correspondance sont retournées comme le Nœud cible, le Nœud cible doit être une instance du type spécifié ou de son sous-type.

Le Tableau 43 définit les paramètres de demande et le Tableau 44 les paramètres de réponse pour le *Service QueryFirst*.

Tableau 43 – Paramètres de demande QueryFirst

Nom	Type	Description
Request		
requestHeader	RequestHeader	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).
view	ViewDescription	Spécifie une <i>Vue</i> et un contexte temporel au <i>Serveur</i> (voir 7.37 pour la définition de <i>ViewDescription</i>).
nodeTypes []	NodeTypeDescription	Il s'agit de la description du type de <i>Nœud</i> . Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.
typeDefinitionNode	ExpandedNodeId	<i>NodeId</i> du <i>TypeDefinitionNode</i> de départ des instances pour lesquelles les données sont à retourner.
includeSubtypes	Boolean	Un fanion qui indique si, oui ou non, il convient que le <i>Serveur</i> inclue des instances des sous-types de <i>TypeDefinitionNode</i> dans la liste des instances du type <i>Node</i> .
dataToReturn []	QueryDataDescription	Spécifie un <i>Attribut</i> ou une <i>Référence</i> partant du <i>typeDefinitionNode</i> de départ le long d'un <i>relativePath</i> donné pour lequel retourner des données. Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.
relativePath	RelativePath	Chemin de navigation relatif au <i>Nœud</i> de départ qui identifie le <i>Nœud</i> contenant les données demandées, le <i>Nœud</i> de départ étant un <i>Nœud</i> d'instance du type défini par le <i>Nœud</i> de définition de type. Les <i>Nœuds</i> d'instance font l'objet d'une limitation supplémentaire par le filtre fourni comme partie de cet appel. Pour une définition de <i>relativePath</i> , voir 7.25. Ce chemin relatif peut se terminer sur une <i>Référence</i> et, dans ce cas, la <i>ReferenceDescription</i> de la <i>Référence</i> est retournée comme étant sa valeur. Le champ <i>targetName</i> du <i>relativePath</i> peut contenir un <i>NodeId</i> de type. Cela est réalisé en mettant le <i>namespaceIndex</i> du <i>targetName</i> à zéro et la partie nom du <i>targetName</i> de la représentation en XML du <i>NodeId</i> . La représentation en XML est définie dans l'IEC 62541-6. Quand les instances de correspondance sont retournées comme le <i>Nœud</i> cible, le <i>Nœud</i> cible doit être une instance du type spécifié ou de son sous-type.
attributId	IntegerId	Identificateur de l' <i>Attribut</i> . Il doit être un identificateur d' <i>Attribut</i> valide. L' <i>IntegerId</i> est défini en 7.13. L' <i>IntegerId</i> pour les <i>Attributs</i> est défini dans l'IEC 62541-6. Si le <i>RelativePath</i> s'est terminé à une <i>Référence</i> , ce paramètre est 0 et ignoré par le serveur.
indexRange	NumericRange	Ce paramètre est utilisé pour identifier un élément simple d'une structure ou d'une matrice ou bien une plage simple d'indices pour les matrices. Si une plage d'éléments est spécifiée, les valeurs sont retournées sous forme de composé. Le premier élément est identifié par l'indice 0 (zéro). Le type <i>NumericRange</i> est défini en 7.21. Ce paramètre est null si l' <i>Attribut</i> spécifié n'est pas une matrice ou une structure. Cependant, si l' <i>Attribut</i> spécifié est une matrice ou une structure, et ce paramètre est donc null, tous les éléments sont à inclure dans la plage.
filter	ContentFilter	Les <i>Nœuds</i> obtenus doivent être limités aux <i>Nœuds</i> qui correspondent aux critères définis par le filtre. <i>ContentFilter</i> est l'objet d'un débat en 7.4. Si un filtre vide est fourni, l' <i>espace d'adresses</i> tout entier doit être examiné et tous les <i>Nœuds</i> contenant un <i>Attribut</i> ou une <i>Référence</i> demandé(e) correspondant aux critères sont retournés.
maxDataSetsToReturn	Counter	Le nombre de <i>QueryDataSets</i> que le <i>Client</i> souhaite que le <i>Serveur</i> retourne dans la réponse et sur chaque réponse ultérieure d'appel de continuation. Le <i>Serveur</i> est autorisé à limiter encore plus la réponse, mais ne doit pas dépasser cette limite. Une valeur 0 indique que le <i>Client</i> n'impose aucune limitation.
maxReferencesToReturn	Counter	Le nombre de <i>Références</i> que le <i>Client</i> souhaite que le <i>Serveur</i> retourne dans la réponse pour chaque <i>QueryDataSet</i> et sur chaque réponse ultérieure d'appel de continuation. Le <i>Serveur</i> est autorisé à limiter encore plus la réponse, mais ne doit pas dépasser cette limite. Une valeur 0 indique que le <i>Client</i> n'impose aucune limitation. Par exemple, dans le cas d'un résultat où quatre <i>Nœuds</i> sont retournés, mais chacun a 100 <i>Références</i> et cette limite est fixée à 50, seules les 50 premières <i>Références</i> pour chaque <i>Nœud</i> seront retournées sur l'appel initial et un point de continuation sera établi pour indiquer l'existence de données supplémentaires.

Tableau 44 – Paramètres de réponse QueryFirst

Nom	Type	Description
Response		
responseHeader	ResponseHeader	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).
queryDataSets []	QueryDataSet	La matrice de <i>QueryDataSets</i> . Cette matrice est vide s'il n'y a pas de <i>Nœuds</i> ou de <i>Références</i> ayant satisfait aux critères de <i>nodeTypes</i> . Dans ce cas, le paramètre <i>continuationPoint</i> doit être vide. Le type <i>QueryDataSet</i> est défini en 7.22.
continuationPoint	ContinuationPoint	Valeur opaque définie par le serveur qui identifie le point de continuation. Le point de continuation est utilisé seulement lorsque les résultats de <i>Query</i> sont trop volumineux pour être retournés dans une seule réponse. «Trop volumineux» signifie dans ce contexte que le <i>Serveur</i> n'est pas capable de retourner une réponse plus volumineuse ou que le nombre de <i>QueryDataSets</i> à retourner dépasse le nombre maximal de <i>QueryDataSets</i> à retourner qui a été spécifié par le <i>Client</i> dans la demande. Le point de continuation est utilisé dans le <i>Service QueryNext</i> . Lorsqu'il n'est pas utilisé, la valeur de ce paramètre est null. Si un point de continuation est retourné, le <i>Client</i> doit appeler <i>QueryNext</i> pour récupérer le prochain jeu de <i>QueryDataSets</i> ou pour libérer des ressources pour le point de continuation dans le <i>Serveur</i> . Un point de continuation doit rester actif jusqu'à ce que le <i>Client</i> passe le point de continuation à <i>QueryNext</i> ou jusqu'à ce que la session soit fermée. Si le nombre maximal de points de continuation a été atteint, le point de continuation le plus ancien doit être réinitialisé. Le type <i>ContinuationPoint</i> est décrit en 7.6.
parsingResults []	ParsingResult	Liste de résultats d'analyse pour <i>QueryFirst</i> . La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>NodeTypes</i> . Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants. Cette liste est alimentée par tous les codes de statut qui se rapportent au traitement des types de nœuds qui font partie de l'interrogation. La matrice peut être vide s'il ne s'est pas produit d'erreurs. Si un quelconque type de nœud a rencontré une erreur, tous les types de nœuds doivent avoir un code de statut associé.
statusCode	StatusCode	Résultat d'analyse pour la <i>NodeTypeDescription</i> demandée.
dataStatusCodes []	StatusCode	Liste de résultats pour <i>dataToReturn</i> . La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>dataToReturn</i> . La matrice peut être vide s'il ne s'est pas produit d'erreurs.
dataDiagnosticInfos []	DiagnosticInfo	Liste d'informations de diagnostic pour les <i>dataToReturn</i> (voir 7.8 pour la définition de <i>DiagnosticInfo</i>). La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>dataToReturn</i> . Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été rencontrée dans le traitement de la demande d'interrogation.
diagnosticInfos []	DiagnosticInfo	Liste d'informations de diagnostic pour la <i>NodeTypeDescription</i> demandée. Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été rencontrée dans le traitement de la demande d'interrogation.
filterResult	ContentFilter Résultat	Une structure qui contient toutes les éventuelles erreurs associées au filtre. Cette structure doit être vide s'il ne s'est pas produit d'erreurs. Le type <i>ContentFilterResult</i> est défini en 7.4.2.

5.9.3.2 Résultats des services

Si la *Query* est invalide ou ne peut pas être traitée, les *QueryDataSets* ne sont pas retournés et seul un résultat de *Service* est retourné, *filterResult*, *parsingResults*, ainsi qu'éventuellement les *DiagnosticInfos* facultatives. Le Tableau 45 définit les résultats de *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 45 – Codes de résultats des services QueryFirst

Id symbolique	Description
Bad_NothingToDo	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_TooManyOperations	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_ContentFilterInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_ViewIdUnknown	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_ViewTimestampInvalid	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_ViewParameterMismatchInvalid	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_ViewVersionInvalid	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_InvalidFilter	Le filtre fourni n'est pas valide, voir le filterResult pour les erreurs spécifiques.
Bad_NodeListError	Les NodeTypes fournis contiennent une erreur, voir les parsingResults pour les erreurs spécifiques.
Bad_InvalidView	La ViewDescription fournie n'est pas une ViewDescription valide.
Good_ResultsMayBeIncomplete	Il convient que le serveur ait suivi une référence à un nœud dans un serveur distant, mais il ne l'a pas fait. Le jeu de résultats peut être incomplet.

5.9.3.3 StatusCodes

Le Tableau 46 définit les valeurs pour le paramètre *statusCode* de parsingResults qui sont spécifiques à ce Service. Les *StatusCodes* communs sont définis au Tableau 166.

Tableau 46 – Codes de résultats de niveau opération de QueryFirst

Id symbolique	Description
Bad_NodeIdInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_NodeIdUnknown	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_NotTypeDefinition	Le NodeId fourni n'était pas un NodeId de définition de type.
Bad_AttributeIdInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_IndexRangeInvalid	Voir le Tableau 166 pour la description de ce code de résultat.

5.9.4 Poursuivre la demande (QueryNext)

5.9.4.1 Descriptions

Ce Service est utilisé pour demander le prochain jeu d'informations de réponse *QueryFirst* ou *QueryNext* qui sont trop volumineuses pour être envoyées dans une seule réponse. «Trop volumineuses» signifie dans ce contexte que le *Serveur* n'est pas capable de retourner une réponse plus volumineuse ou que le nombre de *QueryDataSets* à retourner dépasse le nombre maximal de *QueryDataSets* à retourner qui a été spécifié par le *Client* dans la demande d'origine. Le *QueryNext* doit être présenté dans la même session qui a été utilisée pour présenter le *QueryFirst* ou *QueryNext* qui se poursuit.

5.9.4.2 Paramètres

Le Tableau 47 définit les paramètres du Service.

Tableau 47 – Paramètres du service QueryNext

Nom	Type	Description
Request		
requestHeader	Request Header	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).
releaseContinuationPoint	Boolean	Un paramètre <i>Boolean</i> ayant les valeurs suivantes: TRUE le <i>continuationPoint</i> passé doit être réinitialisé pour libérer des ressources pour le point de continuation dans le <i>Serveur</i> . FALSE le <i>continuationPoint</i> passé doit être utilisé pour récupérer le prochain jeu de <i>QueryDataSets</i> . Un <i>Client</i> doit toujours utiliser le point de continuation retourné par une réponse <i>QueryFirst</i> ou <i>QueryNext</i> pour libérer les ressources pour le point de continuation dans le <i>Serveur</i> . Si le <i>Client</i> ne souhaite pas récupérer le prochain jeu d'informations de <i>Query</i> , <i>QueryNext</i> doit être appelé avec ce paramètre mis à TRUE. Si le paramètre est mis à TRUE, tous les paramètres de matrices dans la réponse doivent contenir des matrices vides.
continuationPoint	ContinuationPoint	Valeur opaque définie par le <i>Serveur</i> qui représente le point de continuation. La valeur pour le point de continuation a été retournée au <i>Client</i> dans une réponse <i>QueryFirst</i> ou <i>QueryNext</i> précédente. Cette valeur est utilisée pour identifier la demande <i>QueryFirst</i> ou <i>QueryNext</i> précédemment traitée qui se poursuit et le point dans le jeu de résultats à partir duquel il faut que la réponse de navigation continue. Le type <i>ContinuationPoint</i> est décrit en 7.6.
Response		
responseHeader	Response Header	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).
queryDataSets []	QueryDataSet	La matrice de <i>QueryDataSets</i> . Le type <i>QueryDataSet</i> est défini en 7.22.
revisedContinuationPoint	ContinuationPoint	Valeur opaque définie par le <i>Serveur</i> qui représente le point de continuation. Elle est utilisée seulement si les informations à retourner sont trop volumineuses pour être contenues dans une seule réponse. Lorsqu'elle n'est pas utilisée ou lorsque le <i>releaseContinuationPoint</i> est mis, la valeur de ce paramètre est null. Le type <i>ContinuationPoint</i> est décrit en 7.6.

5.9.4.3 Résultats des services

Le Tableau 48 définit les résultats des *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 48 – Codes de résultats des services QueryNext

Id symbolique	Description
Bad_ContinuationPointInvalid	Voir le Tableau 166 pour la description de ce code de résultat.

5.10 Jeu de services pour les attributs (Attribute Service Set)

5.10.1 Vue d'ensemble

Ce *Service Set* fournit les *Services* pour accéder à des *Attributs* qui sont partie intégrante des *Nœuds*.

5.10.2 Lecture (Read)

5.10.2.1 Description

Ce *Service* est utilisé pour lire un ou plusieurs *Attributs* d'un ou plusieurs *Nœuds*. Pour les valeurs d'*Attributs* construits dont les éléments ont des indices, comme une matrice, ce *Service* permet aux *Clients* de lire la totalité du jeu de valeurs à indices comme un composé, de lire des éléments individuels ou de lire des plages d'éléments du composé.

Le paramètre *maxAge* est utilisé pour ordonner au *Serveur* d'accéder à la valeur à partir de la source de données sous-jacente (telle qu'un appareil par exemple) si sa copie des données

est plus ancienne que celle que le *maxAge* spécifie. Si le *Serveur* ne peut pas satisfaire à l'âge maximal demandé, il retourne la valeur «de meilleur effort» au lieu de rejeter la demande.

5.10.2.2 Paramètres

Le Tableau 49 définit les paramètres du *Service*.

Tableau 49 – Paramètres du service Read

Nom	Type	Description
Request		
requestHeader	RequestHeader	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).
maxAge	Duration	Âge maximal de la valeur à lire en millisecondes. L'âge de la valeur est basé sur la différence entre le <i>ServeurTimestamp</i> (horodatage du serveur) et l'heure à laquelle le <i>Serveur</i> commence à traiter la demande. Par exemple, si le <i>Client</i> spécifie un <i>maxAge</i> de 500 millisecondes et la durée d'attente est de 100 millisecondes avant que le <i>Serveur</i> ne commence à traiter la demande, l'âge de la valeur retournée peut être 600 millisecondes avant l'heure à laquelle elle a été demandée. Si le <i>Serveur</i> a une ou plusieurs valeurs d'un <i>Attribut</i> qui sont dans la limite de l'âge maximal, il peut retourner n'importe laquelle de ces valeurs ou il peut lire une nouvelle valeur dans la source de données. Le nombre de valeurs d'un <i>Attribut</i> que possède un <i>Serveur</i> dépend du nombre de <i>MonitoredItems</i> qui sont définis pour cet <i>Attribut</i>. Dans tous les cas, le <i>Client</i> ne peut présumer de la copie des données qui sera retournée. Si le <i>Serveur</i> n'a pas de valeur s'inscrivant dans la limite de l'âge maximal, il doit tenter de lire une nouvelle valeur dans la source de données. Si le <i>Serveur</i> ne peut pas satisfaire au <i>maxAge</i> demandé, il retourne sa valeur de "meilleur effort" au lieu de rejeter la demande. Ce cas de figure peut se produire lorsque le temps mis par le <i>Serveur</i> pour traiter et retourner la nouvelle valeur de donnée après accès est supérieur à l'âge maximal spécifié. Si le <i>maxAge</i> est mis à 0, le <i>Serveur</i> doit tenter de lire une nouvelle valeur dans la source de données. Si <i>maxAge</i> est mis à une valeur supérieure ou égale à la valeur Int32 maximale, le <i>Serveur</i> doit tenter de récupérer une valeur mise en cache. Les valeurs négatives ne sont pas valides pour <i>maxAge</i>.
timestampsToReturn	Enum TimestampsToReturn	Une énumération qui spécifie les <i>Timestamps</i> (Horodatages) à retourner pour chaque <i>Variable Value Attribute</i> . L'énumération <i>TimestampsToReturn</i> est définie en 7.34.
nodesToRead []	ReadValueId	Liste des <i>Nœuds</i> et de leurs <i>Attributs</i> à lire. Pour chaque entrée dans la liste, un <i>StatusCode</i> est retourné, et s'il indique un succès, l' <i>Attribut Value</i> est également retourné. Le type de paramètre <i>ReadValueId</i> est défini en 7.23.
Response		
responseHeader	ResponseHeader	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).
results []	DataValue	Liste de valeurs d' <i>Attribut</i> (voir 7.7 pour la définition <i>DataValue</i>). La taille et l'ordre de cette liste correspondent à la taille et à l'ordre du paramètre de demande <i>nodesToRead</i> . Il y a une entrée dans cette liste pour chaque <i>Nœud</i> contenu dans le paramètre <i>nodesToRead</i> .
diagnosticInfos []	DiagnosticInfo	Liste d'informations de diagnostic (voir 7.8 pour la définition de <i>DiagnosticInfo</i>). La taille et l'ordre de cette liste correspondent à la taille et à l'ordre du paramètre de demande <i>nodesToRead</i> . Il y a une entrée dans cette liste pour chaque <i>Nœud</i> contenu dans le paramètre <i>nodesToRead</i> . Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été rencontrée dans le traitement de la demande.

5.10.2.3 Résultats des services

Le Tableau 50 définit les résultats des *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 50 – Codes de résultats des services Read

Id symbolique	Description
Bad_NothingToDo	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_TooManyOperations	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_MaxAgeInvalid	Le paramètre âge maximal n'est pas valide.
Bad_TimestampsToReturnInvalid	Voir le Tableau 165 pour la description de ce code de résultat.

5.10.2.4 StatusCodes

Le Tableau 51 définit les valeurs pour le *statusCode* de niveau opération contenu dans la structure *DataValue* de chaque élément *values*. Les *StatusCodes* communs sont définis au Tableau 166.

Tableau 51 – Codes de résultats de niveau opération de Read

Id symbolique	Description
Bad_NodeIdInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_NodeIdUnknown	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_AttributeIdInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_IndexRangeInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_IndexRangeNoData	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_DataEncodingInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_DataEncodingUnsupported	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_NotReadable	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_UserAccessDenied	Voir le Tableau 165 pour la description de ce code de résultat.

5.10.3 Lecture de l'historique (HistoryRead)

5.10.3.1 Description

Ce *Service* est utilisé pour lire des valeurs ou *Événements* historiques d'un ou plusieurs *Nœuds*. Pour les valeurs d'*Attributs* construits dont les éléments ont des indices, comme une matrice, ce *Service* permet aux *Clients* de lire la totalité du jeu de valeurs à indices comme un composé, de lire des éléments individuels ou de lire des plages d'éléments du composé. Les *Serveurs* peuvent rendre les valeurs historiques disponibles aux *Clients* en utilisant ce *Service*, bien que les valeurs historiques elles-mêmes ne soient pas visibles dans l'*Espace d'Adresses*.

L'*Attribute AccessLevel* (Attribut Niveau d'accès) défini dans l'IEC 62541-3 indique la prise en charge des valeurs historiques par des *Nœuds*. Plusieurs paramètres de demande indiquent comment il faut que le *Serveur* accède à des valeurs de la source sous-jacente des données de l'historique. L'*Attribute EventNotifier* (Attribut Notificateur d'événement) défini dans l'IEC 62541-3 indique la prise en charge d'*Événements* historiques par des *Nœuds*.

Le paramètre *continuationPoint* dans l'*HistoryRead* est utilisé pour marquer le point à partir duquel poursuivre la lecture si toutes les valeurs n'ont pas pu être retournées dans une seule réponse. La valeur n'est pas connue du *Client* et elle est utilisée seulement pour maintenir les informations d'état pour le point de continuation pour le *Serveur*. Un *Serveur* peut utiliser l'horodatage du dernier élément de données retourné si l'horodatage est unique. Cela réduit la nécessité de stocker des informations d'état dans le *Serveur* pour le point de continuation.

Pour des détails supplémentaires relatifs à la lecture de données historiques et d'*Événements* historiques, voir l'IEC 62541-11.

5.10.3.2 Paramètres

Le Tableau 52 définit les paramètres du *Service*.

Tableau 52 – Paramètres du service HistoryRead

Nom	Type	Description
Request		
requestHeader	RequestHeader	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).
historyReadDetails	Extensible Parameter HistoryReadDetails	Les détails définissent les types de lectures d'historiques qui peuvent être effectuées. Le type de paramètre <i>HistoryReadDetails</i> est un type de paramètre extensible qui est défini formellement dans l'IEC 62541-11. Le type <i>ExtensibleParameter</i> est défini en 7.11.
timestampsToReturn	Enum TimestampsToReturn	Une énumération qui spécifie les horodatages à retourner pour chaque <i>Variable Value Attribute</i> . L'énumération <i>TimestampsToReturn</i> est définie en 7.34. Il n'est pas valide de spécifier un <i>TimestampsToReturn</i> de NEITHER. Dans ce cas, le <i>Serveur</i> doit retourner un <i>StatusCode Bad_InvalidTimestampArgument</i> . L'IEC 62541-11 définit les exceptions pour lesquelles ce paramètre doit être ignoré.
releaseContinuation Points	Boolean	Un paramètre <i>Boolean</i> ayant les valeurs suivantes: TRUE les <i>continuationPoints</i> passés doivent être réinitialisés pour libérer des ressources dans le <i>Serveur</i> . FALSE les <i>continuationPoints</i> passés doivent être utilisés pour récupérer le prochain jeu d'informations historiques. Un <i>Client</i> doit toujours utiliser le point de continuation retourné par une réponse <i>HistoryRead</i> pour libérer les ressources pour le point de continuation dans le <i>Serveur</i> . Si le <i>Client</i> ne souhaite pas récupérer le prochain jeu d'informations historiques, <i>HistoryRead</i> doit être appelé avec ce paramètre mis à TRUE.
nodesToRead []	HistoryReadValuelid	Ce paramètre contient la liste d'éléments sur lesquels la récupération d'historique est à accomplir. Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.
nodeId	NodeId	Si l' <i>HistoryReadDetails</i> est RAW, PROCESSED, MODIFIED ou ATTIME: Le <i>nodeId</i> des <i>Nœuds</i> dont les valeurs historiques sont à lire. La valeur retournée doit toujours comprendre un horodatage. Si l' <i>HistoryReadDetails</i> est EVENTS: Le <i>NodeId</i> du <i>Nœud</i> dont l'historique des <i>Événements</i> est à lire. Si le <i>Nœud</i> ne prend pas en charge l'accès demandé pour des valeurs historiques ou des <i>Événements</i> historiques, une réponse d'erreur appropriée pour le <i>Nœud</i> donné doit être générée.
indexRange	NumericRange	Ce paramètre est utilisé pour identifier un élément simple d'une matrice ou une plage simple d'indices pour les matrices. Si une plage d'éléments est spécifiée, les valeurs sont retournées sous forme de composé. Le premier élément est identifié par l'indice 0 (zéro). Le type <i>NumericRange</i> est défini en 7.21. Ce paramètre est null si la valeur n'est pas une matrice. Cependant, si la valeur est une matrice, et ce paramètre est donc null, tous les éléments sont à inclure dans la plage.
dataEncoding	QualifiedName	Un <i>QualifiedName</i> qui spécifie le codage de données à retourner pour la <i>Value</i> à lire (voir 7.23 pour la définition de la façon de spécifier le codage de données). Le paramètre est ignoré lors de la lecture de l'historique d' <i>Événements</i> .
continuationPoint	ByteString	Pour chaque <i>NodesToRead</i> , ce paramètre spécifie un point de continuation retourné à partir d'un précédent appel <i>HistoryRead</i> , permettant au <i>Client</i> de poursuivre cette lecture à partir de la dernière valeur reçue. L' <i>HistoryRead</i> est utilisé pour sélectionner une séquence ordonnée de valeurs ou événements historiques. Un point de continuation marque un point dans cette séquence ordonnée et, ainsi, le <i>Serveur</i> retourne le sous-ensemble de la séquence qui suit le point en question. Une valeur null indique que ce paramètre n'est pas utilisé. Ce point de continuation est décrit plus en détail dans l'IEC 62541-11.
Response		
responseHeader	ResponseHeader	Paramètres communs aux réponses (voir 7.27 pour le type <i>ResponseHeader</i>).
results []	HistoryReadResult	Liste de résultats lus. La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>nodesToRead</i> . Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.
statusCode	StatusCode	<i>StatusCode</i> pour l'élément <i>NodesToRead</i> (voir 7.33 pour la définition de <i>StatusCode</i>).
continuationPoint	ByteString	Ce paramètre est utilisé seulement si le nombre des valeurs à retourner est trop grand pour que celles-ci soient retournées dans une seule réponse. Lorsque ce paramètre n'est pas utilisé, sa valeur est null. Les <i>Serveurs</i> doivent prendre en charge au moins un point de continuation par <i>Session</i> . Les <i>Serveurs</i> spécifient un nombre maximal de points de continuation d'historiques par <i>Session</i> dans l' <i>Object</i> fonctions du <i>Serveur</i> défini dans l'IEC 62541-5. Un point de continuation doit rester actif jusqu'à ce que le <i>Client</i> passe le point de continuation à <i>HistoryRead</i> ou jusqu'à ce que la <i>Session</i> soit fermée. Si le nombre maximal de points de continuation a été atteint, le point

Nom	Type	Description
Request		
		de continuation le plus ancien doit être réinitialisé.
historyData	Extensible Parameter HistoryData	Les données d'historique retournées par le <i>Nœud</i> . Le type de paramètre <i>HistoryData</i> est un type de paramètre extensible qui est défini formellement dans l'IEC 62541-11. Il spécifie les types des données d'historique qui peuvent être retournées. Le type de base <i>ExtensibleParameter</i> est défini en 7.11.
diagnosticInfos []	Diagnostic Info	Liste d'informations de diagnostic. La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>nodesToRead</i> . Il y a une entrée dans cette liste pour chaque <i>Nœud</i> contenu dans le paramètre <i>nodesToRead</i> . Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été rencontrée dans le traitement de la demande.

5.10.3.3 Résultats des services

Le Tableau 53 définit les résultats de *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 53 – Codes de résultats des services HistoryRead

Id symbolique	Description
Bad_NothingToDo	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_TooManyOperations	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_TimestampsToReturnInvalid	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_HistoryOperationInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_HistoryOperationUnsupported	Voir le Tableau 166 pour la description de ce code de résultat. L'opération demandée sur l'historique n'est pas prise en charge par le <i>Serveur</i> .

5.10.3.4 StatusCodes

Le Tableau 54 définit les valeurs pour le paramètre *statusCode* de niveau opération qui sont spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 166. Les *StatusCodes* spécifiques à l'accès aux historiques sont définis dans l'IEC 62541-11.

Tableau 54 – Codes de résultats de niveau opération de HistoryRead

Id symbolique	Description
Bad_NodeIdInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_NodeIdUnknown	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_DataEncodingInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_DataEncodingUnsupported	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_UserAccessDenied	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_ContinuationPointInvalid	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_InvalidTimestampArgument	L'horodatage défini à retourner n'était pas valide.
Bad_HistoryOperationUnsupported	Voir le Tableau 166 pour la description de ce code de résultat. L'opération demandée sur l'historique n'est pas prise en charge pour le <i>Nœud</i> demandé.

5.10.4 Écrire (Write)

5.10.4.1 Description

Ce *Service* est utilisé pour écrire des valeurs dans un ou plusieurs *Attributs* d'un ou plusieurs *Nœuds*. Pour les valeurs d'*Attributs* construits dont les éléments ont des indices, comme une matrice, ce *Service* permet aux *Clients* d'écrire la totalité du jeu de valeurs à indices comme un composé, d'écrire des éléments individuels ou d'écrire des plages d'éléments du composé.

Les valeurs sont écrites dans la source de données (telle qu'un appareil par exemple) et le *Service* ne retourne rien jusqu'à ce qu'il écrive les valeurs ou détermine que la valeur ne peut pas être écrite. Dans certains cas, le *Serveur* écrit avec succès dans un système ou *Serveur* intermédiaire et ne sait pas si la source de données a été correctement mise à jour. Dans ces cas, il convient que le *Serveur* rapporte un code de succès qui indique que l'écriture n'a pas

été vérifiée. Dans les cas où le *Serveur* est à même de vérifier qu'il a réussi à écrire dans la source de données, il rapporte un succès inconditionnel.

L'ordre dans lequel les opérations sont traitées dans le *Serveur* n'est pas défini et dépend des différentes sources de données et de la logique interne du *Serveur*. Si une combinaison *Attribut* et *Nœud* est contenue dans plus d'une opération, l'ordre de traitement n'est pas défini. Si un *Client* exige un traitement séquentiel, le *Client* a besoin d'appels de *Services* séparés.

Il est possible que le *Serveur* puisse réussir à écrire certains *Attributs* et pas d'autres. Le retour en arrière est du ressort du *Client*.

Si un *Serveur* autorise l'écriture d'*Attributs* ayant le *DataType* LocalizedText, le *Client* peut ajouter ou écraser le texte pour un paramètre de lieu en écrivant le texte avec le *LocaleId* associé. Le fait d'écrire une *String* null pour le texte pour un paramètre de lieu doit supprimer la *String* pour le paramètre de lieu en question. Le fait d'écrire une *String* null pour le *paramètre de lieu* et une *String* non null pour le *texte* met le texte à un paramètre de lieu invariable. Le fait d'écrire une *String* null pour le *texte* et une *String* non null pour le *paramètre de lieu* doit supprimer les entrées pour tous les paramètres de lieu. Si un *Client* tente d'écrire un *paramètre de lieu* qui est soit non valide syntaxiquement, soit non pris en charge, le *Serveur* renvoie Bad_LocaleNotSupported.

5.10.4.2 Paramètres

Le Tableau 55 définit les paramètres du *Service*.

Tableau 55 – Paramètres du service Write

Nom	Type	Description
Request		
requestHeader	RequestHeader	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).
nodesToWrite []	WriteValue	Liste des <i>Nœuds</i> et de leurs <i>Attributs</i> à écrire. Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.
nodeId	NodeId	<i>NodeId</i> du <i>Nœud</i> qui contient les <i>Attributs</i> .
AttributeId	IntegerId	Identificateur de l' <i>Attribut</i> . Il doit s'agir d'un identificateur d' <i>Attribut</i> valide. L' <i>IntegerId</i> est défini en 7.13. Les <i>IntegerIds</i> pour les <i>Attributs</i> sont définis dans l'IEC 62541-6.
indexRange	NumericRange	Ce paramètre est utilisé pour identifier un élément simple d'une matrice ou une plage simple d'indices pour les matrices. Le premier élément est identifié par l'indice 0 (zéro). Le type <i>NumericRange</i> est défini en 7.21. Ce paramètre n'est pas utilisé si l' <i>Attribut</i> spécifié n'est pas une matrice. Cependant, si l' <i>Attribut</i> spécifié est une matrice et que ce paramètre n'est pas utilisé, tous les éléments sont à inclure dans la plage. Ce paramètre est null s'il n'est pas utilisé. Un <i>Serveur</i> doit retourner une erreur <i>Bad_WriteNotSupported</i> si une <i>indexRange</i> est fournie et si l'écriture d' <i>indexRange</i> n'est pas possible pour le <i>Nœud</i> .
value	DataValue	La valeur d' <i>Attribut</i> du <i>Nœud</i> (voir 7.7 pour la définition de <i>DataValue</i>). Si le paramètre <i>indexRange</i> est spécifié, la <i>Value</i> doit être une matrice même si un seul élément est en cours d'écriture. Si le <i>SourceTimestamp</i> ou le <i>ServeurTimestamp</i> est spécifié, le <i>Serveur</i> doit utiliser ces valeurs. Le <i>Serveur</i> retourne une erreur <i>Bad_WriteNotSupported</i> s'il ne prend pas en charge l'écriture d'horodatages. Un <i>Serveur</i> doit retourner une erreur <i>Bad_TypeMismatch</i> si le type de données de la valeur écrite n'est pas le même que le type ou le sous-type du <i>DataType</i> de l' <i>Attribut</i> . Sur la base de la hiérarchie du <i>DataType</i> , les sous-types du <i>DataType</i> de l' <i>Attribut</i> doivent être acceptés par le <i>Serveur</i> . Pour le <i>Value Attribute</i> , le <i>DataType</i> est défini au moyen de l' <i>Attribut</i> du <i>DataType</i> . La structure d'une <i>ByteString</i> est la même que celle d'une matrice à une dimension de <i>Byte</i> . Un <i>Serveur</i> doit accepter une <i>ByteString</i> si une matrice de <i>Byte</i> est prévue. Le <i>Serveur</i> retourne une erreur <i>Bad_DataEncodingUnsupported</i> s'il ne prend pas en charge le codage de données fourni. Les <i>Simple DataTypes</i> (voir l'IEC 62541-3) utilisent la même représentation à la volée que leurs supertypes et écrire une valeur d'un <i>DataType</i> simple ne peut être distingué du fait d'écrire une valeur de son supertype. Le <i>Serveur</i> doit supposer qu'en recevant la représentation correcte à la volée pour un <i>DataType</i> simple, le type correct a été choisi. Les <i>Serveurs</i> peuvent imposer des validations supplémentaires de données sur la valeur indépendante du codage (par exemple, une image au format GIF dans une <i>ByteString</i>). Dans ce cas, le <i>Serveur</i> doit retourner une erreur <i>Bad_TypeMismatch</i> si la validation échoue.
Response		
responseHeader	ResponseHeader	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).
results []	StatusCode	Liste des résultats pour les <i>Nœuds</i> à écrire (voir 7.33 pour la définition des <i>StatusCodes</i>). La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>nodesToWrite</i> . Il y a une entrée dans cette liste pour chaque <i>Nœud</i> contenu dans le paramètre <i>nodesToWrite</i> .
diagnosticInfos []	DiagnosticInfo	Liste d'informations de diagnostic pour les <i>Nœuds</i> à écrire (voir 7.8 pour la définition de <i>DiagnosticInfo</i>). La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>nodesToWrite</i> . Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été rencontrée dans le traitement de la demande.

5.10.4.3 Résultats des services

Le Tableau 56 définit les résultats des *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 56 – Codes de résultats des services Write

Id symbolique	Description
Bad_NothingToDo	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_TooManyOperations	Voir le Tableau 165 pour la description de ce code de résultat.

5.10.4.4 StatusCodes

Le Tableau 57 définit les valeurs pour le paramètre *results* qui sont spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 166.

Tableau 57 – Codes de résultats de niveau opération de Write

Id symbolique	Description
Good_CompletesAsynchronously	Voir le Tableau 165 pour la description de ce code de résultat. La valeur a été écrite avec succès dans un système intermédiaire, mais le <i>Serveur</i> ne sait pas si la source de données a été correctement mise à jour.
Bad_NodeIdInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_NodeIdUnknown	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_AttributeIdInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_IndexRangeInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_IndexRangeNoData	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_WriteNotSupported	L'opération d'écriture demandée n'est pas prise en charge. Si un <i>Client</i> tente d'écrire n'importe quelle combinaison de "value", "quality", "timestamp" alors que le <i>Serveur</i> ne prend pas en charge la combinaison demandée (qui pourrait être une seule grandeur telle que juste un horodatage), le <i>Serveur</i> ne doit pas effectuer d'écriture sur ce <i>Nœud</i> et doit retourner ce <i>StatusCode</i> pour ce <i>Nœud</i> . Il est aussi utilisé si l'écriture d'un <i>IndexRange</i> n'est pas prise en charge pour un <i>Nœud</i> .
Bad_NotWritable	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_UserAccessDenied	Voir le Tableau 165 pour la description de ce code de résultat. L'utilisateur courant n'a pas la permission d'écrire l'attribut.
Bad_OutOfRange	Voir le Tableau 166 pour la description de ce code de résultat. Si un <i>Client</i> tente d'écrire une valeur hors de la plage valide telle qu'une valeur non contenue dans le type de données d'énumération du <i>Nœud</i> , le <i>Serveur</i> doit retourner ce <i>StatusCode</i> pour ce <i>Nœud</i> .
Bad_TypeMismatch	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_DataEncodingUnsupported	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_NoCommunication	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_LocaleNotSupported	Le paramètre de lieu dans l'opération d'écriture demandée n'est pas pris en charge.

5.10.5 Mise à jour de l'historique (HistoryUpdate)

5.10.5.1 Description

Ce *Service* est utilisé pour mettre à jour des valeurs ou *Événements* historiques d'un ou plusieurs *Nœuds*. Plusieurs paramètres de demande indiquent comment il faut que le *Serveur* mette à jour une valeur ou un *Événement* historique. Les actions valides sont Insert (Insérer), Replace (Remplacer) ou Delete (Supprimer).

5.10.5.2 Paramètres

Le Tableau 58 définit les paramètres du *Service*.

Tableau 58 – Paramètres du service HistoryUpdate

Nom	Type	Description
Request		
requestHeader	RequestHeader	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).
historyUpdateDetails []	Extensible Parameter HistoryUpdate Details	Les détails définis pour la mise à jour. Le type de paramètre <i>HistoryUpdateDetails</i> est un type de paramètre extensible qui est défini formellement dans l'IEC 62541-11. Il spécifie les types des mises à jour d'historique qui peuvent être accomplies. Le type <i>ExtensibleParameter</i> est défini en 7.11.
Response		
responseHeader	ResponseHeader	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).
results []	HistoryUpdate Result	Liste des résultats de mise à jour pour les détails de mise à jour d'historiques. La taille et l'ordre de la liste correspondent à la taille et à l'ordre des éléments de détails du paramètre <i>historyUpdateDetails</i> spécifié dans la demande. Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.
statusCode	StatusCode	<i>StatusCode</i> pour la mise à jour du <i>Nœud</i> (voir 7.33 pour la définition de <i>StatusCode</i>).
operationResults []	StatusCode	Liste des <i>StatusCodes</i> pour les opérations à exécuter sur un <i>Nœud</i> . La taille et l'ordre de la liste correspondent à la taille et à l'ordre de toute liste définie par l'élément "détails" communiqué dans un rapport par l'entrée de résultat.
diagnosticInfos []	DiagnosticInfo	Liste d'informations de diagnostic pour les opérations à exécuter sur un <i>Nœud</i> (voir 7.8 pour la définition de <i>DiagnosticInfo</i>). La taille et l'ordre de la liste correspondent à la taille et à l'ordre de toute liste définie par l'élément "détails" communiqué dans un rapport par cette entrée de <i>updateResults</i> . Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été rencontrée dans le traitement de la demande.
diagnosticInfos []	DiagnosticInfo	Liste d'informations de diagnostic pour les détails de mise à jour d'historiques. La taille et l'ordre de la liste correspondent à la taille et à l'ordre des éléments de détails du paramètre <i>historyUpdateDetails</i> spécifié dans la demande. Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été rencontrée dans le traitement de la demande.

5.10.5.3 Résultats des services

Le Tableau 59 définit les résultats des *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 59 – Codes de résultats des services HistoryUpdate

Id symbolique	Description
Bad_NothingToDo	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_TooManyOperations	Voir le Tableau 165 pour la description de ce code de résultat.

5.10.5.4 StatusCodes

Le Tableau 60 définit les valeurs pour les paramètres *statusCode* et *operationResults* qui sont spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 166. Les *StatusCodes* spécifiques à l'accès aux historiques sont définis dans l'IEC 62541-11.

Tableau 60 – Codes de résultats de niveau opération de HistoryUpdate

Id symbolique	Description
Bad_NotWritable	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_HistoryOperationInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_HistoryOperationUnsupported	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_UserAccessDenied	Voir le Tableau 165 pour la description de ce code de résultat. L'utilisateur courant n'a pas la permission de mettre à jour l'historique.

5.11 Jeu de services de méthodes (Method Service Set)

5.11.1 Vue d'ensemble

Les *Méthodes* représentent les appels de fonction des *Objects*. Elles sont définies dans l'IEC 62541-3. Les *Méthodes* sont invoquées et retournent un résultat (couronné de succès ou non) seulement à la fin. Les temps d'exécution pour les méthodes peuvent varier, selon la fonction qu'elles accomplissent.

Le *Method Service Set* (Jeu de services de méthodes) définit les moyens pour invoquer des méthodes. Une *méthode* doit être un *composant* d'un *Object*. La découverte est fournie par le biais des *Services Browse* (Navigation) et *Query* (Interrogation). Les *Clients* découvrent les *méthodes* prises en charge par un *Serveur* en naviguant à la recherche des *Références* d'*Objects* propriétaires qui identifient leurs *méthodes* prises en charge.

Parce que des *Méthodes* peuvent commander un certain aspect des opérations d'une installation, l'invocation de méthodes peut dépendre des conditions d'environnement ou autres. Cela peut être particulièrement vrai lors d'une tentative d'invocation d'une méthode immédiatement après que celle-ci a parachevé son exécution. Les conditions indispensables pour invoquer la *Méthode* pourraient ne pas être encore retournées à l'état qui permet à la *Méthode* de redémarrer.

5.11.2 Appel (Call)

5.11.2.1 Description

Ce *Service* est utilisé pour appeler (invoquer) une liste de *Méthodes*. Chaque appel de *Méthode* est invoqué dans le contexte d'une *Session* existante. S'il est mis fin à cette *Session*, les résultats de l'exécution de la *Méthode* ne peuvent pas être retournés au *Client* et sont éliminés. Cela est indépendant de la tâche réellement exécutée au niveau du *Serveur*.

L'ordre dans lequel les opérations sont traitées dans le *Serveur* n'est pas défini et dépend des différentes tâches et de la logique interne du *Serveur*. Si une *Méthode* est contenue dans plusieurs opérations, l'ordre de traitement n'est pas défini. Si un *Client* exige un traitement séquentiel, le *Client* a besoin d'appels de *Services* séparés.

Ce *Service* permet le passage d'arguments d'entrée et de sortie en direction/provenance de la méthode. Ces arguments sont définis par les *Properties* (Propriétés) de la *Méthode*.

5.11.2.2 Paramètres

Le Tableau 61 définit les paramètres du *Service*.

Tableau 61 – Paramètres du service Call

Nom	Type	Description
Request		
requestHeader	RequestHeader	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).
methodsToCall []	CallMethodRequest	Liste des <i>Méthodes</i> à appeler. Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.
objectId	NodeId	Le <i>NodeId</i> doit être celui de l' <i>Object</i> ou <i>ObjectType</i> qui est la source d'une <i>Référence HasComponent</i> (ou sous-type de <i>Référence HasComponent</i>) à la <i>Méthode</i> spécifiée en <i>methodId</i> . Voir l'IEC 62541-3 pour une description des <i>Objects</i> et de leurs <i>Méthodes</i> .
methodId	NodeId	<i>NodeId</i> de la <i>Méthode</i> à invoquer. Si l' <i>objectId</i> est le <i>NodeId</i> d'un <i>Object</i> , il peut utiliser le <i>NodeId</i> d'une <i>Méthode</i> qui est la cible d'une <i>Référence HasComponent</i> de l' <i>ObjectType</i> de l' <i>Object</i> .
inputArguments []	BaseDataType	Liste de valeurs des arguments d'entrée. Une liste vide indique qu'il n'y a pas d'arguments d'entrée. La taille et l'ordre de cette liste correspondent à la taille et à l'ordre des arguments d'entrée définis par la <i>Property InputArguments</i> de la <i>Méthode</i> . Le nom, une description et le type de données de chaque argument sont définis par la structure <i>Argument</i> dans chaque élément de la <i>Property InputArguments</i> de la méthode.
Response		
responseHeader	ResponseHeader	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).
results []	CallMethodResult	Résultat pour les appels de la <i>Méthode</i> . Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.
statusCode	StatusCode	<i>StatusCode</i> de la <i>Méthode</i> exécutée dans le serveur. Ce <i>StatusCode</i> est mis au <i>StatusCode</i> Bad_InvalidArgument si au moins un argument d'entrée a enfreint une contrainte (par exemple type de données erroné, valeur hors plage). Ce <i>StatusCode</i> est mis à bad <i>StatusCode</i> si l'exécution de la <i>Méthode</i> a échoué dans le <i>Serveur</i> , par exemple sur la base d'une exception.
inputArgumentResults []	StatusCode	Liste de <i>StatusCodes</i> correspondant aux <i>inputArguments</i> . Cette liste est vide sauf si le résultat de niveau opération est Bad_InvalidArgument Si cette liste est remplie, elle a la même longueur que la liste <i>inputArgument</i>
inputArgumentDiagnosticInfos []	DiagnosticInfo	Liste d'informations de diagnostic correspondant aux <i>inputArguments</i> . Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été rencontrée dans le traitement de la demande.
outputArguments []	BaseDataType	Liste de valeurs des arguments de sortie. Une liste vide indique qu'il n'y a pas d'arguments de sortie. La taille et l'ordre de cette liste correspondent à la taille et à l'ordre des arguments de sortie définis par la <i>Property OutputArguments</i> de la <i>Méthode</i> . Le nom, une description et le type de données de chaque argument sont définis par la structure <i>Argument</i> dans chaque élément de la <i>Property OutputArguments</i> de la méthode
diagnosticInfos []	DiagnosticInfo	Liste d'informations de diagnostic pour le <i>statusCode</i> des <i>results</i> . Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été rencontrée dans le traitement de la demande.

5.11.2.3 Résultats des services

Le Tableau 62 définit les résultats des *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 62 – Codes de résultats des services Call

Id symbolique	Description
Bad_NothingToDo	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_TooManyOperations	Voir le Tableau 165 pour la description de ce code de résultat.

5.11.2.4 StatusCodes

Le Tableau 63 définit les valeurs pour les paramètres *statusCode* et *inputArgumentResults* qui sont spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 166.

Tableau 63 – Codes de résultats de niveau opération de Call

Id symbolique	Description
Bad_NoIdInvalid	Voir le Tableau 166 pour la description de ce code de résultat. Sert à indiquer que l'objet spécifié n'est pas valide.
Bad_NoIdUnknown	Voir le Tableau 166 pour la description de ce code de résultat. Sert à indiquer que l'objet spécifié n'est pas valide.
Bad_ArgumentsMissing	Le client n'a pas spécifié la totalité des arguments d'entrée pour la méthode.
Bad_TooManyArguments	Le client a spécifié plus d'arguments d'entrée que le nombre défini pour la méthode.
Bad_InvalidArgument	Voir le Tableau 166 pour la description de ce code de résultat. Sert à indiquer dans les résultats de niveau opération qu'un ou plusieurs des arguments d'entrée ne sont pas valides. Les <i>inputArgumentResults</i> contiennent le code de statut spécifique pour chaque argument non valide.
Bad_UserAccessDenied	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_MethodInvalid	L'identificateur de la méthode ne se réfère pas à une méthode pour l'objet spécifié.
Bad_OutOfRange	Voir le Tableau 166 pour la description de ce code de résultat. Sert à indiquer qu'un argument d'entrée se trouve à l'extérieur de la plage acceptable.
Bad_TypeMismatch	Voir le Tableau 166 pour la description de ce code de résultat. Sert à indiquer qu'un argument d'entrée n'a pas le type de données correct. La structure d'une <i>ByteString</i> est la même que celle d'une matrice à une dimension de <i>Byte</i> . Un <i>Serveur</i> doit accepter une <i>ByteString</i> si une matrice de <i>Byte</i> est prévue.
Bad_NoCommunication	Voir le Tableau 166 pour la description de ce code de résultat.

5.12 Jeu de services des éléments surveillés (MonitoredItem Service Set)

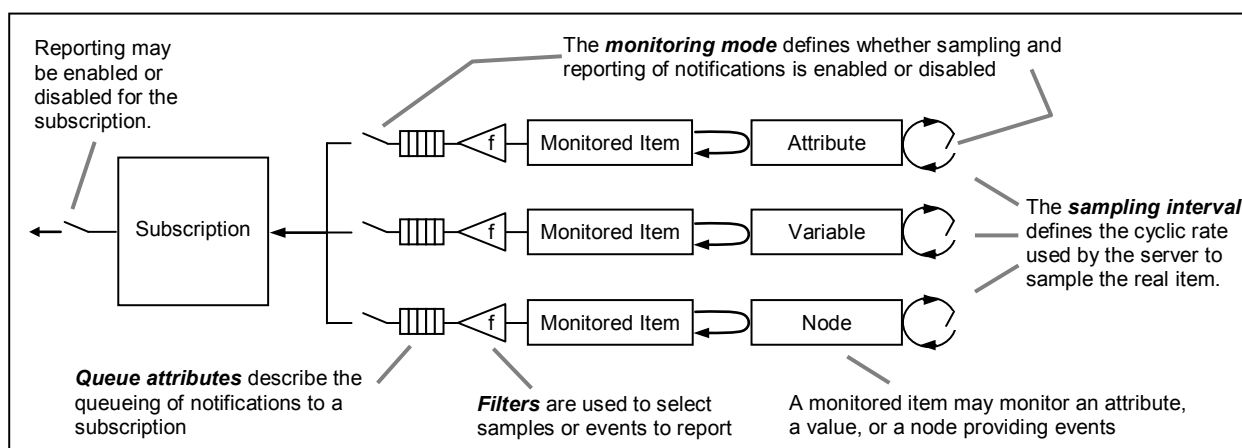
5.12.1 Modèle pour les éléments surveillés (MonitoredItem Model)

5.12.1.1 Vue d'ensemble

Les *Clients* définissent des *MonitoredItems* pour s'abonner à des données et à des *Événements*. Chaque *MonitoredItem* identifie l'élément à surveiller et le *Subscription* à utiliser pour envoyer des *Notifications*. L'élément à surveiller peut être n'importe quel *Node Attribute*.

Les *Notifications* sont des structures de données qui décrivent l'apparition des modifications de données et d'*Événements*. Elles sont empaquetées dans des *NotificationMessages* pour le transfert au *Client*. Le *Subscription* (abonnement) envoie périodiquement des *NotificationMessages* selon un intervalle d'édition spécifié par l'utilisateur, et le cycle durant lequel ces messages sont envoyés est appelé cycle d'édition.

Il a été défini quatre paramètres primaires pour les *MonitoredItems* qui indiquent au *Serveur* comment l'élément est à échantillonner, évaluer et rapporter. Ces paramètres sont l'intervalle d'échantillonnage, le mode de surveillance, le filtre et le paramètre de mise en file d'attente. La Figure 15 illustre ces concepts.



IEC

Anglais	Français
Reporting may be enabled or disabled for the subscription.	La production de rapports peut être activée ou désactivée pour l'abonnement.
Monitored Item	Élément surveillé
Attribute	Attribut
Variable	Variable
Node	Nœud
The <i>sampling interval</i> defines the cyclic rate used by the server to sample the real item.	L' <i>intervalle d'échantillonnage</i> définit la fréquence cyclique utilisée par le serveur pour échantillonner l'élément réel.
Subscription	Abonnement
The <i>monitoring mode</i> defines whether sampling and reporting of notifications are enabled or disabled	Le <i>mode de surveillance</i> définit si l'échantillonnage et les rapports de notifications sont activés ou désactivés
<i>Filters</i> are used to select samples or events to report	Les <i>filtres</i> sont utilisés pour sélectionner des échantillons ou des événements à consigner dans des rapports
A monitored item may monitor an attribute, a value, or a node providing events	Un élément surveillé peut surveiller un attribut, une valeur ou un nœud fournissant des événements
<i>Queue attributes</i> describe the queueing of notifications to a subscription	Les <i>attributs de file d'attente</i> décrivent la mise en file d'attente de notifications destinées à un abonnement

Figure 15 – Modèle MonitoredItem

Les *Attributs* autres que l'*Attribute Value* sont seulement surveillés pour détecter un changement de valeur. Le filtre n'est pas utilisé pour ces *Attributs*. Toute modification de valeur pour ces *Attributs* entraîne la création d'une *Notification*.

L'*Attribute Value* (Valeur Attribut) est utilisé pour surveiller des *Variables*. Les valeurs des *Variables* sont surveillées pour détecter une modification de valeur ou une modification de leur statut. Les filtres définis dans la présente Norme (voir 7.16.2) et dans l'IEC 62541-8 sont utilisés pour déterminer si la modification de la valeur est suffisamment forte pour entraîner la création d'une *Notification* pour cette *Variable*.

Les *Objects* et les vues peuvent être utilisés pour surveiller les *Événements*. Les *Événements* sont disponibles uniquement à partir des *Nœuds* pour lesquels le bit *SubscribeToEvents* de l'*Attribute EventNotifier* est positionné. Le filtre défini dans la présente Norme (voir 7.16.3) est utilisé pour déterminer si un *Événement* reçu du *Nœud* est envoyé au *Client*. Le filtre permet aussi de sélectionner des champs de l'*EventType* qui seront contenus dans l'*Événement* tels que *EventId*, *EventType*, *SourceNode*, *Time* et *Description*.

L'IEC 62541-3 décrit le modèle *Événement* et les *EventTypes* de base.

Les *Properties* des *EventTypes* de base et la représentation des *EventTypes* de base dans l'*Espace d'Adresses* sont spécifiées dans l'IEC 62541-5.

5.12.1.2 Intervalle d'échantillonnage

Chaque *MonitoredItem* créé par le *Client* reçoit un intervalle d'échantillonnage qui lui est affecté et qui est soit hérité de l'intervalle d'édition de *Subscription*, soit défini spécifiquement pour se substituer à cette fréquence. Un nombre négatif indique que l'intervalle d'échantillonnage par défaut défini par l'intervalle d'édition de *Subscription* est demandé. L'intervalle d'échantillonnage indique la fréquence la plus rapide à laquelle il convient que le *Serveur* échantillonne sa source sous-jacente pour les modifications de données.

Un *Client* doit définir un intervalle d'échantillonnage de 0 s'il s'abonne à des *Événements*.

L'intervalle d'échantillonnage affecté définit une fréquence cyclique de «meilleur effort» que le *Serveur* utilise pour échantillonner l'élément à partir de sa source. «Meilleur effort» signifie dans ce contexte que le *Serveur* fait de son mieux pour échantillonner à cette fréquence. L'échantillonnage à des fréquences plus rapides que celle-ci est acceptable, mais n'est pas indispensable pour répondre aux besoins du *Client*. La façon dont le *Serveur* traite de la fréquence d'échantillonnage et la fréquence à laquelle il sonde effectivement ses sources de données en interne sont un détail d'implémentation du *Serveur*. Cependant, le temps entre les valeurs retournées au *Client* doit être supérieur ou égal à l'intervalle d'échantillonnage.

Le *Client* peut aussi spécifier 0 pour l'intervalle d'échantillonnage, qui indique qu'il convient pour le *Serveur* d'utiliser la fréquence pratique la plus rapide. Il est escompté que les *Serveurs* prendront en charge uniquement un jeu limité d'intervalles d'échantillonnage pour optimiser leur fonctionnement. Si l'intervalle exact demandé par le *Client* n'est pas pris en charge par le *Serveur*, le *Serveur* affecte au *MonitoredItem* l'intervalle le plus approprié tel que déterminé par le *Serveur*. Il retourne au *Client* cet intervalle affecté. L'*Object Server Capabilities* (l'objet Fonctions du serveur) défini dans l'IEC 62541-5 identifie les intervalles d'échantillonnage pris en charge par le *Serveur*.

Le *Serveur* peut prendre en charge les données qui sont recueillies sur la base d'un modèle d'échantillonnage ou générées sur la base d'un modèle piloté par des exceptions. L'intervalle d'échantillonnage pris en charge le plus rapide peut être égal à 0, ceci indique que l'élément de donnée est piloté par des exceptions au lieu d'être échantillonné périodiquement. Un modèle piloté par des exceptions signifie que le système sous-jacent ne nécessite pas d'échantillonnage et rapporte les changements apportés aux données.

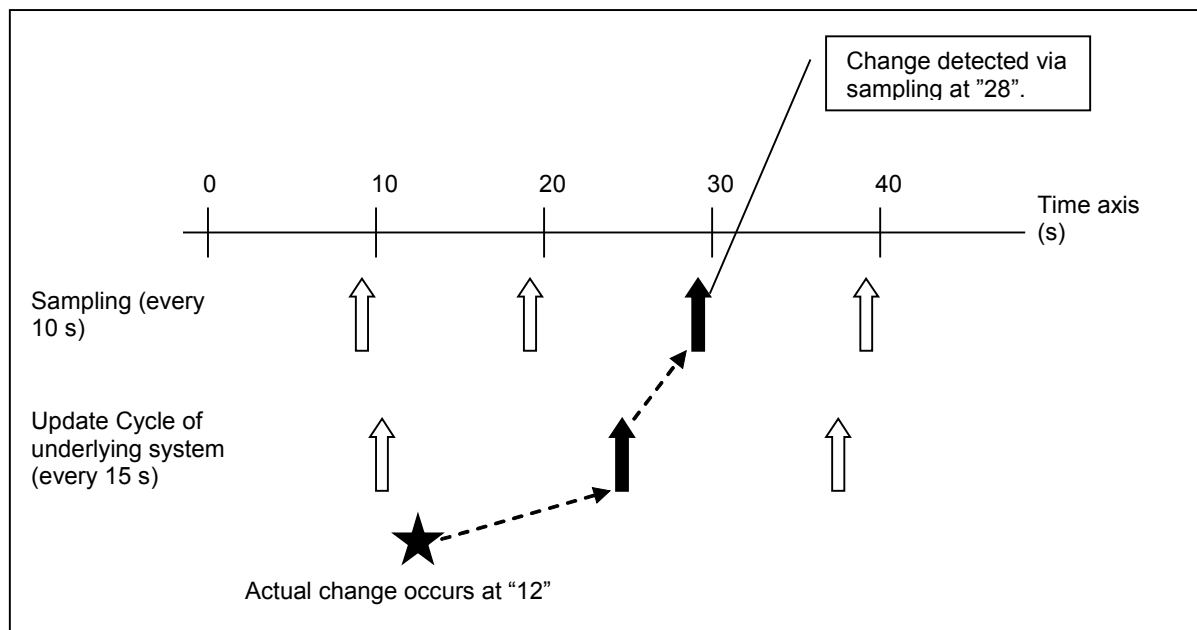
Le *Client* peut utiliser des valeurs révisées de l'intervalle d'échantillonnage comme suggestion pour l'établissement de l'intervalle d'édition ainsi que le compte d'entretien d'un *Subscription*. Si, par exemple, le plus petit intervalle d'échantillonnage révisé pour les *MonitoredItems* est 5 secondes, il convient que le temps avant qu'un entretien ne soit envoyé soit supérieur à 5 secondes.

A noter que, dans de nombreux cas, le *Serveur* OPC UA fournit l'accès à un système découplé et n'a donc aucune connaissance de la logique de mise à jour de données. Dans ce cas, même si le *Serveur* OPC UA échantillonne à la fréquence négociée, les données peuvent être mises à jour par le système sous-jacent à une fréquence beaucoup plus lente. Dans ce cas, les modifications ne peuvent être détectées qu'à cette fréquence plus lente.

Si le comportement par lequel le système sous-jacent met à jour l'élément est connu, il sera disponible via l'*Attribut MinimumSamplingInterval* défini dans l'IEC 62541-3. Si le *Serveur* spécifie une valeur pour l'*Attribut MinimumSamplingInterval*, il doit toujours retourner un *revisedSamplingInterval* qui est supérieur ou égal au *MinimumSamplingInterval* si le *Client* s'abonne à l'*Attribute Value*.

Il convient que les *Clients* sachent que l'échantillonnage par le *Serveur* OPC UA et le cycle de mise à jour du système sous-jacent ne sont pas, en général, synchronisés. Cela peut

introduire des retards supplémentaires dans la détection des modifications, comme illustré à la Figure 16.



IEC

Anglais	Français
Sampling (every 10 s)	Échantillonnage (toutes les 10 s)
Update Cycle of underlying system (every 15 s)	Cycle de mise à jour du système sous-jacent (toutes les 15 s)
Change detected via sampling at "28"	Changement détecté via l'échantillonnage à "28"
Actual change occurs at "12"	Un changement réel se produit à "12"
Time axis (s)	Axe des temps (s)

Figure 16 – Retard type pour détecter des modifications

5.12.1.3 Mode de surveillance

Le paramètre mode de surveillance est utilisé pour activer et désactiver l'échantillonnage d'un *MonitoredItem* et aussi pour permettre d'activer et de désactiver de façon indépendante la production de rapports de *Notifications*. Cette fonction permet de configurer un *MonitoredItem* pour échantillonner, échantillonner et rapporter ou ni l'un ni l'autre. La désactivation de l'échantillonnage ne change les valeurs d'aucun des autres paramètres de *MonitoredItem*, tels que son intervalle d'échantillonnage.

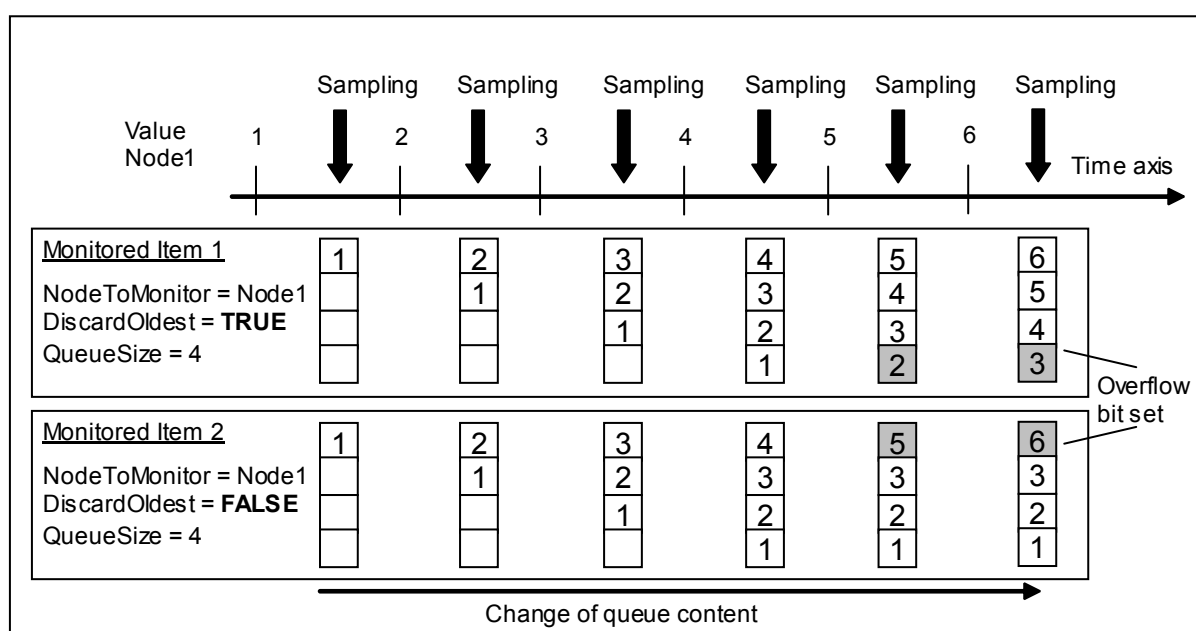
Lorsqu'un *MonitoredItem* est activé (c'est-à-dire lorsque le *MonitoringMode* est passé de *DISABLED* à *SAMPLING* ou à *REPORTING*) ou qu'il est créé dans l'état activé, le *Serveur* doit signaler le premier échantillon dès que possible dans le rapport et l'heure de cet échantillon devient le point de départ pour le prochain intervalle d'échantillonnage.

5.12.1.4 Filtre

Chaque fois qu'un *MonitoredItem* est échantillonné, le *Serveur* évalue l'échantillon en utilisant le filtre défini pour le *MonitoredItem*. Le paramètre *filter* définit les critères que le *Serveur* utilise pour déterminer s'il convient de générer une *Notification* pour l'échantillon. Le type de filtre dépend du type d'élément qui est surveillé. Par exemple, le *DataChangeFilter* et l'*AggregateFilter* sont utilisés pour surveiller les *Variable Values* (Valeurs de variables) et l'*EventFilter* est utilisé pour surveiller des *Événements*. L'échantillonnage et l'évaluation, y compris l'utilisation de filtres, sont décrits dans la présente Norme. Des filtres supplémentaires peuvent être définis dans d'autres parties de la présente série de normes.

5.12.1.5 Paramètres de mise en file d'attente

Si l'échantillon satisfait aux critères de filtrage, une *Notification* est générée et mise en file d'attente pour le transfert par *Subscription*. La taille de la file d'attente est définie lorsque *MonitoredItem* est créé. Lorsque la file d'attente est pleine et qu'une nouvelle *Notification* est reçue, le *Serveur* soit élimine la plus ancienne *Notification* et place la nouvelle dans la file d'attente, soit remplace la dernière valeur ajoutée à la file d'attente par la nouvelle valeur. À sa création, le *MonitoredItem* est configuré pour l'une de ces politiques de rejet. Si une *Notification* est rejetée pour une *DataValue* et que la taille de la file d'attente est supérieure à un, le bit *Overflow* (fanion) dans la partie *InfoBits* du *statusCode* de la *DataValue* est établi. Si *discardOldest* est TRUE, la valeur la plus ancienne est supprimée de la file d'attente et la valeur suivante dans la file d'attente établit le fanion. Si *discardOldest* est FALSE, la dernière valeur ajoutée à la file d'attente est remplacée par la nouvelle valeur. La nouvelle valeur établit le fanion pour indiquer les valeurs perdues dans le *NotificationMessage* suivant. La Figure 17 illustre la gestion du dépassement de capacité de la file d'attente.



IEC

Anglais	Français
Sampling	Échantillonnage
Time axis	Axe des temps
Monitored item	Élément surveillé
Change of queue content	Modification du contenu de la file d'attente
Overflow bit set	Établissement du bit de dépassement de capacité

Figure 17 – Gestion du dépassement de capacité de la file d'attente

Si la taille de la file d'attente est 1, la file d'attente devient un tampon qui contient en permanence la *Notification* la plus récente. Dans ce cas, si l'intervalle d'échantillonnage du *MonitoredItem* est plus court que l'intervalle d'édition de *Subscription*, le *MonitoredItem* sera en suréchantillonnage et le *Client* recevra toujours la valeur la plus récente. La politique de rejet est ignorée si la taille de la file d'attente est un.

Par ailleurs, le *Client* peut souhaiter s'abonner à un flux continu de *Notifications* sans le moindre trou, mais ne souhaite pas que celles-ci soient rapportées à l'intervalle d'échantillonnage. Dans ce cas, le *MonitoredItem* met en place une file d'attente suffisamment longue pour contenir toutes les *Notifications* générées entre deux cycles d'édition consécutifs. Puis, à chaque cycle d'édition, *Subscription* envoie au *Client* toutes les *Notifications* mises en

file d'attente pour le *MonitoredItem*. Le *Serveur* doit retourner les *Notifications* pour tout élément particulier dans l'ordre dans lequel elles sont stockées dans la file d'attente.

Le *Serveur* peut échantillonner à une fréquence plus rapide que l'intervalle d'échantillonnage pour prendre en charge d'autres *Clients*; il convient que le *Client* attende seulement les valeurs à l'intervalle d'échantillonnage négocié. Le *Serveur* peut délivrer moins de valeurs que ne le dicte l'intervalle d'échantillonnage, sur la base des contraintes du filtre et de l'implémentation. Si un *DataChangeFilter* est configuré pour un *MonitoredItem*, il est toujours appliqué à la valeur la plus récente dans la file d'attente en comparaison à l'échantillon courant.

Si, par exemple, l'*AbsoluteDeadband* dans le *DataChangeFilter* est «10», la file d'attente peut être constituée de valeurs dans l'ordre suivant:

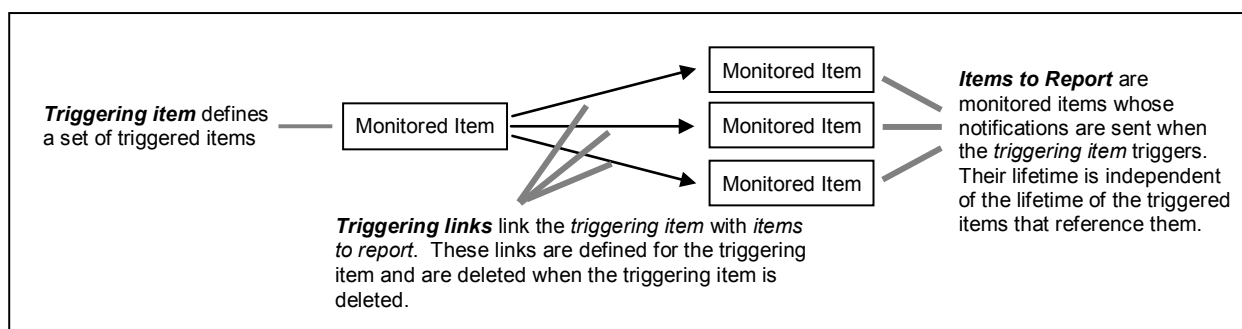
- 100
- 111
- 100
- 89
- 100

La mise des données en file d'attente peut conduire à un comportement inattendu lorsqu'un filtre *Deadband* est utilisé et que le nombre de modifications rencontrées est supérieur au nombre de valeurs qui peuvent être maintenues. La première nouvelle valeur dans la file d'attente peut ne pas dépasser la limite de *Deadband* de la valeur précédente envoyée au *Client*.

La taille de la file d'attente est la valeur maximale prise en charge par le *Serveur* lors de la surveillance d'*Événements*. Dans ce cas, le *Serveur* a la responsabilité du tampon d'*Événements*. Si des *Événements* sont perdus, un *Événement* du type *EventQueueOverflowEventType* est placé dans la file d'attente. Cet *Événement* est généré lorsque le premier *Événement* est à éliminer sur un *MonitoredItem* s'abonnant à des *Événements*. Il est mis dans la File d'attente du *MonitoredItem* en plus de la taille de la File d'attente définie pour ce *MonitoredItem* sans rejeter aucun autre *Événement*. Si *discardOldest* est mis à TRUE, il est placé au début de la file d'attente et n'est jamais rejeté, autrement à la fin. Un *Serveur* combiné ne doit pas transmettre un tel *Événement*. Il doit être géré comme les autres scénarios d'erreur de connexion.

5.12.1.6 Modèle de déclenchement

Le *Service MonitoredItems* permet d'ajouter des éléments qui sont signalés dans un rapport seulement lorsqu'un autre élément est déclenché (l'élément de déclenchement). Cela est réalisé en créant des liens entre les éléments déclenchés et les éléments à rapporter. Le mode de surveillance des éléments à rapporter est mis en échantillonnage seul et, de ce fait, il échantillonnera et mettra les *Notifications* en file d'attente sans les signaler dans un rapport. La Figure 18 illustre ce concept.



IEC

Anglais	Français
Triggering item defines a set of triggered items	Le Triggering item (élément déclencheur) définit un jeu d'éléments déclenchés
Monitored item	Élément surveillé
Triggering links link the <i>triggering item</i> with <i>items to report</i> . These links are defined for the triggering item and are deleted when the triggering item is deleted.	Les Triggering links (liaisons de déclenchement) relient le <i>triggering item</i> à des <i>items to report</i> (<i>éléments à rapporter</i>). Ces liaisons sont définies pour l'élément déclencheur et sont supprimées lorsque l'élément déclencheur est supprimé
Items to report are monitored items whose notifications are sent when the <i>triggering item</i> triggers. Their lifetime is independent of the lifetime of the triggered items that reference them	Les items to report (éléments à rapporter) sont des éléments surveillés dont les notifications sont envoyées lorsque l' <i>élément déclencheur</i> déclenche. Leur durée de vie est indépendante de la durée de vie des éléments déclenchés qui les référencent

Figure 18 – Modèle de déclenchement

Le mécanisme de déclenchement est une caractéristique utile qui permet aux *Clients* de réduire le volume de données à la volée en configurant certains éléments pour qu'ils échantillonnent fréquemment, mais se limitent seulement à produire un rapport lorsqu'un autre *Événement* se produit.

Les comportements de déclenchement suivants sont spécifiés.

- Si le mode monitoring (de surveillance) de l'élément déclencheur est *SAMPLING*, alors, lorsque l'élément déclencheur déclenche les éléments à rapporter, cela ne fait pas l'objet d'un rapport.
- Si le mode monitoring (de surveillance) de l'élément déclencheur est *REPORTING*, alors, lorsque l'élément déclencheur déclenche les éléments à rapporter, cela fait l'objet d'un rapport.
- Si le mode monitoring (de surveillance) de l'élément déclencheur est *DISABLED*, alors l'élément déclencheur ne déclenche pas les éléments à rapporter.
- Si le mode monitoring (de surveillance) de l'élément à rapporter est *SAMPLING*, alors, lorsque l'élément déclencheur déclenche les éléments à rapporter, cela fait l'objet d'un rapport.
- Si le mode monitoring (de surveillance) de l'élément à rapporter est *REPORTING*, de ce fait, l'élément déclencheur est ignoré. Toutes les notifications des éléments à rapporter sont envoyées à l'expiration de l'intervalle d'édition.
- Si le mode monitoring (de surveillance) de l'élément à rapporter est *DISABLED*, alors il n'y a pas d'échantillonnage de l'élément à rapporter et donc aucune notification à rapporter.
- Le premier déclenchement doit se produire lorsque la première notification est mise en file d'attente pour l'élément déclencheur après la création de la liaison.

Les *Clients* créent et suppriment des liaisons de déclenchement entre un élément déclencheur et un ensemble d'éléments à rapporter. Si le *MonitoredItem* qui représente un

élément à rapporter est supprimé avant que sa liaison associée ne soit supprimée, la liaison de déclenchement est également supprimée, mais l'élément déclencheur n'est pas altéré.

Il convient de ne pas confondre la suppression d'un *MonitoredItem* avec le retrait de l'*Attribut* qu'il surveille. Si le Nœud qui contient l'*Attribut* surveillé est supprimé, le *MonitoredItem* génère une *Notification* avec un *StatusCode Bad_NodeIdUnknown* qui indique la suppression, mais le *MonitoredItem* n'est pas supprimé.

5.12.2 Créer des éléments surveillés (*CreateMonitoredItems*)

5.12.2.1 Description

Ce *Service* est utilisé pour créer et ajouter un ou plusieurs *MonitoredItems* à *Subscription*. Un *MonitoredItem* est supprimé automatiquement par le *Serveur* lorsque *Subscription* est supprimé. La suppression d'un *MonitoredItem* entraîne celle de l'ensemble complet des liaisons d'éléments déclenchés, mais n'a pas d'effet sur les *MonitoredItems* référencés par les éléments déclenchés.

L'appel à répétitions du *Service CreateMonitoredItems* pour ajouter chaque fois un petit nombre de *MonitoredItems* peut affecter défavorablement la performance du *Serveur*. Il convient que les *Clients* ajoutent plutôt un jeu complet de *MonitoredItems* à *Subscription* chaque fois que possible.

Lorsqu'un *MonitoredItem* est ajouté, le *Serveur* exécute un traitement d'initialisation pour celui-ci. Le traitement d'initialisation est défini par le type de *Notification* de l'élément surveillé. Les types de *Notification* sont spécifiés dans la présente Norme et dans les parties de la série de normes qui spécifient les types d'accès, telles que l'IEC 62541-8. Voir l'IEC TR 62541-1 pour la description des Parties Types d'accès

Lorsqu'un utilisateur ajoute un élément surveillé dont l'accès en lecture est refusé à l'utilisateur, l'opération d'ajout de l'élément doit réussir et le mauvais statut *Bad_NotReadable* ou *Bad_UserAccessDenied* doit être retourné dans la réponse *Publish* (éditer). Il s'agit du même comportement que dans le cas où les droits d'accès sont modifiés après l'appel à *CreateMonitoredItems*. Si les droits d'accès passent aux droits de lecture, le *Serveur* doit commencer à envoyer des données pour le *MonitoredItem*. La même procédure doit être appliquée pour un *IndexRange* qui ne fournit pas de données pour la valeur courante, mais qui peut fournir des données plus tard.

Les *Nœuds* surveillés peuvent être retirés de l'*Espace d'Adresses* après la création d'un *MonitoredItem*. Cela n'affecte pas la validité du *MonitoredItem*, mais un *Bad_NodeIdUnknown* doit être retourné dans la réponse *Publish*.

Le retour des informations de réglage de diagnostic dans l'en-tête de demande des *CreateMonitoredItems* ou le dernier *Service ModifyMonitoredItems* est appliqué aux *MonitoredItems* et il sert de réglages d'informations de diagnostic lors de l'envoi des *Notifications* dans la réponse *Publish*.

Les valeurs de demande illégales pour les paramètres pouvant être révisés ne génèrent pas d'erreur. En revanche le *Serveur* choisit des valeurs par défaut et les indique dans le paramètre révisé correspondant.

Il est fortement recommandé par l'OPC UA qu'un *Client* réutilise un *Subscription* après une courte interruption du réseau en activant la *Session* existante sur un nouveau *SecureChannel* comme décrit en 6.5. Si un *Client* a appelé *CreateMonitoredItems* pendant l'interruption du réseau et si l'appel a réussi dans le *Serveur*, mais n'est pas retourné au *Client*, alors le *Client* ne sait pas si l'appel a réussi. Le *Client* peut recevoir des changements de données pour ces éléments surveillés, mais n'est pas capable de les retirer, car il ne connaît pas le *handle* du *Serveur* pour chaque élément surveillé. Il n'est pas non plus possible pour le *Client* de détecter si la création a réussi. Pour supprimer et recréer, utiliser *Subscription* n'est pas non

plus possible, car il peut y avoir plusieurs éléments surveillés fonctionnant normalement qu'il convient de ne pas interrompre. Pour résoudre cette situation, le *Server Object* (objet serveur) fournit une *Method GetMonitoredItems* qui retourne la liste des *handles* du *Client* et du *Serveur* pour les éléments surveillés dans *Subscription*. Cette *Method* est définie dans l'IEC 62541-5.

5.12.2.2 Paramètres

Le Tableau 64 définit les paramètres du *Service*.

Tableau 64 – Paramètres du service CreateMonitoredItems

Nom	Type	Description
Request		
requestHeader	RequestHeader	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).
subscriptionId	IntegerId	L'identificateur affecté par le <i>Serveur</i> pour <i>Subscription</i> qui signalera dans un rapport les <i>Notifications</i> pour ce <i>MonitoredItem</i> (voir 7.13 pour la définition d' <i>IntegerId</i>).
timestampsToReturn	Enum Timestamps ToReturn	Une énumération qui spécifie les <i>Attributs</i> d'horodatage à transmettre pour chaque <i>MonitoredItem</i> . L'énumération <i>TimestampsToReturn</i> est définie en 7.34. Lors de la surveillance d' <i>Événements</i> , cela s'applique seulement aux champs <i>Événement</i> qui sont du type <i>DataValue</i> .
itemsToCreate []	MonitoredItem CreateRequest	Une liste des <i>MonitoredItems</i> à créer et à affecter au <i>Subscription</i> spécifié. Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.
itemToMonitor	ReadValueId	Identifie un élément dans l' <i>Espace d'adresses</i> à surveiller. Pour surveiller afin de détecter des <i>Événements</i> , l'élément <i>attributId</i> de la structure <i>ReadValueId</i> est l'identificateur de l' <i>Attribute EventNotifier</i> . Le type <i>ReadValueId</i> est défini en 7.23.
monitoringMode	Enum MonitoringMode	Le mode de surveillance à établir pour le <i>MonitoredItem</i> . L'énumération <i>MonitoringMode</i> est définie en 7.17.
requestedParameters	Monitoring Parameters	Les paramètres de surveillance demandés. Les <i>Serveurs</i> négocient les valeurs de ces paramètres sur la base du <i>Subscription</i> et des fonctions du <i>Serveur</i> . Le type <i>MonitoringParameters</i> est défini en 7.15.
Response		
responseHeader	Response Header	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).
results []	MonitoredItem CreateResult	Liste de résultats pour les <i>MonitoredItems</i> à créer. La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>itemsToCreate</i> . Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.
statusCode	StatusCode	<i>StatusCode</i> pour le <i>MonitoredItem</i> à créer (voir 7.33 pour la définition de <i>StatusCode</i>).
monitoredItemId	IntegerId	Identificateur affecté par le <i>Serveur</i> pour le <i>MonitoredItem</i> (voir 7.13 pour la définition d' <i>IntegerId</i>). Cet identificateur est unique dans <i>Subscription</i> , mais pourrait ne pas être unique dans le <i>Serveur</i> ou dans une <i>Session</i> . Ce paramètre est présent seulement si le <i>statusCode</i> indique que le <i>MonitoredItem</i> a été créé avec succès.
revisedSampling Interval	Duration	L'intervalle d'échantillonnage réel que le <i>Serveur</i> utilisera. Cette valeur est fondée sur plusieurs facteurs, y compris les fonctions du système sous-jacent. Le <i>Serveur</i> doit toujours retourner un <i>revisedSamplingInterval</i> qui est supérieur ou égal au <i>samplingInterval</i> demandé. Si le <i>samplingInterval</i> demandé est supérieur à l'intervalle d'échantillonnage maximal pris en charge par le <i>Serveur</i> , c'est l'intervalle d'échantillonnage maximal qui est retourné.
revisedQueueSize	Counter	La taille réelle de file d'attente que le <i>Serveur</i> utilisera.
filterResult	Extensible Parameter MonitoringFilter Result	Contient toutes les éventuelles valeurs de paramètre révisées ou tous les éventuels résultats d'erreur associés au <i>MonitoringFilter</i> spécifié dans <i>requestedParameters</i> . Ce paramètre peut être omis s'il ne s'est pas produit d'erreurs. Le type de paramètre <i>MonitoringFilterResult</i> est un type de paramètre extensible spécifié en 7.16.
diagnosticInfos []	DiagnosticInfo	Liste d'informations de diagnostic pour les <i>MonitoredItems</i> à créer (voir 7.8 pour la définition de <i>DiagnosticInfo</i>). La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>itemsToCreate</i> . Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été rencontrée dans le traitement de la demande.

5.12.2.3 Résultats des services

Le Tableau 65 définit les résultats des *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 65 – Codes de résultats des services CreateMonitoredItems

Id symbolique	Description
Bad_NothingToDo	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_TooManyOperations	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_TimestampsToReturnInvalid	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_SubscriptionIdInvalid	Voir le Tableau 165 pour la description de ce code de résultat.

5.12.2.4 StatusCodes

Le Tableau 66 définit les valeurs de niveau opération pour le paramètre *statusCode* qui sont spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 166.

Tableau 66 – Codes de résultats de niveau opération de CreateMonitoredItems

Id symbolique	Description
Bad_MonitoringModelInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_NodeIdInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_NodeIdUnknown	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_AttributeIdInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_IndexRangeInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_IndexRangeNoData	Voir le Tableau 166 pour la description de ce code de résultat. Si les <i>ArrayDimensions</i> ont une longueur fixe qui ne peut être modifiée et qu'aucune donnée n'existe dans la plage d'indices spécifiée, <i>Bad_IndexRangeNoData</i> est retourné dans <i>CreateMonitoredItems</i> . Sinon, si la longueur de la matrice est dynamique, le <i>Serveur</i> doit retourner ce statut dans une réponse <i>Publish</i> pour le <i>MonitoredItem</i> si aucune donnée n'existe dans la plage.
Bad_DataEncodingInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_DataEncodingUnsupported	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_MonitoredItemFilterInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_MonitoredItemFilterUnsupported	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_FilterNotAllowed	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_TooManyMonitoredItems	Le <i>Serveur</i> a atteint son nombre maximal d'éléments surveillés.

5.12.3 Modifier des éléments surveillés (ModifyMonitoredItems)

5.12.3.1 Description

Ce *Service* est utilisé pour modifier les *MonitoredItems* d'un *Subscription*. Les modifications apportées aux réglages de *MonitoredItem* doivent être appliquées immédiatement par le *Serveur*. Elles prennent effet le plus tôt possible sans toutefois dépasser une durée égale au double du nouveau *revisedSamplingInterval*.

Le retour des informations de réglage des diagnostics dans l'en-tête de demande de *CreateMonitoredItems* ou le dernier *Service ModifyMonitoredItems* est appliqué aux *MonitoredItems* (éléments surveillés) et sert de réglages d'informations de diagnostic lors de l'envoi de Notifications dans la réponse *Publish*.

Les valeurs de demande illégales pour les paramètres pouvant être révisés ne génèrent pas d'erreur. En revanche le *Serveur* choisit des valeurs par défaut et les indique dans le paramètre révisé correspondant.

5.12.3.2 Paramètres

Le Tableau 67 définit les paramètres du *Service*.

Tableau 67 – Paramètres du service ModifyMonitoredItems

Nom	Type	Description
Request		
requestHeader	RequestHeader	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).
subscriptionId	IntegerId	L'identificateur affecté par le <i>Serveur</i> pour <i>Subscription</i> utilisé pour qualifier le <i>monitoredItemId</i> (voir 7.13 pour la définition d'IntegerId).
timestampsToReturn	Enum Timestamps ToReturn	Une énumération qui spécifie les <i>Attributs</i> d'horodatage à transmettre pour chaque <i>MonitoredItem</i> à modifier. L'énumération <i>TimestampsToReturn</i> est définie en 7.34. Lors de la surveillance d' <i>Événements</i> , cela s'applique seulement aux champs <i>Événement</i> qui sont du type <i>DataValue</i> .
itemsToModify []	MonitoredItemMo difyRequest	La liste des <i>MonitoredItems</i> à modifier. Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.
monitoredItemId	IntegerId	Identificateur affecté par le <i>Serveur</i> pour le <i>MonitoredItem</i> .
requestedParameters	Monitoring Parameters	Les valeurs demandées pour les paramètres de surveillance. Le type <i>MonitoringParameters</i> est défini en 7.15. Si le nombre de notifications dans la file d'attente dépasse la taille de la nouvelle file d'attente, les notifications en excédent doivent être rejetées selon la politique de rejet configurée.
Response		
responseHeader	Response Header	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).
results []	MonitoredItemMo difyResult	Liste de résultats pour les <i>MonitoredItems</i> à modifier. La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>itemsToModify</i> . Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.
statusCode	StatusCode	<i>StatusCode</i> pour le <i>MonitoredItem</i> à modifier (voir 7.33 pour la définition de <i>StatusCode</i>).
revisedSampling Interval	Duration	L'intervalle d'échantillonnage réel que le <i>Serveur</i> utilisera. Le <i>Serveur</i> retourne la valeur qu'il utilisera réellement pour l'intervalle d'échantillonnage. Cette valeur est fondée sur plusieurs facteurs, y compris les fonctions du système sous-jacent. Le <i>Serveur</i> doit toujours retourner un <i>revisedSamplingInterval</i> qui est supérieur ou égal au <i>SamplingInterval</i> demandé. Si le <i>SamplingInterval</i> demandé est supérieur à l'intervalle d'échantillonnage maximal pris en charge par le <i>Serveur</i> , c'est l'intervalle d'échantillonnage maximal qui est retourné.
revisedQueueSize	Counter	La taille réelle de file d'attente que le <i>Serveur</i> utilisera.
filterResult	Extensible Parameter MonitoringFilterR esult	Contient toutes les éventuelles valeurs de paramètre révisées ou tous les éventuels résultats d'erreurs associés au <i>MonitoringFilter</i> spécifié dans la demande. Ce paramètre peut être omis s'il ne s'est pas produit d'erreurs. Le type de paramètre <i>MonitoringFilterResult</i> est un type de paramètre extensible spécifié en 7.16.
diagnosticInfos []	DiagnosticInfo	Liste d'informations de diagnostic pour les <i>MonitoredItems</i> à modifier (voir 7.8 pour la définition de <i>DiagnosticInfo</i>). La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>itemsToModify</i> . Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été rencontrée dans le traitement de la demande.

5.12.3.3 Résultats des services

Le Tableau 68 définit les résultats des *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 68 – Codes de résultats des services ModifyMonitoredItems

Id symbolique	Description
Bad_NothingToDo	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_TooManyOperations	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_TimestampsToReturnInvalid	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_SubscriptionIdInvalid	Voir le Tableau 165 pour la description de ce code de résultat.

5.12.3.4 StatusCodes

Le Tableau 69 définit les valeurs pour le paramètre *statusCode* de niveau opération qui sont spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 166.

Tableau 69 – Codes de résultats de niveau opération de ModifyMonitoredItems

Id symbolique	Description
Bad_MonitoredItemIdInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_MonitoredItemFilterInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_MonitoredItemFilterUnsupported	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_FilterNotAllowed	Voir le Tableau 165 pour la description de ce code de résultat.

5.12.4 Gérer des modes de surveillance (SetMonitoringMode)

5.12.4.1 Description

Ce *Service* est utilisé pour établir le mode de surveillance pour un ou plusieurs *MonitoredItems* d'un *Subscription*. Le fait de mettre le mode à DISABLED entraîne la suppression de toutes les *Notifications* placées en file d'attente.

5.12.4.2 Paramètres

Le Tableau 70 définit les paramètres du *Service*.

Tableau 70 – Paramètres du service SetMonitoringMode

Nom	Type	Description
Request		
requestHeader	RequestHeader	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).
subscriptionId	IntegerId	L'identificateur affecté par le <i>Serveur</i> pour <i>Subscription</i> utilisé pour qualifier les <i>monitoredItemIds</i> (voir 7.13 pour la définition d' <i>IntegerId</i>).
monitoringMode	Enum MonitoringMode	Le mode de surveillance à établir pour les <i>MonitoredItems</i> . L'énumération <i>MonitoringMode</i> est définie en 7.17.
monitoredItemIds []	IntegerId	Liste des identificateurs affectés par le <i>Serveur</i> pour les <i>MonitoredItems</i> .
Response		
responseHeader	Response Header	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).
results []	StatusCode	Liste des <i>StatusCodes</i> pour les <i>MonitoredItems</i> à activer/désactiver (voir 7.33 pour la définition des <i>StatusCode</i>). La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>monitoredItemIds</i> .
diagnosticInfos []	DiagnosticInfo	Liste d'informations de diagnostic pour les <i>MonitoredItems</i> à activer/désactiver (voir 7.8 pour la définition de <i>DiagnosticInfo</i>). La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>monitoredItemIds</i> . Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été rencontrée dans le traitement de la demande.

5.12.4.3 Résultats des services

Le Tableau 71 définit les résultats des *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 71 – Codes de résultats des services SetMonitoringMode

Id symbolique	Description
Bad_NothingToDo	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_TooManyOperations	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_SubscriptionIdInvalid	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_MonitoringModeInvalid	Voir le Tableau 166 pour la description de ce code de résultat.

5.12.4.4 StatusCodes

Le Tableau 72 définit les valeurs de niveau opération pour le paramètre *statusCode* qui sont spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 166.

Tableau 72 – Codes de résultats de niveau opération de SetMonitoringMode

Valeur	Description
Bad_MonitoredItemIdInvalid	Voir le Tableau 166 pour la description de ce code de résultat.

5.12.5 Gérer des déclenchements (SetTriggering)

5.12.5.1 Description

Ce *Service* est utilisé pour créer et supprimer des liaisons de déclenchement pour un élément déclencheur. L'élément déclencheur et les éléments à rapporter doivent appartenir au même *Subscription*.

Chaque liaison de déclenchement relie un élément déclencheur à un élément à rapporter. Chaque liaison est représentée par l'identificateur de *MonitoredItem* pour l'élément à rapporter. Un code d'erreur est retourné si cet identificateur n'est pas valide.

Voir 5.12.1.6 pour une description du modèle de déclenchement.

5.12.5.2 Paramètres

Le Tableau 73 définit les paramètres du *Service*.

Tableau 73 – Paramètres du service SetTriggering

Nom	Type	Description
Request		
requestHeader	Request Header	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).
subscriptionId	IntegerId	L'identificateur affecté par le <i>Serveur</i> pour le <i>Subscription</i> qui contient l'élément déclencheur et les éléments à rapporter (voir 7.13 pour la définition d' <i>IntegerId</i>).
triggeringItemId	IntegerId	Identificateur affecté par le <i>Serveur</i> pour le <i>MonitoredItem</i> utilisé comme l'élément déclencheur.
linksToAdd []	IntegerId	La liste des identificateurs affectés par le <i>Serveur</i> des éléments à rapporter qui sont à ajouter comme liaisons de déclenchement. La liste de <i>linksToRemove</i> est traitée avant le <i>linksToAdd</i> .
linksToRemove []	IntegerId	La liste des identificateurs affectés par le <i>Serveur</i> des éléments à rapporter pour les liaisons de déclenchement à supprimer. La liste de <i>linksToRemove</i> est traitée avant le <i>linksToAdd</i> .
Response		
responseHeader	Response Header	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).
addResults []	StatusCode	Liste des <i>StatusCodes</i> pour les éléments à ajouter (voir 7.33 pour la définition des <i>StatusCode</i>). La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre <i>linksToAdd</i> spécifié dans la demande.
addDiagnosticInfos []	Diagnostic Info	Liste d'informations de diagnostic pour les liaisons à ajouter (voir 7.8 pour la définition de <i>DiagnosticInfo</i>). La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>linksToAdd</i> . Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été rencontrée dans le traitement de la demande.
removeResults []	StatusCode	Liste des <i>StatusCodes</i> pour les éléments à supprimer. La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre <i>linksToRemove</i> spécifié dans la demande.
removeDiagnosticInfos []	Diagnostic Info	Liste d'informations de diagnostic pour les liaisons à supprimer. La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>linksToRemove</i> . Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été rencontrée dans le traitement de la demande.

5.12.5.3 Résultats des services

Le Tableau 74 définit les résultats des *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis en 7.33.

Tableau 74 – Codes de résultats des services SetTriggering

Id symbolique	Description
Bad_NothingToDo	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_TooManyOperations	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_SubscriptionIdInvalid	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_MonitoredItemIdInvalid	Voir le Tableau 166 pour la description de ce code de résultat.

5.12.5.4 StatusCodes

Le Tableau 75 définit les valeurs pour les paramètres results qui sont spécifiques à ce Service. Les StatusCodes communs sont définis au Tableau 166.

Tableau 75 – Codes de résultats de niveau opération de SetTriggering

Id symbolique	Description
Bad_MonitoredItemIdInvalid	Voir le Tableau 166 pour la description de ce code de résultat.

5.12.6 Supprimer des éléments surveillés (DeleteMonitoredItems)

5.12.6.1 Description

Ce Service est utilisé pour retirer un ou plusieurs *MonitoredItems* d'un *Subscription*. Lorsqu'un *MonitoredItem* est supprimé, ses liaisons d'éléments déclenchés sont supprimées aussi.

Le retrait réussi d'un *MonitoredItem* peut toutefois ne pas retirer les *Notifications* pour le *MonitoredItem* qui sont en cours d'envoi par *Subscription*. Par conséquent, les *Clients* peuvent recevoir des *Notifications* pour le *MonitoredItem* après qu'ils aient reçu une réponse positive indiquant que le *MonitoredItem* a été supprimé.

5.12.6.2 Paramètres

Le Tableau 76 définit les paramètres du Service.

Tableau 76 – Paramètres du service DeleteMonitoredItems

Nom	Type	Description
Request		
requestHeader	RequestHeader	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).
subscriptionId	IntegerId	L'identificateur affecté par le <i>Serveur</i> pour <i>Subscription</i> qui contient les <i>MonitoredItems</i> à supprimer (voir 7.13 pour la définition d'IntegerId).
monitoredItemIds []	IntegerId	Liste des identificateurs affectés par le <i>Serveur</i> pour les <i>MonitoredItems</i> à supprimer.
Response		
responseHeader	Response Header	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).
results []	Status Code	Liste des <i>StatusCodes</i> pour les <i>MonitoredItems</i> à supprimer (voir 7.33 pour la définition des <i>StatusCode</i>). La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>monitoredItemIds</i> .
diagnosticInfos []	DiagnosticInfo	Liste d'informations de diagnostic pour les <i>MonitoredItems</i> à supprimer (voir 7.8 pour la définition de <i>DiagnosticInfo</i>). La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>monitoredItemIds</i> . Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été rencontrée dans le traitement de la demande.

5.12.6.3 Résultats des services

Le Tableau 77 définit les résultats des Services spécifiques à ce Service. Les StatusCodes communs sont définis au Tableau 165.

Tableau 77 – Codes de résultats des services DeleteMonitoredItems

Id symbolique	Description
Bad_NothingToDo	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_TooManyOperations	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_SubscriptionIdInvalid	Voir le Tableau 165 pour la description de ce code de résultat.

5.12.6.4 StatusCodes

Le Tableau 78 définit les valeurs pour le paramètre *results* qui sont spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 166.

Tableau 78 – Codes de résultats de niveau opération de DeleteMonitoredItems

Id symbolique	Description
Bad_MonitoredItemIdInvalid	Voir le Tableau 166 pour la description de ce code de résultat.

5.13 Jeu de services d'abonnements (Subscription Service Set)

5.13.1 Modèle d'abonnement (SubscriptionModel)

5.13.1.1 Description

Les *Subscriptions* sont utilisés pour rapporter des *Notifications* au *Client*. Leur comportement général est récapitulé ci-après. Leur comportement précis est décrit en 5.13.1.2.

- Les *Subscriptions* ont un jeu de *MonitoredItems* qui leur sont affectés par le *Client*. Les *MonitoredItems* génèrent des *Notifications* que *Subscription* est tenu de signaler dans un rapport au *Client* (voir 5.12.1 pour une description des *MonitoredItems*).
- Les *Subscriptions* ont un intervalle d'édition. L'intervalle d'édition d'un *Subscription* définit la fréquence cyclique à laquelle le *Subscription* s'exécute. Chaque fois qu'il s'exécute, il tente d'envoyer un *NotificationMessage* au *Client*. Les *NotificationMessages* contiennent des *Notifications* qui n'ont pas encore été rapportées au *Client*.
- Les *NotificationMessages* sont envoyés au *Client* en réponse à des demandes *Publish*. Les demandes *Publish* sont normalement placées en file d'attente dans la *Session* à mesure qu'elles sont reçues, et une est sortie de la file d'attente et traitée par un abonnement relatif à cette *Session* à chaque cycle d'édition, s'il y a des *Notifications* à rapporter. Lorsqu'il n'y en a pas, la demande *Publish* est sortie de la file d'attente de la *Session* et le *Serveur* attend le prochain cycle et vérifie de nouveau la présence de *Notifications*.
- Au début d'un cycle, s'il y a des *Notifications* à envoyer, mais s'il n'y a pas de demandes *Publish* placées en file d'attente, le *Serveur* entre dans l'état d'attente (wait) pour qu'une demande *Publish* soit reçue. Lorsqu'une demande est reçue, elle est traitée immédiatement sans attendre le prochain cycle d'édition.
- Les *NotificationMessages* sont identifiés de manière unique par des numéros de séquence qui permettent à des *Clients* de détecter des *Messages* manqués. L'intervalle d'édition définit aussi l'intervalle d'échantillonnage par défaut pour ses *MonitoredItems*.
- Les *Subscriptions* ont un compteur d'entretien qui compte le nombre de cycles d'édition consécutifs dans lesquels il n'y a pas de *Notifications* à rapporter au *Client*. Lorsque le compte d'entretien maximal est atteint, une demande *Publish* est sortie de la file d'attente et utilisée pour retourner le *Message* d'entretien. Ce *Message* d'entretien informe le *Client* que *Subscription* est encore actif. Chaque *Message* d'entretien est une réponse à une demande *Publish* dans laquelle le paramètre *notificationMessage* ne contient pas de *Notifications* et qui contient le numéro de séquence du prochain *NotificationMessage* à envoyer. Dans les articles qui suivent, le terme *NotificationMessage* se réfère à une réponse à une demande *Publish* dans laquelle le paramètre *notificationMessage* contient réellement une ou plusieurs *Notifications*, par opposition à un *keep-alive Message* (Message d'entretien) dans lequel ce paramètre ne contient pas de *Notifications*. Le compte d'entretien maximal est établi par le *Client* pendant la création d'un *Subscription* et peut être modifié ultérieurement en utilisant le *Service ModifySubscription*. D'une manière

similaire au traitement des *Notifications* décrit en c) ci-dessus, s'il n'y a pas de demandes *Publish* placées en file d'attente, le *Serveur* attend la réception de la prochaine demande et envoie immédiatement le message d'entretien sans attendre le prochain cycle d'édition.

- g) L'édition par un *Subscription* peut être activée ou désactivée par le *Client* à sa création ou ultérieurement en utilisant le *Service SetPublishingMode*. La désactivation force *Subscription* à cesser d'envoyer des *NotificationMessages* au *Client*. Cependant, *Subscription* continue à s'exécuter de façon cyclique et continue d'envoyer des *Messages* d'entretien au *Client*.
- h) Les *Subscriptions* ont un compteur de durée de vie qui compte le nombre de cycles d'édition consécutifs dans lesquels il n'y a pas de demandes *Publish* disponibles pour envoyer une réponse *Publish* pour *Subscription*. Tout appel de *Service* qui utilise le *SubscriptionId* ou le traitement d'une réponse *Publish* réinitialise le compteur de durée de vie de *Subscription*. Lorsque ce compteur atteint la valeur calculée pour la durée de vie d'un *Subscription* en fonction du paramètre *MaxKeepAliveCount* du *Service CreateSubscription* (5.13.2), *Subscription* est fermé. La fermeture d'un *Subscription* entraîne la suppression de ses *MonitoredItems*. De plus, le *Serveur* doit émettre un *notificationMessage* de *StatusChangeNotification* avec le code de statut *Bad_Timeout*. Le type du *notificationMessage* de *StatusChangeNotification* est défini en 7.19.4.
- i) Les *Sessions* maintiennent une file d'attente de transmission des *NotificationMessages* envoyés. Les *NotificationMessages* sont retenus dans cette file d'attente jusqu'à ce qu'ils soient acquittés. La *Session* doit maintenir une file d'attente de transmission d'une taille au moins égale à deux fois le nombre de demandes *Publish* par *Session* prises en charge par le *Serveur*. La taille minimale de la file d'attente de transmission peut être modifiée par un *Profil* dans l'IEC 62541-7. Le nombre minimal de demandes *Publish* par *Session* prises en charge par le *Serveur* est défini dans l'IEC 62541-7. Il est nécessaire que les *Clients* acquittent les *NotificationMessages* à leur réception. Dans le cas d'un dépassement de capacité de la file d'attente de transmission, le *NotificationMessage* le plus anciennement envoyé est supprimé. Si un *Subscription* est transféré vers une autre *Session*, les *NotificationMessages* placés dans la file d'attente pour ce *Subscription* sont déplacés de l'ancienne *Session* vers la nouvelle.

Le numéro de séquence est un entier non signé de 32 bits qui est incrémenté de 1 pour chaque *NotificationMessage* envoyé. La valeur 0 n'est jamais utilisée comme numéro de séquence. Le premier *NotificationMessage* envoyé sur un *Subscription* a le numéro de séquence 1. Si le numéro de séquence atteint son maximum, il repasse à 1.

Lorsqu'un *Subscription* est créé, le premier *Message* est envoyé à la fin du premier cycle d'édition pour signaler au *Client* que *Subscription* est opérationnel. Un *NotificationMessage* est envoyé s'il y a des *Notifications* prêtes à être rapportées. S'il n'y en a pas, il est envoyé à la place un *Message* d'entretien qui contient un numéro de séquence de 1, indiquant que le premier *NotificationMessage* n'a pas encore été envoyé. C'est la seule fois qu'un *Message* d'entretien est envoyé sans attendre d'avoir atteint le compte d'entretien maximal, comme spécifié en f) ci-dessus.

La valeur du numéro de séquence n'est jamais réinitialisée au cours de la durée de vie d'un *Subscription*. Par conséquent, ce même numéro de séquence ne doit jamais être réutilisé par un *Subscription* avant que plus de quatre milliards de *NotificationMessages* aient été envoyés. À une fréquence continue de mille *NotificationMessages* par seconde sur un *Subscription* donné, il faudrait environ cinquante jours pour que le même numéro de séquence soit réutilisé. Cela permet aux *Clients* de traiter en toute sécurité les numéros de séquence comme étant uniques.

Les numéros de séquence sont également utilisés par les *Clients* pour acquitter réception des *NotificationMessages*. Les demandes *Publish* permettent au *Client* d'acquitter toutes les *Notifications* jusqu'à un numéro de séquence spécifique et permettent d'acquitter le numéro de séquence du dernier *NotificationMessage* reçu. Un ou plusieurs trous peuvent exister entre elles. Les acquittements permettent au *Serveur* d'effacer des *NotificationMessages* de sa file d'attente de retransmission.

Les *Clients* peuvent demander la retransmission de *NotificationMessages* sélectionnés en utilisant le *Service Republish*. Ce *Service* retourne le *Message* demandé.

5.13.1.2 Table d'états

La table d'états décrit le fonctionnement du *Subscription*. Le modèle de fonctionnement des opérations est décrit par cette table d'états. Cette description s'applique lorsque l'édition est activée ou désactivée pour *Subscription*.

Après la création d'un *Subscription*, le *Serveur* lance la minuterie d'édition et la redémarre chaque fois qu'elle expire. Si la minuterie expire le nombre de fois défini pour la durée de vie d'un *Subscription* sans avoir reçu de demande du *Service* de *Subscription* de la part du *Client*, *Subscription* suppose que le *Client* n'est plus présent et se termine.

Les *Clients* envoient des demandes *Publish* aux *Serveurs* pour recevoir des *Notifications*. Les demandes *Publish* ne sont dirigées à aucun *Subscription* spécifique et, par conséquent, peuvent être utilisées par n'importe quel *Subscription*. Chacune contient des acquittements pour un ou plusieurs *Subscriptions*. Ces acquittements sont traités lorsque la demande *Publish* est reçue. Le *Serveur* met alors la demande dans une file d'attente partagée par tous les *Subscriptions*, sauf dans les cas suivants.

- a) La réponse *Publish* précédente indiquait qu'il y avait encore d'autres *Notifications* prêtes à être transférées et il n'y avait plus de demandes *Publish* en file d'attente pour les transférer.
- b) La minuterie d'édition d'un *Subscription* a expiré et il y avait soit des *Notifications* à envoyer, soit un *Message* d'entretien à envoyer.

Dans ces cas, la demande *Publish* nouvellement reçue est immédiatement traitée par le premier *Subscription* qui rencontrera le cas a) ou le cas b).

Chaque fois que la minuterie d'édition expire, elle est immédiatement réinitialisée. S'il y a des *Notifications* ou un *Message* d'entretien à envoyer, il sort de la file d'attente et traite une demande *Publish*. Lorsqu'un *Subscription* traite une demande *Publish*, il accède aux files d'attente de ses *MonitoredItems* et sort ses éventuelles *Notifications* de la file d'attente. Il retourne ces *Notifications* dans la réponse, positionnant le fanion *moreNotifications* s'il n'a pas été capable de retourner toutes les *Notifications* disponibles dans la réponse.

S'il y avait des *Notifications* ou un *Message* d'entretien à envoyer, mais s'il n'y avait pas de demandes *Publish* en file d'attente, *Subscription* suppose que la demande *Publish* est tardive et attend que la prochaine demande *Publish* soit reçue, comme décrit dans le cas b).

Si *Subscription* est désactivé lorsque la minuterie d'édition expire ou s'il n'y a pas de *Notifications* disponibles, il entre dans l'état d'entretien et met le compteur d'entretien à sa valeur maximale comme défini pour *Subscription*.

Pendant qu'il est dans l'état d'entretien, il recherche des *Notifications* chaque fois que la minuterie d'édition expire. Si une ou plusieurs *Notifications* ont été générées, une demande *Publish* est sortie de la file d'attente et un *NotificationMessage* est retourné dans la réponse. En revanche, si la minuterie d'édition expire sans qu'une *Notification* ne devienne disponible, une demande *Publish* est sortie de la file d'attente et un *Message* d'entretien est retourné dans la réponse. Le *Subscription* revient alors dans son état normal qui est d'attendre que la minuterie d'édition expire de nouveau. Si, dans l'un ou l'autre de ces cas, il n'y a pas de demande *Publish* dans la file d'attente, *Subscription* attend que la prochaine demande *Publish* soit reçue, comme décrit dans le cas b).

Les états de *Subscription* sont définis au Tableau 79.

Tableau 79 – États d'un abonnement

État	Description
CLOSED	Le <i>Subscription</i> n'a pas encore été créé ou s'est terminé.
CREATING	Le <i>Subscription</i> est en cours de création.
NORMAL	Le <i>Subscription</i> vérifie de façon cyclique s'il y a des <i>Notifications</i> issues de ses <i>MonitoredItems</i> . Le compteur d'entretien n'est pas utilisé dans cet état.
LATE	La minuterie d'édition a expiré et il y a des <i>Notifications</i> disponibles ou un <i>Message</i> d'entretien prêt à être envoyé, mais il n'y a pas de demandes <i>Publish</i> en file d'attente. Dans cet état, la prochaine demande <i>Publish</i> est traitée lorsqu'elle est reçue. Le compteur d'entretien n'est pas utilisé dans cet état.
KEEPALIVE	Le <i>Subscription</i> vérifie de façon cyclique s'il y a des <i>Notifications</i> issues de ses <i>MonitoredItems</i> ou si le compteur d'entretien compte à rebours jusqu'à 0 à partir de son maximum.

La table d'états est décrite au Tableau 80. Les règles et conventions suivantes s'appliquent.

- a) Les *Événements* représentent la réception de demandes de *Service* et l'occurrence d'*Événements* internes, tels que les expirations de temporisateurs.
- b) Les *Événements* de demandes de *Services* peuvent être accompagnés de conditions qui soumettent à essai les valeurs des paramètres de *Service*. Les noms de paramètres commencent par une lettre minuscule.
- c) Des *Événements* internes peuvent être accompagnés de conditions qui soumettent à essai les valeurs des *Variables* d'état. Les *Variables* d'état sont définies en 5.13.1.3. Elles commencent par une lettre majuscule.
- d) Les *Événements* internes et les demandes de *Services* peuvent être accompagnés de conditions représentées par des fonctions dont la valeur de retour est soumise à essai. Les fonctions sont identifiées par “()” après leur nom. Elles sont décrites en 5.13.1.4.
- e) Lorsqu'un *Événement* est reçu, la première transition pour l'état courant est localisée et les transitions sont scrutées séquentiellement pour trouver la première transition qui satisfait aux critères de l'*Événement* ou des conditions. Si aucune transition n'est trouvée, l'*Événement* est ignoré.
- f) Les actions sont décrites par des manipulations de fonctions et des *Variables* d'état.
- g) L'*Événement LifetimeTimerExpires* est déclenché lorsque son compteur associé atteint zéro.

Tableau 80 – Table d'états d'un abonnement

#	État courant	Événement/Conditions	Action	Prochain état
1	CLOSED	Recevoir la demande <i>CreateSubscription</i>	<i>CreateSubscription()</i>	CREATING
2	CREATING	<i>CreateSubscription</i> échoue	<i>ReturnNegativeResponse()</i>	CLOSED
3	CREATING	<i>CreateSubscription</i> réussit	<i>InitializeSubscription()</i> <i>MessageSent</i> = FALSE <i>ReturnResponse()</i>	NORMAL
4	NORMAL	Recevoir la demande <i>Publish</i> && (<i>PublishingEnabled</i> == FALSE (<i>PublishingEnabled</i> == TRUE && <i>MoreNotifications</i> == FALSE))	<i>DeleteAkedNotificationMsgs()</i> <i>EnqueuePublishingReq()</i>	NORMAL
5	NORMAL	Recevoir la demande <i>Publish</i> && <i>PublishingEnabled</i> == TRUE && <i>MoreNotifications</i> == TRUE	<i>ResetLifetimeCounter()</i> <i>DeleteAkedNotificationMsgs()</i> <i>ReturnNotifications()</i> <i>MessageSent</i> = TRUE	NORMAL
6	NORMAL	<i>PublishingTimer</i> expire && <i>PublishingReqQueued</i> == TRUE && <i>PublishingEnabled</i> == TRUE && <i>NotificationsAvailable</i> == TRUE	<i>ResetLifetimeCounter()</i> <i>StartPublishingTimer()</i> <i>DequeuePublishReq()</i> <i>ReturnNotifications()</i> <i>MessageSent</i> == TRUE	NORMAL
7	NORMAL	<i>PublishingTimer</i> expire && <i>PublishingReqQueued</i> == TRUE && <i>MessageSent</i> == FALSE && (<i>PublishingEnabled</i> == FALSE (<i>PublishingEnabled</i> == TRUE && <i>NotificationsAvailable</i> == FALSE))	<i>ResetLifetimeCounter()</i> <i>StartPublishingTimer()</i> <i>DequeuePublishReq()</i> <i>ReturnKeepAlive()</i> <i>MessageSent</i> == TRUE	NORMAL
8	NORMAL	<i>PublishingTimer</i> expire && <i>PublishingReqQueued</i> == FALSE && (<i>MessageSent</i> == FALSE (<i>PublishingEnabled</i> == TRUE && <i>NotificationsAvailable</i> == TRUE))	<i>StartPublishingTimer()</i>	LATE
9	NORMAL	<i>PublishingTimer</i> expire && <i>MessageSent</i> == TRUE && (<i>PublishingEnabled</i> == FALSE (<i>PublishingEnabled</i> == TRUE && <i>NotificationsAvailable</i> == FALSE))	<i>StartPublishingTimer()</i> <i>ResetKeepAliveCounter()</i>	KEEPALIVE
10	LATE	Recevoir la demande <i>Publish</i> && <i>PublishingEnabled</i> == TRUE && (<i>NotificationsAvailable</i> == TRUE <i>MoreNotifications</i> == TRUE)	<i>ResetLifetimeCounter()</i> <i>DeleteAkedNotificationMsgs()</i> <i>ReturnNotifications()</i> <i>MessageSent</i> = TRUE	NORMAL
11	LATE	Recevoir demande <i>Publish</i> && (<i>PublishingEnabled</i> == FALSE (<i>PublishingEnabled</i> == TRUE && <i>NotificationsAvailable</i> == FALSE && <i>MoreNotifications</i> == FALSE))	<i>ResetLifetimeCounter()</i> <i>DeleteAkedNotificationMsgs()</i> <i>ReturnKeepAlive()</i> <i>MessageSent</i> = TRUE	KEEPALIVE
12	LATE	<i>PublishingTimer</i> expire	<i>StartPublishingTimer()</i>	LATE
13	KEEPALIVE	Recevoir la demande <i>Publish</i>	<i>DeleteAkedNotificationMsgs()</i> <i>EnqueuePublishingReq()</i>	KEEPALIVE

#	État courant	Événement/Conditions	Action	Prochain état
14	KEEPALIVE	PublishingTimer expire && PublishingEnabled == TRUE && NotificationsAvailable == TRUE && PublishingReqQueued == TRUE	ResetLifetimeCounter() StartPublishingTimer() DequeuePublishReq() ReturnNotifications() MessageSent == TRUE	NORMAL
15	KEEPALIVE	PublishingTimer expire && PublishingReqQueued == TRUE && KeepAliveCounter == 1 && (PublishingEnabled == FALSE (PublishingEnabled == TRUE && NotificationsAvailable == FALSE))	StartPublishingTimer() DequeuePublishReq() ReturnKeepAlive() ResetKeepAliveCounter()	KEEPALIVE
16	KEEPALIVE	PublishingTimer expire && KeepAliveCounter > 1 && (PublishingEnabled == FALSE (PublishingEnabled == TRUE && NotificationsAvailable == FALSE)))	StartPublishingTimer() KeepAliveCounter--	KEEPALIVE
17	KEEPALIVE	PublishingTimer expire && PublishingReqQueued == FALSE && (KeepAliveCounter == 1 (KeepAliveCounter > 1 && PublishingEnabled == TRUE && NotificationsAvailable == TRUE)))	StartPublishingTimer()	LATE
18	NORMAL LATE KEEPALIVE	Recevoir la demande ModifySubscription	ResetLifetimeCounter() UpdateSubscriptionParams() ReturnResponse()	SAME
19	NORMAL LATE KEEPALIVE	Recevoir la demande SetPublishingMode	ResetLifetimeCounter() SetPublishingEnabled() MoreNotifications = FALSE ReturnResponse()	SAME
20	NORMAL LATE KEEPALIVE	Recevoir la demande Republish && RequestedMessageFound == TRUE	ResetLifetimeCounter() ReturnResponse()	SAME
21	NORMAL LATE KEEPALIVE	Recevoir la demande Republish && RequestedMessageFound == FALSE	ResetLifetimeCounter() ReturnNegativeResponse()	SAME
22	NORMAL LATE KEEPALIVE	Recevoir la demande TransferSubscriptions && SessionChanged() == FALSE	ResetLifetimeCounter() ReturnNegativeResponse ()	SAME
23	NORMAL LATE KEEPALIVE	Recevoir la demande TransferSubscriptions && SessionChanged() == TRUE && ClientValidated() ==TRUE	SetSession() ResetLifetimeCounter() ReturnResponse() IssueStatusChangeNotification()	SAME
24	NORMAL LATE KEEPALIVE	Recevoir la demande TransferSubscriptions && SessionChanged() == TRUE && ClientValidated() == FALSE	ReturnNegativeResponse()	SAME
25	NORMAL LATE KEEPALIVE	Recevoir la demande DeleteSubscriptions && SubscriptionAssignedToClient ==TRUE	DeleteMonitoredItems() DeleteClientPublReqQueue()	CLOSED

#	État courant	Événement/Conditions	Action	Prochain état
26	NORMAL LATE KEEPALIVE	Recevoir la demande DeleteSubscriptions && SubscriptionAssignedToClient ==FALSE	ResetLifetimeCounter() ReturnNegativeResponse()	SAME
27	NORMAL LATE KEEPALIVE	LifetimeCounter == 1 Le LifetimeCounter est décrémenté si PublishingTimer expire et PublishingReqQueued == FALSE Le LifetimeCounter est réinitialisé si PublishingReqQueued == TRUE.	DeleteMonitoredItems() IssueStatusChangeNotification()	CLOSED

5.13.1.3 Variables d'état et paramètres

Les *Variables* d'état sont définies dans l'ordre alphabétique au Tableau 81.

Tableau 81 – Variables d'état et paramètres

Variable d'état	Description
MoreNotifications	Une valeur booléenne qui est mise à TRUE seulement par la fonction CreateNotificationMsg() lorsqu'il y avait trop de <i>Notifications</i> pour un seul <i>NotificationMessage</i> .
LatePublishRequest	Une valeur booléenne qui est mise à TRUE pour refléter que, la dernière fois que la minuterie d'édition a expiré, il n'y avait pas de demandes <i>Publish</i> en file d'attente.
LifetimeCounter	Une valeur qui contient le nombre d'expirations consécutives de la minuterie d'édition sans activité du <i>Client</i> avant que la <i>Subscription</i> soit terminée.
MessageSent	Une valeur booléenne qui est mise à TRUE pour signifier que soit un <i>NotificationMessage</i> , soit un <i>Message</i> d'entretien a été envoyé sur <i>Subscription</i> . Il s'agit d'un fanion qui est utilisé pour assurer qu'un <i>NotificationMessage</i> ou un <i>Message</i> d'entretien est envoyé la première fois que la minuterie d'édition expire.
NotificationsAvailable	Une valeur booléenne qui est mise à TRUE seulement lorsqu'il y a au moins un <i>MonitoredItem</i> qui est dans le mode production de rapport et qui a une <i>Notification</i> en file d'attente ou lorsqu'il y a au moins un élément à rapporter dont l'élément déclencheur a déclenché et a une <i>Notification</i> en file d'attente. La transition de cette <i>Variable</i> d'état de FALSE à TRUE crée l' <i>Événement</i> «New <i>Notification</i> Queued» (Nouvelle notification placée en file d'attente) dans la table d'états.
PublishingEnabled	Le paramètre qui demande que l'édition soit activée ou désactivée.
PublishingReqQueued	Une valeur booléenne qui est mise à TRUE seulement lorsqu'il y a un <i>Message</i> de demande <i>Publish</i> mis en file d'attente de <i>Subscription</i> .
RequestedMessageFound	Une valeur booléenne qui est mise à TRUE seulement lorsque le <i>Message</i> dont la retransmission est demandée a été trouvé dans la file d'attente de retransmission.
SeqNum	La valeur qui enregistre la valeur du numéro de séquence utilisé dans les <i>NotificationMessages</i> .
SubscriptionAssignedToClient	Une valeur booléenne qui est mise à TRUE seulement lorsque <i>Subscription</i> dont la suppression est demandée est affecté au <i>Client</i> qui a émis la demande. Un <i>Subscription</i> est affecté au <i>Client</i> qui l'a créé. Cette affectation peut seulement être changée par l'achèvement réussi du <i>Service</i> TransferSubscriptions.

5.13.1.4 Fonctions

Les fonctions d'action sont définies dans l'ordre alphabétique au Tableau 82.

Tableau 82 – Fonctions

Fonction	Description
ClientValidated()	Une fonction booléenne qui retourne TRUE seulement lorsque le <i>Client</i> qui présente une demande de TransferSubscriptions opère pour le compte du même utilisateur et prend en charge les mêmes <i>Profiles</i> que le <i>Client</i> de la <i>Session</i> précédente.
CreateNotificationMsg()	Incrémenter le SeqNum et créer un <i>NotificationMessage</i> à partir des <i>MonitoredItems</i> affectés à <i>Subscription</i> . Sauvegarder dans la file d'attente de retransmission le <i>NotificationMessage</i> nouvellement créé. Si toutes les <i>Notifications</i> disponibles peuvent être envoyées dans la réponse <i>Publish</i> , la <i>Variable</i> d'état MoreNotifications est mise à FALSE. Autrement, elle est mise à TRUE.
CreateSubscription()	Tenter de créer un <i>Subscription</i> .
DeleteAckedNotificationMsgs()	Supprimer de la file d'attente de retransmission les <i>NotificationMessages</i> qui ont été acquittés par la demande.
DeleteClientPublReqQueue()	Vider la file d'attente des demandes <i>Publish</i> pour le <i>Client</i> qui envoie la demande DeleteSubscriptions, s'il n'y a plus de <i>Subscriptions</i> affectés au <i>Client</i> en question.
DeleteMonitoredItems()	Supprimer tous les <i>MonitoredItems</i> affectés à la <i>Subscription</i> .
DequeuePublishReq()	Sortir de la file d'attente une demande d'édition selon la règle du premier entré premier sorti. Valider si la demande d'édition est encore valide en vérifiant le timeoutHint dans le RequestHeader. Si la demande a dépassé le délai, envoyer un résultat de service <i>Bad_Timeout</i> pour la demande et sortir de la file d'attente une autre demande d'édition. ResetLifetimeCounter()
EnqueuePublishingReq()	Placer en file d'attente la demande d'édition.
InitializeSubscription()	ResetLifetimeCounter() MoreNotifications = FALSE PublishRateChange = FALSE PublishingEnabled = valeur du paramètre publishingEnabled dans la demande CreateSubscription PublishingReqQueued = FALSE SeqNum = 0 SetSession() StartPublishingTimer()
IssueStatusChangeNotification()	Émettre un <i>notificationMessage StatusChangeNotification</i> avec un code de statut pour le changement de statut du <i>Subscription</i> . Le type <i>notificationMessage StatusChangeNotification</i> est défini en 7.19.4. Le code de statut <i>Bad_Timeout</i> est utilisé si la durée de vie expire et <i>Good_SubscriptionTransferred</i> est utilisé si les <i>Subscriptions</i> ont été transférés à une autre <i>Session</i> .
ResetKeepAliveCounter()	Réinitialiser le compteur d'entretien au compte d'entretien maximal de <i>Subscription</i> . Le compte d'entretien maximal est établi par le <i>Client</i> lorsque <i>Subscription</i> est créé et peut être modifié en utilisant le <i>Service ModifySubscription</i> .
ResetLifetimeCounter()	Réinitialiser la <i>Variable</i> LifetimeCounter à la valeur spécifiée pour la durée de vie d'un <i>Subscription</i> dans le <i>Service CreateSubscription</i> (5.13.2).
ReturnKeepAlive()	CreateKeepAliveMsg() ReturnResponse()
ReturnNegativeResponse ()	Retourner une réponse <i>Service</i> indiquant l'erreur de niveau <i>Service</i> appropriée. Il n'est retourné aucun paramètre autre que le responseHeader qui contient le <i>StatusCode</i> de niveau <i>Service</i> .
ReturnNotifications()	CreateNotificationMsg() ReturnResponse() If (MoreNotifications == TRUE) && (PublishingReqQueued == TRUE) { DequeuePublishReq() Boucler de nouveau à travers cette fonction. }
ReturnResponse()	Retourner la réponse appropriée, établissant les valeurs appropriées de paramètres <i>StatusCodes</i> définis pour le <i>Service</i> .
SessionChanged()	Une fonction booléenne qui retourne TRUE seulement lorsque la <i>Session</i> utilisée pour envoyer une demande TransferSubscriptions est différente de la <i>Session Client</i> actuellement associée à <i>Subscription</i> .
SetPublishingEnabled ()	Mettre la <i>Variable</i> d'état PublishingEnabled à la valeur du paramètre publishingEnabled reçu dans la demande.
SetSession	Établir les informations de <i>Session</i> pour que <i>Subscription</i> corresponde à la <i>Session</i> sur laquelle la demande TransferSubscriptions a été émise.
StartPublishingTimer()	Démarrer ou redémarrer la minuterie d'édition et décrémenter la <i>Variable</i> LifetimeCounter.
UpdateSubscriptionParams()	Négocier et mettre à jour les paramètres de <i>Subscription</i> . Si le nouvel intervalle d'entretien est inférieur à la valeur courante du compteur d'entretien, exécuter les fonctions ResetKeepAliveCounter() et ResetLifetimeCounter().

5.13.2 Créer un abonnement (CreateSubscription)

5.13.2.1 Description

Ce *Service* est utilisé pour créer un *Subscription*. Les *Subscriptions* surveillent un jeu de *MonitoredItems* pour détecter des *Notifications* et les retourner au *Client* en réponse aux demandes *Publish*.

Les valeurs de demande illégales pour les paramètres pouvant être révisés ne génèrent pas d'erreur. En revanche le *Serveur* choisit des valeurs par défaut et les indique dans le paramètre révisé correspondant.

5.13.2.2 Paramètres

Le Tableau 83 définit les paramètres du *Service*.

Tableau 83 – Paramètres du service CreateSubscription

Nom	Type	Description
Request		
requestHeader	Request Header	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).
requestedPublishingInterval	Duration	Cet intervalle définit la fréquence cyclique à laquelle il est demandé à <i>Subscription</i> de retourner des <i>Notifications</i> au <i>Client</i> . Cet intervalle est exprimé en millisecondes. Cet intervalle est représenté par la minuterie d'édition dans la table d'états de <i>Subscription</i> (voir 5.13.1.2). La valeur négociée pour ce paramètre retournée dans la réponse est utilisée comme intervalle d'échantillonnage par défaut pour les <i>MonitoredItems</i> affectés à ce <i>Subscription</i> . Si la valeur demandée est 0 ou négative, le <i>Serveur</i> doit réviser avec l'intervalle d'édition pris en charge le plus rapide.
requestedLifetimeCount	Counter	Compte de durée de vie demandé (voir 7.5 pour la définition de <i>Counter</i>). Le compte de durée de vie doit être au minimum égal à trois fois le compte d'entretien. Lorsque la minuterie d'édition a expiré ce nombre de fois sans qu'une demande <i>Publish</i> soit disponible pour envoyer un <i>NotificationMessage</i> , <i>Subscription</i> doit être supprimé par le <i>Serveur</i> .
requestedMaxKeepAliveCount	Counter	Compte d'entretien maximal demandé (voir 7.5 pour la définition de <i>Counter</i>). Lorsque la minuterie d'édition a expiré ce nombre de fois sans exiger qu'un moindre <i>NotificationMessage</i> soit envoyé, <i>Subscription</i> envoie un <i>Message</i> d'entretien au <i>Client</i> . La valeur négociée pour ce paramètre est retournée dans la réponse. Si la valeur demandée est 0, le <i>Serveur</i> doit réviser avec le compte d'entretien le plus petit pris en charge.
maxNotificationsPerPublish	Counter	Le nombre maximal des notifications que le <i>Client</i> souhaite recevoir dans une seule réponse <i>Publish</i> . Une valeur de zéro indique qu'il n'y a pas de limite. Le nombre de notifications par <i>Publish</i> est la somme de <i>monitoredItems</i> dans la <i>DataChangeNotification</i> et d'événements dans l' <i>EventNotificationList</i> .
publishingEnabled	Boolean	Un paramètre <i>Boolean</i> ayant les valeurs suivantes: TRUE l'édition est activée pour <i>Subscription</i> . FALSE l'édition est désactivée pour <i>Subscription</i> . La valeur de ce paramètre n'altère pas la valeur de l' <i>Attribut</i> de mode de surveillance des <i>MonitoredItems</i> .
priority	Byte	Indique la priorité relative de <i>Subscription</i> . Lorsqu'il est nécessaire que plus d'un <i>Subscription</i> envoie des <i>Notifications</i> , il convient que le <i>Serveur</i> sorte de la file d'attente une demande <i>Publish</i> adressée à <i>Subscription</i> ayant le nombre <i>priority</i> le plus élevé. Pour les <i>Subscriptions</i> d'égale <i>priority</i> , il convient que le <i>Serveur</i> sorte de la file d'attente les demandes <i>Publish</i> en «round-robin» (chacune à tour de rôle). Il convient qu'un <i>Client</i> qui n'exige pas de réglages de priorité spéciaux mette cette valeur à zéro.
Response		
responseHeader	Response Header	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).
subscriptionId	IntegerId	L'identificateur affecté par le <i>Serveur</i> pour le <i>Subscription</i> (voir 7.13 pour la définition d' <i>IntegerId</i>). Cet identificateur doit être unique pour le <i>Serveur</i> tout entier, pas uniquement pour la <i>Session</i> , afin de permettre le transfert d'un <i>Subscription</i> vers une autre <i>Session</i> au moyen du service <i>TransferSubscriptions</i> .

Nom	Type	Description
Request		
revisedPublishingInterval	Duration	L'intervalle d'édition réel que le <i>Serveur</i> utilisera, exprimé en millisecondes. Il convient que le <i>Serveur</i> essaie d'honorer la demande du <i>Client</i> pour ce paramètre, mais il peut négocier cette valeur à la hausse ou à la baisse pour satisfaire à ses propres contraintes.
revisedLifetimeCount	Counter	La durée de vie d'un <i>Subscription</i> doit être au moins égale à trois fois l'intervalle d'entretien négocié par le <i>Serveur</i> .
revisedMaxKeepAliveCount	Counter	Le compte d'entretien maximal réel. (voir 7.5 pour la définition de <i>Counter</i>). Il convient que le <i>Serveur</i> essaie d'honorer la demande du <i>Client</i> pour ce paramètre, mais il peut négocier cette valeur à la hausse ou à la baisse pour satisfaire à ses propres contraintes.

5.13.2.3 Résultats des services

Le Tableau 84 définit les résultats des *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 84 – Codes de résultats des services CreateSubscription

Id symbolique	Description
Bad_TooManySubscriptions	Le <i>Serveur</i> a atteint le nombre maximal d'abonnements.

5.13.3 Modifier un abonnement (ModifySubscription)

5.13.3.1 Description

Ce *Service* est utilisé pour modifier un *Subscription*.

Les valeurs de demande illégales pour les paramètres pouvant être révisés ne génèrent pas d'erreur. En revanche le *Serveur* choisit des valeurs par défaut et les indique dans le paramètre révisé correspondant.

Les modifications apportées aux réglages de *Subscription* doivent être appliquées immédiatement par le *Serveur*. Elles prennent effet le plus tôt possible sans toutefois dépasser une durée égale au double du nouveau *revisedPublishingInterval*.

5.13.3.2 Paramètres

Le Tableau 85 définit les paramètres du *Service*.

Tableau 85 – Paramètres du service ModifySubscription

Nom	Type	Description
Request		
requestHeader	RequestHeader	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).
subscriptionId	IntegerId	L'identificateur affecté par le <i>Serveur</i> pour <i>Subscription</i> (voir 7.13 pour la définition d' <i>IntegerId</i>).
requestedPublishingInterval	Duration	Cet intervalle définit la fréquence cyclique à laquelle il est demandé au <i>Subscription</i> de retourner des <i>Notifications</i> au <i>Client</i> . Cet intervalle est exprimé en millisecondes. Cet intervalle est représenté par la minuterie d'édition dans la table d'états de <i>Subscription</i> (voir 5.13.1.2). La valeur négociée pour ce paramètre retournée dans la réponse est utilisée comme intervalle d'échantillonnage par défaut pour les <i>MonitoredItems</i> affectés à ce <i>Subscription</i> . Si la valeur demandée est 0 ou négative, le <i>Serveur</i> doit réviser l'intervalle d'édition le plus rapide pris en charge.
requestedLifetimeCount	Counter	Compte de durée de vie demandé (voir 7.5 pour la définition de <i>Counter</i>). Le compte de durée de vie doit être au minimum égal à trois fois le compte d'entretien. Lorsque la minuterie d'édition a expiré ce nombre de fois sans qu'une demande <i>Publish</i> soit disponible pour envoyer un <i>NotificationMessage</i> , <i>Subscription</i> doit être supprimé par le <i>Serveur</i> .
requestedMaxKeepAliveCount	Counter	Le compte d'entretien maximal demandé. (voir 7.5 pour la définition de <i>Counter</i>). Lorsque la minuterie d'édition a expiré ce nombre de fois

Nom	Type	Description
Request		
		sans exiger qu'un moindre <i>NotificationMessage</i> soit envoyé, <i>Subscription</i> envoie un <i>Message</i> d'entretien au <i>Client</i> . La valeur négociée pour ce paramètre est retournée dans la réponse. Si la valeur demandée est 0, le <i>Serveur</i> doit réviser avec le compte d'entretien le plus petit pris en charge.
maxNotificationsPerPublish	Counter	Le nombre maximal des notifications que le <i>Client</i> souhaite recevoir dans une seule réponse <i>Publish</i> . Une valeur de zéro indique qu'il n'y a pas de limite.
Priority	Byte	Indique la priorité relative de <i>Subscription</i> . Lorsqu'il est nécessaire que plus d'un <i>Subscription</i> envoie des <i>Notifications</i> , il convient que le <i>Serveur</i> sorte de la file d'attente une demande <i>Publish</i> adressée au <i>Subscription</i> ayant le nombre <i>priority</i> le plus élevé. Pour les <i>Subscriptions</i> d'égale <i>priority</i> , il convient que le <i>Serveur</i> sorte de la file d'attente les demandes <i>Publish</i> en «round-robin» (chacune à tour de rôle). Tout <i>Subscription</i> qui a besoin d'envoyer un <i>Message</i> d'entretien doit avoir la préséance quelle que soit sa <i>priority</i> , afin d'éviter que <i>Subscription</i> n'expire. Il convient qu'un <i>Client</i> qui n'exige pas de réglages de priorité spéciaux mette cette valeur à zéro.
Response		
responseHeader	ResponseHeader	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).
revisedPublishingInterval	Duration	L'intervalle d'édition réel que le <i>Serveur</i> utilisera, exprimé en millisecondes. Il convient que le <i>Serveur</i> essaie d'honorer la demande du <i>Client</i> pour ce paramètre, mais il peut négocier cette valeur à la hausse ou à la baisse pour satisfaire à ses propres contraintes.
revisedLifetimeCount	Counter	La durée de vie de <i>Subscription</i> doit être au moins égale à trois fois l'intervalle d'entretien négocié avec le <i>Serveur</i> .
revisedMaxKeepAliveCount	Counter	Le compte d'entretien maximal réel (voir 7.5 pour la définition de <i>Counter</i>). Il convient que le <i>Serveur</i> essaie d'honorer la demande du <i>Client</i> pour ce paramètre, mais il peut négocier cette valeur à la hausse ou à la baisse pour satisfaire à ses propres contraintes.

5.13.3.3 Résultats des services

Le Tableau 86 définit les résultats des *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 86 – Codes de résultats des services ModifySubscription

Id symbolique	Description
Bad_SubscriptionIdInvalid	Voir le Tableau 165 pour la description de ce code de résultat.

5.13.4 Demander un mode d'édition (SetPublishingMode)

5.13.4.1 Description

Ce *Service* est utilisé pour permettre l'envoi de *Notifications* sur un ou plusieurs *Subscriptions*.

5.13.4.2 Paramètres

Le Tableau 87 définit les paramètres du *Service*.

Tableau 87 – Paramètres du service SetPublishingMode

Nom	Type	Description
Request		
requestHeader	RequestHeader	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).
publishingEnabled	Boolean	Un paramètre <i>Boolean</i> ayant les valeurs suivantes: TRUE l'édition de <i>NotificationMessages</i> est activée pour <i>Subscription</i> . FALSE l'édition de <i>NotificationMessages</i> est désactivée pour <i>Subscription</i> . La valeur de ce paramètre n'altère pas la valeur de l' <i>Attribut</i> de mode de surveillance des <i>MonitoredItems</i> . La mise de cette valeur à FALSE n'interrompt

Nom	Type	Description
		pas l'envoi de <i>Messages</i> d'entretien.
subscriptionIds []	IntegerId	Liste d'identificateurs affectés par le <i>Serveur</i> pour les <i>Subscriptions</i> à activer ou désactiver (voir 7.13 pour la définition d' <i>IntegerId</i>).
Response		
responseHeader	ResponseHeader	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).
results []	StatusCode	Liste des <i>StatusCodes</i> pour les <i>Subscriptions</i> à activer/désactiver (voir 7.33 pour la définition des <i>StatusCodes</i>). La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>subscriptionIds</i> .
diagnosticInfos []	DiagnosticInfo	Liste d'informations de diagnostic pour les <i>Subscriptions</i> à activer/désactiver (voir 7.8 pour la définition de <i>DiagnosticInfo</i>). La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>subscriptionIds</i> . Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été rencontrée dans le traitement de la demande.

5.13.4.3 Résultats des services

Le Tableau 88 définit les résultats des *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 88 – Codes de résultats des services SetPublishingMode

Id symbolique	Description
Bad_NothingToDo	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_TooManyOperations	Voir le Tableau 165 pour la description de ce code de résultat.

5.13.4.4 StatusCodes

Le Tableau 89 définit les valeurs pour le paramètre *results* qui sont spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 166.

Tableau 89 – Codes de résultats de niveau opération de SetPublishingMode

Id symbolique	Description
Bad_SubscriptionIdInvalid	Voir le Tableau 165 pour la description de ce code de résultat.

5.13.5 Editer (Publish)

5.13.5.1 Description

Ce *Service* est utilisé à deux fins. Premièrement, il est utilisé pour acquitter réception des *NotificationMessages* pour un ou plusieurs *Subscriptions*. Deuxièmement, il est utilisé pour demander au *Serveur* de retourner un *NotificationMessage* ou un *Message* d'entretien. Comme les demandes *Publish* ne sont pas dirigées à un *Subscription* spécifique, elles peuvent être utilisées par n'importe quel *Subscription*. Le paragraphe 5.13.1.2 décrit l'utilisation du *Service Publish*.

Les stratégies du *Client* pour émettre des demandes *Publish* peuvent varier en fonction des retards réseau entre le *Client* et le *Serveur*. Dans de nombreux cas, le *Client* peut souhaiter émettre une demande *Publish* immédiatement après avoir créé un *Subscription* et, par la suite, immédiatement après avoir reçu une réponse *Publish*.

Dans d'autres cas, notamment dans les réseaux à temps d'attente élevée, le *Client* peut souhaiter placer en cascade les demandes *Publish* pour assurer une production cyclique de rapports par le *Serveur*. Le placement en cascade implique d'envoyer plus d'une demande *Publish* pour chaque *Subscription* avant de recevoir une réponse. Par exemple, si le réseau introduit entre le *Client* et le *Serveur* un retard de 5 secondes et l'intervalle d'édition pour une *Subscription* est 1 seconde le *Client* aura à émettre des demandes *Publish* toutes les secondes au lieu d'attendre qu'une réponse soit reçue avant d'envoyer la prochaine demande.

Il convient qu'un *Serveur* limite le nombre de demandes *Publish* actives afin d'éviter que le nombre soit infini, car les demandes *Publish* sont censées être placées en file d'attente dans le *Serveur*. Mais un *Serveur* doit accepter plus de demandes *Publish* mises en file d'attente que d'*Abonnements* créés. Il est attendu qu'un *Serveur* prenne en charge plusieurs demandes *Publish* par *Subscription*. Lorsqu'un *Serveur* reçoit une nouvelle demande *Publish* qui dépasse sa limite, il doit sortir de la file d'attente la demande *Publish* la plus ancienne et retourner une réponse avec le résultat mis à *Bad_TooManyPublishRequests*. Si le *Client* reçoit ce *Service* pour une demande *Publish*, il ne doit pas émettre d'autre demande *Publish* avant que l'une de ses demandes *Publish* en cours ne soit retournée du *Serveur*.

Les *Clients* peuvent limiter la taille des réponses *Publish* avec le paramètre *maxNotificationsPerPublish* passé au *Service CreateSubscription*. Cependant, cela peut encore produire un message trop volumineux pour être traité par le *Client* ou le *Serveur*. Dans cette situation, le *Client* s'apercevra soit que le *Canal Sécurisé* passe à un état de défaut et a besoin d'être rétabli, soit que la réponse *Publish* retourne une erreur et l'appel du *Service Republish* retourne aussi une erreur. Si l'une de ces situations se produit, le *Client* aura à ajuster ses limites de traitement de message ou ses paramètres pour *Subscription* et/ou les *MonitoredItems*.

Le réglage des informations de diagnostic de retour dans l'en-tête de demande des *CreateMonitoredItems* ou le dernier *Service ModifyMonitoredItems* est appliqué aux *MonitoredItems* (Éléments surveillés) et sert de réglages d'informations de diagnostic lors de l'envoi de Notifications dans la réponse *Publish*.

5.13.5.2 Paramètres

Le Tableau 90 définit les paramètres du *Service*.

Tableau 90 – Paramètres du service Publish

Nom	Type	Description
Request		
requestHeader	RequestHeader	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).
subscriptionAcknowledgements []	SubscriptionAcknowledgement	La liste des acquittements pour un ou plusieurs <i>Subscriptions</i> . Cette liste peut contenir plusieurs acquittements pour le même <i>Subscription</i> (plusieurs entrées ayant le même <i>subscriptionId</i>). Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.
subscriptionId	IntegerId	L'identificateur affecté par le <i>Serveur</i> pour un <i>Subscription</i> (voir 7.13 pour la définition d' <i>IntegerId</i>).
sequenceNumber	Counter	Le numéro de séquence acquitté (voir 7.5 pour la définition de <i>Counter</i>). Le <i>Serveur</i> peut supprimer de sa file d'attente de retransmission le <i>Message</i> ayant ce numéro de séquence.
Response		
responseHeader	ResponseHeader	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).
subscriptionId	IntegerId	L'identificateur affecté par le <i>Serveur</i> pour <i>Subscription</i> pour lequel les <i>Notifications</i> sont retournées (voir 7.13 pour la définition d' <i>IntegerId</i>). La valeur 0 est utilisée pour indiquer qu'il n'y avait pas de <i>Subscriptions</i> définis pour lesquels une réponse peut être envoyée.
availableSequenceNumbers []	Counter	Une liste de plages de numéros de séquence qui identifient des <i>NotificationMessages</i> non acquittés qui sont disponibles pour la retransmission à partir de la file d'attente de retransmission d'un <i>Subscription</i> . Cette liste est élaborée après traitement des acquittements dans la réponse (voir 7.5 pour la définition de <i>Counter</i>).
moreNotifications	Boolean	Un paramètre <i>Boolean</i> ayant les valeurs suivantes: TRUE le nombre des <i>Notifications</i> qui étaient prêtes à être envoyées n'a pas pu être envoyé dans une seule réponse. FALSE toutes les <i>Notifications</i> qui étaient prêtes sont incluses dans la réponse.
notificationMessage	NotificationMessage	Le <i>NotificationMessage</i> qui contient la liste des <i>Notifications</i> . Le type de paramètre <i>NotificationMessage</i> est spécifié en 7.20.
results []	StatusCode	Liste de résultats pour les acquittements (voir 7.33 pour la définition de <i>StatusCode</i>). La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>subscriptionAcknowledgements</i> .
diagnosticInfos []	DiagnosticInfo	Liste d'informations de diagnostic pour les acquittements (voir 7.8 pour la définition de <i>DiagnosticInfo</i>). La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>subscriptionAcknowledgements</i> . Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été rencontrée dans le traitement de la demande.

5.13.5.3 Résultats des services

Le Tableau 91 définit les résultats des *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 91 – Codes de résultats des services Publish

Id symbolique	Description
Bad_TooManyPublishRequests	Le serveur a atteint le nombre maximal de demandes d'édition placées en file d'attente.
Bad_NoSubscription	Aucun abonnement n'est disponible pour cette session.

5.13.5.4 StatusCodes

Le Tableau 92 définit les valeurs pour le paramètre *results* qui sont spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 166.

Tableau 92 – Codes de résultats de niveau opération de Publish

Id symbolique	Description
Bad_SubscriptionIdInvalid	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_SequenceNumberUnknown	Le numéro de séquence est inconnu du serveur.

5.13.6 Éditer à nouveau (Republish)

5.13.6.1 Description

Ce *Service* demande au *Subscription* de rééditer un *NotificationMessage* à partir de sa file d'attente de retransmission. Si le *Serveur* n'a pas le *Message* demandé dans sa file d'attente de retransmission, il retourne une réponse d'erreur.

Voir 5.13.1.2 pour la description détaillée du comportement de ce *Service*.

Voir 6.5 pour une description des problèmes et des stratégies de la gestion de reconnexion et de *Republish*.

5.13.6.2 Paramètres

Le Tableau 93 définit les paramètres du *Service*.

Tableau 93 – Paramètres du service Republish

Nom	Type	Description
Request		
requestHeader	RequestHeader	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).
subscriptionId	IntegerId	L'identificateur affecté par le <i>Serveur</i> pour <i>Subscription</i> à rééditer (voir 7.13 pour la définition d' <i>IntegerId</i>).
retransmitSequence Number	Counter	Le numéro de séquence d'un <i>NotificationMessage</i> spécifique à rééditer (voir 7.5 pour la définition de <i>Counter</i>).
Response		
responseHeader	ResponseHeader	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).
notificationMessage	Notification Message	Le <i>NotificationMessage</i> demandé. Le type de paramètre <i>NotificationMessage</i> est spécifié en 7.18.

5.13.6.3 Résultats des services

Le Tableau 94 définit les résultats des *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 94 – Codes de résultats des services Republish

Id symbolique	Description
Bad_SubscriptionIdInvalid	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_MessageNotAvailable	Le message demandé n'est plus disponible.

5.13.7 Transférer un abonnement (TransferSubscriptions)

5.13.7.1 Description

Ce *Service* est utilisé pour transférer un *Subscription* et ses *MonitoredItems* d'une *Session* à l'autre. Par exemple, un *Client* peut avoir besoin de rouvrir une *Session* et ensuite de transférer ses *Subscriptions* à la *Session* en question. Il peut aussi être utilisé par un *Client* pour prendre un *Subscription* à un autre *Client* en transférant le *Subscription* à sa *Session*.

L'*authenticationToken* contenu dans l'en-tête de demande identifie la *Session* à laquelle le *Subscription* et les *MonitoredItems* doivent être transférés. Le *Serveur* doit valider que le *Client* de la *Session* en question opère pour le compte du même utilisateur et que le *Client* potentiellement nouveau prend en charge les *Profiles* qui sont nécessaires pour le *Subscription*. Si le *Serveur* transfère le *Subscription*, il retourne les numéros de séquence des *NotificationMessages* qui sont disponibles pour une retransmission. Il convient que le *Client* acquitte tous les *Messages* de la liste pour lesquels il ne demandera pas de retransmission.

Si le *Serveur* transfère le *Subscription* à la nouvelle *Session*, le *Serveur* doit émettre un *notificationMessage StatusChangeNotification* avec le code de statut *Good_SubscriptionTransferred* à destination de l'ancienne *Session*. Le type *notificationMessage StatusChangeNotification* est défini en 7.19.4.

5.13.7.2 Paramètres

Le Tableau 95 définit les paramètres du *Service*.

Tableau 95 – Paramètres du service TransferSubscriptions

Nom	Type	Description
Request		
requestHeader	RequestHeader	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).
subscriptionIds []	IntegerId	Liste d'identificateurs pour les <i>Subscriptions</i> à transférer au nouveau <i>Client</i> (voir 7.13 pour la définition d' <i>IntegerId</i>). Ces identificateurs sont transférés du <i>Client</i> principal vers un <i>Client</i> de secours via des mécanismes externes.
sendInitialValues	Boolean	Un paramètre <i>Boolean</i> ayant les valeurs suivantes: TRUE la première réponse <i>Publish</i> après l'appel de <i>TransferSubscriptions</i> doit contenir les valeurs courantes de tous les <i>Monitored Items</i> (éléments surveillés) dans l'Abonnement dans lequel le Mode "Monitoring" est mis à "Reporting". FALSE la première réponse <i>Publish</i> après l'appel de <i>TransferSubscriptions</i> doit seulement contenir les changements de valeur depuis que la dernière réponse <i>Publish</i> a été envoyée. Ce paramètre ne s'applique qu'aux <i>MonitoredItems</i> utilisés pour surveiller les modifications d'Attribut.
Response		
responseHeader	ResponseHeader	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).
results []	TransferResult	Liste de résultats pour les <i>Subscriptions</i> à transférer. La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>subscriptionIds</i> . Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.
statusCode	StatusCode	<i>StatusCode</i> pour chaque <i>Subscription</i> à transférer (voir 7.33 pour la définition de <i>StatusCode</i>).
availableSequence Numbers []	Counter	Une liste de plages de numéros de séquence qui identifie des <i>NotificationMessages</i> qui sont dans la file d'attente de retransmission d'un <i>Subscription</i> . Ce paramètre est null si le transfert d'un <i>Subscription</i> a échoué. Le type <i>Counter</i> est défini en 7.5.
diagnosticInfos []	DiagnosticInfo	Liste d'informations de diagnostic pour les <i>Subscriptions</i> à transférer (voir 7.8 pour la définition de <i>DiagnosticInfo</i>). La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>subscriptionIds</i> . Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été rencontrée dans le traitement de la demande.

5.13.7.3 Résultats des services

Le Tableau 96 définit les résultats des *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 96 – Codes de résultats des services TransferSubscriptions

Id symbolique	Description
Bad_NothingToDo	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_TooManyOperations	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_InsufficientClientProfile	Le <i>Client</i> de la <i>Session</i> courante ne prend pas en charge un ou plusieurs <i>Profiles</i> nécessaires à un <i>Subscription</i> .

5.13.7.4 StatusCodes

Le Tableau 97 définit les valeurs de niveau opération pour le paramètre *statusCode* qui sont spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 166.

Tableau 97 – Codes de résultats de niveau opération de TransferSubscriptions

Id symbolique	Description
Bad_SubscriptionIdInvalid	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_UserAccessDenied	Voir le Tableau 165 pour la description de ce code de résultat. Le <i>Client</i> de la <i>Session</i> courante n'opère pas pour le compte du même utilisateur que la <i>Session</i> qui est propriétaire d'un <i>Subscription</i> .

5.13.8 Supprimer des abonnements (DeleteSubscriptions)

5.13.8.1 Description

Ce *Service* est invoqué pour supprimer un ou plusieurs *Subscriptions* qui appartiennent à la *Session* du *Client*.

L'achèvement réussi de ce *Service* entraîne la suppression de tous les *MonitoredItems* qui utilisent *Subscription*. S'il s'agit du dernier *Subscription* pour la *Session*, alors toutes les demandes *Publish* encore dans la file d'attente pour cette *Session* sont sorties de la file d'attente et doivent être retournées avec Bad_NoSubscription.

Les *Subscriptions* transférés à une autre *Session* doivent être supprimés par le *Client* qui est propriétaire de la *Session*.

5.13.8.2 Paramètres

Le Tableau 98 définit les paramètres du *Service*.

Tableau 98 – Paramètres du service DeleteSubscriptions

Nom	Type	Description
Request		
requestHeader	RequestHeader	Paramètres communs aux demandes (voir 7.26 pour la définition de <i>RequestHeader</i>).
subscriptionIds []	IntegerId	L'identificateur affecté par le <i>Serveur</i> pour <i>Subscription</i> ((voir 7.13 pour la définition d' <i>IntegerId</i>)).
Response		
responseHeader	ResponseHeader	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).
results []	StatusCode	Liste des <i>StatusCodes</i> pour les <i>Subscriptions</i> à supprimer (voir 7.33 pour la définition de <i>StatusCode</i>). La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>subscriptionIds</i> .
diagnosticInfos []	DiagnosticInfo	Liste d'informations de diagnostic pour les <i>Subscriptions</i> à supprimer (voir 7.8 pour la définition de <i>DiagnosticInfo</i>). La taille et l'ordre de la liste correspondent à la taille et à l'ordre du paramètre de demande <i>subscriptionIds</i> . Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été rencontrée dans le traitement de la demande.

5.13.8.3 Résultats des services

Le Tableau 99 définit les résultats des *Services* spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 165.

Tableau 99 – Codes de résultats des services DeleteSubscriptions

Id symbolique	Description
Bad_NothingToDo	Voir le Tableau 165 pour la description de ce code de résultat.
Bad_TooManyOperations	Voir le Tableau 165 pour la description de ce code de résultat.

5.13.8.4 StatusCodes

Le Tableau 100 définit les valeurs pour le paramètre *results* qui sont spécifiques à ce *Service*. Les *StatusCodes* communs sont définis au Tableau 166.

Tableau 100 – Codes de résultats de niveau opération de DeleteSubscriptions

Id symbolique	Description
Bad_SubscriptionIdInvalid	Voir le Tableau 165 pour la description de ce code de résultat.

6 Comportements des services

6.1 Sécurité

6.1.1 Vue d'ensemble

Les *Services OPC UA* définissent un certain nombre de mécanismes pour satisfaire aux exigences de sécurité décrites dans les grandes lignes dans l'IEC TR 62541-2. Cet article décrit un certain nombre d'importantes procédures de sécurité que les *Applications OPC UA* doivent suivre.

6.1.2 Obtenir et installer un certificat d'instance d'application (Application Instance Certificate)

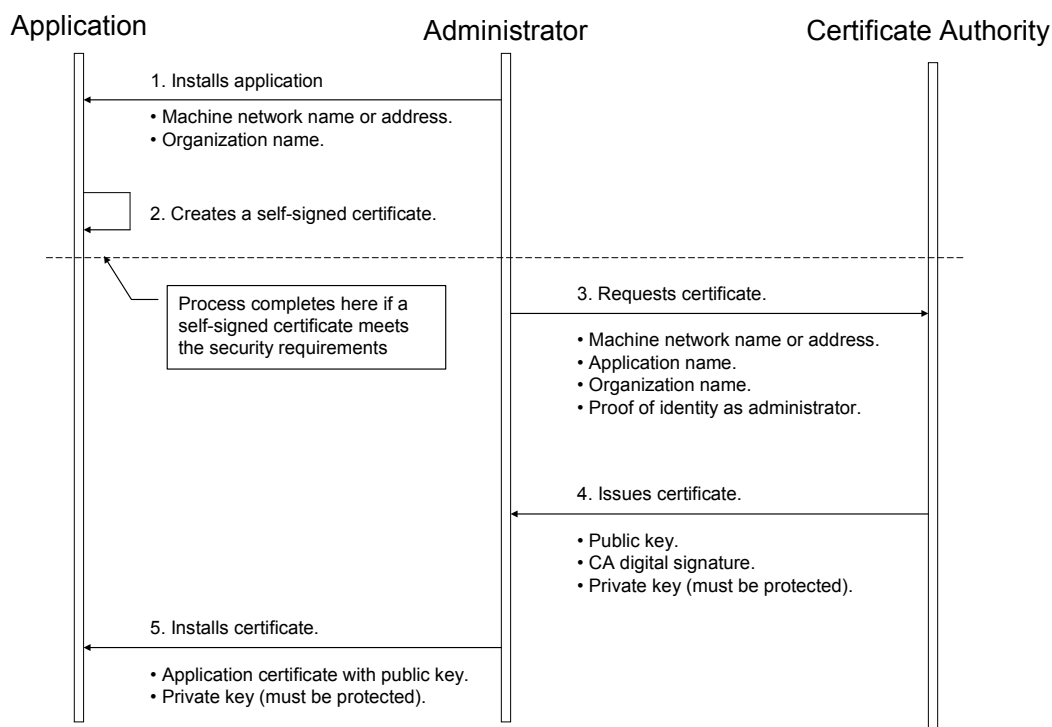
Toutes les *Applications OPC UA* exigent un *Certificat d'Instance d'Application* qui doit contenir les renseignements suivants:

- Le nom du réseau ou l'adresse de l'ordinateur sur lequel l'application s'exécute;
- Le nom de l'organisation qui gère ou est le propriétaire de l'application;
- Le nom de l'application;
- L'URI de l'instance d'application;
- Le nom de l'*Autorité de Certification* qui a émis le *Certificat*;
- La date d'émission et d'expiration du *Certificat*;
- La clé publique émise pour l'application par l'*Autorité de Certification* (CA, *certificate authority*);
- Une signature numérique créée par l'*Autorité de Certification* (CA).

En outre, chaque *Certificat d'Instance d'Application* a une clé privée qu'il convient de conserver dans un endroit uniquement accessible par l'application. Si l'intégrité de cette clé privée est compromise, l'administrateur doit affecter un nouveau *Certificat d'Instance d'Application* et une nouvelle clé privée à l'application.

Ce *Certificat* peut être généré automatiquement lorsque l'application est installée. Dans cette situation, la clé privée affectée au *Certificat* doit être utilisée pour créer la signature du *Certificat*. Les certificats créés de cette manière sont appelés *Certificats* autosignés.

Si l'administrateur responsable de l'application décide qu'un *Certificat* autosigné ne satisfait pas aux exigences de sécurité de l'organisation, il convient que l'administrateur installe un *Certificat* émis par une *Autorité de Certification*. Les étapes impliquées dans la demande du *Certificat d'Instance d'Application* auprès d'une *Autorité de Certification* sont montrées à la Figure 19.



IEC

Anglais	Français
Administrator	Administrateur
Certificate authority	Autorité de certification
Installs application	Installe l'application
Machine network name or address	Nom ou adresse réseau de la machine
Organization name	Nom d'organisation
Creates a self-signed certificate	Crée un certificat autosigné
Process completes here if a self-signed certificate meets the security requirements	Le processus se termine ici si un certificat autosigné satisfait aux exigences relatives à la sécurité
Requests certificate	Demande un certificat
Application name	Nom de l'application
Proof of identity as administrator	Preuve de l'identité comme administrateur
Issues certificate	Émet un certificat
Installs certificate	Installe un certificat
Application certificate with public key	Certificat d'application avec clé publique
Private key (must be protected)	Clé privée (doit être protégée)
Public key	Clé publique
CA digital signature	Signature numérique de CA

Figure 19 – Obtenir et installer un certificat d'instance d'application

La figure ci-dessus illustre les interactions entre l'application, l'*Administrateur* et l'*Autorité de Certification*. L'*Application* est en tant qu'*Application OPC UA* installée sur une seule machine. L'*Administrateur* est la personne chargée de gérer la machine et l'*Application OPC UA*. L'*Autorité de Certification* est une entité qui peut émettre des *Certificats* numériques qui satisfont aux exigences de l'organisation qui déploie l'*Application OPC UA*.

Si l'*Administrateur* décide qu'un *Certificat* autosigné satisfait aux exigences de sécurité pour l'organisation, l'*Administrateur* peut sauter les Étapes 3 à 5. Les fournisseurs d'application

doivent toujours créer un *Certificat* autosigné par défaut au cours du processus d'installation. Chaque *Application OPC UA* doit permettre aux *Administrateurs* de remplacer des *Certificats d'instance d'application* par des *Certificats* qui satisfont à leurs exigences.

Lorsque l'*Administrateur* demande un nouveau *Certificat* auprès d'une *Autorité de Certification*, l'*Autorité de Certification* peut exiger que l'*Administrateur* fournisse la preuve de l'autorisation de demander des *Certificats* pour l'organisation qui sera propriétaire du *Certificat*. Le mécanisme exact utilisé pour fournir cette preuve dépend de l'*Autorité de Certification*.

Les fournisseurs peuvent choisir d'automatiser le processus d'acquisition de *Certificats* auprès d'une autorité. Si tel est le cas, l'*Administrateur* passe toujours par les étapes illustrées à la Figure 19. En revanche, le programme d'installation pour l'application le fait automatiquement et sollicite seulement l'*Administrateur* pour qu'il fournisse les renseignements relatifs à l'instance d'application installée.

6.1.3 Déterminer si un Certificat est Fiable

Les *Applications* ne doivent jamais communiquer avec une autre application en laquelle elles n'ont pas confiance. Une *Application* décide qu'une autre application est fiable en vérifiant si le *Certificat d'instance d'application* de l'autre application est fiable. Les applications doivent s'appuyer sur les listes de *Certificats* fournies par l'*Administrateur* pour déterminer la confiance. Il existe deux listes distinctes: une liste d'*Applications* fiables et une liste d'*Autorités de certification* (CA) fiables. Si une application n'est pas directement fiable (c'est-à-dire que son *Certificat* ne figure pas dans la liste d'applications fiables), l'application doit construire une chaîne de *Certificats* ramenant à une CA fiable.

Lors de la construction d'une chaîne, chaque *Certificat* de la chaîne doit être validé. S'il se produit une quelconque erreur de validation, la vérification de la confiance a échoué. Certaines erreurs de validation ne sont pas critiques, ce qui signifie qu'elles peuvent être éliminées par l'utilisateur d'une *Application* ayant les privilèges appropriés. Les erreurs de validation supprimées sont toujours consignées dans un rapport via un audit (cela signifie qu'un événement d'Audit approprié est soulevé).

La construction d'une chaîne de confiance nécessite l'accès à tous les *Certificats* de la chaîne. Ces *Certificats* peuvent être stockés localement ou ils peuvent être fournis avec le *Certificat* d'application. Le traitement échoue avec `Bad_SecurityChecksFailed` si un *Certificat* de CA ne peut pas être trouvé.

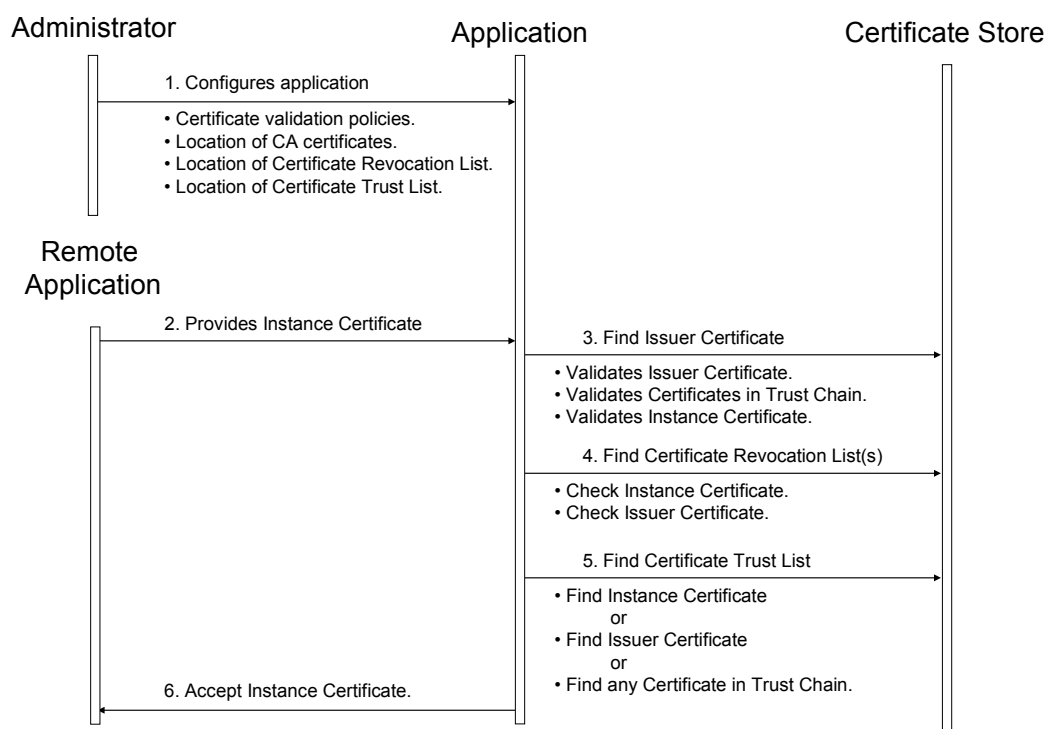
Le Tableau 101 spécifie les étapes utilisées pour valider un *Certificat* et l'ordre dans lequel on doit les suivre. Ces étapes sont répétées pour chaque *Certificat* de la chaîne. Chaque étape de validation a un statut d'erreur et un type d'événement d'audit qui sont uniques et doivent être consignés dans un rapport si la vérification échoue. L'événement d'audit s'ajoute à tout événement d'audit qui a été généré pour le *Service* particulier qui a été invoqué. L'événement d'audit du *Service* dans le texte de son message doit inclure l'*EventID* d'audit de l'*AuditCertificateEventType* (pour plus de détails, voir 6.2). Le traitement s'arrête si une erreur se produit, à moins qu'elle ne soit pas critique et qu'elle ait été éliminée.

Les *ApplicationInstanceCertificates* ne doivent pas être utilisés dans un *Client* ou un *Serveur* jusqu'à ce qu'ils soient évalués et marqués comme fiables. Cela peut se produire de manière automatique par une chaîne de confiance ICP ou de manière hors ligne où le *Certificat* est marqué comme étant fiable par un administrateur après évaluation.

Tableau 101 – Étapes de validation d'un certificat

Étape	Erreur/Événement d'audit	Description
Structure du certificat	Bad_SecurityChecksFailed AuditCertificateInvalidEventType	La structure du <i>Certificat</i> est vérifiée. Cette erreur ne peut pas être supprimée.
Signature	Bad_SecurityChecksFailed AuditCertificateInvalidEventType	Un <i>Certificat</i> avec une signature non valide doit toujours être rejeté. Une signature de <i>Certificat</i> est non valide si le <i>Certificat</i> de l'émetteur est inconnu. Un <i>Certificat</i> autosigné est son propre émetteur.
Vérification de la liste de confiance	Bad_SecurityChecksFailed AuditCertificateUntrustedEventType	Si le <i>Certificat d'Instance d'Application</i> n'est pas fiable et qu'aucun des <i>Certificats</i> de CA dans la chaîne n'est fiable, le résultat de la validation du <i>Certificat</i> doit être <i>Bad_SecurityChecksFailed</i> .
Période de validité	Bad_CertificateTimeInvalid Bad_CertificateIssuerTimeInvalid AuditCertificateExpiredEventType	L'instant actuel doit se situer après le début de la période de validité et avant la fin. Cette erreur peut être supprimée.
Nom de l'hôte	Bad_CertificateHostNameInvalid AuditCertificateDataMismatchEventType	Le <i>HostName</i> dans l'URL utilisé pour se connecter au <i>Serveur</i> doit être le même que l'un des <i>HostNames</i> spécifiés dans le <i>Certificat</i> . Cette vérification est ignorée pour les <i>Certificats</i> de CA. Cette erreur peut être supprimée.
URI	Bad_CertificateUriInvalid AuditCertificateDataMismatchEventType	Les <i>Certificats d'Application</i> et de <i>Logiciel</i> contiennent un URI d'application ou de produit qui doit correspondre à l'URI spécifié dans l' <i>ApplicationDescription</i> fournie avec le <i>Certificat</i> . Cette vérification est ignorée pour les <i>Certificats</i> de CA. Cette erreur ne peut pas être supprimée. Le <i>gatewayServerUri</i> est utilisé pour valider un <i>Application Certificate</i> lors de la connexion à un <i>Gateway Server</i> (voir 7.1).
Utilisation du certificat	Bad_CertificateUseNotAllowed Bad_CertificateIssuerUseNotAllowed AuditCertificateMismatchEventType	Chaque <i>Certificat</i> a un ensemble d'utilisations pour le <i>Certificat</i> (voir IEC 62541-6). Ces utilisations doivent correspondre à l'utilisation demandée pour le <i>Certificat</i> (à savoir <i>Application</i> , <i>Logiciel</i> ou <i>CA</i>). Cette erreur peut être supprimée à moins que le <i>Certificat</i> n'indique que l'utilisation est obligatoire.
Trouver la Liste d'annulation	Bad_CertificateRevocationUnknown Bad_CertificateIssuerRevocationUnknown AuditCertificateRevokedEventType	Chaque <i>Certificat</i> de CA peut avoir une liste d'annulation. Cette vérification échoue si cette liste n'est pas disponible (à savoir, une interruption du réseau empêche l'application d'accéder à la liste). Aucune erreur n'est rapportée si l' <i>Administrateur</i> désactive les vérifications d'annulation pour un <i>Certificat</i> de CA. Cette erreur peut être supprimée.
Vérification d'annulation	Bad_CertificateRevoked Bad_CertificateIssuerRevoked AuditCertificateRevokedEventType	Le <i>Certificat</i> a été annulé et ne peut pas être utilisé. Cette erreur ne peut pas être supprimée.

Les *Certificats* sont en général placés dans un lieu central appelé *CertificateStore*. La Figure 20 illustre les interactions entre l'*Application*, l'*Administrateur* et le *CertificateStore*. Le *CertificateStore* peut être sur la machine locale ou dans quelque serveur central. Les mécanismes exacts utilisés pour accéder au *CertificateStore* dépendent de l'application et de l'environnement ICP installé par l'*Administrateur*.



IEC

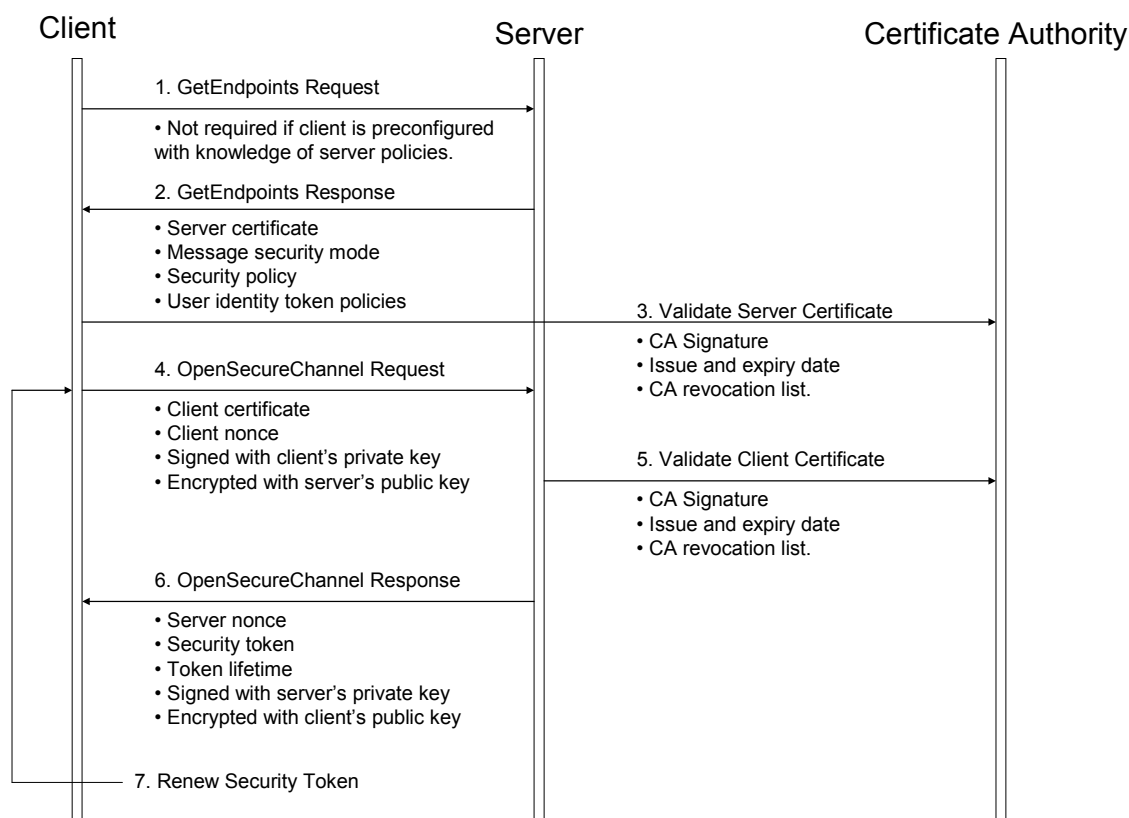
Anglais	Français
Administrator	Administrateur
Certificate store	Stockage de certificats
Configures application	Configure l'application
Certificate validation policies	Politiques de validation de certificat
Location of CA certificates	Emplacement des certificats de CA
Location of Certificate Revocation List	Emplacement de la Liste d'annulation de certificats
Location of Certificate Trust List	Emplacement de la Liste de certificats de confiance
Remote application	Application distante
Provides Instance Certificate	Fournit un certificat d'instance
Find Issuer Certificate	Trouver le certificat de l'émetteur
Validates Issuer Certificate	Valide le certificat d'émetteur
Validates certificates in Trust Chain	Valide les certificats dans la chaîne de confiance
Validates Instance Certificate	Valide le certificat d'instance
Find Certificate Revocation List(s)	Trouver la/les liste(s) d'annulation de certificats
Check Instance Certificate	Vérifier le certificat d'instance
Check Issuer Certificate	Vérifier le certificat d'émetteur
Find Certificate Trust List	Trouver la Liste de certificats de confiance
Find Instance Certificate	Trouver le certificat d'instance
Or	Ou
Find Issuer Certificate	Trouver le certificat de l'émetteur
Find any certificate in Trust Chain	Trouver n'importe quel certificat dans la chaîne de confiance
Accept instance certificate	Accepter le certificat d'instance

Figure 20 – Déterminer si un Certificat d'Instance d'Application est Fiable

6.1.4 Créer un Canal Sécurisé (SecureChannel)

Toutes les *Applications OPC UA* doivent établir un *Canal Sécurisé* avant de créer une *Session*. Ce *Canal Sécurisé* exige que les deux applications aient accès aux *Certificats* qui peuvent être utilisés pour chiffrer et signer l'échange des *Messages*. Les *Application Instance Certificates* installés en suivant le processus décrit en 6.1.2 peuvent être utilisés à cette fin.

Les étapes impliquées dans l'établissement d'un *Canal Sécurisé* sont montrées à la Figure 21.



IEC

Anglais	Français
Server	Serveur
Certificate authority	Autorité de certification
GetEndpoints Request	Demande de GetEndpoints
Not required if client is preconfigured with knowledge of server policies	Non requise si le client est préconfiguré avec connaissance des politiques serveur
GetEndpoints response	Réponse GetEndpoints
Server certificate	Certificat de Serveur
Message security mode	Mode sécurité de message
Security policy	Politique de sécurité
User identity token policies	Politiques de jeton d'identité d'utilisateur
Validate Server Certificate	Valider le certificat de Serveur
CA signature	Signature de CA
Issue and expiry date	Date d'émission et d'expiration
CA revocation list	Liste d'annulation de CA
OpenSecureChannel Request	Demande OpenSecureChannel
Client certificate	Certificat Client

Anglais	Français
Client nonce	Nonce de Client
Signed with client's private key	Signée avec la clé privée du client
Encrypted with server's public key	Chiffrée avec la clé publique du serveur
Validate Client Certificate	Valider le certificat de Client
Server nonce	Nonce de Serveur
Security token	Jeton de sécurité
Token lifetime	Durée de vie du jeton
Signed with server's private key	Signée avec la clé privée du serveur
Encrypted with client's public key	Chiffrée avec la clé publique du client
Renew Security Token	Renouveler le jeton de sécurité

Figure 21 – Etablir un Canal Sécurisé

La Figure 21 ci-dessus suppose que le *Client* et le *Serveur* ont un accès en ligne à une *Autorité de Certification* (CA). Si un accès en ligne n'est pas disponible et si l'administrateur a installé une clé publique de CA sur la machine locale, le *Client* et *Serveur* doivent encore valider les *Certificats* d'application en utilisant la clé en question. La figure montre une seule CA, mais il n'y a aucune exigence pour que les *Certificats* du *Client* et du *Serveur* soient émis par la même autorité. Un *Certificat d'Instance d'Application* autosigné n'a pas besoin d'être vérifié par une CA. Tout *Certificat* doit être rejeté s'il ne figure pas dans une liste de confiance fournie par l'administrateur.

Le *Client* et le *Serveur* doivent avoir une liste de *Certificats* à laquelle ils font confiance (parfois appelée Liste de *Certificats* de confiance ou CTL) en raison de leur configuration. Ces *Certificats* de confiance peuvent être des *Certificats* pour des *Autorités de Certification* ou bien ils peuvent être des *Certificats d'Instance d'Application OPC UA*. Les *Applications OPC UA* doivent être configurées pour refuser les connexions à des applications qui n'ont pas de *Certificat* fiable.

L'intégrité des *certificats* peut être compromise, ce qui signifie qu'il convient de ne plus leur accorder de confiance. Les administrateurs peuvent annuler un *Certificat* en le retirant de la liste de confiance pour toutes les applications ou bien la CA peut ajouter le *Certificat* à la Liste d'annulation de *Certificats* (CRL) pour le *Certificat de l'émetteur*. Les administrateurs peuvent sauvegarder une copie de la CRL pour chaque *Certificat d'émetteur* lorsque l'accès en ligne n'est pas disponible.

Un *Client* n'a pas besoin d'appeler *GetEndpoints* chaque fois qu'il se connecte au *Serveur*. Il convient que cette information change rarement et le *Client* peut la placer en cache localement. Si le *Serveur* rejette la demande *OpenSecureChannel*, il convient que le *Client* appelle *GetEndpoints* et s'assure que la configuration du *Serveur* n'a pas changé.

Il existe deux risques pour la sécurité dont un *Client* doit être conscient lorsqu'il utilise le *Service GetEndpoints*. Le premier peut provenir d'un *Server Discovery* malveillant qui essaye de diriger le *Client* vers un *Serveur* malveillant. Pour cette raison, le *Client* doit vérifier que le *ServerCertificate* dans l'*EndpointDescription* est un *Certificat* fiable avant d'appeler *CreateSession*.

Le second risque pour la sécurité provient d'une tierce partie qui altère le contenu des *EndpointDescriptions* à mesure qu'elles sont transférées sur le réseau pour revenir au *Client*. Le *Client* s'en protège en comparant la liste des *EndpointDescriptions* retournée par le *Service GetEndpoints* à la liste retournée dans la réponse *CreateSession*.

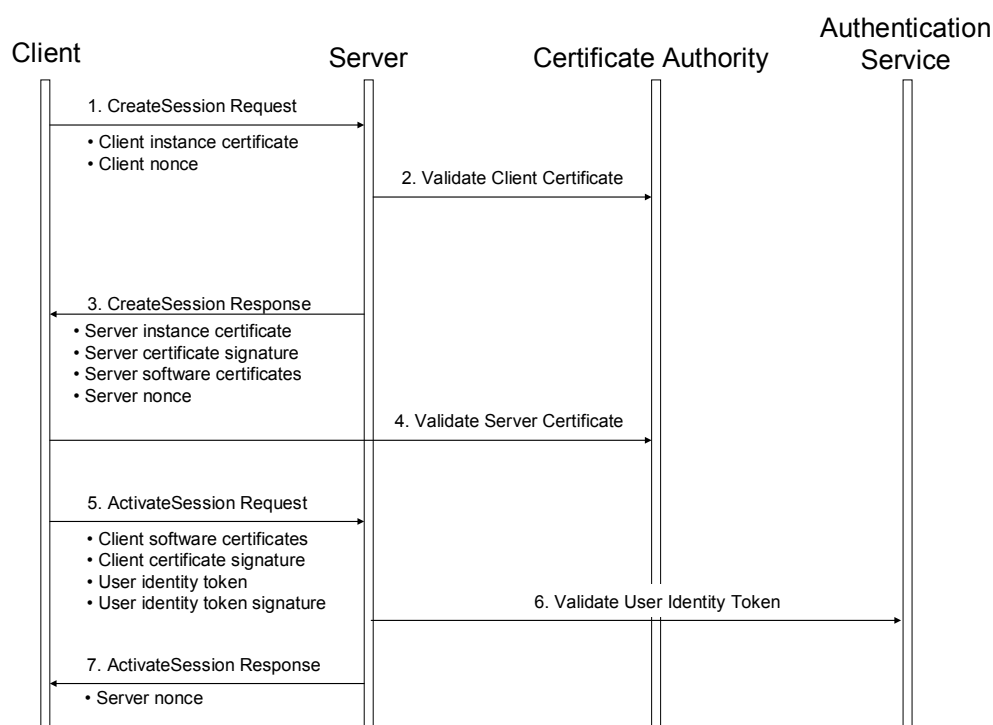
Les mécanismes exacts pour utiliser le jeton de sécurité pour signer et chiffrer les *Messages* échangés sur le *Canal Sécurisé* sont décrits dans l'IEC 62541-6. Le processus de renouvellement des jetons est également décrit en détail dans l'IEC 62541-6.

Dans de nombreux cas, les *Certificats* utilisés pour établir le *Canal Sécurisé* sont les *Certificats d'Instance d'Application*. Cependant, certaines *Piles de Communication* peuvent ne pas prendre en charge des *Certificats* qui sont spécifiques à une seule application. Elles attendent plutôt que toute la communication soit sécurisée par un *Certificat* spécifique à un utilisateur ou à toute la machine. Pour cette raison, les *Applications OPC UA* ont besoin d'échanger leurs *Certificats d'Instance d'Application* pour créer une *Session*.

6.1.5 Créer une Session

Une fois qu'un *Client* OPC UA a établi un *Canal Sécurisé* avec un *Serveur*, il peut créer une *Session* OPC UA.

Les étapes impliquées dans l'établissement d'une *Session* sont montrées à la Figure 22.



IEC

Anglais	Français
Server	Serveur
Certificate authority	Autorité de certification
Authentication service	Service d'authentification
CreateSession Request	Demande CreateSession
Client instance certificate	Certificat d'instance client
Client nonce	Nonce Client
Validate Client Certificate	Valider le certificat client
CreateSession Response	Réponse CreateSession
Server instance certificate	Certificat d'instance Serveur
Server certificate signature	Signature de certificat Serveur
Server software certificates	Certificats de logiciel Serveur
Server nonce	Nonce de Serveur
Validate Server Certificate	Valider le certificat de Serveur
ActivateSession Request	Demande ActivateSession
Client software certificates	Certificats de logiciel Client

Anglais	Français
Client certificate signature	Signature de certificat Client
User identity token	Jeton d'identité d'utilisateur
User identity token signature	Signature de jeton d'identité d'utilisateur
Validate user identity token	Valider le jeton d'identité d'utilisateur
ActivateSession Response	Réponse ActivateSession

Figure 22 – Etablir une Session

La Figure 22 ci-dessus illustre les interactions entre un *Client*, un *Serveur*, une *Autorité de Certification* (CA) et un service d'authentification. La CA est chargée d'émettre les *Certificats d'Instance d'Application*. Si le *Client* ou le *Serveur* n'a pas un accès en ligne à la CA, il doit valider les *Certificats d'Instance d'Application* en utilisant la clé publique de la CA que l'administrateur doit installer sur la machine locale.

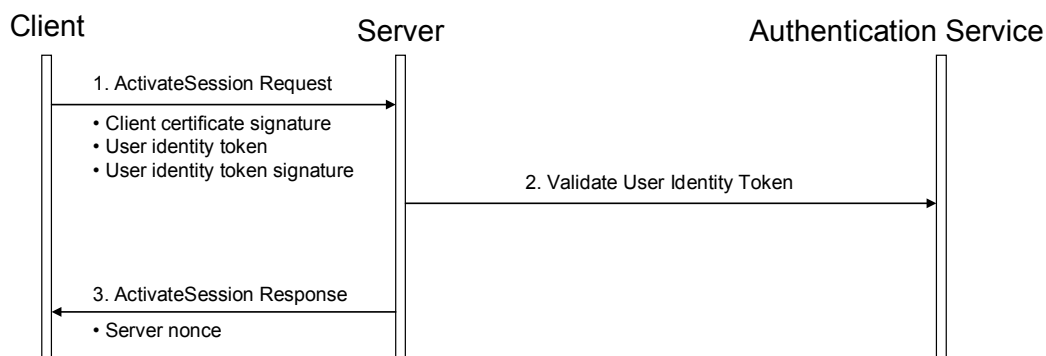
Le service d'authentification est une base de données centrale qui peut vérifier ce jeton d'utilisateur fourni par le *Client*. Ce service d'authentification peut également indiquer au *Serveur* les droits d'accès que l'utilisateur possède. Le service d'authentification dépend du jeton d'identité d'utilisateur. Il peut s'agir d'une *Autorité de Certification*, d'un service d'octroi de ticket Kerberos, d'un *Serveur* WS-Trust ou d'une base de données propriétaires quelconque.

Le *Client* et le *Serveur* doivent prouver la possession de leurs *Certificats d'Instance d'Application* en signant les *Certificats* avec un nonce accolé. Le mécanisme exact utilisé pour créer les signatures de preuve de possession est décrit en 5.6.2. De même, le *Client* doit prouver la possession de certains types de jetons d'identité d'utilisateur en créant des signatures avec le secret associé au jeton.

6.1.6 Se substituer à un utilisateur

Une fois qu'un *Client* OPC UA a établi une *Session* avec un *Serveur*, il peut changer l'identité d'utilisateur associée à la *Session* en appelant le service *ActivateSession*.

Les étapes impliquées dans la substitution à un utilisateur sont montrées à la Figure 23.



IEC

Anglais	Français
Server	Serveur
Authentication service	Service d'authentification
ActivateSession Request	Demande ActivateSession
Client certificate signature	Signature de certificat Client

Anglais	Français
User identity token	Jeton d'identité d'utilisateur
User identity token signature	Signature de jeton d'identité d'utilisateur
Validate User Identity Token	Valider le jeton d'identité d'utilisateur
ActivateSession Response	Réponse ActivateSession
Server nonce	Nonce de serveur

Figure 23 – Se substituer à un utilisateur

6.2 Certificats de logiciel (Software Certificates)

6.2.1 Vue d'ensemble

Toutes les *Applications OPC UA* peuvent avoir un ou plusieurs *Certificats* de logiciel qui sont émis par des autorités de certification et spécifient les profils que l'application prend en charge. Le présent Article décrit comment sont obtenus, installés et validés les *SoftwareCertificates*.

Les *SoftwareCertificates* sont utilisés pour identifier un produit logiciel qui est une *Application OPC UA* et pour décrire les fonctions du produit. Les fonctions sont décrites par des *Profils* définis dans l'IEC 62541-7. Les *SoftwareCertificates* ne sont pas pertinents pour la sécurité en OPC UA.

6.2.2 Obtenir et installer un Certificat de logiciel

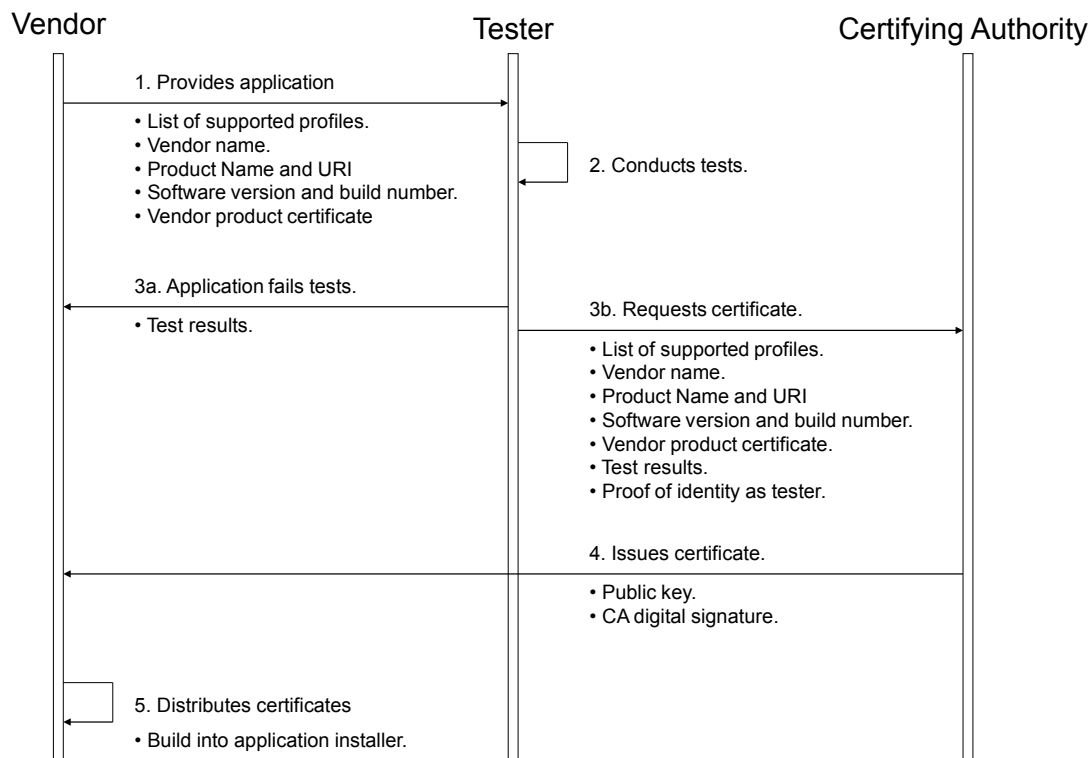
Les *Certificats* de logiciel contiennent les informations suivantes:

- Le nom du fournisseur responsable du produit;
- Le nom du produit;
- L'URI du produit;
- Le numéro de version et de mouture du logiciel;
- Le *Certificat* du produit du fournisseur;
- Une liste des profils pris en charge par l'application;
- Le statut d'essais de certification pour chaque profil pris en charge;
- Le nom de l'autorité de contrôle qui a émis le *Certificat*;
- La date d'émission et d'expiration du *Certificat*;
- La clé publique émise pour l'application par l'autorité de certification;
- Une signature numérique créée par l'autorité de certification.

Le fournisseur du produit a la responsabilité de mener à bien le processus de certification et de demander des *certificats* de logiciel auprès des autorités de certification. Le fournisseur doit installer les *Certificats* de logiciel lorsqu'une application est installée sur une machine. Lorsqu'ils distribuent ces *Certificats*, il convient que les fournisseurs d'application prennent des précautions pour empêcher que des utilisateurs non autorisés acquièrent leurs *Certificats* de logiciel et les utilisent pour des applications que le fournisseur n'a pas développées. Les *Certificats* de logiciel improprement utilisés ne sont pas un risque pour la sécurité, mais les fournisseurs peuvent être réprimandés pour des problèmes d'interopérabilité dus à l'utilisation de *Certificats* par des applications non autorisées.

Des fournisseurs peuvent choisir d'affecter leur propre *Certificat* à l'application (appelé *Certificat Produit du Fournisseur*). Ce *Certificat* serait alors incorporé dans le *SoftwareCertificate*. La clé privée pour le *Certificat* produit du fournisseur est gérée par le fournisseur.

Les étapes impliquées dans l'acquisition et l'installation d'un *Certificat* de logiciel auprès d'une *Autorité de Certification* sont montrées à la Figure 24.



IEC

Anglais	Français
Vendor	Fournisseur
Tester	Vérificateur
Certifying authority	Autorité de certification
Provides application	Fournit l'application
List of supported profiles	Liste des profils pris en charge
Vendor name	Nom de fournisseur
Product name and URI	Nom et URI de produit
Software version and build number	Numéro de version et de mouture du logiciel
Vendor product certificate	Certificat de produit du fournisseur
Conduct tests	Conduit les essais
Application fails tests	L'application échoue aux essais
Test results	Résultats d'essai
Requests certificate	Demande un certificat
Proof of identity as tester	Preuve de l'identité comme vérificateur
Issues certificate	Émet un certificat
Public key	Clé publique
CA digital signature	Signature numérique de CA
Distributes certificates	Distribue des certificats
Build into application installer	Bâtit dans un installateur d'application

Figure 24 – Obtenir et installer un Certificat de logiciel

La figure ci-dessus illustre les interactions entre le *Fournisseur (Vendor)*, le *Vérificateur (Tester)* et l'autorité de certification (*Certifying Authority*). Le *Fournisseur* est l'organisation

responsable de l'*Application OPC UA*. Le *Vérificateur* peut être le fournisseur (pour l'autocertification) ou peut être une installation d'essai de tierce partie. La *Certifying Authority* est une Autorité de certification ICP (infrastructure à clé publique) gérée par l'organisation qui a créé les profils d'*Application OPC UA* et les programmes de certification.

L'autorité de certification émettra des *Certificats* uniquement pour les fournisseurs en qui elle a confiance. Pour cette raison, le *Vérificateur* doit fournir une preuve d'identité avant que l'*Autorité de Certification* n'émette un *Certificat*.

6.2.3 Valider un Certificat de logiciel

Les *Applications OPC UA* doivent valider les *Certificats de logiciel* fournis par les applications avec lesquelles elles communiquent.

Un *Certificat de logiciel* est valide si:

- La signature sur le *Certificat de logiciel* est valide;
- le *Certificat de logiciel* a dépassé sa date d'émission et il n'a pas expiré;
- le *Certificat de logiciel* n'a pas été annulé par l'émetteur;
- le *Certificat d'émetteur* est valide et n'a pas été annulé par la CA qui l'avait émis.

Les étapes utilisées pour valider des *Certificats de logiciel* sont pratiquement les mêmes que celles décrites pour les *Certificats d'Instance d'Application* en 6.1.3. La seule différence est que les *Certificats de logiciel* ne sont pas vérifiés par rapport à la *TrustList* de l'*Application*.

Les *Certificats de logiciel* contiennent toujours un *Certificat d'Instance d'Application* qui est la propriété du fournisseur qui distribue l'application. Le *Certificat* de l'application peut aussi être dans la chaîne des *Certificats* utilisée pour émettre des *Certificats d'Instance d'Application*. Pour cette raison, les *Applications OPC UA* peuvent rejeter des *SoftwareCertificates* fournis par des applications si le *Certificat* de l'application ne fait pas partie de la chaîne de *Certificats* pour le *Certificat d'Instance d'Application*. Il convient que les *Applications OPC UA* permettent aux *Administrateurs* d'exiger ce comportement. Les profils définis dans l'IEC 62541-7 peuvent spécifier plus précisément les comportements de gestion attendus des *Certificats*.

6.3 Audits

6.3.1 Vue d'ensemble

L'audit est une exigence dans de nombreux systèmes. Il fournit un moyen de suivre les activités qui ont lieu comme partie intégrante du fonctionnement normal du système. Il fournit aussi un moyen de suivre un comportement anormal. Il est également une exigence du point de vue de la sécurité. Pour plus d'informations sur les aspects sécuritaires des audits, voir l'IEC TR 62541-2. Ce paragraphe décrit ce qui est attendu d'un *Serveur* et d'un *Client OPC UA* quant aux audits et il détaille les exigences relatives à l'audit pour chaque jeu de services. L'audit peut être accompli en utilisant l'une des deux méthodes suivantes ou les deux:

- a) L'*Application OPC UA* qui génère un événement d'audit peut journaliser l'entrée d'audit dans un fichier journal ou autre lieu de stockage;
- b) Le *Serveur OPC UA* qui génère l'événement d'audit peut l'éditer en utilisant le mécanisme d'événements OPC UA. Cela permet à un *Client OPC UA* de s'abonner aux entrées d'audit et de les journaliser dans un fichier journal ou dans un autre lieu de stockage.

6.3.2 Journaux d'audits généraux

Chaque demande de *Service OPC UA* contient un paramètre chaîne qui est utilisé pour contenir un id d'enregistrement d'audit. Un *Client* ou tout *Serveur* opérant comme *Client*, tel qu'un *Serveur* combiné par exemple, peut créer une entrée locale de journal d'audits pour une demande qu'il présente. Ce paramètre permet à ce *Client* de transmettre l'identificateur pour

cette entrée avec la demande. Si ce *Serveur* maintient aussi un journal d'audits, il convient qu'il inclue cet identificateur dans son entrée du journal d'audits qu'il écrit. Lorsque ce journal est examiné et l'entrée trouvée, l'examineur peut relier celle-ci directement à l'entrée de journal d'audits créée par le *Client*. Cette fonction permet la traçabilité à travers les journaux d'audits dans un système.

6.3.3 Événements d'audit généraux

Un *Serveur* qui maintient un journal d'audits doit fournir les entrées de journal d'audits via des *Messages d'Événement*. L'*AuditEventType* et ses sous-types sont définis dans l'IEC 62541-3. Un *Message d'Événement* d'audit inclut aussi l'identificateur d'enregistrement d'audit. Les détails de l'*AuditEventType* et de ses sous-types sont définis dans l'IEC 62541-5. Un *Serveur* qui est un *Serveur* combiné qui prend en charge les audits doit aussi s'abonner à des événements d'audit pour tous les *Serveurs* qu'il combine (en supposant qu'ils fournissent des audits). Il convient que le flux combiné soit disponible à partir du *Serveur* combiné.

6.3.4 Audits pour jeu de services Discovery

Ce *Jeu de Services* peut être divisé en deux groupes: les *Services* appelés par les *Clients* OPC UA et les *Services* qui sont invoqués par des *Serveurs* OPC UA. Les *Services* *FindServers* et *GetEndpoints* qui sont appelés par les *Clients* OPC UA peuvent générer des entrées d'audit pour des invocations de *Service* non abouties. Le *Service RegisterServer* qui est invoqué par les *Serveurs* OPC UA doit générer des entrées d'audit pour tous les nouveaux enregistrements et pour les invocations de *Service* non abouties. Ces entrées d'audit doivent inclure l'URI de *Serveur*, les noms de *Serveur*, les URI *Discovery* et le statut *isOnline*. Il convient que des entrées d'audit ne soient pas générées pour l'invocation *RegisterServer* qui n'induit pas de changements aux *Serveurs* enregistrés.

6.3.5 Audits pour jeu de services SecureChannel

Tous les *Services* dans ce *Service Set* pour des *Serveurs* qui prennent en charge les audits peuvent générer des entrées d'audit et doivent générer des *Événements* d'audit pour les invocations de service non abouties et pour l'invocation réussie des *Services* *OpenSecureChannel* et *CloseSecureChannel*. Il convient que les entrées d'audit générées par le *Client* soient installées avant l'appel effectif, permettant la fourniture de l'id correct d'enregistrement d'audit. Le *Service OpenSecureChannel* doit générer un *Événement* d'audit du type *AuditOpenSecureChannelEventType* ou d'un sous-type de celui-ci pour *requestType* *ISSUE_0*. Les *Événements* d'audit pour *requestType* *RENEW_1* ne sont créés que si *renew* échoue. Le service *CloseSecureChannel* doit générer un *Événement* d'audit du type *AuditChannelEventType* ou d'un sous-type de celui-ci. Ces deux types d'*Événement* sont des sous-types de l'*AuditChannelEventType*. Voir l'IEC 62541-5 pour l'affectation détaillée des paramètres *SourceNode*, *SourceName* et paramètres supplémentaires. Pour les cas d'échec, il convient d'inclure dans le *Message* pour les *Événements* de ce type une description des raisons de l'échec du service. Il convient que la description soit plus détaillée que celle qui a été retournée au client. D'un point de vue sécurité, un client a seulement besoin de savoir qu'il y a eu échec. En revanche, du point de vue de l'*Audit*, il est nécessaire de connaître les détails exacts de l'échec. Dans le cas d'erreurs de validation de *Certificat*, il convient d'inclure dans la description l'*EventId* d'audit de l'*AuditCertificateEventType* spécifique qui a été généré pour rapporter l'erreur du *Certificat*. L'*AuditCertificateEvent* doit aussi contenir dans le détail l'erreur de validation du *Certificat*. Il convient d'inclure dans les paramètres supplémentaires les détails de la demande. Il est bien entendu que ces événements peuvent être générés dans de nombreux cas par les *Piles de Communication* sous-jacentes, mais ils doivent être à la disposition du *Serveur* et le *Serveur* doit les consigner dans un rapport.

6.3.6 Audits pour jeu de services Session

Tous les *Services* de ce *Service Set* pour les *Serveurs* qui prennent en charge les audits peuvent générer des entrées d'audit et doivent générer des *Événements* d'audit pour les invocations de *Service* abouties et non abouties. Ces *Services* doivent générer un *Événement* d'audit du type *AuditSessionEventType* ou d'un sous-type de celui-ci. En particulier, ils doivent générer l'*EventType* de base ou le sous-type approprié, en fonction du service qui a

été invoqué. Le service *CreateSession* doit générer des événements de l'*AuditActivateSessionEventType* ou des sous-types de celui-ci. Le service *ActivateSession* doit générer des événements de l'*AuditActivateSessionEventType* ou des sous-types de celui-ci. Lorsque le Service *ActivateSession* est appelé pour changer l'identité d'utilisateur, le serveur doit générer des événements de l'*AuditActivateSessionEventType* ou sous-types de celui-ci. Le service *CloseSession* service doit générer l'*EventType* de base de l'*AuditSessionEventType* ou sous-types de celui-ci. Voir l'IEC 62541-5 pour l'affectation détaillée des paramètres *SourceNode*, *SourceName* et paramètres supplémentaires. Pour les cas d'échec, il convient d'inclure dans le *Message* pour les *Événements* de ce type une description des raisons de l'échec du *Service*. Il convient d'inclure dans les paramètres supplémentaires les détails de la demande.

Ce *Service Set* doit également générer des événements d'audit supplémentaires lorsqu'il se produit des erreurs de validation de *Certificat*. Ces événements d'audit sont générés en plus des *AuditSessionEventTypes*. Voir l'IEC 62541-3 pour la définition de l'*AuditCertificateEventType* et de ses sous-types.

Pour les *Clients* prenant en charge les audits, l'accès aux services dans le *Session Service Set* doit générer des entrées d'audit pour les invocations abouties et non abouties du *Service*. Il convient que ces entrées d'audit soient installées avant l'invocation effective du *Service*, permettant à l'invocation de contenir l'id correct d'enregistrement d'audit.

6.3.7 Audits pour jeu de services NodeManagement

Tous les Services de ce *Service Set* pour les *Serveurs* qui prennent en charge les audits peuvent générer des entrées d'audit et doivent générer des *Événements* d'audit pour les invocations de *Service* abouties et non abouties. Ces *Services* doivent générer un *Événement* d'audit du type *AuditNodeManagementEventType* ou sous-types de celui-ci. Voir l'IEC 62541-5 pour l'affectation détaillée des paramètres *SourceNode*, *SourceName* et paramètres supplémentaires. Pour les cas d'échec, il convient d'inclure dans le *Message* pour les *Événements* de ce type une description des raisons de l'échec du service. Il convient d'inclure dans les paramètres supplémentaires les détails de la demande.

Pour les *Clients* prenant en charge les audits, l'accès aux *Services* dans le *NodeManagement Service Set*, doit générer des entrées d'audit pour les invocations abouties et non abouties du *Service*. Il convient que toutes les entrées d'audit soient installées avant l'invocation effective du *Service*, permettant à l'invocation de contenir l'id correct d'enregistrement d'audit.

6.3.8 Audits pour jeu de services Attribute

Les Services *Write* ou *HistoryUpdate* de ce *Service Set* pour les *Serveurs* qui prennent en charge les audits peuvent générer des entrées d'audit et doivent générer des *Événements* d'audit pour les invocations de *Service* abouties et non abouties. Ces *Services* doivent générer un *Événement* d'audit du type *AuditUpdateEventType* ou sous-types de celui-ci. En particulier, le Service *Write* doit générer un événement d'audit du type *AuditWriteUpdateEventType* ou d'un sous-type de celui-ci. Le Service *HistoryUpdate* doit générer un *Événement* d'audit du type *AuditHistoryUpdateEvent* ou d'un sous-type de celui-ci. Trois sous-types de l'*AuditHistoryUpdateEvent* sont définis, à savoir *AuditHistoryEventUpdateEventType*, *AuditHistoryValueUpdateEventType* et *AuditHistoryDeleteEventType*. Le sous-type dépend du type d'opération effectuée, à savoir mise à jour d'événements historiques, mise à jour de valeurs de données historiques ou suppression d'historique. Voir l'IEC 62541-5 pour l'affectation détaillée des paramètres *SourceNode*, *SourceName* et des paramètres supplémentaires. Pour les cas d'échec, il convient d'inclure dans le *Message* pour les *Événements* de ce type une description des raisons de l'échec du *Service*. Il convient d'inclure dans les paramètres supplémentaires les détails de la demande.

Les Services *Read* et *HistoryRead* peuvent générer des entrées d'audit et des *Événements* d'audit pour les invocations de *Service* qui n'ont pas abouti. Il convient que ces *Services* génèrent un *Événement* d'audit du type *AuditEventType* ou d'un sous-type de celui-ci. Voir

l'IEC 62541-5 pour l'affectation détaillée des paramètres *SourceNode*, *SourceName* et paramètres supplémentaires. Il convient d'inclure dans le *Message* pour les *Événements* de ce type une description des raisons de l'échec du *Service*.

Pour les *Clients* prenant en charge les audits, l'accès aux services *Write* ou *HistoryUpdate* dans l'*Attribute Service Set* doit générer des entrées d'audit pour les invocations abouties et non abouties du *Service*. Les invocations des autres *Services* dans ce *Service Set* peuvent générer des entrées d'audit. Il convient que toutes les entrées d'audit soient installées avant l'invocation effective du *Service*, permettant à l'invocation de contenir l'id correct d'enregistrement d'audit.

6.3.9 Audits pour jeu de services de Méthodes

Tous les *Services* de ce *Service Set* pour les *Serveurs* qui prennent en charge les audits peuvent générer des entrées d'audit et doivent générer des *Événements* d'audit pour les invocations abouties et non abouties si l'invocation modifie l'espace d'adresses, écrit une valeur ou modifie l'état du système (acquiescement d'alarme, séquençement par lots ou autres modifications de système). Ces appels de méthodes doivent générer un *Événement* d'audit du type *AuditUpdateMethodEventType* ou sous-types de celui-ci. Les méthodes qui ne modifient pas l'*Espace d'Adresses*, n'écrivent pas de valeurs ou ne modifient pas l'état du système peuvent générer des événements. Voir l'IEC 62541-5 pour l'affectation détaillée des paramètres *SourceNode*, *SourceName* et des paramètres supplémentaires.

Pour les *Clients* prenant en charge les audits, l'accès au *jeu de services de méthodes* doit générer des entrées d'audit pour les invocations abouties et non abouties du *Service* si l'invocation modifie l'espace d'adresses, écrit une valeur ou modifie l'état du système (acquiescement d'alarme, séquençement par lots ou autres modifications de système). Les invocations des autres *Méthodes* peuvent générer des entrées d'audit. Il convient que toutes les entrées d'audit soient installées avant l'invocation effective du *Service*, permettant à l'invocation de contenir l'id correct d'enregistrement d'audit.

6.3.10 Audits pour le View Service Set, le Query Service Set, le MonitoredItem Service Set et le Subscription Service Set

Tous les *Services* de ces quatre *Service Sets* fournissent seulement des informations au *Client*, avec l'exception du *Service TransferSubscriptions* dans le *Subscription Service Set*. En général, ces services ne génèrent pas d'entrées d'audit ou de *Messages* d'événement d'audit. Le *Service TransferSubscriptions* doit générer un *Événement* d'audit du type *AuditSessionEventType* ou sous-types de celui-ci pour les invocations de *Service* abouties et non abouties. Voir l'IEC 62541-5 pour l'affectation détaillée des paramètres *SourceNode*, *SourceName* et des paramètres supplémentaires. Pour les cas d'échec, il convient d'inclure dans le *Message* pour les *Événements* de ce type une description des raisons de l'échec du service.

Pour les *Clients* prenant en charge les audits, l'accès au *Service TransferSubscriptions* dans le *Subscription Service Set* doit générer des entrées d'audit pour les invocations abouties et non abouties du *Service*. Les invocations des autres *Services* dans ce *Service Set* n'exigent pas d'entrées d'audit. Il convient que toutes les entrées d'audit soient installées avant l'invocation effective du *Service*, permettant à l'invocation de contenir l'id correct d'enregistrement d'audit.

6.4 Redondance

6.4.1 Vue d'ensemble de la redondance

La redondance dans OPC UA garantit que les *Clients* tout comme le *Serveur* peuvent être redondants. OPC UA ne fournit pas la redondance, elle fournit les structures de données et les services permettant d'obtenir la redondance d'une manière normalisée.

6.4.2 Vue d'ensemble de la redondance du serveur

6.4.2.1 Généralités

La redondance du serveur se présente sous deux modes, transparent et non transparent. Par définition, dans la redondance transparente, le basculement des responsabilités de *Serveur* d'un *Serveur* à un autre est transparent au *Client*: le *Client* ne se soucie pas de savoir, ou même ne sait pas, qu'un basculement a eu lieu; le *Client* n'a pas besoin de faire quoi que ce soit pour maintenir le flux de données. En revanche, le basculement non transparent exige une action du *Client*.

L'*Object ServerRedundancy* défini dans l'IEC 62541-5 indique le mode pris en charge par le *Serveur*. L'*ObjectType ServerRedundancyType* et ses sous-types *TransparentRedundancyType* et *NonTransparentRedundancyType* définis dans l'IEC 62541-5 spécifient les informations pour le mode de redondance pris en charge.

Les deux domaines où la redondance crée des besoins spécifiques sont le maintien de la synchronisation des informations du *Serveur* et du *Client* parmi les *Serveurs* et la régulation du basculement de flux de données d'un *Serveur* à l'autre.

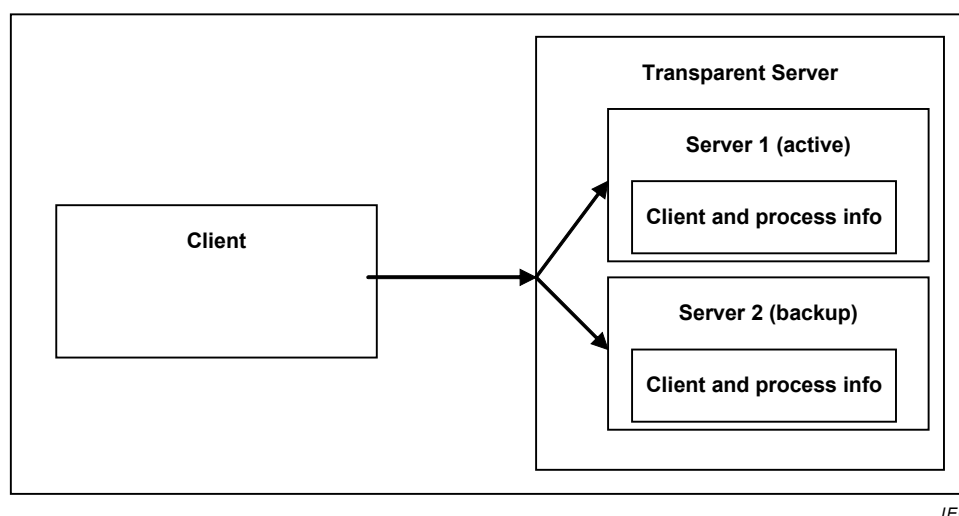
Quel que soit le mode de redondance utilisé, il est prévu que tous les *Serveurs* dans le jeu redondant aient des espaces d'adresses identiques, y compris les *NodeIds* identiques et une logique identique pour le réglage du niveau de service.

6.4.2.2 Redondance transparente

Pour la redondance transparente, tout ce qu'OPC UA fournit se résume aux structures de données pour permettre au *Client* d'identifier les *Serveurs* dans le jeu redondant, le niveau de service de chaque *Serveur* et le *Serveur* actuellement responsable de la *Session Client*. Ces informations sont spécifiées dans l'*ObjectType* de *TransparentRedundancyType* défini dans l'IEC 62541-5.

Toutes les interactions OPC UA au sein d'une session donnée doivent être prises en charge par un *Serveur* donné et le *Client* est à même d'identifier le *Serveur* en question, ce qui permet une piste d'audit complète pour les données. Il est de la responsabilité des *Serveurs* d'assurer que les informations sont synchronisées entre les *Serveurs*. Un *Serveur* fonctionnel reprendra la *Session* et les *Subscriptions* du *Serveur* défaillant. Le basculement peut exiger une reconnexion à la couche transport du *Client*, mais l'URL d'*Endpoint* du *Serveur* ne doit pas changer. Voir 6.5 pour plus de détails sur le fait de rétablir une connexion.

La Figure 25 montre une configuration type de la redondance transparente.



IEC

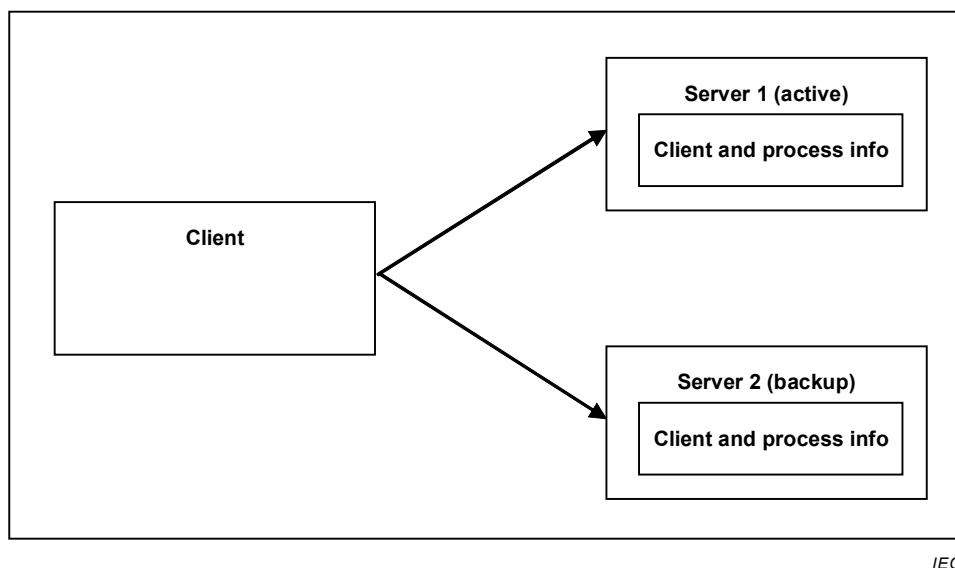
Anglais	Français
Transparent server	Serveur transparent
Server 1 (active)	Serveur 1 (actif)
Client and process info	Info de client et de processus
Server 2 (backup)	Serveur 2 (secours)

Figure 25 – Configuration type de la redondance transparente

6.4.2.3 Redondance non transparente

Pour la redondance non transparente, OPC UA fournit les structures de données pour permettre au *Client* d'identifier les *Serveurs* disponibles dans le jeu redondant et aussi les informations du *Serveur* qui précisent au *Client* les modes de basculement que le *Serveur* prend en charge. Ces informations permettent au *Client* de déterminer les actions qu'il peut avoir à prendre pour accomplir la reprise sur défaillance. Ces informations sont spécifiées dans l'*ObjectType NonTransparentRedundancyType* défini dans l'IEC 62541-5.

La Figure 26 montre une configuration type de la redondance non transparente.



IEC

Anglais	Français
Server 1 (active)	Serveur 1 (actif)
Client and process info	Info de client et de processus
Server 2 (backup)	Serveur 2 (secours)

Figure 26 – Configuration de la redondance non transparente

Pour la redondance non transparente, le *Serveur* a des concepts supplémentaires de modes de basculement *Cold* (à froid), *Warm* (tiède), *Hot* (à chaud) et *HotAndMirrored* (à chaud et reflété). Le mode de basculement fournit des informations sur les fonctions de basculement prises en charge par le *Serveur* et permet au *Client* de déterminer les actions possibles de basculement décrites dans le Tableau 102.

Les basculements *Cold* concernent les *Serveurs* parmi lesquels un seul *Serveur* peut être actif à la fois. Le *Client* ne peut se connecter qu'à l'un des *Serveurs* dans le jeu redondant.

Les basculements *Warm* concernent les *Serveurs* parmi lesquels les *Serveurs* de secours peuvent être actifs, mais ne peuvent pas se connecter à des points de données réels (typiquement un système où les appareils sous-jacents sont limités à une seule connexion). La *Variable ServiceLevel* définie dans l'IEC 62541-5 indique la capacité du *Serveur* à fournir ses données au *Client*. Le *Client* doit utiliser le *Serveur* avec le niveau de service le plus élevé. La désignation du *Serveur* actif est spécifique au fournisseur.

Les basculements *Hot* sont pour les *Serveurs* pour lesquels plus d'un *Serveur* peut être actif et entièrement opérationnel. Il convient que la *Variable ServiceLevel* définie dans l'IEC 62541-5 soit utilisée par le *Client* pour trouver les *Serveurs* avec le niveau de service le plus élevé afin d'atteindre l'équilibre de la charge.

Les basculements *HotAndMirrored* sont pour les *Serveurs* qui reflètent les états internes pour tous les *Serveurs* dans le jeu redondant et pour lesquels plus d'un *Serveur* peut être actif et entièrement opérationnel. L'état de reflet comprend au moins les *Sessions*, les *Subscriptions*, les *Nodes* enregistrés, les numéros de séquence et les *Notifications* envoyées. Cela permet aux *Clients* de basculer sans créer de nouveau contexte pour la communication. Il convient que la *Variable ServiceLevel* définie dans l'IEC 62541-5 soit utilisée par le *Client* pour trouver les *Serveurs* avec le niveau de service le plus élevé afin d'atteindre l'équilibre de la charge. Ce mode de basculement est similaire à la redondance transparente. L'avantage est que le *Client* a un contrôle total sur la sélection du *Serveur*. L'inconvénient est que le *Client* a besoin d'être capable de gérer les basculements.

Chaque *Serveur* maintient une liste de *ServerUris* pour tous les *Serveurs* redondants dans le jeu redondant. Cette liste est fournie avec le mode de basculement dans l'*Object ServerRedundancy* défini dans l'IEC 62541-5. Pour permettre aux clients de se connecter à tous les *Serveurs* dans la liste, chaque *Serveur* de la liste doit fournir l'*ApplicationDescription* pour tous les *Serveurs* du jeu redondant par le *Service FindServers*. Ces informations sont nécessaires au *Client* pour traduire le *ServerUri* en informations nécessaires pour se connecter aux autres *Serveurs* du jeu redondant. Un *Client* n'a donc besoin de connaître qu'un seul des *Serveurs* redondants pour trouver les autres *Serveurs* en se basant sur les informations fournies.

Le Tableau 102 définit la liste des actions de basculement.

Tableau 102 – Actions de basculement de redondance

Mode de basculement et options du <i>Client</i>	Cold (à froid)	Warm (tiède)	Hot (1) (à chaud)	Hot (2) (à chaud)	HotAndMirrored (à chaud et reflété)
Sur connexion initiale en plus des actions sur le serveur actif:					
Connexion à plus d'un <i>Serveur</i> OPC UA		X	X	X	Optionnel pour la vérification de l'état
Création de <i>Subscriptions</i> et ajout d'éléments surveillés		X	X	X	
Activation de l'échantillonnage sur les <i>Subscriptions</i>			X	X	
Activation de l'édition				X	
Au basculement:					
CreateSecureChannel au <i>Serveur</i> OPC UA de secours	X				X
CreateSession sur <i>Serveur</i> OPC UA de secours	X				
ActivateSession sur <i>Serveur</i> OPC UA de secours	X				X
Création de <i>Subscriptions</i> et ajout d'éléments surveillés	X				
Activation de l'échantillonnage sur les <i>Subscriptions</i>	X	X			
Activation de l'édition	X	X	X		

Un *Client* connecté à un jeu de basculements *Cold* redondants ne peut se connecter qu'à un *Serveur*. Pour le basculement, le *Client* se connecte à un *Serveur* de secours, crée des *Subscriptions* et des *MonitoredItems* et active l'édition. Il y a perte de données entre le moment où la connexion au *Serveur* actif est interrompue et le moment où le *Client* obtient des *Réponses Publish* du *Serveur* de secours.

Un *Client* connecté à un jeu de basculements *Warm* redondants peut se connecter et créer des *Subscriptions* et des *MonitoredItems* sur plus d'un *Serveur*. L'échantillonnage et l'édition ne peuvent être actifs que sur un *Serveur*. Ce *Serveur* peut être trouvé en lisant la *Variable ServiceLevel* de tous les *Serveurs*. Le *Serveur* avec le *ServiceLevel* le plus élevé est utilisé. Pour le basculement, le *Client* active l'échantillonnage et l'édition sur le *Serveur* avec le *ServiceLevel* le plus élevé. Il y a perte de données entre le moment où la connexion au *Serveur* actif est interrompue et le moment où le *Client* obtient des *Réponses Publish* du *Serveur* de secours.

Un *Client* connecté à un jeu de basculements *Hot* redondants a deux options pour assurer qu'il n'y a pas de perte de données dans le scénario de basculement.

- **Hot (À chaud) (1)**
Le *Client* crée des *MonitoredItems* et échantillonne sur plus d'un *Serveur*, mais active l'édition uniquement sur le *Serveur* avec le *ServiceLevel* le plus élevé. Il convient que le *Client* définisse la taille de la file d'attente pour les *MonitoredItems* de sorte qu'il puisse servir de tampon pour tous les changements se produisant pendant la durée du basculement. La durée du basculement correspond au moment entre l'interruption de la connexion et le moment où le *Client* obtient des *Réponses Publish* du *Serveur* de secours.
- **Hot (À chaud) (2)**
Le *Client* reçoit les mêmes informations de plus d'un *Serveur* en échantillonnant et en éditant sur plus d'un *Serveur*. Ce mode garantit qu'il n'y a pas d'interruption dans le

flux de données et qu'il n'y a aucune perte de données, mais il convient que le *Client* assure qu'il détecte les données dupliquées et il doit traiter deux flux de données.

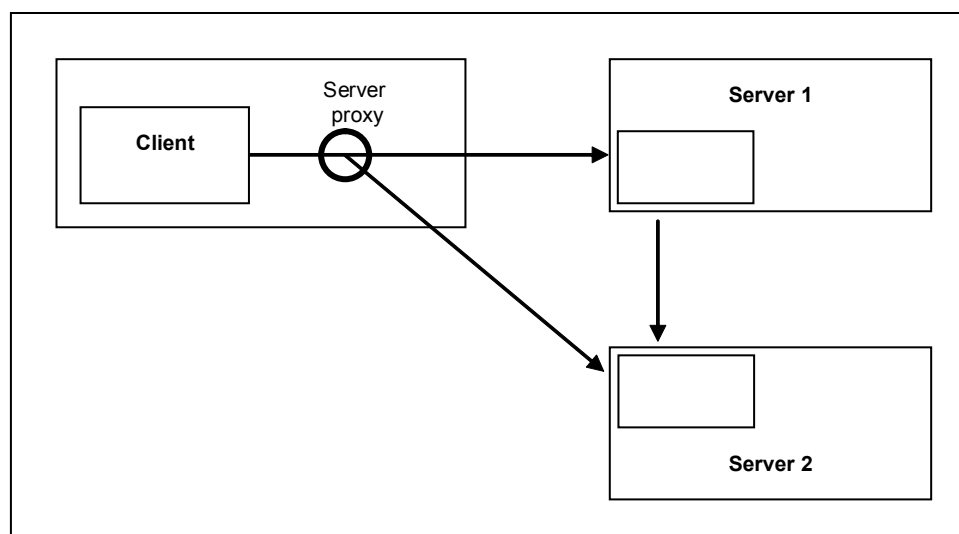
Un *Client* connecté à un jeu de basculements *HotAndMirrored* (à chaud et reflétés) redondants peut supposer que les *Serveurs* reflètent leur état, y compris les informations de connexion et les files d'attente. Le *Client* se connecte donc seulement à un *Serveur* et dans le cas d'un basculement rétablit sa connexion à l'aide d'un autre *Serveur*. Afin de valider la capacité à se connecter à d'autres *Serveurs* redondants, il est autorisé à créer des *Sessions* avec d'autres *Serveurs* et à maintenir les connexions ouvertes en lisant régulièrement le *ServiceLevel*. Un *Client* ne doit pas créer de *Subscriptions* sur les *Serveurs* de secours pour la surveillance des états. Le basculement fonctionne comme décrit en 6.5. La seule différence est que le nouveau *SecureChannel* est créé sur un autre *Serveur* avec le *ServiceLevel* le plus élevé. Les *Sessions* et les *Subscriptions* peuvent être réutilisés, car ils sont reflétés vers tous les *Serveurs*.

Un *Client* peut toujours utiliser un mode inférieur à celui pris en charge par le *Serveur*. Par exemple, le *Serveur* prend en charge le mode de basculement *Hot* et le *Client* peut utiliser les actions *Warm*. Dans le cas d'un mode de basculement *HotAndMirrored*, le *Client* ne doit pas utiliser un mode inférieur, car cela génère une charge inutile sur les *Serveurs*.

6.4.2.4 Cacher le basculement avec un Proxy Serveur (Informative)

Un fournisseur peut utiliser les caractéristiques de redondance non transparente pour créer un proxy *Serveur* tournant sur la machine du *Client* qui rend la redondance transparente pour les *Clients*. Cela réduit la quantité de fonctionnalité qui doit être conçue dans le *Client* et cela permet à des *Clients* plus simples de tirer parti de la redondance non transparente. Le proxy *Serveur* duplique tout simplement les *Subscriptions* et les modifications des *Subscriptions*, en transmettant les appels aux deux *Serveurs*, mais en n'activant que l'édition ou l'échantillonnage sur un *Serveur*. Lorsque le proxy détecte une défaillance, il active l'édition et/ou l'échantillonnage sur le *Serveur* de secours, juste comme l'aurait fait le *Client* s'il s'agissait d'un *Client* conscient de la redondance. Le proxy doit tourner sur la machine du *Client* pour éviter un autre seul point de défaillance.

La Figure 27 montre le proxy *Serveur* utilisé pour fournir la redondance à des *Clients* non conscients de la redondance.



IEC

Anglais	Français
Server proxy	Proxy serveur
Server	Serveur

Figure 27 – Proxy Serveur pour la redondance

6.4.3 Redondance client

La redondance *Client* est prise en charge dans OPC UA par l'appel *TransferSubscriptions* et en expose les informations *Client* dans les structures d'informations *Serveur*. Sachant que la durée de vie de *Subscription* n'est pas liée à la *Session* dans laquelle il a été créé, les *Clients* de secours peuvent surveiller la *Session* du *Client* actif avec le *Serveur*, tout comme ils surveilleraient n'importe quelle autre variable de données. Si le *Client* actif cesse d'être actif, le *Serveur* doit envoyer une mise à jour de données à tout *Client* dont cette variable est surveillée. Lorsqu'il a reçu une telle notification, un *Client* de secours ordonne au *Serveur* de transférer les *Subscriptions* vers sa propre session. Si *Subscription* est soigneusement conçu, avec des ressources suffisantes pour mémoriser les données lors de la permutation, il est nécessaire qu'il n'y ait pas de perte de données due à un basculement de *Client*.

OPC UA ne fournit pas de mécanisme normalisé pour acheminer le *SessionId* et les *SubscriptionIds* du *Client* actif vers les *Clients* de secours, mais tant que les *Clients* de secours connaissent le nom de *Client* du *Client* actif, ces informations sont immédiatement disponibles en utilisant les parties *SessionDiagnostics* et *SubscriptionDiagnostics* des données de *ServerDiagnostics*.

6.4.4 Redondance du réseau

Les réseaux redondants peuvent être utilisés avec OPC UA, transparente ou non transparente.

Dans le cas d'une OPC UA transparente un seul *Server Endpoint* peut être atteint par les différents chemins de réseau. Ce cas est complètement géré par l'infrastructure réseau. Le chemin de réseau sélectionné et le basculement sont transparents pour le *Client*.

Dans le cas d'une OPC UA non transparente, le *Serveur* fournit différents *Endpoints* pour les différents chemins de réseau. Cela nécessite que le *Serveur* et le *Client* prennent en charge de multiples connexions de réseau. Dans ce cas, le *Client* est responsable de la sélection de l'*Endpoint* et responsable du basculement. Pour le basculement, le scénario de reconnexion normal décrit en 6.5 peut être utilisé. Seul le *SecureChannel* est créé avec un autre *Endpoint*. Les *Sessions* et les *Subscriptions* peuvent être réutilisés.

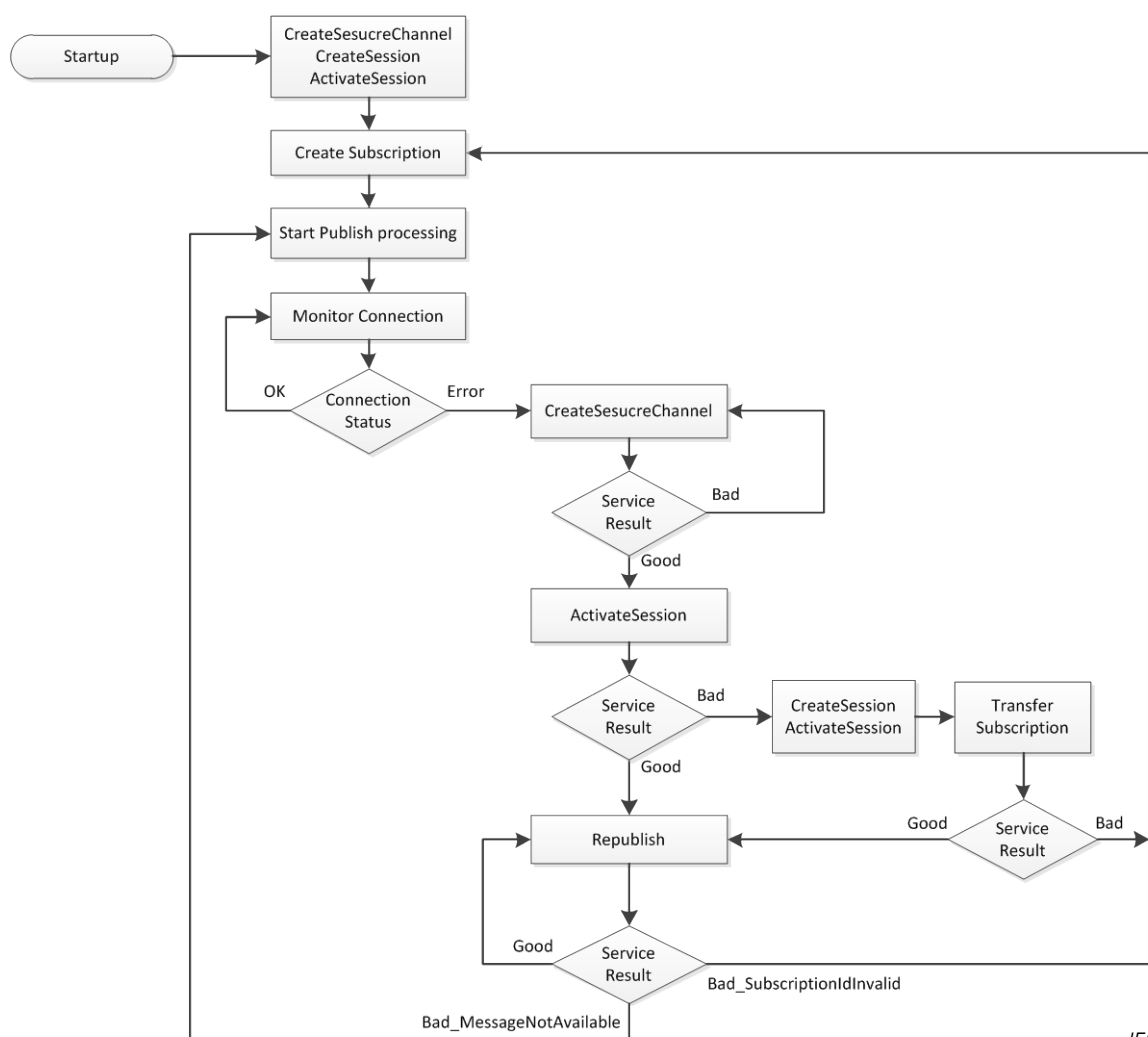
Les informations relatives aux différents chemins de réseau sont spécifiées dans l'*ObjectType NonTransparentRedundancyType* défini dans l'IEC 62541-5.

La redondance du réseau peut être combinée à la redondance du serveur.

6.5 Rétablir les connexions

Après qu'un *Client* a établi une connexion avec un *Serveur* et crée un *Subscription*, le *Client* surveille l'état de connexion. La Figure 28 montre les étapes pour connecter un *Client* à un *Serveur* et la logique générale de la gestion de reconnexion. Les scénarii d'erreur possibles ne sont pas tous couverts.

Le mécanisme préférentiel de surveillance de l'état de connexion par un *Client* est le maintien de *Subscription*. Il convient qu'un *Client* s'abonne pour la *Variable State* dans le *ServerStatus* afin de détecter les fermetures ou autres états de défaillance. Si aucun *Subscription* n'est créé ou que le *Serveur* ne prend pas en charge les *Subscriptions*, la connexion peut être surveillée en lisant régulièrement la *Variable State*.



IEC

Anglais	Français
Startup	Démarrage
Create Subscription	Créer Abonnement
Start Publish processing	Démarrer le traitement de Publish
Monitor Connection	Surveiller la connexion
Connection Status	État de la connexion
Error	Erreur
Bad	Mauvais
Good	Bien
Service Result	Résultat de Service
Transfer subscription	Transférer l'abonnement
Republish	Réédition

Figure 28 – Séquence de reconnexion

Lorsqu'un *Client* perd la connexion avec le *Serveur*, l'objectif est de se reconnecter sans perdre d'informations. Pour cela le *Client* doit rétablir la connexion en créant un nouveau *SecureChannel* et en activant la *Session* avec le *Service ActivateSession*. Cela attribue le nouveau *SecureChannel* à la *Session* existante et permet au *Client* de réutiliser la *Session* et les *Subscriptions* dans le *Serveur*. Par conséquent le *Client* reçoit des *Notifications* de données et d'événements sans perte d'informations, à condition que les files d'attente dans les *MonitoredItems* ne connaissent pas de dépassement de capacité.

Le *Client* ne doit créer une nouvelle *Session* que si *ActivateSession* échoue. *TransferSubscriptions* est utilisé pour transférer le *Subscription* vers la nouvelle *Session*. Si *TransferSubscriptions* échoue, il est nécessaire que le *Client* crée un nouveau *Subscription*.

Lorsque la connexion est perdue, les réponses *Publish* peuvent avoir été envoyées, mais peuvent ne pas avoir été reçues par le *Client*.

Une fois la connexion rétablie le *Client* doit appeler *Republish* dans une boucle, en commençant par le numéro de séquence suivant attendu et en incrémentant le numéro de séquence jusqu'à ce que le *Serveur* retourne le statut *Bad_MessageNotAvailable*. Après avoir reçu ce statut, le *Client* doit commencer à envoyer des demandes *Publish* avec la gestion *Publish* normale. Cette séquence garantit que les *NotificationMessages* perdus placés dans la file d'attente dans le *Serveur* ne sont pas écrasés par les nouvelles réponses *Publish*.

Si le *Client* détecte des numéros de séquence manquants dans les réponses *Publish* et n'est pas capable d'obtenir les *NotificationMessages* perdus via *Republish*, il convient que le *Client* lise les valeurs de tous les *MonitoredItems* des données afin de s'assurer qu'il possède les dernières valeurs pour tous les *MonitoredItems*.

Indépendamment de la stratégie de reprise détaillée, il convient que le *Client* s'assure que des valeurs plus anciennes fournies via *Republish* n'écrasent pas des données plus récentes dans le *Client*.

Si *Republish* retourne *Bad_SubscriptionIdInvalid*, alors il est nécessaire que le *Client* crée un nouveau *Subscription*.

7 Définitions de types de paramètres communs

7.1 Description d'une application (ApplicationDescription)

Les composantes de ce paramètre sont définies au Tableau 103.

Tableau 103 – ApplicationDescription

Nom	Type	Description
ApplicationDescription	structure	Spécifie une application qui est disponible.
applicationUri	String	L'identificateur globalement unique pour l'instance d'application. Cet URI est utilisé comme <i>ServerUri</i> dans des <i>Services</i> si l'application est un <i>Serveur</i> .
productUri	String	L'identificateur globalement unique pour le produit
applicationName	LocalizedText	Un nom descriptif localisé pour l'application.
applicationType	Enum ApplicationType	Le type d'application. Cette valeur est une énumération avec l'une des valeurs suivantes: SERVER_0 L'application est un <i>Serveur</i> . CLIENT_1 L'application est un <i>Client</i> . CLIENTANDSERVER_2 L'application est un <i>Client</i> et un <i>Serveur</i> . DISCOVERYSERVER_3 L'application est un <i>DiscoveryServer</i> .
gatewayServerUri	String	Un URI qui identifie le <i>Gateway Server</i> associé aux <i>discoveryUrls</i> . Cette valeur n'est pas spécifiée si le <i>Serveur</i> n'est pas directement accessible. Ce champ n'est pas utilisé si l' <i>applicationType</i> est CLIENT_1.
discoveryProfileUri	String	Un URI qui identifie le profil de découverte pris en charge par les URL fournis. Ce champ n'est pas utilisé si l' <i>applicationType</i> est CLIENT_1. Si cette valeur n'est pas spécifiée, les points d'extrémité (EndPoints) doivent prendre en charge les services Discovery définis en 5.4. Des profils de découverte de remplacement sont définis dans l'IEC 62541-7.
discoveryUrls []	String	Une liste d'URL pour les <i>Endpoints</i> de découverte fournis par l'application. Si l' <i>applicationType</i> est CLIENT_1, ce champ doit contenir une liste vide.

7.2 Certificat d'instance d'application (ApplicationInstanceCertificate)

Un *ApplicationInstanceCertificate* est une *ByteString* contenant un *Certificat* codé. Le codage d'un *ApplicationInstanceCertificate* dépend de la mise en correspondance de la technologie

de sécurité et il est complètement défini dans l'IEC 62541-6. Le Tableau 104 spécifie les informations qui doivent être contenues dans un *ApplicationInstanceCertificate*.

Tableau 104 – ApplicationInstanceCertificate

Nom	Type	Description
ApplicationInstanceCertificate	structure	<i>ApplicationInstanceCertificate</i> avec une signature créée par une <i>Certificate Authority</i> .
version	String	Un identificateur pour la version du codage du <i>Certificat</i> .
serialNumber	ByteString	Un identificateur unique pour le <i>Certificat</i> affecté par son Émetteur.
signatureAlgorithm	String	L'algorithme utilisé pour signer le <i>Certificat</i> . La syntaxe de ce champ dépend du codage du <i>Certificat</i> .
signature	ByteString	La signature créée par l'Émetteur.
issuer	Structure	Un nom qui identifie le <i>Certificat de l'Émetteur</i> utilisé pour créer la signature.
validFrom	UTCTime	Le moment où le <i>Certificat</i> devient valide.
validTo	UTCTime	Le moment où le <i>Certificat</i> expire.
subject	Structure	Un nom qui identifie l'instance d'application que le <i>Certificat</i> décrit. Ce champ doit contenir le <i>productName</i> et le nom de l'organisation responsable de l'instance d'application.
applicationUri	String	L' <i>applicationUri</i> spécifié dans l' <i>ApplicationDescription</i> . L' <i>ApplicationDescription</i> est décrit en 7.1.
hostnames []	String	Le nom de la machine où l'instance d'application s'exécute. Une machine peut avoir plusieurs noms si elle est accessible via plusieurs réseaux. Le hostname (nom d'hôte) peut être une adresse réseau numérique ou un nom descriptif. Les <i>Certificats</i> du <i>Serveur</i> doivent avoir au moins un hostname défini.
publicKey	ByteString	La clé publique associée au <i>Certificat</i> .
keyUsage []	String	Spécifie comment la clé de <i>Certificat</i> peut être utilisée. Les <i>ApplicationInstanceCertificates</i> doivent prendre en charge Digital Signature, Non-Repudiation Key Encryption, Data Encryption et Client/Server Authorization. Le contenu de ce champ dépend du codage du <i>Certificat</i> .

7.3 Parcourir les résultats (BrowseResult)

Les composantes de ce paramètre sont définies au Tableau 105.

Tableau 105 – BrowseResult

Nom	Type	Description
BrowseResult	structure	Les résultats d'une opération Browse (navigation)
statusCode	StatusCode	Le statut de la <i>BrowseDescription</i> . Cette valeur est mise à <i>Good</i> s'il y a encore des références à retourner pour la <i>BrowseDescription</i> .
continuationPoint	ContinuationPoint	Une valeur opaque définie par le <i>Serveur</i> qui identifie le point de continuation. Le type <i>ContinuationPoint</i> est défini en 7.6.
References []	ReferenceDescription	Le jeu de références qui satisfait aux critères spécifiés dans la <i>BrowseDescription</i> . Vide, s'il n'y a pas de <i>Références</i> satisfaisant aux critères. Le type <i>Reference Description</i> est défini en 7.24.

7.4 Composantes d'un filtre (ContentFilter)

7.4.1 Structure de composantes d'un filtre (ContentFilter structure)

La structure *ContentFilter* définit un ensemble d'éléments qui définissent des critères de filtrage. Chaque élément dans l'ensemble décrit un opérateur et une matrice d'opérandes à utiliser par l'opérateur. Les opérateurs qui peuvent être utilisés dans un *ContentFilter* sont décrits au Tableau 110. Le filtre est contrôlé par l'évaluation de la première entrée dans la matrice d'éléments en commençant par le premier opérande de la matrice d'opérandes. Les opérandes d'un élément peuvent contenir des *Références* à des sous-éléments faisant que l'évaluation se poursuive aux éléments référencés de la matrice d'éléments. L'évaluation ne doit pas introduire de boucles. Par exemple, l'évaluation commençant par l'élément A ne doit jamais pouvoir retourner à l'élément A. Cependant, il peut exister plusieurs chemins menant à l'autre élément B. Un élément qui ne peut pas être rattaché en retour à l'élément de départ

est ignoré. Pour tout opérateur, les opérandes supplémentaires doivent se traduire par une erreur. L'Annexe B fournit des exemples utilisant la structure *ContentFilter*.

Le Tableau 106 définit la structure *ContentFilter*.

Tableau 106 – Structure *ContentFilter*

Nom	Type	Description
<i>ContentFilter</i>	structure	
elements []	<i>ContentFilterElement</i>	Liste des opérateurs et de leurs opérandes qui composent les critères de filtrage. Le filtre est évalué en commençant par la première entrée de cette matrice. Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.
filterOperator	Enum <i>FilterOperator</i>	L'opérateur de filtre à évaluer. L'énumération <i>FilterOperator</i> est définie au Tableau 110.
filterOperands []	Extensible Parameter <i>FilterOperand</i>	Opérandes utilisés par l'opérateur sélectionné. Le nombre et l'utilisation dépendent des opérateurs définis au Tableau 110. Cette matrice a besoin d'une entrée au moins. Ce type de paramètre extensible est le type de paramètre <i>FilterOperand</i> spécifié en 7.4.4. Il spécifie la liste des valeurs valides de <i>FilterOperand</i> .

7.4.2 Composantes des résultats d'un filtre (*ContentFilterResult*)

Les composantes de ce type de données sont définies au Tableau 107.

Tableau 107 – Structure *ContentFilterResult*

Nom	Type	Description
<i>ContentFilterResult</i>	structure	Une structure qui contient toutes les éventuelles erreurs associées au filtre.
elementResults []	<i>ContentFilterElementResult</i>	Une liste de résultats pour les éléments individuels du filtre. La taille et l'ordre de la liste correspondent à la taille et à l'ordre des éléments dans le paramètre <i>ContentFilter</i> . Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.
statusCode	<i>StatusCode</i>	Le code de statut pour un élément seul.
operandStatusCodes []	<i>StatusCode</i>	Une liste de codes de statut pour les opérandes dans un élément. La taille et l'ordre de la liste correspondent à la taille et à l'ordre des opérandes dans le <i>ContentFilterElement</i> . Cette liste est vide s'il ne s'est pas produit d'erreurs d'opérande.
operandDiagnosticInfos []	<i>DiagnosticInfo</i>	Une liste d'informations de diagnostic pour les opérandes dans un élément. La taille et l'ordre de la liste correspondent à la taille et à l'ordre des opérandes dans le <i>ContentFilterElement</i> . Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été rencontrée dans le traitement des opérandes.
elementDiagnosticInfos []	<i>DiagnosticInfo</i>	Une liste d'informations de diagnostic pour les éléments individuels du filtre. La taille et l'ordre de la liste correspondent à la taille et à l'ordre des éléments dans le paramètre de demande <i>filter</i> . Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été rencontrée dans le traitement des éléments.

Le Tableau 108 définit les valeurs pour le paramètre *statusCode* qui sont spécifiques à cette structure. Les *StatusCodes* communs sont définis au Tableau 166.

Tableau 108 – Codes de résultats *ContentFilterResult*

Id symbolique	Description
Bad_FilterOperandCountMismatch	Le nombre d'opérandes fournis pour l'opérateur de filtre était plus faible que prévu pour l'opérande fourni.
Bad_FilterOperatorInvalid	Un opérateur non reconnu a été fourni dans un filtre.
Bad_FilterOperatorUnsupported	Un opérateur valide était fourni, mais le serveur ne prévoit pas de prise en charge pour l'opérateur du filtre.

Le Tableau 109 définit les valeurs pour le paramètre *operandStatusCode* qui sont spécifiques à cette structure. Les *StatusCodes* communs sont définis au Tableau 166.

Tableau 109 – Codes de résultats d'opérande ContentFilterResult

Id symbolique	Description
Bad_FilterOperandInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_FilterElementInvalid	L'élément référencé n'est pas un élément valide dans le filtre de contenu.
Bad_FilterLiteralInvalid	Le littéral référencé n'est pas un BaseDataType valide.
Bad_AttributeIdInvalid	L'identificateur d'attribut n'est pas un identificateur d'attribut valide dans le système.
Bad_IndexRangeInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_NodeIdInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_NodeIdUnknown	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_NotTypeDefinition	Le NodeId fourni n'était pas un NodeId de définition de type.

7.4.3 Opérateurs d'un filtre (FilterOperator)

Le Tableau 110 définit les opérateurs de base qui peuvent être utilisés dans un *ContentFilter*. Voir le Tableau 111 pour une description d'opérateurs avancés. Voir 7.4.4 pour une définition des opérandes.

Tableau 110 – Définition de FilterOperator de base

Opérateur	Nombre d'opérandes	Description
Equals_0	2	TRUE si operand[0] est égal à operand[1]. Si les opérandes sont de types différents, le système doit exécuter toute conversion implicite en un type commun. Cet opérateur décide FALSE si aucune conversion implicite n'est disponible et les opérandes sont de types différents. Cet opérateur retourne FALSE si la conversion implicite échoue. Voir le débat relatif à la préséance des types de données au Tableau 114 pour plus d'informations sur la façon de convertir des opérandes de types différents.
IsNull_1	1	TRUE si operand[0] est une valeur null.
GreaterThan_2	2	TRUE si operand[0] est supérieur à operand[1]. Les restrictions suivantes s'appliquent aux opérandes: [0]: Tout opérande qui se résout en une valeur ordonnée. [1]: Tout opérande qui se résout en une valeur ordonnée. Les mêmes règles de conversion définies pour <i>Equals</i> s'appliquent.
LessThan_3	2	TRUE si operand[0] est inférieur à operand[1]. Les mêmes règles de conversion et restrictions définies pour <i>GreaterThan</i> s'appliquent.
GreaterThanOrEqual_4	2	TRUE si operand[0] est supérieur ou égal à operand[1]. Les mêmes règles de conversion et restrictions définies pour <i>GreaterThan</i> s'appliquent.
LessThanOrEqual_5	2	TRUE si operand[0] est inférieur ou égal à operand[1]. Les mêmes règles de conversion et restrictions définies pour <i>GreaterThan</i> s'appliquent.
Like_6	2	TRUE si operand[0] correspond à un motif défini par operand[1]. Voir le Tableau 112 pour la définition de la syntaxe des motifs. Les restrictions suivantes s'appliquent aux opérandes: [0]: Tout opérande qui se résout en une String. [1]: Tout opérande qui se résout en une String. Cet opérateur décide FALSE si aucun opérande ne peut se résoudre en une string.
Not_7	1	TRUE si operand[0] est FALSE. Les restrictions suivantes s'appliquent aux opérandes: [0]: Tout opérande qui se résout en un booléen. Si l'opérande ne peut pas se résoudre en un booléen, le résultat est NULL. Voir ci-dessous un débat relatif à la gestion de NULL.
Between_8	3	TRUE si operand[0] est supérieur ou égal à operand[1] et inférieur ou égal à operand[2]. Les restrictions suivantes s'appliquent aux opérandes: [0]: Tout opérande qui se résout en une valeur ordonnée. [1]: Tout opérande qui se résout en une valeur ordonnée. [2]: Tout opérande qui se résout en une valeur ordonnée. Si les opérandes sont de types différents, le système doit exécuter toute conversion implicite pour faire correspondre tous les opérandes à un type commun. Si aucune conversion implicite n'est disponible et les opérandes sont de types différents, le résultat particulier est FALSE. Voir le débat relatif à la préséance des types de données au Tableau 114 pour plus d'informations sur la façon de convertir des opérandes de types différents.
InList_9	2..n	TRUE si operand[0] est égal à l'un ou plusieurs des opérandes restants.

Opérateur	Nombre d'opérandes	Description
		L'opérateur Equals est évalué pour l'operand[0] et chaque opérande restant de la liste. Si une évaluation Equals quelconque est TRUE, InList retourne TRUE.
And_10	2	TRUE si operand[0] et operand[1] sont TRUE. Les restrictions suivantes s'appliquent aux opérandes: [0]: Tout opérande qui se résout en un booléen. [1]: Tout opérande qui se résout en un booléen. Si un opérande quelconque ne peut pas se résoudre en un booléen, il est considéré NULL. Voir ci-dessous un débat relatif à la gestion de NULL.
Or_11	2	TRUE si operand[0] ou operand[1] est TRUE. Les restrictions suivantes s'appliquent aux opérandes: [0]: Tout opérande qui se résout en un booléen. [1]: Tout opérande qui se résout en un booléen. Si un opérande quelconque ne peut pas se résoudre en un booléen, il est considéré NULL. Voir ci-dessous un débat relatif à la gestion de NULL.
Cast_12	2	Convertit operand[0] en une valeur ayant un type de données avec un NodeId identifié par operand[1]. Les restrictions suivantes s'appliquent aux opérandes: [0]: N'importe quel opérande. [1]: N'importe quel opérande qui se résout en un NodeId ou ExpandedNodeId lorsque le <i>Noeud</i> est le <i>NodeClass DataType</i> . S'il y a une erreur de conversion ou dans l'un des paramètres, l'opération Cast évalue à NULL. Voir ci-dessous un débat relatif à la gestion de NULL.
BitwiseAnd_16	2	Le résultat est un nombre entier qui correspond à la taille du plus grand opérande et contient une opération And sur les bits appliquée aux deux opérandes lorsque tous deux ont été convertis à la même taille (celle du plus grand des deux opérandes) Les restrictions suivantes s'appliquent aux opérandes: [0]: Tout opérande qui se résout en un nombre entier. [1]: Tout opérande qui se résout en un nombre entier. Si un opérande quelconque ne peut pas se résoudre en un nombre entier, il est considéré NULL. Voir ci-dessous un débat relatif à la gestion de NULL.
BitwiseOr_17	2	Le résultat est un nombre entier qui correspond à la taille du plus grand opérande et contient une opération Or sur les bits appliquée aux deux opérandes lorsque tous deux ont été convertis à la même taille (celle du plus grand des deux opérandes) Les restrictions suivantes s'appliquent aux opérandes: [0]: Tout opérande qui se résout en un nombre entier. [1]: Tout opérande qui se résout en un nombre entier. Si un opérande quelconque ne peut pas se résoudre en un Nombre entier, il est considéré NULL. Voir ci-dessous un débat relatif à la gestion de NULL.

De nombreux opérandes ont des restrictions dues à leur type. Cela exige que l'opérande soit évalué pour déterminer son type. Dans certains cas, il s'agit du type spécifié dans l'opérande (c'est-à-dire un *LiteralOperand*). Dans d'autres cas, le type exige que la valeur d'un attribut soit lue. Un *ElementOperand* évalué à une valeur booléenne à moins que l'opérateur ne soit Cast ou un opérateur *RelatedTo* imbriqué.

Le Tableau 111 définit des opérateurs complexes qui exigent un nœud cible (c'est-à-dire rangée) à évaluer. Ces opérateurs doivent être réévalués pour chaque nœud cible possible dans le jeu de résultats.

Tableau 111 – Définition de FilterOperator complexe

Opérateur	Nombre d'opérandes	Description
InView_13	1	TRUE si le <i>Nœud</i> cible est contenu dans la <i>Vue</i> définie par operand[0]. Les restrictions suivantes s'appliquent aux opérandes: [0]: Tout opérande qui se résout en un <i>NodeId</i> qui identifie une <i>Vue</i> du Nœud. Si operand[0] ne se résout pas en un <i>NodeId</i> qui identifie une <i>Vue</i> du Nœud, cette opération doit toujours être False.
OfType_14	1	TRUE si le <i>Nœud</i> cible est du type operand[0] ou d'un sous-type de operand[0]. Les restrictions suivantes s'appliquent aux opérandes: [0]: Tout opérande qui se résout en un <i>NodeId</i> qui identifie un Nœud ObjectType ou VariableType. Si operand[0] ne se résout pas en un <i>NodeId</i> qui identifie un Nœud ObjectType ou VariableType, cette opération doit toujours être False.
RelatedTo_15	6	TRUE si le <i>Nœud</i> cible est du type Operand[0] et est relié à un <i>NodeId</i> du type défini dans Operand[1] par le type <i>Référence</i> défini dans Operand[2]. Operand[0] ou Operand[1] peut aussi être un élément <i>Référence</i> où l'élément auquel il est fait référence est un autre opérateur RelatedTo. Cela permet le chaînage de relations (par exemple A est relié à B qui lui-même est relié à C), la relation étant définie par le <i>ReferenceType</i> défini dans Operand[2]. Dans ce cas, l'élément auquel il est fait référence retourne une liste de <i>NodeIds</i> au lieu de TRUE ou FALSE. Dans ce cas, si des erreurs apparaissent ou l'un quelconque des opérandes ne peut se résoudre en une valeur appropriée, le résultat de la relation chaînée est une liste vide de nœuds. Operand[3] définit le nombre de sauts pour lequel il convient que la relation suive. Si Operand[3] est 1, les objets doivent être reliés directement. Si un saut est supérieur à 1, un <i>NodeId</i> du type décrit dans l'Operand[1] est recherché à la profondeur spécifiée par le saut. Dans ce cas, le type de <i>Nœud</i> intermédiaire est indéfini, et seul le type <i>Référence</i> utilisé pour atteindre le <i>Nœud</i> final est défini. Si le nombre demandé de sauts ne peut pas être suivi, le résultat est FALSE, à savoir, une liste de <i>Nœuds</i> vide. Si Operand[3] est 0, la relation est suivie jusqu'à sa fin logique dans le sens direct et chaque <i>Nœud</i> est vérifié pour savoir s'il est du type spécifié dans Operand[1]. Si un <i>Nœud</i> quelconque satisfait à ce critère, le résultat est TRUE, à savoir, le <i>NodeId</i> est inclus dans une sous-liste. Operand[4] définit s'il convient que les opérandes [0] et [1] incluent la prise en charge des sous-types des types définis par ces opérandes. Un TRUE indique la prise en charge des sous-types. Operand[5] définit s'il convient qu'Operand[2] inclue la prise en charge pour les sous-types du type de référence. Un TRUE indique la prise en charge des sous-types. Les restrictions suivantes s'appliquent aux opérandes: [0]: Tout opérande qui se résout en un <i>NodeId</i> ou <i>ExpandedNodeId</i> qui identifie un Nœud ObjectType ou VariableType ou une référence à un autre élément qui est un opérateur RelatedTo. [1]: Tout opérande qui se résout en un <i>NodeId</i> ou <i>ExpandedNodeId</i> qui identifie un Nœud ObjectType ou VariableType ou une référence à un autre élément qui est un opérateur RelatedTo. [2]: Tout opérande qui se résout en un <i>NodeId</i> qui identifie un Nœud ReferenceType. [3]: Tout opérande qui se résout en une valeur implicitement convertible en Uint32. [4]: Tout opérande qui se résout en une valeur implicitement convertible en un booléen. Si l'opérande ne se résout pas en un booléen, une valeur FALSE est utilisée. [5]: Tout opérande qui se résout en une valeur implicitement convertible en un booléen. Si l'opérande ne se résout pas en un booléen, une valeur FALSE est utilisée. Si aucun des opérandes [0], [1], [2], [3] ne se résout en une valeur appropriée, le résultat de cette opération doit toujours être False (ou un jeu vide "Empty" dans le cas d'un opérande <i>RelatedTo</i> imbriqué). Voir des exemples de RelatedTo en B.2.

L'opérateur RelatedTo peut être utilisé pour identifier si un type donné, établi comme operand[1], est un sous-type d'un autre type établi comme operand[0] en mettant operand[2] au *ReferenceType HasSubtype* et operand[3] à 0.

L'opérateur *Like* peut être utilisé pour effectuer des comparaisons de caractères de remplacement (Jokers). Plusieurs caractères spéciaux peuvent être inclus dans le second opérande de l'opérateur *Like*. Les caractères valides sont définis au Tableau 112. Les caractères de remplacement peuvent être combinés en une seule chaîne (à savoir 'Th[ia][ts]%' pourrait correspondre à 'That is fine', 'This is fine', 'That as one', 'This it is', 'Then at any', etc.). L'opérateur *Like* est sensible à la casse.

Tableau 112 – Caractères de remplacement

Caractère spécial	Description
%	Correspond à toute chaîne de zéro ou plus de caractères (à savoir 'main%' correspondrait à toute chaîne qui commence par 'main', '%en%' correspondrait à toute chaîne qui contient les lettres 'en' telle que 'entail', 'green' et 'content'.) Si un signe '%' est prévu dans une chaîne, l'opérande de liste peut être utilisé (à savoir 5[%] correspondrait à '5%').
_	Correspond à tout caractère seul (à savoir '_ould' correspondrait à 'would', 'could'). Si un signe '_' est prévu dans une chaîne, l'opérande de liste peut être utilisé (à savoir 5[_] correspondrait à '5_').
\	Le caractère Escape (échappement) permet l'interprétation littérale (à savoir \ est \, \% est %, _ est _).
[]	Correspond à tout caractère seul dans une liste (à savoir 'abc[13-68]' correspondrait à 'abc1', 'abc3', 'abc4', 'abc5', 'abc6', et 'abc8'. 'xyz[c-f]' correspondrait à 'xyzc', 'xyzd', 'xyze', 'xyzf').
[^]	Ne correspondant à aucun caractère dans une liste. Le ^ doit être le premier caractère à l'intérieur du []. (à savoir 'ABC[^13-5]' NE correspondrait PAS à 'ABC1', 'ABC3', 'ABC4', et 'ABC5'. 'xyz[^dgh]' NE correspondrait PAS 'xyzd', 'xyzg', 'xyzh'.)

Le Tableau 113 définit les règles de conversion pour les valeurs des opérandes. Les types sont automatiquement convertis si une conversion implicite existe (I). Si une conversion explicite existe (E), le type peut être converti avec l'opérateur cast. Si aucune conversion n'existe (X), les types ne peuvent pas être convertis, mais certains serveurs peuvent toutefois prendre en charge des conversions explicites spécifiques à l'application. Les types utilisés dans le tableau sont définis dans l'IEC 62541-3. Un type de données qui ne figure pas dans le tableau n'a pas de conversions définies.

Tableau 113 – Règles de conversion

Type cible (To)	Type source (From)																	
	Boolean	Byte	ByteString	DateTime	Double	ExpandedNodeId	Float	Guid	Int16	Int32	Int64	NodeId	SByte	StatusCode	String	LocalizedText	QualifiedName	UInt16
Boolean	-	I	X	X	I	X	I	X	I	I	I	X	I	X	E	X	X	I
Byte	E	-	X	X	I	X	I	X	I	I	I	X	I	X	E	X	X	I
ByteString	X	X	-	X	X	X	X	E	X	X	X	X	X	X	X	X	X	X
DateTime	X	X	X	-	X	X	X	X	X	X	X	X	X	X	E	X	X	X
Double	E	E	X	X	-	X	E	X	E	E	E	X	E	X	E	X	X	E
ExpandedNodeId	X	X	X	X	X	-	X	X	X	X	X	E	X	X	I	X	X	X
Float	E	E	X	X	I	X	-	X	E	E	E	X	E	X	E	X	X	E
Guid	X	X	E	X	X	X	X	-	X	X	X	X	X	X	E	X	X	X
Int16	E	E	X	X	I	X	I	X	-	I	I	X	E	X	E	X	X	E
Int32	E	E	X	X	I	X	I	X	E	-	I	X	E	E	E	X	X	E
Int64	E	E	X	X	I	X	I	X	E	E	-	X	E	E	E	X	X	E
NodeId	X	X	X	X	X	I	X	X	X	X	X	-	X	X	I	X	X	X
SByte	E	E	X	X	I	X	I	X	I	I	I	X	-	X	E	X	X	I
StatusCode	X	X	X	X	X	X	X	X	X	I	I	X	X	-	X	X	X	E
String	I	I	X	E	I	E	I	I	I	I	I	E	I	X	-	E	E	I
LocalizedText	X	X	X	X	X	X	X	X	X	X	X	X	X	X	I	-	X	X
QualifiedName	X	X	X	X	X	X	X	X	X	X	X	X	X	X	I	I	-	X
UInt16	E	E	X	X	I	X	I	X	I	I	I	X	E	I	E	X	X	-
UInt32	E	E	X	X	I	X	I	X	E	I	I	X	E	E	E	X	X	E
UInt64	E	E	X	X	I	X	I	X	E	E	I	X	E	E	E	X	X	E
XmlElement	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X

Les matrices d'un type source peuvent être converties en matrices d'un type cible par conversion de chaque élément. Une erreur de conversion pour un élément quelconque entraîne l'échec de toute la conversion.

Les matrices de longueur 1 peuvent être converties implicitement en une valeur scalaire du même type.

Guid, *NodeId* et *ExpandedNodeId* sont convertis en *String*, et inversement, en utilisant la syntaxe définie dans l'IEC 62541-6.

Les valeurs à virgule flottante sont arrondies en ajoutant 0,5 et en les tronquant lorsqu'elles sont converties en valeurs entières.

La conversion d'une valeur négative en un type non signé provoque une erreur de conversion. Si la conversion échoue, le résultat est une valeur null.

La conversion d'une valeur hors de la plage du type cible provoque une erreur de conversion. Si la conversion échoue, le résultat est une valeur null.

ByteString est convertie en *String* en formatant les octets sous la forme d'une séquence de chiffres hexadécimaux.

Les valeurs *LocalizedText* sont converties en *Strings* en abandonnant le *Locale* (paramètre de lieu). Les chaînes *String* sont converties en valeurs *LocalizedText* en mettant le *Locale* à "".

Les valeurs *QualifiedName* sont converties en *Strings* en abandonnant le *NamespaceIndex* (indice d'espace de noms). Les chaînes *String* sont converties en valeurs *QualifiedName* en mettant le *NamespaceIndex* à 0.

Un *StatusCode* peut être converti en *UInt32* et *Int32*, et inversement, en copiant les bits. Seuls les 16 bits de poids fort dans le *StatusCode* sont copiés lorsqu'il est converti en une valeur *UInt16* ou *Int16*, et inversement.

Les valeurs *booléennes* sont converties en '1' lorsqu'elles sont true (vraies) et en '0' lorsqu'elles sont false (fausses). Les valeurs numériques non nulles sont converties en valeurs *booléennes* vraies ("true"). Les valeurs numériques de 0 sont converties en valeurs *booléennes* false ("fausses"). Les valeurs *String* contenant "true", "false", "1" ou "0" peuvent être converties en valeurs *booléennes*. Les autres valeurs *String* (chaînes) provoquent une erreur de conversion. Dans ce cas, les *Strings* ne sont pas sensibles à la casse.

Il est parfois possible d'utiliser des opérateurs cast implicites lorsque des opérandes ayant des types différents sont utilisés dans une opération. Dans cette situation, les règles de préséance définies au Tableau 114 sont utilisées pour déterminer la conversion implicite à utiliser. Le premier type de données dans la liste (de haut en bas) a la préséance la plus élevée. Si un type de données ne figure pas dans ce tableau, il ne peut pas être converti implicitement lors de l'évaluation d'une opération.

Par exemple, supposons que A = 1,1 (*Float*) et B = 1 (*Int32*) et que ces valeurs sont utilisées avec un opérateur *Equals*. Cette opération serait évaluée en forçant la conversion de la valeur *Int32* en *Float*, car le type de données *Float* a une préséance plus élevée.

Tableau 114 – Règles de préséance des données

Rang	Type de données
1	Double
2	Float
3	Int64
4	UInt64
5	Int32
6	UInt32
7	StatusCode
8	Int16
9	UInt16
10	SByte
11	Byte
12	Boolean
13	Guid
14	String
15	ExpandedNodeId
16	NodeId
17	LocalizedText
18	QualifiedName

Les opérandes peuvent contenir des valeurs null (à savoir des valeurs qui n'existent pas). Lorsque cela se produit, l'élément s'évalue toujours à NULL (sauf si l'opérateur *IsNull_1* a été spécifié). Le Tableau 115 définit comment combiner des éléments qui s'évaluent à NULL avec d'autres éléments dans une opération AND logique.

Tableau 115 – Table de vérité du AND (ET) logique

	TRUE (vrai)	FALSE (faux)	NULL
TRUE (vrai)	TRUE (vrai)	FALSE (faux)	NULL
FALSE (faux)	FALSE (faux)	FALSE (faux)	FALSE (faux)
NULL	NULL	FALSE (faux)	NULL

Le Tableau 116 définit comment combiner des éléments qui s'évaluent à NULL avec d'autres éléments dans une opération OR logique.

Tableau 116 – Table de vérité du OR (OU) logique

	TRUE (vrai)	FALSE (faux)	NULL
TRUE (vrai)	TRUE (vrai)	TRUE (vrai)	TRUE (vrai)
FALSE (faux)	TRUE (vrai)	FALSE (faux)	NULL
NULL	TRUE (vrai)	NULL	NULL

L'opérateur NOT évalue toujours à NULL s'il est appliqué à un opérande NULL.

Un *ContentFilter* qui évalue à NULL après que tous les éléments ont été évalués est évalué comme false.

7.4.4 Paramètres de FilterOperand

7.4.4.1 Vue d'ensemble

La structure *ContentFilter* spécifiée en 7.4 définit un ensemble d'éléments qui constituent les critères de filtrage et contient différents types de *FilterOperands*. Le paramètre *FilterOperand* est un paramètre extensible. Ce paramètre est défini au Tableau 117. Le type *ExtensibleParameter* est défini en 7.11.

Tableau 117 – Typelds du paramètre FilterOperand

Id symbolique	Description
Element	Spécifie un indice dans la matrice d'éléments. Ce type est utilisé pour construire une arborescence logique de sous-éléments en reliant l'opérande d'un élément à un sous-élément.
Literal	Spécifie une valeur littérale.
Attribute	Spécifie n'importe quel <i>Attribut</i> d'un <i>Nœud Object</i> ou <i>Variable</i> utilisant un <i>Nœud</i> dans le système de type et le chemin relatif construits à partir de <i>ReferenceTypes</i> et de <i>BrowseNames</i> .
SimpleAttribute	Spécifie n'importe quel <i>Attribut</i> d'un <i>Nœud Object</i> ou <i>Variable</i> utilisant une <i>TypeDefinition</i> et un chemin relatif construit à partir de <i>BrowseNames</i> .

7.4.4.2 ElementOperand

L'*ElementOperand* fournit la liaison à des sous-éléments dans un *ContentFilter*. La liaison se présente sous la forme d'un entier qui est utilisé pour indexer dans la matrice d'éléments contenus dans le *ContentFilter*. Un indice est considéré valide si sa valeur est supérieure à l'indice d'élément dont il fait partie et il ne *réfère* pas un élément inexistant. Les *Clients* doivent construire des filtres de cette manière afin d'éviter des *Références* circulaires non valides. Il convient que les *Serveurs* se protègent des indices non valables en vérifiant l'indice avant de l'utiliser.

Le Tableau 118 définit le type *ElementOperand*.

Tableau 118 – ElementOperand

Nom	Type	Description
ElementOperand	structure	Valeur d'ElementOperand.
index	UInt32	Indice dans la matrice d'éléments.

7.4.4.3 LiteralOperand

Le Tableau 119 définit le type *LiteralOperand*.

Tableau 119 – LiteralOperand

Nom	Type	Description
LiteralOperand	structure	Valeur de LiteralOperand.
value	BaseDataType	Une valeur littérale.

7.4.4.4 AttributeOperand

Le Tableau 120 définit le type *AttributeOperand*.

Tableau 120 – AttributeOperand

Nom	Type	Description
AttributeOperand	structure	Attribut d'un <i>Nœud</i> dans l'espace d'adresses.
nodeId	NodeId	<i>NodeId</i> d'un <i>Nœud</i> issu du système de type.
alias	String	Un paramètre facultatif utilisé pour identifier un pseudonyme ou s'y référer. Un pseudonyme est un nom symbolique qui peut être utilisé pour désigner cet opérande et l'utiliser dans d'autres endroits de la structure de fichiers.
browsePath	RelativePath	Chemin de navigation relatif au <i>Nœud</i> identifié par le paramètre <i>nodeId</i> . Voir 7.25 pour la définition de <i>RelativePath</i> .
AttributeID	IntegerId	Identificateur de l' <i>Attribut</i> . Il doit s'agir d'un <i>AttributeId</i> valide. Le type <i>IntegerId</i> est défini en 7.13. Les <i>IntegerIds</i> pour les attributs sont définis dans l'IEC 62541-6.
indexRange	NumericRange	Ce paramètre est utilisé pour identifier un élément simple d'une matrice ou une plage simple d'indices pour une matrice. Le premier élément est identifié par l'indice 0 (zéro). Le type <i>NumericRange</i> est défini en 7.21. Ce paramètre n'est pas utilisé si l' <i>Attribut</i> spécifié n'est pas une matrice. Cependant, si l' <i>Attribut</i> spécifié est une matrice et ce paramètre n'est pas utilisé, tous les éléments sont à inclure dans la plage. Ce paramètre est null s'il n'est pas utilisé.

7.4.4.5 SimpleAttributeOperand

Le *SimpleAttributeOperand* est une forme simplifiée de l'*AttributeOperand* et toutes les règles qui s'appliquent à l'*AttributeOperand* s'appliquent aussi au *SimpleAttributeOperand*. Les exemples fournis en B.1 utilisent seulement *AttributeOperand*. Cependant, l'*AttributeOperand* peut être remplacé par un *SimpleAttributeOperand* chaque fois que tous les *ReferenceTypes* dans le *RelativePath* sont des sous-types des *HierarchicalReferences* et les cibles sont des *Nœuds Object* ou *Variable* et un pseudonyme (*Alias*) n'est pas requis.

Le Tableau 121 définit le type *SimpleAttributeOperand*.

Tableau 121 – SimpleAttributeOperand

Nom	Type	Description
SimpleAttributeOperand	structure	Attribut d'un <i>Nœud</i> dans l'espace d'adresses.
typeId	NodeId	<i>NodeId</i> d'un <i>TypeDefinitionNode</i> . Ce paramètre limite l'opérande à des instances du <i>TypeDefinitionNode</i> ou l'un de ses sous-types.
browsePath []	QualifiedName	Un chemin relatif vers un <i>Nœud</i> . Ce paramètre spécifie un chemin relatif utilisant une liste de <i>BrowseNames</i> en lieu et place de la structure <i>RelativePath</i> utilisée dans l' <i>AttributeOperand</i> . La liste des <i>BrowseNames</i> équivaut à un <i>RelativePath</i> qui spécifie des références directes qui sont des sous-types du <i>ReferenceType HierarchicalReferences</i> . Tous les <i>Nœuds</i> suivis par le <i>browsePath</i> doivent être de la <i>NodeClass Object</i> ou <i>Variable</i> . Si cette liste est vide, le <i>Nœud</i> est l'instance de la <i>TypeDefinition</i> .
attributeId	IntegerId	Identificateur de l' <i>Attribut</i> . L' <i>IntegerId</i> est défini en 7.13. L' <i>Attribut Value</i> doit être pris en charge par tous les <i>Serveurs</i> . La prise en charge d'autres <i>Attributs</i> dépend des exigences établies dans des Profils ou autres parties de cette spécification.
indexRange	NumericRange	Ce paramètre est utilisé pour identifier un élément simple d'une matrice ou une plage simple d'indices pour une matrice. Le premier élément est identifié par l'indice 0 (zéro). Ce paramètre est ignoré si le <i>Nœud</i> sélectionné n'est pas une <i>Variable</i> ou la <i>Valeur</i> d'une <i>Variable</i> n'est pas une matrice. Ce paramètre est null s'il n'est pas spécifié. Toutes les valeurs dans la matrice sont utilisées si ce paramètre n'est pas spécifié. Le type <i>NumericRange</i> est défini en 7.21.

7.5 Compteur (Counter)

Ce type de données de primitive est un *UInt32* qui représente la valeur d'un compteur. La valeur initiale d'un compteur est spécifiée par son utilisation. L'arithmétique modulaire est utilisée pour tous les calculs, le module y étant $(\text{max value} + 1)$. Par conséquent,

$$x + y = (x + y) \bmod (\text{max value} + 1)$$

Par exemple:

$$\text{max value} + 1 = 0$$

$$\text{max value} + 2 = 1$$

7.6 Point de continuation (ContinuationPoint)

Un *ContinuationPoint* est utilisé pour suspendre une opération *Browse* ou *QueryFirst* et permettre qu'elle soit relancée en appelant *BrowseNext* ou *QueryNext*. Les opérations sont suspendues lorsque le nombre de résultats trouvés dépasse la limite fixée par le *Client* ou le *Serveur*.

Le *Client* spécifie le nombre maximal de résultats par opération dans le message de demande. Un *Serveur* ne doit pas en retourner plus que ce nombre de résultats, mais il peut en retourner moins. Le *Serveur* attribue un *ContinuationPoint* s'il y a plus de résultats à retourner. Le *Serveur* doit toujours retourner au moins un résultat s'il retourne un *ContinuationPoint*.

Les *Serveurs* doivent prendre en charge au moins un *ContinuationPoint* par *Session*. Les *Serveurs* spécifient un nombre maximal de *ContinuationPoints* par *Session* dans l'*Object ServerCapabilities* défini dans l'IEC 62541-5. Les *ContinuationPoints* restent actifs jusqu'à ce que le *Client* récupère les résultats restants, le *Client* libère le *ContinuationPoint* ou la *Session* est fermée. Le *Serveur* doit automatiquement libérer les *ContinuationPoints* des demandes précédentes s'ils sont nécessaires au traitement d'une demande. Le *Serveur* retourne une erreur *Bad_ContinuationPointInvalid* si un *Client* essaye d'utiliser un *ContinuationPoint* qui a été libéré.

Les demandes spécifient souvent plusieurs opérations qui peuvent ou peuvent ne pas nécessiter un *ContinuationPoint*. Un *Serveur* doit traiter les opérations jusqu'à ce qu'il n'ait plus de *ContinuationPoints*. Une fois que cela se produit, le *Serveur* doit retourner une erreur *Bad_NoContinuationPoints* pour les opérations restantes.

Un *Client* redémarre une opération en rendant le *ContinuationPoint* au *Serveur*. Il convient que le *Serveur* soit toujours capable de réutiliser le *ContinuationPoint* fourni, donc les *Serveurs* ne doivent jamais retourner une erreur *Bad_NoContinuationPoints* lorsqu'ils exécutent une opération précédemment arrêtée.

Un *ContinuationPoint* est un sous-type du type de données *ByteString*.

7.7 Valeur d'une donnée (DataValue)

7.7.1 Généralités

Les composantes de ce paramètre sont définies au Tableau 122.

Tableau 122 – DataValue

Nom	Type	Description
DataValue	structure	La valeur et les informations associées.
value	BaseDataType	La valeur de donnée. Si le <i>StatusCode</i> indique une erreur, la valeur est à ignorer et le <i>Serveur</i> doit la mettre à null.
statusCode	StatusCode	Le <i>StatusCode</i> qui définit la capacité du <i>Serveur</i> à accéder/fournir la valeur. Le type <i>StatusCode</i> est défini en 7.33
sourceTimestamp	UTCTime	Horodatage de la source pour la valeur.
sourcePicoSeconds	UInteger	Spécifie le nombre d'intervalles de 10 picosecondes (1,0 e-11 secondes) qui doit être ajouté au <i>sourceTimestamp</i> .
serverTimestamp	UTCTime	Horodatage du <i>Serveur</i> pour la valeur.
serverPicoSeconds	UInteger	Spécifie le nombre d'intervalles de 10 picosecondes (1,0 e-11 secondes) qui doit être ajouté au <i>serverTimestamp</i> .

7.7.2 PicoSeconds

Certaines applications requièrent des horodatages de haute résolution. Les champs *PicoSeconds* permettent aux applications de spécifier des horodatages avec une résolution de 10 picosecondes. La taille réelle du champ *PicoSeconds* dépend de la résolution du *DataType UtcTime*. Par exemple, si le *DataType UtcTime* a une résolution de 100 nanosecondes, le champ *PicoSeconds* aurait à stocker des valeurs jusqu'à 10 000 afin de fournir la résolution de 10 picosecondes. La résolution du *DataType UtcTime* (Type de données Temps TUC) dépend des *Mappings* (Mises en correspondance) définies dans l'IEC 62541-6.

7.7.3 SourceTimestamp

Le *sourceTimestamp* est utilisé pour refléter l'horodatage qui a été appliqué à la valeur *Variable* par la source de données. Une fois qu'une valeur a été affectée à un horodatage de source, l'horodatage de source pour cette instance de valeur ne change jamais. Dans ce contexte, l'instance de valeur (Value instance) se réfère à la valeur reçue, quelle que soit sa valeur réelle.

Le *sourceTimestamp* doit être l'heure TUC et il convient qu'il indique l'heure du dernier changement de la *value* ou du *statusCode*.

Il convient que le *sourceTimestamp* soit généré le plus près possible de la source de la valeur, mais il est nécessaire que l'horodatage soit placé par la même horloge physique. Dans le cas des sources redondantes, il convient que les horloges des sources soient synchronisées.

Si le *Serveur* OPC UA reçoit la valeur *Variable* d'un autre *Serveur* OPC UA, le *Serveur* OPC UA doit toujours transmettre l'horodatage de la source sans changements. Si la source qui applique l'horodatage n'est pas disponible, l'horodatage de la source est mis à null. Par exemple, si une valeur ne peut pas être lue en raison d'une erreur au cours du traitement comme arguments invalides passés dans la demande, le *sourceTimestamp* doit être null.

Dans le cas d'un statut incertain ou mauvais, *sourceTimestamp* est utilisé pour refléter l'heure à laquelle la source a reconnu le statut autre que good (correct) ou l'heure de la dernière fois que le *Serveur* a essayé une reprise sur statut incertain ou mauvais.

Le *sourceTimestamp* est seulement retourné avec un *Attribut Value*. Pour tous les autres *Attributs*, le *sourceTimestamp* retourné est mis à null.

7.7.4 ServerTimestamp

Le *serveurTimestamp* est utilisé pour refléter l'heure à laquelle le *Serveur* a reçu une valeur *Variable* ou a su qu'elle était exacte.

Dans le cas d'un statut incertain ou mauvais, *serverTimestamp* est utilisé pour refléter l'heure à laquelle le *Serveur* a reçu le statut ou l'heure correspondant à la dernière fois que le *Serveur* a essayé une reprise sur statut incertain ou mauvais.

Dans le cas où le *Serveur* OPC UA s'abonne à une valeur issue d'un autre *Serveur* OPC UA, chaque *Serveur* applique son propre *serverTimestamp*. Ce cas de figure contraste avec celui du *sourceTimestamp* dans lequel seul l'émetteur des données est autorisé à appliquer le *sourceTimestamp*.

Si le *Serveur* s'abonne à la valeur issue d'un autre *Serveur* toutes les dix secondes et que la valeur change, le *serverTimestamp* est mis à jour chaque fois qu'une nouvelle valeur est reçue. Si la valeur ne change pas, les nouvelles valeurs ne seront pas reçues lors de la *Subscription*. Cependant, en l'absence d'erreurs, le *Serveur* de réception applique un nouveau *serverTimestamp* toutes les dix secondes, car ne pas recevoir une valeur signifie que la valeur n'a pas changé. Ainsi, le *serverTimestamp* reflète l'heure à laquelle le *Serveur* a su que la valeur était exacte.

Ce concept s'applique aussi aux *Serveurs* OPC UA qui reçoivent des valeurs issues de sources de données pilotées par des exceptions. Par exemple, soient les hypothèses suivantes: le *Serveur* reçoit des valeurs issues d'un appareil piloté par des exceptions et

- a) l'appareil vérifie les valeurs toutes les 0,5 seconde,
- b) la connexion à l'appareil est bonne,
- c) l'appareil a envoyé une mise à jour il y a 3 minutes avec une valeur de 1,234.

Dans ce cas, la valeur du *Serveur* serait de 1,234 et le *serverTimestamp* serait mis à jour toutes les 0,5 seconde après la réception de la valeur.

7.7.5 StatusCode affecté à une valeur

Le *StatusCode* est utilisé pour indiquer les conditions dans lesquelles une valeur *Variable* a été générée et, de ce fait, peut être utilisé comme un indicateur de validité de la valeur. Le *StatusCode* est défini en 7.33.

Condition globale (sévérité)

- Un *StatusCode* avec une sévérité Good signifie que la valeur est de bonne qualité.
- Un *StatusCode* avec une sévérité Uncertain signifie que la qualité de la valeur est incertaine pour des raisons indiquées par le *SubCode*.

- Un *StatusCode* avec une sévérité Bad signifie que la valeur n'est pas utilisable pour des raisons indiquées par le *SubCode*.

Règles

- Le *StatusCode* indique la validité d'une valeur. Par conséquent, il est requis que les *Clients* vérifient au minimum le *StatusCode Severity* de tous les résultats, même s'ils ne vérifient pas les autres champs, avant d'accéder à la valeur et de l'utiliser.
- Un *Serveur*, qui ne prend pas en charge les Informations sur le statut, doit retourner un code de sévérité Good. Il est également acceptable qu'un *Serveur* retourne simplement une sévérité et un sous-statut non spécifique (0).
- Si le *Serveur* n'a pas de valeur connue, notamment si la *Severity* est BAD – il doit retourner une valeur NULL.

7.8 Information de diagnostic (DiagnosticInfo)

Les composantes de ce paramètre sont définies au Tableau 123.

Tableau 123 – DiagnosticInfo

Nom	Type	Description
DiagnosticInfo	structure	Informations de diagnostic spécifiques au fournisseur.
namespaceUri	Int32	Le <i>symbolicId</i> est défini dans le contexte d'un espace de noms. Cet espace de noms est représenté sous la forme d'une chaîne et est acheminé au <i>Client</i> dans le paramètre <i>stringTable</i> du paramètre <i>ResponseHeader</i> défini en 7.27. Le paramètre <i>namespaceIndex</i> contient l'indice dans la <i>stringTable</i> pour cette chaîne. -1 indique qu'aucune chaîne n'est spécifiée. Le <i>namespaceUri</i> ne doit pas être l'espace de noms OPC UA normalisé. Aucun <i>symbolicId</i> n'est fourni pour les <i>StatusCodes</i> normalisés.
symbolicId	Int32	Le <i>symbolicId</i> doit être utilisé pour identifier une erreur ou une condition spécifique au fournisseur; typiquement le résultat de certaines opérations internes du <i>Serveur</i> . La longueur maximale de cette chaîne est de 32 caractères. Il convient que les <i>Serveurs</i> souhaitant retourner un code de retour numérique convertissent le code de retour en chaîne et utilisent cette chaîne comme <i>symbolicId</i> (par exemple, "0xC0040007" ou "-4"). La chaîne d'identificateur symbolique est acheminée au <i>Client</i> dans le paramètre <i>stringTable</i> du paramètre <i>ResponseHeader</i> défini en 7.27. Le paramètre <i>symbolicId</i> contient l'indice dans la <i>stringTable</i> pour cette chaîne. -1 indique qu'aucune chaîne n'est spécifiée. Le <i>symbolicId</i> ne doit pas contenir de <i>StatusCodes</i> . Si le <i>localizedText</i> contient une traduction pour la description d'un <i>StatusCode</i> , le <i>symbolicId</i> est -1.
locale	Int32	La partie paramètre de lieu du texte localisé spécifique au fournisseur décrivant l'identificateur symbolique. La chaîne de texte localisé est acheminée au <i>Client</i> dans le paramètre <i>stringTable</i> du paramètre <i>ResponseHeader</i> défini en 7.27. Le paramètre <i>localizedTextIndex</i> contient l'indice dans la <i>stringTable</i> pour cette chaîne. -1 indique qu'aucune chaîne n'est spécifiée.
localizedText	Int32	Une chaîne de texte localisé spécifique au fournisseur décrit l'identificateur symbolique. La longueur maximale de cette chaîne de texte est de 256 caractères. La chaîne de texte localisé est acheminée au <i>Client</i> dans le paramètre <i>stringTable</i> du paramètre <i>ResponseHeader</i> défini en 7.27. Le paramètre <i>localizedTextIndex</i> contient l'indice dans la <i>stringTable</i> pour cette chaîne. -1 indique qu'aucune chaîne n'est spécifiée. Le <i>localizedText</i> fait référence à l'éventuel <i>symbolicId</i> ou à la chaîne qui décrit le <i>StatusCode</i> normalisé si le <i>Serveur</i> fournit les traductions. Si l'indice est -1, le <i>Serveur</i> n'a pas de traduction à retourner et il convient que le <i>Client</i> utilise la description du <i>StatusCode</i> invariable de la spécification.
additionalInfo	String	Informations de diagnostic spécifiques au fournisseur.
innerStatusCode	StatusCode	Le <i>StatusCode</i> issu de l'opération interne. De nombreuses applications effectuent des appels dans des systèmes sous-jacents au cours du traitement de la demande OPC UA. Un <i>Serveur</i> OPC UA a l'option de consigner le statut issu du système sous-jacent dans les informations de diagnostic.
innerDiagnosticInfo	DiagnosticInfo	Les informations de diagnostic associées au <i>StatusCode</i> interne.

7.9 Description d'un point d'extrémité (EndpointDescription)

Les composantes de ce paramètre sont définies au Tableau 124.

Tableau 124 – EndpointDescription

Nom	Type	Description
EndpointDescription	structure	Décrit un <i>Endpoint</i> pour un <i>Serveur</i> .
endpointUrl	String	L'URL de l' <i>Endpoint</i> décrit.
Server	ApplicationDescription	La description pour le <i>Serveur</i> auquel l' <i>Endpoint</i> appartient. Le type <i>ApplicationDescription</i> est défini en 7.1.
serverCertificate	ApplicationInstance Certificate	Le <i>Certificat d'Instance d'Application</i> émis au <i>Serveur</i> . Le type <i>ApplicationInstanceCertificate</i> est défini en 7.2.
securityMode	Enum MessageSecurityMode	Le type de sécurité à appliquer aux messages. Le type <i>MessageSecurityMode</i> est défini en 7.14. Un <i>Canal Sécurisé</i> peut avoir à être créé même si le <i>securityMode</i> est NONE. Le comportement exact dépend de la mise en correspondance utilisée et est décrit dans l'IEC 62541-6.
securityPolicyUri	String	L'URI pour la <i>SecurityPolicy</i> à utiliser pour sécuriser des messages. Le jeu d'URI connu et les <i>SecurityPolicies</i> qui leur sont associées sont définis dans l'IEC 62541-7.
userIdentityTokens []	UserTokenPolicy	Les jetons d'identité d'utilisateur que le <i>Serveur</i> acceptera. Le <i>Client</i> doit passer l'un des <i>UserIdentityTokens</i> dans la demande <i>ActivateSession</i> . Le type <i>UserTokenPolicy</i> est décrit en 7.36
transportProfileUri	String	L'URI du <i>Profile Transport</i> (profil de transport) pris en charge par l' <i>Endpoint</i> . L'IEC 62541-7 définit les URI pour les <i>Transport Profiles</i> (profils de transport).
securityLevel	Byte	Une valeur numérique qui indique le niveau de sécurité de l' <i>EndpointDescription</i> en comparaison à d'autres <i>EndpointDescriptions</i> pour le même <i>Serveur</i> . Une valeur de 0 indique que l' <i>EndpointDescription</i> n'est pas recommandée et n'est prise en charge que pour la rétrocompatibilité. Une valeur plus élevée est signe d'un meilleur niveau de sécurité.

7.10 Identifiant étendu d'un nœud (ExpandedNodeId)

Les composantes de ce paramètre sont définies au Tableau 125. L'*ExpandedNodeId* permet de spécifier explicitement l'espace de noms sous la forme d'une chaîne ou avec un indice dans le tableau d'espaces de noms du *Serveur*.

Tableau 125 – ExpandedNodeId

Nom	Type	Description
ExpandedNodeId	structure	Le <i>NodeId</i> avec l'espace de noms étendu à sa représentation sous forme de chaîne.
serverIndex	Index	Indice qui identifie le <i>Serveur</i> qui contient le <i>TargetNode</i> . Ce <i>Serveur</i> peut être le <i>Serveur</i> local ou un <i>Serveur</i> distant. Cet indice est l'indice du <i>Serveur</i> en question dans le tableau de <i>Serveurs</i> du <i>Serveur</i> local. L'indice du <i>Serveur</i> local dans le tableau de <i>Serveurs</i> est toujours 0. Tous les <i>Serveurs</i> distants ont des indices supérieurs à 0. Le tableau de <i>Serveurs</i> est contenu dans l' <i>Object Server</i> (objet serveur) dans l'Espace d'Adresses (voir l'IEC 62541-3 et l'IEC 62541-5). Le <i>Client</i> peut lire la <i>Serveur</i> table <i>Variable</i> (variable tableau de serveurs) pour accéder à la description du <i>Serveur</i> cible.
namespaceUri	String	L'URI de l'espace de noms. Si ce paramètre est spécifié, l'indice d'espace de noms est ignoré. Le paragraphe 5.4 et l'IEC 62541-12 ³ décrivent le mécanisme de découverte qui peut être utilisé pour résoudre les URI en des URL.
namespaceIndex	Index	L'indice dans le tableau d'espace de noms du <i>Serveur</i> Ce paramètre doit être 0 et est ignoré dans le <i>Serveur</i> si l'URI d'espace de noms est spécifié.
identifiantType	IdType	Type de l'élément identificateur du <i>NodeId</i> .
identifiant	*	L'identificateur pour un <i>Nœud</i> dans l'Espace d'Adresses d'un <i>Serveur</i> OPC UA (voir la définition de <i>NodeId</i> dans l'IEC 62541-3).

7.11 Paramètre extensible (ExtensibleParameter)

Les types de paramètres extensibles peuvent seulement être étendus par des parties supplémentaires de la série de normes.

L'*ExtensibleParameter* définit une structure de données à deux éléments. Le *parameterTypeId* spécifie le codage de type de données du second élément. Par conséquent, le second élément est spécifié comme étant "--". Le type de base *ExtensibleParameter* est défini au Tableau 126.

Les paramètres extensibles concrets qui sont communs à OPC UA sont définis à l'Article 7. Des parties supplémentaires de la série de normes peuvent définir de types de paramètres extensibles supplémentaires.

Tableau 126 – Type de base ExtensibleParameter

Nom	Type	Description
ExtensibleParameter	structure	Spécifie les détails d'un type de paramètre extensible.
parameterTypeId	NodeId	Identifie le type de données du paramètre qui suit.
parameterData	--	Les détails pour le type de paramètre extensible.

7.12 Indice (Index)

Ce type de données de primitive est un UInt32 qui identifie un élément d'une matrice.

7.13 IntegerId

Ce type de données de primitive est un UInt32 qui est utilisé comme identificateur, tel qu'un descripteur. Toutes les valeurs, autres que 0, sont valides.

7.14 Mode de sécurité des messages (MessageSecurityMode)

Le *MessageSecurityMode* est une énumération qui spécifie la sécurité qu'il convient d'appliquer aux échanges de messages au cours d'une Session. Les valeurs possibles sont décrites au Tableau 127.

Tableau 127 – Valeurs de MessageSecurityMode

Valeur	Description
INVALID_0	Le MessageSecurityMode n'est pas valide. Cette valeur est la valeur par défaut afin d'éviter qu'un choix accidentel de "no security" (aucune sécurité) ne soit appliqué. Ce choix sera toujours rejeté.
NONE_1	Aucune sécurité n'est appliquée.
SIGN_2	Tous les messages sont signés, mais ne sont pas chiffrés.
SIGNANDENCRYPT_3	Tous les messages sont signés et chiffrés.

7.15 Surveillance des paramètres (MonitoringParameters)

Les composantes de ce paramètre sont définies au Tableau 128.

Tableau 128 – MonitoringParameters

Nom	Type	Description												
MonitoringParameters	structure	Paramètres qui définissent les caractéristiques de surveillance relatives à un <i>MonitoredItem</i> .												
clientHandle	IntegerId	Identificateur fourni par le <i>Client</i> pour le <i>MonitoredItem</i> . Cet identificateur est utilisé dans les <i>Notifications</i> générées pour le <i>Nœud</i> de liste. Le type <i>IntegerId</i> est défini en 7.13.												
samplingInterval	Duration	L'intervalle qui définit la fréquence la plus rapide à laquelle il convient d'accéder et évaluer le(s) <i>MonitoredItem</i>(s). Cet intervalle est défini en millisecondes. La valeur 0 indique qu'il convient que le <i>Serveur</i> utilise la fréquence pratique la plus rapide. La valeur -1 indique que l'intervalle d'échantillonnage par défaut défini par l'intervalle d'édition de <i>Subscription</i> est demandé. Un autre intervalle d'échantillonnage est utilisé si l'intervalle d'édition n'est pas pris en charge. Tout nombre négatif est interprété comme -1. L'intervalle d'échantillonnage n'est pas modifié si l'intervalle d'édition est modifié par un appel consécutif au <i>Service ModifySubscription</i>. Le <i>Serveur</i> utilise ce paramètre pour affecter les <i>MonitoredItems</i> à un intervalle d'échantillonnage qu'il prend en charge. L'intervalle assigné est fourni dans le paramètre <i>revisedSamplingInterval</i>. Le <i>Serveur</i> doit toujours retourner un <i>revisedSamplingInterval</i> supérieur ou égal au <i>samplingInterval</i> demandé. Si le <i>samplingInterval</i> demandé est supérieur à l'intervalle d'échantillonnage maximal pris en charge par le <i>Serveur</i>, l'intervalle d'échantillonnage maximal est retourné.												
filter	Extensible Parameter MonitoringFilter	Un filtre utilisé par le <i>Serveur</i> pour déterminer s'il convient que le <i>MonitoredItem</i> génère une <i>Notification</i> . S'il n'est pas utilisé, ce paramètre est null. Le type de paramètre <i>MonitoringFilter</i> est un type de paramètre extensible spécifié en 7.16. Il spécifie les types de filtres qui peuvent être utilisés.												
queueSize	Counter	La taille demandée de la file d'attente des <i>MonitoredItems</i>. Les valeurs suivantes ont une signification spéciale pour les éléments surveillés de données: <table><tr><th><u>Valeur</u></th><th><u>Signification</u></th></tr><tr><td>0 ou 1</td><td>le <i>Serveur</i> retourne la taille par défaut de la file d'attente qui doit être de 1, comme <i>revisedQueueSize</i> pour les éléments surveillés de données. La file d'attente a une seule entrée, désactivant effectivement la mise en file d'attente.</td></tr></table> Pour les valeurs supérieures à un, une file d'attente en "premier entré premier sorti" est à utiliser. Le <i>Serveur</i> peut limiter la taille dans <i>revisedQueueSize</i>. Dans le cas d'un dépassement de capacité de la file d'attente, le bit <i>Overflow</i> (fanion) dans la partie <i>InfoBits</i> du <i>statusCode DataValue</i> est mis à la nouvelle valeur. Les valeurs suivantes ont une signification spéciale pour les éléments surveillés de données: <table><tr><th><u>Valeur</u></th><th><u>Signification</u></th></tr><tr><td>0</td><td>le <i>Serveur</i> retourne la taille par défaut de la file d'attente pour les <i>Event Notifications</i> comme <i>revisedQueueSize</i> pour les éléments surveillés de données.</td></tr><tr><td>1</td><td>le <i>Serveur</i> retourne la taille minimale de la file d'attente requise par le <i>Serveur</i> pour les <i>Event Notifications</i> comme <i>revisedQueueSize</i>.</td></tr><tr><td>MaxUInt32</td><td>le <i>Serveur</i> retourne la taille maximale de la file d'attente prise en charge par le <i>Serveur</i> pour les <i>Event Notifications</i> comme <i>revisedQueueSize</i>.</td></tr></table> Si un <i>Client</i> choisit une valeur entre la valeur minimale et la valeur maximale du <i>Serveur</i>, la valeur doit être retournée dans <i>revisedQueueSize</i>. Si la <i>queueSize</i> demandée dépasse la valeur minimale ou maximale, le <i>Serveur</i> doit retourner la valeur liée correspondante. Dans le cas d'un dépassement de capacité de la file d'attente, un <i>Event</i> du type <i>EventQueueOverflowEventType</i> est généré.	<u>Valeur</u>	<u>Signification</u>	0 ou 1	le <i>Serveur</i> retourne la taille par défaut de la file d'attente qui doit être de 1, comme <i>revisedQueueSize</i> pour les éléments surveillés de données. La file d'attente a une seule entrée, désactivant effectivement la mise en file d'attente.	<u>Valeur</u>	<u>Signification</u>	0	le <i>Serveur</i> retourne la taille par défaut de la file d'attente pour les <i>Event Notifications</i> comme <i>revisedQueueSize</i> pour les éléments surveillés de données.	1	le <i>Serveur</i> retourne la taille minimale de la file d'attente requise par le <i>Serveur</i> pour les <i>Event Notifications</i> comme <i>revisedQueueSize</i> .	MaxUInt32	le <i>Serveur</i> retourne la taille maximale de la file d'attente prise en charge par le <i>Serveur</i> pour les <i>Event Notifications</i> comme <i>revisedQueueSize</i> .
<u>Valeur</u>	<u>Signification</u>													
0 ou 1	le <i>Serveur</i> retourne la taille par défaut de la file d'attente qui doit être de 1, comme <i>revisedQueueSize</i> pour les éléments surveillés de données. La file d'attente a une seule entrée, désactivant effectivement la mise en file d'attente.													
<u>Valeur</u>	<u>Signification</u>													
0	le <i>Serveur</i> retourne la taille par défaut de la file d'attente pour les <i>Event Notifications</i> comme <i>revisedQueueSize</i> pour les éléments surveillés de données.													
1	le <i>Serveur</i> retourne la taille minimale de la file d'attente requise par le <i>Serveur</i> pour les <i>Event Notifications</i> comme <i>revisedQueueSize</i> .													
MaxUInt32	le <i>Serveur</i> retourne la taille maximale de la file d'attente prise en charge par le <i>Serveur</i> pour les <i>Event Notifications</i> comme <i>revisedQueueSize</i> .													
discardOldest	Boolean	Un paramètre booléen qui spécifie la politique de rejet lorsque la file d'attente est pleine et une nouvelle <i>Notification</i> est à placer en file d'attente. Ce paramètre a les valeurs suivantes: <table><tr><td>TRUE</td><td>La plus ancienne (première) <i>Notification</i> dans la file d'attente est rejetée. La nouvelle <i>Notification</i> est ajoutée à la fin de la file d'attente.</td></tr><tr><td>FALSE</td><td>la dernière <i>Notification</i> ajoutée à la file d'attente est remplacée par la nouvelle <i>Notification</i>.</td></tr></table>	TRUE	La plus ancienne (première) <i>Notification</i> dans la file d'attente est rejetée. La nouvelle <i>Notification</i> est ajoutée à la fin de la file d'attente.	FALSE	la dernière <i>Notification</i> ajoutée à la file d'attente est remplacée par la nouvelle <i>Notification</i> .								
TRUE	La plus ancienne (première) <i>Notification</i> dans la file d'attente est rejetée. La nouvelle <i>Notification</i> est ajoutée à la fin de la file d'attente.													
FALSE	la dernière <i>Notification</i> ajoutée à la file d'attente est remplacée par la nouvelle <i>Notification</i> .													

7.16 Paramètres des filtres de surveillance (MonitoringFilter)

7.16.1 Vue d'ensemble

Le *Service CreateMonitoredItem* permet de spécifier un filtre pour chaque *MonitoredItem*. Le *MonitoringFilter* est un paramètre extensible dont la structure dépend du type d'élément surveillé. Les *parameterTypeIds* sont définis au Tableau 129. D'autres types peuvent être définis par des parties supplémentaires de la présente spécification à plusieurs parties ou par d'autres spécifications basées sur OPC UA. Le type *ExtensibleParameter* est défini en 7.11.

Chaque *MonitoringFilter* peut avoir une structure associée *MonitoringFilterResult* qui retourne des paramètres révisés et/ou des informations d'erreur aux clients dans la réponse. Les structures de résultats, lorsqu'elles existent, sont décrites dans la section qui définit le *MonitoringFilter*.

Tableau 129 – Typelds du paramètre MonitoringFilter

Id symbolique	Description
DataChangeFilter	Le changement apporté à une valeur de donnée qui doit entraîner la création d'une <i>Notification</i> .
EventFilter	Si une <i>Notification</i> est conforme à l' <i>EventFilter</i> , la <i>Notification</i> est envoyée au <i>Client</i> .
AggregateFilter	L' <i>Aggregate</i> et ses intervalles lorsqu'il sera calculé et une <i>Notification</i> est générée.

7.16.2 DataChangeFilter

Le *DataChangeFilter* définit les conditions dans lesquelles il convient de rapporter une *Notification* de *DataChange* et, facultativement, une plage ou une bande de valeurs lorsqu'il n'est pas généré de *Notification* de *DataChange*. Cette plage est appelée *Deadband* (bande morte). Le *DataChangeFilter* est défini au Tableau 130.

Tableau 130 – DataChangeFilter

Nom	Type	Description
DataChangeFilter	structure	
trigger	Enum DataChangeTrigger	Spécifie les conditions dans lesquelles il convient de rapporter une notification de changement de donnée. Cette énumération a les valeurs suivantes: STATUS_0 Rapporter une notification SEULEMENT si le <i>StatusCode</i> associé aux changements de la valeur. Voir le Tableau 166 pour les <i>StatusCodes</i> définis dans la présente Norme. L'IEC 62541-8 spécifie des <i>StatusCodes</i> supplémentaires qui sont en particulier valides pour des données d'appareil. STATUS_VALUE_1 Rapporter une notification si le <i>StatusCode</i> ou la valeur change. Le filtre <i>Deadband</i> peut être ajouté en plus pour filtrer les changements de valeur. Il s'agit du réglage par défaut si aucun filtre n'est installé. STATUS_VALUE_TIMESTAMP_2 Rapporter une notification si le <i>StatusCode</i>, la valeur ou le <i>SourceTimestamp</i> change. Si un filtre <i>Deadband</i> est spécifié, ce <i>trigger</i> a le même comportement que STATUS_VALUE_1. Le DataChangeFilter n'est pas appliqué à l'élément surveillé, STATUS_VALUE_1 est le comportement par défaut pour la production de rapports.
deadbandType	UInt32	Une valeur qui définit le type et le comportement <i>Deadband</i>. Valeur <i>deadbandType</i> None_0 Il convient de n'appliquer aucun calcul <i>Deadband</i>. Absolute_1 AbsoluteDeadband (Voir ci-dessous) Percent_2 PercentDeadband (Ce type est spécifié dans l'IEC 62541-8).
deadbandValue	Double	La <i>Deadband</i> est appliquée uniquement si * le <i>trigger</i> inclut des changements de valeur et * le <i>deadbandType</i> est établi de manière appropriée. La bande morte est ignorée si le statut de l'élément de données change. DeadbandType = AbsoluteDeadband: Pour ce type, la <i>deadbandValue</i> contient le changement absolu apporté à une valeur de données qui doit provoquer la création d'une <i>Notification</i>. Ce paramètre s'applique uniquement aux <i>Variables</i> de n'importe quel type de données numérique. Une exception qui provoque une <i>Notification</i> de <i>DataChange</i> basée sur une AbsoluteDeadband est déterminée comme suit: Génère une Notification si (valeur absolue de (dernière valeur en cache – valeur courante) > AbsoluteDeadband) La dernière valeur en cache est définie comme étant la dernière valeur poussée dans la file d'attente. Si l'élément est une matrice de valeurs, toute la matrice est retournée si un élément de matrice dépasse l'AbsoluteDeadband, ou la taille ou la dimension des changements de la matrice. DeadbandType = PercentDeadband: Ce type est spécifié dans l'IEC 62541-8

Le *DataChangeFilter* n'a pas de structure de résultats associée.

7.16.3 EventFilter

L'*EventFilter* permet le filtrage et la sélection de contenu des *Event Subscriptions*.

Si une *Notification* d'Événement est conforme au filtre défini par le paramètre *where* de l'*EventFilter*, la *Notification* est envoyée au *Client*.

Chaque *Notification* d'Événement doit inclure les champs définis par le paramètre *selectClauses* de l'*EventFilter*. Les *EventTypes* définis sont spécifiés dans l'IEC 62541-5.

Les paramètres *selectClause* et *whereClause* sont spécifiés avec la structure *SimpleAttributeOperand* (voir 7.4.4.5). Cette structure exige le *NodeId* d'un *EventType* pris en charge par le *Serveur* et un chemin vers une *InstanceDeclaration*. Une *InstanceDeclaration* est un *Nœud* qui peut être trouvé en suivant des références hiérarchiques directes à partir de

L'*EventType* totalement hérité où le *Nœud* est aussi la source d'une référence *HasModellingRule*. Les *EventTypes*, les *InstanceDeclarations* et les *Modelling Rules* sont décrits de manière complète dans l'IEC 62541-3.

Dans certains cas, le même *BrowsePath* s'applique à plusieurs *EventTypes*. Si le *Client* spécifie le *BaseEventType* dans le *SimpleAttributeOperand*, le *Serveur* doit évaluer le *BrowsePath* sans tenir compte du *Type*.

Chaque *InstanceDeclaration* dans le chemin doit être un *Object* ou *Variable Node*. Le *Nœud* final dans le chemin peut être un *Object Node*; cependant, les *Object Nodes* sont disponibles uniquement pour les *Événements* qui sont visibles dans l'*Espace d'Adresses du Serveur*.

La structure *SimpleAttributeOperand* permet au *Client* de spécifier n'importe quel *Attribut*; cependant, le *Serveur* est seulement tenu de prendre en charge le *Value Attribute* pour des *Variable Nodes* et le *NodeId Attribute* pour des *Object Nodes*. Cela dit, les profils définis dans l'IEC 62541-7 peuvent rendre obligatoire la prise en charge d'*Attributs* supplémentaires.

La structure *SimpleAttributeOperand* est utilisée dans la *selectClause* pour sélectionner la valeur à retourner si un *Événement* satisfait aux critères spécifiés par la *whereClause*. Une valeur null est retournée dans le champ événement correspondant dans la réponse Publish si le *field* sélectionné ne fait pas partie de l'*Événement* ou une erreur a été retournée dans les *selectClauseResults* de l'*EventFilterResult*. Si le *field* sélectionné est pris en charge, mais non disponible au moment de la notification d'événement, le champ d'événement doit contenir un *StatusCode* qui indique la raison de l'indisponibilité. Par exemple, le *Serveur* doit mettre le champ d'événement sur *Bad_UserAccessDenied* si la valeur n'est pas accessible à l'utilisateur associé à la *Session*. Si un *Attribut Value* a un *StatusCode* incertain ou mauvais qui lui est associé, le *Serveur* doit fournir le *StatusCode* au lieu de l'*Attribut Value*. Le *Serveur* doit mettre le champ d'événement sur *Bad_EncodingLimitsExceeded* si une valeur dépasse *maxResponseMessageSize*. L'*EventId*, l'*EventType* et le *ReceiveTime* ne peuvent pas contenir un *StatusCode* ou une valeur null.

Le *Serveur* doit valider les *selectClauses* lorsqu'un *Client* crée ou met à jour l'*EventFilter*. Toutes les éventuelles erreurs qui sont vraies pour tous les *Événements* possibles sont retournées dans le paramètre *selectClauseResults* décrit au Tableau 132. Certains *Serveurs*, comme les *Serveurs* combinés, peuvent ne pas connaître tous les *EventTypes* possibles au moment où l'*EventFilter* est établi. Ces *Serveurs* ne retournent pas d'erreurs pour les *EventTypes* ou *BrowsePaths* inconnus. Le *Serveur* ne doit pas rapporter les erreurs susceptibles de se produire en fonction de l'état du *Serveur* ou du type d'*Événement*. Par exemple, un *selectClause* qui demande un seul élément d'une matrice produirait toujours une erreur si le *DataType* de l'*Attribut* est un scalaire. Cependant, même si le *DataType* est une matrice, une erreur peut se produire si l'indice demandé n'existe pas pour un *Événement* particulier. Dans cette dernière situation, le *Serveur* ne rapporterait pas une erreur dans le paramètre *selectClauseResults*.

Le *SimpleAttributeOperand* est utilisé dans la *whereClause* pour sélectionner une valeur qui est partie intégrante d'une expression logique. Ces expressions logiques sont alors utilisées pour déterminer s'il convient de rapporter un *Événement* particulier au *Client*. Le *Serveur* doit utiliser une valeur null si une erreur se produit lorsqu'une *whereClause* est évaluée pour un *Événement* particulier. Si un *Attribut Value* a un *StatusCode* incertain ou mauvais qui lui est associé, le *Serveur* doit utiliser une valeur null en lieu et place de la *Value*.

Tout *FilterOperator* de base contenu dans le Tableau 110 peut être utilisé dans la *whereClause*; cependant, seul l'*OfType FilterOperator* du Tableau 111 est autorisé.

Le *Serveur* doit valider les *whereClauses* lorsqu'un *Client* crée ou met à jour l'*EventFilter*. Toutes les éventuelles erreurs de structure dans la construction du filtre et toutes les éventuelles erreurs qui sont vraies pour tous les *Événements* possibles sont retournées dans le paramètre *whereClauseResult* décrit au Tableau 132. Les erreurs qui peuvent se produire en fonction de l'état du *Serveur* ou de l'*Événement* ne font pas l'objet d'un rapport. Certains

Serveurs, comme les *Serveurs* combinés, peuvent ne pas connaître tous les *EventTypes* possibles au moment où l'*EventFilter* est établi. Ces *Serveurs* ne retournent pas d'erreurs pour les *EventTypes* ou *BrowsePaths* inconnus.

Les *SubscriptionControlEvents* sont des *Événements* spéciaux qui sont utilisés pour fournir des informations de commande au *Client*. Ces *Événements* ne sont édités qu'aux *MonitoredItems* dans le *Subscription* qui a produit le *SubscriptionControlEvent*. Ces *Événements* contournent la *whereClause*.

Le Tableau 131 définit la structure *EventFilter*.

Tableau 131 – Structure EventFilter

Nom	Type	Description
EventFilter	structure	
selectClauses []	SimpleAttribute Operand	Liste des valeurs à retourner avec chaque <i>Événement</i> dans une <i>Notification</i> . Au moins une clause valide doit être spécifiée. Voir 7.4.4.5 pour la définition de <i>SimpleAttributeOperand</i> .
whereClause	ContentFilter	Limiter les <i>Notifications</i> aux <i>Événements</i> qui correspondent aux critères définis par ce ContentFilter. La structure ContentFilter est décrite au 7.4. La structure <i>AttributeOperand</i> ne peut pas être utilisée dans les <i>EventFilter</i> .

Le Tableau 132 définit la structure *EventFilterResult*. Il s'agit du *MonitoringFilterResult* associé à l'*EventFilter MonitoringFilter*.

Tableau 132 – Structure EventFilterResult

Nom	Type	Description
EventFilterResult	structure	
selectClauseResults []	StatusCode	Liste de codes de statut pour les éléments dans la clause select. La taille et l'ordre de la liste correspondent à la taille et à l'ordre des éléments dans le paramètre de demande <i>selectClauses</i> . Le <i>Serveur</i> retourne null pour les champs <i>Événement</i> indisponibles ou rejetés.
selectClauseDiagnosticInfos []	DiagnosticInfo	Une liste d'informations de diagnostic pour les éléments individuels dans la clause select. La taille et l'ordre de la liste correspondent à la taille et à l'ordre des éléments dans le paramètre de demande <i>selectClauses</i> . Cette liste est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande ou si aucune information de diagnostic n'a été rencontrée dans le traitement des clauses select.
whereClauseResult	ContentFilter Result	Un résultat associé avec le paramètre de demande <i>whereClause</i> . Le type <i>ContentFilterResult</i> est défini en 7.4.2.

Le Tableau 133 définit des valeurs pour le paramètre *selectClauseResults*. Les *StatusCodes* communs sont définis au Tableau 166.

Tableau 133 – Codes de résultats EventFilterResult

Id symbolique	Description
Bad_TypeDefinitionInvalid	Voir le Tableau 166 pour la description de ce code de résultat. Le typeId n'est pas le NodeId pour BaseEventType ou un sous-type de celui-ci.
Bad_NodeIdUnknown	Voir le Tableau 166 pour la description de ce code de résultat. Le browsePath est spécifié, mais il n'existera jamais dans un Event.
Bad_BrowseNameInvalid	Voir le Tableau 166 pour la description de ce code de résultat. Le browsePath est spécifié et contient un élément null.
Bad_AttributeIdInvalid	Voir le Tableau 166 pour la description de ce code de résultat. Le nœud spécifié par le chemin de navigation ne permettra jamais que l'identificateur d'attribut donné soit retourné.
Bad_IndexRangeInvalid	Voir le Tableau 166 pour la description de ce code de résultat.
Bad_TypeMismatch	Voir le Tableau 166 pour la description de ce code de résultat. L'indexRange est valide, mais la valeur de l'Attribut n'est jamais une matrice.

7.16.4 AggregateFilter

L'*AggregateFilter* définit la fonction d'*agrégat* qu'il convient d'utiliser pour calculer les valeurs à retourner. Voir l'IEC 62541-13 pour les détails sur les fonctions d'*agrégat* possibles. Il spécifie un *startTime* du premier *agrégat* à calculer. Le *samplingInterval* des

MonitoringAttributes (voir 7.15) définit la manière dont il convient que le serveur échantillonne en interne la source de données sous-jacente. Le processingInterval spécifie la durée d'une période pendant laquelle l'*agrégat* est calculé. La queueSize des MonitoringAttributes spécifie le nombre de valeurs traitées qu'il convient de conserver.

L'objectif de l'*AggregateFilter* n'est pas de lire des données historiques; pour ce faire, il convient d'utiliser le service HistoryRead. Cependant, il est permis de mettre le startTime à une heure qui se situe dans le passé lorsqu'elles sont reçues par le serveur. Il convient que le nombre d'*agrégats* à calculer dans le passé ne dépasse pas la queueSize définie dans les MonitoringAttributes, car les valeurs supérieures à la queueSize seraient directement rejetées et ne seraient jamais retournées au client.

Le startTime et le processingInterval peuvent être révisés par le serveur, mais il convient que le startTime reste dans les mêmes limites ($\text{startTime} + \text{revisedProcessingInterval} * n = \text{revisedStartTime}$). Ce comportement simplifie l'accès aux valeurs historiques des *agrégats* qui utilisent les mêmes limites en appelant le service HistoryRead. Le paramètre extensible AggregateFilterResult est utilisé pour retourner les valeurs révisées pour l'*AggregateFilter*.

Certains systèmes sous-jacents peuvent sonder les données et produire plusieurs échantillons avec la même valeur. D'autres systèmes peuvent seulement rapporter les changements apportés aux valeurs. La définition pour chaque type d'*agrégat* explique la manière de gérer ces deux scénarios différents.

Le *MonitoredItem* rapporte uniquement les valeurs pour des intervalles qui sont achevés lorsque la minuterie d'édition expire. Les données non utilisées sont reportées et utilisées pour calculer une valeur retournée dans la prochaine édition.

Le *ServerTimestamp* pour chaque intervalle doit être l'heure de la fin de l'intervalle de traitement.

L'*AggregateFilter* est défini au Tableau 134.

Tableau 134 – Structure AggregateFilter

Nom	Type	Description
AggregateFilter	structure	
startTime	UTCTime	Début de la période pour calculer l' <i>agrégat</i> la première fois. La taille de chaque période utilisée pour calculer l' <i>agrégat</i> est définie par le samplingInterval des MonitoringAttributes (voir 7.15).
aggregateType	NodeId	Le NodeId de l'objet <i>AggregateFunctionType</i> qui indique l' <i>agrégat</i> à utiliser pour récupérer les données traitées. Voir l'IEC 62541-13 pour les détails.
processingInterval	Duration	La période à utiliser pour calculer l' <i>agrégat</i> .
aggregateConfiguration	Aggregate Configuration	Ce paramètre permet aux Clients de passer outre les paramètres de configuration des <i>Aggregate</i> fournis par l'objet <i>AggregateConfiguration</i> , élément surveillé par élément surveillé. Voir l'IEC 62541-13 pour plus d'informations sur les configurations des <i>Aggregate</i> . Si le <i>Serveur</i> ne prend pas en charge la capacité de passer outre aux paramètres de configuration des <i>Aggregate</i> , il doit retourner un <i>StatusCode</i> de Bad_AggregateListMismatch. Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.
useSeverCapabilities Defaults	Boolean	Si value = TRUE, utiliser les paramètres de configuration des agrégats tels qu'indiqués par l'objet AggregateConfiguration. Si value = FALSE utiliser les paramètres de configuration tels qu'indiqués dans les paramètres AggregateConfiguration suivants. La valeur par défaut est TRUE.
treatUncertainAsBad	Boolean	Comme décrit dans l'IEC 62541-13.
percentDataBad	Byte	Comme décrit dans l'IEC 62541-13.
percentDataGood	Byte	Comme décrit dans l'IEC 62541-13.
useSloped Extrapolation	Boolean	Comme décrit dans l'IEC 62541-13.

L'AggregateFilterResult définit l'AggregateFilter révisé que le serveur peut retourner lorsqu'un AggregateFilter est défini pour un MonitoredItem dans les services CreateMonitoredItems ou ModifyMonitoredItems. L'AggregateFilterResult est défini au Tableau 135. Il s'agit de MonitoringFilterResult associé à l'AggregateFilter MonitoringFilter.

Tableau 135 – Structure AggregateFilterResult

Nom	Type	Description
AggregateFilterResult	structure	
revisedStartTime	UTCTime	L'intervalle StartTime réel que le <i>Serveur</i> doit utiliser. Cette valeur est basée sur plusieurs facteurs, notamment les capacités du serveur à accéder aux données historiques. Il convient que le revisedStartTime reste dans les mêmes limites que le startTime (startTime + samplingInterval * n = revisedStartTime).
revisedProcessingInterval	Duration	Le processingInterval réel que le <i>Serveur</i> doit utiliser. Le revisedProcessingInterval doit être au moins deux fois le revisedSamplingInterval pour le MonitoredItem.

7.17 Mode de surveillance (MonitoringMode)

Le *MonitoringMode* est une énumération qui spécifie si l'échantillonnage et la production de rapports sont activés ou désactivés pour un *MonitoredItem*. La valeur du paramètre d'édition activé pour un *Subscription* n'a pas d'incidence sur la valeur du mode de surveillance pour un *MonitoredItem* de *Subscription*. Les valeurs de ce paramètre sont définies au Tableau 136.

Tableau 136 – Valeurs de MonitoringMode

Valeur	Description
DISABLED_0	L'élément surveillé n'est ni échantillonné ni évalué, et les <i>Notifications</i> ne sont ni générées ni placées en file d'attente. La production de rapports relatifs aux <i>Notifications</i> est désactivée.
SAMPLING_1	L'élément surveillé est échantillonné et évalué, et les <i>Notifications</i> sont générées et placées en file d'attente. La production de rapports relatifs aux <i>Notifications</i> est désactivée.
REPORTING_2	L'élément surveillé est échantillonné et évalué, et les <i>Notifications</i> sont générées et placées en file d'attente. La production de rapports relatifs aux <i>Notifications</i> est activée.

7.18 Paramètres des attributs d'un nœud (NodeAttributes)

7.18.1 Vue d'ensemble

Le *Service AddNodes* permet de spécifier les *Attributs* pour les *Nœuds* à ajouter. Le *NodeAttributes* est un paramètre extensible dont la structure dépend du type de l'*Attribut* ajouté. Il identifie la *NodeClass* qui définit la structure des *Attributs* qui suivent. Les *parameterTypeIds* sont définis au Tableau 137. Le type *ExtensibleParameter* est défini en 7.11.

Tableau 137 –TypeIds du paramètre NodeAttributes

Id symbolique	Description
ObjectAttributes	Définit les <i>Attributs</i> pour l' <i>Object NodeClass</i> .
VariableAttributes	Définit les <i>Attributs</i> pour la <i>Variable NodeClass</i> .
MethodAttributes	Définit les <i>Attributs</i> pour la <i>Method NodeClass</i> .
ObjectTypeAttributes	Définit les <i>Attributs</i> pour l' <i>ObjectType NodeClass</i> .
VariableTypeAttributes	Définit les <i>Attributs</i> pour la <i>VariableType NodeClass</i> .
ReferenceTypeAttributes	Définit les <i>Attributs</i> pour la <i>ReferenceType NodeClass</i> .
DataTypeAttributes	Définit les <i>Attributs</i> pour la <i>DataType NodeClass</i> .
ViewAttributes	Définit les <i>Attributs</i> pour la <i>View NodeClass</i> .

Le Tableau 138 définit le masque de bits utilisé dans les paramètres *NodeAttributes* pour spécifier les *Attributs* qui sont établis par le *Client*.

Tableau 138 – Masque de bits pour les Attributs spécifiés

Champ	Bit	Description
AccessLevel	0	Indique si l'attribut AccessLevel est positionné.
ArrayDimensions	1	Indique si l'attribut ArrayDimensions est positionné.
Reserved	2	Réservé pour être cohérent avec le WriteMask défini dans l'IEC 62541-3.
ContainsNoLoops	3	Indique si l'attribut ContainsNoLoops est positionné.
DataType	4	Indique si l'attribut DataType est positionné.
Description	5	Indique si l'attribut Description est positionné.
DisplayName	6	Indique si l'attribut DisplayName est positionné.
EventNotifier	7	Indique si l'attribut EventNotifier est positionné.
Executable	8	Indique si l'attribut Executable est positionné.
Historizing	9	Indique si l'attribut Historizing est positionné.
InverseName	10	Indique si l'attribut InverseName est positionné.
IsAbstract	11	Indique si l'attribut IsAbstract est positionné.
MinimumSamplingInterval	12	Indique si l'attribut MinimumSamplingInterval est positionné.
Reserved	13	Réservé pour être cohérent avec le WriteMask défini dans l'IEC 62541-3.
Reserved	14	Réservé pour être cohérent avec le WriteMask défini dans l'IEC 62541-3.
Symmetric	15	Indique si l'attribut Symmetric est positionné.
UserAccessLevel	16	Indique si l'attribut UserAccessLevel est positionné.
UserExecutable	17	Indique si l'attribut UserExecutable est positionné.
UserWriteMask	18	Indique si l'attribut UserWriteMask est positionné.
ValueRank	19	Indique si l'attribut ValueRank est positionné.
WriteMask	20	Indique si l'attribut WriteMask est positionné.
Value	21	Indique si l'attribut Value est positionné.
Reserved	22:32	Réservé pour utilisation ultérieure. Doit toujours être zéro.

7.18.2 Paramètre d'ObjectAttributes

Le Tableau 139 définit le paramètre *ObjectAttributes*.

Tableau 139 – ObjectAttributes

Nom	Type	Description
ObjectAttributes	structure	Définit les <i>Attributs</i> pour l' <i>Object NodeClass</i> .
specifiedAttributes	UInt32	Un masque de bits qui indique les champs qui contiennent des valeurs valides. Un champ doit être ignoré si le bit correspondant est positionné à 0. Les valeurs des bits sont définies au Tableau 138.
displayName	LocalizedText	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
Description	LocalizedText	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
eventNotifier	Byte	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
writeMask	UInt32	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
userWriteMask	UInt32	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .

7.18.3 Paramètre de VariableAttributes

Le Tableau 140 définit le paramètre *VariableAttributes*.

Tableau 140 – VariableAttributes

Nom	Type	Description
VariableAttributes	structure	Définit les <i>Attributs</i> pour la <i>Variable NodeClass</i> .
specifiedAttributes	UInt32	Un masque de bits qui indique les champs qui contiennent des valeurs valides. Un champ doit être ignoré si le bit correspondant est mis à 0. Les valeurs des bits sont définies au Tableau 138.
displayName	LocalizedText	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
Description	LocalizedText	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
value	Défini par le <i>Data Type Attribute</i>	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
dataType	NodeId	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
valueRank	Int32	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
arrayDimensions	UInt32 []	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
accessLevel	Byte	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
userAccessLevel	Byte	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
minimumSamplingInterval	Duration	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
historizing	Boolean	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
writeMask	UInt32	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
userWriteMask	UInt32	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .

7.18.4 Paramètre de MethodAttributes

Le Tableau 141 définit le paramètre *MethodAttributes*.

Tableau 141 – MethodAttributes

Nom	Type	Description
BaseAttributes	structure	Définit les <i>Attributs</i> pour la <i>Method NodeClass</i> .
specifiedAttributes	UInt32	Un masque de bits qui indique les champs qui contiennent des valeurs valides. Un champ doit être ignoré si le bit correspondant est mis à 0. Les valeurs des bits sont définies au Tableau 138.
displayName	LocalizedText	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
Description	LocalizedText	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
executable	Boolean	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
userExecutable	Boolean	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
writeMask	UInt32	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
userWriteMask	UInt32	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .

7.18.5 Paramètre d'ObjectTypeAttributes

Le Tableau 142 définit le paramètre *ObjectTypeAttributes*.

Tableau 142 – ObjectTypeAttributes

Nom	Type	Description
ObjectTypeAttributes	structure	Définit les <i>Attributs</i> pour l' <i>ObjectType NodeClass</i> .
specifiedAttributes	UInt32	Un masque de bits qui indique les champs qui contiennent des valeurs valides. Un champ doit être ignoré si le bit correspondant est mis à 0. Les valeurs des bits sont définies au Tableau 138.
displayName	LocalizedText	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
description	LocalizedText	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
isAbstract	Boolean	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
writeMask	UInt32	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
userWriteMask	UInt32	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .

7.18.6 Paramètre de VariableTypeAttributes

Le Tableau 143 définit le paramètre *VariableTypeAttributes*.

Tableau 143 – VariableTypeAttributes

Nom	Type	Description
VariableTypeAttributes	structure	Définit les <i>Attributs</i> pour la <i>VariableType NodeClass</i> .
specifiedAttributes	UInt32	Un masque de bits qui indique les champs qui contiennent des valeurs valides. Un champ doit être ignoré si le bit correspondant est mis à 0. Les valeurs des bits sont définies au Tableau 138.
displayName	LocalizedText	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
Description	LocalizedText	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
value	Défini par le <i>Data Type Attribute</i>	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
dataType	NodeId	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
valueRank	Int32	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
arrayDimensions	UInt32 []	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
isAbstract	Boolean	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
writeMask	UInt32	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
userWriteMask	UInt32	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .

7.18.7 Paramètre de ReferenceTypeAttributes

Le Tableau 144 définit le paramètre *ReferenceTypeAttributes*.

Tableau 144 – ReferenceTypeAttributes

Nom	Type	Description
ReferenceTypeAttributes	structure	Définit les <i>Attributs</i> pour la <i>ReferenceType NodeClass</i> .
specifiedAttributes	UInt32	Un masque de bits qui indique les champs qui contiennent des valeurs valides. Un champ doit être ignoré si le bit correspondant est mis à 0. Les valeurs des bits sont définies au Tableau 138.
displayName	LocalizedText	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
Description	LocalizedText	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
isAbstract	Boolean	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
symmetric	Boolean	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
inverseName	LocalizedText	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
writeMask	UInt32	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
userWriteMask	UInt32	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .

7.18.8 Paramètre de DataTypeAttributes

Le Tableau 145 définit le paramètre *DataTypeAttributes*.

Tableau 145 – DataTypeAttributes

Nom	Type	Description
DataTypeAttributes	structure	Définit les <i>Attributs</i> pour la <i>DataType NodeClass</i> .
specifiedAttributes	UInt32	Un masque de bits qui indique les champs qui contiennent des valeurs valides. Un champ doit être ignoré si le bit correspondant est mis à 0. Les valeurs des bits sont définies au Tableau 138.
displayName	LocalizedText	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
Description	LocalizedText	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
isAbstract	Boolean	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
writeMask	UInt32	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
userWriteMask	UInt32	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .

7.18.9 Paramètre de ViewAttributes

Le Tableau 146 définit le paramètre *ViewAttributes*.

Tableau 146 – ViewAttributes

Nom	Type	Description
ViewAttributes	structure	Définit les <i>Attributs</i> pour la <i>View NodeClass</i> .
specifiedAttributes	UInt32	Un masque de bits qui indique les champs qui contiennent des valeurs valides. Un champ doit être ignoré si le bit correspondant est mis à 0. Les valeurs des bits sont définies au Tableau 138.
displayName	LocalizedText	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
Description	LocalizedText	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
containsNoLoops	Boolean	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
eventNotifier	Byte	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
writeMask	UInt32	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .
userWriteMask	UInt32	Voir l'IEC 62541-3 pour la description de cet <i>Attribut</i> .

7.19 Paramètres des données de notification (NotificationData)

7.19.1 Vue d'ensemble

La structure *NotificationMessage* utilisée dans le jeu de *Services Subscription* permet de spécifier différents types de *NotificationData*. Le *NotificationData* est un paramètre extensible dont la structure dépend du type de *Notification* envoyée. Ce paramètre est défini au Tableau 147. D'autres types peuvent être définis par des parties supplémentaires de la série de normes ou par d'autres spécifications basées sur OPC UA. Le type *ExtensibleParameter* est défini en 7.11.

Il peut y avoir plusieurs notifications pour un seul *MonitoredItem* dans une seule structure *NotificationData*. Lorsque cela se produit, le *Serveur* doit assurer que les notifications apparaissent dans l'ordre dans lequel elles étaient placées en file d'attente dans le *MonitoredItem*. Ces notifications n'ont pas besoin d'apparaître dans un bloc contigu.

Tableau 147 – Typelds du paramètre NotificationData

Id symbolique	Description
DataChange	Paramètre de données <i>Notification</i> utilisé pour les <i>Notifications</i> de changements de données.
Événement	Paramètre de données <i>Notification</i> utilisé pour les <i>Event Notifications</i> .
StatusChange	Paramètre de données <i>Notification</i> utilisé pour les <i>Notifications</i> de changements de statut d'abonnement.

7.19.2 Paramètre de DataChangeNotification

Le Tableau 148 définit le paramètre *NotificationData* utilisé pour des notifications de changements de données. Cette structure contient les éléments de données surveillés qui sont à rapporter. Les éléments de données surveillés sont rapportés dans deux conditions:

- si le *MonitoringMode* est mis à REPORTING et si un changement est apporté à une valeur ou à son statut (représenté par son *StatusCode*) est détecté;
- si le *MonitoringMode* est mis à SAMPLING, le *MonitoredItem* est relié à un élément déclencheur qui déclenche.

Voir 5.12 pour une description du jeu de *Services MonitoredItem* et en particulier le *MonitoringItemModel* et le *TriggeringModel*.

Après la création d'un *MonitoredItem*, la valeur ou le statut courant(e) de l'Attribut surveillé doit être placé en file d'attente sans appliquer de filtre. Si la valeur courante n'est pas disponible après le premier intervalle d'échantillonnage, la première *Notification* doit être placée en file d'attente après récupération de la valeur ou du statut initial(e) dans la source de données.

Tableau 148 – DataChangeNotification

Nom	Type	Description
DataChangeNotification	structure	Données de <i>Notification</i> de changements de données.
monitoredItems []	MonitoredItem Notification	La liste des <i>MonitoredItems</i> pour lesquels un changement a été détecté. Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.
clientHandle	IntegerId	Descripteur fourni par le <i>Client</i> pour le <i>MonitoredItem</i> . Le type <i>IntegerId</i> est défini en 7.13
Value	DataValue	Le <i>StatusCode</i> , la valeur et le(s) horodatage(s) de l' <i>Attribut</i> surveillé en fonction de la configuration d'échantillonnage et de mise en file d'attente. Si le <i>StatusCode</i> indique une erreur, la valeur est à ignorer. Si les changements détectés n'ont pas tous été retournés depuis que le tampon de file d'attente du <i>Serveur</i> pour les <i>MonitoredItem</i> a atteint sa limite et a eu à purger les données, et si la taille de la file d'attente est supérieure à un, le bit <i>Overflow</i> dans les <i>DataValue InfoBits</i> du <i>statusCode</i> est positionné. <i>DataValue</i> est un type commun défini en 7.7.
diagnosticInfos []	DiagnosticInfo	Liste d'informations de diagnostic. La taille et l'ordre de cette liste correspondent à la taille et à l'ordre du paramètre de demande <i>monitoredItems</i> . Il y a une entrée dans cette liste pour chaque <i>Nœud</i> contenu dans le paramètre <i>monitoredItems</i> . Cette liste est vide si les informations de diagnostic n'ont pas été demandées ou ne sont disponibles pour aucun des <i>MonitoredItems</i> . <i>DiagnosticInfo</i> est un type commun défini en 7.8

7.19.3 Paramètre d'EventNotificationList

Le Tableau 149 définit le paramètre *NotificationData* utilisé pour les *EventNotifications*.

L'*EventNotificationList* définit une structure de tableau qui est utilisée pour retourner des champs d'*Événement* à un *Abonnement de Client (Client Subscription)*. La structure a la forme d'un tableau constitué d'un ou plusieurs *Événements*, contenant chacun une matrice d'un ou plusieurs champs. La sélection et l'ordre des champs retournés pour chaque *Événement* sont identiques à ceux du paramètre sélectionné de l'*EventFilter*.

Tableau 149 – EventNotificationList

Nom	Type	Description
EventNotificationList	structure	Données de <i>Notification</i> d'événement.
events []	EventFieldList	La liste des <i>Events</i> distribués. Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.
clientHandle	IntegerId	Descripteur fourni par le <i>Client</i> pour le <i>MonitoredItem</i> . Le type <i>IntegerId</i> est défini en 7.13.
eventFields []	BaseDataType	Liste de champs d' <i>Event</i> sélectionnés. Ceux-ci doivent être en correspondance univoque avec les champs sélectionnés dans l' <i>EventFilter</i> . Le paragraphe 7.16.3 spécifie la manière dont le <i>Serveur</i> doit traiter les conditions d'erreur.

7.19.4 Paramètre de StatusChangeNotification

Le Tableau 150 définit le paramètre *NotificationData* utilisé pour un *StatusChangeNotification*.

La *StatusChangeNotification* informe le client d'un changement de statut de *Subscription*.

Tableau 150 – StatusChangeNotification

Nom	Type	Description
StatusChangeNotification	structure	Données de <i>Notification</i> d'événement.
Status	StatusCode	Le <i>StatusCode</i> qui indique le changement de statut.
diagnosticInfo	DiagnosticInfo	Informations de diagnostic pour le changement de statut.

7.20 Message de notification (NotificationMessage)

Les composantes de ce paramètre sont définies au Tableau 151.

Tableau 151 – NotificationMessage

Nom	Type	Description
NotificationMessage	structure	Le <i>Message</i> qui contient une ou plusieurs <i>Notifications</i> .
sequenceNumber	Counter	Le numéro de séquence du <i>NotificationMessage</i> .
publishTime	UTCTime	L'heure à laquelle ce <i>Message</i> a été envoyé au <i>Client</i> . Si ce <i>Message</i> est retransmis au <i>Client</i> , ce paramètre contient l'heure à laquelle il a été transmis la première fois au <i>Client</i> .
notificationData []	Extensible Parameter NotificationData	La liste des <i>structures NotificationData</i> . Le type de paramètre <i>NotificationData</i> est un type de paramètre extensible spécifié en 7.19. Il spécifie les types des <i>Notifications</i> qui peuvent être envoyées. Le type <i>ExtensibleParameter</i> est spécifié en 7.11. Il convient de regrouper les <i>Notifications</i> du même type dans un seul élément <i>NotificationData</i> . Si un <i>Abonnement</i> contient des <i>MonitoredItems</i> pour des événements et des données, il convient que cette matrice n'ait pas plus de deux éléments. Si <i>Subscription</i> contient des <i>MonitoredItems</i> uniquement pour des données ou uniquement pour des événements, il convient que la taille de la matrice soit toujours 1 pour ce <i>Subscription</i> .

7.21 Plage numérique (NumericRange)

Ce paramètre est défini au Tableau 152. Une définition BNF formelle de la plage numérique peut être trouvée en A.3.

La syntaxe pour la chaîne contient l'une des deux constructions suivantes. La première construction est la représentation sous forme de chaîne d'un nombre entier individuel. Par exemple, "6" est valide, mais "6,0" et "3,2" ne le sont pas. Les valeurs minimales et maximales qui peuvent être exprimées sont définies par l'utilisation de ce paramètre, et non par la définition du type de ce paramètre. La seconde construction est une plage représentée par deux nombres entiers séparés par le caractère deux-points (":"). Le premier nombre entier doit toujours avoir une valeur inférieure à celle du second. Par exemple, "5:7" est valide, alors que "7:5" et "5:5" ne le sont pas. Les valeurs minimales et maximales qui peuvent être exprimées par ces nombres entiers sont définies par l'utilisation de ce paramètre, et non par la définition du type de ce paramètre. Aucun autre caractère, y compris les caractères espace, n'est autorisé.

Les matrices multidimensionnelles peuvent être indexées en spécifiant une plage pour chaque dimension séparée par une ','. Par exemple, un bloc 2x2 dans une matrice 4x4 peut être sélectionné avec la plage "1:2,0:1". Un élément individuel de la matrice multidimensionnelle peut être sélectionné en spécifiant un seul nombre au lieu d'une plage. Par exemple, "1,1" sélectionne l'élément [1,1] dans une matrice à deux dimensions.

Les dimensions sont spécifiées dans l'ordre de leur apparition dans l'attribut *ArrayDimensions*. Toutes les dimensions doivent être spécifiées pour qu'une *NumericRange* soit valide.

Tous les indices commencent à 0. La valeur maximale pour un indice quelconque est inférieure de 1 à la longueur de la dimension.

Lors de la lecture d'une valeur, les indices ne peuvent pas spécifier une plage qui est dans les limites de la matrice. Le *Serveur* doit retourner un résultat partiel si certains éléments existent dans la plage. Le *Serveur* doit retourner un *Bad_IndexRangeNoData* si aucun élément n'existe dans la plage.

Bad_IndexRangeInvalid est utilisé uniquement pour la syntaxe invalide de *NumericRange*. Toutes les autres demandes invalides avec une syntaxe valide doivent avoir pour résultat *Bad_IndexRangeNoData*.

Lors de l'écriture d'une valeur, la taille de la matrice doit correspondre à la taille spécifiée par la *NumericRange*. Le *Serveur* doit retourner une erreur s'il ne peut pas écrire tous les éléments spécifiés par le *Client*.

La *NumericRange* peut aussi être utilisée pour spécifier des sous-chaînes pour les valeurs *ByteString* et *String*. Les matrices de valeurs *ByteString* et *String* sont traitées comme des matrices à deux dimensions dans lesquelles l'indice final spécifie la plage de la sous-chaîne dans la valeur *ByteString* ou *String*. La valeur complète *ByteString* ou *String* est sélectionnée si l'indice final est omis.

Tableau 152 – NumericRange

Nom	Type	Description
NumericRange	String	Un nombre ou une plage numérique. Une chaîne null indique que ce paramètre n'est pas utilisé.

7.22 Jeu de données de demande (QueryDataSet)

Les composantes de ce paramètre sont définies au Tableau 153.

Tableau 153 – QueryDataSet

Nom	Type	Description
QueryDataSet	structure	Données relatives à un <i>Nœud</i> retournées dans une réponse Query.
nodeId	ExpandedNodeId	Le <i>NodeId</i> pour la description de ce <i>Nœud</i> .
typeDefinitionNode	ExpandedNodeId	Le <i>NodeId</i> pour la définition de type pour la description de ce <i>Nœud</i> .
values []	BaseDataType	Valeurs pour les <i>Attributs</i> sélectionnés. L'ordre des éléments retournés correspond à l'ordre des éléments demandés. Il y a une entrée pour chaque élément demandé pour le <i>TypeDefinitionNode</i> donné qui correspond à l'instance sélectionnée; cela inclut tous les éventuels nœuds connexes qui avaient été spécifiés en utilisant le chemin relatif partant du <i>TypeDefinitionNode</i> de l'instance sélectionnée. Si aucune valeur n'a été trouvée pour un élément demandé donné, une valeur null est retournée pour l'élément en question. Si une valeur a un statut bad, le <i>StatusCode</i> est retourné à la place de la valeur. Si plusieurs valeurs existent pour un élément demandé, une matrice de valeurs est retournée. Si l'élément demandé est une référence, une <i>ReferenceDescription</i> ou une matrice de <i>ReferenceDescription</i> est retournée pour l'élément en question. Si le <i>QueryDataSet</i> est retourné dans un <i>QueryNext</i> pour continuer une liste de <i>ReferenceDescription</i> , les matrices de <i>valeurs</i> auront la même taille, mais les autres valeurs déjà retournées sont null.

7.23 Lecture de l'identifiant d'une valeur (ReadValueId)

Les composantes de ce paramètre sont définies au Tableau 154.

Tableau 154 – ReadValueId

Nom	Type	Description
ReadValueId	structure	Identificateur pour un élément à lire ou à surveiller.
nodeId	NodeId	<i>NodeId</i> d'un <i>Nœud</i> .
AttributeID	IntegerId	Identificateur de l' <i>Attribut</i> . Il doit être un identificateur d' <i>Attribut</i> valide. L' <i>IntegerId</i> est défini en 7.13. Les <i>IntegerIds</i> pour les attributs sont définis dans l'IEC 62541-6.
indexRange	NumericRange	Ce paramètre est utilisé pour identifier un élément simple d'une matrice ou une plage simple d'indices pour les matrices. Si une plage d'éléments est spécifiée, les valeurs sont retournées sous forme de composé. Le premier élément est identifié par l'indice 0 (zéro). Le type <i>NumericRange</i> est défini en 7.21. Ce paramètre est null si l' <i>Attribut</i> spécifié n'est pas une matrice. Cependant, si l' <i>Attribut</i> spécifié est une matrice et ce paramètre est null, tous les éléments sont à inclure dans la plage.

Nom	Type	Description
dataEncoding	QualifiedName	Ce paramètre spécifie le <i>BrowseName</i> du <i>DataTypeEncoding</i> qu'il convient que le <i>Serveur</i> utilise pour retourner l'attribut <i>Value</i> d'une <i>Variable</i>. C'est une erreur de spécifier ce paramètre pour d'autres <i>Attributs</i>. Un <i>Client</i> peut découvrir les <i>DataTypeEncodings</i> qui sont disponibles en suivant la <i>Référence HasEncoding</i> à partir du <i>Nœud DataType</i> pour une <i>Variable</i>. OPC UA définit des <i>BrowseNames</i> que les <i>Serveurs</i> doivent reconnaître même si les <i>Nodes DataType</i> ne sont pas visibles dans l'Espace d'Adresses du <i>Serveur</i>. Ces <i>BrowseNames</i> sont: DefaultBinary The default or native binary (or non-XML) encoding. DefaultXML The default XML encoding. Chaque <i>DataType</i> doit prendre en charge au moins l'un de ces codages. Il convient que les <i>DataTypes</i> qui n'ont pas un véritable codage binaire (par exemple ils ont seulement un codage de texte non-XML) utilisent le nom Default Binary pour identifier le codage qui est considéré comme le codage non XML par défaut. Les <i>DataTypes</i> qui prennent en charge au moins un codage XML doivent identifier l'un des codages comme étant le codage XML par défaut. D'autres organismes de normalisation peuvent définir d'autres codages de données bien connus qui peuvent être pris en charge. Si ce paramètre n'est pas spécifié, le <i>Serveur</i> doit choisir soit le codage Default Binary, soit le codage XML par défaut en fonction du codage de <i>Message</i> (voir l'IEC 62541-6) qui est utilisé pour la <i>Session</i>. Si le <i>Serveur</i> ne prend pas en charge le codage qui correspond au codage de <i>Message</i>, le <i>Serveur</i> doit choisir le codage par défaut qu'il prend effectivement en charge. Si ce paramètre est spécifié pour un <i>MonitoredItem</i>, le <i>Serveur</i> doit mettre le bit <i>StructureChanged</i> dans le <i>StatusCode</i> (voir 7.33) si le <i>DataTypeEncoding</i> change. Le <i>DataTypeEncoding</i> change si la <i>DataTypeVersion</i> de la <i>DataTypeDescription</i> ou du <i>DataTypeDictionary</i> associé au <i>DataTypeEncoding</i> change.

7.24 Description d'une référence (ReferenceDescription)

Les composantes de ce paramètre sont définies au Tableau 155.

Tableau 155 – ReferenceDescription

Nom	Type	Description
ReferenceDescription	structure	Les paramètres de référence retournés pour le <i>Browse Service</i> (service navigation).
referenceTypeId	NodeId	<i>NodeId</i> du <i>ReferenceType</i> qui définit la <i>Référence</i> .
isForward	Boolean	Si la valeur est TRUE, le <i>Serveur</i> a suivi une <i>Référence</i> directe (forward <i>Reference</i>). Si la valeur est FALSE, le <i>Serveur</i> a suivi une <i>Référence</i> inverse (inverse <i>Reference</i>).
nodeId	Expanded NodeId	<i>NodeId</i> du <i>TargetNode</i> tel qu'affecté par le <i>Serveur</i> identifié par l'indice de <i>Serveur</i> . Le type <i>ExpandedNodeId</i> est défini en 7.10. Si le <i>serverIndex</i> indique que le <i>TargetNode</i> est un <i>Nœud</i> distant, le <i>nodeId</i> doit contenir l'URI d'espace de noms absolu. Si le <i>TargetNode</i> est un <i>Nœud</i> local, le <i>nodeId</i> doit contenir l'indice d'espace de noms.
browseName ¹⁾	QualifiedName	Le <i>BrowseName</i> du <i>TargetNode</i> .
displayName	LocalizedText	Le <i>DisplayName</i> du <i>TargetNode</i> .
nodeClass ¹⁾	NodeClass	<i>NodeClass</i> du <i>TargetNode</i> .
typeDefinition ¹⁾	Expanded NodeId	<i>NodeId</i> de définition de type du <i>TargetNode</i> . Les définitions de type sont disponibles uniquement pour les <i>NodeClasses Object</i> et <i>Variable</i> . Pour toutes les autres <i>NodeClasses</i> , un <i>NodeId</i> null doit être retourné.

^{a)} Si l'indice du *Serveur* indique que le *TargetNode* est un *Nœud* distant, le *browseName*, la *nodeClass* et la *typeDefinition* du *TargetNode* peuvent être null ou vides. S'ils ne le sont pas, ils peuvent ne pas être à jour, car le *Serveur* local peut ne pas surveiller en permanence le *Serveur* distant pour détecter les changements.

7.25 Chemin relatif (RelativePath)

Les composantes de ce paramètre sont définies au Tableau 156.

Tableau 156 – RelativePath

Nom	Type	Description
RelativePath	structure	Définit une séquence de <i>Références</i> et de <i>BrowseNames</i> à suivre.
elements []	RelativePath Element	Une séquence de <i>Références</i> et de <i>BrowseNames</i> à suivre. Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants. Chaque élément dans la séquence est traité en trouvant les cibles et ensuite en utilisant ces cibles comme nœuds de départ pour le prochain élément. Les cibles de l'élément final sont la cible du <i>RelativePath</i> .
referenceTypeld	Nodeld	Le type de référence à suivre à partir du nœud courant. Le chemin courant ne peut pas être suivi plus loin si le referenceTypeld n'est pas disponible sur l'instance du nœud. Si ce n'est pas spécifié, toutes les <i>Références</i> sont incluses et le paramètre includeSubtypes est ignoré.
isInverse	Boolean	Indique s'il convient de suivre la <i>Référence</i> inverse. La référence inverse est suivie si cette valeur est TRUE.
includeSubtypes	Boolean	Indique s'il convient de suivre les sous-types du <i>ReferenceType</i> . Les sous-types sont inclus si cette valeur est TRUE.
targetName	QualifiedName	Le <i>BrowseName</i> du nœud cible L'élément final peut avoir un <i>targetName</i> vide. Dans cette situation, toutes les cibles des références identifiées par le referenceTypeld sont les cibles du <i>RelativePath</i> . Le <i>targetName</i> doit être spécifié pour tous les autres éléments. Le chemin courant ne peut pas être suivi si aucune cible avec le <i>BrowseName</i> spécifié n'existe.

Un *RelativePath* peut être appliqué à tout *Nœud* de départ. Les cibles du *RelativePath* sont le jeu de *Nœuds* qui sont trouvés en suivant séquentiellement les éléments dans le *RelativePath*.

Un format texte pour le *RelativePath* peut être trouvé en A.2. Ce format est utilisé dans des exemples qui expliquent les *Services* qui se servent de la structure du *RelativePath*.

7.26 Entête d'une demande (RequestHeader)

Les composantes de ce paramètre sont définies au Tableau 157.

Tableau 157 – RequestHeader

Nom	Type	Description																						
RequestHeader	structure	Paramètres communs pour toutes les demandes présentées sur une <i>Session</i> .																						
authenticationToken	Session AuthenticationToken	L'identificateur de <i>Session</i> secret utilisé pour vérifier que la demande est associée à la <i>Session</i> . Le type <i>SessionAuthenticationToken</i> est défini en 7.29.																						
timestamp	UTCTime	L'heure à laquelle le <i>Client</i> a envoyé la demande. Le paramètre est utilisé uniquement à des fins de diagnostic et de journaux dans le <i>Serveur</i> .																						
requestHandle	IntegerId	Un <i>requestHandle</i> associé à la demande. Ce descripteur défini par le client peut être utilisé pour annuler la demande. Il est également retourné dans la réponse.																						
returnDiagnostics	UInt32	Un masque de bits qui identifie les types de diagnostics spécifiques au fournisseur à retourner dans les paramètres de réponse <i>diagnosticInfo</i>. La valeur du paramètre peut être constituée de zéro, d'une ou de plusieurs des valeurs ci-après. Aucune valeur n'indique que les diagnostics ne sont pas à retourner. <table><tr><th>Valeur de bit</th><th>Diagnostic à retourner</th></tr><tr><td>0x0000 0001</td><td>ServiceLevel / SymbolicId</td></tr><tr><td>0x0000 0002</td><td>ServiceLevel / LocalizedText</td></tr><tr><td>0x0000 0004</td><td>ServiceLevel / AdditionalInfo</td></tr><tr><td>0x0000 0008</td><td>ServiceLevel / <i>StatusCode</i> interne</td></tr><tr><td>0x0000 0010</td><td>ServiceLevel / Diagnostics internes</td></tr><tr><td>0x0000 0020</td><td>OperationLevel / SymbolicId</td></tr><tr><td>0x0000 0040</td><td>OperationLevel / LocalizedText</td></tr><tr><td>0x0000 0080</td><td>OperationLevel / AdditionalInfo</td></tr><tr><td>0x0000 0100</td><td>OperationLevel / <i>StatusCode</i> interne</td></tr><tr><td>0x0000 0200</td><td>OperationLevel / Diagnostics internes</td></tr></table> Chacune de ces valeurs est composée de deux composants, à savoir <i>level</i> (niveau) et <i>type</i>, comme décrit ci-dessous. Si aucun n'est demandé, comme indiqué par une valeur 0, ou si aucune information de diagnostic n'a été	Valeur de bit	Diagnostic à retourner	0x0000 0001	ServiceLevel / SymbolicId	0x0000 0002	ServiceLevel / LocalizedText	0x0000 0004	ServiceLevel / AdditionalInfo	0x0000 0008	ServiceLevel / <i>StatusCode</i> interne	0x0000 0010	ServiceLevel / Diagnostics internes	0x0000 0020	OperationLevel / SymbolicId	0x0000 0040	OperationLevel / LocalizedText	0x0000 0080	OperationLevel / AdditionalInfo	0x0000 0100	OperationLevel / <i>StatusCode</i> interne	0x0000 0200	OperationLevel / Diagnostics internes
Valeur de bit	Diagnostic à retourner																							
0x0000 0001	ServiceLevel / SymbolicId																							
0x0000 0002	ServiceLevel / LocalizedText																							
0x0000 0004	ServiceLevel / AdditionalInfo																							
0x0000 0008	ServiceLevel / <i>StatusCode</i> interne																							
0x0000 0010	ServiceLevel / Diagnostics internes																							
0x0000 0020	OperationLevel / SymbolicId																							
0x0000 0040	OperationLevel / LocalizedText																							
0x0000 0080	OperationLevel / AdditionalInfo																							
0x0000 0100	OperationLevel / <i>StatusCode</i> interne																							
0x0000 0200	OperationLevel / Diagnostics internes																							

Nom	Type	Description
		rencontrée dans le traitement de la demande, les informations de diagnostic ne sont pas retournées. <i>Level:</i> ServiceLevel retourner des diagnostics dans le <i>diagnosticInfo</i> du <i>Service</i>. OperationLevel retourner des diagnostics dans le <i>diagnosticInfo</i> défini pour des opérations individuelles demandées par le <i>Service</i>. <i>Type:</i> SymbolicId retourner un identificateur symbolique qualifié pour un espace de noms pour une erreur ou une condition. La longueur maximale de cet identificateur est de 32 caractères. LocalizedText retourner jusqu'à 256 octets de texte localisé qui décrit l'identificateur symbolique. AdditionalInfo retourner une chaîne d'octets qui contient des informations de diagnostic supplémentaires, telles qu'une image mémoire. Le format de cette chaîne d'octets est spécifique au fournisseur et peut dépendre du type d'erreur ou de condition rencontrée. InnerStatusCode retourner le <i>StatusCode</i> interne associé à l'opération ou au <i>Service</i>. InnerDiagnostics retourner les informations de diagnostic internes associées à l'opération ou au <i>Service</i>. Le contenu de la structure d'informations de diagnostic internes est déterminé par d'autres bits dans le masque. À noter que le positionnement de ce bit peut induire que plusieurs niveaux de structures d'informations de diagnostic imbriquées sont retournés.
auditEntryId	String	Un identificateur qui identifie l'entrée de journal d'audits de sécurité du <i>Client</i> associée à cette demande. Une valeur de chaîne vide signifie que ce paramètre n'est pas utilisé. L'<i>AuditEntryId</i> contient typiquement l'initiateur de l'action et l'endroit où elle a été initiée. L'<i>AuditEventId</i> est inclus dans l'<i>AuditEvent</i> pour permettre à des lecteurs humains de corréler un <i>Event</i> à une action initiatrice. Plus de détails des mécanismes d'<i>Audit</i> sont définis en 6.2 et dans l'IEC 62541-3.
timeoutHint	UInt32	Cette temporisation en millisecondes est utilisée dans la <i>Pile de communication</i> côté <i>Client</i> pour établir la temporisation sur une base appel par appel. Pour un <i>Serveur</i>, cette temporisation est seulement une suggestion et peut être utilisée pour annuler des opérations de longue durée pour libérer des ressources. Si le <i>Serveur</i> détecte une temporisation, il peut annuler l'opération en envoyant le résultat de <i>Service Bad_Timeout</i>. Il convient que le <i>Serveur</i> attende au minimum la temporisation après avoir reçu la demande avant d'annuler l'opération. Le <i>Serveur</i> doit vérifier le paramètre <i>timeoutHint</i> d'une <i>PublishRequest</i> avant de traiter une <i>PublishResponse</i>. Si la demande a dépassé le délai, un résultat de <i>Service Bad_Timeout</i> est envoyé et une autre <i>PublishRequest</i> est utilisée. La valeur de 0 indique l'absence de temporisation.
additionalHeader	Extensible Parameter AdditionalHeader	Réservé pour utilisation ultérieure. Il convient que les applications qui ne comprennent pas l'en-tête l'ignorent.

7.27 Entête d'une réponse (ResponseHeader)

Les composantes de ce paramètre sont définies au Tableau 158.

Tableau 158 – ResponseHeader

Nom	Type	Description
ResponseHeader	structure	Paramètres communs pour toutes les réponses.
timestamp	UTCtime	L'heure à laquelle le <i>Serveur</i> a envoyé la réponse.
requestHandle	IntegerId	Le requestHandle donné par le <i>Client</i> à la demande.
serviceResult	StatusCode	Résultat défini par l'OPC UA de l'invocation du <i>Service</i> . Le type <i>StatusCode</i> est défini en 7.33.
serviceDiagnostics	DiagnosticInfo	Informations de diagnostic pour l'invocation du <i>Service</i> . Ce paramètre est vide si les informations de diagnostic n'ont pas été demandées dans l'en-tête de demande. Le type <i>DiagnosticInfo</i> est défini en 7.8.
stringTable []	String	Il y a une chaîne dans cette liste pour chaque espace de noms unique, chaque identificateur symbolique, et chaque chaîne de texte localisé contenue dans l'ensemble de tous les paramètres d'informations de diagnostic contenus dans la réponse (voir 7.8). Chacun est identifié dans ce tableau par son indice à base zéro.
additionalHeader	Extensible Parameter AdditionalHeader	Réservé pour utilisation ultérieure. Il convient que les applications qui ne comprennent pas l'en-tête l'ignorent.

7.28 Service en défaut (ServiceFault)

Les composantes de ce paramètre sont définies au Tableau 159.

Le paramètre *ServiceFault* est retourné au lieu du message de réponse du *Service* lorsqu'une erreur au niveau service se produit. Il convient que le *requestHandle* dans le *ResponseHeader* soit mis à ce qui a été fourni dans le *RequestHeader* même si ces valeurs n'étaient pas valides. Le niveau de diagnostic retourné dans le *ResponseHeader* est spécifié par le paramètre *returnDiagnostics* dans le *RequestHeader*.

L'utilisation exacte de ce paramètre dépend des mises en correspondance définies dans l'IEC 62541-6.

Tableau 159 – ServiceFault

Nom	Type	Description
ServiceFault	structure	Une réponse d'erreur envoyée lorsqu'une erreur de niveau de service se produit.
responseHeader	ResponseHeader	Paramètres communs aux réponses (voir 7.27 pour la définition de <i>ResponseHeader</i>).

7.29 Authentification du jeton d'une session (SessionAuthenticationToken)

Le type *SessionAuthenticationToken* est un identificateur opaque qui est utilisé pour identifier les demandes associées à une Session particulière. Cet identificateur est utilisé en conjonction avec le *SecureChannelId* ou le *Client Certificate* pour authentifier les messages entrants. Il s'agit de la forme secrète du *sessionId* à usage interne dans les *Applications Client* et *Serveur*.

Un *Serveur* retourne un *SessionAuthenticationToken* dans la réponse *CreateSession*. Le *Client* envoie alors cette valeur avec chaque demande, ce qui permet au *Serveur* de vérifier que l'expéditeur de la demande est le même que l'expéditeur de la demande *CreateSession* d'origine.

Pour les besoins de ce débat, un *Serveur* consiste en une application (code) et une *Pile de Communication* (*Communication Stack*) comme présenté à la Figure 29. La sécurité assurée par le *SessionAuthenticationToken* dépend de la relation de confiance entre l'application *Serveur* et la *Pile de Communication*. La *Pile de Communication* doit être capable de s'assurer de l'expéditeur du message et elle utilise le *SecureChannelId* ou le *Client Certificate* pour identifier l'expéditeur auprès du *Serveur*. Dans ces cas, le *SessionAuthenticationToken* est un identificateur UInt32 qui permet au *Serveur* de distinguer entre des Sessions différentes créées par le même expéditeur.

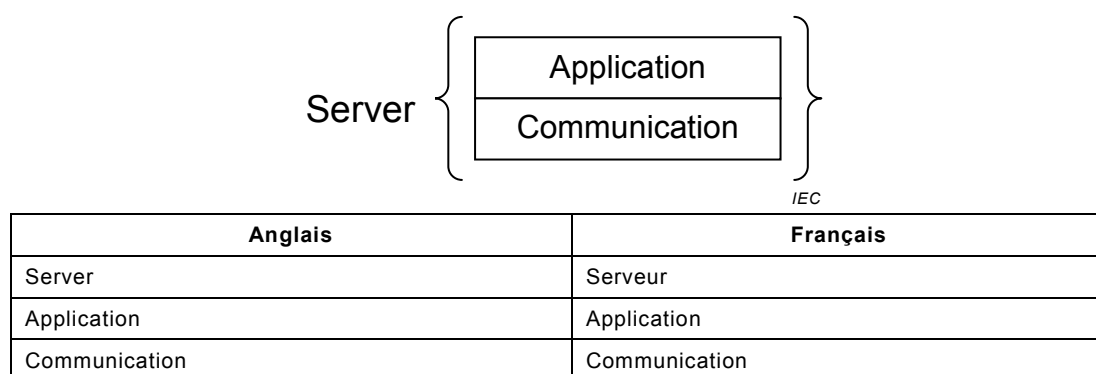
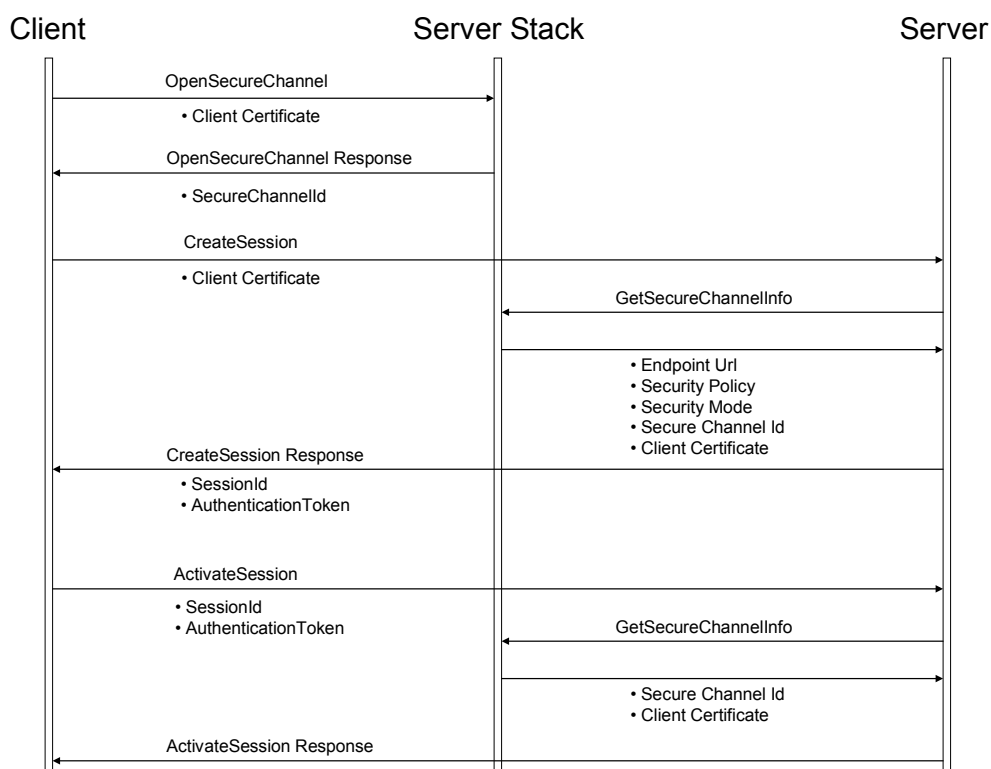


Figure 29 – Couches logiques d'un Serveur

Dans certains cas, l'application et la *Pile de Communication* ne peuvent pas échanger d'informations en mode exécution, ce qui signifie que l'application n'a pas accès au *SecureChannelId* ou au *Certificat* utilisé pour créer le *Canal Sécurisé*. Dans ces cas, l'application doit créer une valeur *ByteString* aléatoire qui a une longueur d'au moins 32 octets. Cette valeur doit être gardée secrète et doit toujours être échangée sur un *Canal Sécurisé* avec le chiffrement activé. L'Administrateur a la responsabilité de s'assurer que le chiffrement est activé. Les *Profiles* dans l'IEC 62541-7 peuvent définir des exigences supplémentaires pour un *ByteString SessionAuthenticationToken*.

Il convient que les applications *Client* et *Serveur* soient écrites de manière à être indépendantes de l'implémentation du *Canal Sécurisé*. Par conséquent, il convient qu'elles traitent toujours le *SessionAuthenticationToken* comme une information secrète, même si cela n'est pas requis lors de l'utilisation de certaines implémentations du *Canal Sécurisé*.

La Figure 30 illustre les informations échangées entre le *Client*, le *Serveur* et la *Pile de Communication* du *Serveur* lorsque le *Client* obtient un *SessionAuthenticationToken*. Dans la Figure ci-dessous, l'étape *GetSecureChannelInfo* représente une API qui dépend de l'implémentation de la *Pile de Communication*.



IEC

Anglais	Français
Server stack	Pile serveur
Server	Serveur
Client certificate	Certificat client
OpenSecureChannel Response	Réponse OpenSecureChannel
Client Certificate	Certificat Client
CreateSession Response	Réponse CreateSession
ActivateSession Response	Réponse ActivateSession
Endpoint Url	Url de point d'extrémité
Security policy	Politique de sécurité
Security mode	Mode sécurité
Secure Channel Id	Id de canal sécurisé

Figure 30 – Obtenir un SessionAuthenticationToken

Le *SessionAuthenticationToken* est un sous-type du type de données *NodeId*; cependant, il n'est jamais utilisé pour identifier un *Nœud* dans l'Espace d'Adresses. Des *Serveurs* peuvent affecter une valeur au *NamespaceIndex*; cependant, sa signification est spécifique au *Serveur*.

7.30 Signature d'une donnée (SignatureData)

Les composantes de ce paramètre sont définies au Tableau 160.

Tableau 160 – SignatureData

Nom	Type	Description
SignatureData	structure	Contient une signature numérique créée avec un <i>Certificat</i> .
signature	ByteString	Il s'agit d'une signature générée par la clé privée associée à un <i>Certificat</i> .
algorithme	String	Une chaîne contenant l'URI de l' <i>algorithme</i> . Les valeurs de la chaîne URI sont définies comme partie des profils de sécurité spécifiés dans l'IEC 62541-7.

7.31 Certificat d'un logiciel signé (SignedSoftwareCertificate)

Un *SignedSoftwareCertificate* est une *ByteString* contenant un *Certificat* codé. Le codage d'un *SignedSoftwareCertificate* dépend de la mise en correspondance de la technologie de sécurité et il est complètement défini dans l'IEC 62541-6. Le Tableau 161 spécifie les informations qui doivent être contenues dans un *SignedSoftwareCertificate*.

Tableau 161 – SignedSoftwareCertificate

Nom	Type	Description
SignedSoftwareCertificate	structure	Un <i>SoftwareCertificate</i> avec une signature créée par l'autorité de certification.
version	String	Un identificateur pour la version du codage du <i>Certificat</i> .
serialNumber	ByteString	Un Identificateur unique pour le <i>Certificat</i> affecté par son Émetteur.
signatureAlgorithm	String	L'algorithme utilisé pour signer le <i>Certificat</i> . La syntaxe de ce champ dépend du codage du <i>Certificat</i> .
signature	ByteString	La signature créée par l'Émetteur.
issuer	structure	Un nom qui identifie le <i>Certificat de l'Émetteur</i> utilisé pour créer la signature.
validFrom	UTCTime	Le moment où le <i>Certificat</i> devient valide.
validTo	UTCTime	Le moment où le <i>Certificat</i> expire.
subject	structure	Un nom qui identifie le produit que le <i>Certificat</i> décrit. Ce champ doit contenir le <i>productName</i> et le <i>vendorName</i> issus du <i>SoftwareCertificate</i> .
subjectAltName []	structure	Une liste de noms de remplacement pour le produit. Cette liste doit inclure le <i>productUri</i> spécifié dans le <i>SoftwareCertificate</i> .
publicKey	ByteString	La clé publique associée au <i>Certificat</i> .
keyUsage []	String	Spécifie comment la clé de <i>Certificat</i> peut être utilisée. Le <i>SignedSoftwareCertificate</i> peut seulement être utilisé pour créer des signatures numériques et le non-rejet. Le contenu de ce champ dépend du codage du <i>Certificat</i> .
softwareCertificate	ByteString	La forme codée XML du <i>SoftwareCertificate</i> stockée sous forme de texte UTF8. L'IEC 62541-6 décrit la représentation en XML pour le <i>SoftwareCertificate</i> .

7.32 Certificat d'un logiciel (SoftwareCertificate)

Les composantes de ce paramètre sont définies au Tableau 162.

Tableau 162 – SoftwareCertificate

Nom	Type	Description
SoftwareCertificate	structure	Un <i>Certificat</i> décrivant un produit.
productName	String	Le nom du produit qui est certifié. Ce champ doit être spécifié.
productUri	String	Un identificateur globalement unique pour le produit qui est certifié. Ce champ doit être spécifié.
vendorName	String	Le nom du fournisseur responsable du produit. Ce champ doit être spécifié.
vendorProductCertificate	ByteString	La forme codée DER (<i>Distinguished Encoding Rules</i> , règles de codage distinctives) du <i>Certificat</i> X.509 qui est affecté au produit par le fournisseur. Ce champ peut être omis.
softwareVersion	String	Version du logiciel. Ce champ doit être spécifié.
buildNumber	String	Numéro de mouture. Ce champ doit être spécifié.
buildDate	UTCTime	Date et heure de la mouture. Ce champ doit être spécifié.
issuedBy	String	URI de l'autorité de certification. Ce champ doit être spécifié.
issueDate	UTCTime	Spécifie l'heure à laquelle le <i>Certificat</i> a été émis par l'autorité de certification. Ce champ doit être spécifié.
supportedProfiles []	structure	Liste des <i>Profiles</i> pris en charge. Cette structure est définie de façon cohérente par rapport aux éléments prévus suivants.
organizationUri	String	Un URI qui identifie l'organisation qui a défini le profil.
profileID	String	Une chaîne qui identifie le <i>Profile</i> .
complianceTool	String	Une chaîne qui identifie l'outil ou la méthode de certification utilisé(e) pour les essais de conformité.
complianceDate	UTCTime	Date et heure de l'essai de conformité.
complianceLevel	Enum Compliance Level	Une énumération qui spécifie le niveau de conformité du <i>Profile</i> . Cette énumération a les valeurs suivantes: UNTESTED_0 la fonction profilée n'a pas été soumise à essai avec succès. PARTIAL_1 la fonction profilée a été soumise partiellement à essai et a réussi aux essais critiques, tels que définis par l'autorité de certification. SELFTESTED_2 la fonction profilée a été soumise à essai avec succès en utilisant un système d'autotest autorisé par l'autorité de certification. CERTIFIED_3 la fonction profilée a été soumise à essai avec succès par un organisme d'essai autorisé par l'autorité de certification.
unsupportedUnitIds []	String	Les identificateurs pour les unités de conformité facultative qui n'ont pas été soumises à essai. Voir l'IEC 62541-7 pour une explication détaillée.

7.33 Code de statut (StatusCode)

7.33.1 Généralités

Un *StatusCode* dans OPC UA est une valeur numérique qui est utilisée pour rapporter le résultat d'une opération accomplie par le *Serveur* OPC UA. Ce code peut avoir des informations de diagnostic associées qui décrivent le statut avec plus de détails; cependant, le code tout seul est censé fournir à des applications *Client* assez d'informations pour prendre des décisions quant à la manière de traiter les résultats d'un *Service* OPC UA.

Le *StatusCode* est un nombre entier non signé de 32 bits. Les 16 bits de poids fort représentent la valeur numérique du code qui doit être utilisé pour détecter des erreurs ou des conditions spécifiques. Les 16 bits de poids faible sont des fanions binaires qui contiennent des informations supplémentaires, mais n'altèrent pas la signification du *StatusCode*.

Tous les *Clients* OPC UA doivent toujours vérifier le *StatusCode* associé à un résultat avant de l'utiliser. Les résultats auxquels est associé un statut incertain/warning (incertain/avertissement) doivent être utilisés avec prudence, car ces résultats peuvent ne pas être valides pour toutes les situations. Les résultats avec un statut bad/failed (mauvais/défaillant) ne doivent jamais être utilisés.

Il convient que les *Serveurs* OPC UA retournent des *StatusCodes* good/success (Bon/succès) si l'opération s'est terminée normalement et le résultat est toujours valide. Des valeurs de *StatusCode* différentes peuvent fournir des informations supplémentaires au *Client*.

Il convient que les *Serveurs* OPC UA utilisent des *StatusCodes* incertain/warning s'ils n'ont pas pu achever l'opération comme demandé par le *Client*, mais l'opération n'a toutefois pas complètement échoué.

Les affectations exactes des bits sont montrées au Tableau 163.

Tableau 163 – Affectations de bits de StatusCode

Champ	Plage de bits	Description
Severity	30:31	Indique si le <i>StatusCode</i> représente un état good (correct), bad (mauvais), incertain (incertain). Ces bits ont les significations suivantes: Bonne 00 Indique que l'opération s'est déroulée correctement et que les résultats associés peuvent être utilisés. Avertissement aléatoire 01 Indique que l'opération s'est en partie déroulée correctement et que les résultats associés peuvent dans certains cas ne pas être appropriés. Échec sérieux 10 Indique que l'opération ne s'est pas déroulée correctement et que les résultats associés ne peuvent pas être utilisés. Réservé 11 Réservé pour utilisation ultérieure. Il convient que tous les <i>Clients</i> traitent le <i>StatusCode</i> avec un niveau de sévérité "Mauvais".
Reserved	29:29	Réservé pour usage dans des API spécifiques à l'application OPC UA. Ce bit doit toujours être zéro à la volée, mais peut être utilisé par des API spécifiques à l'application OPC UA pour des codes de statut spécifiques aux API.
Reserved	28:28	Réservé pour utilisation ultérieure. Doit toujours être zéro.
SubCode	16:27	Le code est une valeur numérique affectée pour représenter différents états. Chaque code a un nom symbolique et une valeur numérique. Toutes les descriptions dans la spécification OPC UA se réfèrent au nom symbolique. L'IEC 62541-6 fait correspondre les noms symboliques à une valeur numérique.
StructureChanged	15:15	Indique que la structure de la valeur de données associée a changé depuis la dernière <i>Notification</i> . Il convient que les <i>Clients</i> ne traitent pas la donnée sauf s'ils relisent les métadonnées. Les <i>Serveurs</i> doivent positionner ce bit si le <i>DataTypeEncoding</i> utilisé pour une <i>Variable</i> change. Le paragraphe 7.23 décrit comment le <i>DataTypeEncoding</i> est spécifié pour une <i>Variable</i> . Les <i>Serveurs</i> doivent aussi positionner ce bit si la <i>Property EnumStrings</i> des <i>DataType</i> de la <i>Variable</i> change. Ce bit est fourni pour avertir les <i>Clients</i> qui analysent des valeurs de données complexes que leurs programmes d'analyse peuvent faillir parce que la forme sérialisée de la valeur de donnée a changé. Ce bit n'a de sens que pour les <i>StatusCodes</i> retournés comme partie d'une <i>Notification</i> de changement de données ou de l' <i>HistoryRead</i> . Les <i>StatusCodes</i> utilisés dans d'autres contextes doivent toujours mettre ce bit à zéro.
SemanticsChanged	14:14	Indique que la sémantique de la valeur de données associée a changé. Il convient que les <i>Clients</i> traitent la valeur de données jusqu'à ce qu'ils relisent les métadonnées associées à la <i>Variable</i> . Il convient que les <i>Serveurs</i> positionnent ce bit si les métadonnées ont changé d'une manière susceptible de provoquer des erreurs d'application si le <i>Client</i> ne relit pas les métadonnées. Par exemple, un changement d'unités techniques peut créer des problèmes si le <i>Client</i> utilise la valeur pour effectuer des calculs. L'IEC 62541-8 définit les conditions dans lesquelles un <i>Serveur</i> doit positionner ce bit pour une <i>Variable</i> DA. D'autres spécifications peuvent définir des conditions supplémentaires. Un <i>Serveur</i> peut définir d'autres conditions qui provoquent le positionnement de ce bit. Ce bit n'a de sens que pour les <i>StatusCodes</i> retournés comme partie d'une <i>Notification</i> de changement de données ou de l' <i>HistoryRead</i> . Les <i>StatusCodes</i> utilisés dans d'autres contextes doivent toujours mettre ce bit à zéro.
Reserved	12:13	Réservé pour utilisation ultérieure. Doit toujours être zéro.
InfoType	10:11	Le type d'informations contenu dans les bits d'informations. Ces bits ont les significations suivantes: Pas utilisé 00 Les bits d'information ne sont pas utilisés et doivent être positionnés à zéro. Valeur de la donnée 01 Le <i>StatusCode</i> et ses bits d'information sont positionnés en fonction de la valeur des données retournées par le serveur. Ce fanion n'est utilisé qu'en combinaison avec les <i>StatusCodes</i> définis dans l'IEC 62541-8. Réservé 1X Réservé pour utilisation ultérieure. Les bits d'information doivent être ignorés.
InfoBits	0:9	Bits d'informations supplémentaires qui qualifient le <i>StatusCode</i> . La structure de ces bits dépend du champ Info Type.

Le Tableau 164 décrit la structure des *InfoBits* lorsque l'Info Type est mis à *DataValue* (01).

Tableau 164 – DataValue InfoBits

Info Type	Plage de bits	Description																					
LimitBits	8:9	Les bits de limites associés à la valeur de données. Les bits de limites ont les significations suivantes: <table> <tr> <th>Limite</th><th>Bits</th><th>Description</th></tr> <tr> <td>Aucune</td><td>00</td><td>La valeur peut être modifiée.</td></tr> <tr> <td>Faible</td><td>01</td><td>La valeur est à la limite inférieure pour la source de données.</td></tr> <tr> <td>Haute</td><td>10</td><td>La valeur est à la limite supérieure pour la source de données.</td></tr> <tr> <td>Constante</td><td>11</td><td>La valeur est constante et ne peut pas être modifiée.</td></tr> </table>	Limite	Bits	Description	Aucune	00	La valeur peut être modifiée.	Faible	01	La valeur est à la limite inférieure pour la source de données.	Haute	10	La valeur est à la limite supérieure pour la source de données.	Constante	11	La valeur est constante et ne peut pas être modifiée.						
Limite	Bits	Description																					
Aucune	00	La valeur peut être modifiée.																					
Faible	01	La valeur est à la limite inférieure pour la source de données.																					
Haute	10	La valeur est à la limite supérieure pour la source de données.																					
Constante	11	La valeur est constante et ne peut pas être modifiée.																					
Overflow	7	Ce bit ne doit être mis que si la taille de la file d'attente du <i>MonitoredItem</i> est supérieure à 1. Si ce bit est positionné, les changements détectés n'ont pas tous été retournés depuis que le tampon de file d'attente du <i>Serveur</i> pour le <i>MonitoredItem</i> a atteint sa limite et a dû purger des données.																					
Reserved	5:6	Réservé pour utilisation ultérieure. Doit toujours être zéro.																					
HistorianBits	0:4	Ces bits ne sont positionnés que lors de la lecture de données historiques. Ils indiquent la source de la valeur de données et fournissent des informations qui ont une incidence sur la manière dont le <i>Client</i> utilise la valeur de données. Les bits historiques ont la signification suivante: <table> <tr> <td>Raw</td><td>XXX00</td><td>Valeur de données brute.</td></tr> <tr> <td>Calculated</td><td>XXX01</td><td>Valeur de données ayant fait l'objet d'un calcul.</td></tr> <tr> <td>Interpolated</td><td>XXX10</td><td>Valeur de données interpolée.</td></tr> <tr> <td>Reserved</td><td>XXX11</td><td>Non défini.</td></tr> <tr> <td>Partial</td><td>XX1XX</td><td>Valeur de données calculée avec intervalle incomplet.</td></tr> <tr> <td>Extra Data</td><td>X1XXX</td><td>Valeur de données brute cachant une autre donnée au niveau du même horodatage.</td></tr> <tr> <td>Multi Value</td><td>1XXXX</td><td>Des valeurs multiples correspondent au critère d'<i>Aggregate</i> (c'est-à-dire des valeurs minimales multiples au niveau de différents horodatages dans le même intervalle).</td></tr> </table> L'IEC 62541-11 décrit avec plus de détails comment ces bits sont utilisés.	Raw	XXX00	Valeur de données brute.	Calculated	XXX01	Valeur de données ayant fait l'objet d'un calcul.	Interpolated	XXX10	Valeur de données interpolée.	Reserved	XXX11	Non défini.	Partial	XX1XX	Valeur de données calculée avec intervalle incomplet.	Extra Data	X1XXX	Valeur de données brute cachant une autre donnée au niveau du même horodatage.	Multi Value	1XXXX	Des valeurs multiples correspondent au critère d' <i>Aggregate</i> (c'est-à-dire des valeurs minimales multiples au niveau de différents horodatages dans le même intervalle).
Raw	XXX00	Valeur de données brute.																					
Calculated	XXX01	Valeur de données ayant fait l'objet d'un calcul.																					
Interpolated	XXX10	Valeur de données interpolée.																					
Reserved	XXX11	Non défini.																					
Partial	XX1XX	Valeur de données calculée avec intervalle incomplet.																					
Extra Data	X1XXX	Valeur de données brute cachant une autre donnée au niveau du même horodatage.																					
Multi Value	1XXXX	Des valeurs multiples correspondent au critère d' <i>Aggregate</i> (c'est-à-dire des valeurs minimales multiples au niveau de différents horodatages dans le même intervalle).																					

7.33.2 StatusCodes communs

Le Tableau 165 définit les *StatusCodes* communs pour tous les résultats des *Services* utilisés dans plus d'un *Service*. Il ne fournit pas une liste complète. Ces *StatusCodes* peuvent aussi être utilisés comme code de résultat de niveau opération. L'IEC 62541-6 fait correspondre les noms symboliques à une valeur numérique et fournit une liste complète des *StatusCodes*, y compris les codes définis dans d'autres parties.

Tableau 165 – Codes de résultats des services communs

Id symbolique	Description
Good	L'opération a réussi.
Good_CompletesAsynchronously	Le traitement sera accompli de manière asynchrone.
Good_SubscriptionTransferred	L'abonnement a été transféré à une autre session.
Bad_CertificateHostNameInvalid	Le <i>HostName</i> utilisé pour se connecter à un <i>Serveur</i> ne correspond pas au <i>HostName</i> contenu dans le <i>Certificat</i> .
Bad_CertificateIssuerRevocationUnknown	Il n'était pas possible de déterminer si le <i>Certificat</i> de l'émetteur a été annulé.
Bad_CertificateIssuerUseNotAllowed	Le <i>Certificat de l'émetteur</i> ne peut pas être utilisé pour l'opération demandée.
Bad_CertificateIssuerTimeInvalid	Le <i>Certificat de l'émetteur</i> a expiré ou n'est pas encore valide.
Bad_CertificateIssuerRevoked	Le <i>Certificat de l'émetteur</i> a été annulé.
Bad_CertificateInvalid	Le <i>Certificat</i> fourni comme paramètre n'est pas valide.
Bad_CertificateRevocationUnknown	Il n'était pas possible de déterminer si le <i>Certificat</i> a été annulé.
Bad_CertificateRevoked	Le <i>Certificat</i> a été annulé.
Bad_CertificateTimeInvalid	Le <i>Certificat</i> a expiré ou n'est pas encore valide.
Bad_CertificateUriInvalid	L'URI spécifié dans l' <i>ApplicationDescription</i> ne correspond pas à l'URI contenu dans le <i>Certificat</i> .
Bad_CertificateUntrusted	Le <i>Certificat</i> n'est pas fiable.
Bad_CertificateUseNotAllowed	Le <i>Certificat</i> ne peut pas être utilisé pour l'opération demandée.
Bad_CommunicationError	Une erreur de communication de bas niveau s'est produite.
Bad_DataTypedUnknown	L'objet d'extension ne peut pas être (dé)sérialisé, car l'identificateur de type de données n'est pas reconnu.
Bad_DecodingError	Décodage arrêté en raison d'une donnée non valide dans le flux.
Bad_EncodingError	Codage arrêté en raison d'une donnée non valide dans les objets sérialisés.
Bad_EncodingLimitsExceeded	Les limites de codage/décodage des messages imposées par la <i>Pile de Communication</i> ont été dépassées.
Bad_IdentityTokenInvalid	Le jeton d'identité d'utilisateur n'est pas valide.
Bad_IdentityTokenRejected	Le jeton d'identité d'utilisateur est valide, mais le serveur l'a rejeté.
Bad_InternalError	Une erreur interne s'est produite à la suite d'une erreur de programmation ou de configuration.
Bad_InvalidArgument	Un ou plusieurs arguments ne sont pas valides. Chaque service définit des <i>StatusCodes</i> spécifiques au paramètre et ces <i>StatusCodes</i> doivent être utilisés au lieu de ce code d'erreur générale. Ce code d'erreur doit être uniquement utilisé par la <i>Pile de Communication</i> et dans des services où il est défini dans la liste de <i>StatusCodes</i> valides pour le service.
Bad_InvalidState	L'opération ne peut pas être menée à bien, car l'objet est fermé, non initialisé ou dans quelque autre état non valide.
Bad_InvalidTimestamp	L'horodatage est hors la plage permise par le serveur.
Bad_NonceInvalid	Le nonce s'avère être une valeur non aléatoire ou il n'a pas la longueur correcte.
Bad_NothingToDo	Il n'y avait rien à faire, car le client a passé une liste d'opérations sans éléments.
Bad_OutOfMemory	Mémoire insuffisante pour parachever l'opération.
Bad_RequestCancelledByClient	La demande a été annulée par le client.
Bad_RequestTooLarge	La taille du message de demande dépasse les limites fixées par le serveur.
Bad_ResponseTooLarge	La taille du message de réponse dépasse les limites fixées par le client.
Bad_RequestHeaderInvalid	L'en-tête pour la demande est absent ou n'est pas valide.
Bad_ResourceUnavailable	Une ressource du système d'exploitation n'est pas disponible.
Bad_SecureChannelIdInvalid	Le canal sécurisé spécifié n'est plus valide.
Bad_SecurityChecksFailed	Une erreur s'est produite lors de la vérification de la sécurité.
Bad_ServerHalted	Le serveur s'est arrêté et ne peut traiter aucune demande.
Bad_ServerNotConnected	L'opération ne pouvait pas être achevée, car le client n'est pas connecté au serveur.
Bad_ServerUriInvalid	L'URI de <i>Serveur</i> n'est pas valide.
Bad_ServiceUnsupported	Le serveur ne prend pas en charge le service demandé.
Bad_SessionIdInvalid	L'identificateur de session n'est pas valide.
Bad_SessionClosed	La session était fermée par le client.
Bad_SessionNotActivated	La session ne peut pas être utilisée, car <i>ActivateSession</i> n'a pas été appelé.
Bad_Shutdown	L'opération a été annulée, car l'application se fermait.
Bad_SubscriptionIdInvalid	L'identificateur d'abonnement n'est pas valide.
Bad_Timeout	L'opération s'est interrompue par temporisation.
Bad_TimestampsToReturnInvalid	Le paramètre horodatages à retourner n'est pas valide.
Bad_TooManyOperations	La requête n'a pas pu être traitée, car elle spécifiait de trop nombreuses opérations.
Bad_UnexpectedError	Une erreur inattendue s'est produite.
Bad_UnknownResponse	Une réponse non reconnue a été reçue du serveur.
Bad_UserAccessDenied	L'utilisateur n'a pas la permission d'effectuer l'opération demandée.
Bad_ViewIdUnknown	L'identificateur de vue ne renvoie pas à un Nœud de vue valide.

Id symbolique	Description
Bad_ViewTimestampInvalid	L'horodatage de la vue n'est pas disponible ou n'est pas pris en charge.
Bad_ViewParameterMismatchInvalid	Les paramètres de vue ne sont pas cohérents les uns avec les autres.
Bad_ViewVersionInvalid	La version de la vue n'est pas disponible ou n'est pas prise en charge.

Le Tableau 166 définit les *StatusCodes* communs pour tous les résultats de niveau opération utilisés dans plus d'un service. Il ne fournit pas une liste complète. L'IEC 62541-6 fait correspondre les noms symboliques à une valeur numérique et fournit une liste complète des *StatusCodes*, y compris les codes définis dans d'autres parties. Les codes de résultats de *Services* communs peuvent aussi être contenus dans le niveau opération.

Tableau 166 – Codes de résultats de niveau opération communs

Id symbolique	Description
Good_Clamped	La valeur écrite a été acceptée, mais a été bloquée.
Good_Overload	L'échantillonnage a ralenti en raison de limitations de ressources.
Uncertain	La valeur est incertaine, mais aucune cause spécifique n'est connue.
Bad	La valeur est bad (mauvaise), mais aucune raison spécifique n'est connue.
Bad_AttributeIdInvalid	L'attribut n'est pas pris en charge pour le nœud spécifié.
Bad_BrowseDirectionInvalid	Le sens du parcours dans la navigation n'est pas valide.
Bad_BrowseNameInvalid	Le nom de navigation n'est pas valide.
Bad_ContentFilterInvalid	Le filtre de contenu n'est pas valide.
Bad_ContinuationPointInvalid	Le point de continuation fourni n'est plus valide. Ce statut est retourné si le point de continuation a été supprimé ou si l'espace d'adresses a été changé entre les appels de browse.
Bad_DataEncodingInvalid	Le codage de données n'est pas valide. Ce résultat est utilisé si aucun <i>dataEncoding</i> ne peut être appliqué, car un <i>Attribute</i> autre que <i>Value</i> a été demandé ou le <i>DataType</i> de <i>Attribute Value</i> n'est pas un sous-type du <i>DataType Structure</i> .
Bad_DataEncodingUnsupported	Le serveur ne prend pas en charge le codage de données demandé pour le nœud. Ce résultat est utilisé si un <i>dataEncoding</i> peut être appliqué, mais que le codage des données passé n'est pas connu du <i>Serveur</i> .
Bad_EventFilterInvalid	Le filtre d'événements n'est pas valide.
Bad_FilterNotAllowed	Un filtre de surveillance ne peut pas être utilisé en combinaison avec l'attribut spécifié.
Bad_FilterOperandInvalid	L'opérande utilisé dans le filtre de contenu n'est pas valide.
Bad_HistoryOperationInvalid	Le paramètre détails d'historique n'est pas valide.
Bad_HistoryOperationUnsupported	Le serveur ne prend pas en charge l'opération demandée.
Bad_IndexRangeInvalid	La syntaxe du paramètre plage d'indices n'est pas valide.
Bad_IndexRangeNoData	Il n'existe pas de données dans la plage d'indices spécifiée.
Bad_MonitoredItemFilterInvalid	Le paramètre filtre d'éléments surveillés n'est pas valide.
Bad_MonitoredItemFilterUnsupported	Le serveur ne prend pas en charge le filtre d'éléments surveillés demandé.
Bad_MonitoredItemIdInvalid	L'identificateur d'élément surveillé ne renvoie pas à un élément surveillé valide.
Bad_MonitoringModelInvalid	Le mode de surveillance n'est pas valide.
Bad_NoCommunication	La communication avec la source de données est définie, mais n'a pas été établie et aucune dernière valeur connue n'est disponible. Cet état/sous-état est utilisé pour les valeurs placées en cache avant que la première valeur soit reçue ou pour Write (Ecrire) et Call (Appeler) si la communication n'est pas établie.
Bad_NoContinuationPoints	L'opération n'a pas pu être traitée, car tous les points de continuation ont été attribués.
Bad_NodeClassInvalid	La classe de nœuds n'est pas valide.
Bad_NodeIdInvalid	La syntaxe de l'identificateur de nœud n'est pas valide.
Bad_NodeIdUnknown	L'identificateur de nœud renvoie à un nœud qui n'existe pas dans l'Espace d'Adresses du serveur.
Bad_NoDeleteRights	Le <i>Serveur</i> ne permettra pas la suppression du nœud.
Bad_NodeNotInView	Le <i>nodesToBrowse</i> (nœud à parcourir) ne fait pas partie de la vue.
Bad_NotFound	Un élément demandé n'a pas été trouvé ou l'opération de recherche s'est terminée sans succès.
Bad_NotImplemented	L'opération demandée n'est pas implémentée.
Bad_NotReadable	Le niveau d'accès ne permet pas la lecture ou l'abonnement au <i>Nœud</i> .
Bad_NotSupported	L'opération demandée n'est pas prise en charge.
Bad_NotWritable	Le niveau d'accès ne permet pas l'écriture sur le <i>Nœud</i> .
Bad_ObjectDeleted	L'objet ne peut pas être utilisé, car il a été supprimé.
Bad_OutOfRange	La valeur était hors plage.
Bad_ReferenceTypeIdInvalid	L'identificateur de type "référence" ne renvoie pas à un nœud de type "référence" valide.

Id symbolique	Description
Bad_SourceNodeIdInvalid	L'identificateur de nœud source ne renvoie pas à un nœud valide.
Bad_StructureMissing	Un paramètre structuré obligatoire était absent ou null.
Bad_TargetNodeIdInvalid	L'identificateur de nœud cible ne renvoie pas à un nœud valide.
Bad_TypeDefinitionInvalid	L'identificateur de nœud de définition de type ne renvoie pas à un nœud de type approprié.
Bad_TypeMismatch	La valeur fournie pour l'attribut n'est pas du même type que la valeur de l'attribut.
Bad_WaitingForInitialData	En attente que le serveur obtienne des valeurs de la part de la source de données sous-jacente. Après avoir créé un <i>MonitoredItem</i> , il peut prendre un certain temps au serveur pour obtenir effectivement des valeurs pour ces éléments. Dans ces cas, le serveur peut envoyer une <i>Notification</i> avec ce statut avant la <i>Notification</i> avec la première valeur ou le statut de la source de données.

7.34 Horodatages à retourner (TimestampsToReturn)

Le *TimestampsToReturn* est une énumération qui spécifie les *Attributs Timestamp* à transmettre pour des *MonitoredItems* ou des *Nœuds* dans *Read* et *HistoryRead*. Les valeurs de ce paramètre sont définies au Tableau 167.

Tableau 167 – Valeurs de TimestampsToReturn

Valeur	Description
SOURCE_0	Retourner l'horodatage de la source.
SERVER_1	Retourner l'horodatage du <i>Serveur</i> .
BOTH_2	Retourner les horodatages tant de la source que du <i>Serveur</i> .
NEITHER_3	Ne retourner aucun des horodatages. Il s'agit de la valeur par défaut pour les <i>MonitoredItems</i> si une valeur <i>Variable</i> n'est pas actuellement accessible. Pour <i>HistoryRead</i> , il ne s'agit pas d'un paramétrage valide.

7.35 Paramètres du jeton d'identité d'utilisateur (UserIdentityToken)

7.35.1 Vue d'ensemble

La structure *UserIdentityToken* utilisée dans le *Server Service Set* (Jeu de services serveur) permet aux *Clients* de spécifier l'identité de l'utilisateur pour le compte duquel ils agissent. Le mécanisme exact utilisé pour identifier des utilisateurs dépend de la configuration du système. Les différents types de jetons d'identité reposent sur les mécanismes les plus communs qui sont utilisés dans les systèmes à l'heure actuelle. Le Tableau 168 définit le jeu courant de jetons d'identité d'utilisateur. Le type *ExtensibleParameter* est défini en 7.11.

Tableau 168 – Typelds du paramètre UserIdentityToken

Id symbolique	Description
AnonymousIdentityToken	Aucune information d'utilisateur n'est disponible.
UserNameIdentityToken	Un utilisateur identifié par le nom utilisateur et le mot de passe.
X509IdentityToken	Un utilisateur identifié par un <i>Certificat X509v3</i> .
IssuedIdentityToken	Un utilisateur identifié par un <i>WS-SecurityToken</i> .

Le Client doit toujours prouver la possession d'un *UserIdentityToken* lorsqu'il le passe au *Serveur*. Certains jetons incluent un secret tel qu'un mot de passe que le *Serveur* accepte comme preuve. Afin de protéger ces secrets, le *Token* peut être chiffré avant qu'il ne soit passé au *Serveur*. Les profils définis dans l'IEC 62541-7 spécifient où le chiffrement doit être appliqué. D'autres types de jetons permettent au *Client* de créer une signature avec le secret associé au *Token*. Dans ces cas, le *Client* prouve la possession d'un *UserIdentityToken* en accolant le dernier *ServerNonce* au *ServerCertificate* et utilise le secret pour produire une *Signature* qui est passée au *Serveur*.

Chaque *UserIdentityToken* autorisé par un *Endpoint* doit avoir une *UserTokenPolicy* spécifiée dans l'*EndpointDescription*. L'*UserTokenPolicy* spécifie la *SecurityPolicy* qui est à utiliser pour le chiffrement ou la signature. Si cette *SecurityPolicy* est omise, le *Client* utilise la

SecurityPolicy figurant dans l'*EndpointDescription*. Si la *SecurityPolicy* concordante est mise à None, il n'est pas requis de chiffrement ou de signature. Il est recommandé que les Applications ne mettent jamais la *SecurityPolicy* à None pour des UserTokens qui incluent un secret, car ces secrets peuvent être utilisés par un attaquant pour avoir accès au système.

Le Tableau 169 décrit comment sérialiser des *UserIdentityTokens* avant d'appliquer le chiffrement.

Tableau 169 – Format de jeton chiffré UserIdentityToken

Nom	Type	Description
length	Byte[4]	La longueur des données chiffrées, y compris le <i>ServerNonce</i> , mais non compris le champ <i>length</i> . Ce champ est un nombre entier non signé de quatre octets où les octets de poids faible apparaissent en premier.
tokenData	Byte[*]	La valeur de jeton.
serverNonce	Byte[*]	Le dernier <i>ServerNonce</i> retourné par le serveur dans la réponse <i>CreateSession</i> ou <i>ActivateSession</i> .

7.35.2 AnonymousIdentityToken

L'*AnonymousIdentityToken* est utilisé pour indiquer que le Client n'a pas d'authentifiants d'utilisateur.

Le Tableau 170 définit le paramètre *AnonymousIdentityToken*.

Tableau 170 – AnonymousIdentityToken

Nom	Type	Description
AnonymousIdentityToken	structure	Une identité d'utilisateur anonyme.
policyId	String	Un identificateur pour l' <i>UserTokenPolicy</i> à laquelle le jeton se conforme. La structure <i>UserTokenPolicy</i> est définie en 7.36.

7.35.3 UserNameIdentityToken

L'*UserNameIdentityToken* est utilisé pour passer de simples authentifiants de nom d'utilisateur/mot de passe au *Serveur*.

Ce jeton doit être chiffré par le *Client* si le *Serveur* l'exige dans la *SecurityPolicy* de l'*UserTokenPolicy* dans la *EndpointDescription*. Il convient que le *Serveur* spécifie une *SecurityPolicy* pour l'*UserTokenPolicy* si le *Canal Sécurisé* a une *SecurityPolicy* None et qu'aucun chiffrement de couche transport n'est disponible. Si None est spécifié pour l'*UserTokenPolicy* et que la *SecurityPolicy* est None, alors le mot de passe ne contient que le mot de passe codé UTF-8. Il convient de ne pas utiliser cette configuration, sauf si le réseau est chiffré d'une autre façon, par exemple un RPV. L'utilisation de cette configuration sans chiffrement de réseau provoquerait un sérieux défaut de sécurité, dans la mesure où cela donnerait l'aspect d'un accès sécurisé pour l'utilisateur, alors qu'en fait il rendrait visible le mot de passe en texte clair.

Si le jeton est chiffré, le mot de passe doit être converti en *ByteString* UTF8 puis sérialisé comme montré au Tableau 169.

Le *Serveur* doit déchiffrer le mot de passe et vérifier le *ServerNonce*.

Si la *SecurityPolicy* est None, le mot de passe contient seulement le mode de passe codé UTF-8.

Le Tableau 171 définit le paramètre *UserNameIdentityToken*.

Tableau 171 – UserNameIdentityToken

Nom	Type	Description
UserNameIdentityToken	structure	UserName value.
policyId	String	Un identificateur pour l' <i>UserTokenPolicy</i> à laquelle le jeton se conforme. La structure <i>UserTokenPolicy</i> est définie en 7.36.
userName	String	Une chaîne qui identifie l'utilisateur.
password	ByteString	Le mot de passe pour l'utilisateur. Le mot de passe peut être une chaîne vide. Ce paramètre doit être chiffré avec le Certificat du <i>Serveur</i> à l'aide de l'algorithme spécifié par la <i>SecurityPolicy</i> .
encryptionAlgorithm	String	Une chaîne contenant l'URI de l' <i>AsymmetricEncryptionAlgorithm</i> . Les valeurs de la chaîne URI sont des noms définis qui peuvent être utilisés comme partie des profils de sécurité spécifiés dans l'IEC 62541-7. Ce paramètre est null si le mot de passe n'est pas chiffré.

Le Tableau 172 décrit les dépendances pour sélectionner l'*AsymmetricEncryptionAlgorithm* pour l'*UserNameIdentityToken*. L'URI de la *SecurityPolicy* de *SecureChannel* est spécifié dans l'*EndpointDescription* et utilisé dans la demande *OpenSecureChannel* ultérieure. L'URI de la *SecurityPolicy* de l'*UserTokenPolicy* est spécifié dans l'*EndpointDescription*. L'*encryptionAlgorithm* est spécifié dans l'*UserNameIdentityToken* ou l'*IssuedIdentityToken* fourni par le *Client* dans l'appel *ActivateSession*.

Tableau 172 – Sélection d'EncryptionAlgorithm

SecurityPolicy de SecureChannel	SecurityPolicy de l'UserTokenPolicy	EncryptionAlgorithm de l'UserNameIdentityToken
Politique de Sécurité – None ("aucune")	Null	Null
Politique de Sécurité – None	Politique de Sécurité – None	Null
Politique de Sécurité – None	Politique de Sécurité – Other	Algorithme asymétrique pour "Other"
Politique de Sécurité – Other ("Autre")	Null	Algorithme asymétrique pour "Other"
Politique de Sécurité – Other	Politique de Sécurité – Yet another ("Encore une autre")	Algorithme asymétrique pour "Yet another"
Politique de Sécurité – Other	Politique de Sécurité – Other	Algorithme asymétrique pour "Other"
Politique de Sécurité – Other	Politique de Sécurité – None	Null

7.35.4 X509IdentityTokens

Le *X509IdentityToken* est utilisé pour passer un *Certificat* X509v3 qui est émis par l'utilisateur.

Ce jeton doit toujours être accompagné par une signature dans le paramètre *userTokenSignature* d'*ActivateSession* si la *SecurityPolicy* l'exige. Il convient que le *Serveur* spécifie une *SecurityPolicy* pour *UserTokenPolicy* si le *Canal Sécurisé* a une *SecurityPolicy* None.

Le Tableau 173 définit le paramètre *X509IdentityToken*.

Tableau 173 – Jeton d'identité X509

Nom	Type	Description
X509IdentityToken	structure	Valeur X509v3.
policyId	String	Un identificateur pour l' <i>UserTokenPolicy</i> à laquelle le jeton se conforme. La structure <i>UserTokenPolicy</i> est définie en 7.36.
certificateData	ByteString	Le <i>Certificat</i> X509 au format DER.

7.35.5 IssuedIdentityToken

L'*IssuedIdentityToken* est utilisé pour passer des *SecurityTokens* conformes à WS-Security au *Serveur*.

WS-Security définit un certain nombre de profils de jetons qui peuvent être utilisés pour représenter différents types de *SecurityTokens*. Par exemple, les jetons Kerberos et SAML

ont des profils de jetons WSS et doivent être échangés dans OPC UA comme des jetons de sécurité XML.

Il convient que les jetons WSS X509 et UserName ne soient pas échangés comme des jetons de sécurité XML. Il convient que les applications OPC UA utilisent les jetons d'identité OPC UA pour passer les informations contenues dans ces types de *SecurityTokens* WSS.

Ces jetons peuvent être chiffrés ou exiger une signature. L'IEC 62541-7 définit des profils qui incluent la sécurité relative à l'utilisateur et qui incluent également toutes les exigences éventuelles relatives au chiffrement et aux signatures. Des profils de sécurité supplémentaires spécifient des algorithmes de chiffrement et de signature.

Si le jeton est chiffré, l'expression en XML doit être convertie en *ByteString* UTF8 puis sérialisée comme montré au Tableau 169.

Le *Serveur* doit déchiffrer le tokenData et vérifier le *ServerNonce*.

Si la *SecurityPolicy* est None ou si le jeton exige seulement une signature, le tokenData contient la représentation en XML codée UTF-8 du jeton.

Le Tableau 174 définit le paramètre *IssuedIdentityToken*.

Tableau 174 – IssuedIdentityToken

Nom	Type	Description
IssuedIdentityToken	structure	Valeur WSS.
policyId	String	Un identificateur pour l' <i>UserTokenPolicy</i> à laquelle le jeton se conforme. La structure <i>UserTokenPolicy</i> est définie en 7.36.
tokenData	ByteString	La représentation en XML du jeton. Ce paramètre peut être chiffré avec le <i>Certificat</i> du <i>Serveur</i> .
encryptionAlgorithm	String	Une chaîne contenant l'URI de l' <i>AsymmetricEncryptionAlgorithm</i> . La liste des noms définis par l'OPC UA qui peuvent être utilisés est spécifiée dans l'IEC 62541-7. Ce paramètre est null si le jeton n'est pas chiffré. L' <i>encryptionAlgorithm</i> est <i>AsymmetricEncryptionAlgorithm</i> pour la politique de sécurité identifiée par le <i>securityPolicyUri</i> pour l' <i>UserTokenPolicy</i> .

7.36 Politique pour le jeton utilisateur (UserTokenPolicy)

Les composantes de ce paramètre sont définies au Tableau 175.

Tableau 175 – UserTokenPolicy

Nom	Type	Description
UserTokenPolicy	structure	Spécifie un <i>UserIdentityToken</i> qu'un <i>Serveur</i> acceptera.
policyId	String	Un identificateur pour l' <i>UserTokenPolicy</i> affecté par le <i>Serveur</i> . Le Client spécifie cette valeur lorsqu'il construit un <i>UserIdentityToken</i> qui se conforme à la politique. Cette valeur est seulement unique dans le contexte d'un <i>Serveur</i> seul.
tokenType	Enum <i>UserIdentityTokenType</i>	Le type de jeton d'identité d'utilisateur requis. Cette valeur est une énumération avec l'une des valeurs suivantes: ANONYMOUS_0 Aucun jeton n'est requis. USERNAME_1 Un jeton de nom d'utilisateur/de mot de passe. CERTIFICATE_2 Un jeton de <i>Certificate</i> X509v3. ISSUEDTOKEN_3 N'importe quel jeton défini par WS-Security. Un <i>tokenType</i> ANONYMOUS indique que le <i>Serveur</i> n'exige pas d'identification d'utilisateur. Dans ce cas, le <i>Certificate d'Instance d'Application Client</i> est utilisé comme identification d'utilisateur.
issuedTokenType	String	Un URI pour le type de jeton. L'IEC 62541-6 définit les URI pour les types communs de jetons émis. Les fournisseurs peuvent spécifier leur propre jeton. Ce champ peut seulement être spécifié si <i>TokenType</i> est ISSUEDTOKEN.
issuerEndpointUrl	String	Un URL facultatif pour le service émettant le jeton. La signification de cette valeur dépend de l' <i>issuedTokenType</i> .
securityPolicyUri	String	La politique de sécurité à appliquer pour chiffrer ou signer l' <i>UserIdentityToken</i> lorsqu'il est passé au <i>Serveur</i> dans la demande <i>ActivateSession</i> . L'Article 7.35 décrit comment ce paramètre est utilisé. La politique de sécurité pour le Canal Sécurisé est appliquée si cette valeur est omise.

7.37 Description d'une vue (ViewDescription)

Les composantes de ce paramètre sont définies au Tableau 176.

Tableau 176 – ViewDescription

Nom	Type	Description
ViewDescription	structure	Spécifie une Vue.
viewId	NodeId	<i>NodeId</i> de la <i>Vue</i> à interroger (<i>Query</i>). Une valeur null indique l' <i>Espace d'Adresses</i> tout entier.
timestamp	UTCTime	La date et l'heure souhaitées. La version correspondante est celle qui a l'horodatage de création précédent le plus proche. Le paramètre <i>Timestamp</i> ou bien le paramètre <i>viewVersion</i> peut être établi par un <i>Client</i> , mais pas les deux. Si <i>ViewVersion</i> est positionné, ce paramètre doit être null.
viewVersion	UInt32	Le numéro de version pour la <i>Vue</i> souhaitée. Lorsque des <i>Nœuds</i> sont ajoutés à une <i>Vue</i> ou en sont retirés, la valeur de <i>ViewVersion Property</i> d'une <i>Vue</i> est mise à jour. Le paramètre <i>Timestamp</i> ou bien le paramètre <i>viewVersion</i> peut être établi par un <i>Client</i> , mais pas les deux. La <i>Property ViewVersion</i> est définie dans l'IEC 62541-3. Si <i>timestamp</i> est mis, ce paramètre doit être 0. La vue courante est utilisée si <i>timestamp</i> est null et <i>viewVersion</i> est 0.

Annexe A (informative)

Définitions BNF

A.1 Vue d'ensemble sur BNF

La notation BNF (Backus-Naur form) utilisée dans la présente Annexe utilise `<` et `>` pour marquer des symboles, `[` et `]` pour identifier des chemins facultatifs et `|` pour identifier des alternatives. Si les symboles `(` et `)` sont utilisés, ils indiquent des ensembles.

A.2 BNF de RelativePath

Un *RelativePath* est une structure qui décrit une séquence de *Références* et de *Nœuds* à suivre. La présente Annexe décrit un format de texte pour un *RelativePath* qui peut être utilisé dans une documentation ou dans des fichiers servant à stocker des informations de configuration.

Les composantes d'un format de texte pour *RelativePath* sont spécifiées au Tableau A.1.

Tableau A.1 – RelativePath

Symbole	Signification										
/	Le caractère slash (barre oblique) indique que le <i>Serveur</i> est tenu de suivre tout sous-type de <i>HierarchicalReferences</i> .										
.	Le caractère point final (point) indique que le <i>Serveur</i> est tenu de suivre tout sous-type d'un <i>Aggregates ReferenceType</i> (type de référence agrégats).										
<[#!ns:]ReferenceType>	Une chaîne délimitée par les symboles «<» et «>» spécifie le <i>BrowseName</i> d'un <i>ReferenceType</i> à suivre. Par défaut, n'importe quelles <i>Références</i> des sous-types de <i>ReferenceType</i> sont suivies également. Un «#» placé devant le <i>BrowseName</i> indique qu'il convient de ne pas suivre les sous-types. Un «!» devant le <i>BrowseName</i> sert à indiquer qu'il convient de suivre la <i>Référence</i> inverse. Le <i>BrowseName</i> peut être qualifié par un indice d'espace de noms (indiqué par un préfixe numérique suivi de deux-points). Cet indice d'espace de noms est utilisé pour spécifier la composante espace de noms du <i>BrowseName</i> pour le <i>ReferenceType</i> . Si le préfixe d'espace de noms est omis, l'indice d'espace de noms 0 est utilisé.										
[ns:]BrowseName	Une chaîne qui suit un symbole «/», «.» ou «>» spécifie le <i>BrowseName</i> d'un <i>Nœud</i> à retourner ou à suivre. Ce <i>BrowseName</i> peut être préfixé par son indice d'espace de noms. Si le préfixe d'espace de noms est omis, l'indice d'espace de noms 0 est utilisé. L'omission du <i>BrowseName</i> final d'un chemin équivaut à une opération joker qui correspond à tous les <i>Nœuds</i> qui sont la cible de la <i>Référence</i> spécifiée par le chemin.										
&	Le signe & est le caractère d'échappement. Il sert à spécifier des caractères réservés qui apparaissent au sein d'un <i>BrowseName</i> . Un caractère réservé est échappé en insérant le `&` devant lui. Des exemples de <i>BrowseNames</i> avec des caractères échappés sont: <table> <tr> <td><u>Nom de chemin de navigation reçu</u></td><td><u>Se résout en</u></td></tr> <tr> <td>"&/Name_1"</td><td>"/Name_1"</td></tr> <tr> <td>"&.Name_2"</td><td>".Name_2"</td></tr> <tr> <td>"&:Name_3"</td><td>":Name_3"</td></tr> <tr> <td>"&&Name_4"</td><td>"&Name_4"</td></tr> </table>	<u>Nom de chemin de navigation reçu</u>	<u>Se résout en</u>	"&/Name_1"	"/Name_1"	"&.Name_2"	".Name_2"	"&:Name_3"	":Name_3"	"&&Name_4"	"&Name_4"
<u>Nom de chemin de navigation reçu</u>	<u>Se résout en</u>										
"&/Name_1"	"/Name_1"										
"&.Name_2"	".Name_2"										
"&:Name_3"	":Name_3"										
"&&Name_4"	"&Name_4"										

Le Tableau A.2 donne des exemples de *RelativePaths* dans le format de texte.

Tableau A.2 – Exemples de *RelativePath*

Chemin de navigation	Description
"/2:Block&.Output"	Suit toute <i>Référence</i> hiérarchique directe avec la cible <i>BrowseName</i> = "2:Block.Output".
"/3:Truck.0:NodeVersion"	Suit toute <i>Référence</i> hiérarchique directe avec la cible <i>BrowseName</i> = "3:Truck" et de là une <i>Aggregates Reference</i> directe jusqu'à la cible avec <i>BrowseName</i> "0:NodeVersion".
"<1:ConnectedTo>1:Boiler/1:HeatSensor"	Suit toute <i>Référence</i> directe avec un <i>BrowseName</i> = '1:ConnectedTo' et trouve les cibles avec <i>BrowseName</i> = '1:Boiler'. De là, il suit toute <i>Référence</i> hiérarchique et trouve des cibles avec <i>BrowseName</i> = '1:HeatSensor'.
"<1:ConnectedTo>1:Boiler/"	Suit toute <i>Référence</i> directe avec un <i>BrowseName</i> = '1:ConnectedTo' et trouve les cibles avec <i>BrowseName</i> = '1:Boiler'. De là, il trouve toutes les cibles de <i>Références</i> hiérarchiques.
"<0:HasChild>2:Wheel"	Suit toute <i>Référence</i> directe avec <i>BrowseName</i> = 'HasChild' et qualifié par l'espace de noms par défaut de l'OPC UA. Puis il trouve les cibles avec <i>BrowseName</i> = 'Wheel' qualifiées avec l'indice d'espace de noms '2'.
"<!HasChild>Truck"	Suit toute <i>Référence</i> inverse avec un <i>BrowseName</i> = 'HasChild'. Puis il trouve les cibles avec <i>BrowseName</i> = 'Truck'. Dans les deux cas, la composante espace de noms du <i>BrowseName</i> est supposée être 0.
"<0:HasChild>"	Trouve toutes les cibles des <i>Références</i> directes avec un <i>BrowseName</i> = 'HasChild' et qualifié avec l'espace de noms par défaut de l'OPC UA.

La notation BNF ci-après décrit la syntaxe du format de texte de *RelativePath*.

```

<relative-path> ::= <reference-type> <browse-name> [relative-path]

<reference-type> ::= '/' | '.' | '<' ['#'] ['!'] <browse-name> '>'

<browse-name> ::= [<namespace-index> ':' ] <name>

<namespace-index> ::= <digit> [<digit>]

<digit> ::= '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'

<name> ::= (<name-char> | '&' <reserved-char>) [<name>]

<reserved-char> ::= '/' | '.' | '<' | '>' | ':' | '#' | '!' | '&'

<name-char> ::= Tous les caractères valides pour une chaîne String (voir l'IEC 62541-3) en excluant les reserved-chars (caractères réservés).
```

A.3 BNF de NumericRange

La notation BNF ci-après décrit la syntaxe du type de paramètre *NumericRange*.

```

<numeric-range> ::= <dimension> [',' <dimension>]

<dimension> ::= <index> [':' <index>]

<index> ::= <digit> [<digit>]

<digit> ::= '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'
```

Annexe B (informative)

Composition des Filtres et exemples d'interrogation (Query)

B.1 Exemples simples de ContentFilter

B.1.1 Vue d'ensemble

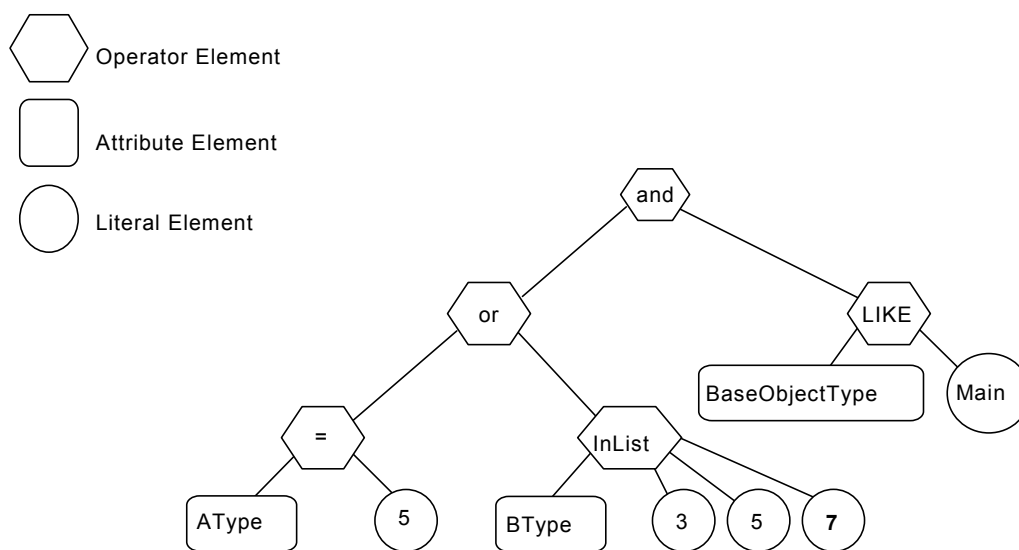
Ces exemples présentent des compositions de filtres très simples. Un filtre semblable à ces exemples peut être utilisé pour traiter des événements.

Les conventions suivantes s'appliquent à ces exemples pour ce qui est de la manière d'utiliser les opérandes des Attributs (pour une définition de cet opérande, voir 7.4.4):

- **AttributeOperand:** Se réfère à un *Nœud*, à l'*Attribut* d'un *Nœud* ou à l'*Attribut Value* d'une *Property* associée à un *Nœud*. Dans les exemples, des noms en caractères de Nodelds sont utilisés en lieu et place d'un nodeld réel; cela s'applique aussi aux identificateurs d'attributs (Attribute Ids).
- La représentation des chemins relatifs sous forme de chaînes est utilisée en lieu et place de la structure réelle.
- Le NamespaceIndex utilisé dans tous les exemples est 12 (il aurait tout aussi bien pu être 4 ou 23 ou n'importe quelle valeur). Pour plus d'informations relatives à NamespaceIndex, voir l'IEC 62541-3. L'utilisation du NamespaceIndex montre que le modèle d'informations utilisé dans les exemples n'est pas un modèle défini par la présente Norme, mais un modèle créé pour les exemples.

B.1.2 Exemple 1

Par exemple, la logique décrite par «(((AType.A = 5) or InList(BType.B, 3,5,7)) an BaseObjectType.displayName LIKE "Main%")» se traduirait par un arbre logique tel que présenté à la Figure B.1 et un ContentFilter tel que présenté au Tableau B.1. Pour que cet exemple retourne quoi que ce soit, AType et BType doivent tous deux être des sous-types de BaseObjectType, ou bien l'opération "And" qui en résulte serait toujours fausse.



IEC

Anglais	Français
Operator element	Élément opérateur

Anglais	Français
Attribute element	Élément attribut
Literal element	Élément littéral
and	et
or	ou

Figure B.1 – Exemple d'arbre logique de filtre

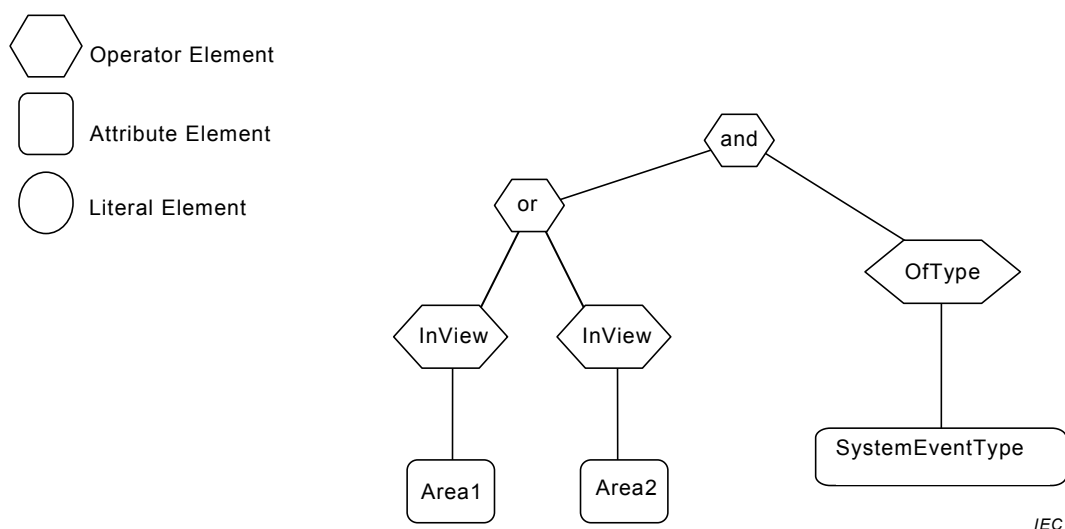
Le Tableau B.1 décrit les éléments, les opérateurs et les opérandes utilisés dans l'exemple.

Tableau B.1 – Exemple de ContentFilter

Element []	Opérateur	Operand[0]	Operand[1]	Operand[2]	Operand[3]
0	And	ElementOperand = 1	ElementOperand = 4		
1	Or	ElementOperand = 2	ElementOperand = 3		
2	Equals	AttributeOperand = NodeId: AType, BrowsePath: ".12:A", Attribute:value	LiteralOperand = '5'		
3	InList	AttributeOperand = NodeId: BType, BrowsePath: ".12:B", Attribute:value	LiteralOperand = '3'	LiteralOperand = '5'	LiteralOperand = '7'
4	Like	AttributeOperand = NodeId: BaseObjectType, BrowsePath: ".", Attribute: displayName	LiteralOperand = "Main%"		

B.1.3 Exemple 2

À titre d'autre exemple, un filtre pour sélectionner tous les *SystemEvents* (y compris les types dérivés) qui sont contenus dans la *View Area1* ou la *View Area2* se traduirait par un arbre logique tel que montré à la Figure B.2 et par un ContentFilter tel que montré au Tableau B.2.



Anglais	Français
Operator element	Élément opérateur
Attribute element	Élément attribut
Literal element	Élément littéral
and	et
or	ou

Figure B.2 – Exemple d'arbre logique de filtre

Le Tableau B.2 décrit les éléments, les opérateurs et les opérandes utilisés dans l'exemple.

Tableau B.2 – Exemple de ContentFilter

Element []	Opérateur	Operand[0]	Operand[1]
0	And	ElementOperand = 1	ElementOperand = 4
1	Or	ElementOperand = 2	ElementOperand = 3
2	InView	AttributeOperand = Nodeld: Area1, BrowsePath: ".", Attribute: Nodeld	
3	InView	AttributeOperand = Nodeld: Area2, BrowsePath: ".", Attribute: Nodeld	
4	OfType	AttributeOperand = Nodeld: SystemEventType, BrowsePath: ".", Attribute: Nodeld"	

B.2 Exemples complexes de Filtre d'Interrogation (Query Filter)

B.2.1 Vue d'ensemble

Ces exemples d'interrogation illustrent des filtres complexes. Les conventions suivantes s'appliquent à ces exemples en ce qui concerne la manière d'utiliser les opérandes d'attribut (pour une définition de ces opérandes, voir 7.4.4):

- *AttributeOperand*: Se réfère à un *Nœud*, à l'*Attribut* d'un *Nœud* ou à l'*Attribut Value* d'une *Property* associée à un *Nœud*. Dans les exemples, des noms en caractères d'*ExpandedNodeld* sont utilisés en lieu et place d'un *ExpandedNodeId* réel; cela s'applique aussi aux identificateurs d'attributs (*Attribute Ids*).
- La représentation des chemins relatifs sous forme de chaînes est utilisée en lieu et place de la structure réelle.
- Le *NamespaceIndex* utilisé dans tous les exemples est 12 (il aurait tout aussi bien pu être 4 ou 23 ou n'importe quelle valeur). Pour plus d'informations relatives à *NamespacesIndex*, voir l'IEC 62541-3. L'utilisation du *NamespaceIndex* montre que le modèle d'informations utilisé dans les exemples n'est pas un modèle défini par la présente Norme, mais un modèle créé pour les exemples.

B.2.2 Modèle de type utilisé

Les exemples suivants utilisent le modèle de type décrit ci-dessous. Toutes les valeurs *Property* sont supposées être sous forme de chaîne sauf indication contraire

Nouveaux types de références:

- "HasChild" dérivé de HierarchicalReference.
- "HasAnimal" dérivé de HierarchicalReference.
- "HasPet" dérivé de HasAnimal.
- "HasFarmAnimal" dérivé de HasAnimal.
- "HasSchedule" dérivé de HierarchicalReference.

PersonType dérivé de BaseObjectType ajoute:

- HasProperty "LastName".
- HasProperty "FirstName".
- HasProperty "StreetAddress".
- HasProperty "City".
- HasProperty "ZipCode".
- Peut avoir une référence HasChild à un nœud de type PersonType.
- Peut avoir une référence HasAnimal à un nœud de type AnimalType (ou sous-type de ce type *Reference*).

AnimalType dérivé de BaseObjectType ajoute:

- Peut avoir une référence HasSchedule à un nœud de type FeedingScheduleType.
- HasProperty "Name".

DogType dérivé de AnimalType ajoute:

- HasProperty "NickName".
- HasProperty "DogBreed".
- HasProperty "License".

CatType dérivé de AnimalType ajoute:

HasProperty "NickName".

HasProperty "CatBreed".

PigType dérivé de AnimalType ajoute:

HasProperty "PigBreed".

ScheduleType dérivé de BaseObjectType ajoute:

HasProperty "Period".

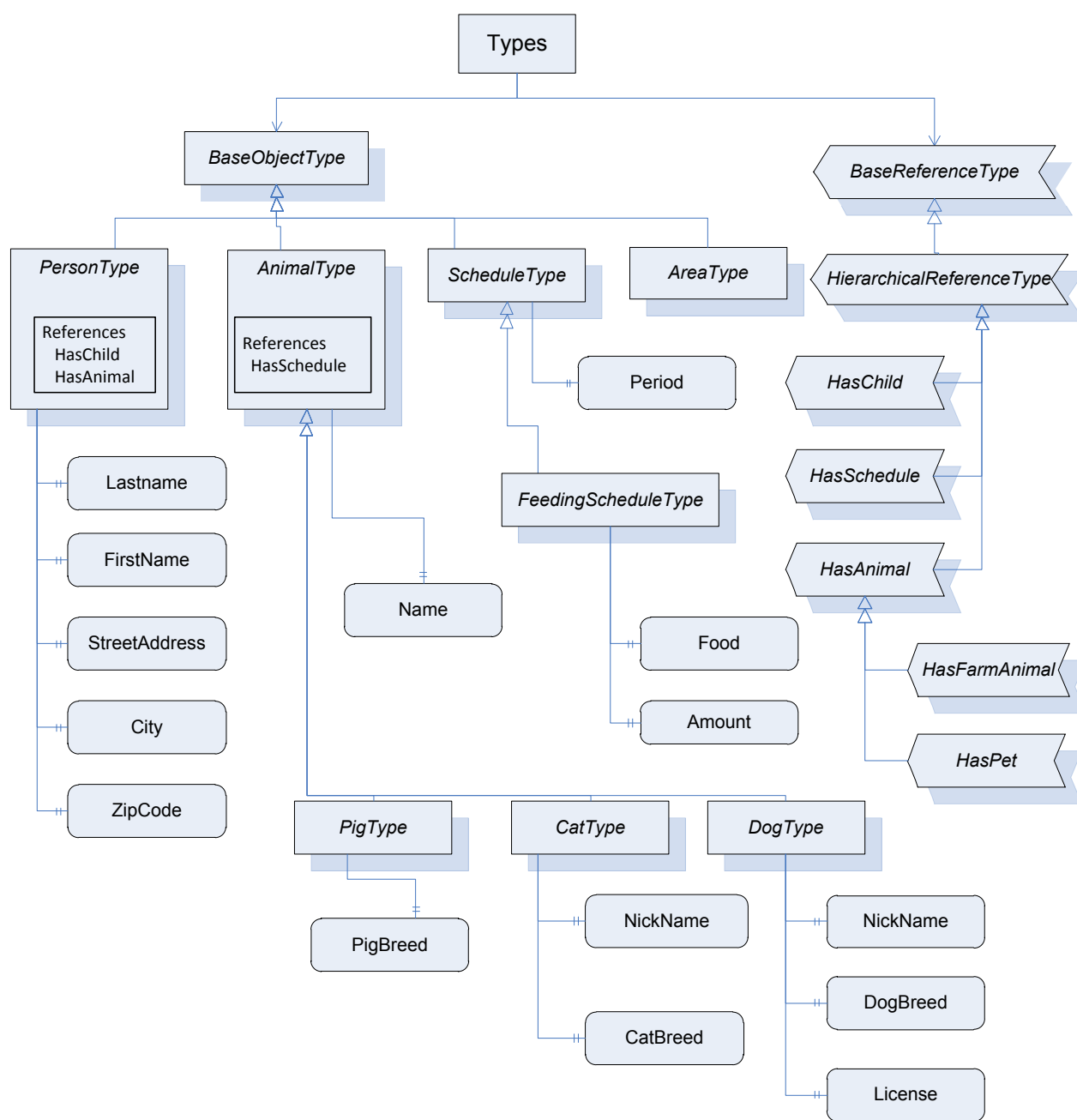
FeedingScheduleType dérivé de ScheduleType ajoute:

HasProperty "Food".

HasProperty "Amount" (Stored as an *Int32*).

AreaType dérivé de *BaseObjectType* est juste un simple *Folder* et ne contient pas de *Properties*.

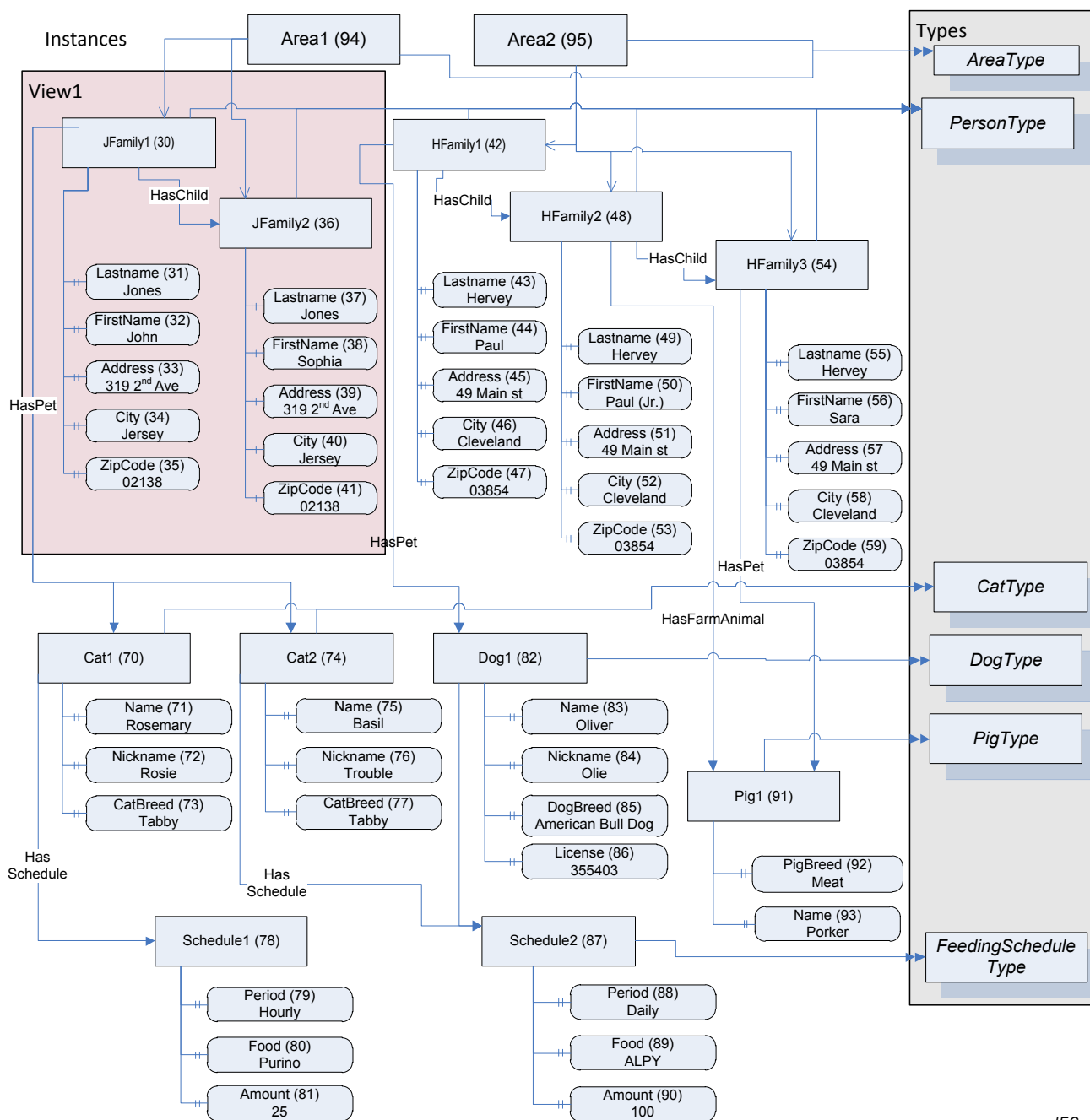
Cet exemple de système de type est montré à la Figure B.3. Dans cette Figure, la notation OPC UA est utilisée pour toutes les *Références* à des *ObjectTypes*, *Variables*, *Properties* et sous-types. En outre, les *Références* prises en charge sont contenues dans une boîte interne. Les références réelles existent uniquement dans les instances et, de ce fait, la Figure ne montre pas de connexions aux autres *Objects* et elles sont des sous-types de la *Référence* répertoriée.



IEC

Figure B.3 – Exemples de nœuds Type

Un exemple correspondant de jeu d'instances est montré à la Figure B.4. Ces instances comprennent des *Références* de type pour des *Objets*. Les *Propriétés* ont aussi des *Références* de type, mais les *Références* sont omises pour des raisons de simplicité. Le nom de l'*Objet* est fourni dans la boîte, ainsi qu'une instance numérique *NodeId* entre parenthèses. Les *ReferenceTypes* normalisés utilisent la notation OPC UA, les *ReferenceTypes* personnalisés sont énumérés comme une *Référence* nommée. Pour les *Propriétés*, le *BrowseName*, le *NodeId* et la *Value* sont montrés. Les *Nœuds* qui sont inclus dans une Vue (View1) sont enfermés dans une boîte colorée. Deux nœuds Area sont inclus pour regrouper les nœuds personne existants. Tous les nœuds personnalisés sont définis dans l'espace de noms 12 qui n'est pas inclus dans la Figure B.4.



IEC

Figure B.4 – Exemple de nœuds Instance

B.2.3 Remarques relatives aux exemples

Pour tous les exemples en 7.4.4, la définition de type *Nœud* est énumérée dans sa forme symbolique; dans l'appel réel ce serait l'*ExpandedNodeId* affecté au *Nœud*. L'*Attribut* est le nom symbolique de l'*Attribut*; dans l'appel réel, il serait traduit en *IntegerId* de l'*Attribut*. Par ailleurs, dans tous les exemples, le *BrowseName* est inclus dans le tableau de résultats pour des raisons de clarté, mais normalement il n'est pas retourné.

Tous les exemples comprennent les éléments suivants:

- une description en anglais de l'objet de l'interrogation,
- une description de l'interrogation telle qu'une description SQL,

- un tableau comprenant une *NodeTypeDescription* des éléments à retourner,
- une figure illustrant le filtre d'interrogation,
- un tableau décrivant le filtre de contenu,
- un tableau décrivant le jeu de données résultant.

Les exemples supposent que l'espace de noms 12 est l'espace de noms pour toutes les définitions personnalisées décrites pour les exemples.

B.2.4 Exemple 1

Cet exemple illustre un filtre de simple niveau, une personne a un animal familier et l'animal familier a un programme.

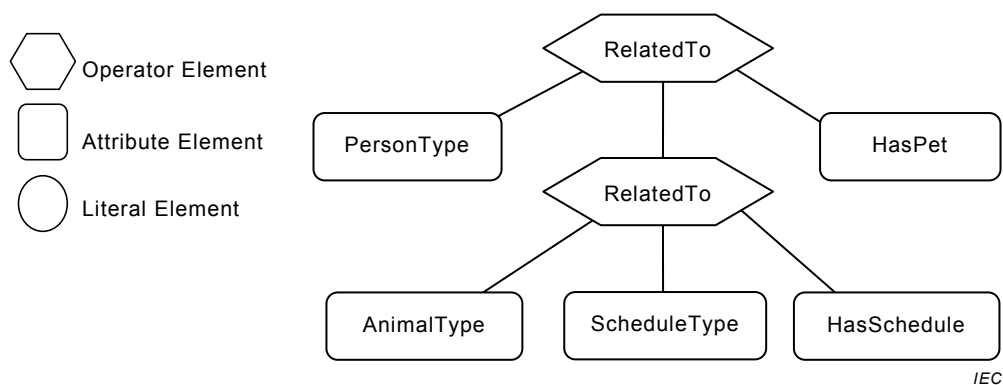
Exemple 1: Get PersonType.LastName, AnimalType.Name, ScheduleType.Period, la personne ayant un animal familier et l'animal familier un programme.

Les paramètres *NodeTypeDescription* utilisés dans l'exemple sont décrits au Tableau B.3.

Tableau B.3 – Exemple 1 NodeTypeDescription

Type Definition Node	Include Subtypes	QueryDataDescription		
		Relative Path	Attribute	Plage d'indices
PersonType	FALSE	".12:LastName"	value	N/A
		"<12:HasPet>12:AnimalType. 12:Name"	value	N/A
		"<12:HasPet>12:AnimalType<12:HasSchedule>12:Schedule. 12:Period"	value	N/A

Le *ContentFilter* correspondant est illustré à la Figure B.5.



Anglais	Français
Operator element	Élément opérateur
Attribute element	Élément attribut
Literal element	Élément littéral

Figure B.5 – Exemple 1 Filtre

Le Tableau B.4 décrit les éléments, les opérateurs et les opérandes de *ContentFilter* utilisés dans l'exemple.

Tableau B.4 – Exemple 1 ContentFilter

Element[]	filterOperator	filterOperand[0]	filterOperand[1]	filterOperand[2]	filterOperand[3]
1	RelatedTo_15	AttributeOperand = Nodeid: PersonType, BrowsePath “.”, Attribute: NodeId	ElementOperand = 2	AttributeOperand = Nodeid: HasPet, BrowsePath “.”, Attribute: NodeId	LiteralOperand = ‘1’
2	RelatedTo_15	AttributeOperand = Nodeid: AnimalType, BrowsePath “.”, Attribute: NodeId	AttributeOperand = Nodeid: ScheduleType, BrowsePath “.”, Attribute: NodeId	AttributeOperand = Nodeid: HasSchedule, BrowsePath “.”, Attribute: NodeId	LiteralOperand= ‘1’

Le Tableau B.5 décrit le *QueryDataSet* qui résulterait de cette interrogation si elle était exécutée par rapport aux instances décrites à la Figure B.4.

Tableau B.5 – Exemple 1 QueryDataSets

Nodeid	TypeDefinition Nodeid	RelativePath	Valeur
12:30 (JFamily1)	PersonType	“.12:LastName”	Jones
		“<12:HasPet>12:AnimalType. 12:Name”	Rosemary
			Basil
		“<12:HasPet>12:AnimalType<12:HasSchedule> 12:Schedule.12:Period”	Hourly
12:42(HFamily1)	PersonType	“.12:LastName”	Hervey
		“<12:HasPet>12:AnimalType. 12:Name”	Oliver
		“<12:HasPet>12:AnimalType<12:HasSchedule> 12:Schedule.12:Period”	Daily

NOTE La colonne RelativePath (chemin relatif) et le nom de navigation (entre parenthèses dans la colonne *Nodeid*) ne sont pas dans le *QueryDataSet* et ne sont montrés ici que par souci de clarté. Le *TypeDefinition Nodeid* serait un nombre entier et non le nom symbolique qui est inclus dans le tableau.

La colonne Valeur est retournée comme une matrice pour chaque description de *Nœud*, où l'ordre des éléments dans la matrice correspond à l'ordre des éléments qui étaient demandés pour le Type de Nœud donné. De plus, si un *Attribut* seul a plusieurs valeurs, il est retourné sous forme d'une matrice à l'intérieur de la matrice la plus grande, par exemple dans le tableau ci-dessus Rosemary et Basil seraient retournés dans une matrice pour l'élément *.<HasPet>.AnimalType.Name*. Ils sont montrés sur des lignes séparées pour faciliter la visualisation. La matrice de la valeur réelle pour JFamily1 serait (“Jones”, {“RoseMary”, “Basil”}, {“Hourly”, “Daily”})

B.2.5 Exemple 2

Le deuxième exemple illustre la réception d'une liste de *Nœuds* disjoints et montre aussi qu'une matrice de résultats peut être reçue.

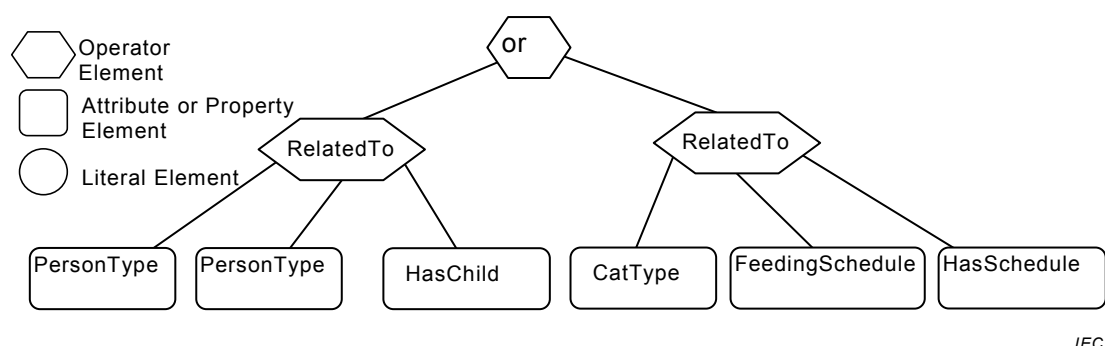
Exemple 2: Get PersonType.LastName, AnimalType.Name, où une personne a un enfant ou (un animal familier est du type chat et a un programme d'alimentation).

Les paramètres *NodeTypeDescription* utilisés dans l'exemple sont décrits au Tableau B.6.

Tableau B.6 – Exemple 2 NodeTypeDescription

Type Definition Node	Include Subtypes	QueryDataDescription		
		Relative Path	Attribute	Plage d'indices
PersonType	FALSE	“.12:LastName”	Value	N/A
AnimalType	TRUE	“.12:Name”	Value	N/A

Le ContentFilter correspondant est illustré à la Figure B.6.



IEC

Anglais	Français
Operator element	Élément Opérateur
Attribute or property element	Élément Attribut ou Propriété
Literal element	Élément littéral

Figure B.6 – Exemple 2 Arbre logique d'un filtre

Le Tableau B.7 décrit les éléments, les opérateurs et les opérandes utilisés dans l'exemple. Il est à noter qu'un CatType est un sous-type d'AnimalType.

Tableau B.7 – Exemple 2 ContentFilter

Element []	Opérateur	Operand[0]	Operand[1]	Operand[2]	Operand[3]
0	Or	ElementOperand=1	ElementOperand = 2		
1	RelatedTo	AttributeOperand = Nodeld: PersonType, BrowsePath ".", Attribute: Nodeld	AttributeOperand = Nodeld: PersonType, BrowsePath ".", Attribute: Nodeld	AttributeOperand = Nodeld: HasChild, BrowsePath ".", Attribute: Nodeld	LiteralOperand = '1'
2	RelatedTo	AttributeOperand = Nodeld: CatType, BrowsePath ".", Attribute: Nodeld	AttributeOperand = Nodeld: FeedingScheduleType, BrowsePath ".", Attribute: Nodeld	AttributeOperand = Nodeld: HasSchedule, BrowsePath ".", Attribute: Nodeld	LiteralOperand = '1'

Les résultats de cette interrogation contiendraient les *QueryDataSets* présentés au Tableau B.8.

Tableau B.8 – Exemple 2 QueryDataSets

Nodeld	TypeDefinition Nodeld	RelativePath	Valeur
12:30 (Jfamily1)	PersonType	. 12:LastName	Jones
12:42 (HFamily1)	PersonType	. 12:LastName	Hervey
12:48 (HFamily2)	PersonType	. 12:LastName	Hervey
12:70 (Cat1)	CatType	. 12:Name	Rosemary
12:74 (Cat2)	CatType	. 12:Name	Basil

NOTE La colonne RelativePath (chemin relatif) et le nom de navigation (entre parenthèses dans la colonne *Nodeld*) ne sont pas dans le *QueryDataSet* et ne sont montrés ici que par souci de clarté. Le *TypeDefinitionNodeld* serait un *Nodeld* et non le nom symbolique qui est inclus dans le tableau.

B.2.6 Exemple 3

Le troisième exemple fournit une *Query* plus complexe dans laquelle les résultats sont filtrés suivant plusieurs critères.

Exemple 3: Get PersonType.LastName, AnimalType.Name, ScheduleType.Period où une personne a un animal familial et l'animal a un programme d'alimentation et la personne

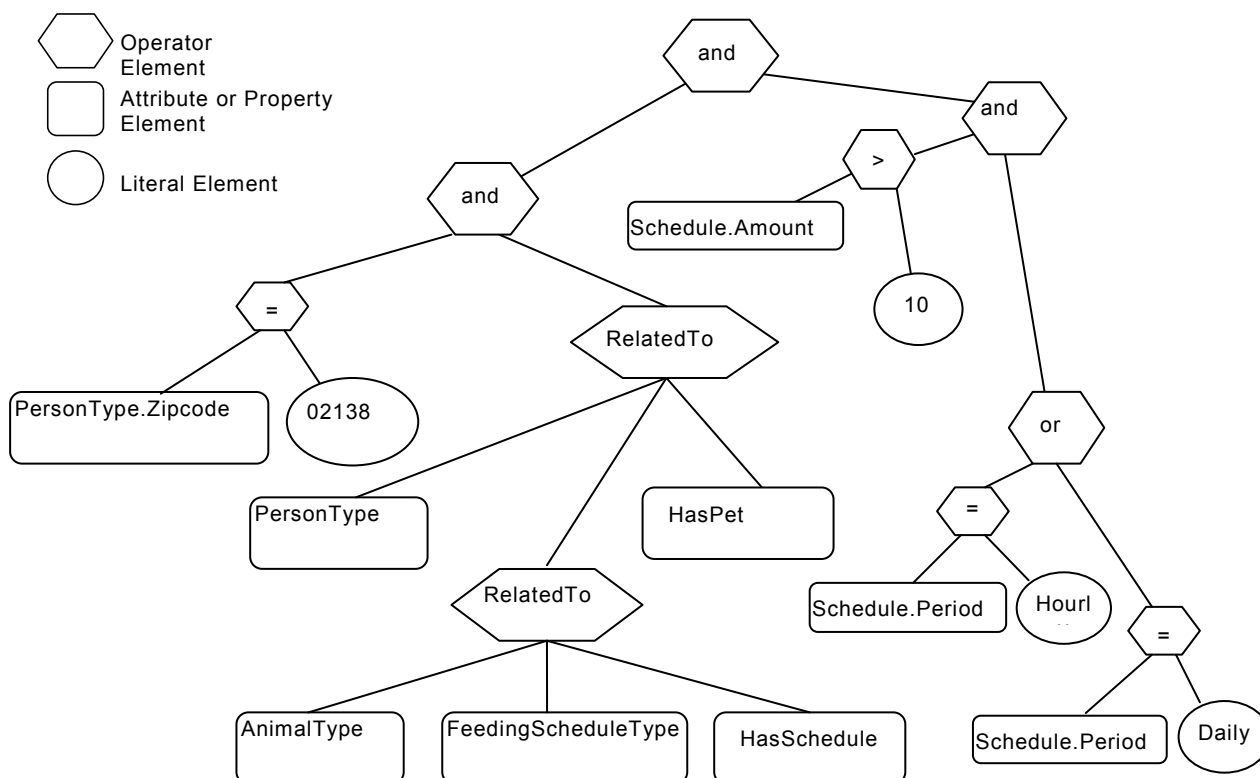
a un code postal Zipcode = '02138' et (la Schedule.Period – périodicité du programme – est Daily (tous les jours) ou Hourly (toutes les heures)) et l'Amount (la quantité) à alimenter est > 10.

Le Tableau B.9 décrit les paramètres NodeTypeDescription utilisés dans l'exemple.

Tableau B.9 – Exemple 3 – NodeTypeDescription

Type Definition Node	Include Subtypes	QueryDataDescription		
		RelativePath	Attribute	Plage d'indices
PersonType	FALSE	"12:LastName"	Value	N/A
		"<12:HasPet>12:AnimalType. 12:Name"	Value	N/A
		"<12:HasPet>12:AnimalType<12:HasSchedule> 12:FeedingSchedule.Period"	Value	N/A

Le *ContentFilter* correspondant est illustré à la Figure B.7.



IEC

Anglais	Français
Operator element	Élément Opérateur
Attribute or property element	Élément Attribut ou Propriété
Literal element	Élément littéral

Figure B.7 – Exemple 3 Arbre logique d'un filtre

Le Tableau B.10 décrit les éléments, les opérateurs et les opérandes utilisés dans l'exemple.

Tableau B.10 – Exemple 3 ContentFilter

Element []	Opérateur	Operand[0]	Operand[1]	Operand[2]	Operand[3]
0	And	ElementOperand = 1	ElementOperand = 2		
1	And	ElementOperand = 3	ElementOperand = 5		
2	And	ElementOperand = 3	ElementOperand = 9		
3	Or	ElementOperand = 7	ElementOperand = 8		
4	RelatedTo	AttributeOperand = NodeId: 12:PersonType, BrowsePath ".", Attribute: NodeId	ElementOperand = 5	AttributeOperand = NodeId: 12:HasPet, BrowsePath ".", Attribute: NodeId	LiteralOperand = '1'
5	RelatedTo	AttributeOperand = Node: 12:AnimalType, BrowsePath ".", Attribute: NodeId	AttributeOperand = NodeId: 12:FeedingScheduleType, BrowsePath ".", Attribute: NodeId Alias: FST	AttributeOperand = NodeId: 12:HasSchedule, BrowsePath ".", Attribute: NodeId	LiteralOperand = '1'
6	Equals	AttributeOperand = NodeId: 12:PersonType BrowsePath 12:Zipcode ".", Attribute: Value	LiteralOperand = '02138'		
7	Equals	AttributeOperand = NodeId: 12:PersonType BrowsePath "12:HasPet>12:AnimalType<12: HasSchedule>12: FeedingSchedule/12:Period", Attribute: Value Alias: FST	LiteralOperand = 'Daily'		
8	Equals	AttributeOperand = NodeId: 12:PersonType BrowsePath "12:HasPet>12:AnimalType<12: HasSchedule>12: FeedingSchedule/12:Period", Attribute: Value Alias: FST	LiteralOperand = 'Hourly'		
9	Greater Than	AttributeOperand = NodeId: 12:PersonType BrowsePath "12:HasPet>12:AnimalType<12: HasSchedule>12: FeedingSchedule/12:Amount", Attribute: Value Alias: FST	ElementOperand = 10		
10	Cast	LiteralOperand = 10	AttributeOperand = NodeId: Int32, BrowsePath ".", Attribute: NodeId		

Les résultats de cette interrogation contiendraient les *QueryDataSets* montrés au Tableau B.11.

Tableau B.11 – Exemple 3 QueryDataSets

NodeId	TypeDefinition NodeId	RelativePath	Valeur
12:30 (JFamily1)	PersonType	". 12:LastName"	Jones
		"<12:HasPet>12:PersonType. 12:Name"	Rosemary
			Basil
		"<12:HasPet>12:AnimalType<12:HasSchedule>12:FeedingSchedule. 12:Period"	Hourly
			Daily

NOTE La colonne RelativePath (chemin relatif) et le nom de navigation (entre parenthèses dans la colonne *NodeId*) ne sont pas dans le *QueryDataSet* et ne sont montrés que par souci de clarté. Le TypeDefinitionNodeId serait un nombre entier et non le nom symbolique qui est inclus dans le tableau.

B.2.7 Exemple 4

Le quatrième exemple fournit une illustration du paramètre Hop (saut) qui fait partie de l'opérateur RelatedTo.

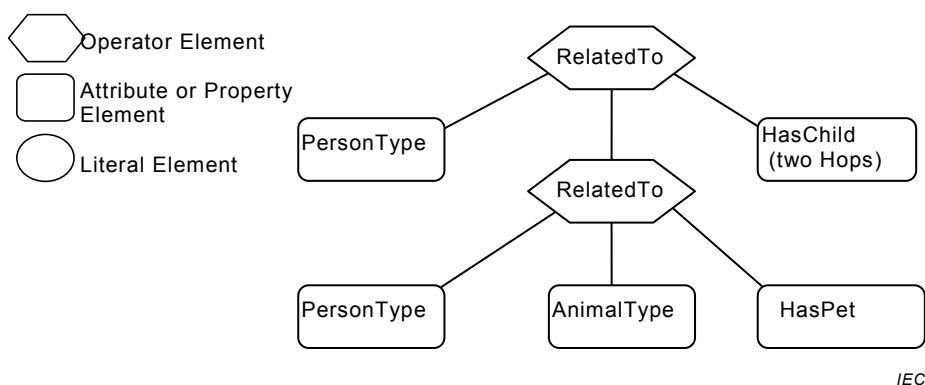
Exemple 4: Get PersonType.LastName, avec une personne ayant un enfant qui a un animal qui a un animal familial.

Le Tableau B.12 décrit les paramètres NodeTypeDescription utilisés dans l'exemple.

Tableau B.12 – Exemple 4 NodeTypeDescription

Type Definition Node	Include Subtypes	QueryDataDescription		
		Relative Path	Attribute	Plage d'indices
PersonType	FALSE	“.12:LastName”	value	N/A

Le *ContentFilter* correspondant est illustré à la Figure B.8.



Anglais	Français
Operator element	Élément Opérateur
Attribute or property element	Élément Attribut ou Propriété
Literal element	Élément littéral

Figure B.8 – Exemple 4 Arbre logique d'un filtre

Le Tableau B.13 décrit les éléments, les opérateurs et les opérandes utilisés dans l'exemple.

Tableau B.13 – Exemple 4 ContentFilter

Element []	Opérateur	Operand[0]	Operand[1]	Operand[2]	Operand[3]
0	RelatedTo	AttributeOperand = NodId: 12:PersonType, BrowsePath “.”, Attribute: NodId	ElementOperand= 1	AttributeOperand = NodId: 12:HasChild, BrowsePath “.”, Attribute: NodId	LiteralOperand = '2'
1	RelatedTo	AttributeOperand = NodId: 12:PersonType, BrowsePath “.”, Attribute: NodId	AttributeOperand = NodId: 12:AnimalType, BrowsePath “.”, Attribute: NodId	AttributeOperand = NodId: 12:HasPet, BrowsePath “.”, Attribute: NodId	LiteralOperand = '1'

Les résultats de cette interrogation contiendraient les *QueryDataSets* montrés au Tableau B.14. Il est à noter que le cochon “Pig1” est référencé comme étant un animal familial par Sara, mais est référencé comme animal de ferme par Paul, le père de Sara.

Tableau B.14 – Exemple 4 QueryDataSets

NodId	TypeDefinition NodId	RelativePath	Valeur
12:42 (HFamily1)	PersonType	“.12:LastName”	Hervey

NOTE La colonne RelativePath (chemin relatif) et le nom de navigation (entre parenthèses dans la colonne *NodId*) ne sont pas dans le *QueryDataSet* et ne sont montrés ici que par souci de clarté. Le NodId de TypeDefinition serait un nombre entier et non le nom symbolique qui est inclus dans le tableau.

B.2.8 Exemple 5

Le cinquième exemple fournit une illustration de l'utilisation d'un pseudonyme (alias).

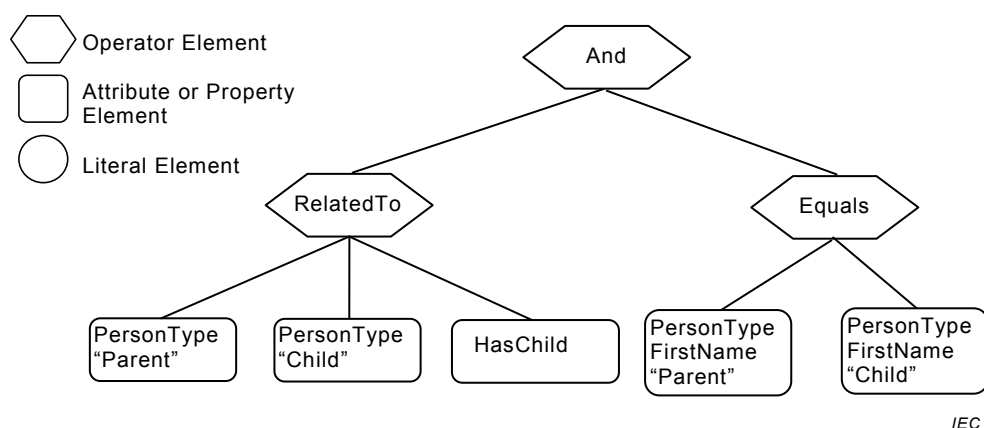
Exemple 5: Récupérer les noms de famille d'enfants qui ont le même prénom qu'un de leurs parents.

Le Tableau B.15 décrit les paramètres NodeTypeDescription utilisés dans l'exemple

Tableau B.15 – Exemple 5 NodeTypeDescription

Type Definition Node	Include Subtypes	QueryDataDescription		
		Relative Path	Attribute	Plage d'indices
PersonType	FALSE	"<12:HasChild>12:PersonType.12:LastName"	Value	N/A

Le *ContentFilter* correspondant est illustré à la Figure B.9.



Anglais	Français
Operator element	Élément Opérateur
Attribute or property element	Élément Attribut ou Propriété
Literal element	Élément littéral

Figure B.9 – Exemple 5 Arbre logique d'un filtre

Dans cet exemple, une *Référence* à PersonType a le pseudonyme "Parent" et une autre *Référence* à PersonType a le pseudonyme "Child". La valeur de Parent.firstName et celle de Child.firstName sont alors comparées. Le Tableau B.16 décrit les éléments, les opérateurs et les opérandes utilisés dans l'exemple.

Tableau B.16 – Exemple 5 ContentFilter

Element []	Opérateur	Operand[0]	Operand[1]	Operand[2]	Operand[3]
0	And	ElementOperand = 1	ElementOperand = 2		
1	RelatedTo	AttributeOperand = NodId: 12:PersonType, BrowsePath ".", Attribute: NodId, Alias: "Parent"	AttributeOperand = NodId: 12:PersonType, BrowsePath ".", Attribute: NodId, Alias: "Child"	AttributeOperand = NodId: 12:HasChild, Attribute: NodId	LiteralOperand = "1"
2	Equals	AttributeOperand = NodId: 12:PersonType, BrowsePath ""/12:FirstName", Attribute: Value, Alias: "Parent"	AttributeOperand = NodId: 12:PersonType, BrowsePath ""/12:FirstName", Attribute: Value, Alias: "Child"		

Les résultats de cette interrogation contiendraient les *QueryDataSets* montrés au Tableau B.17.

Tableau B.17 – Exemple 5 QueryDataSets

NodeId	TypeDefinition NodeId	RelativePath	Valeur
12:42 (HFamily1)	PersonType	"<12:HasChild>12:PersonType.12:LastName"	Hervey

NOTE La colonne RelativePath (chemin relatif) et le nom de navigation (entre parenthèses dans la colonne *NodeId*) ne sont pas dans le *QueryDataSet* et ne sont montrés ici que par souci de clarté. Le *TypeDefinitionNodeId* serait un nombre entier et non le nom symbolique qui est inclus dans le tableau.

B.2.9 Exemple 6

Le sixième exemple montre une illustration d'un type de demande différent, dans lequel le *Client* veut afficher une partie de l'Espace d'Adresses du serveur. Cette demande impose d'énumérer une *Référence* comme quelque chose à retourner.

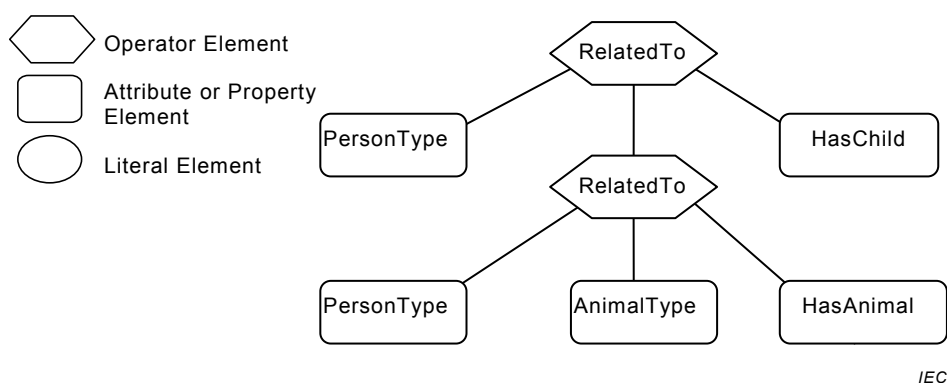
Exemple 6: Get PersonType.NodeId, AnimalType.NodeId, PersonType.HasChild Reference, PersonType.HasAnimal Reference, avec une personne ayant un enfant qui a un Animal.

Le Tableau B.18 décrit les paramètres NodeDescription utilisés dans l'exemple.

Tableau B.18 – Exemple 6 NodeDescription

Type Definition Node	Include Subtypes	QueryDataDescription		
		Relative Path	Attribute	Plage d'indices
PersonType	FALSE	".12:NodeId"	value	N/A
		<12:HasChild>12:PersonType<12:HasAnimal>12:AnimalType.NodeId	value	N/A
		<12:HasChild>	value	N/A
		<12:HasChild>12:PersonType<12:HasAnimal>	value	N/A

Le *ContentFilter* correspondant est illustré à la Figure B.10.



Anglais	Français
Operator element	Élément Opérateur
Attribute or property element	Élément Attribut ou Propriété
Literal element	Élément littéral

Figure B.10 – Exemple 6 Arbre logique d'un filtre

Le Tableau B.19 décrit les éléments, les opérateurs et les opérandes utilisés dans l'exemple.

Tableau B.19 – Exemple 6 ContentFilter

Element []	Opérateur	Operand[0]	Operand[1]	Operand[2]	Operand[3]
0	RelatedTo	AttributeOperand = NodeId: 12:PersonType, BrowsePath “.”, Attribute: NodeId	ElementOperand = 1	AttributeOperand = Node: 12:HasChild, BrowsePath “.”, Attribute: NodeId	LiteralOpera nd = '1'
1	RelatedTo	AttributeOperand = NodeId: 12:PersonType, BrowsePath “.”, Attribute: NodeId	AttributeOperand = NodeId: 12:AnimalType, BrowsePath “.”, Attribute: NodeId	AttributeOperand = NodeId: 12:HasAnimal, BrowsePath “.”, Attribute: NodeId	LiteralOpera nd = '1'

Les résultats de cette interrogation contiendraient les *QueryDataSets* montrés au Tableau B.20.

Tableau B.20 – Exemple 6 QueryDataSets

NodeId	TypeDefinition NodeId	RelativePath	Valeur
12:42 (HFamily1)	PersonType	“.NodeId”	12:42 (HFamily1)
		<12:HasChild>12:PersonType<12:HasAnimal> 12:AnimalType.NodeId	12:91 (Pig1)
		<12:HasChild>	HasChild <i>ReferenceDescription</i>
		<12:HasChild>12:PersonType<12:HasAnimal>	HasFarmAnimal <i>ReferenceDescription</i>
12:48 (HFamily2)	PersonType	“.NodeId”	12:48 (HFamily2)
		<12:HasChild>12:PersonType<12:HasAnimal> 12:AnimalType.NodeId	12:91 (Pig1)
		<12:HasChild>	HasChild <i>ReferenceDescription</i>
		<12:HasChild>12:PersonType<12:HasAnimal>	HasPet <i>ReferenceDescription</i>

NOTE Le RelativePath (chemin relatif) et le nom de navigation (entre parenthèses) ne sont pas dans le *QueryDataSet* et ne sont montrés que par souci de clarté, et le TypeDefinitionNodeId serait un nombre entier et non le nom symbolique qui est inclus dans le tableau. Le champ value serait dans ce cas le *NodeId* où il était demandé, mais pour l'exemple, le nom de navigation est fourni entre parenthèses et dans le cas où des types *Références* sur le nom de navigation sont donnés. Pour les *Références* énumérées au Tableau B.20, la valeur serait une *ReferenceDescription* qui est décrite en 7.24.

Le Tableau B.21 fournit un exemple du même *QueryDataSet* montré au Tableau B.20 sans champs supplémentaires et avec des identificateurs symboliques minimaux. Il y a une entrée pour chaque Attribut demandé; dans le cas où un Attribut retourne plusieurs entrées, les entrées sont séparées par des virgules. Si une structure est retournée, la structure est placée entre crochets. Dans le cas d'une *ReferenceDescription*, la structure contient une structure. Par ailleurs, DisplayName et BrowseName sont supposés être les mêmes et définis à la Figure B.4.

Tableau B.21 – Exemple 6 QueryDataSets sans informations supplémentaires

NodeId	TypeDefinition NodeId	Valeur
12:42	PersonType	12:42
		12:91
		[HasChild, TRUE, [48, HFamily2, HFamily2, PersonType]],
		[HasFarmAnimal, TRUE, [91, Pig1, Pig1, PigType]]
12:48	PersonType	12:54
		12:91
		[HasChild, TRUE, [54, HFamily3, HFamily3, PersonType]]
		[HasPet, TRUE, [91, Pig1, Pig1, PigType]]

Les identificateurs PersonType, HasChild, PigType, HasPet, HasFarmAnimal utilisés dans le Tableau ci-dessus se traduiraient en *ExtendedNodeIds* réels.

B.2.10 Exemple 7

Le septième exemple montre une illustration d'une demande dans laquelle un *Client* souhaite afficher une partie de l'Espace d'Adresses en se basant sur un point de départ obtenu par navigation. Cette demande inclut d'énumérer des *Références* comme quelque chose à retourner. Dans ce cas, la personne a navigué jusqu'à Area2 et a souhaité interroger (*Query*) pour obtenir des informations sous ce point de départ.

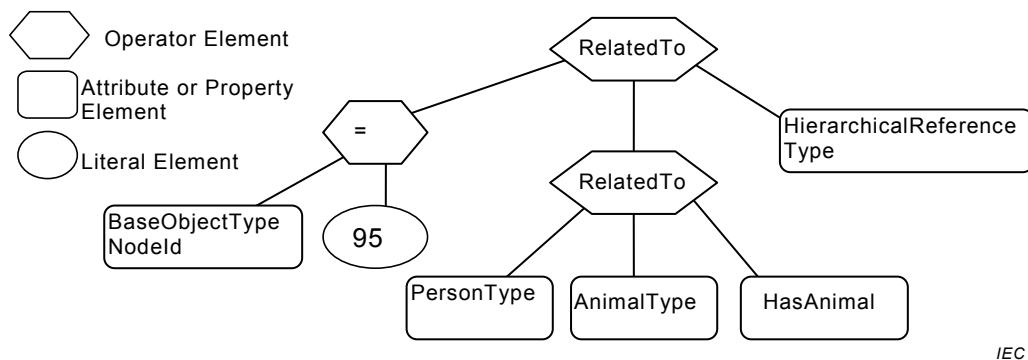
Exemple 7: Get PersonType.NodeId, AnimalType.NodeId, PersonType.HasChild Reference, PersonType.HasAnimal Reference, la personne étant dans Area2 (nœuds Cleveland) et la personne ayant un enfant.

Le Tableau B.22 décrit les paramètres NodeTypeDescription utilisés dans l'exemple.

Tableau B.22 – Exemple 7 NodeTypeDescription

Type Definition Node	Include Subtypes	QueryDataDescription		
		Relative Path	Attribute	Plage d'indices
PersonType	FALSE	".NodeId"	Value	N/A
		<12:HasChild>	Value	N/A
		<12:HasAnimal>NodeId	Value	N/A
		<12:HasAnimal>	Value	N/A

Le *ContentFilter* correspondant est illustré à la Figure B.11. À noter que l'appel *Browse* retournerait typiquement un *NodeId*; ainsi le premier filtre est pour le *BaseObjectType* avec un *NodeId* de 95, où 95 est le *NodeId* associé au nœud Area2, tous les *Nœuds* descendent de *BaseObjectType*, et *NodeId* est une *Propriété* de base et, donc, ce filtre fonctionnera pour toutes les *Queries* de cette nature.



Anglais	Français
Operator element	Élément Opérateur
Attribute or property element	Élément Attribut ou Propriété
Literal element	Élément littéral

Figure B.11 – Exemple 7 Arbre logique d'un filtre

Le Tableau B.23 décrit les éléments, les opérateurs et les opérandes utilisés dans l'exemple.

Tableau B.23 – Exemple 7 ContentFilter

Element []	Opérateur	Operand[0]	Operand[1]	Operand[2]	Operand[3]
0	RelatedTo	ElementOperand = 2	ElementOperand = 1	AttributeOperand = Node:HierachicalReference, BrowsePath “.”, Attribute:NodeId	LiteralOperand = ‘1’
1	RelatedTo	AttributeOperand = NodeId: 12:PersonType, BrowsePath “.”, Attribute: NodeId	AttributeOperand = NodeId: 12:PersonType, BrowsePath “.”, Attribute: NodeId	AttributeOperand = NodeId: 12:HasChild, BrowsePath “.”, Attribute: NodeId	LiteralOperand = ‘1’
2	Equals	AttributeOperand = NodeId: BaseObjectType, BrowsePath “.”, Attribute: NodeId,	LiteralOperand = ‘95		

Les résultats de cette *Query* contiendraient les *QueryDataSets* montrés au Tableau B.24.

Tableau B.24 – Exemple 7 QueryDataSets

NodeId	TypeDefinition NodeId	RelativePath	Valeur
12:42 (HFamily1)	PersonType	“.NodeId”	12:42 (HFamily1)
		<12:HasChild>	HasChild <i>ReferenceDescription</i>
		<12:HasAnimal>12:AnimalType.NodeId	NULL
		<12:HasAnimal>	HasFarmAnimal <i>ReferenceDescription</i>
12:48 (HFamily2)	PersonType	“.NodeId”	12:48 (HFamily2)
		<12:HasChild>	HasChild <i>ReferenceDescription</i>
		<12:HasAnimal>12:AnimalType.NodeId	12:91 (Pig1)
		<12:HasAnimal>	HasFarmAnimal <i>ReferenceDescription</i>

NOTE Le RelativePath (chemin relatif) et le nom de navigation (entre parenthèses) ne sont pas dans le *QueryDataSet* et ne sont montrés que par souci de clarté et le TypeDefinitionNodeId serait un nombre entier et non le nom symbolique qui est inclus dans le tableau. Le champ valeur serait dans ce cas le *NodeId* où il était demandé, mais pour l'exemple, le nom de navigation est fourni entre parenthèses et dans le cas où des types *Références* sur le nom de navigation sont donnés. Pour les *Références* énumérées au Tableau B.24, la valeur serait une *ReferenceDescription* qui est décrite en 7.24.

B.2.11 Exemple 8

Le huitième exemple montre une illustration d'une demande dans laquelle l'*Espace d'Adresses* est limité par une *Vue* définie par un *Serveur*. Cette demande est la même que dans le deuxième exemple qui illustre la réception d'une liste de *Nœuds* disjoints et montre aussi qu'une matrice de résultats peut être reçue. Il est **important** de noter que tous les paramètres et le *ContentFilter* sont les mêmes, seule la description de la *Vue* serait spécifiée comme étant “View1”.

Exemple 8: Get PersonType.LastName, AnimalType.Name, où une personne a un enfant ou (un animal familial est du type chat et a un programme d'alimentation) limité par l'Espace d'Adresses dans View1.

Les paramètres NodeTypeDescription utilisés dans l'exemple sont décrits au Tableau B.25.

Tableau B.25 – Exemple 8 NodeTypeDescription

Type Definition Node	Include Subtypes	QueryDataDescription		
		Relative Path	Attribute	Plage d'indices
PersonType	FALSE	“.12:LastName”	value	N/A
AnimalType	TRUE	“12.Name”	value	N/A

Le *ContentFilter* correspondant est illustré à la Figure B.12.

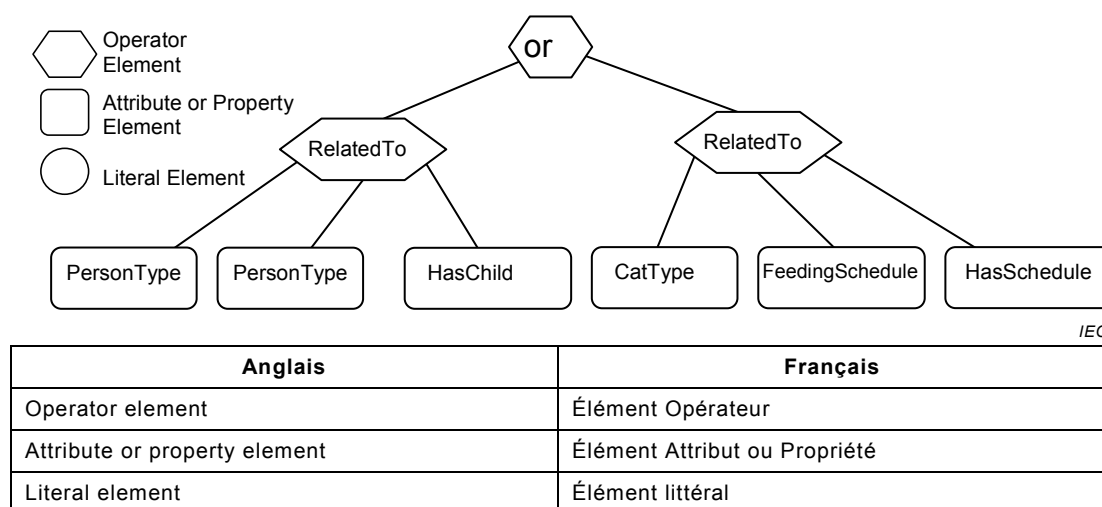


Figure B.12 – Exemple 8 Arbre logique d'un filtre

Le Tableau B.26 décrit les éléments, les opérateurs et les opérandes utilisés dans l'exemple. Il est à noter qu'un CatType est un sous-type d'Animaltype.

Tableau B.26 – Exemple 8 ContentFilter

Element []	Opérateur	Operand[0]	Operand[1]	Operand[2]	Operand[3]
0	Ou	ElementOperand=1	ElementOperand = 2		
1	RelatedTo	AttributeOperand = Nodeld: 12:PersonType, BrowsePath “.”, Attribute: Nodeld	AttributeOperand = Nodeld: 12:PersonType, BrowsePath “.”, Attribute: Nodeld	AttributeOperand = Nodeld: 12:HasChild, BrowsePath “.”, Attribute: Nodeld	LiteralOperand = ‘1’
2	RelatedTo	AttributeOperand = Nodeld: 12:CatType, BrowsePath “.”, Attribute: Nodeld	AttributeOperand = Nodeld: 12:FeedingScheduleType, BrowsePath “.”, Attribute: Nodeld	AttributeOperand = Nodeld: 12:HasSchedule, BrowsePath “.”, Attribute: Nodeld	LiteralOperand = ‘1’

Les résultats de cette interrogation contiendraient les *QueryDataSets* montrés au Tableau B.27. Si ces résultats sont comparés au jeu de résultats de l'exemple 2, la seule différence est l'omission des *Nœuds* Cat. Ces *Nœuds* ne sont pas dans la *Vue* et ne sont donc pas inclus dans le jeu de résultats.

Tableau B.27 – Exemple 8 QueryDataSets

Nodeld	TypeDefinition Nodeld	RelativePath	Valeur
12:30 (Jfamily1)	Persontype	.12:LastName	Jones

NOTE La colonne RelativePath (chemin relatif) et le nom de navigation (entre parenthèses dans la colonne *Nodeld*) ne sont pas dans le *QueryDataSet* et ne sont montrés ici que par souci de clarté. Le TypeDefinition Nodeld serait un nombre entier et non le nom symbolique qui est inclus dans le tableau.

B.2.12 Exemple 9

Le neuvième exemple montre une autre illustration d'une demande dans laquelle l'*Espace d'Adresses* est limité par une *Vue* définie par un *Serveur*. Cette demande est semblable au deuxième exemple, excepté que certains des nœuds demandés sont exprimés en termes d'un chemin relatif. Il est **important** de noter que le *ContentFilter* est le même, seule la description de la *Vue* serait spécifiée comme étant “View1”.

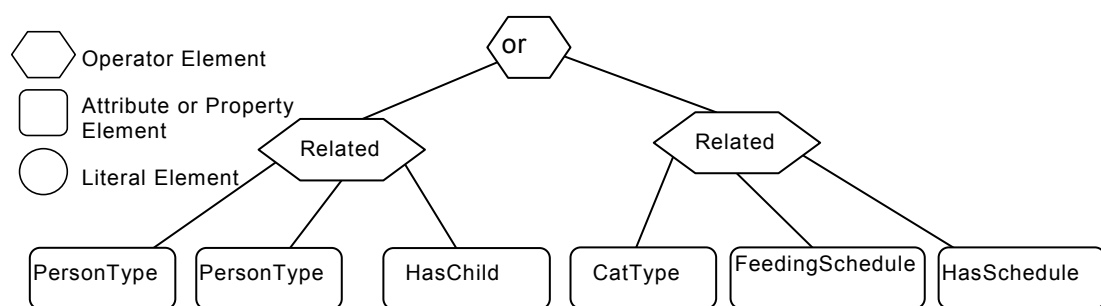
Exemple 9: Get PersonType.LastName, AnimalType.Name, où une personne a un enfant ou (un animal familier est du type chat et a un programme d'alimentation) limité par l'Espace d'Adresses dans View1.

Le Tableau B.28 décrit les paramètres NodeTypeDescription utilisés dans l'exemple.

Tableau B.28 – Exemple 9 NodeTypeDescription

Type Definition Node	Include Subtypes	QueryDataDescription		
		Relative Path	Attribute	Plage d'indices
PersonType	FALSE	“.NodeId”	value	N/A
		<12:HasChild>12:PersonType<12:HasAnimal>12:AnimalType.NodeId	value	N/A
		<12:HasChild>	value	N/A
		<12:HasChild>12:PersonType<12:HasAnimal>	value	N/A
PersonType	FALSE	“.12:LastName”	value	N/A
		<12:HasAnimal>12:AnimalType.12:Name	value	N/A
AnimalType	TRUE	“.12:name”	value	N/A

Le *ContentFilter* correspondant est illustré à la Figure B.13.



IEC

Anglais	Français
Operator element	Élément Opérateur
Attribute or property element	Élément Attribut ou Propriété
Literal element	Élément littéral

Figure B.13 – Exemple 9 Arbre logique d'un filtre

Le Tableau B.29 décrit les éléments, les opérateurs et les opérandes utilisés dans l'exemple.

Tableau B.29 – Exemple 9 ContentFilter

Element []	Opérateur	Operand[0]	Operand[1]	Operand[2]	Operand[3]
0	Ou	ElementOperand=1	ElementOperand = 2		
1	RelatedTo	AttributeOperand = NodeId: 12:PersonType, BrowsePath “.”, Attribute: NodeId	AttributeOperand = NodeId: 12:PersonType, BrowsePath “.”, Attribute: NodeId	AttributeOperand = NodeId: 12:HasChild, BrowsePath “.”, Attribute: NodeId	LiteralOperand = ‘1’
2	RelatedTo	AttributeOperand = NodeId: 12:CatType, BrowsePath “.”, Attribute: NodeId	AttributeOperand = NodeId: 12:FeedingScheduleType, BrowsePath “.”, Attribute: NodeId	AttributeOperand = NodeId: 12:HasSchedule, BrowsePath “.”, Attribute: NodeId	LiteralOperand = ‘1’

Les résultats de cette interrogation (*Query*) contiendraient les *QueryDataSets* montrés au Tableau B.30. Si ces résultats sont comparés au jeu de résultats de l'exemple 2, les *Nœuds*

Pet sont inclus dans la liste, même s'ils sont à l'extérieur de la *Vue*. Cela est possible, car le nom référencé via le chemin relatif et le *Nœud* racine se trouvent dans la *Vue*.

Tableau B.30 – Exemple 9 QueryDataSets

NodeId	TypeDefinition NodeId	RelativePath	Valeur
12:30 (Jfamily1)	PersonType	. 12:LastName	Jones
		<12:HasAnimal>12:AnimalType. 12:Name	Rosemary
		<12:HasAnimal>12:AnimalType. 12:Name	Basil

NOTE La colonne RelativePath (chemin relatif) et le nom de navigation (entre parenthèses dans la colonne *NodeId*) ne sont pas dans le *QueryDataSet* et ne sont montrés ici que par souci de clarté. Le TypeDefinition NodeId serait un nombre entier et non le nom symbolique qui est inclus dans le tableau.

INTERNATIONAL
ELECTROTECHNICAL
COMMISSION

3, rue de Varembé
PO Box 131
CH-1211 Geneva 20
Switzerland

Tel: + 41 22 919 02 11
Fax: + 41 22 919 03 00
info@iec.ch
www.iec.ch